Hình 1: Quì Hoa Bảo Điển

# Mục lục


**1 Tổ hợp** **19**

1.1 Tìm lũy thừa của ước giai thừa . . . 19

1.1.1 Trường hợp $k$ là số nguyên tố . . . 19

1.1.2 Trường hợp $k$ là số hợp số . . . 20

1.2 Hệ số nhị thức . . . 20

1.2.1 Tính toán . . . 21

1.2.2 Tính chất . . . 21

1.2.3 Tính toán . . . 22

1.3 Số Catalan . . . 23

1.3.1 Ứng dụng trong một số bài toán tổ hợp . . . 23

1.3.2 Tính toán . . . 24

1.4 Nguyên lý bao hàm - loại trừ . . . 26

1.4.1 Phát biểu . . . 26

1.4.2 Chứng minh . . . 27

1.4.3 Khái quát hóa để tính số phần tử nằm trong đúng $r$ tập . . . 28

1.4.4 Sử dụng khi giải bài toán . . . 29

1.4.5 Bài toán đơn giản về hoán vị . . . 29

1.4.6 Bài toán đơn giản về dãy (0, 1, 2) . . . 30

1.4.7 Số tổng số nguyên có giới hạn trên . . . 30

1.4.8 Số số nguyên tương đối nguyên tố trong một khoảng cho trước . . . 31

1.4.9 Số số nguyên trong một khoảng cho trước là bội của ít nhất một trong các số đã cho . . . 32

1.4.10 Số chuỗi thỏa mãn một mẫu cho trước . . . 33

1.4.11 Số cách đi từ một ô đến ô khác . . . 34

1.4.12 Số bộ bốn tương đối nguyên tố . . . 35

1.4.13 Số bộ ba hài hòa . . . 36

1.4.14 Số hoán vị không có điểm cố định (derangements) . . . 38

1.5 Định lý Burnside / Định lý đếm Pólya . . . 39

1.5.1 Định lý Burnside . . . 39

1.5.2 Định lý đếm Pólya . . . 42

1.6 Sao và Thanh . . . 45
1.6.1 Định lý . . . 46
1.6.2 Số tổng số nguyên không âm . . . 46
1.6.3 Số tổng số nguyên dương . . . 46
1.6.4 Số tổng số nguyên có giới hạn dưới . . . 47
1.6.5 Số tổng số nguyên có giới hạn trên . . . 47
1.7 Sinh tất cả các tổ hợp $K$-phần tử . . . 47
1.7.1 Sinh tổ hợp $K$-phần tử tiếp theo theo thứ tự từ điển . . . 47
1.7.2 Sinh tất cả tổ hợp $K$-phần tử sao cho các tổ hợp liền kề khác nhau bởi một phần tử . . . 48
1.8 Đặt các quân Tượng trên bàn cờ . . . 50
1.8.1 Thuật toán . . . 50
1.8.2 Triển khai . . . 51
1.9 Dãy ngoặc cân bằng . . . 52
1.9.1 Kiểm tra tính cân bằng . . . 52
1.9.2 Số lượng dãy cân bằng . . . 53
1.9.3 Tìm dãy cân bằng tiếp theo theo thứ tự từ điển . . . 53
1.9.4 Tìm tất cả các dãy cân bằng . . . 54
1.9.5 Chỉ số của dãy . . . 55
1.9.6 Tìm dãy thứ $k$ . . . 56
1.10 Đếm đồ thị có nhãn . . . 58
1.10.1 Đồ thị có nhãn . . . 58
1.10.2 Đồ thị có nhãn liên thông . . . 59
1.10.3 Đồ thị có nhãn với $k$ thành phần liên thông . . . 59
1.11 Tính liên thông cạnh / Tính liên thông đỉnh . . . 60
1.11.1 Định nghĩa . . . 60
1.11.2 Thuộc tính . . . 61
1.11.3 Tính toán các giá trị . . . 61
1.12 2-SAT . . . 62
1.12.1 Thuật toán: . . . 63
1.12.2 Triển khai: . . . 64

**2 String** **67**
2.1 Hàm băm chuỗi . . . 67
2.1.1 Tính toán giá trị băm của một chuỗi . . . 68
2.1.2 Các bài toán ví dụ . . . 69
2.1.3 Ứng dụng của hàm băm . . . 70
2.1.4 Cải thiện xác suất không va chạm . . . 71
2.2 Thuật toán Rabin-Karp cho tìm kiếm chuỗi . . . 72
2.2.1 Thuật toán: . . . 72
2.2.2 Triển khai . . . 72

2.3 Hàm tiền tố. Thuật toán Knuth–Morris–Pratt . . . 73
2.3.1 Định nghĩa hàm tiền tố . . . 73
2.3.2 Thuật toán đơn giản . . . 73
2.3.3 Thuật toán hiệu quả . . . 74
2.3.4 Triển khai . . . 75
2.3.5 Ứng dụng . . . 76
2.4 Hàm Z và cách tính toán . . . 82
2.4.1 Ví dụ . . . 82
2.4.2 Thuật toán đơn giản . . . 82
2.4.3 Thuật toán hiệu quả để tính hàm Z . . . 83
2.4.4 Triển khai . . . 84
2.4.5 Bình luận về triển khai này . . . 84
2.4.6 Hành vi tiệm cận của thuật toán . . . 85
2.4.7 Ứng dụng . . . 86
2.5 Mảng hậu tố . . . 87
2.5.1 Định nghĩa . . . 87
2.5.2 Xây dựng . . . 88
2.5.3 Ứng dụng . . . 92

**3 Đồ thị 97**
3.1 Tìm kiếm theo chiều rộng . . . 97
3.1.1 Mô tả thuật toán . . . 97
3.1.2 Triển khai . . . 98
3.1.3 Ứng dụng của BFS . . . 99
3.2 Tìm kiếm theo chiều sâu . . . 100
3.2.1 Mô tả thuật toán . . . 100
3.2.2 Ứng dụng của Tìm kiếm theo chiều sâu . . . 101
3.2.3 Phân loại các cạnh của đồ thị . . . 102
3.2.4 Triển khai . . . 103
3.3 Tìm kiếm các thành phần liên thông trong đồ thị . . . 104
3.3.1 Thuật toán giải bài toán . . . 104
3.3.2 Triển khai . . . 104
3.3.3 Triển khai lặp . . . 105
3.4 Tìm cầu trong đồ thị trong $O(N + M)$ . . . 106
3.4.1 Thuật toán . . . 107
3.4.2 Triển khai . . . 107
3.5 Tìm cầu trực tuyến . . . 109
3.5.1 Thuật toán . . . 110
3.5.2 Cấu trúc dữ liệu để lưu trữ rừng . . . 110
3.5.3 Triển khai . . . 112

3.6 Tìm điểm phân cắt trong đồ thị trong $O(N+M)$ . . . 116
3.6.1 Thuật toán . . . 116
3.6.2 Triển khai . . . 117
3.7 Thành phần liên thông mạnh và đồ thị nén . . . 118
3.7.1 Định nghĩa . . . 118
3.7.2 Mô tả thuật toán . . . 120
3.7.3 Triển khai . . . 122
3.7.4 Tài liệu tham khảo . . . 124
3.8 Định hướng mạnh . . . 124
3.8.1 Giải pháp . . . 125
3.8.2 Mở rộng bài toán . . . 125
3.8.3 Triển khai . . . 125
3.9 Thuật toán Dijkstra . . . 127
3.9.1 Thuật toán . . . 127
3.9.2 Khôi phục đường đi ngắn nhất . . . 128
3.9.3 Chứng minh . . . 128
3.9.4 Triển khai . . . 129
3.9.5 Tài liệu tham khảo . . . 131
3.10 Dijkstra trên đồ thị thưa . . . 132
3.10.1 Thuật toán . . . 132
3.10.2 Triển khai . . . 133
3.11 Thuật toán Bellman-Ford . . . 135
3.11.1 Đường đi ngắn nhất từ một nguồn với cạnh có trọng số âm . . . 135
3.11.2 Mô tả thuật toán . . . 136
3.11.3 Triển khai . . . 136
3.11.4 Chứng minh thuật toán . . . 139
3.11.5 Trường hợp có chu trình âm . . . 140
3.11.6 Thuật toán đường đi ngắn nhất nhanh hơn (SPFA) . . . 142
3.12 BFS 0-1 . . . 143
3.12.1 Thuật toán . . . 143
3.12.2 Thuật toán của Dial . . . 145
3.13 Thuật toán D´Esopo-Pape . . . 145
3.13.1 Mô tả . . . 145
3.13.2 Triển khai . . . 146
3.13.3 Độ phức tạp . . . 147
3.14 Thuật toán Floyd-Warshall . . . 147
3.14.1 Mô tả thuật toán . . . 148
3.14.2 Triển khai . . . 149
3.14.3 Khôi phục chuỗi đỉnh trong đường đi ngắn nhất . . . 150
3.14.4 Trường hợp trọng số thực . . . 150

3.14.5 Trường hợp chu trình âm . . . . . 150
3.15 Số lượng đường đi có độ dài cố định / Đường đi ngắn nhất có độ dài cố định 151
3.15.1 Số lượng đường đi có độ dài cố định . . . . . 151
3.15.2 Đường đi ngắn nhất có độ dài cố định . . . . . 152
3.15.3 Khái quát hóa các bài toán cho đường đi có độ dài tối đa $k$ . . . . . 153
3.16 Cây bao trùm nhỏ nhất - Thuật toán Prim . . . . . 153
3.16.1 Thuật toán Prim . . . . . 154
3.16.2 Triển khai . . . . . 155
3.17 Cây bao trùm nhỏ nhất - Thuật toán Kruskal . . . . . 159
3.17.1 Đặc điểm của cây bao trùm nhỏ nhất . . . . . 159
3.17.2 Thuật toán Kruskal . . . . . 160
3.18 Cây bao trùm nhỏ nhất - Kruskal với Disjoint Set Union . . . . . 162
3.18.1 Mô tả . . . . . 162
3.18.2 Triển khai . . . . . 162
3.19 Cây bao trùm nhỏ nhất thứ hai . . . . . 164
3.19.1 Quan sát . . . . . 164
3.19.2 Sử dụng thuật toán Kruskal . . . . . 164
3.19.3 Mô hình hóa thành bài toán Tổ tiên chung thấp nhất (LCA) . . . . 165
3.19.4 Triển khai . . . . . 166
3.19.5 Tài liệu tham khảo . . . . . 169
3.20 Định lý Kirchhoff. Tìm số lượng cây bao trùm . . . . . 169
3.20.1 Định lý ma trận cây của Kirchhoff . . . . . 170
3.20.2 Mối liên hệ với định luật mạch điện của Kirchhoff . . . . . 170
3.21 Mã Prüfer . . . . . 170
3.21.1 Mã Prüfer . . . . . 171
3.21.2 Song ánh giữa cây và mã Prüfer . . . . . 176
3.21.3 Công thức Cayley . . . . . 176
3.21.4 Số cách làm cho một đồ thị liên thông . . . . . 177
3.22 Kiểm tra đồ thị có chu trình và tìm chu trình trong $O(M)$ . . . . . 178
3.22.1 Thuật toán . . . . . 178
3.22.2 Triển khai . . . . . 179
3.23 Tìm chu trình âm trong đồ thị . . . . . 181
3.23.1 Sử dụng thuật toán Bellman-Ford . . . . . 182
3.23.2 Sử dụng thuật toán Floyd-Warshall . . . . . 183
3.24 Tìm đường đi Euler trong $O(M)$ . . . . . 184
3.24.1 Thuật toán . . . . . 184
3.24.2 Bài toán Domino . . . . . 185
3.24.3 Triển khai . . . . . 185
3.25 Tổ tiên chung thấp nhất - $O(\sqrt{N})$ và $O(\log N)$ với tiền xử lý $O(N)$ . . . . 188
3.25.1 Ý tưởng của thuật toán . . . . . 188

3.25.2 Triển khai . . . . . 190
3.26 Tổ tiên chung thấp nhất - Nâng nhị phân . . . . . 191
3.26.1 Thuật toán . . . . . 192
3.26.2 Triển khai . . . . . 192
3.27 Tổ tiên chung thấp nhất - Thuật toán Farach-Colton và Bender . . . . . 194
3.27.1 Thuật toán . . . . . 194
3.27.2 Triển khai . . . . . 196
3.28 Giải bài toán RMQ (Truy vấn giá trị nhỏ nhất trong đoạn) bằng cách tìm LCA (Tổ tiên chung thấp nhất) . . . . . 200
3.28.1 Thuật toán . . . . . 200
3.28.2 Xây dựng cây Descartes . . . . . 201
3.29 Tổ tiên chung thấp nhất - Thuật toán ngoại tuyến của Tarjan . . . . . 202
3.29.1 Thuật toán . . . . . 203
3.29.2 Triển khai . . . . . 204
3.30 Luồng cực đại - Ford-Fulkerson và Edmonds-Karp . . . . . 205
3.30.1 Mạng luồng . . . . . 205
3.30.2 Phương pháp Ford-Fulkerson . . . . . 206
3.30.3 Thuật toán Edmonds-Karp . . . . . 209
3.30.4 Triển khai . . . . . 209
3.30.5 Định lý luồng nguyên . . . . . 212
3.30.6 Định lý luồng cực đại - cắt nhỏ nhất . . . . . 212
3.31 Luồng cực đại - Thuật toán Push-relabel . . . . . 213
3.31.1 Định nghĩa . . . . . 213
3.31.2 Thuật toán . . . . . 214
3.31.3 Độ phức tạp . . . . . 215
3.31.4 Triển khai . . . . . 215
3.32 Luồng cực đại - Phương pháp Push-relabel cải tiến . . . . . 217
3.32.1 Mô tả . . . . . 217
3.32.2 Triển khai . . . . . 218
3.33 Luồng cực đại - Thuật toán Dinic . . . . . 219
3.33.1 Định nghĩa . . . . . 220
3.33.2 Thuật toán . . . . . 220
3.33.3 Chứng minh tính đúng đắn . . . . . 220
3.33.4 Số lượng giai đoạn . . . . . 220
3.33.5 Tìm luồng chặn . . . . . 221
3.33.6 Độ phức tạp . . . . . 221
3.33.7 Mạng đơn vị . . . . . 221
3.33.8 Mạng có dung lượng đơn vị . . . . . 222
3.33.9 Triển khai . . . . . 222
3.34 Luồng cực đại - Thuật toán MPM . . . . . 224

3.34.1 Thuật toán . . . . . 224
3.34.2 Triển khai . . . . . 225
3.35 Luồng với Yêu Cầu . . . . . 229
3.35.1 Tìm một luồng bất kỳ . . . . . 230
3.35.2 Luồng tối thiểu . . . . . 231
3.36 Luồng Cực Tiểu Chi Phí - Thuật Toán Đường Đi Ngắn Nhất Liên Tiếp . . 231
3.36.1 Thuật Toán . . . . . 231
3.36.2 Độ Phức Tạp . . . . . 233
3.36.3 Triển Khai . . . . . 233
3.37 Giải Bài Toán Gán Việc Sử Dụng Luồng Cực Tiểu Chi Phí . . . . . 235
3.37.1 Mô Tả . . . . . 235
3.37.2 Triển Khai . . . . . 236
3.38 Kiểm Tra Xem Một Đồ Thị Có Phải Là Đồ Thị Hai Phía Hay Không . . . 238
3.38.1 Thuật Toán . . . . . 238
3.38.2 Triển Khai . . . . . 239
3.39 Thuật Toán Kuhn cho Tìm Kiếm Ghép Cặp Lớn Nhất trong Đồ Thị Hai Phía . . . . . 240
3.39.1 Bài Toán . . . . . 240
3.39.2 Mô Tả Thuật Toán . . . . . 240
3.39.3 Triển Khai . . . . . 243
3.39.4 Ghi Chú . . . . . 246
3.40 Thuật Toán Hungary để Giải Bài Toán Gán Việc . . . . . 246
3.40.1 Phát Biểu Bài Toán Gán Việc . . . . . 246
3.40.2 Thuật Toán Hungary . . . . . 247
3.40.3 Triển Khai Thuật Toán Hungary . . . . . 251
3.40.4 Kết Nối với Thuật Toán Đường Đi Ngắn Nhất Liên Tiếp . . . . . 253
3.40.5 Ví Dụ Bài Toán . . . . . 254
3.40.6 Tài Liệu Tham Khảo . . . . . 256
3.41 Sắp Xếp Tô-pô (Topological Sorting) . . . . . 257
3.41.1 Bài Toán . . . . . 257
3.41.2 Thuật Toán . . . . . 257
3.41.3 Triển Khai . . . . . 258
3.42 Kết Nối Cạnh / Kết Nối Đỉnh . . . . . 259
3.42.1 Định Nghĩa . . . . . 259
3.42.2 Tính Chất . . . . . 260
3.42.3 Tính Toán Các Giá Trị . . . . . 260
3.43 Phân Rã Heavy-Light . . . . . 261
3.43.1 Mô tả . . . . . 261
3.43.2 Thuật toán xây dựng . . . . . 261
3.43.3 Chứng minh tính đúng đắn . . . . . 262

3.43.4 Các bài toán mẫu . . . 263
3.43.5 Triển khai . . . 265

**4 Cấu trúc dữ liệu 269**
4.1 Stack Tối Thiểu / Queue Tối Thiểu . . . 269
4.1.1 Sửa đổi Stack . . . 269
4.1.2 Sửa đổi Queue (phương pháp 1) . . . 270
4.1.3 Sửa đổi Queue (phương pháp 2) . . . 271
4.1.4 Sửa đổi Queue (phương pháp 3) . . . 272
4.1.5 Tìm giá trị nhỏ nhất cho tất cả các mảng con có độ dài cố định . . 273
4.2 Bảng Thưa (Sparse Table) . . . 273
4.2.1 Trực giác . . . 273
4.2.2 Tính toán trước . . . 274
4.2.3 Truy vấn Tổng Phạm vi . . . 274
4.2.4 Truy vấn Phạm vi Tối thiểu (RMQ) . . . 275
4.2.5 Các cấu trúc dữ liệu tương tự hỗ trợ nhiều loại truy vấn hơn . . . 276
4.3 Tập Hợp Không Giao Nhau (Disjoint Set Union) . . . 277
4.3.1 Xây dựng một cấu trúc dữ liệu hiệu quả . . . 277
4.3.2 Triển khai ngây thơ . . . 278
4.3.3 Tối ưu hóa nén đường đi . . . 279
4.3.4 Hợp nhất theo kích thước / cấp bậc . . . 280
4.3.5 Độ phức tạp thời gian . . . 281
4.3.6 Liên kết theo chỉ số / Liên kết tung đồng xu . . . 282
4.3.7 Ứng dụng và các cải tiến khác nhau . . . 283
4.3.8 Hồi cố lịch sử . . . 291
4.4 Cây Fenwick . . . 292
4.5 Mô tả . . . 292
4.5.1 Tổng quan . . . 292
4.5.2 Định nghĩa của $g(i)$ . . . 294
4.6 Triển khai . . . 295
4.6.1 Tìm tổng trong mảng một chiều . . . 295
4.6.2 Xây dựng tuyến tính . . . 296
4.6.3 Tìm giá trị nhỏ nhất của $[0, r]$ trong mảng một chiều . . . 297
4.6.4 Tìm tổng trong mảng hai chiều . . . 298
4.6.5 Phương pháp lập chỉ mục dựa trên một . . . 298
4.6.6 Các thao tác phạm vi . . . 300
4.7 Phân Rã Căn Bậc Hai (Sqrt Decomposition) . . . 303
4.8 Cấu trúc dữ liệu dựa trên phân rã Sqrt . . . 303
4.8.1 Mô tả . . . 303
4.8.2 Triển khai . . . 304

4.9 Các vấn đề khác . . . 305
4.10 Thuật toán Mo . . . 306
4.10.1 Triển khai . . . 307
4.10.2 Độ phức tạp . . . 308
4.10.3 Lời khuyên để cải thiện thời gian chạy . . . 309
4.11 Cây Đoạn (Segment Tree) . . . 309
4.12 Dạng đơn giản nhất của Cây Đoạn . . . 310
4.12.1 Cấu trúc của Cây Đoạn . . . 310
4.12.2 Xây dựng . . . 311
4.12.3 Truy vấn tổng . . . 312
4.12.4 Truy vấn cập nhật . . . 314
4.13 Triển khai . . . 315
4.14 Triển khai tiết kiệm bộ nhớ . . . 317
4.15 Các phiên bản nâng cao của Cây Đoạn . . . 317
4.15.1 Các truy vấn phức tạp hơn . . . 317
4.16 Treap (Cây Descartes) . . . 323
4.16.1 Lợi ích của cách tổ chức dữ liệu này . . . 324
4.16.2 Các thao tác . . . 324
4.16.3 Mô tả triển khai . . . 325
4.16.4 Triển khai . . . 328
4.17 Treap Ngầm . . . 329
4.18 Tài liệu tham khảo . . . 333
4.19 Cây Căn Bậc Hai . . . 333
4.19.1 Mô tả . . . 334
4.19.2 Cập nhật phần tử . . . 336
4.19.3 Triển khai . . . 339
4.20 Heap Ngẫu Nhiên . . . 343
4.20.1 Cấu trúc dữ liệu . . . 343
4.20.2 Thao tác . . . 344
4.20.3 Độ phức tạp . . . 345
4.20.4 Độ phức tạp của thuật toán . . . 346
4.21 Xóa khỏi cấu trúc dữ liệu trong $O(T(n)\log n)$ . . . 346
4.21.1 Thuật toán . . . 346
4.21.2 Ghi chú . . . 346
4.21.3 Triển khai . . . 346

**5 Đại Số** **351**
5.1 Lũy thừa nhị phân . . . 351
5.1.1 Thuật toán . . . 351
5.1.2 Triển khai . . . 352
5.1.3 Ứng dụng . . . 353

5.2 Thuật toán Euclid để tính ước chung lớn nhất . . . 355
5.2.1 Thuật toán . . . 355
5.2.2 Triển khai . . . 356
5.2.3 Độ phức tạp thời gian . . . 356
5.2.4 Bội chung nhỏ nhất . . . 357
5.2.5 UCLN nhị phân . . . 357
5.3 Thuật toán Euclid mở rộng . . . 358
5.3.1 Thuật toán . . . 358
5.3.2 Triển khai . . . 359
5.3.3 Phiển bản không đệ quy . . . 360
5.4 Phương trình Diophantine tuyến tính . . . 361
5.4.1 Trường hợp thoái hóa . . . 361
5.4.2 Nghiệm phân tích . . . 361
5.4.3 Nghiệm thuật toán . . . 362
5.4.4 Lấy tất cả các nghiệm . . . 363
5.4.5 Tìm số lượng nghiệm và các nghiệm trong một khoảng cho trước . . 364
5.4.6 Tìm nghiệm có giá trị $x + y$ nhỏ nhất . . . 366
5.5 Số Fibonacci . . . 366
5.5.1 Tính chất . . . 366
5.5.2 Mã hóa Fibonacci . . . 367
5.5.3 Công thức cho số Fibonacci thứ $n$ . . . 368
5.6 Sàng Eratosthenes . . . 371
5.6.1 Triển khai . . . 372
5.6.2 Phân tích tiệm cận . . . 372
5.6.3 Các tối ưu khác nhau của Sàng Eratosthenes . . . 373
5.7 Sàng tuyến tính . . . 378
5.7.1 Thuật toán . . . 378
5.7.2 Triển khai . . . 379
5.7.3 Chứng minh tính đúng đắn . . . 379
5.7.4 Thời gian chạy và Bộ nhớ . . . 380
5.7.5 Tài liệu tham khảo . . . 380
5.8 Kiểm tra số nguyên tố . . . 380
5.8.1 Thử chia . . . 380
5.8.2 Kiểm tra nguyên tố Fermat . . . 381
5.8.3 Kiểm tra nguyên tố Miller-Rabin . . . 382
5.8.4 Phiên bản xác định . . . 384
5.9 Phân tích số nguyên . . . 385
5.9.1 Phân tích thử . . . 385
5.9.2 Phân tích bánh xe . . . 386
5.9.3 Số nguyên tố tính trước . . . 387

5.9.4 Phương pháp phân tích của Fermat . . . . . 388
5.9.5 Phương pháp Pollard $p-1$ . . . . . 388
5.9.6 Thuật toán rho của Pollard . . . . . 390
5.10 Hàm Totient của Euler . . . . . 392
5.10.1 Tính chất . . . . . 392
5.10.2 Triển khai . . . . . 393
5.10.3 Hàm totient từ 1 đến $n$ trong $O(n \log \log n)$ . . . . . 393
5.10.4 Tính chất tổng ước . . . . . 394
5.10.5 Tính totient từ 1 đến $n$ bằng tính chất tổng ước . . . . . 394
5.10.6 Ứng dụng trong định lý Euler . . . . . 395
5.10.7 Lý thuyết nhóm . . . . . 395
5.10.8 Khái quát hóa . . . . . 395
5.11 Số lượng ước số / Tổng các ước số . . . . . 396
5.11.1 Số lượng ước số . . . . . 396
5.11.2 Tổng các ước số . . . . . 398
5.11.3 Hàm nhân tính . . . . . 399
5.12 Nghịch đảo nhân modulo . . . . . 399
5.12.1 Định nghĩa . . . . . 399
5.12.2 Tìm nghịch đảo modulo bằng thuật toán Euclid mở rộng . . . . . 400
5.12.3 Tìm nghịch đảo modulo bằng lũy thừa nhị phân . . . . . 400
5.12.4 Tìm nghịch đảo modulo cho modulo nguyên tố bằng phép chia Euclid 401
5.12.5 Tìm nghịch đảo modulo cho mảng số modulo $m$ . . . . . 402
5.13 Phương trình đồng dư tuyến tính . . . . . 403
5.13.1 Giải bằng cách tìm phần tử nghịch đảo . . . . . 403
5.13.2 Giải bằng thuật toán Euclid mở rộng . . . . . 404
5.14 Định lý Thặng dư Trung Hoa . . . . . 404
5.14.1 Công thức . . . . . 404
5.14.2 Hệ quả . . . . . 405
5.14.3 Nghiệm cho hai modulo . . . . . 405
5.14.4 Nghiệm cho trường hợp tổng quát . . . . . 406
5.14.5 Triển khai . . . . . 406
5.14.6 Nghiệm cho các modulo không nguyên tố cùng nhau . . . . . 407
5.14.7 Thuật toán Garner . . . . . 408
5.15 Thuật toán Garner . . . . . 409
5.15.1 Biểu diễn cơ số hỗn hợp . . . . . 409
5.16 Thuật toán Garner . . . . . 409
5.17 Triển khai thuật toán Garner . . . . . 410
5.18 Giai thừa modulo $p$ . . . . . 412
5.18.1 Thuật toán . . . . . 413
5.18.2 Triển khai . . . . . 414

5.19 Bội số của $p$ . . . 415
5.20 Logarit rời rạc . . . 415
5.20.1 Thuật toán . . . 416
5.20.2 Độ phức tạp . . . 416
5.20.3 Triển khai . . . 417
5.20.4 Khi $a$ và $m$ không nguyên tố cùng nhau . . . 419
5.21 Căn nguyên thủy . . . 420
5.21.1 Định nghĩa . . . 420
5.21.2 Tính tồn tại . . . 421
5.21.3 Mối quan hệ với hàm Euler . . . 421
5.21.4 Thuật toán tìm căn nguyên thủy . . . 421
5.21.5 Triển khai . . . 422
5.22 Căn rời rạc . . . 423
5.22.1 Thuật toán . . . 423
5.22.2 Tìm tất cả các nghiệm từ một nghiệm đã biết . . . 424
5.22.3 Triển khai . . . 424
5.23 Phép nhân Montgomery . . . 426
5.23.1 Biểu diễn Montgomery . . . 427
5.23.2 Giảm Montgomery . . . 427
5.23.3 Mẹo tính nghịch đảo nhanh . . . 428
5.23.4 Triển khai . . . 429
5.23.5 Chuyển đổi nhanh . . . 430
5.24 Hệ tam phân cân bằng . . . 431
5.24.1 Thuật toán chuyển đổi . . . 432
5.25 Mã Gray . . . 433
5.25.1 Tìm mã Gray . . . 434
5.25.2 Tìm mã Gray ngược . . . 434
5.25.3 Ứng dụng thực tiễn . . . 434
5.26 Thao tác bit . . . 435
5.26.1 Số nhị phân . . . 435
5.26.2 Các toán tử bit . . . 436
5.26.3 Mẹo hữu ích . . . 437
5.26.4 Thuật toán của Brian Kernighan . . . 439
5.26.5 Đếm số bit được đặt đến $n$ . . . 439
5.26.6 Các mẹo bổ sung . . . 440
5.26.7 Hỗ trợ ngôn ngữ và trình biên dịch . . . 440
5.27 Liệt kê tất cả các mặt nạ con . . . 441
5.27.1 Liệt kê tất cả các mặt nạ con của một mặt nạ cho trước . . . 441
5.27.2 Duyệt qua tất cả các mặt nạ với các mặt nạ con của chúng. Độ phức tạp $O(3^n)$ . . . 442

5.28 Số học chính xác tùy ý . . . 443
5.28.1 Số học dài nguyên cổ điển . . . 443
5.28.2 Số học dài nguyên theo biểu diễn phân tích thừa số . . . 446
5.28.3 Số học dài nguyên trong các modulo nguyên tố (Thuật toán Garner) 447
5.28.4 Số học chính xác tùy ý phân số . . . 447
5.29 Biến đổi Fourier Nhanh . . . 448
5.29.1 Biến đổi Fourier Rời rạc . . . 448
5.29.2 Ứng dụng của DFT: Nhân đa thức nhanh . . . 449
5.29.3 Biến đổi Fourier Nhanh . . . 450
5.29.4 FFT Ngược . . . 451
5.29.5 Triển khai . . . 452
5.29.6 Triển khai cải tiến: Tính toán tại chỗ . . . 454
5.29.7 Biến đổi Số học . . . 458
5.29.8 Nhân với modulo tùy ý . . . 460
5.29.9 Ứng dụng . . . 461
5.30 Biến đổi Số học . . . 463
5.30.1 Nhân với modulo tùy ý . . . 465
5.30.2 Ứng dụng . . . 466
5.31 Phép toán trên đa thức và chuỗi . . . 468
5.31.1 Khái niệm cơ bản và các dữ kiện . . . 468
5.31.2 Triển khai cơ bản . . . 471
5.31.3 Số học . . . 472
5.31.4 Tính các hàm của đa thức . . . 474
5.31.5 Đánh giá và Nội suy . . . 477
5.31.6 UCLN và Kết quả . . . 478
5.32 Tính Lũy Thừa Nhị Phân Bằng Phân Tích Thừa Số . . . 480
5.32.1 Tính $4L(x)$ từ $x$ . . . 481
5.32.2 Tính $x$ từ $4L(x)$ . . . 482
5.32.3 Tối ưu hóa thêm . . . 483
5.32.4 Tính bảng logarit . . . 484

**6 Quy hoạch động 485**
6.1 Giới thiệu về Quy hoạch Động . . . 485
6.1.1 Tăng tốc Fibonacci bằng Quy hoạch Động (Ghi nhớ) . . . 485
6.1.2 Quy hoạch Động Từ Dưới Lên . . . 487
6.1.3 Các Bài Toán Quy hoạch Động Kinh Điển . . . 489
6.1.4 Các Chủ Đề Liên Quan . . . 489
6.2 Bài toán Cái Túi (Knapsack Problem) . . . 490
6.2.1 Kiến thức cần thiết: Nhập môn Quy hoạch động . . . 490
6.2.2 Giới thiệu . . . 490

6.3 Bài toán Cái Túi 0-1 . . . . . 490
6.3.1 Giải thích . . . . . 490
6.3.2 Triển khai . . . . . 491
6.4 Bài toán Cái Túi Hoàn Chỉnh (Complete Knapsack) . . . . . 491
6.4.1 Giải thích . . . . . 491
6.4.2 Triển khai . . . . . 492
6.5 Bài toán Cái Túi Bội Số (Multiple Knapsack) . . . . . 492
6.5.1 Giải thích . . . . . 492
6.5.2 Tối ưu hóa Nhóm Nhị Phân (Binary Grouping Optimization) . . . . 492
6.5.3 Triển khai . . . . . 493
6.6 Tối ưu hóa Hàng đợi Đơn điệu (Monotone Queue Optimization) . . . . . 493
6.7 Bài toán Cái Túi Hỗn Hợp (Mixed Knapsack) . . . . . 494
6.8 Quy hoạch động Chia để trị (Divide and Conquer DP) . . . . . 494
6.8.1 Điều kiện tiên quyết . . . . . 495
6.8.2 Triển khai tổng quát . . . . . 495
6.8.3 Những điều cần lưu ý . . . . . 496
6.9 Tối ưu hóa Knuth (Knuth's Optimization) . . . . . 497
6.9.1 Điều kiện . . . . . 497
6.9.2 Thuật toán . . . . . 497
6.9.3 Triển khai tổng quát . . . . . 497
6.9.4 Độ phức tạp . . . . . 498
6.9.5 Trong thực tế . . . . . 499
6.9.6 Chứng minh tính đúng đắn . . . . . 499
6.10 Quy hoạch động trên Profile bị hỏng. Bài toán "Parquet" . . . . . 501
6.11 Bài toán "Parquet" . . . . . 501
6.11.1 Mô tả bài toán . . . . . 501
6.11.2 Định nghĩa trạng thái quy hoạch động . . . . . 501
6.11.3 Triển khai . . . . . 502
6.12 Tìm ma trận con toàn số 0 lớn nhất . . . . . 503
6.13 Thuật toán . . . . . 503
6.13.1 Bước 1: Quy hoạch động phụ trợ . . . . . 503
6.13.2 Bước 2: Giải quyết bài toán . . . . . 504
6.14 Triển khai . . . . . 505

**7 Phương pháp số 507**
7.1 Tìm kiếm nhị phân (Binary Search) . . . . . 507
7.2 Tìm kiếm trong mảng đã sắp xếp . . . . . 507
7.3 Cận dưới và cận trên . . . . . 508
7.4 Triển khai . . . . . 509
7.5 Tìm kiếm trên vị từ tùy ý . . . . . 510
7.6 Tìm kiếm nhị phân trên đáp án . . . . . 510

7.7 Tìm kiếm liên tục . . . 511
7.8 Tìm kiếm với lũy thừa của 2 . . . 511
7.9 Tìm kiếm Tam phân (Ternary Search) . . . 512
7.10 Thuật toán . . . 512
7.11 Phân tích thời gian chạy . . . 513
7.12 Trường hợp đối số số nguyên . . . 513
7.13 Triển khai . . . 513
7.14 Phương pháp Newton để tìm nghiệm . . . 514
7.15 Thuật toán . . . 514
7.16 Ứng dụng để tính căn bậc hai . . . 516
7.17 Tích phân bằng công thức Simpson . . . 517
7.18 Công thức Simpson . . . 517
7.19 Sai số . . . 518
7.20 Triển khai . . . 518

**8 Vài thứ linh tinh 521**
8.1 Truy vấn Tìm kiếm Giá trị Nhỏ nhất trong Khoảng (Range Minimum Query) 521
8.2 Giải pháp . . . 521
8.3 Dãy con tăng dài nhất (Longest Increasing Subsequence) . . . 522
8.4 Giải pháp trong $O(n^2)$ với quy hoạch động . . . 522
8.4.1 Tìm độ dài . . . 523
8.4.2 Triển khai . . . 523
8.4.3 Khôi phục dãy con . . . 524
8.4.4 Triển khai khôi phục . . . 524
8.4.5 Cách thay thế để khôi phục dãy con . . . 525
8.5 Giải pháp trong $O(n \log n)$ với quy hoạch động và tìm kiếm nhị phân . . . 525
8.5.1 Triển khai . . . 527
8.5.2 Khôi phục dãy con . . . 528
8.6 Giải pháp trong $O(n \log n)$ với cấu trúc dữ liệu . . . 528
8.7 Các nhiệm vụ liên quan . . . 529
8.7.1 Dãy con không giảm dài nhất . . . 529
8.7.2 Số lượng dãy con tăng dài nhất . . . 529
8.7.3 Số lượng nhỏ nhất của các dãy con không tăng bao phủ một dãy . . . 530
8.8 Tìm mảng con có tổng lớn nhất/nhỏ nhất . . . 530
8.9 Phát biểu bài toán . . . 530
8.10 Thuật toán 1 . . . 531
8.10.1 Mô tả thuật toán . . . 531
8.10.2 Triển khai . . . 531
8.11 Thuật toán 2 . . . 532
8.11.1 Mô tả thuật toán . . . 532
8.11.2 Triển khai . . . 533

8.12 Các nhiệm vụ liên quan . . . . . 534
8.12.1 Tìm mảng con lớn nhất/nhỏ nhất với các ràng buộc . . . . . 534
8.12.2 Trường hợp hai chiều của bài toán: tìm kiếm ma trận con lớn nhất/nhỏ nhất . . . . . 534
8.12.3 Tìm kiếm một mảng con có giá trị trung bình lớn nhất/nhỏ nhất . 535
8.12.4 Giải quyết bài toán trực tuyến . . . . . 535
8.13 Thống kê thứ tự thứ $K$ trong $O(N)$ . . . . . 535
8.14 Triển khai (không đệ quy) . . . . . 536
8.15 Ghi chú . . . . . 537
8.16 MEX (minimal excluded) của một dãy . . . . . 537
8.17 Triển khai . . . . . 538
8.18 MEX với cập nhật mảng . . . . . 539
8.19 Các trò chơi trên đồ thị tùy ý . . . . . 540
8.20 Mô tả thuật toán . . . . . 541
8.21 Triển khai . . . . . 542
8.22 Ví dụ: "Cảnh sát và tên trộm" . . . . . 543
8.23 Các trò chơi trên đồ thị tùy ý . . . . . 547
8.24 Mô tả thuật toán . . . . . 547
8.25 Triển khai . . . . . 548
8.26 Ví dụ: "Cảnh sát và tên trộm" . . . . . 549
8.27 Lên lịch công việc trên một máy . . . . . 553
8.28 Các giải pháp cho các trường hợp đặc biệt . . . . . 553
8.28.1 Hàm phạt tuyến tính . . . . . 553
8.28.2 Hàm phạt mũ . . . . . 554
8.28.3 Hàm phạt đơn điệu giống hệt nhau . . . . . 554
8.29 Định lý Livshits-Kladov . . . . . 554
8.30 Lên lịch công việc trên hai máy . . . . . 555
8.31 Xây dựng thuật toán . . . . . 555
8.32 Triển khai . . . . . 557

# Chương 1

# Tổ hợp

## 1.1 Tìm lũy thừa của ước giai thừa

Cho hai số $n$ và $k$. Tìm lũy thừa lớn nhất $x$ của $k$ sao cho $n!$ chia hết cho $k^x$.

### 1.1.1 Trường hợp $k$ là số nguyên tố

Trước tiên, hãy xem xét trường hợp $k$ là số nguyên tố. Công thức tường minh cho giai thừa là:

$$n! = 1 \cdot 2 \cdot 3 \ldots (n-1) \cdot n$$

Lưu ý rằng mỗi phần tử thứ $k$ của tích chia hết cho $k$, tức là thêm $+1$ vào kết quả; số lượng các phần tử như vậy là:

$$\left\lfloor \frac{n}{k} \right\rfloor$$

Tiếp theo, mỗi phần tử thứ $k^2$ chia hết cho $k^2$, tức là thêm một $+1$ nữa vào kết quả (lũy thừa đầu tiên của $k$ đã được tính ở đoạn trước). Số lượng các phần tử như vậy là:

$$\left\lfloor \frac{n}{k^2} \right\rfloor$$

Và cứ tiếp tục như vậy, với mỗi $i$, mỗi phần tử thứ $k^i$ thêm một $+1$ nữa vào kết quả, và có:

$$\left\lfloor \frac{n}{k^i} \right\rfloor$$

các phần tử như vậy.

Kết quả cuối cùng là:

$$\left\lfloor \frac{n}{k} \right\rfloor + \left\lfloor \frac{n}{k^2} \right\rfloor + \ldots + \left\lfloor \frac{n}{k^i} \right\rfloor + \ldots$$

Kết quả này còn được gọi là công thức Legendre. Tổng này tất nhiên là hữu hạn, vì chỉ khoảng $\log_k n$ phần tử đầu tiên không bằng 0. Do đó, thời gian chạy của thuật toán này là $O(\log_k n)$.

**Triển khai**

```
int fact_pow(int n, int k) {
    int res = 0;
    while (n) {
        n /= k;
        res += n;
    }
    return res;
}
```

### 1.1.2 Trường hợp $k$ là số hợp số

Ý tưởng tương tự không thể áp dụng trực tiếp. Thay vào đó, ta có thể phân tích $k$, biểu diễn nó dưới dạng:

$$k = k_1^{p_1} \cdot \ldots \cdot k_m^{p_m}$$

Với mỗi $k_i$, ta tìm số lần nó xuất hiện trong $n!$ bằng cách sử dụng thuật toán đã mô tả ở trên - gọi giá trị này là $a_i$. Kết quả cho $k$ hợp số sẽ là:

$$\min_{i=1\ldots m} \frac{a_i}{p_i}$$

## 1.2 Hệ số nhị thức

Hệ số nhị thức $\binom{n}{k}$ là số cách để chọn một tập hợp gồm $k$ phần tử từ $n$ phần tử khác nhau mà không tính đến thứ tự sắp xếp của các phần tử này (tức là số tập hợp không thứ tự).

Hệ số nhị thức cũng là các hệ số trong khai triển của $(a+b)^n$ (định lý nhị thức):

$$(a+b)^n = \binom{n}{0}a^n + \binom{n}{1}a^{n-1}b + \binom{n}{2}a^{n-2}b^2 + \cdots + \binom{n}{k}a^{n-k}b^k + \cdots + \binom{n}{n}b^n$$

Người ta tin rằng công thức này, cùng với tam giác cho phép tính toán hiệu quả các hệ số, được Blaise Pascal phát hiện vào thế kỷ 17. Tuy nhiên, nó đã được nhà toán học

Trung Quốc Dương Huy biết đến, người sống vào thế kỷ 13. Có lẽ nó cũng được học giả Ba Tư Omar Khayyam khám phá. Hơn nữa, nhà toán học Ấn Độ Pingala, sống sớm hơn vào thế kỷ 3 trước Công nguyên, cũng đạt được kết quả tương tự. Công lao của Newton là đã khái quát hóa công thức này cho các lũy thừa không phải là số tự nhiên.

### 1.2.1 Tính toán

Công thức phân tích để tính toán:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Công thức này có thể dễ dàng suy ra từ bài toán sắp xếp có thứ tự (số cách chọn $k$ phần tử khác nhau từ $n$ phần tử khác nhau). Đầu tiên, hãy đếm số cách chọn có thứ tự của $k$ phần tử. Có $n$ cách để chọn phần tử đầu tiên, $n-1$ cách để chọn phần tử thứ hai, $n-2$ cách để chọn phần tử thứ ba, và cứ tiếp tục như vậy. Kết quả là, ta được công thức cho số sắp xếp có thứ tự:

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!}$$

Ta có thể dễ dàng chuyển sang sắp xếp không thứ tự, lưu ý rằng mỗi sắp xếp không thứ tự tương ứng với đúng $k!$ sắp xếp có thứ tự ($k!$ là số hoán vị có thể có của $k$ phần tử). Công thức cuối cùng thu được bằng cách chia $\frac{n!}{(n-k)!}$ cho $k!$.

Công thức đệ quy (liên quan đến "Tam giác Pascal" nổi tiếng):

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Có thể dễ dàng suy ra điều này bằng cách sử dụng công thức phân tích.

Lưu ý rằng khi $n < k$, giá trị của $\binom{n}{k}$ được giả định là 0.

### 1.2.2 Tính chất

Hệ số nhị thức có nhiều tính chất khác nhau. Dưới đây là những tính chất đơn giản nhất:

Quy tắc đối xứng:

$$\binom{n}{k} = \binom{n}{n-k}$$

Phân tích nhân tử:

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

Tổng theo $k$:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n$$

Tổng theo $n$:

$$\sum_{m=0}^{n} \binom{m}{k} = \binom{n+1}{k+1}$$

Tổng theo $n$ và $k$:

$$\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m}$$

Tổng các bình phương:

$$\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$$

Tổng có trọng số:

$$1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n2^{n-1}$$

Liên hệ với số Fibonacci:

$$\binom{n}{0} + \binom{n-1}{1} + \cdots + \binom{n-k}{k} + \cdots + \binom{0}{n} = F_{n+1}$$

### 1.2.3 Tính toán

**Tính toán trực tiếp bằng công thức phân tích**

Công thức đơn giản đầu tiên rất dễ lập trình, nhưng phương pháp này dễ gây tràn số ngay cả với các giá trị $n$ và $k$ tương đối nhỏ (ngay cả khi kết quả hoàn toàn nằm trong một kiểu dữ liệu nào đó, việc tính toán các giai thừa trung gian có thể dẫn đến tràn). Do đó, phương pháp này thường chỉ có thể được sử dụng với số học dài:

```
int C(int n, int k) {
    int res = 1;
    for (int i = n - k + 1; i <= n; ++i)
        res *= i;
    for (int i = 2; i <= k; ++i)
```

```
        res /= i;
    return res;
}
```

**Triển khai cải tiến**

Lưu ý rằng trong triển khai trên, tử số và mẫu số có cùng số lượng nhân tử ($k$), mỗi nhân tử đều lớn hơn hoặc bằng 1. Do đó, ta có thể thay thế phân số của mình bằng tích của $k$ phân số, mỗi phân số là một số thực. Tuy nhiên, sau mỗi bước khi nhân kết quả hiện tại với từng phân số tiếp theo, kết quả vẫn sẽ là số nguyên (điều này xuất phát từ tính chất phân tích nhân tử).

Triển khai bằng C++:

```
int C(int n, int k) {
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (int)(res + 0.01);
}
```

Ở đây, ta cẩn thận ép kiểu số thực sang số nguyên, tính đến việc do lỗi tích lũy, giá trị có thể hơi nhỏ hơn giá trị thực (ví dụ, 2.99999 thay vì 3).

## 1.3 Số Catalan

Số Catalan là một dãy số được tìm thấy hữu ích trong nhiều bài toán tổ hợp, thường liên quan đến các đối tượng được định nghĩa đệ quy.

Dãy số này được đặt tên theo nhà toán học người Bỉ Catalan, người sống vào thế kỷ 19. (Thực tế, nó đã được Euler biết đến trước đó, người sống sớm hơn Catalan một thế kỷ).

Một vài số Catalan đầu tiên $C_n$ (bắt đầu từ 0):

$$1, 1, 2, 5, 14, 42, 132, 429, 1430, \ldots$$

### 1.3.1 Ứng dụng trong một số bài toán tổ hợp

Số Catalan $C_n$ là lời giải cho các bài toán sau:

- Số dãy dấu ngoặc đúng gồm $n$ dấu ngoặc mở và $n$ dấu ngoặc đóng.
- Số cây nhị phân đầy đủ có gốc với $n+1$ lá (các đỉnh không được đánh số). Một cây nhị phân có gốc là đầy đủ nếu mỗi đỉnh có hoặc hai con hoặc không có con.

- Số cách đặt dấu ngoặc hoàn toàn cho $n+1$ thừa số.
- Số cách phân chia một đa giác lồi có $n+2$ cạnh thành các tam giác không giao nhau bằng cách sử dụng các đường chéo.
- Số cách nối $2n$ điểm trên một đường tròn để tạo thành $n$ dây cung không giao nhau.
- Số cây nhị phân đầy đủ không đồng dạng với $n$ nút bên trong (tức là các nút có ít nhất một con).
- Số đường đi đơn điệu trên lưới từ điểm $(0,0)$ đến điểm $(n,n)$ trong lưới vuông kích thước $n \times n$, không vượt qua đường chéo chính (tức là nối $(0,0)$ đến $(n,n)$).
- Số hoán vị độ dài $n$ có thể được sắp xếp bằng ngăn xếp (tức là có thể chứng minh rằng hoán vị đó được sắp xếp bằng ngăn xếp nếu và chỉ nếu không tồn tại chỉ số $i < j < k$ sao cho $a_k < a_i < a_j$).
- Số phân hoạch không giao nhau của tập hợp $n$ phần tử.
- Số cách phủ thang $1 \ldots n$ bằng $n$ hình chữ nhật (Thang gồm $n$ cột, trong đó cột thứ $i$ có chiều cao $i$).

### 1.3.2 Tính toán

Có hai công thức cho số Catalan: Công thức đệ quy và công thức phân tích. Vì chúng ta tin rằng tất cả các bài toán được đề cập ở trên là tương đương (có cùng lời giải), để chứng minh các công thức dưới đây, chúng ta sẽ chọn bài toán dễ thực hiện nhất.

**Công thức đệ quy**

$$C_0 = C_1 = 1$$

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}, \quad n \geq 2$$

Công thức đệ quy này có thể dễ dàng suy ra từ bài toán về dãy dấu ngoặc đúng.

Dấu ngoặc mở ngoài cùng bên trái $l$ tương ứng với một dấu ngoặc đóng $r$, chia dãy thành hai phần, mỗi phần lần lượt phải là một dãy dấu ngoặc đúng. Do đó, công thức cũng được chia thành hai phần. Nếu ta ký hiệu $k = r - l - 1$, thì với $r$ cố định, sẽ có chính xác $C_k C_{n-1-k}$ dãy dấu ngoặc như vậy. Tổng hợp trên tất cả các $k$ hợp lệ, ta được quan hệ đệ quy cho $C_n$.

Bạn cũng có thể nghĩ theo cách này. Theo định nghĩa, $C_n$ biểu thị số dãy dấu ngoặc đúng. Bây giờ, dãy có thể được chia thành hai phần có độ dài $k$ và $n-k$, mỗi phần phải là một dãy dấu ngoặc đúng. Ví dụ:

$$()(())$$

có thể được chia thành () và (()), nhưng không thể chia thành ()( và ()). Tổng hợp trên tất cả các $k$ hợp lệ, ta lại được quan hệ đệ quy cho $C_n$.

**Triển khai bằng C++:**

```
const int MOD = ....;
const int MAX = ....;
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
            if (catalan[i] >= MOD) {
                catalan[i] -= MOD;
            }
        }
    }
}
```

**Công thức phân tích**

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

(ở đây $\binom{n}{k}$ biểu thị hệ số nhị thức thông thường, tức là số cách chọn $k$ đối tượng từ tập hợp $n$ đối tượng).

Công thức trên có thể dễ dàng kết luận từ bài toán về đường đi đơn điệu trong lưới vuông. Tổng số đường đi đơn điệu trong lưới kích thước $n \times n$ được cho bởi $\binom{2n}{n}$.

Bây giờ, ta đếm số đường đi đơn điệu vượt qua đường chéo chính. Xét các đường đi như vậy vượt qua đường chéo chính và tìm cạnh đầu tiên nằm phía trên đường chéo. Phản xạ đường đi về phía đường chéo từ cạnh này trở đi. Kết quả luôn là một đường đi đơn điệu trong lưới $(n-1) \times (n+1)$. Mặt khác, bất kỳ đường đi đơn điệu nào trong lưới $(n-1) \times (n+1)$ đều phải giao với đường chéo. Do đó, ta đã liệt kê tất cả các đường đi đơn điệu vượt qua đường chéo chính trong lưới $n \times n$.

Số đường đi đơn điệu trong lưới $(n-1) \times (n+1)$ là $\binom{2n}{n-1}$. Gọi các đường đi này là "đường đi xấu". Kết quả là, để có được số đường đi đơn điệu không vượt qua đường chéo chính, ta trừ đi các "đường đi xấu" ở trên, thu được công thức:

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}, \quad n \geq 0$$

## 1.4 Nguyên lý bao hàm - loại trừ

Nguyên lý bao hàm - loại trừ là một phương pháp tổ hợp quan trọng để tính kích thước của một tập hợp hoặc xác suất của các sự kiện phức tạp. Nó liên kết kích thước của các tập hợp riêng lẻ với hợp của chúng.

### 1.4.1 Phát biểu

#### Công thức bằng lời

Nguyên lý bao hàm - loại trừ có thể được diễn đạt như sau:

Để tính kích thước của hợp của nhiều tập hợp, cần cộng kích thước của các tập hợp này riêng lẻ, sau đó trừ đi kích thước của tất cả các giao đôi của các tập hợp, rồi cộng lại kích thước của các giao ba của các tập hợp, trừ đi kích thước của các giao bốn, và tiếp tục như vậy, cho đến giao của tất cả các tập hợp.

#### Công thức bằng tập hợp

Định nghĩa trên có thể được biểu diễn toán học như sau:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{i=1}^{n} |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1} |A_1 \cap \cdots \cap A_n|$$

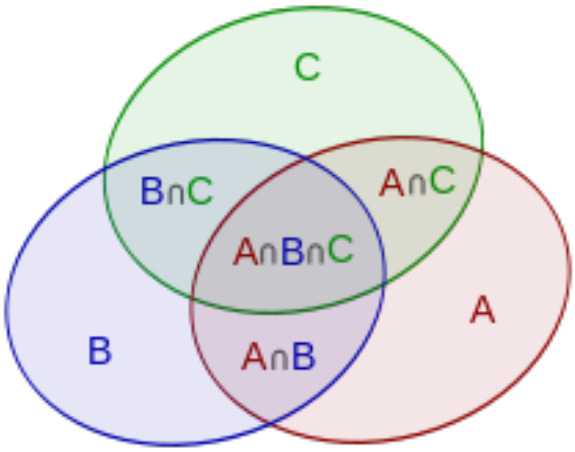
Và theo cách ngắn gọn hơn:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

#### Công thức bằng sơ đồ Venn

Giả sử sơ đồ thể hiện ba tập hợp $A$, $B$ và $C$:

Khi đó diện tích của hợp $A \cup B \cup C$ bằng tổng diện tích $A$, $B$ và $C$, trừ đi các diện tích bị phủ đôi $A \cap B$, $A \cap C$, $B \cap C$, nhưng cộng lại diện tích bị phủ bởi ba tập $A \cap B \cap C$:




Hình 1.1: Sơ đồ venn

$$S(A \cup B \cup C) = S(A) + S(B) + S(C) - S(A \cap B) - S(A \cap C) - S(B \cap C) + S(A \cap B \cap C)$$

Công thức này cũng có thể được khái quát hóa cho hợp của $n$ tập hợp.

**Công thức trong lý thuyết xác suất**

Nếu $A_i$ $(i = 1, 2, \ldots, n)$ là các sự kiện và $\mathcal{P}(A_i)$ là xác suất xảy ra sự kiện $A_i$, thì xác suất của hợp của chúng (tức là xác suất để ít nhất một sự kiện xảy ra) bằng:

$$\begin{aligned}\mathcal{P}\left(\bigcup_{i=1}^{n} A_i\right) = &\sum_{i=1}^{n} \mathcal{P}(A_i) - \sum_{1 \le i < j \le n} \mathcal{P}(A_i \cap A_j) \\ &+ \sum_{1 \le i < j < k \le n} \mathcal{P}(A_i \cap A_j \cap A_k) - \cdots \\ &+ (-1)^{n-1} \mathcal{P}(A_1 \cap \cdots \cap A_n)\end{aligned}$$

Và theo cách ngắn gọn hơn:

$$\mathcal{P}\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{\emptyset \ne J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \mathcal{P}\left(\bigcap_{j \in J} A_j\right)$$

### 1.4.2 Chứng minh

Để chứng minh, tiện lợi nhất là sử dụng công thức toán học trong lý thuyết tập hợp:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

Chúng ta muốn chứng minh rằng bất kỳ phần tử nào nằm trong ít nhất một tập $A_i$ sẽ chỉ được đếm một lần trong công thức (lưu ý rằng các phần tử không có trong bất kỳ tập $A_i$ nào sẽ không bao giờ xuất hiện ở vế phải của công thức).

Xét một phần tử $x$ xuất hiện trong $k \geq 1$ tập $A_i$. Chúng ta sẽ chứng minh nó chỉ được đếm một lần trong công thức. Lưu ý rằng:

- Trong các hạng mà $|J| = 1$, phần tử $x$ sẽ được đếm $+k$ lần;
- Trong các hạng mà $|J| = 2$, phần tử $x$ sẽ được đếm $-\binom{k}{2}$ lần - vì nó sẽ được đếm trong các hạng bao gồm hai trong số $k$ tập chứa $x$;
- Trong các hạng mà $|J| = 3$, phần tử $x$ sẽ được đếm $+\binom{k}{3}$ lần;
- $\cdots$
- Trong các hạng mà $|J| = k$, phần tử $x$ sẽ được đếm $(-1)^{k-1} \cdot \binom{k}{k}$ lần;
- Trong các hạng mà $|J| > k$, phần tử $x$ sẽ được đếm 0 lần;

Điều này dẫn chúng ta đến tổng của các hệ số nhị thức sau:

$$T = \binom{k}{1} - \binom{k}{2} + \binom{k}{3} - \cdots + (-1)^{i-1} \cdot \binom{k}{i} + \cdots + (-1)^{k-1} \cdot \binom{k}{k}$$

Biểu thức này rất giống với khai triển nhị thức của $(1-x)^k$:

$$(1-x)^k = \binom{k}{0} - \binom{k}{1} \cdot x + \binom{k}{2} \cdot x^2 - \binom{k}{3} \cdot x^3 + \cdots + (-1)^k \cdot \binom{k}{k} \cdot x^k$$

Khi $x = 1$, $(1-x)^k$ trông rất giống $T$. Tuy nhiên, biểu thức này có thêm $\binom{k}{0} = 1$ và được nhân với $-1$. Điều đó dẫn đến $(1-1)^k = 1 - T$. Do đó $T = 1 - (1-1)^k = 1$, điều cần chứng minh. Phần tử chỉ được đếm một lần.

### 1.4.3 Khái quát hóa để tính số phần tử nằm trong đúng $r$ tập

Nguyên lý bao hàm - loại trừ có thể được viết lại để tính số phần tử không nằm trong tập nào:

$$\left|\bigcap_{i=1}^{n}\overline{A_i}\right| = \sum_{m=0}^{n}(-1)^m \sum_{|X|=m}\left|\bigcap_{i\in X} A_i\right|$$

Xem xét khái quát hóa của nó để tính số phần tử nằm trong đúng $r$ tập:

$$\left|\bigcup_{|B|=r}\left[\bigcap_{i\in B} A_i \cap \bigcap_{j\notin B}\overline{A_j}\right]\right| = \sum_{m=r}^{n}(-1)^{m-r}\binom{m}{r}\sum_{|X|=m}\left|\bigcap_{i\in X} A_i\right|$$

Để chứng minh công thức này, xét một $B$ cụ thể. Theo nguyên lý bao hàm - loại trừ cơ bản, ta có thể nói rằng:

$$\left|\bigcap_{i\in B} A_i \cap \bigcap_{j\notin B}\overline{A_j}\right| = \sum_{m=r}^{n}(-1)^{m-r}\sum_{\substack{|X|=m\\ B\subset X}}\left|\bigcap_{i\in X} A_i\right|$$

Các tập ở vế trái không giao nhau với các $B$ khác nhau, do đó ta có thể cộng chúng trực tiếp. Cũng cần lưu ý rằng bất kỳ tập $X$ nào cũng sẽ luôn có hệ số $(-1)^{m-r}$ nếu nó xuất hiện và nó sẽ xuất hiện trong đúng $\binom{m}{r}$ tập $B$.

### 1.4.4 Sử dụng khi giải bài toán

Nguyên lý bao hàm - loại trừ khó hiểu nếu không nghiên cứu các ứng dụng của nó.

Trước tiên, chúng ta sẽ xem xét ba bài toán đơn giản nhất "trên giấy", minh họa ứng dụng của nguyên lý, sau đó xem xét các bài toán thực tế hơn, khó giải nếu không có nguyên lý bao hàm - loại trừ.

Các bài toán yêu cầu "tìm số cách" đáng chú ý, vì đôi khi chúng dẫn đến các nghiệm đa thức, không nhất thiết là lũy thừa.

### 1.4.5 Bài toán đơn giản về hoán vị

**Bài toán:** Đếm số hoán vị của các số từ 0 đến 9 sao cho phần tử đầu tiên lớn hơn 1 và phần tử cuối nhỏ hơn 8.

Hãy đếm số hoán vị "xấu", tức là các hoán vị mà phần tử đầu tiên $\leq 1$ và/hoặc phần tử cuối $\geq 8$.

Gọi $X$ là tập các hoán vị mà phần tử đầu tiên $\leq 1$ và $Y$ là tập các hoán vị mà phần tử cuối $\geq 8$. Khi đó, số hoán vị "xấu" theo công thức bao hàm - loại trừ sẽ là:

$$|X \cup Y| = |X| + |Y| - |X \cap Y|$$

Sau một phép tính tổ hợp đơn giản, ta được:

$$2 \cdot 9! + 2 \cdot 9! - 2 \cdot 2 \cdot 8!$$

Chỉ cần trừ số này khỏi tổng số 10! để được số hoán vị "tốt".

### 1.4.6 Bài toán đơn giản về dãy (0, 1, 2)

**Bài toán:** Đếm số dãy có độ dài $n$ chỉ gồm các số $0, 1, 2$ sao cho mỗi số xuất hiện ít nhất một lần.

Hãy chuyển sang bài toán ngược, tức là tính số dãy không chứa ít nhất một trong các số.

Gọi $A_i (i = 0, 1, 2)$ là tập các dãy mà chữ số $i$ không xuất hiện. Công thức bao hàm - loại trừ cho số dãy "xấu" sẽ là:

$$|A_0 \cup A_1 \cup A_2| = |A_0| + |A_1| + |A_2| - |A_0 \cap A_1| - |A_0 \cap A_2| - |A_1 \cap A_2| + |A_0 \cap A_1 \cap A_2|$$

Kích thước của mỗi $A_i$ là $2^n$, vì mỗi dãy chỉ có thể chứa hai trong số các chữ số. Kích thước của mỗi giao đôi $A_i \cap A_j$ bằng 1, vì chỉ có một chữ số để tạo dãy. Kích thước của giao của cả ba tập bằng 0, vì không còn chữ số nào để tạo dãy.

Vì ta đã giải bài toán ngược, ta trừ kết quả này khỏi tổng số $3^n$ dãy:

$$3^n - (3 \cdot 2^n - 3 \cdot 1 + 0)$$

### 1.4.7 Số tổng số nguyên có giới hạn trên

Xét phương trình sau:

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 = 20$$

với điều kiện $0 \leq x_i \leq 8$ $(i = 1, 2, \ldots, 6)$.

**Bài toán:** Đếm số nghiệm của phương trình.

Tạm quên điều kiện giới hạn trên $x_i$ và chỉ đếm số nghiệm không âm của phương trình này. Điều này dễ dàng thực hiện bằng phương pháp "Sao và Thanh": ta muốn chia một dãy gồm 20 đơn vị thành 6 nhóm, tương đương với việc sắp xếp 5 thanh và 20 sao:

$$N_0 = \binom{25}{5}$$

Bây giờ ta sẽ tính số nghiệm "xấu" bằng nguyên lý bao hàm - loại trừ. Các nghiệm "xấu" là những nghiệm mà một hoặc nhiều $x_i$ lớn hơn hoặc bằng 9.

Gọi $A_k$ $(k = 1, 2, \ldots, 6)$ là tập các nghiệm mà $x_k \geq 9$, và các $x_i \geq 0$ $(i \neq k)$ khác (có thể $\geq 9$ hoặc không). Để tính kích thước của $A_k$, lưu ý rằng ta có cùng bài toán tổ hợp

như đã giải ở trên, nhưng giờ đây 9 đơn vị bị loại khỏi các ô và chắc chắn thuộc về nhóm đầu tiên. Do đó:

$$|A_k| = \binom{16}{5}$$

Tương tự, kích thước của giao giữa hai tập $A_k$ và $A_p$ (với $k \neq p$) bằng:

$$|A_k \cap A_p| = \binom{7}{5}$$

Kích thước của mỗi giao ba tập bằng 0, vì 20 đơn vị không đủ cho ba hoặc nhiều biến lớn hơn hoặc bằng 9.

Kết hợp tất cả vào công thức bao hàm - loại trừ và vì ta đã giải bài toán ngược, ta được kết quả cuối cùng:

$$\binom{25}{5} - \left(\binom{6}{1} \cdot \binom{16}{5} - \binom{6}{2} \cdot \binom{7}{5}\right)$$

Công thức này dễ dàng khái quát hóa cho $d$ số có tổng bằng $s$ với giới hạn $0 \leq x_i \leq b$:

$$\sum_{i=0}^{d} (-1)^i \binom{d}{i} \binom{s+d-1-(b+1)i}{d-1}$$

Như trên, ta coi các hệ số nhị thức với chỉ số trên âm là 0.

Lưu ý rằng bài toán này cũng có thể được giải bằng quy hoạch động hoặc hàm sinh. Kết quả từ bao hàm - loại trừ được tính trong thời gian $O(d)$ (giả sử các phép toán như hệ số nhị thức là thời gian không đổi), trong khi cách tiếp cận quy hoạch động đơn giản sẽ mất thời gian $O(ds)$.

### 1.4.8 Số số nguyên tương đối nguyên tố trong một khoảng cho trước

**Bài toán:** Cho hai số $n$ và $r$, đếm số số nguyên trong khoảng $[1; r]$ tương đối nguyên tố với $n$ (ước chung lớn nhất của chúng là 1).

Hãy giải bài toán ngược - tính số số không tương đối nguyên tố với $n$.

Gọi các thừa số nguyên tố của $n$ là $p_i (i = 1, \ldots, k)$.

Có bao nhiêu số trong khoảng $[1; r]$ chia hết cho $p_i$? Câu trả lời là:

$$\left\lfloor \frac{r}{p_i} \right\rfloor$$

Tuy nhiên, nếu chỉ đơn giản cộng các số này, một số số sẽ được tính nhiều lần (những số chia hết cho nhiều $p_i$). Do đó, cần sử dụng nguyên lý bao hàm - loại trừ.

Ta sẽ duyệt qua tất cả $2^k$ tập con của $p_i$, tính tích của chúng và cộng hoặc trừ số bội của tích đó.

Dưới đây là triển khai bằng C++:

```
int solve(int n, int r) {
    vector<int> p;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            p.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        p.push_back(n);

    int sum = 0;
    for (int msk = 1; msk < (1 << p.size()); ++msk) {
        int mult = 1,
            bits = 0;
        for (int i = 0; i < (int)p.size(); ++i)
            if (msk & (1 << i)) {
                ++bits;
                mult *= p[i];
            }

        int cur = r / mult;
        if (bits % 2 == 1)
            sum += cur;
        else
            sum -= cur;
    }

    return r - sum;
}
```

Độ phức tạp của giải pháp là $O(\sqrt{n})$.

### 1.4.9 Số số nguyên trong một khoảng cho trước là bội của ít nhất một trong các số đã cho

Cho $n$ số $a_i$ và số $r$. Bạn muốn đếm số số nguyên trong khoảng $[1; r]$ là bội của ít nhất một trong các $a_i$.

Thuật toán giải gần giống với bài toán trước — xây dựng công thức bao hàm - loại trừ trên các số $a_i$, tức là mỗi hạng trong công thức này là số các số chia hết cho một tập con của $a_i$ (nói cách khác, chia hết cho bội chung nhỏ nhất của chúng).

Vì vậy, ta sẽ duyệt qua tất cả $2^n$ tập con của các số nguyên $a_i$ với $O(n \log r)$ phép toán để tìm bội chung nhỏ nhất của chúng, cộng hoặc trừ số bội của nó trong khoảng. Độ phức tạp là $O(2^n \cdot n \cdot \log r)$.

### 1.4.10 Số chuỗi thỏa mãn một mẫu cho trước

Xét $n$ mẫu chuỗi có cùng độ dài, chỉ gồm các chữ cái $(a \dots z)$ hoặc dấu hỏi chấm. Bạn cũng được cho một số $k$. Một chuỗi khớp với một mẫu nếu nó có cùng độ dài với mẫu, và tại mỗi vị trí, hoặc các ký tự tương ứng bằng nhau, hoặc ký tự trong mẫu là dấu hỏi chấm. Nhiệm vụ là đếm số chuỗi khớp chính xác $k$ mẫu (bài toán thứ nhất) và ít nhất $k$ mẫu (bài toán thứ hai).

Trước tiên, lưu ý rằng ta có thể dễ dàng đếm số chuỗi thỏa mãn đồng thời tất cả các mẫu đã cho. Để làm điều này, chỉ cần "ghép" các mẫu: duyệt qua các vị trí ("khe") và xem xét một vị trí trên tất cả các mẫu. Nếu tất cả các mẫu đều có dấu hỏi chấm ở vị trí này, ký tự có thể là bất kỳ chữ cái nào từ $a$ đến $z$. Ngược lại, ký tự ở vị trí này được xác định duy nhất bởi các mẫu không chứa dấu hỏi chấm.

Bây giờ, hãy học cách giải phiên bản đầu tiên của bài toán: khi chuỗi phải thỏa mãn chính xác $k$ mẫu.

Để giải, duyệt và cố định một tập con cụ thể $X$ từ tập các mẫu gồm $k$ mẫu. Sau đó, ta phải đếm số chuỗi thỏa mãn tập mẫu này và chỉ khớp với nó, tức là không khớp với bất kỳ mẫu nào khác. Ta sẽ sử dụng nguyên lý bao hàm - loại trừ theo cách hơi khác: ta cộng trên tất cả các tập cha $Y$ (các tập con từ tập chuỗi ban đầu chứa $X$), và hoặc cộng vào đáp án hiện tại hoặc trừ đi từ số chuỗi:

$$ans(X) = \sum_{Y \supseteq X} (-1)^{|Y|-k} \cdot f(Y)$$

Trong đó $f(Y)$ là số chuỗi khớp với $Y$ (ít nhất $Y$).

(Nếu bạn khó hình dung điều này, có thể thử vẽ sơ đồ Venn.)

Nếu ta cộng tất cả $ans(X)$, ta sẽ được đáp án cuối cùng:

$$ans = \sum_{X \,:\, |X|=k} ans(X)$$

Tuy nhiên, độ phức tạp của giải pháp này là $O(3^k \cdot k)$. Để cải thiện, lưu ý rằng các phép tính $ans(X)$ khác nhau rất thường xuyên chia sẻ các tập $Y$.

Ta sẽ đảo ngược công thức bao hàm - loại trừ và tính tổng theo các tập $Y$. Bây giờ rõ ràng rằng cùng một tập $Y$ sẽ được tính đến trong phép tính $ans(X)$ của $\binom{|Y|}{k}$ tập với cùng dấu $(-1)^{|Y|-k}$.

$$ans = \sum_{Y\ :\ |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|}{k} \cdot f(Y)$$

Giờ đây, giải pháp của chúng ta có độ phức tạp $O(2^k \cdot k)$.

Bây giờ, ta sẽ giải phiên bản thứ hai của bài toán: tìm số chuỗi khớp ít nhất $k$ mẫu.

Tất nhiên, ta có thể sử dụng giải pháp của phiên bản đầu tiên và cộng các đáp án cho các tập có kích thước lớn hơn $k$. Tuy nhiên, bạn có thể nhận thấy rằng trong bài toán này, một tập $|Y|$ được xem xét trong công thức cho tất cả các tập có kích thước $\geq k$ nằm trong $Y$. Do đó, ta có thể viết phần của biểu thức được nhân với $f(Y)$ như sau:

$$(-1)^{|Y|-k} \cdot \binom{|Y|}{k} + (-1)^{|Y|-k-1} \cdot \binom{|Y|}{k+1} + (-1)^{|Y|-k-2} \cdot \binom{|Y|}{k+2} + \cdots + (-1)^{|Y|-|Y|} \cdot \binom{|Y|}{|Y|}$$

Xem xét Graham (Graham, Knuth, Patashnik. "Concrete mathematics" [1998]), ta thấy một công thức nổi tiếng cho hệ số nhị thức:

$$\sum_{k=0}^{m} (-1)^k \cdot \binom{n}{k} = (-1)^m \cdot \binom{n-1}{m}$$

Áp dụng ở đây, ta thấy toàn bộ tổng của các hệ số nhị thức được rút gọn thành:

$$(-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k}$$

Do đó, cho bài toán này, ta cũng thu được một giải pháp với độ phức tạp $O(2^k \cdot k)$:

$$ans = \sum_{Y\ :\ |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k} \cdot f(Y)$$

### 1.4.11 Số cách đi từ một ô đến ô khác

Có một bảng $n \times m$, và $k$ ô của nó là các tường không thể vượt qua. Một robot ban đầu ở ô $(1, 1)$ (góc dưới bên trái). Robot chỉ có thể di chuyển sang phải hoặc lên trên, và cuối cùng cần đến ô $(n, m)$, tránh tất cả các chướng ngại vật. Bạn cần đếm số cách nó có thể thực hiện.

Giả sử kích thước $n$ và $m$ rất lớn (ví dụ, $10^9$), và số $k$ nhỏ (khoảng 100).

Trước tiên, sắp xếp các chướng ngại vật theo tọa độ $x$, và trong trường hợp bằng nhau — theo tọa độ $y$.

Cũng cần học cách giải bài toán không có chướng ngại vật: tức là đếm số cách đi từ một ô đến ô khác. Trên một trục, ta cần đi qua $x$ ô, và trên trục kia, $y$ ô. Từ tổ hợp đơn giản, ta có công thức sử dụng hệ số nhị thức:

$$\binom{x+y}{x}$$

Bây giờ, để đếm số cách đi từ một ô đến ô khác, tránh tất cả chướng ngại vật, bạn có thể sử dụng bao hàm - loại trừ để giải bài toán ngược: đếm số cách đi qua bảng, bước qua một tập con các chướng ngại vật (và trừ nó khỏi tổng số cách).

Khi duyệt qua một tập con các chướng ngại vật mà ta sẽ bước qua, để đếm số cách thực hiện, chỉ cần nhân số tất cả các đường đi từ ô bắt đầu đến chướng ngại vật đầu tiên được chọn, từ chướng ngại vật đầu tiên đến thứ hai, và tiếp tục như vậy, rồi cộng hoặc trừ số này vào đáp án, theo công thức bao hàm - loại trừ tiêu chuẩn.

Tuy nhiên, điều này lại có độ phức tạp không đa thức $O(2^k \cdot k)$.

Dưới đây là một giải pháp đa thức:

Ta sẽ sử dụng quy hoạch động. Để tiện lợi, thêm $(1, 1)$ vào đầu và $(n, m)$ vào cuối mảng chướng ngại vật. Hãy tính các số $d[i]$ — số cách để đi từ điểm bắt đầu (thứ 0) đến ô thứ $i$, mà không bước qua bất kỳ chướng ngại vật nào khác (trừ ô $i$, tất nhiên). Ta sẽ tính số này cho tất cả các ô chướng ngại vật, và cả ô kết thúc.

Tạm quên các chướng ngại vật và chỉ đếm số đường đi từ ô 0 đến $i$. Ta cần xem xét một số đường đi "xấu", những đường đi qua các chướng ngại vật, và trừ chúng khỏi tổng số cách đi từ 0 đến $i$.

Khi xem xét một chướng ngại vật $t$ giữa 0 và $i$ $(0 < t < i)$, mà ta có thể bước qua, ta thấy số đường đi từ 0 đến $i$ đi qua $t$ mà có $t$ là chướng ngại vật đầu tiên giữa điểm bắt đầu và $i$. Ta có thể tính điều này bằng: $d[t]$ nhân với số đường đi bất kỳ từ $t$ đến $i$. Ta có thể đếm số cách "xấu" bằng cách cộng điều này cho tất cả $t$ giữa 0 và $i$.

Ta có thể tính $d[i]$ trong $O(k)$ cho $O(k)$ chướng ngại vật, nên giải pháp này có độ phức tạp $O(k^2)$.

### 1.4.12 Số bộ bốn tương đối nguyên tố

Bạn được cho $n$ số: $a_1, a_2, \ldots, a_n$. Bạn cần đếm số cách chọn bốn số sao cho ước chung lớn nhất tổng hợp của chúng bằng 1.

Ta sẽ giải bài toán ngược — tính số bộ bốn "xấu", tức là các bộ bốn mà tất cả các số đều chia hết cho một số $d > 1$.

Ta sẽ sử dụng nguyên lý bao hàm - loại trừ trong khi tính tổng trên tất cả các nhóm bốn số có thể chia hết cho một ước $d$.

$$ans = \sum_{d \geq 2} (-1)^{deg(d)-1} \cdot f(d)$$

Trong đó $deg(d)$ là số thừa số nguyên tố trong phân tích của số $d$ và $f(d)$ là số bộ bốn chia hết cho $d$.

Để tính hàm $f(d)$, bạn chỉ cần đếm số bội của $d$ (như đã đề cập trong bài toán trước) và sử dụng hệ số nhị thức để đếm số cách chọn bốn trong số chúng.

Do đó, sử dụng công thức bao hàm - loại trừ, ta cộng số nhóm bốn chia hết cho một số nguyên tố, sau đó trừ số bộ bốn chia hết cho tích của hai số nguyên tố, cộng các bộ bốn chia hết cho ba số nguyên tố, v.v.

### 1.4.13 Số bộ ba hài hòa

Bạn được cho một số $n \leq 10^6$. Bạn cần đếm số bộ ba $2 \leq a < b < c \leq n$ thỏa mãn một trong các điều kiện sau:

Hoặc $\gcd(a, b) = \gcd(a, c) = \gcd(b, c) = 1$, Hoặc $\gcd(a, b) > 1, \gcd(a, c) > 1, \gcd(b, c) > 1$.

Trước tiên, đi thẳng vào bài toán ngược — tức là đếm số bộ ba không hài hòa.

Thứ hai, lưu ý rằng bất kỳ bộ ba không hài hòa nào cũng được tạo thành từ một cặp tương đối nguyên tố và một số thứ ba không tương đối nguyên tố với ít nhất một số trong cặp.

Do đó, số bộ ba không hài hòa chứa $i$ bằng số số nguyên từ 2 đến $n$ tương đối nguyên tố với $i$ nhân với số số nguyên không tương đối nguyên tố với $i$.

Hoặc $gcd(a, b) = 1 \wedge gcd(a, c) > 1 \wedge gcd(b, c) > 1$, Hoặc $gcd(a, b) = 1 \wedge gcd(a, c) = 1 \wedge gcd(b, c) > 1$.

Trong cả hai trường hợp này, nó sẽ được đếm hai lần. Trường hợp đầu tiên sẽ được đếm khi $i = a$ và khi $i = b$. Trường hợp thứ hai sẽ được đếm khi $i = b$ và khi $i = c$. Do đó, để tính số bộ ba không hài hòa, ta tính tổng phép tính này qua tất cả $i$ từ 2 đến $n$ và chia cho 2.

Bây giờ, điều còn lại cần giải là học cách đếm số số tương đối nguyên tố với $i$ trong khoảng $[2; n]$. Mặc dù bài toán này đã được đề cập, giải pháp trên không phù hợp ở đây — nó sẽ yêu cầu phân tích thừa số của từng số nguyên từ 2 đến $n$, rồi duyệt qua tất cả các tập con của các số nguyên tố này.

Một giải pháp nhanh hơn có thể thực hiện với việc sửa đổi sàng Eratosthenes như sau:

Trước tiên, ta tìm tất cả các số trong khoảng $[2; n]$ sao cho phân tích thừa số đơn giản của nó không chứa thừa số nguyên tố nào lặp lại hai lần. Ta cũng cần biết, đối với các số này, nó bao gồm bao nhiêu thừa số.

Để làm điều này, ta sẽ duy trì một mảng $deg[i]$ để lưu số thừa số nguyên tố trong phân tích của $i$, và một mảng $good[i]$, để đánh dấu liệu $i$ chứa mỗi thừa số tối đa một lần ($good[i] = 1$) hay không ($good[i] = 0$). Khi duyệt từ 2 đến $n$, nếu ta đến một số có $deg$ bằng 0, thì đó là số nguyên tố và $deg$ của nó là 1. Trong quá trình sàng Eratosthenes, ta sẽ duyệt $i$ từ 2 đến $n$. Khi xử lý một số nguyên tố, ta đi qua tất cả bội của nó và tăng $deg[]$ của chúng. Nếu một trong các bội này là bội của bình phương của $i$, thì ta có thể đặt $good$ là false.

Thứ hai, ta cần tính đáp án cho tất cả $i$ từ 2 đến $n$, tức là mảng $cnt[]$ — số số nguyên không tương đối nguyên tố với $i$.

Để làm điều này, nhớ cách hoạt động của công thức bao hàm - loại trừ — thực ra ở đây ta thực hiện cùng khái niệm, nhưng với logic đảo ngược: ta duyệt qua một thành phần (tích của các số nguyên tố từ phân tích) và cộng hoặc trừ hạng của nó trong công thức bao hàm - loại trừ của mỗi bội của nó. Vậy, giả sử ta đang xử lý một số $i$ sao cho $good[i] = true$, tức là nó tham gia vào công thức bao hàm - loại trừ. Duyệt qua tất cả các số là bội của $i$, và hoặc cộng hoặc trừ $\lfloor N/i \rfloor$ khỏi $cnt[]$ của chúng (dấu phụ thuộc vào $deg[i]$: nếu $deg[i]$ lẻ thì cộng, ngược lại thì trừ).

Dưới đây là triển khai bằng C++:

```
int n;
bool good[MAXN];
int deg[MAXN], cnt[MAXN];

long long solve() {
    memset(good, 1, sizeof good);
    memset(deg, 0, sizeof deg);
    memset(cnt, 0, sizeof cnt);

    long long ans_bad = 0;
    for (int i = 2; i <= n; ++i) {
        if (good[i]) {
            if (deg[i] == 0) deg[i] = 1;
            for (int j = 1; i * j <= n; ++j) {
                if (j > 1 && deg[i] == 1)
                    if (j % i == 0)
                        good[i * j] = false;
                    else
                        ++deg[i * j];
                cnt[i * j] += (n / i) * (deg[i] % 2 == 1 ? +1 : -1);
            }
        }
        ans_bad += (cnt[i] - 1) * 1ll * (n - 1 - cnt[i]);
    }

    return (n - 1) * 1ll * (n - 2) * (n - 3) / 6 - ans_bad / 2;
}
```

Độ phức tạp của giải pháp là $O(n \log n)$, vì với hầu hết mọi số đến $n$, ta thực hiện $n/i$ lần lặp trong vòng lặp lồng nhau.

### 1.4.14 Số hoán vị không có điểm cố định (derangements)

Chứng minh rằng số hoán vị độ dài $n$ không có điểm cố định (tức là không có số $i$ nào ở vị trí $i$ - còn gọi là derangement) bằng số sau:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \cdots \pm \binom{n}{n} \cdot (n-n)!$$

và xấp xỉ bằng:

$$\frac{n!}{e}$$

(nếu bạn làm tròn biểu thức này đến số nguyên gần nhất — bạn sẽ được chính xác số hoán vị không có điểm cố định).

Gọi $A_k$ là tập các hoán vị độ dài $n$ có điểm cố định tại vị trí $k$ ($1 \leq k \leq n$) (tức là phần tử $k$ ở vị trí $k$).

Bây giờ ta sử dụng công thức bao hàm - loại trừ để đếm số hoán vị có ít nhất một điểm cố định. Để làm điều này, ta cần học cách đếm kích thước của giao các tập $A_i$, như sau:

$$\begin{aligned} |A_p| &= (n-1)!, \\ |A_p \cap A_q| &= (n-2)!, \\ |A_p \cap A_q \cap A_r| &= (n-3)!, \\ &\vdots \end{aligned}$$

vì nếu ta biết số điểm cố định bằng $x$, thì ta biết vị trí của $x$ phần tử của hoán vị, và tất cả các phần tử khác $(n-x)$ có thể được đặt ở bất kỳ đâu.

Thay điều này vào công thức bao hàm - loại trừ, và với số cách chọn một tập con kích thước $x$ từ tập $n$ phần tử bằng $\binom{n}{x}$, ta thu được công thức cho số hoán vị có ít nhất một điểm cố định:

$$\binom{n}{1} \cdot (n-1)! - \binom{n}{2} \cdot (n-2)! + \binom{n}{3} \cdot (n-3)! - \cdots \pm \binom{n}{n} \cdot (n-n)!$$

Vậy số hoán vị không có điểm cố định bằng:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \cdots \pm \binom{n}{n} \cdot (n-n)!$$

Rút gọn biểu thức này, ta thu được biểu thức chính xác và xấp xỉ cho số hoán vị không có điểm cố định:

$$n!\left(1-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}+\cdots\pm\frac{1}{n!}\right)\approx\frac{n!}{e}$$

(vì tổng trong dấu ngoặc là $n+1$ hạng đầu tiên của khai triển chuỗi Taylor của $e^{-1}$).

Đáng chú ý rằng một bài toán tương tự có thể được giải theo cách này: khi bạn cần các điểm cố định không nằm trong $m$ phần tử đầu tiên của hoán vị (không phải trong tất cả, như vừa giải). Công thức thu được giống công thức chính xác đã cho ở trên, nhưng sẽ tính tổng đến $k$, thay vì $n$.

## 1.5 Định lý Burnside / Định lý đếm Pólya

### 1.5.1 Định lý Burnside

Định lý Burnside được Burnside công thức hóa và chứng minh vào năm 1897, nhưng lịch sử cho thấy nó đã được Frobenius phát hiện vào năm 1887, và thậm chí sớm hơn vào năm 1845 bởi Cauchy. Do đó, đôi khi nó cũng được gọi là định lý Cauchy-Frobenius.

Định lý Burnside cho phép chúng ta đếm số lớp tương đương trong các tập hợp, dựa trên tính đối xứng nội tại.

#### Đối tượng và biểu diễn

Chúng ta cần phân biệt rõ ràng giữa số lượng đối tượng và số lượng biểu diễn.

Các biểu diễn khác nhau có thể tương ứng với cùng một đối tượng, nhưng tất nhiên mỗi biểu diễn chỉ tương ứng với một đối tượng duy nhất. Do đó, tập hợp tất cả các biểu diễn được chia thành các lớp tương đương. Nhiệm vụ của chúng ta là tính số lượng đối tượng, hoặc tương đương, số lượng lớp tương đương. Ví dụ sau sẽ làm rõ sự khác biệt giữa đối tượng và biểu diễn.

**Ví dụ: Tô màu cây nhị phân**

Giả sử chúng ta có bài toán sau. Chúng ta cần đếm số cách tô màu một cây nhị phân có gốc với $n$ đỉnh bằng hai màu, trong đó tại mỗi đỉnh, chúng ta không phân biệt giữa con trái và con phải.

Ở đây, tập hợp các đối tượng là tập hợp các cách tô màu khác nhau của cây.

Bây giờ chúng ta định nghĩa tập hợp các biểu diễn. Một biểu diễn của cách tô màu là một hàm $f(v)$, gán mỗi đỉnh một màu (ở đây chúng ta dùng các màu 0 và 1). Tập hợp các biểu diễn là tập chứa tất cả các hàm kiểu này, và kích thước của nó rõ ràng bằng $2^n$.

Đồng thời, chúng ta đưa ra một cách phân chia tập hợp này thành các lớp tương đương.

Ví dụ, giả sử $n=3$, và cây gồm gốc 1 và hai con của nó 2 và 3. Khi đó, các hàm sau $f_1$ và $f_2$ được coi là tương đương:

$$\begin{array}{ll} f_1(1) = 0 & f_2(1) = 0 \\ f_1(2) = 1 & f_2(2) = 0 \\ f_1(3) = 0 & f_2(3) = 1 \end{array}$$

### Hoán vị bất biến

Tại sao hai hàm $f_1$ và $f_2$ này thuộc cùng một lớp tương đương? Theo trực giác, điều này dễ hiểu - chúng ta có thể hoán đổi các con của đỉnh 1, tức là các đỉnh 2 và 3, và sau khi biến đổi hàm $f_1$ như vậy, nó sẽ trùng với $f_2$.

Nhưng theo hình thức, điều này có nghĩa là tồn tại một hoán vị bất biến $\pi$ (tức là một hoán vị không thay đổi bản thân đối tượng, mà chỉ thay đổi biểu diễn của nó), sao cho:

$$f_2\pi \equiv f_1$$

Vậy, bắt đầu từ định nghĩa của các đối tượng, chúng ta có thể tìm tất cả các hoán vị bất biến, tức là tất cả các hoán vị không thay đổi đối tượng khi áp dụng hoán vị vào biểu diễn. Sau đó, chúng ta có thể kiểm tra liệu hai hàm $f_1$ và $f_2$ có tương đương không (tức là chúng có tương ứng với cùng một đối tượng không) bằng cách kiểm tra điều kiện $f_2\pi \equiv f_1$ cho mỗi hoán vị bất biến (hoặc tương đương $f_1\pi \equiv f_2$). Nếu tìm được ít nhất một hoán vị mà điều kiện được thỏa mãn, thì $f_1$ và $f_2$ tương đương, ngược lại thì không.

Việc tìm tất cả các hoán vị bất biến như vậy theo định nghĩa của đối tượng là bước quan trọng để áp dụng cả định lý Burnside và định lý đếm Pólya. Rõ ràng, các hoán vị bất biến này phụ thuộc vào bài toán cụ thể, và việc tìm chúng là một quá trình hoàn toàn dựa trên trực giác. Tuy nhiên, trong hầu hết các trường hợp, chỉ cần tìm thủ công một vài hoán vị "cơ bản", từ đó tất cả các hoán vị khác có thể được tạo ra (và phần công việc này có thể được giao cho máy tính).

Không khó để hiểu rằng các hoán vị bất biến tạo thành một nhóm, vì tích (hợp thành) của các hoán vị bất biến vẫn là một hoán vị bất biến. Chúng ta ký hiệu nhóm các hoán vị bất biến bằng $G$.

### Phát biểu của định lý

Để phát biểu định lý, chúng ta cần thêm một định nghĩa từ đại số. Một điểm cố định $f$ đối với một hoán vị $\pi$ là một phần tử không thay đổi dưới hoán vị này: $f \equiv f\pi$. Ví dụ, trong ví dụ của chúng ta, các điểm cố định là những hàm $f$ tương ứng với các cách tô màu không thay đổi khi hoán vị $\pi$ được áp dụng (tức là không thay đổi theo nghĩa chính thức của sự bằng nhau của các hàm). Chúng ta ký hiệu $I(\pi)$ là số điểm cố định của hoán vị $\pi$.

Khi đó, định lý Burnside được phát biểu như sau: số lớp tương đương bằng tổng số điểm cố định đối với tất cả các hoán vị từ nhóm $G$, chia cho kích thước của nhóm này:

$$|\text{Classes}| = \frac{1}{|G|}\sum_{\pi \in G} I(\pi)$$

Mặc dù bản thân định lý Burnside không quá tiện lợi để sử dụng trong thực tế (không rõ cách tìm nhanh giá trị $I(\pi)$), nhưng nó thể hiện rõ ràng nhất bản chất toán học mà ý tưởng tính toán các lớp tương đương dựa vào.

### Chứng minh định lý Burnside

Chứng minh định lý Burnside được mô tả ở đây không quan trọng đối với các ứng dụng thực tế, nên có thể bỏ qua khi đọc lần đầu.

Chứng minh ở đây là chứng minh đơn giản nhất được biết đến, và không sử dụng lý thuyết nhóm. Chứng minh này được Kenneth P. Bogart công bố vào năm 1991.

Chúng ta cần chứng minh phát biểu sau:

$$|\text{Classes}| \cdot |G| = \sum_{\pi \in G} I(\pi)$$

Giá trị ở vế phải không gì khác ngoài số "cặp bất biến" $(f, \pi)$, tức là các cặp sao cho $f\pi \equiv f$. Rõ ràng, chúng ta có thể thay đổi thứ tự tính tổng. Chúng ta để tổng duyệt qua tất cả các phần tử $f$ và tính tổng các giá trị $J(f)$ - số hoán vị mà $f$ là điểm cố định.

$$|\text{Classes}| \cdot |G| = \sum_{f} J(f)$$

Để chứng minh công thức này, chúng ta sẽ lập một bảng với các cột được gắn nhãn bằng tất cả các hàm $f_i$ và các hàng được gắn nhãn bằng tất cả các hoán vị $\pi_j$. Và chúng ta điền các ô bằng $f_i\pi_j$. Nếu nhìn các cột trong bảng này như các tập hợp, thì một số cột sẽ trùng nhau, và điều này có nghĩa là các hàm $f$ tương ứng với các cột này cũng tương đương. Do đó, số cột khác nhau (theo nghĩa tập hợp) bằng số lớp tương đương. Nhân tiện, từ quan điểm lý thuyết nhóm, cột được gắn nhãn bằng $f_i$ là quỹ đạo của phần tử này. Đối với các phần tử tương đương, các quỹ đạo trùng nhau, và số quỹ đạo chính xác bằng số lớp.

Vậy, các cột của bảng phân tách thành các lớp tương đương. Hãy cố định một lớp và xem xét các cột trong đó. Trước tiên, lưu ý rằng các cột này chỉ có thể chứa các phần tử $f_i$ của lớp tương đương (nếu không, một hoán vị $\pi_j$ nào đó đã chuyển một trong các hàm sang một lớp tương đương khác, điều này là không thể vì chúng ta chỉ xem xét các hoán vị bất biến). Thứ hai, mỗi phần tử $f_i$ sẽ xuất hiện cùng số lần trong mỗi cột (điều này cũng xuất phát từ việc các cột tương ứng với các phần tử tương đương). Từ đây, chúng ta có thể kết luận rằng tất cả các cột trong cùng một lớp tương đương trùng nhau dưới dạng tập đa hợp.

Bây giờ, cố định một phần tử bất kỳ $f$. Một mặt, nó xuất hiện trong cột của nó chính xác $J(f)$ lần (theo định nghĩa). Mặt khác, tất cả các cột trong cùng một lớp tương đương đều giống nhau dưới dạng tập đa hợp. Do đó, trong mỗi cột của một lớp tương đương cho trước, bất kỳ phần tử $g$ nào cũng xuất hiện chính xác $J(g)$ lần.

Vậy, nếu chúng ta tùy ý lấy một cột từ mỗi lớp tương đương và tính tổng số phần tử trong chúng, một mặt ta thu được $|\text{Classes}| \cdot |G|$ (đơn giản bằng cách nhân số cột với số hàng), và mặt khác là tổng các giá trị $J(f)$ cho tất cả $f$ (điều này xuất phát từ tất cả các lập luận trước):

$$|\text{Classes}| \cdot |G| = \sum_f J(f)$$

### 1.5.2 Định lý đếm Pólya

Định lý đếm Pólya là một khái quát hóa của định lý Burnside, và nó cũng cung cấp một công cụ tiện lợi hơn để tìm số lớp tương đương. Cần lưu ý rằng định lý này đã được Redfield phát hiện trước Pólya vào năm 1927, nhưng ấn phẩm của ông không được các nhà toán học chú ý. Pólya độc lập đạt được cùng kết quả vào năm 1937, và ấn phẩm của ông thành công hơn.

Ở đây, chúng ta chỉ thảo luận một trường hợp đặc biệt của định lý đếm Pólya, vốn sẽ rất hữu ích trong thực tế. Công thức tổng quát của định lý sẽ không được thảo luận.

Chúng ta ký hiệu $C(\pi)$ là số chu trình trong hoán vị $\pi$. Khi đó, công thức sau (một trường hợp đặc biệt của định lý đếm Pólya) được thỏa mãn:

$$|\text{Classes}| = \frac{1}{|G|} \sum_{\pi \in G} k^{C(\pi)}$$

$k$ là số giá trị mà mỗi phần tử biểu diễn có thể nhận, trong trường hợp tô màu cây nhị phân thì $k = 2$.

#### Chứng minh

Công thức này là hệ quả trực tiếp của định lý Burnside. Để có được nó, chúng ta chỉ cần tìm một biểu thức tường minh cho $I(\pi)$, xuất hiện trong định lý. Nhắc lại rằng $I(\pi)$ là số điểm cố định trong hoán vị $\pi$.

Vậy, chúng ta xét một hoán vị $\pi$ và một phần tử $f$ nào đó. Trong quá trình áp dụng $\pi$, các phần tử trong $f$ di chuyển qua các chu trình trong hoán vị. Vì kết quả cần đạt được $f \equiv f\pi$, các phần tử bị ảnh hưởng bởi một chu trình phải đều bằng nhau. Đồng thời, các chu trình khác nhau là độc lập. Do đó, với mỗi chu trình hoán vị $\pi$, chúng ta có thể chọn một giá trị (trong số $k$ giá trị có thể) và từ đó ta có số điểm cố định:

$$I(\pi) = k^{C(\pi)}$$

**Ứng dụng: Tô màu vòng cổ**

Bài toán "Vòng cổ" là một trong những bài toán tổ hợp kinh điển. Nhiệm vụ là đếm số vòng cổ khác nhau từ $n$ hạt, mỗi hạt có thể được sơn bằng một trong $k$ màu. Khi so sánh hai vòng cổ, chúng có thể được xoay, nhưng không được lật ngược (tức là phép dịch chuyển vòng được phép).

Trong bài toán này, chúng ta có thể ngay lập tức tìm ra nhóm các hoán vị bất biến:

$$\begin{aligned}
\pi_0 &= 1 \quad 2 \quad 3 \quad \dots \quad n, \\
\pi_1 &= 2 \quad 3 \quad \dots \quad n \quad 1, \\
\pi_2 &= 3 \quad \dots \quad n \quad 1 \quad 2, \\
&\vdots \\
\pi_{n-1} &= n \quad 1 \quad 2 \quad 3 \quad \dots .
\end{aligned}$$

Hãy tìm một công thức tường minh để tính $C(\pi_i)$. Trước tiên, lưu ý rằng hoán vị $\pi_i$ có giá trị tại vị trí thứ $j$ là $i+j$ (lấy modulo $n$). Nếu kiểm tra cấu trúc chu trình của $\pi_i$, ta thấy 1 chuyển thành $1+i$, $1+i$ chuyển thành $1+2i$, rồi thành $1+3i$, v.v., cho đến khi đến một số dạng $1+kn$. Các phát biểu tương tự có thể được đưa ra cho các phần tử còn lại. Do đó, ta thấy tất cả các chu trình đều có cùng độ dài, cụ thể là:

$$\frac{\operatorname{lcm}(i,n)}{i} = \frac{n}{\gcd(i,n)}$$

Vậy, số chu trình trong $\pi_i$ sẽ bằng $\gcd(i,n)$.

Thay các giá trị này vào định lý đếm Pólya, ta thu được giải pháp:

$$\frac{1}{n}\sum_{i=1}^{n} k^{\gcd(i,n)}$$

Bạn có thể giữ công thức này ở dạng này, hoặc đơn giản hóa thêm. Hãy chuyển tổng để nó duyệt qua tất cả các ước của $n$. Trong tổng ban đầu, sẽ có nhiều hạng tương đương: nếu $i$ không phải là ước của $n$, thì một ước như vậy có thể được tìm thấy sau khi tính $\gcd(i,n)$. Do đó, với mỗi ước $d \mid n$, hạng $k^{\gcd(d,n)} = k^d$ sẽ xuất hiện nhiều lần trong tổng, tức là đáp án của bài toán có thể được viết lại thành:

$$\frac{1}{n}\sum_{d \mid n} C_d k^d,$$

trong đó $C_d$ là số các số $i$ sao cho $\gcd(i,n) = d$. Chúng ta có thể tìm một biểu thức tường minh cho giá trị này. Bất kỳ số $i$ nào như vậy đều có dạng $i = dj$ với $\gcd(j, n/d) = 1$ (nếu không, $\gcd(i,n) > d$). Vậy, ta có thể đếm số $j$ có tính chất này. Hàm phi của Euler cho ta kết quả $C_d = \phi(n/d)$, và do đó ta có đáp án:

$$\frac{1}{n}\sum_{d \mid n} \phi\left(\frac{n}{d}\right) k^d$$

**Ứng dụng: Tô màu mặt xuyến**

Thông thường, chúng ta không thể thu được một công thức tường minh cho số lớp tương đương. Trong nhiều bài toán, số hoán vị trong một nhóm có thể quá lớn để tính toán thủ công và không thể tính toán phân tích số chu trình trong chúng.

Trong trường hợp đó, chúng ta nên tìm thủ công một vài hoán vị "cơ bản", để chúng có thể tạo ra toàn bộ nhóm $G$. Tiếp theo, ta có thể viết một chương trình tạo ra tất cả các hoán vị của nhóm $G$, đếm số chu trình trong chúng, và tính đáp án bằng công thức.

Xét ví dụ về bài toán tô màu mặt xuyến. Có một tờ giấy kẻ ô $n \times m$ ($n < m$), một số ô được tô đen. Sau đó, một hình trụ được tạo ra từ tờ giấy này bằng cách dán hai cạnh có độ dài $m$ lại với nhau. Rồi một mặt xuyến được tạo từ hình trụ bằng cách dán hai vòng tròn (trên và dưới) lại mà không xoắn. Nhiệm vụ là tính số mặt xuyến được tô màu khác nhau, giả sử ta không thể thấy các đường dán, và mặt xuyến có thể được xoay và lật.

Chúng ta lại bắt đầu với một mảnh giấy $n \times m$. Dễ thấy rằng các loại biến đổi sau giữ nguyên lớp tương đương: dịch chuyển vòng các hàng, dịch chuyển vòng các cột, và xoay tờ giấy 180 độ. Cũng dễ thấy rằng các biến đổi này có thể tạo ra toàn bộ nhóm các biến đổi bất biến. Nếu chúng ta đánh số các ô của tờ giấy theo cách nào đó, thì ta có thể viết ba hoán vị $p_1$, $p_2$, $p_3$ tương ứng với các loại biến đổi này.

Tiếp theo, chỉ cần tạo ra tất cả các hoán vị thu được dưới dạng tích. Rõ ràng, tất cả các hoán vị như vậy có dạng $p_1^{i_1} p_2^{i_2} p_3^{i_3}$ với $i_1 = 0 \ldots m-1$, $i_2 = 0 \ldots n-1$, $i_3 = 0 \ldots 1$.

Vậy, chúng ta có thể viết triển khai cho bài toán này:

```
using Permutation = vector<int>;

void operator*=(Permutation& p, Permutation const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++) {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] != -1;) {
                int next = p[j];
```

```
                p[j] = -1;
                j = next;
            }
        }
    }
    return cnt;
}

int solve(int n, int m) {
    Permutation p(n*m), p1(n*m), p2(n*m), p3(n*m);
    for (int i = 0; i < n*m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }

    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
        for (int i2 = 0; i2 < m; i2++) {
            for (int i3 = 0; i3 < 2; i3++) {
                s.insert(p);
                p *= p3;
            }
            p *= p2;
        }
        p *= p1;
    }

    int sum = 0;
    for (Permutation const& p : s) {
        sum += 1 << count_cycles(p);
    }
    return sum / s.size();
}
```

## 1.6 Sao và Thanh

Sao và Thanh là một kỹ thuật toán học để giải quyết một số bài toán tổ hợp. Nó xuất hiện bất cứ khi nào bạn muốn đếm số cách phân nhóm các đối tượng giống hệt nhau.

### 1.6.1 Định lý

Số cách để đặt $n$ đối tượng giống hệt nhau vào $k$ hộp có nhãn là:

$$\binom{n+k-1}{n}.$$

Chứng minh liên quan đến việc biến các đối tượng thành các ngôi sao và phân tách các hộp bằng các thanh (do đó có tên gọi này). Ví dụ, ta có thể biểu diễn bằng$\star \mid \star\star \mid \star\star$ tình huống sau: trong hộp đầu tiên có một đối tượng, trong hộp thứ hai có hai đối tượng, hộp thứ ba trống và trong hộp cuối cùng có hai đối tượng. Đây là một cách chia 5 đối tượng vào 4 hộp.

Rõ ràng, mỗi phân hoạch có thể được biểu diễn bằng $n$ ngôi sao và $k-1$ thanh, và mỗi hoán vị sao và thanh sử dụng $n$ ngôi sao và $k-1$ thanh đại diện cho một phân hoạch. Do đó, số cách để chia $n$ đối tượng giống hệt nhau vào $k$ hộp có nhãn bằng với số hoán vị của $n$ ngôi sao và $k-1$ thanh. Hệ số nhị thức cho chúng ta công thức mong muốn.

### 1.6.2 Số tổng số nguyên không âm

Bài toán này là một ứng dụng trực tiếp của định lý.

Bạn muốn đếm số nghiệm của phương trình:

$$x_1 + x_2 + \cdots + x_k = n$$

với $x_i \geq 0$.

Ta lại có thể biểu diễn một nghiệm bằng sao và thanh. Ví dụ, nghiệm $1+3+0=4$ cho $n=4$, $k=3$ có thể được biểu diễn bằng $\star \mid \star\star\star \mid$.

Dễ thấy rằng đây chính là định lý sao và thanh. Do đó, nghiệm là $\binom{n+k-1}{n}$.

### 1.6.3 Số tổng số nguyên dương

Một định lý thứ hai cung cấp một cách diễn giải đẹp cho các số nguyên dương. Xét các nghiệm của:

$$x_1 + x_2 + \cdots + x_k = n$$

với $x_i \geq 1$.

Ta có thể xem xét $n$ ngôi sao, nhưng lần này ta chỉ có thể đặt tối đa một thanh giữa các ngôi sao, vì hai thanh giữa các ngôi sao sẽ biểu thị $x_i = 0$, tức là một hộp trống. Có $n-1$ khoảng trống giữa các ngôi sao để đặt $k-1$ thanh, nên nghiệm là $\binom{n-1}{k-1}$.

### 1.6.4 Số tổng số nguyên có giới hạn dưới

Điều này có thể dễ dàng mở rộng cho các tổng số nguyên với các giới hạn dưới khác nhau. Tức là ta muốn đếm số nghiệm của phương trình:

$$x_1 + x_2 + \cdots + x_k = n$$

với $x_i \geq a_i$.

Sau khi thay $x'_i := x_i - a_i$, ta nhận được phương trình đã sửa đổi:

$$(x'_1 + a_1) + (x'_2 + a_2) + \cdots + (x'_k + a_k) = n$$

$$\Leftrightarrow \quad x'_1 + x'_2 + \cdots + x'_k = n - a_1 - a_2 - \cdots - a_k$$

với $x'_i \geq 0$. Vậy, ta đã rút gọn bài toán về trường hợp đơn giản hơn với $x'_i \geq 0$ và lại có thể áp dụng định lý sao và thanh.

### 1.6.5 Số tổng số nguyên có giới hạn trên

Với sự trợ giúp của Nguyên lý Bao hàm - Loại trừ, bạn cũng có thể giới hạn các số nguyên với các giới hạn trên. Xem phần Số tổng số nguyên có giới hạn trên trong bài viết tương ứng.

## 1.7 Sinh tất cả các tổ hợp $K$-phần tử

Trong bài viết này, chúng ta sẽ thảo luận về bài toán sinh tất cả các tổ hợp $K$-phần tử. Cho các số tự nhiên $N$ và $K$, và xét một tập hợp các số từ 1 đến $N$. Nhiệm vụ là tìm tất cả các tập con có kích thước $K$.

### 1.7.1 Sinh tổ hợp $K$-phần tử tiếp theo theo thứ tự từ điển

Trước tiên, chúng ta sẽ sinh chúng theo thứ tự từ điển. Thuật toán cho việc này rất đơn giản. Tổ hợp đầu tiên sẽ là $\{1, 2, \ldots, K\}$. Bây giờ, hãy xem cách tìm tổ hợp ngay sau tổ hợp này theo thứ tự từ điển. Để làm điều đó, chúng ta xét tổ hợp hiện tại và tìm phần tử cực phải chưa đạt đến giá trị cao nhất có thể của nó. Sau khi tìm thấy phần tử này, chúng ta tăng nó lên 1, và gán giá trị hợp lệ nhỏ nhất cho tất cả các phần tử tiếp theo.

```
bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
```

```
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}
```

### 1.7.2 Sinh tất cả tổ hợp $K$-phần tử sao cho các tổ hợp liền kề khác nhau bởi một phần tử

Lần này, chúng ta muốn sinh tất cả các tổ hợp $K$-phần tử theo thứ tự sao cho các tổ hợp liền kề khác nhau chính xác bởi một phần tử.

Điều này có thể được giải quyết bằng cách sử dụng Mã Gray: Nếu chúng ta gán một mặt nạ bit cho mỗi tập con, thì bằng cách sinh và duyệt qua các mặt nạ bit này với mã Gray, chúng ta có thể thu được đáp án.

Nhiệm vụ sinh các tổ hợp $K$-phần tử cũng có thể được giải bằng Mã Gray theo cách khác: Sinh Mã Gray cho các số từ 0 đến $2^N - 1$ và chỉ giữ lại những mã chứa đúng $K$ bit 1. Điều đáng ngạc nhiên là trong dãy kết quả của các tập bit $K$, bất kỳ hai mặt nạ lân cận nào (bao gồm cả mặt nạ đầu tiên và cuối cùng - lân cận theo nghĩa vòng tròn) sẽ khác nhau chính xác bởi hai bit, điều này phù hợp với mục tiêu của chúng ta (loại bỏ một số, thêm một số).

Hãy chứng minh điều này:

Để chứng minh, chúng ta nhớ lại thực tế rằng dãy $G(N)$ (biểu thị Mã Gray thứ $N$) có thể được thu nhận như sau:

$$G(N) = 0G(N-1) \cup 1G(N-1)^{\mathrm{R}}$$

Tức là, xét dãy Mã Gray cho $N-1$, và thêm tiền tố 0 trước mỗi hạng. Sau đó, xét dãy Mã Gray đảo ngược cho $N-1$ và thêm tiền tố 1 trước mỗi mặt nạ, rồi nối hai dãy này lại.

Bây giờ, chúng ta có thể đưa ra chứng minh.

Trước tiên, chúng ta chứng minh rằng mặt nạ đầu tiên và cuối cùng khác nhau chính xác ở hai bit. Để làm điều này, chỉ cần lưu ý rằng mặt nạ đầu tiên của dãy $G(N)$ sẽ có dạng $N-K$ bit 0, tiếp theo là $K$ bit 1. Vì bit đầu tiên được đặt là 0, sau đó có $(N-K-1)$ bit 0 theo sau, rồi $K$ bit 1 theo sau, và mặt nạ cuối cùng sẽ có dạng 1, rồi $(N-K)$ bit 0, rồi $K-1$ bit 1. Áp dụng nguyên lý quy nạp toán học và sử dụng công thức cho $G(N)$, ta kết thúc chứng minh.

Bây giờ, nhiệm vụ của chúng ta là chứng minh rằng bất kỳ hai mã liền kề nào cũng khác nhau chính xác ở hai bit, chúng ta có thể làm điều này bằng cách xét phương trình

đệ quy để sinh Mã Gray. Giả sử nội dung của hai nửa được tạo bởi $G(N-1)$ là đúng. Bây giờ, chúng ta cần chứng minh rằng cặp liên tiếp mới được tạo ra tại điểm nối (bằng cách nối hai nửa này) cũng hợp lệ, tức là chúng khác nhau chính xác ở hai bit.

Điều này có thể thực hiện được, vì chúng ta biết mặt nạ cuối cùng của nửa đầu tiên và mặt nạ đầu tiên của nửa thứ hai. Mặt nạ cuối cùng của nửa đầu tiên sẽ là 1, rồi $(N-K-1)$ bit 0, rồi $K-1$ bit 1. Và mặt nạ đầu tiên của nửa thứ hai sẽ là 0, rồi $(N-K-2)$ bit 0 theo sau, rồi $K$ bit 1. Do đó, so sánh hai mặt nạ, ta thấy chính xác hai bit khác nhau.

Dưới đây là một triển khai đơn giản hoạt động bằng cách sinh tất cả $2^n$ tập con có thể có, và tìm các tập con có kích thước $K$:

```
int gray_code(int n) {
    return n ^ (n >> 1);
}

int count_bits(int n) {
    int res = 0;
    for (; n; n >>= 1)
        res += n & 1;
    return res;
}

void all_combinations(int n, int k) {
    for (int i = 0; i < (1 << n); i++) {
        int cur = gray_code(i);
        if (count_bits(cur) == k) {
            for (int j = 0; j < n; j++) {
                if (cur & (1 << j))
                    cout << j + 1;
            }
            cout << "\n";
        }
    }
}
```

Đáng chú ý rằng tồn tại một triển khai hiệu quả hơn chỉ xây dựng các tổ hợp hợp lệ và do đó hoạt động trong $O\left(N \cdot \binom{N}{K}\right)$, tuy nhiên nó mang tính đệ quy và với các giá trị nhỏ của $N$, có lẽ nó có hằng số lớn hơn so với giải pháp trước.

Triển khai này được suy ra từ công thức:

$$G(N,K) = 0G(N-1,K) \cup 1G(N-1,K-1)^{\mathrm{R}}$$

Công thức này được thu nhận bằng cách sửa đổi phương trình tổng quát để xác định mã Gray, và hoạt động bằng cách chọn dãy con từ các phần tử thích hợp.

Triển khai của nó như sau:

```
vector<int> ans;

void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);
}
```

## 1.8 Đặt các quân Tượng trên bàn cờ

Tìm số cách đặt $K$ quân Tượng trên bàn cờ $N \times N$ sao cho không có hai quân Tượng nào tấn công lẫn nhau.

### 1.8.1 Thuật toán

Bài toán này có thể được giải bằng quy hoạch động.

Hãy đánh số các đường chéo của bàn cờ như sau: các đường chéo đen có chỉ số lẻ, các đường chéo trắng có chỉ số chẵn, và các đường chéo được đánh số theo thứ tự không giảm của số ô trong chúng. Dưới đây là một ví dụ cho bàn cờ $5 \times 5$:

$$\begin{array}{ccccc} \mathbf{1} & 2 & \mathbf{5} & 6 & \mathbf{9} \\ 2 & \mathbf{5} & 6 & \mathbf{9} & 8 \\ \mathbf{5} & 6 & \mathbf{9} & 8 & \mathbf{7} \\ 6 & \mathbf{9} & 8 & \mathbf{7} & 4 \\ \mathbf{9} & 8 & \mathbf{7} & 4 & \mathbf{3} \end{array}$$

Gọi $D[i][j]$ là số cách đặt $j$ quân Tượng trên các đường chéo có chỉ số tối đa là $i$ và cùng màu với đường chéo $i$. Khi đó, $i = 1 \ldots 2N - 1$ và $j = 0 \ldots K$.

Chúng ta có thể tính $D[i][j]$ chỉ sử dụng các giá trị của $D[i-2]$ (chúng ta trừ 2 vì chỉ xét các đường chéo cùng màu với $i$). Có hai cách để đạt được $D[i][j]$. Hoặc là đặt tất cả $j$ quân Tượng trên các đường chéo trước đó: khi đó có $D[i-2][j]$ cách để thực hiện điều này. Hoặc là đặt một quân Tượng trên đường chéo $i$ và $j-1$ quân Tượng trên các đường chéo trước đó. Số cách để làm điều này bằng số ô trong đường chéo $i$ trừ đi $j-1$, vì mỗi trong số $j-1$ quân Tượng đặt trên các đường chéo trước sẽ chặn một ô trên đường chéo hiện tại. Số ô trong đường chéo $i$ có thể được tính như sau:

```
int squares(int i) {
    if (i & 1)
        return i / 4 * 2 + 1;
    else
        return (i - 1) / 4 * 2 + 2;
}
```

Trường hợp cơ sở rất đơn giản: $D[i][0] = 1$, $D[1][1] = 1$.

Sau khi đã tính tất cả các giá trị của $D[i][j]$, đáp án có thể được thu nhận như sau: xét tất cả các số lượng quân Tượng có thể đặt trên các đường chéo đen $i = 0 \ldots K$, với số quân Tượng tương ứng trên các đường chéo trắng là $K - i$. Các quân Tượng đặt trên các đường chéo đen và trắng không bao giờ tấn công lẫn nhau, nên việc đặt có thể được thực hiện độc lập. Chỉ số của đường chéo đen cuối cùng là $2N - 1$, đường chéo trắng cuối cùng là $2N - 2$. Với mỗi $i$, ta cộng $D[2N-1][i] \cdot D[2N-2][K-i]$ vào đáp án.

### 1.8.2 Triển khai

```
int bishop_placements(int N, int K)
{
    if (K > 2 * N - 1)
        return 0;

    vector<vector<int>> D(N * 2, vector<int>(K + 1));
    for (int i = 0; i < N * 2; ++i)
        D[i][0] = 1;
```

```
    D[1][1] = 1;
    for (int i = 2; i < N * 2; ++i)
        for (int j = 1; j <= K; ++j)
            D[i][j] = D[i-2][j] + D[i-2][j-1] * (squares(i) - j + 1);

    int ans = 0;
    for (int i = 0; i <= K; ++i)
        ans += D[N*2-1][i] * D[N*2-2][K-i];
    return ans;
}
```

## 1.9 Dãy ngoặc cân bằng

Dãy ngoặc cân bằng là một chuỗi chỉ gồm các dấu ngoặc, sao cho chuỗi này, khi thêm vào một số số và phép toán học, tạo thành một biểu thức toán học hợp lệ. Hình thức, bạn có thể định nghĩa dãy ngoặc cân bằng như sau:

- $e$ (chuỗi rỗng) là một dãy ngoặc cân bằng. - Nếu $s$ là một dãy ngoặc cân bằng, thì $(s)$ cũng là một dãy ngoặc cân bằng. - Nếu $s$ và $t$ là các dãy ngoặc cân bằng, thì $st$ cũng là một dãy ngoặc cân bằng.

Ví dụ, (())() là một dãy ngoặc cân bằng, nhưng ())( thì không.

Tất nhiên, bạn cũng có thể định nghĩa các dãy ngoặc khác với nhiều loại ngoặc theo cách tương tự.

Trong bài viết này, chúng ta sẽ thảo luận một số bài toán kinh điển liên quan đến dãy ngoặc cân bằng (để đơn giản, chúng ta sẽ chỉ gọi chúng là dãy): kiểm tra tính cân bằng, số lượng dãy, tìm dãy tiếp theo theo thứ tự từ điển, sinh tất cả các dãy có kích thước nhất định, tìm chỉ số của dãy, và sinh dãy thứ $k$. Chúng ta cũng sẽ thảo luận hai biến thể cho các bài toán, phiên bản đơn giản hơn khi chỉ cho phép một loại ngoặc, và trường hợp khó hơn khi có nhiều loại ngoặc.

### 1.9.1 Kiểm tra tính cân bằng

Chúng ta muốn kiểm tra xem một chuỗi đã cho có cân bằng hay không.

Trước tiên, giả sử chỉ có một loại ngoặc. Trong trường hợp này, tồn tại một thuật toán rất đơn giản. Gọi depth là số lượng ngoặc mở hiện tại. Ban đầu depth = 0. Chúng ta duyệt qua tất cả các ký tự của chuỗi, nếu ký tự ngoặc hiện tại là ngoặc mở, thì tăng depth, ngược lại giảm nó. Nếu tại bất kỳ thời điểm nào biến depth trở thành âm, hoặc cuối cùng nó khác 0, thì chuỗi không phải là dãy cân bằng. Ngược lại, nó là dãy cân bằng.

Nếu có nhiều loại ngoặc, thì thuật toán cần được thay đổi. Thay vì một bộ đếm depth, chúng ta tạo một ngăn xếp, trong đó sẽ lưu trữ tất cả các ngoặc mở mà chúng ta gặp. Nếu ký tự ngoặc hiện tại là ngoặc mở, chúng ta đặt nó vào ngăn xếp. Nếu là ngoặc đóng,

thì kiểm tra xem ngăn xếp có rỗng không, và phần tử trên cùng của ngăn xếp có cùng loại với ngoặc đóng hiện tại không. Nếu cả hai điều kiện đều được thỏa mãn, thì loại bỏ ngoặc mở khỏi ngăn xếp. Nếu tại bất kỳ thời điểm nào một trong các điều kiện không được thỏa mãn, hoặc cuối cùng ngăn xếp không rỗng, thì chuỗi không cân bằng. Ngược lại, nó cân bằng.

### 1.9.2 Số lượng dãy cân bằng

#### Công thức

Số lượng dãy ngoặc cân bằng với chỉ một loại ngoặc có thể được tính bằng số Catalan. Số lượng dãy ngoặc cân bằng có độ dài $2n$ ($n$ cặp ngoặc) là:

$$\frac{1}{n+1}\binom{2n}{n}$$

Nếu cho phép $k$ loại ngoặc, thì mỗi cặp có thể thuộc bất kỳ loại nào trong $k$ loại (độc lập với các loại khác), do đó số lượng dãy ngoặc cân bằng là:

$$\frac{1}{n+1}\binom{2n}{n}k^n$$

#### Quy hoạch động

Mặt khác, các số này có thể được tính bằng quy hoạch động. Gọi $d[n]$ là số lượng dãy ngoặc thông thường với $n$ cặp ngoặc. Lưu ý rằng ở vị trí đầu tiên luôn là một ngoặc mở. Và đâu đó sau đó là ngoặc đóng tương ứng của cặp này. Rõ ràng rằng bên trong cặp này là một dãy ngoặc cân bằng, và tương tự sau cặp này là một dãy ngoặc cân bằng. Vì vậy, để tính $d[n]$, chúng ta sẽ xem có bao nhiêu dãy cân bằng với $i$ cặp ngoặc bên trong cặp ngoặc đầu tiên này, và bao nhiêu dãy cân bằng với $n-1-i$ cặp sau cặp này. Do đó, công thức có dạng:

$$d[n] = \sum_{i=0}^{n-1} d[i] \cdot d[n-1-i]$$

Giá trị ban đầu cho quan hệ đệ quy này là $d[0] = 1$.

### 1.9.3 Tìm dãy cân bằng tiếp theo theo thứ tự từ điển

Ở đây, chúng ta chỉ xét trường hợp với một loại ngoặc hợp lệ.

Cho một dãy cân bằng, chúng ta phải tìm dãy cân bằng tiếp theo (theo thứ tự từ điển).

Rõ ràng, chúng ta phải tìm ngoặc mở cực phải mà chúng ta có thể thay bằng ngoặc đóng mà không vi phạm điều kiện rằng số ngoặc đóng không vượt quá số ngoặc mở tính đến vị trí này. Sau khi thay vị trí này, chúng ta có thể điền phần còn lại của chuỗi bằng dãy nhỏ nhất theo thứ tự từ điển: tức là đầu tiên với càng nhiều ngoặc mở càng tốt, rồi điền các vị trí còn lại bằng ngoặc đóng. Nói cách khác, chúng ta cố gắng giữ nguyên tiền tố dài nhất có thể, và phần hậu tố được thay bằng dãy nhỏ nhất theo thứ tự từ điển.

Để tìm vị trí này, chúng ta có thể duyệt qua các ký tự từ phải sang trái, và duy trì độ cân bằng depth của ngoặc mở và đóng. Khi gặp một ngoặc mở, chúng ta giảm depth, và khi gặp một ngoặc đóng, chúng ta tăng nó. Nếu tại một điểm nào đó gặp một ngoặc mở, và độ cân bằng sau khi xử lý ký tự này là dương, thì chúng ta đã tìm thấy vị trí cực phải mà chúng ta có thể thay đổi. Chúng ta thay đổi ký tự, tính số ngoặc mở và đóng cần thêm vào phía bên phải, và sắp xếp chúng theo cách nhỏ nhất theo thứ tự từ điển.

Nếu không tìm thấy vị trí phù hợp, thì dãy này đã là dãy tối đa có thể, và không có đáp án.

```
bool next_balanced_sequence(string & s) {
    int n = s.size();
    int depth = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == '(')
            depth--;
        else
            depth++;

        if (s[i] == '(' && depth > 0) {
            depth--;
            int open = (n - i - 1 - depth) / 2;
            int close = n - i - 1 - open;
            string next = s.substr(0, i) + ')' + string(open, '(') + string(close, ')
            s.swap(next);
            return true;
        }
    }
    return false;
}
```

Hàm này tính dãy ngoặc cân bằng tiếp theo trong thời gian $O(n)$, và trả về false nếu không có dãy tiếp theo.

### 1.9.4 Tìm tất cả các dãy cân bằng

Đôi khi cần tìm và xuất ra tất cả các dãy ngoặc cân bằng có độ dài cụ thể $n$.

Để sinh chúng, chúng ta có thể bắt đầu với dãy nhỏ nhất theo thứ tự từ điển $((\ldots(())\ldots))$, rồi tiếp tục tìm các dãy tiếp theo theo thứ tự từ điển với thuật toán được mô tả ở phần trước.

Tuy nhiên, nếu độ dài của dãy không quá dài (ví dụ $n$ nhỏ hơn 12), thì chúng ta cũng có thể sinh tất cả các hoán vị một cách tiện lợi với hàm next_permutation của C++ STL, và kiểm tra tính cân bằng của từng dãy.

Ngoài ra, chúng cũng có thể được sinh bằng cách sử dụng các ý tưởng mà chúng ta đã dùng để đếm tất cả các dãy với quy hoạch động. Chúng ta sẽ thảo luận các ý tưởng này trong hai phần tiếp theo.

### 1.9.5 Chỉ số của dãy

Cho một dãy ngoặc cân bằng với $n$ cặp ngoặc. Chúng ta phải tìm chỉ số của nó trong danh sách tất cả các dãy cân bằng với $n$ cặp ngoặc, được sắp xếp theo thứ tự từ điển.

Hãy định nghĩa một mảng phụ trợ $d[i][j]$, trong đó $i$ là độ dài của dãy ngoặc (nửa cân bằng, mỗi ngoặc đóng có một ngoặc mở tương ứng, nhưng không phải mọi ngoặc mở đều có ngoặc đóng tương ứng), và $j$ là độ cân bằng hiện tại (hiệu giữa ngoặc mở và đóng). $d[i][j]$ là số lượng các dãy như vậy phù hợp với các tham số. Chúng ta sẽ tính các số này với chỉ một loại ngoặc.

Với giá trị khởi đầu $i = 0$, đáp án hiển nhiên: $d[0][0] = 1$, và $d[0][j] = 0$ với $j > 0$. Bây giờ, giả sử $i > 0$, và chúng ta xem xét ký tự cuối cùng trong dãy. Nếu ký tự cuối là ngoặc mở (, thì trạng thái trước đó là $(i-1, j-1)$, nếu là ngoặc đóng ), thì trạng thái trước đó là $(i-1, j+1)$. Do đó, ta thu được công thức đệ quy:

$$d[i][j] = d[i-1][j-1] + d[i-1][j+1]$$

$d[i][j] = 0$ rõ ràng đúng với $j$ âm. Vậy, chúng ta có thể tính mảng này trong $O(n^2)$.

Bây giờ, hãy sinh chỉ số cho một dãy đã cho.

Trước tiên, giả sử chỉ có một loại ngoặc. Chúng ta sẽ sử dụng bộ đếm depth cho biết mức độ lồng hiện tại của chúng ta, và duyệt qua các ký tự của dãy. Nếu ký tự hiện tại $s[i]$ bằng (, thì tăng depth. Nếu ký tự hiện tại $s[i]$ bằng ), thì phải cộng $d[2n-i-1][\text{depth}+1]$ vào đáp án, xét đến tất cả các kết thúc có thể bắt đầu bằng ( (là các dãy nhỏ hơn theo thứ tự từ điển), rồi giảm depth.

Bây giờ, giả sử có $k$ loại ngoặc khác nhau.

Vậy, khi xem xét ký tự hiện tại $s[i]$ trước khi tính lại depth, chúng ta phải duyệt qua tất cả các loại ngoặc nhỏ hơn ký tự hiện tại, và thử đặt loại ngoặc này vào vị trí hiện tại (thu được độ cân bằng mới $\text{ndepth} = \text{depth} \pm 1$), rồi cộng số cách hoàn thành dãy (độ dài $2n-i-1$, độ cân bằng ndepth) vào đáp án:

$$d[2n-i-1][\text{ndepth}] \cdot k^{\frac{2n-i-1-\text{ndepth}}{2}}$$

Công thức này có thể được suy ra như sau: Trước tiên, chúng ta "quên" rằng có nhiều loại ngoặc, và chỉ lấy đáp án $d[2n-i-1][\text{ndepth}]$. Bây giờ, xét xem đáp án sẽ thay đổi như thế nào nếu có $k$ loại ngoặc. Chúng ta có $2n-i-1$ vị trí chưa xác định, trong đó ndepth đã được định trước do các ngoặc mở. Nhưng tất cả các ngoặc khác $((2n-i-1-\text{ndepth})/2$ cặp) có thể thuộc bất kỳ loại nào, do đó chúng ta nhân số này với lũy thừa của $k$ như vậy.

### 1.9.6 Tìm dãy thứ $k$

Gọi $n$ là số cặp ngoặc trong dãy. Chúng ta phải tìm dãy cân bằng thứ $k$ trong danh sách tất cả các dãy cân bằng được sắp xếp theo thứ tự từ điển cho một $k$ đã cho.

Như ở phần trước, chúng ta tính mảng phụ trợ $d[i][j]$, số lượng dãy ngoặc nửa cân bằng có độ dài $i$ với độ cân bằng $j$.

Trước tiên, chúng ta bắt đầu với chỉ một loại ngoặc.

Chúng ta sẽ duyệt qua các ký tự trong chuỗi mà chúng ta muốn sinh. Như trong bài toán trước, chúng ta lưu một bộ đếm depth, độ sâu lồng hiện tại. Tại mỗi vị trí, chúng ta phải quyết định đặt ngoặc mở hay ngoặc đóng. Để đặt một ngoặc mở, $d[2n-i-1][\text{depth}+1] \geq k$ phải đúng. Nếu đúng, chúng ta tăng bộ đếm depth, và chuyển sang ký tự tiếp theo. Ngược lại, chúng ta giảm $k$ đi $d[2n-i-1][\text{depth}+1]$, đặt một ngoặc đóng, và tiếp tục.

```
string kth_balanced(int n, int k) {
    vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }

    string ans;
    int depth = 0;
    for (int i = 0; i < 2*n; i++) {
        if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k) {
            ans += '(';
            depth++;
        } else {
            ans += ')';
            if (depth + 1 <= n)
                k -= d[2*n-i-1][depth+1];
```

```
            depth--;
        }
    }
    return ans;
}
```

Bây giờ, giả sử có $k$ loại ngoặc. Giải pháp chỉ khác một chút ở chỗ chúng ta phải nhân giá trị $d[2n-i-1][\text{ndepth}]$ với $k^{(2n-i-1-\text{ndepth})/2}$ và xét đến việc có thể có các loại ngoặc khác nhau cho ký tự tiếp theo.

Dưới đây là triển khai sử dụng hai loại ngoặc: ngoặc tròn và ngoặc vuông:

```
string kth_balanced2(int n, int k) {
    vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }

    string ans;
    int shift, depth = 0;

    stack<char> st;
    for (int i = 0; i < 2*n; i++) {

        // '('
        shift = ((2*n-i-1-depth-1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2*n-i-1][depth+1] << shift;
            if (cnt >= k) {
                ans += '(';
                st.push('(');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ')'
```

```
        shift = ((2*n-i-1-depth+1) / 2);
        if (shift >= 0 && depth && st.top() == '(') {
            int cnt = d[2*n-i-1][depth-1] << shift;
            if (cnt >= k) {
                ans += ')';
                st.pop();
                depth--;
                continue;
            }
            k -= cnt;
        }

        // '['
        shift = ((2*n-i-1-depth-1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2*n-i-1][depth+1] << shift;
            if (cnt >= k) {
                ans += '[';
                st.push('[');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ']'
        ans += ']';
        st.pop();
        depth--;
    }
    return ans;
}
```

## 1.10 Đếm đồ thị có nhãn

### 1.10.1 Đồ thị có nhãn

Giả sử số đỉnh trong một đồ thị là $n$. Chúng ta cần tính số $G_n$ là số đồ thị có nhãn với $n$ đỉnh (có nhãn nghĩa là các đỉnh được đánh dấu bằng các số từ 1 đến $n$). Các cạnh của đồ thị được coi là không định hướng, và không cho phép vòng lặp hay cạnh bội.

Chúng ta xét tập hợp tất cả các cạnh có thể có của đồ thị. Với mỗi cạnh $(i, j)$, ta có thể giả sử $i < j$ (vì đồ thị không định hướng và không có vòng lặp). Do đó, tập hợp tất cả các cạnh có số phần tử là $\binom{n}{2}$, tức là:

$$\frac{n(n-1)}{2}$$

Vì bất kỳ đồ thị có nhãn nào cũng được xác định duy nhất bởi các cạnh của nó, số đồ thị có nhãn với $n$ đỉnh bằng:

$$G_n = 2^{\frac{n(n-1)}{2}}$$

### 1.10.2 Đồ thị có nhãn liên thông

Ở đây, chúng ta thêm điều kiện ràng buộc rằng đồ thị phải liên thông.

Gọi số đồ thị liên thông cần tìm với $n$ đỉnh là $C_n$.

Chúng ta sẽ thảo luận trước về số lượng đồ thị không liên thông. Sau đó, số đồ thị liên thông sẽ là $G_n$ trừ đi số đồ thị không liên thông. Hơn nữa, chúng ta sẽ đếm số đồ thị không liên thông, có gốc. Đồ thị có gốc là một đồ thị mà chúng ta nhấn mạnh một đỉnh bằng cách đánh dấu nó là gốc. Rõ ràng, ta có $n$ cách để chọn gốc cho một đồ thị với $n$ đỉnh có nhãn, do đó cuối cùng chúng ta sẽ cần chia số đồ thị không liên thông có gốc cho $n$ để thu được số đồ thị không liên thông.

Đỉnh gốc sẽ nằm trong một thành phần liên thông có kích thước từ 1 đến $n-1$. Có $k\binom{n}{k}C_kG_{n-k}$ đồ thị sao cho đỉnh gốc nằm trong một thành phần liên thông với $k$ đỉnh (có $\binom{n}{k}$ cách để chọn $k$ đỉnh trongNh thành phần, các đỉnh này được kết nối theo một trong $C_k$ cách, đỉnh gốc có thể là bất kỳ đỉnh nào trong số $k$ đỉnh, và $n-k$ đỉnh còn lại có thể được kết nối/không kết nối theo bất kỳ cách nào, điều này cho một hệ số $G_{n-k}$). Do đó, số đồ thị không liên thông với $n$ đỉnh là:

$$\frac{1}{n}\sum_{k=1}^{n-1} k\binom{n}{k}C_kG_{n-k}$$

Và cuối cùng, số đồ thị liên thông là:

$$C_n = G_n - \frac{1}{n}\sum_{k=1}^{n-1} k\binom{n}{k}C_kG_{n-k}$$

### 1.10.3 Đồ thị có nhãn với $k$ thành phần liên thông

Dựa trên công thức từ phần trước, chúng ta sẽ học cách đếm số đồ thị có nhãn với $n$ đỉnh và $k$ thành phần liên thông.

Số này có thể được tính bằng quy hoạch động. Chúng ta sẽ tính $D[i][j]$ - số đồ thị có nhãn với $i$ đỉnh và $j$ thành phần - cho mỗi $i \leq n$ và $j \leq k$.

Hãy thảo luận cách tính phần tử tiếp theo $D[n][k]$ nếu chúng ta đã biết các giá trị trước đó. Chúng ta sử dụng một cách tiếp cận phổ biến, lấy đỉnh cuối cùng (chỉ số $n$). Đỉnh này thuộc về một thành phần nào đó. Nếu kích thước của thành phần này là $s$, thì có $\binom{n-1}{s-1}$ cách để chọn tập hợp các đỉnh như vậy, và $C_s$ cách để kết nối chúng. Sau khi loại bỏ thành phần này khỏi đồ thị, ta còn lại $n-s$ đỉnh với $k-1$ thành phần liên thông. Do đó, ta thu được quan hệ đệ quy sau:

$$D[n][k] = \sum_{s=1}^{n} \binom{n-1}{s-1} C_s D[n-s][k-1]$$

## 1.11 Tính liên thông cạnh / Tính liên thông đỉnh

### 1.11.1 Định nghĩa

Cho một đồ thị không định hướng $G$ với $n$ đỉnh và $m$ cạnh. Cả tính liên thông cạnh và tính liên thông đỉnh đều là các đặc trưng mô tả đồ thị.

#### Tính liên thông cạnh

Tính liên thông cạnh $\lambda$ của đồ thị $G$ là số lượng tối thiểu các cạnh cần xóa để đồ thị $G$ trở thành không liên thông.

Ví dụ, một đồ thị đã không liên thông có tính liên thông cạnh là 0, một đồ thị liên thông với ít nhất một cầu (bridge) có tính liên thông cạnh là 1, và một đồ thị liên thông không có cầu có tính liên thông cạnh ít nhất là 2.

Ta nói rằng một tập $S$ các cạnh phân tách các đỉnh $s$ và $t$, nếu sau khi loại bỏ tất cả các cạnh trong $S$ khỏi đồ thị $G$, các đỉnh $s$ và $t$ nằm trong các thành phần liên thông khác nhau.

Rõ ràng, tính liên thông cạnh của một đồ thị bằng kích thước tối thiểu của tập như vậy phân tách hai đỉnh $s$ và $t$, xét trên tất cả các cặp có thể $(s, t)$.

#### Tính liên thông đỉnh

Tính liên thông đỉnh $\kappa$ của đồ thị $G$ là số lượng tối thiểu các đỉnh cần xóa để đồ thị $G$ trở thành không liên thông.

Ví dụ, một đồ thị đã không liên thông có tính liên thông đỉnh là 0, và một đồ thị liên thông với một điểm khớp (articulation point) có tính liên thông đỉnh là 1. Ta định nghĩa rằng một đồ thị đầy đủ có tính liên thông đỉnh là $n-1$. Với tất cả các đồ thị khác, tính liên thông đỉnh không vượt quá $n-2$, vì bạn có thể tìm một cặp đỉnh không được nối bởi một cạnh, và xóa tất cả $n-2$ đỉnh còn lại.

Ta nói rằng một tập $T$ các đỉnh phân tách các đỉnh $s$ và $t$, nếu sau khi loại bỏ tất cả các đỉnh trong $T$ khỏi đồ thị $G$, các đỉnh này nằm trong các thành phần liên thông khác nhau.

Rõ ràng, tính liên thông đỉnh của một đồ thị bằng kích thước tối thiểu của tập như vậy phân tách hai đỉnh $s$ và $t$, xét trên tất cả các cặp có thể $(s, t)$.

### 1.11.2 Thuộc tính

#### Bất đẳng thức Whitney

Bất đẳng thức Whitney (1932) đưa ra mối quan hệ giữa tính liên thông cạnh $\lambda$, tính liên thông đỉnh $\kappa$, và bậc tối thiểu của bất kỳ đỉnh nào trong đồ thị $\delta$:

$$\kappa \leq \lambda \leq \delta$$

Theo trực giác, nếu ta có một tập các cạnh có kích thước $\lambda$, làm cho đồ thị không liên thông, ta có thể chọn một trong hai đầu mút của mỗi cạnh, và tạo ra một tập các đỉnh cũng làm đồ thị không liên thông. Và tập này có kích thước $\leq \lambda$.

Và nếu ta chọn đỉnh có bậc tối thiểu $\delta$, và xóa tất cả các cạnh nối với nó, thì ta cũng sẽ có một đồ thị không liên thông. Do đó, bất đẳng thức thứ hai $\lambda \leq \delta$.

Điều thú vị là bất đẳng thức Whitney không thể cải thiện: tức là với bất kỳ bộ ba số nào thỏa mãn bất đẳng thức này, tồn tại ít nhất một đồ thị tương ứng. Một đồ thị như vậy có thể được xây dựng theo cách sau: Đồ thị sẽ gồm $2(\delta + 1)$ đỉnh, $\delta + 1$ đỉnh đầu tiên tạo thành một đám đầy đủ (clique, tất cả các cặp đỉnh đều được nối bởi một cạnh), và $\delta + 1$ đỉnh thứ hai tạo thành một đám đầy đủ thứ hai. Ngoài ra, ta nối hai đám này bằng $\lambda$ cạnh, sao cho sử dụng $\lambda$ đỉnh khác nhau trong đám đầu tiên, và chỉ $\kappa$ đỉnh trong đám thứ hai. Đồ thị kết quả sẽ có ba đặc trưng này.

#### Định lý Ford-Fulkerson

Định lý Ford-Fulkerson ngụ ý rằng số lượng lớn nhất các đường đi không giao nhau về cạnh nối hai đỉnh bằng số lượng nhỏ nhất các cạnh phân tách các đỉnh này.

### 1.11.3 Tính toán các giá trị

#### Tính liên thông cạnh bằng luồng tối đa

Phương pháp này dựa trên định lý Ford-Fulkerson.

Ta duyệt qua tất cả các cặp đỉnh $(s, t)$ và giữa mỗi cặp, ta tìm số lượng lớn nhất các đường đi không giao nhau giữa chúng. Giá trị này có thể được tìm bằng thuật toán luồng tối đa: ta dùng $s$ làm nguồn, $t$ làm đích, và gán mỗi cạnh một dung lượng là 1. Khi đó, luồng tối đa là số lượng các đường đi không giao nhau.

Độ phức tạp của thuật toán sử dụng Edmonds-Karp là $O(V^2VE^2) = O(V^3E^2)$. Nhưng cần lưu ý rằng điều này bao gồm một hệ số ẩn, vì thực tế rất khó tạo ra một đồ thị sao cho thuật toán luồng tối đa chạy chậm với tất cả các nguồn và đích. Đặc biệt, thuật toán sẽ chạy khá nhanh với các đồ thị ngẫu nhiên.

#### Thuật toán đặc biệt cho tính liên thông cạnh

Nhiệm vụ tìm tính liên thông cạnh tương đương với nhiệm vụ tìm cắt tối thiểu toàn cục.

Các thuật toán đặc biệt đã được phát triển cho nhiệm vụ này. Một trong số đó là thuật toán Stoer-Wagner, hoạt động trong thời gian $O(V^3)$ hoặc $O(VE)$.

#### Tính liên thông đỉnh

Ta lại duyệt qua tất cả các cặp đỉnh $s$ và $t$, và với mỗi cặp, ta tìm số lượng tối thiểu các đỉnh phân tách $s$ và $t$.

Bằng cách này, ta có thể áp dụng cách tiếp cận luồng tối đa như đã mô tả trong các phần trước.

Ta chia mỗi đỉnh $x$ với $x \neq s$ và $x \neq t$ thành hai đỉnh $x_1$ và $x_2$. Ta nối hai đỉnh này bằng một cạnh có hướng $(x_1, x_2)$ với dung lượng 1, và thay tất cả các cạnh $(u, v)$ bằng hai cạnh có hướng $(u_2, v_1)$ và $(v_2, u_1)$, cả hai đều có dung lượng 1. Theo cách xây dựng này, giá trị của luồng tối đa sẽ bằng số lượng tối thiểu các đỉnh cần thiết để phân tách $s$ và $t$.

Cách tiếp cận này có độ phức tạp tương tự như cách tiếp cận luồng để tìm tính liên thông cạnh.

## 1.12 2-SAT

SAT (Bài toán thỏa mãn Boolean) là bài toán gán các giá trị Boolean cho các biến để thỏa mãn một công thức Boolean đã cho. Công thức Boolean thường được đưa ra dưới dạng CNF (dạng chuẩn hội), là một hội của nhiều mệnh đề, trong đó mỗi mệnh đề là một tuyển của các nguyên tố (biến hoặc phủ định của biến). 2-SAT (2-thỏa mãn) là một hạn chế của bài toán SAT, trong 2-SAT mỗi mệnh đề có chính xác hai nguyên tố. Dưới đây là một ví dụ về bài toán 2-SAT. Tìm một gán cho $a, b, c$ sao cho công thức sau là đúng:

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

SAT là NP-đầy đủ, không có giải pháp hiệu quả nào được biết đến cho nó. Tuy nhiên, 2-SAT có thể được giải hiệu quả trong $O(n + m)$ với $n$ là số biến và $m$ là số mệnh đề.

### 1.12.1 Thuật toán:

Đầu tiên, chúng ta cần chuyển đổi bài toán sang một dạng khác, gọi là dạng chuẩn hàm ý. Lưu ý rằng biểu thức $a \vee b$ tương đương với $\neg a \Rightarrow b \wedge \neg b \Rightarrow a$ (nếu một trong hai biến là sai, thì biến còn lại phải đúng).

Bây giờ chúng ta xây dựng một đồ thị có hướng của các hàm ý này: với mỗi biến $x$ sẽ có hai đỉnh $v_x$ và $v_{\neg x}$. Các cạnh sẽ tương ứng với các hàm ý.

Hãy xem xét ví dụ ở dạng 2-CNF:

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

Đồ thị có hướng sẽ chứa các đỉnh và cạnh sau:

$$\begin{array}{cccc} \neg a \Rightarrow \neg b & a \Rightarrow b & a \Rightarrow \neg b & \neg a \Rightarrow \neg c \\ b \Rightarrow a & \neg b \Rightarrow \neg a & b \Rightarrow \neg a & c \Rightarrow a \end{array}$$

Bạn có thể thấy đồ thị hàm ý trong hình sau:

"Đồ thị hàm ý của ví dụ 2-SAT"

Đáng chú ý là đặc tính của đồ thị hàm ý: nếu có cạnh $a \Rightarrow b$, thì cũng có cạnh $\neg b \Rightarrow \neg a$.

Cũng lưu ý rằng, nếu $x$ có thể tiếp cận từ $\neg x$, và $\neg x$ có thể tiếp cận từ $x$, thì bài toán không có nghiệm. Bất kể giá trị nào chúng ta chọn cho biến $x$, nó sẽ luôn dẫn đến mâu thuẫn - nếu $x$ được gán true thì hàm ý cho ta biết $\neg x$ cũng phải là true và ngược lại. Hóa ra, điều kiện này không chỉ cần thiết, mà còn đủ. Chúng ta sẽ chứng minh điều này trong vài đoạn dưới đây. Trước tiên, nhớ rằng nếu một đỉnh có thể tiếp cận từ đỉnh thứ hai, và đỉnh thứ hai có thể tiếp cận từ đỉnh đầu tiên, thì hai đỉnh này nằm trong cùng một thành phần liên thông mạnh. Do đó, chúng ta có thể phát biểu tiêu chí về sự tồn tại của nghiệm như sau:

Để bài toán 2-SAT này có nghiệm, điều kiện cần và đủ là với bất kỳ biến $x$ nào, các đỉnh $x$ và $\neg x$ phải nằm trong các thành phần liên thông mạnh khác nhau của đồ thị hàm ý.

Tiêu chí này có thể được kiểm tra trong thời gian $O(n + m)$ bằng cách tìm tất cả các thành phần liên thông mạnh.

Hình sau hiển thị tất cả các thành phần liên thông mạnh cho ví dụ. Như chúng ta có thể kiểm tra dễ dàng, không có thành phần nào trong bốn thành phần chứa đỉnh $x$ và phủ định của nó $\neg x$, do đó ví dụ này có nghiệm. Chúng ta sẽ tìm hiểu cách tính một gán hợp lệ trong các đoạn tiếp theo, nhưng chỉ để minh họa, nghiệm $a = \text{false}, b = \text{false}, c = \text{false}$ được đưa ra.

"Các thành phần liên thông mạnh của ví dụ 2-SAT"

Bây giờ chúng ta xây dựng thuật toán để tìm nghiệm của bài toán 2-SAT với giả định rằng nghiệm tồn tại.

Lưu ý rằng, mặc dù nghiệm tồn tại, có thể xảy ra trường hợp $\neg x$ có thể tiếp cận từ $x$ trong đồ thị hàm ý, hoặc (nhưng không đồng thời) $x$ có thể tiếp cận từ $\neg x$. Trong trường hợp đó, việc chọn true hoặc false cho $x$ sẽ dẫn đến mâu thuẫn, trong khi chọn giá trị còn lại thì không. Hãy tìm hiểu cách chọn một giá trị để không tạo ra mâu thuẫn.

Hãy sắp xếp các thành phần liên thông mạnh theo thứ tự tô-pô (tức là $\text{comp}[v] \leq \text{comp}[u]$ nếu có đường đi từ $v$ đến $u$) và gọi $\text{comp}[v]$ là chỉ số của thành phần liên thông mạnh mà đỉnh $v$ thuộc về. Sau đó, nếu $\text{comp}[x] < \text{comp}[\neg x]$ thì gán $x$ là false, và true trong trường hợp ngược lại.

Hãy chứng minh rằng với cách gán biến này, chúng ta không gặp mâu thuẫn. Giả sử $x$ được gán true. Trường hợp kia có thể được chứng minh tương tự.

Trước tiên, chứng minh rằng đỉnh $x$ không thể tiếp cận đỉnh $\neg x$. Vì chúng ta gán true, điều này đòi hỏi chỉ số của thành phần liên thông mạnh của $x$ phải lớn hơn chỉ số của thành phần của $\neg x$. Điều này có nghĩa là $\neg x$ nằm bên trái thành phần chứa $x$, và đỉnh sau không thể tiếp cận đỉnh trước.

Thứ hai, chứng minh rằng không tồn tại biến $y$ sao cho cả hai đỉnh $y$ và $\neg y$ đều có thể tiếp cận từ $x$ trong đồ thị hàm ý. Điều này sẽ gây mâu thuẫn, vì $x = \text{true}$ ngụ ý rằng $y = \text{true}$ và $\neg y = \text{true}$. Hãy chứng minh bằng phản chứng. Giả sử $y$ và $\neg y$ đều có thể tiếp cận từ $x$, thì theo tính chất của đồ thị hàm ý, $\neg x$ có thể tiếp cận từ cả $y$ và $\neg y$. Theo tính chất bắc cầu, điều này dẫn đến $\neg x$ có thể tiếp cận từ $x$, mâu thuẫn với giả định.

Vậy, chúng ta đã xây dựng một thuật toán tìm các giá trị cần thiết của các biến với giả định rằng với bất kỳ biến $x$ nào, các đỉnh $x$ và $\neg x$ nằm trong các thành phần liên thông mạnh khác nhau. Phía trên đã chứng minh tính đúng đắn của thuật toán này. Do đó, chúng ta đồng thời chứng minh tiêu chí về sự tồn tại của nghiệm ở trên.

### 1.12.2 Triển khai:

Bây giờ, chúng ta có thể triển khai toàn bộ thuật toán. Đầu tiên, xây dựng đồ thị hàm ý và tìm tất cả các thành phần liên thông mạnh. Điều này có thể được thực hiện bằng thuật toán Kosaraju trong thời gian $O(n+m)$. Trong lần duyệt thứ hai của đồ thị, thuật toán Kosaraju thăm các thành phần liên thông mạnh theo thứ tự tô-pô, do đó dễ dàng tính $\text{comp}[v]$ cho mỗi đỉnh $v$.

Sau đó, chúng ta có thể chọn gán cho $x$ bằng cách so sánh $\text{comp}[x]$ và $\text{comp}[\neg x]$. Nếu $\text{comp}[x] = \text{comp}[\neg x]$, ta trả về false để chỉ ra rằng không tồn tại gán hợp lệ thỏa mãn bài toán 2-SAT.

Dưới đây là triển khai giải pháp của bài toán 2-SAT cho đồ thị hàm ý đã xây dựng *adj* và đồ thị chuyển vị $\mathsf{Adj}^{\mathsf{T}}$(trong đó hướng của mỗi cạnh được đảo ngược). Trong đồ thị, các đỉnh với chỉ số $2k$ và $2k+1$ là hai đỉnh tương ứng với biến $k$ với $2k+1$ tương ứng với biến phủ định.

```
struct TwoSatSolver {
    int n_vars;
```

```
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;

    TwoSatSolver(int _n_vars) : n_vars(_n_vars), n_vertices(2 * n_vars), adj(n_vertic
                                adj_t(n_vertices), used(n_vertices), order(),
                                comp(n_vertices, -1), assignment(n_vars) {
        order.reserve(n_vertices);
    }
    void dfs1(int v) {
        used[v] = true;
        for (int u : adj[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }

    void dfs2(int v, int cl) {
        comp[v] = cl;
        for (int u : adj_t[v]) {
            if (comp[u] == -1)
                dfs2(u, cl);
        }
    }

    bool solve_2SAT() {
        order.clear();
        used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i])
                dfs1(i);
        }

        comp.assign(n_vertices, -1);
        for (int i = 0, j = 0; i < n_vertices; ++i) {
            int v = order[n_vertices - i - 1];
            if (comp[v] == -1)
```

```
                dfs2(v, j++);
        }

        assignment.assign(n_vars, false);
        for (int i = 0; i < n_vertices; i += 2) {
            if (comp[i] == comp[i + 1])
                return false;
            assignment[i / 2] = comp[i] > comp[i + 1];
        }
        return true;
    }

    void add_disjunction(int a, bool na, int b, bool nb) {
        // na và nb biểu thị liệu a và b có bị phủ định không
        a = 2 * a ^ na;
        b = 2 * b ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        adj[neg_a].push_back(b);
        adj[neg_b].push_back(a);
        adj_t[b].push_back(neg_a);
        adj_t[a].push_back(neg_b);
    }

    static void example_usage() {
        TwoSatSolver solver(3); // a, b, c
        solver.add_disjunction(0, false, 1, true);  //     a  v  not b
        solver.add_disjunction(0, true, 1, true);   // not a  v  not b
        solver.add_disjunction(1, false, 2, false); //     b  v      c
        solver.add_disjunction(0, false, 0, false); //     a  v      a
        assert(solver.solve_2SAT() == true);
        auto expected = vector<bool>{true, false, true};
        assert(solver.assignment == expected);
    }
};
```

# Chương 2

# String

## 2.1 Hàm băm chuỗi

Các thuật toán băm rất hữu ích trong việc giải quyết nhiều bài toán.

Chúng ta muốn giải quyết bài toán so sánh các chuỗi một cách hiệu quả. Cách đơn giản nhất là so sánh từng ký tự của hai chuỗi, điều này có độ phức tạp thời gian là $O(\min(n_1, n_2))$ nếu $n_1$ và $n_2$ là độ dài của hai chuỗi. Chúng ta muốn làm tốt hơn. Ý tưởng đằng sau hàm băm chuỗi là như sau: chúng ta ánh xạ mỗi chuỗi thành một số nguyên và so sánh các số đó thay vì so sánh chuỗi. Làm như vậy cho phép giảm thời gian thực thi của việc so sánh chuỗi xuống còn $O(1)$.

Để chuyển đổi, chúng ta cần một hàm gọi là hàm băm. Mục tiêu của nó là chuyển đổi một chuỗi thành một số nguyên, được gọi là giá trị băm của chuỗi. Điều kiện sau phải được thỏa mãn: nếu hai chuỗi $s$ và $t$ bằng nhau ($s = t$), thì giá trị băm của chúng cũng phải bằng nhau ($\text{hash}(s) = \text{hash}(t)$). Nếu không, chúng ta sẽ không thể so sánh các chuỗi.

Lưu ý, chiều ngược lại không cần phải đúng. Nếu giá trị băm bằng nhau ($\text{hash}(s) = \text{hash}(t)$), thì các chuỗi không nhất thiết phải bằng nhau. Ví dụ, một hàm băm hợp lệ có thể là $\text{hash}(s) = 0$ cho mỗi $s$. Đây chỉ là một ví dụ ngớ ngẩn, vì hàm này sẽ hoàn toàn vô dụng, nhưng nó là một hàm băm hợp lệ. Lý do tại sao chiều ngược lại không cần phải đúng là vì có số lượng chuỗi tăng theo cấp số nhân. Nếu chúng ta chỉ muốn hàm băm này phân biệt giữa tất cả các chuỗi gồm các ký tự chữ cái thường với độ dài nhỏ hơn 15, thì giá trị băm đã không thể nằm trong một số nguyên 64-bit (ví dụ: unsigned long long) nữa, vì có quá nhiều chuỗi như vậy. Và tất nhiên, chúng ta không muốn so sánh các số nguyên dài tùy ý, vì điều này cũng sẽ có độ phức tạp $O(n)$.

Vì vậy, thường thì chúng ta muốn hàm băm ánh xạ các chuỗi vào các số trong một phạm vi cố định $[0, m)$, khi đó việc so sánh chuỗi chỉ là so sánh hai số nguyên có độ dài cố định. Và tất nhiên, chúng ta muốn $\text{hash}(s) \neq \text{hash}(t)$ có xác suất rất cao nếu $s \neq t$.

Đó là phần quan trọng bạn cần ghi nhớ. Việc sử dụng băm sẽ không chính xác 100% một cách chắc chắn, vì hai chuỗi hoàn toàn khác nhau có thể có cùng giá trị băm (các

giá trị băm va chạm). Tuy nhiên, trong phần lớn các bài toán, điều này có thể được bỏ qua một cách an toàn vì xác suất va chạm giữa giá trị băm của hai chuỗi khác nhau vẫn rất nhỏ. Và chúng ta sẽ thảo luận một số kỹ thuật trong bài viết này để giữ xác suất va chạm rất thấp.

### 2.1.1 Tính toán giá trị băm của một chuỗi

Cách tốt và được sử dụng rộng rãi để định nghĩa giá trị băm của một chuỗi $s$ có độ dài $n$ là:

$$\begin{aligned}\text{hash}(s) &= \left(s[0] + s[1] \cdot p + s[2] \cdot p^2 + \cdots + s[n-1] \cdot p^{n-1}\right) \bmod m \\ &= \left(\sum_{i=0}^{n-1} s[i] \cdot p^i\right) \bmod m.\end{aligned}$$

nơi $p$ và $m$ là một số nguyên dương được chọn. Nó được gọi là hàm băm đa thức lăn.

Việc chọn $p$ là một số nguyên tố gần bằng số ký tự trong bảng chữ cái đầu vào là hợp lý. Ví dụ, nếu đầu vào chỉ gồm các chữ cái thường của bảng chữ cái tiếng Anh, $p = 31$ là một lựa chọn tốt. Nếu đầu vào có thể chứa cả chữ cái in hoa và thường, thì $p = 53$ là một lựa chọn khả thi. Mã trong bài viết này sẽ sử dụng $p = 31$.

Rõ ràng $m$ nên là một số lớn vì xác suất hai chuỗi ngẫu nhiên va chạm là khoảng $\approx \frac{1}{m}$. Đôi khi $m = 2^{64}$ được chọn, vì khi đó tràn số nguyên của số nguyên 64-bit hoạt động giống như phép lấy modulo. Tuy nhiên, tồn tại một phương pháp tạo ra các chuỗi va chạm (hoạt động độc lập với lựa chọn của $p$). Vì vậy, trong thực tế, $m = 2^{64}$ không được khuyến nghị. Một lựa chọn tốt cho $m$ là một số nguyên tố lớn. Mã trong bài viết này sẽ chỉ sử dụng $m = 10^9 + 9$. Đây là một số lớn, nhưng vẫn đủ nhỏ để chúng ta có thể thực hiện phép nhân của hai giá trị bằng số nguyên 64-bit.

Dưới đây là ví dụ tính giá trị băm của một chuỗi $s$, chỉ chứa các chữ cái thường. Chúng ta chuyển mỗi ký tự của $s$ thành một số nguyên. Ở đây chúng ta sử dụng quy ước $a \to 1$, $b \to 2$, ..., $z \to 26$. Việc chuyển $a \to 0$ không phải là ý hay, vì khi đó giá trị băm của các chuỗi $a$, $aa$, $aaa$, ... đều bằng 0.

```
long long compute_hash(string const& s) {
    const int p = 31;
    const int m = 1e9 + 9;
    long long hash_value = 0;
    long long p_pow = 1;
    for (char c : s) {
        hash_value = (hash_value + (c - 'a' + 1) * p_pow) % m;
        p_pow = (p_pow * p) % m;
    }
    return hash_value;
```

```
}
```

Việc tính trước các lũy thừa của $p$ có thể mang lại hiệu suất tốt hơn.

### 2.1.2 Các bài toán ví dụ

#### Tìm kiếm chuỗi trùng lặp trong mảng chuỗi

**Bài toán:** Cho một danh sách gồm $n$ chuỗi $s_i$, mỗi chuỗi không dài quá $m$ ký tự, tìm tất cả các chuỗi trùng lặp và chia chúng thành các nhóm.

Từ thuật toán hiển nhiên liên quan đến việc sắp xếp các chuỗi, ta sẽ có độ phức tạp thời gian là $O(nm \log n)$ nơi việc sắp xếp cần $O(n \log n)$ phép so sánh và mỗi phép so sánh mất $O(m)$ thời gian. Tuy nhiên, bằng cách sử dụng băm, ta giảm thời gian so sánh xuống $O(1)$, cho chúng ta một thuật toán chạy trong $O(nm + n \log n)$ thời gian.

Chúng ta tính giá trị băm cho mỗi chuỗi, sắp xếp các giá trị băm cùng với chỉ số, sau đó nhóm các chỉ số theo các giá trị băm giống nhau.

```
vector<vector<int>> group_identical_strings(vector<string> const& s) {
    int n = s.size();
    vector<pair<long long, int>> hashes(n);
    for (int i = 0; i < n; i++)
        hashes[i] = {compute_hash(s[i]), i};

    sort(hashes.begin(), hashes.end());

    vector<vector<int>> groups;
    for (int i = 0; i < n; i++) {
        if (i == 0 || hashes[i].first != hashes[i-1].first)
            groups.emplace_back();
        groups.back().push_back(hashes[i].second);
    }
    return groups;
}
```

#### Tính nhanh giá trị băm của chuỗi con của chuỗi đã cho

**Bài toán:** Cho một chuỗi $s$ và các chỉ số $i$ và $j$, tìm giá trị băm của chuỗi con $s[i \dots j]$.

Theo định nghĩa, ta có:

$$\text{hash}(s[i \dots j]) = \sum_{k=i}^{j} s[k] \cdot p^{k-i} \mod m$$

Nhân với $p^i$ cho:

$$\begin{aligned}\text{hash}(s[i:j]) \cdot p^i &= \left(\sum_{k=i}^{j} s[k] \cdot p^k\right) \bmod m \\ &= (\text{hash}(s[0:j]) - \text{hash}(s[0:i-1])) \bmod m\end{aligned}$$

Vậy, bằng cách biết giá trị băm của mỗi tiền tố của chuỗi $s$, ta có thể tính giá trị băm của bất kỳ chuỗi con nào trực tiếp bằng công thức này. Vấn đề duy nhất chúng ta gặp phải khi tính toán là phải có khả năng chia $\text{hash}(s[0 \dots j]) - \text{hash}(s[0 \dots i-1])$ cho $p^i$. Do đó, ta cần tìm nghịch đảo nhân modulo của $p^i$ và sau đó thực hiện phép nhân với nghịch đảo này. Chúng ta có thể tính trước nghịch đảo của mỗi $p^i$, điều này cho phép tính giá trị băm của bất kỳ chuỗi con nào của $s$ trong thời gian $O(1)$.

Tuy nhiên, tồn tại một cách đơn giản hơn. Trong hầu hết các trường hợp, thay vì tính chính xác giá trị băm của chuỗi con, chỉ cần tính giá trị băm nhân với một lũy thừa nào đó của $p$ là đủ. Giả sử ta có hai giá trị băm của hai chuỗi con, một nhân với $p^i$ và cái kia nhân với $p^j$. Nếu $i < j$ thì ta nhân giá trị băm đầu tiên với $p^{j-i}$, ngược lại, ta nhân giá trị băm thứ hai với $p^{i-j}$. Bằng cách này, ta có cả hai giá trị băm được nhân với cùng một lũy thừa của $p$ (là tối đa của $i$ và $j$), và giờ đây các giá trị băm này có thể được so sánh dễ dàng mà không cần phép chia.

### 2.1.3 Ứng dụng của hàm băm

Dưới đây là một số ứng dụng điển hình của hàm băm:

- Thuật toán Rabin-Karp để tìm kiếm mẫu trong chuỗi trong thời gian $O(n)$ - Tính số chuỗi con khác nhau của một chuỗi trong $O(n^2)$ (xem bên dưới) - Tính số chuỗi con đối xứng trong một chuỗi. - Xác định số chuỗi con khác nhau trong một chuỗi

#### Xác định số chuỗi con khác nhau trong một chuỗi

**Bài toán:** Cho một chuỗi $s$ có độ dài $n$, chỉ gồm các chữ cái thường tiếng Anh, tìm số chuỗi con khác nhau trong chuỗi này.

Để giải bài toán này, ta duyệt qua tất cả các độ dài chuỗi con $l = 1 \dots n$. Với mỗi độ dài chuỗi con $l$, ta xây dựng một mảng các giá trị băm của tất cả các chuỗi con có độ dài $l$ nhân với cùng một lũy thừa của $p$. Số phần tử khác nhau trong mảng bằng số chuỗi con khác nhau có độ dài $l$ trong chuỗi. Số này được cộng vào đáp án cuối cùng.

Để thuận tiện, ta sẽ sử dụng $h[i]$ là giá trị băm của tiền tố với $i$ ký tự, và định nghĩa $h[0] = 0$.

```
int count_unique_substrings(string const& s) {
    int n = s.size();
```

```
    const int p = 31;
    const int m = 1e9 + 9;
    vector<long long> p_pow(n);
    p_pow[0] = 1;
    for (int i = 1; i < n; i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(n + 1, 0);
    for (int i = 0; i < n; i++)
        h[i+1] = (h[i] + (s[i] - 'a' + 1) * p_pow[i]) % m;

    int cnt = 0;
    for (int l = 1; l <= n; l++) {
        unordered_set<long long> hs;
        for (int i = 0; i <= n - l; i++) {
            long long cur_h = (h[i + l] + m - h[i]) % m;
            cur_h = (cur_h * p_pow[n-i-1]) % m;
            hs.insert(cur_h);
        }
        cnt += hs.size();
    }
    return cnt;
}
```

Lưu ý rằng $O(n^2)$ không phải là độ phức tạp thời gian tốt nhất cho bài toán này. Một giải pháp với $O(n \log n)$ được mô tả trong bài viết về Mảng hậu tố (Suffix Arrays), và thậm chí có thể tính trong $O(n)$ bằng cách sử dụng Cây hậu tố (Suffix Tree) hoặc Máy tự động hậu tố (Suffix Automaton).

### 2.1.4 Cải thiện xác suất không va chạm

Thông thường, hàm băm đa thức đã nêu ở trên là đủ tốt, và không có va chạm nào xảy ra trong quá trình kiểm tra. Hãy nhớ rằng, xác suất xảy ra va chạm chỉ là $\approx \frac{1}{m}$. Với $m = 10^9 + 9$, xác suất là $\approx 10^{-9}$, rất thấp. Nhưng lưu ý, chúng ta chỉ thực hiện một phép so sánh. Nếu ta so sánh một chuỗi $s$ với $10^6$ chuỗi khác nhau, xác suất xảy ra ít nhất một va chạm giờ đây là $\approx 10^{-3}$. Và nếu ta muốn so sánh $10^6$ chuỗi khác nhau với nhau (ví dụ: bằng cách đếm số chuỗi duy nhất), thì xác suất xảy ra ít nhất một va chạm đã là $\approx 1$. Gần như chắc chắn rằng bài toán này sẽ kết thúc với một va chạm và trả về kết quả sai.

Có một mẹo rất đơn giản để có xác suất tốt hơn. Chúng ta có thể tính hai giá trị băm khác nhau cho mỗi chuỗi (bằng cách sử dụng hai $p$ khác nhau, và/hoặc hai $m$ khác nhau, và so sánh các cặp này thay vì chỉ một giá trị). Nếu $m$ khoảng $10^9$ cho mỗi hàm băm, thì

điều này tương đương với việc có một hàm băm với $m \approx 10^{18}$. Khi so sánh $10^6$ chuỗi với nhau, xác suất xảy ra ít nhất một va chạm giờ đây giảm xuống còn $\approx 10^{-6}$.

## 2.2 Thuật toán Rabin-Karp cho tìm kiếm chuỗi

Thuật toán này dựa trên khái niệm băm, vì vậy nếu bạn chưa quen với băm chuỗi, hãy tham khảo bài viết về băm chuỗi.

Thuật toán này được phát triển bởi Rabin và Karp vào năm 1987.

**Bài toán:** Cho hai chuỗi - một mẫu $s$ và một văn bản $t$, xác định xem mẫu có xuất hiện trong văn bản không và nếu có, liệt kê tất cả các vị trí xuất hiện của nó trong thời gian $O(|s| + |t|)$.

### 2.2.1 Thuật toán:

Tính giá trị băm cho mẫu $s$. Tính giá trị băm cho tất cả các tiền tố của văn bản $t$. Bây giờ, chúng ta có thể so sánh một chuỗi con có độ dài $|s|$ với $s$ trong thời gian không đổi bằng cách sử dụng các giá trị băm đã tính. Vì vậy, so sánh mỗi chuỗi con có độ dài $|s|$ với mẫu. Việc này sẽ mất tổng cộng $O(|t|)$ thời gian. Do đó, độ phức tạp cuối cùng của thuật toán là $O(|t|+|s|)$: $O(|s|)$ cần để tính giá trị băm của mẫu và $O(|t|)$ để so sánh mỗi chuỗi con có độ dài $|s|$ với mẫu.

### 2.2.2 Triển khai

```
vector<int> rabin_karp(string const& s, string const& t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<long long> p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < (int)p_pow.size(); i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i+1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
    long long h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;
```

```
    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        long long cur_h = (h[i+S] + m - h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}
```

## 2.3 Hàm tiền tố. Thuật toán Knuth–Morris–Pratt

### 2.3.1 Định nghĩa hàm tiền tố

Cho một chuỗi $s$ có độ dài $n$. Hàm tiền tố cho chuỗi này được định nghĩa là một mảng $\pi$ có độ dài $n$, trong đó $\pi[i]$ là độ dài của tiền tố thực sự dài nhất của chuỗi con $s[0 \dots i]$ mà cũng là một hậu tố của chuỗi con này. Tiền tố thực sự của một chuỗi là tiền tố không bằng chính chuỗi đó. Theo định nghĩa, $\pi[0] = 0$.

Về mặt toán học, định nghĩa của hàm tiền tố có thể được viết như sau:

$$\pi[i] = \max_{k=0\dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ví dụ, hàm tiền tố của chuỗi "abcabcd" là $[0, 0, 0, 1, 2, 3, 0]$, và hàm tiền tố của chuỗi "aabaaab" là $[0, 1, 0, 1, 2, 2, 3]$.

### 2.3.2 Thuật toán đơn giản

Một thuật toán tuân theo đúng định nghĩa của hàm tiền tố là như sau:

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 0; i < n; i++)
        for (int k = 0; k <= i; k++)
            if (s.substr(0, k) == s.substr(i-k+1, k))
                pi[i] = k;
    return pi;
}
```

Dễ thấy rằng độ phức tạp của nó là $O(n^3)$, điều này có thể được cải thiện.

### 2.3.3 Thuật toán hiệu quả

Thuật toán này được đề xuất bởi Knuth và Pratt, và độc lập với họ bởi Morris vào năm 1977. Nó được sử dụng như hàm chính của thuật toán tìm kiếm chuỗi con.

#### Tối ưu hóa đầu tiên

Quan sát quan trọng đầu tiên là các giá trị của hàm tiền tố chỉ có thể tăng tối đa một đơn vị.

Thật vậy, nếu không, nếu $\pi[i+1] > \pi[i]+1$, thì ta có thể lấy hậu tố kết thúc ở vị trí $i+1$ với độ dài $\pi[i+1]$ và loại bỏ ký tự cuối cùng khỏi nó. Ta sẽ có một hậu tố kết thúc ở vị trí $i$ với độ dài $\pi[i+1]-1$, điều này tốt hơn $\pi[i]$, tức là ta gặp mâu thuẫn.

Hình minh họa sau cho thấy mâu thuẫn này. Hậu tố thực sự dài nhất ở vị trí $i$ cũng là một tiền tố có độ dài 2, và ở vị trí $i+1$ nó có độ dài 4. Do đó, chuỗi $s_0\ s_1\ s_2\ s_3$ bằng chuỗi $s_{i-2}\ s_{i-1}\ s_i\ s_{i+1}$, điều này có nghĩa là các chuỗi $s_0\ s_1\ s_2$ và $s_{i-2}\ s_{i-1}\ s_i$ cũng bằng nhau, do đó $\pi[i]$ phải là 3.

$$\underbrace{\overbrace{s_0\ s_1}^{\pi[i]=2}\ s_2\ s_3}_{\pi[i+1]=4} \cdots \underbrace{s_{i-2}\ \overbrace{s_{i-1}\ s_i}^{\pi[i]=2}\ s_{i+1}}_{\pi[i+1]=4}$$

Vậy khi chuyển sang vị trí tiếp theo, giá trị của hàm tiền tố có thể tăng thêm một, giữ nguyên, hoặc giảm đi một lượng nào đó. Sự thật này đã cho phép ta giảm độ phức tạp của thuật toán xuống $O(n^2)$, vì trong một bước hàm tiền tố chỉ có thể tăng tối đa một đơn vị. Tổng cộng hàm này chỉ có thể tăng tối đa $n$ bước, và do đó cũng chỉ có thể giảm tổng cộng $n$ bước. Điều này có nghĩa là ta chỉ cần thực hiện $O(n)$ phép so sánh chuỗi, và đạt độ phức tạp $O(n^2)$.

#### Tối ưu hóa thứ hai

Hãy tiến xa hơn, ta muốn loại bỏ việc so sánh chuỗi. Để làm điều này, ta phải sử dụng tất cả thông tin đã tính toán ở các bước trước.

Vậy hãy tính giá trị của hàm tiền tố $\pi$ cho $i+1$. Nếu $s[i+1] = s[\pi[i]]$, thì ta có thể chắc chắn nói rằng $\pi[i+1] = \pi[i]+1$, vì ta đã biết rằng hậu tố ở vị trí $i$ có độ dài $\pi[i]$ bằng tiền tố có độ dài $\pi[i]$. Điều này lại được minh họa bằng một ví dụ.

$$\underbrace{\overbrace{s_0\ s_1\ s_2}^{\pi[i]}\ \overbrace{s_3}^{s_3=s_{i+1}}}_{\pi[i+1]=\pi[i]+1} \cdots \underbrace{\overbrace{s_{i-2}\ s_{i-1}\ s_i}^{\pi[i]}\ \overbrace{s_{i+1}}^{s_3=s_{i+1}}}_{\pi[i+1]=\pi[i]+1}$$

Nếu không phải trường hợp này, $s[i+1] \neq s[\pi[i]]$, thì ta cần thử một chuỗi ngắn hơn. Để tăng tốc độ, ta muốn ngay lập tức chuyển đến độ dài dài nhất $j < \pi[i]$, sao cho tính chất tiền tố ở vị trí $i$ vẫn đúng, tức là $s[0 \ldots j-1] = s[i-j+1 \ldots i]$:

$$\overbrace{\underbrace{s_0\ s_1}_{j}\ s_2\ s_3}^{\pi[i]}\ \ldots\ \overbrace{s_{i-3}\ s_{i-2}\ \underbrace{s_{i-1}\ s_i}_{j}}^{\pi[i]}\ s_{i+1}$$

Thật vậy, nếu ta tìm được độ dài $j$ như vậy, thì ta chỉ cần so sánh các ký tự $s[i+1]$ và $s[j]$. Nếu chúng bằng nhau, thì ta có thể gán $\pi[i+1] = j+1$. Nếu không, ta sẽ cần tìm giá trị lớn nhất nhỏ hơn $j$, mà tại đó tính chất tiền tố vẫn đúng, và cứ tiếp tục như vậy. Có thể điều này kéo dài đến khi $j = 0$. Nếu lúc đó $s[i+1] = s[0]$, ta gán $\pi[i+1] = 1$, và nếu không thì $\pi[i+1] = 0$.

Vậy ta đã có một lược đồ tổng quát của thuật toán. Câu hỏi duy nhất còn lại là làm thế nào để tìm các độ dài cho $j$ một cách hiệu quả. Hãy ôn lại: với độ dài hiện tại $j$ ở vị trí $i$ mà tại đó tính chất tiền tố đúng, tức là $s[0 \ldots j-1] = s[i-j+1 \ldots i]$, ta muốn tìm $k < j$ lớn nhất, mà tại đó tính chất tiền tố vẫn đúng.

$$\overbrace{\underbrace{s_0\ s_1}_{k}\ s_2\ s_3}^{j}\ \ldots\ \overbrace{s_{i-3}\ s_{i-2}\ \underbrace{s_{i-1}\ s_i}_{k}}^{j}\ s_{i+1}$$

Hình minh họa cho thấy điều này phải là giá trị của $\pi[j-1]$, mà ta đã tính trước đó.

**Thuật toán cuối cùng**

Vậy cuối cùng ta có thể xây dựng một thuật toán không thực hiện bất kỳ phép so sánh chuỗi nào và chỉ thực hiện $O(n)$ hành động.

Dưới đây là quy trình cuối cùng:

Ta tính các giá trị tiền tố $\pi[i]$ trong một vòng lặp bằng cách lặp từ $i = 1$ đến $i = n-1$ ($\pi[0]$ chỉ được gán bằng 0). Để tính giá trị hiện tại $\pi[i]$, ta đặt biến $j$ biểu thị độ dài của hậu tố tốt nhất cho $i-1$. Ban đầu $j = \pi[i-1]$. Kiểm tra xem hậu tố có độ dài $j+1$ cũng là một tiền tố bằng cách so sánh $s[j]$ và $s[i]$. Nếu chúng bằng nhau thì ta gán $\pi[i] = j+1$, nếu không ta giảm $j$ xuống $\pi[j-1]$ và lặp lại bước này. Nếu ta đã đạt đến độ dài $j = 0$ mà vẫn không khớp, thì ta gán $\pi[i] = 0$ và chuyển sang chỉ số tiếp theo $i+1$.

## 2.3.4 Triển khai

Việc triển khai cuối cùng trở nên ngắn gọn và biểu cảm đáng ngạc nhiên.

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
```

```
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

Đây là một thuật toán trực tuyến, tức là nó xử lý dữ liệu ngay khi nhận được - ví dụ, bạn có thể đọc từng ký tự của chuỗi một và xử lý chúng ngay lập tức, tìm giá trị của hàm tiền tố cho mỗi ký tự tiếp theo. Thuật toán vẫn yêu cầu lưu trữ chính chuỗi và các giá trị đã tính trước của hàm tiền tố, nhưng nếu ta biết trước giá trị tối đa $M$ mà hàm tiền tố có thể nhận trên chuỗi, ta chỉ cần lưu trữ $M + 1$ ký tự đầu tiên của chuỗi và cùng số lượng giá trị của hàm tiền tố.

### 2.3.5 Ứng dụng

**Tìm kiếm chuỗi con trong chuỗi. Thuật toán Knuth-Morris-Pratt**

Nhiệm vụ này là ứng dụng cổ điển của hàm tiền tố.

Cho một văn bản $t$ và một chuỗi $s$, ta muốn tìm và hiển thị các vị trí của tất cả các lần xuất hiện của chuỗi $s$ trong văn bản $t$.

Để thuận tiện, ta ký hiệu $n$ là độ dài của chuỗi $s$ và $m$ là độ dài của văn bản $t$.

Ta tạo chuỗi $s + \# + t$, trong đó $\#$ là một ký tự phân cách không xuất hiện trong $s$ cũng như trong $t$. Hãy tính hàm tiền tố cho chuỗi này. Bây giờ, hãy nghĩ về ý nghĩa của các giá trị của hàm tiền tố, ngoại trừ $n + 1$ mục đầu tiên (thuộc về chuỗi $s$ và ký tự phân cách). Theo định nghĩa, giá trị $\pi[i]$ cho thấy độ dài lớn nhất của một chuỗi con kết thúc ở vị trí $i$ mà trùng với tiền tố. Nhưng trong trường hợp của chúng ta, điều này không gì khác ngoài khối lớn nhất trùng với $s$ và kết thúc ở vị trí $i$. Độ dài này không thể lớn hơn $n$ do ký tự phân cách. Nhưng nếu đạt được đẳng thức $\pi[i] = n$, thì điều đó có nghĩa là chuỗi $s$ xuất hiện hoàn toàn ở vị trí này, tức là nó kết thúc ở vị trí $i$. Đừng quên rằng các vị trí được đánh chỉ số trong chuỗi $s + \# + t$.

Vậy nếu ở một vị trí $i$ nào đó ta có $\pi[i] = n$, thì ở vị trí $i - (n + 1) - n + 1 = i - 2n$ trong chuỗi $t$, chuỗi $s$ xuất hiện.

Như đã đề cập trong phần mô tả tính toán hàm tiền tố, nếu ta biết rằng các giá trị tiền tố không bao giờ vượt quá một giá trị nhất định, thì ta không cần lưu trữ toàn bộ chuỗi và toàn bộ hàm, mà chỉ cần phần đầu của nó. Trong trường hợp của chúng ta, điều này có nghĩa là ta chỉ cần lưu chuỗi $s + \#$ và các giá trị của hàm tiền tố cho nó. Ta có thể đọc từng ký tự của chuỗi $t$ một và tính giá trị hiện tại của hàm tiền tố.

Do đó, thuật toán Knuth-Morris-Pratt giải quyết bài toán trong thời gian $O(n + m)$ và bộ nhớ $O(n)$.

### Đếm số lần xuất hiện của mỗi tiền tố

Ở đây, ta thảo luận hai bài toán cùng lúc. Cho một chuỗi $s$ có độ dài $n$. Trong biến thể đầu tiên của bài toán, ta muốn đếm số lần xuất hiện của mỗi tiền tố $s[0 \dots i]$ trong cùng chuỗi đó. Trong biến thể thứ hai, một chuỗi $t$ khác được cho và ta muốn đếm số lần xuất hiện của mỗi tiền tố $s[0 \dots i]$ trong $t$.

Trước tiên, ta giải bài toán thứ nhất. Xét giá trị của hàm tiền tố $\pi[i]$ tại một vị trí $i$. Theo định nghĩa, nó có nghĩa là tiền tố có độ dài $\pi[i]$ của chuỗi $s$ xuất hiện và kết thúc tại vị trí $i$, và không có tiền tố nào dài hơn thỏa mãn định nghĩa này. Đồng thời, các tiền tố ngắn hơn có thể kết thúc tại vị trí này. Không khó để thấy rằng ta có cùng câu hỏi mà ta đã trả lời khi tính chính hàm tiền tố: Cho một tiền tố có độ dài $j$ là một hậu tố kết thúc tại vị trí $i$, tiền tố nhỏ hơn tiếp theo $< j$ cũng là một hậu tố kết thúc tại vị trí $i$ là gì. Do đó, tại vị trí $i$ kết thúc tiền tố có độ dài $\pi[i]$, tiền tố có độ dài $\pi[\pi[i] - 1]$, tiền tố $\pi[\pi[\pi[i] - 1] - 1]$, và cứ thế, cho đến khi chỉ số trở thành 0. Vậy ta có thể tính đáp án theo cách sau:

```
vector<int> ans(n + 1);
for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n-1; i > 0; i--)
    ans[pi[i-1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++;
```

Ở đây, với mỗi giá trị của hàm tiền tố, ta trước tiên đếm xem nó xuất hiện bao nhiêu lần trong mảng $\pi$, sau đó tính các đáp án cuối cùng: nếu ta biết rằng tiền tố độ dài $i$ xuất hiện chính xác ans$[i]$ lần, thì số này phải được cộng vào số lần xuất hiện của hậu tố dài nhất của nó cũng là một tiền tố. Cuối cùng, ta cần cộng 1 vào mỗi kết quả, vì ta cũng cần đếm cả các tiền tố ban đầu.

Bây giờ, hãy xem xét bài toán thứ hai. Ta áp dụng mẹo từ Knuth-Morris-Pratt: ta tạo chuỗi $s + \# + t$ và tính hàm tiền tố của nó. Điểm khác biệt duy nhất so với bài toán đầu tiên là ta chỉ quan tâm đến các giá trị tiền tố liên quan đến chuỗi $t$, tức là $\pi[i]$ cho $i \geq n + 1$. Với những giá trị này, ta có thể thực hiện các phép tính tương tự như trong bài toán đầu tiên.

### Số chuỗi con khác nhau trong một chuỗi

Cho một chuỗi $s$ có độ dài $n$. Ta muốn tính số chuỗi con khác nhau xuất hiện trong nó.

Ta sẽ giải bài toán này một cách lặp. Cụ thể, ta sẽ học cách, biết số chuỗi con khác nhau hiện tại, làm thế nào để tính lại số này bằng cách thêm một ký tự vào cuối.

Giả sử $k$ là số chuỗi con khác nhau hiện tại trong $s$, và ta thêm ký tự $c$ vào cuối $s$. Rõ ràng một số chuỗi con mới kết thúc bằng $c$ sẽ xuất hiện. Ta muốn đếm những chuỗi con mới này mà chưa xuất hiện trước đó.

Ta lấy chuỗi $t = s + c$ và đảo ngược nó. Bây giờ, nhiệm vụ được chuyển thành tính số tiền tố không xuất hiện ở bất kỳ nơi nào khác. Nếu ta tính giá trị tối đa của hàm tiền tố $\pi_{\max}$ của chuỗi đảo ngược $t$, thì tiền tố dài nhất xuất hiện trong $s$ có độ dài $\pi_{\max}$. Rõ ràng tất cả các tiền tố có độ dài nhỏ hơn cũng xuất hiện trong đó.

Do đó, số chuỗi con mới xuất hiện khi ta thêm một ký tự mới $c$ là $|s| + 1 - \pi_{\max}$.

Vậy với mỗi ký tự được thêm vào, ta có thể tính số chuỗi con mới trong thời gian $O(n)$, điều này cho độ phức tạp thời gian tổng cộng là $O(n^2)$.

Đáng chú ý là ta cũng có thể tính số chuỗi con khác nhau bằng cách thêm các ký tự vào đầu, hoặc bằng cách xóa các ký tự từ đầu hoặc cuối.

### Nén một chuỗi

Cho một chuỗi $s$ có độ dài $n$. Ta muốn tìm biểu diễn "nén" ngắn nhất của chuỗi, tức là ta muốn tìm một chuỗi $t$ có độ dài nhỏ nhất sao cho $s$ có thể được biểu diễn dưới dạng nối của một hoặc nhiều bản sao của $t$.

Rõ ràng, ta chỉ cần tìm độ dài của $t$. Biết độ dài, đáp án của bài toán sẽ là tiền tố của $s$ với độ dài này.

Hãy tính hàm tiền tố cho $s$. Sử dụng giá trị cuối cùng của nó, ta định nghĩa giá trị $k = n - \pi[n-1]$. Ta sẽ chứng minh rằng nếu $k$ chia hết cho $n$, thì $k$ sẽ là đáp án, nếu không thì không có nén hiệu quả và đáp án là $n$.

Giả sử $n$ chia hết cho $k$. Khi đó, chuỗi có thể được chia thành các khối có độ dài $k$. Theo định nghĩa của hàm tiền tố, tiền tố có độ dài $n - k$ sẽ bằng với hậu tố của nó. Nhưng điều này có nghĩa là khối cuối cùng bằng với khối trước nó. Và khối trước đó phải bằng với khối trước nó nữa. Và cứ thế. Kết quả là, tất cả các khối đều bằng nhau, do đó ta có thể nén chuỗi $s$ thành độ dài $k$.

Tất nhiên, ta vẫn cần chứng minh rằng đây thực sự là tối ưu. Thật vậy, nếu có một nén nhỏ hơn $k$, thì hàm tiền tố ở cuối sẽ lớn hơn $n - k$. Do đó, $k$ thực sự là đáp án.

Bây giờ, giả sử $n$ không chia hết cho $k$. Ta chứng minh rằng điều này ngụ ý độ dài của đáp án là $n$. Ta chứng minh bằng phản chứng. Giả sử tồn tại một đáp án, và độ dài nén là $p$ ($p$ chia hết cho $n$). Khi đó, giá trị cuối cùng của hàm tiền tố phải lớn hơn $n - p$, tức là hậu tố sẽ bao phủ một phần khối đầu tiên. Bây giờ, xét khối thứ hai của chuỗi. Vì tiền tố bằng với hậu tố, và cả tiền tố và hậu tố đều bao phủ khối này và độ dịch chuyển tương đối của chúng $k$ không chia hết cho độ dài khối $p$ (nếu không thì $k$ chia hết cho $n$), thì tất cả các ký tự của khối phải giống nhau. Nhưng khi đó chuỗi chỉ gồm một ký tự lặp đi lặp lại, do đó ta có thể nén nó thành chuỗi có kích thước 1, điều này cho $k = 1$, và $k$ chia hết cho $n$. Mâu thuẫn.

$$\overbrace{s_0\ s_1\ s_2\ s_3}^{p}\ \overbrace{s_4\ s_5\ s_6\ s_7}^{p}$$

$$s_0\ s_1\ s_2\ \underbrace{\overbrace{s_3\ s_4\ s_5\ s_6}^{p}\ s_7}_{\pi[7]=5}$$

$$s_4 = s_3,\ s_5 = s_4,\ s_6 = s_5,\ s_7 = s_6\ \Rightarrow\ s_0 = s_1 = s_2 = s_3$$

### Xây dựng một automaton dựa trên hàm tiền tố

Hãy quay lại việc nối hai chuỗi qua một ký tự phân cách, tức là với các chuỗi $s$ và $t$, ta tính hàm tiền tố cho chuỗi $s + \# + t$. Rõ ràng, vì # là ký tự phân cách, giá trị của hàm tiền tố sẽ không bao giờ vượt quá $|s|$. Do đó, chỉ cần lưu trữ chuỗi $s + \#$ và các giá trị của hàm tiền tố cho nó là đủ, và ta có thể tính hàm tiền tố cho tất cả các ký tự tiếp theo một cách tức thời:

$$\underbrace{s_0\ s_1\ \ldots\ s_{n-1}\ \#}_{\text{cần lưu trữ}}\ \underbrace{t_0\ t_1\ \ldots\ t_{m-1}}_{\text{không cần lưu trữ}}$$

Thật vậy, trong tình huống như vậy, biết ký tự tiếp theo $c \in t$ và giá trị của hàm tiền tố ở vị trí trước đó là đủ thông tin để tính giá trị tiếp theo của hàm tiền tố, mà không cần sử dụng bất kỳ ký tự trước đó của chuỗi $t$ và giá trị của hàm tiền tố trong chúng.

Nói cách khác, ta có thể xây dựng một automaton (máy trạng thái hữu hạn): trạng thái trong đó là giá trị hiện tại của hàm tiền tố, và việc chuyển từ trạng thái này sang trạng thái khác sẽ được thực hiện qua ký tự tiếp theo.

Do đó, ngay cả khi không có chuỗi $t$, ta vẫn có thể xây dựng một bảng chuyển trạng thái $(\text{old}_\pi, c) \to \text{new}_\pi$ bằng cùng thuật toán như khi tính bảng chuyển trạng thái:

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            int j = i;
            while (j > 0 && 'a' + c != s[j])
                j = pi[j-1];
            if ('a' + c == s[j])
                j++;
```

```
            aut[i][c] = j;
        }
    }
}
```

Tuy nhiên, ở dạng này, thuật toán chạy trong thời gian $O(n^2 \cdot 26)$ cho các chữ cái thường của bảng chữ cái. Lưu ý rằng ta có thể áp dụng quy hoạch động và sử dụng các phần đã tính toán của bảng. Bất cứ khi nào ta chuyển từ giá trị $j$ sang giá trị $\pi[j-1]$, ta thực sự ngụ ý rằng chuyển trạng thái $(j, c)$ dẫn đến cùng trạng thái như chuyển trạng thái $(\pi[j-1], c)$, và câu trả lời này đã được tính toán chính xác.

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i-1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

Kết quả là ta xây dựng automaton trong thời gian $O(26 \cdot n)$.

Khi nào thì automaton như vậy hữu ích? Trước tiên, hãy nhớ rằng ta sử dụng hàm tiền tố cho chuỗi $s + \# + t$ và các giá trị của nó chủ yếu cho một mục đích duy nhất: tìm tất cả các lần xuất hiện của chuỗi $s$ trong chuỗi $t$.

Do đó, lợi ích rõ ràng nhất của automaton này là tăng tốc độ tính toán hàm tiền tố cho chuỗi $s + \# + t$. Bằng cách xây dựng automaton cho $s + \#$, ta không còn cần lưu trữ chuỗi $s$ hoặc các giá trị của hàm tiền tố trong đó. Tất cả các chuyển trạng thái đã được tính toán trong bảng.

Nhưng còn có một ứng dụng thứ hai, ít rõ ràng hơn. Ta có thể sử dụng automaton khi chuỗi $t$ là một chuỗi khổng lồ được xây dựng theo một số quy tắc. Điều này có thể là các chuỗi Gray, hoặc một chuỗi được hình thành bằng cách kết hợp đệ quy của một số chuỗi ngắn từ đầu vào.

Để hoàn chỉnh, ta sẽ giải một bài toán như vậy: cho một số $k \leq 10^5$ và một chuỗi $s$ có độ dài $\leq 10^5$. Ta phải tính số lần xuất hiện của $s$ trong chuỗi Gray thứ $k$. Nhớ lại rằng các chuỗi Gray được định nghĩa như sau:

$$g_1 = \texttt{"a"}$$
$$g_2 = \texttt{"aba"}$$
$$g_3 = \texttt{"abacaba"}$$
$$g_4 = \texttt{"abacabadabacaba"}$$

Trong những trường hợp như vậy, ngay cả việc xây dựng chuỗi $t$ cũng là không thể, vì độ dài của nó rất lớn. Chuỗi Gray thứ $k$ có độ dài $2^k - 1$ ký tự. Tuy nhiên, ta có thể tính giá trị của hàm tiền tố ở cuối chuỗi một cách hiệu quả, chỉ bằng cách biết giá trị của hàm tiền tố ở đầu.

Ngoài chính automaton, ta cũng tính các giá trị $G[i][j]$ - giá trị của automaton sau khi xử lý chuỗi $g_i$ bắt đầu từ trạng thái $j$. Và thêm vào đó, ta tính các giá trị $K[i][j]$ - số lần xuất hiện của $s$ trong $g_i$, trước trong quá trình xử lý $g_i$ bắt đầu từ trạng thái $j$. Thực tế, $K[i][j]$ là số lần hàm tiền tố nhận giá trị $|s|$ trong khi thực hiện các thao tác. Đáp án của bài toán sẽ là $K[k][0]$.

Làm thế nào để tính các giá trị này? Trước tiên, các giá trị cơ bản là $G[0][j] = j$ và $K[0][j] = 0$. Và tất cả các giá trị tiếp theo có thể được tính từ các giá trị trước đó và sử dụng automaton. Để tính giá trị cho một $i$ nào đó, ta nhớ rằng chuỗi $g_i$ bao gồm $g_{i-1}$, ký tự thứ $i$ của bảng chữ cái, và $g_{i-1}$. Do đó, automaton sẽ chuyển sang trạng thái:

$$\text{mid} = \text{aut}[G[i-1][j]][i]$$

$$G[i][j] = G[i-1][\text{mid}]$$

Các giá trị cho $K[i][j]$ cũng có thể được đếm dễ dàng.

$$K[i][j] = K[i-1][j] + (\text{mid} == |s|) + K[i-1][\text{mid}]$$

Vậy ta có thể giải bài toán cho các chuỗi Gray, và tương tự cũng giải được rất nhiều bài toán tương tự khác. Ví dụ, phương pháp tương tự cũng giải được bài toán sau: ta được cho một chuỗi $s$ và một số mẫu $t_i$, mỗi mẫu được xác định như sau: đó là một chuỗi các ký tự thông thường, và có thể có một số chèn đệ quy của các chuỗi trước dưới dạng $t_k^{\text{cnt}}$, nghĩa là tại vị trí này ta phải chèn chuỗi $t_k$ cnt lần. Một ví dụ về các mẫu như vậy:

$$t_1 = \texttt{"abdeca"}$$
$$t_2 = \texttt{"abc"} \circ t_1^{30} \circ \texttt{"abd"}$$
$$t_3 = t_2^{50} \circ t_1^{100}$$
$$t_4 = t_2^{10} \circ t_3^{100}$$

Các phép thay thế đệ quy làm phình chuỗi lên, đến mức độ dài của chúng có thể đạt tới $100^{100}$.

Ta phải tìm số lần chuỗi $s$ xuất hiện trong mỗi chuỗi.

Bài toán có thể được giải theo cách tương tự bằng cách xây dựng automaton của hàm tiền tố, sau đó ta tính các chuyển trạng thái cho mỗi mẫu bằng cách sử dụng các kết quả trước đó.

## 2.4 Hàm Z và cách tính toán

Giả sử ta được cho một chuỗi $s$ có độ dài $n$. Hàm Z cho chuỗi này là một mảng có độ dài $n$ trong đó phần tử thứ $i$ bằng với số lượng ký tự lớn nhất bắt đầu từ vị trí $i$ mà trùng với các ký tự đầu tiên của $s$.

Nói cách khác, $z[i]$ là độ dài của chuỗi dài nhất vừa là tiền tố của $s$ vừa là tiền tố của hậu tố của $s$ bắt đầu tại $i$.

**Lưu ý:** Trong bài viết này, để tránh nhầm lẫn, ta giả sử các chỉ số bắt đầu từ 0; nghĩa là: ký tự đầu tiên của $s$ có chỉ số 0 và ký tự cuối cùng có chỉ số $n-1$.

Phần tử đầu tiên của hàm Z, $z[0]$, thường không được định nghĩa rõ ràng. Trong bài viết này, ta sẽ giả sử nó bằng 0 (dù điều này không thay đổi gì trong việc triển khai thuật toán).

Bài viết này trình bày một thuật toán để tính hàm Z trong thời gian $O(n)$, cũng như các ứng dụng khác nhau của nó.

### 2.4.1 Ví dụ

Ví dụ, dưới đây là các giá trị của hàm Z được tính cho các chuỗi khác nhau:

- "aaaaa" - $[0, 4, 3, 2, 1]$ - "aaabaab" - $[0, 2, 1, 0, 2, 1, 0]$ - "abacaba" - $[0, 0, 1, 0, 3, 0, 1]$

### 2.4.2 Thuật toán đơn giản

Định nghĩa chính thức có thể được biểu diễn trong triển khai đơn giản $O(n^2)$ sau:

```
vector<int> z_function_trivial(string s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1; i < n; i++) {
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
    }
    return z;
}
```

Ta chỉ cần duyệt qua từng vị trí $i$ và cập nhật $z[i]$ cho mỗi vị trí, bắt đầu từ $z[i] = 0$ và tăng dần miễn là không tìm thấy sự không khớp (và miễn là chưa đến cuối chuỗi).

Tất nhiên, đây không phải là một triển khai hiệu quả. Bây giờ ta sẽ trình bày việc xây dựng một triển khai hiệu quả.

### 2.4.3 Thuật toán hiệu quả để tính hàm Z

Để có một thuật toán hiệu quả, ta sẽ tính các giá trị $z[i]$ lần lượt từ $i = 1$ đến $n - 1$, nhưng đồng thời, khi tính một giá trị mới, ta sẽ cố gắng tận dụng tối đa các giá trị đã tính trước đó.

Để ngắn gọn, ta gọi các đoạn khớp là những chuỗi con trùng với một tiền tố của $s$. Ví dụ, giá trị mong muốn của hàm Z $z[i]$ là độ dài của đoạn khớp bắt đầu tại vị trí $i$ (và kết thúc tại vị trí $i + z[i] - 1$).

Để làm điều này, ta sẽ giữ các chỉ số $[l, r)$ của đoạn khớp cực phải. Nghĩa là, trong tất cả các đoạn đã phát hiện, ta sẽ giữ đoạn kết thúc ở vị trí cực phải. Theo một cách nào đó, chỉ số $r$ có thể được xem như "ranh giới" mà thuật toán đã quét qua chuỗi $s$; mọi thứ vượt quá điểm đó vẫn chưa được biết.

Sau đó, nếu chỉ số hiện tại (mà ta phải tính giá trị tiếp theo của hàm Z) là $i$, ta có hai lựa chọn:

- $i \geq r$ – vị trí hiện tại nằm ngoài những gì ta đã xử lý.

Ta sẽ tính $z[i]$ bằng thuật toán đơn giản (tức là so sánh từng giá trị một). Lưu ý rằng cuối cùng, nếu $z[i] > 0$, ta sẽ phải cập nhật các chỉ số của đoạn khớp cực phải, vì chắc chắn rằng $r = i + z[i]$ mới sẽ tốt hơn $r$ trước đó.

- $i < r$ – vị trí hiện tại nằm trong đoạn khớp hiện tại $[l, r)$.

Khi đó, ta có thể sử dụng các giá trị Z đã tính để "khởi tạo" giá trị của $z[i]$ thành một giá trị nào đó (chắc chắn tốt hơn "bắt đầu từ zero"), thậm chí là một số lớn.

Để làm điều này, ta quan sát rằng các chuỗi con $s[l \dots r)$ và $s[0 \dots r - l)$ khớp nhau. Điều này có nghĩa là để ước lượng ban đầu cho $z[i]$, ta có thể lấy giá trị đã tính cho đoạn tương ứng $s[0 \dots r - l)$, tức là $z[i - l]$.

Tuy nhiên, giá trị $z[i - l]$ có thể quá lớn: khi áp dụng cho vị trí $i$, nó có thể vượt quá chỉ số $r$. Điều này không được phép vì ta không biết gì về các ký tự bên phải của $r$: chúng có thể khác với những ký tự cần thiết.

Dưới đây là một ví dụ về tình huống tương tự:

$$s = "aaaabaa"$$

Khi ta đến vị trí cuối cùng ($i = 6$), đoạn khớp hiện tại sẽ là $[5, 7)$. Vị trí 6 sẽ khớp với vị trí $6 - 5 = 1$, tại đó giá trị của hàm Z là $z[1] = 3$. Rõ ràng, ta không thể khởi tạo $z[6]$ thành 3, điều đó hoàn toàn sai. Giá trị tối đa ta có thể khởi tạo là 1 – vì đó là giá trị lớn nhất không đưa ta vượt quá chỉ số $r$ của đoạn khớp $[l, r)$.

Do đó, như một ước lượng ban đầu cho $z[i]$, ta có thể an toàn lấy:

$$z_0[i] = \min(r - i,\ z[i - l])$$

Sau khi có $z[i]$ được khởi tạo thành $z_0[i]$, ta cố gắng tăng $z[i]$ bằng cách chạy thuật toán đơn giản – vì nói chung, sau ranh giới $r$, ta không thể biết liệu đoạn có tiếp tục khớp hay không.

Do đó, toàn bộ thuật toán được chia thành hai trường hợp, chỉ khác nhau ở giá trị ban đầu của $z[i]$: trong trường hợp đầu tiên, nó được giả định là zero, trong trường hợp thứ hai, nó được xác định bởi các giá trị đã tính trước (sử dụng công thức trên). Sau đó, cả hai nhánh của thuật toán này đều có thể được rút gọn thành triển khai của thuật toán đơn giản, bắt đầu ngay sau khi ta xác định giá trị ban đầu.

Thuật toán hóa ra rất đơn giản. Mặc dù trong mỗi lần lặp, thuật toán đơn giản được chạy, ta đã đạt được tiến bộ đáng kể, có một thuật toán chạy trong thời gian tuyến tính. Sau này, ta sẽ chứng minh rằng thời gian chạy là tuyến tính.

### 2.4.4 Triển khai

Triển khai hóa ra khá ngắn gọn:

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

### 2.4.5 Bình luận về triển khai này

Giải pháp hoàn chỉnh được cung cấp dưới dạng một hàm trả về một mảng có độ dài $n$ – hàm Z của $s$.

Mảng $z$ ban đầu được điền bằng các số không. Đoạn khớp cực phải hiện tại được giả định là $[0; 0)$ (tức là một đoạn nhỏ cố ý không chứa bất kỳ $i$ nào).

Bên trong vòng lặp cho $i = 1 \ldots n-1$, ta trước tiên xác định giá trị ban đầu $z[i]$ – nó sẽ hoặc giữ nguyên là zero hoặc được tính bằng công thức trên.

Sau đó, thuật toán đơn giản cố gắng tăng giá trị của $z[i]$ càng nhiều càng tốt.

Cuối cùng, nếu cần thiết (tức là nếu $i + z[i] > r$), ta cập nhật đoạn khớp cực phải $[l, r)$.

### 2.4.6 Hành vi tiệm cận của thuật toán

Ta sẽ chứng minh rằng thuật toán trên có thời gian chạy tuyến tính với độ dài của chuỗi – tức là $O(n)$.

Bằng chứng rất đơn giản.

Ta quan tâm đến vòng lặp while lồng nhau, vì mọi thứ khác chỉ là một loạt các phép toán không đổi tổng cộng thành $O(n)$.

Ta sẽ chỉ ra rằng mỗi lần lặp của vòng lặp while sẽ làm tăng ranh giới bên phải $r$ của đoạn khớp.

Để làm điều đó, ta sẽ xem xét cả hai nhánh của thuật toán:

- $i \geq r$

Trong trường hợp này, hoặc vòng lặp while sẽ không thực hiện bất kỳ lần lặp nào (nếu $s[0] \neq s[i]$), hoặc nó sẽ thực hiện một vài lần lặp, bắt đầu tại vị trí $i$, mỗi lần di chuyển một ký tự sang phải. Sau đó, ranh giới bên phải $r$ chắc chắn sẽ được cập nhật.

Vậy ta đã thấy rằng, khi $i \geq r$, mỗi lần lặp của vòng lặp while làm tăng giá trị của chỉ số $r$ mới.

- $i < r$

Trong trường hợp này, ta khởi tạo $z[i]$ thành một giá trị nào đó $z_0$ theo công thức trên. Hãy so sánh giá trị ban đầu $z_0$ này với giá trị $r - i$. Ta sẽ có ba trường hợp:

- $z_0 < r - i$

Ta chứng minh rằng trong trường hợp này, không có lần lặp nào của vòng lặp while sẽ xảy ra.

Điều này dễ chứng minh, ví dụ, bằng phản chứng: nếu vòng lặp while thực hiện ít nhất một lần lặp, điều đó có nghĩa là ước lượng ban đầu $z[i] = z_0$ không chính xác (nhỏ hơn độ dài thực tế của đoạn khớp). Nhưng vì $s[l \ldots r)$ và $s[0 \ldots r - l)$ giống nhau, điều này sẽ ngụ ý rằng $z[i - l]$ chứa giá trị sai (nhỏ hơn giá trị cần có).

Do đó, vì $z[i - l]$ là đúng và nhỏ hơn $r - i$, điều đó có nghĩa là giá trị này trùng với giá trị cần thiết $z[i]$.

- $z_0 = r - i$

Trong trường hợp này, vòng lặp while có thể thực hiện một vài lần lặp, nhưng mỗi lần trong số đó sẽ dẫn đến việc tăng giá trị của chỉ số $r$ vì ta sẽ bắt đầu so sánh từ $s[r]$, điều này sẽ vượt ra ngoài khoảng $[l, r)$.

- $z_0 > r - i$

Tùy chọn này là không thể, theo định nghĩa của $z_0$.

Vậy, ta đã chứng minh rằng mỗi lần lặp của vòng lặp bên trong làm cho con trỏ $r$ tiến về bên phải. Vì $r$ không thể lớn hơn $n-1$, điều này có nghĩa là vòng lặp bên trong sẽ không thực hiện quá $n-1$ lần lặp.

Vì phần còn lại của thuật toán rõ ràng hoạt động trong $O(n)$, ta đã chứng minh rằng toàn bộ thuật toán tính hàm Z chạy trong thời gian tuyến tính.

### 2.4.7 Ứng dụng

Bây giờ ta sẽ xem xét một số ứng dụng của hàm Z cho các bài toán cụ thể.

Các ứng dụng này sẽ phần lớn tương tự như ứng dụng của hàm tiền tố.

#### Tìm kiếm chuỗi con

Để tránh nhầm lẫn, ta gọi $t$ là chuỗi văn bản, và $p$ là mẫu. Bài toán là: tìm tất cả các lần xuất hiện của mẫu $p$ trong văn bản $t$.

Để giải bài toán này, ta tạo một chuỗi mới $s = p + \diamond + t$, tức là ta nối chuỗi $p$ và $t$ nhưng cũng đặt một ký tự phân cách $\diamond$ ở giữa (ta sẽ chọn $\diamond$ sao cho chắc chắn nó không xuất hiện ở bất kỳ đâu trong các chuỗi $p$ hoặc $t$).

Tính hàm Z cho $s$. Sau đó, với bất kỳ $i$ nào trong khoảng $[0; \text{length}(t) - 1]$, ta sẽ xem xét giá trị tương ứng $k = z[i + \text{length}(p) + 1]$. Nếu $k$ bằng $\text{length}(p)$ thì ta biết có một lần xuất hiện của $p$ tại vị trí $i$ của $t$, nếu không thì không có lần xuất hiện của $p$ tại vị trí $i$ của $t$.

Thời gian chạy (và mức tiêu thụ bộ nhớ) là $O(\text{length}(t) + \text{length}(p))$.

#### Số chuỗi con khác nhau trong một chuỗi

Cho một chuỗi $s$ có độ dài $n$, đếm số chuỗi con khác nhau của $s$.

Ta sẽ giải bài toán này một cách lặp. Nghĩa là: biết số chuỗi con khác nhau hiện tại, tính lại số lượng này sau khi thêm một ký tự vào cuối $s$.

Giả sử $k$ là số chuỗi con khác nhau hiện tại của $s$. Ta thêm một ký tự mới $c$ vào cuối $s$. Rõ ràng, có thể có một số chuỗi con mới kết thúc bằng ký tự mới này $c$ (cụ thể là tất cả những chuỗi kết thúc bằng ký tự này mà ta chưa gặp trước đó).

Lấy chuỗi $t = s + c$ và đảo ngược nó (viết các ký tự của nó theo thứ tự ngược lại). Nhiệm vụ của ta bây giờ là đếm số tiền tố của $t$ không xuất hiện ở bất kỳ nơi nào khác trong $t$. Hãy tính hàm Z của $t$ và tìm giá trị tối đa của nó $z_{\max}$. Rõ ràng, tiền tố của $t$ có độ dài $z_{\max}$ cũng xuất hiện đâu đó ở giữa $t$. Rõ ràng, các tiền tố ngắn hơn cũng xuất hiện.

Vậy, ta đã thấy rằng số chuỗi con mới xuất hiện khi ký tự $c$ được thêm vào $s$ bằng $\text{length}(t) - z_{\max}$.

Do đó, thời gian chạy của giải pháp này là $O(n^2)$ cho một chuỗi có độ dài $n$.

Đáng chú ý là theo cách hoàn toàn tương tự, ta có thể tính lại, vẫn trong thời gian $O(n)$, số chuỗi con khác nhau khi thêm một ký tự vào đầu chuỗi, cũng như khi xóa nó (từ cuối hoặc đầu).

**Nén chuỗi**

Cho một chuỗi $s$ có độ dài $n$. Tìm biểu diễn "nén" ngắn nhất của nó, tức là: tìm một chuỗi $t$ có độ dài ngắn nhất sao cho $s$ có thể được biểu diễn dưới dạng nối của một hoặc nhiều bản sao của $t$.

Một giải pháp là: tính hàm Z của $s$, duyệt qua tất cả $i$ sao cho $i$ chia hết cho $n$. Dừng lại tại $i$ đầu tiên sao cho $i + z[i] = n$. Khi đó, chuỗi $s$ có thể được nén thành độ dài $i$.

Bằng chứng cho sự thật này giống với giải pháp sử dụng hàm tiền tố.

## 2.5 Mảng hậu tố

### 2.5.1 Định nghĩa

Cho $s$ là một chuỗi có độ dài $n$. Hậu tố thứ $i$ của $s$ là chuỗi con $s[i \dots n-1]$.

Mảng hậu tố sẽ chứa các số nguyên biểu thị các chỉ số bắt đầu của tất cả các hậu tố của một chuỗi đã cho, sau khi các hậu tố này được sắp xếp.

Ví dụ, xem chuỗi $s = abaab$. Tất cả các hậu tố bao gồm như sau:

0. *abaab*
1. *baab*
2. *aab*
3. *ab*
4. *b*

Sau khi sắp xếp các chuỗi này:

2. *aab*
3. *ab*
0. *abaab*
4. *b*
1. *baab*

Do đó, mảng hậu tố cho $s$ sẽ là (2, 3, 0, 4, 1).

Là một cấu trúc dữ liệu, nó được sử dụng rộng rãi trong các lĩnh vực như nén dữ liệu, tin sinh học và nói chung, trong bất kỳ lĩnh vực nào liên quan đến chuỗi và các bài toán khớp chuỗi.

### 2.5.2 Xây dựng

**Phương pháp** $O(n^2 \log n)$

Đây là phương pháp ngây thơ nhất. Lấy tất cả các hậu tố và sắp xếp chúng bằng cách sử dụng quicksort hoặc mergesort và đồng thời giữ lại các chỉ số ban đầu của chúng. Việc sắp xếp sử dụng $O(n \log n)$ phép so sánh, và vì việc so sánh hai chuỗi sẽ mất thêm thời gian $O(n)$, ta có độ phức tạp cuối cùng là $O(n^2 \log n)$.

**Phương pháp** $O(n \log n)$

Nói một cách chặt chẽ, thuật toán sau đây sẽ không sắp xếp các hậu tố, mà là các dịch vòng của một chuỗi. Tuy nhiên, ta có thể dễ dàng suy ra một thuật toán để sắp xếp hậu tố từ nó: chỉ cần thêm một ký tự bất kỳ vào cuối chuỗi mà nhỏ hơn bất kỳ ký tự nào trong chuỗi. Thường sử dụng ký hiệu \$. Khi đó, thứ tự của các dịch vòng đã sắp xếp sẽ tương đương với thứ tự của các hậu tố đã sắp xếp, như được thể hiện ở đây với chuỗi *dabbb*:

| | | |
|---|---|---|
| 1. | *abbb\$d* | *abbb* |
| 4. | *b\$dabb* | *b* |
| 3. | *bb\$dab* | *bb* |
| 2. | *bbb\$da* | *bbb* |
| 0. | *dabbb\$* | *dabbb* |

Vì ta sẽ sắp xếp các dịch vòng, ta sẽ xem xét các chuỗi con vòng. Ta sẽ sử dụng ký hiệu $s[i \dots j]$ cho chuỗi con của $s$ ngay cả khi $i > j$. Trong trường hợp này, ta thực sự muốn nói đến chuỗi $s[i \dots n-1] + s[0 \dots j]$. Ngoài ra, ta sẽ lấy tất cả các chỉ số theo modulo độ dài của $s$, và bỏ qua phép toán modulo để đơn giản hóa.

Thuật toán mà ta thảo luận sẽ thực hiện $\lceil \log n \rceil + 1$ lần lặp. Trong lần lặp thứ $k$ (với $k = 0 \dots \lceil \log n \rceil$), ta sắp xếp $n$ chuỗi con vòng của $s$ có độ dài $2^k$. Sau lần lặp thứ $\lceil \log n \rceil$, các chuỗi con có độ dài $2^{\lceil \log n \rceil} \geq n$ sẽ được sắp xếp, do đó điều này tương đương với việc sắp xếp toàn bộ các dịch vòng.

Trong mỗi lần lặp của thuật toán, ngoài hoán vị $p[0 \dots n-1]$, trong đó $p[i]$ là chỉ số của chuỗi con thứ $i$ (bắt đầu tại $i$ và có độ dài $2^k$) theo thứ tự đã sắp xếp, ta cũng sẽ duy trì một mảng $c[0 \dots n-1]$, trong đó $c[i]$ tương ứng với lớp tương đương mà chuỗi con thuộc về. Vì một số chuỗi con sẽ giống nhau, và thuật toán cần xử lý chúng như nhau. Để tiện lợi, các lớp sẽ được đánh nhãn bằng số bắt đầu từ zero. Ngoài ra, các số $c[i]$ sẽ được gán sao cho chúng giữ thông tin về thứ tự: nếu một chuỗi con nhỏ hơn chuỗi con khác, thì nó cũng nên có nhãn lớp nhỏ hơn. Số lượng lớp tương đương sẽ được lưu trong một biến classes.

Hãy xem một ví dụ. Xét chuỗi $s = aaba$. Các chuỗi con vòng và các mảng tương ứng $p[]$ và $c[]$ được đưa ra cho mỗi lần lặp:

| | | | |
|---|---|---|---|
| $0:$ | $(a,\ a,\ b,\ a)$ | $p = (0,\ 1,\ 3,\ 2)$ | $c = (0,\ 0,\ 1,\ 0)$ |
| $1:$ | $(aa,\ ab,\ ba,\ aa)$ | $p = (0,\ 3,\ 1,\ 2)$ | $c = (0,\ 1,\ 2,\ 0)$ |
| $2:$ | $(aaba,\ abaa,\ baaa,\ aaab)$ | $p = (3,\ 0,\ 1,\ 2)$ | $c = (1,\ 2,\ 3,\ 0)$ |

Đáng chú ý là các giá trị của $p[]$ có thể khác nhau. Ví dụ, trong lần lặp thứ 0, mảng cũng có thể là $p = (3,\ 1,\ 0,\ 2)$ hoặc $p = (3,\ 0,\ 1,\ 2)$. Tất cả các tùy chọn này đều hoán vị các chuỗi con thành thứ tự đã sắp xếp. Vì vậy, chúng đều hợp lệ. Đồng thời, mảng $c[]$ là cố định, không thể có sự mơ hồ.

Bây giờ hãy tập trung vào việc triển khai thuật toán. Ta sẽ viết một hàm nhận một chuỗi $s$ và trả về các hoán vị của các dịch vòng đã sắp xếp:

```
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;
```

Ở bước đầu tiên (trong lần lặp thứ 0), ta phải sắp xếp các chuỗi con vòng có độ dài 1, tức là ta phải sắp xếp tất cả các ký tự của chuỗi và chia chúng thành các lớp tương đương (các ký tự giống nhau được gán vào cùng một lớp). Điều này có thể được thực hiện một cách đơn giản, ví dụ, bằng cách sử dụng sắp xếp đếm. Với mỗi ký tự, ta đếm xem nó xuất hiện bao nhiêu lần trong chuỗi, sau đó sử dụng thông tin này để tạo mảng $p[]$. Sau đó, ta duyệt qua mảng $p[]$ và xây dựng $c[]$ bằng cách so sánh các ký tự liền kề:

```
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i-1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i-1]])
            classes++;
        c[p[i]] = classes - 1;
    }
```

Bây giờ ta phải nói về bước lặp. Giả sử ta đã thực hiện bước thứ $k-1$ và tính được các giá trị của mảng $p[]$ và $c[]$ cho nó. Ta muốn tính các giá trị cho bước thứ $k$ trong thời gian $O(n)$. Vì ta thực hiện bước này $O(\log n)$ lần, thuật toán hoàn chỉnh sẽ có độ phức tạp thời gian là $O(n \log n)$.

Để làm điều này, lưu ý rằng các chuỗi con vòng có độ dài $2^k$ bao gồm hai chuỗi con có độ dài $2^{k-1}$ mà ta có thể so sánh với nhau trong $O(1)$ bằng cách sử dụng thông tin từ giai đoạn trước - các giá trị của lớp tương đương $c[]$. Do đó, đối với hai chuỗi con có độ dài $2^k$ bắt đầu tại vị trí $i$ và $j$, tất cả thông tin cần thiết để so sánh chúng được chứa trong các cặp $(c[i],\ c[i+2^{k-1}])$ và $(c[j],\ c[j+2^{k-1}])$:

$$\cdots \overbrace{\underbrace{s_i \ldots s_{i+2^{k-1}-1}}_{\text{độ dài}=2^{k-1},\ \text{lớp}=c[i]} \quad \underbrace{s_{i+2^{k-1}} \ldots s_{i+2^k-1}}_{\text{độ dài}=2^{k-1},\ \text{lớp}=c[i+2^{k-1}]}}^{\text{độ dài}=2^k} \cdots \overbrace{\underbrace{s_j \ldots s_{j+2^{k-1}-1}}_{\text{độ dài}=2^{k-1},\ \text{lớp}=c[j]} \quad \underbrace{s_{j+2^{k-1}} \ldots s_{j+2^k-1}}_{\text{độ dài}=2^{k-1},\ \text{lớp}=c[j+2^{k-1}]}}^{\text{độ dài}=2^k} \cdots$$

Điều này cho ta một giải pháp rất đơn giản: sắp xếp các chuỗi con có độ dài $2^k$ theo các cặp số này. Điều này sẽ cho ta thứ tự cần thiết $p[]$. Tuy nhiên, một phép sắp xếp thông thường chạy trong thời gian $O(n \log n)$, điều mà ta không hài lòng. Điều này sẽ chỉ cho ta một thuật toán để xây dựng mảng hậu tố trong thời gian $O(n \log^2 n)$.

Làm thế nào để thực hiện nhanh chóng việc sắp xếp các cặp như vậy? Vì các phần tử của cặp không vượt quá $n$, ta có thể sử dụng lại sắp xếp đếm. Tuy nhiên, sắp xếp cặp bằng sắp xếp đếm không phải là hiệu quả nhất. Để đạt được một hằng số ẩn tốt hơn trong độ phức tạp, ta sẽ sử dụng một mẹo khác.

Ta sử dụng kỹ thuật mà sắp xếp radix dựa vào: để sắp xếp các cặp, ta trước tiên sắp xếp chúng theo phần tử thứ hai, sau đó theo phần tử thứ nhất (với một sắp xếp ổn định, tức là sắp xếp mà không làm phá vỡ thứ tự tương đối của các phần tử bằng nhau). Tuy nhiên, các phần tử thứ hai đã được sắp xếp trong lần lặp trước. Do đó, để sắp xếp các cặp theo phần tử thứ hai, ta chỉ cần trừ $2^{k-1}$ từ các chỉ số trong $p[]$ (ví dụ, nếu chuỗi con nhỏ nhất có độ dài $2^{k-1}$ bắt đầu tại vị trí $i$, thì chuỗi con có độ dài $2^k$ với nửa sau nhỏ nhất bắt đầu tại $i - 2^{k-1}$).

Vậy chỉ bằng các phép trừ đơn giản, ta có thể sắp xếp các phần tử thứ hai của các cặp trong $p[]$. Bây giờ ta cần thực hiện một sắp xếp ổn định theo các phần tử thứ nhất. Như đã đề cập, điều này có thể được thực hiện với sắp xếp đếm.

Điều duy nhất còn lại là tính các lớp tương đương $c[]$, nhưng như trước đây, điều này có thể được thực hiện bằng cách chỉ cần lặp qua hoán vị đã sắp xếp $p[]$ và so sánh các cặp lân cận.

Dưới đây là phần triển khai còn lại. Ta sử dụng các mảng tạm thời $pn[]$ và $cn[]$ để lưu hoán vị theo phần tử thứ hai và các chỉ số lớp tương đương mới:

```
vector<int> pn(n), cn(n);
for (int h = 0; (1 << h) < n; ++h) {
    for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0)
            pn[i] += n;
```

```
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i-1];
        for (int i = n-1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
            pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}
```

Thuật toán yêu cầu thời gian $O(n \log n)$ và bộ nhớ $O(n)$. Để đơn giản, ta đã sử dụng toàn bộ phạm vi ASCII làm bảng chữ cái. Nếu biết rằng chuỗi chỉ chứa một tập hợp con của các ký tự, ví dụ chỉ có chữ cái thường, thì triển khai có thể được tối ưu hóa, nhưng yếu tố tối ưu hóa có thể không đáng kể, vì kích thước của bảng chữ cái chỉ quan trọng trong lần lặp đầu tiên. Mọi lần lặp khác phụ thuộc vào số lượng lớp tương đương, mà có thể nhanh chóng đạt $O(n)$ ngay cả khi ban đầu là một chuỗi trên bảng chữ cái có kích thước 2.

Cũng lưu ý rằng thuật toán này chỉ sắp xếp các dịch vòng. Như đã đề cập ở đầu phần này, ta có thể tạo ra thứ tự đã sắp xếp của các hậu tố bằng cách thêm một ký tự nhỏ hơn tất cả các ký tự khác của chuỗi, và sắp xếp chuỗi kết quả này theo dịch vòng, ví dụ, bằng cách sắp xếp các dịch vòng của $s + \$$. Điều này rõ ràng sẽ cho mảng hậu tố của $s$, tuy nhiên được thêm vào trước $|s|$:

```
vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}
```

### 2.5.3 Ứng dụng

#### Tìm dịch vòng nhỏ nhất

Thuật toán trên sắp xếp tất cả các dịch vòng (mà không thêm ký tự vào chuỗi), và do đó $p[0]$ cho vị trí của dịch vòng nhỏ nhất.

#### Tìm chuỗi con trong một chuỗi

Nhiệm vụ là tìm một chuỗi $s$ bên trong một văn bản $t$ trực tuyến - ta biết trước văn bản $t$, nhưng không biết chuỗi $s$. Ta có thể tạo mảng hậu tố cho văn bản $t$ trong thời gian $O(|t| \log |t|)$. Bây giờ ta có thể tìm kiếm chuỗi con $s$ theo cách sau. Sự xuất hiện của $s$ phải là tiền tố của một số hậu tố từ $t$. Vì ta đã sắp xếp tất cả các hậu tố, ta có thể thực hiện tìm kiếm nhị phân cho $s$ trong $p$. Việc so sánh hậu tố hiện tại và chuỗi con $s$ trong tìm kiếm nhị phân có thể được thực hiện trong thời gian $O(|s|)$, do đó độ phức tạp để tìm chuỗi con là $O(|s| \log |t|)$. Cũng lưu ý rằng nếu chuỗi con xuất hiện nhiều lần trong $t$, thì tất cả các lần xuất hiện sẽ nằm cạnh nhau trong $p$. Do đó, số lần xuất hiện có thể được tìm thấy với một tìm kiếm nhị phân thứ hai, và tất cả các lần xuất hiện có thể được in ra dễ dàng.

#### So sánh hai chuỗi con của một chuỗi

Ta muốn có thể so sánh hai chuỗi con có cùng độ dài của một chuỗi đã cho $s$ trong thời gian $O(1)$, tức là kiểm tra xem chuỗi con đầu tiên có nhỏ hơn chuỗi con thứ hai hay không.

Để làm điều này, ta xây dựng mảng hậu tố trong thời gian $O(|s| \log |s|)$ và lưu trữ tất cả các kết quả trung gian của các lớp tương đương $c[]$.

Sử dụng thông tin này, ta có thể so sánh bất kỳ hai chuỗi con nào có độ dài là lũy thừa của hai trong $O(1)$: để làm điều này, chỉ cần so sánh các lớp tương đương của cả hai chuỗi con. Bây giờ ta muốn khái quát hóa phương pháp này cho các chuỗi con có độ dài bất kỳ.

Hãy so sánh hai chuỗi con có độ dài $l$ với các chỉ số bắt đầu $i$ và $j$. Ta tìm độ dài lớn nhất của một khối nằm trong chuỗi con có độ dài này: $k$ lớn nhất sao cho $2^k \leq l$. Khi đó, việc so sánh hai chuỗi con có thể được thay thế bằng việc so sánh hai khối chồng lấp có độ dài $2^k$: đầu tiên ta cần so sánh hai khối bắt đầu tại $i$ và $j$, và nếu chúng bằng nhau thì so sánh hai khối kết thúc tại các vị trí $i + l - 1$ và $j + l - 1$:

$$\ldots \overbrace{\underbrace{s_i \ldots s_{i+l-2^k} \ldots s_{i+2^k-1}}_{2^k} \ldots s_{i+l-1}}^{\text{thứ nhất}} \ldots \overbrace{\underbrace{s_j \ldots s_{j+l-2^k} \ldots s_{j+2^k-1}}_{2^k} \ldots s_{j+l-1}}^{\text{thứ hai}} \ldots$$

$$\ldots \overbrace{s_i \ldots \underbrace{s_{i+l-2^k} \ldots s_{i+2^k-1} \ldots s_{i+l-1}}_{2^k}}^{\text{thứ nhất}} \ldots \overbrace{s_j \ldots \underbrace{s_{j+l-2^k} \ldots s_{j+2^k-1} \ldots s_{j+l-1}}_{2^k}}^{\text{thứ hai}} \ldots$$

Dưới đây là triển khai của việc so sánh. Lưu ý rằng giả định rằng hàm được gọi với $k$ đã được tính trước. $k$ có thể được tính bằng $\lfloor \log l \rfloor$, nhưng hiệu quả hơn là tính trước tất cả các giá trị $k$ cho mỗi $l$. Xem ví dụ bài viết về Bảng Thưa, sử dụng ý tưởng tương tự và tính tất cả các giá trị log:

```
int compare(int i, int j, int l, int k) {
    pair<int, int> a = {c[k][i], c[k][(i+l-(1 << k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+l-(1 << k))%n]};
    return a == b ? 0 : a < b ? -1 : 1;
}
```

### Tiền tố chung dài nhất của hai chuỗi con với bộ nhớ bổ sung

Với một chuỗi đã cho $s$, ta muốn tính tiền tố chung dài nhất (LCP) của hai hậu tố bất kỳ với vị trí $i$ và $j$.

Phương pháp được mô tả ở đây sử dụng bộ nhớ bổ sung $O(|s| \log |s|)$. Một cách tiếp cận hoàn toàn khác chỉ sử dụng lượng bộ nhớ tuyến tính sẽ được mô tả trong phần tiếp theo.

Ta xây dựng mảng hậu tố trong thời gian $O(|s| \log |s|)$, và ghi nhớ các kết quả trung gian của các mảng $c[]$ từ mỗi lần lặp.

Hãy tính LCP cho hai hậu tố bắt đầu tại $i$ và $j$. Ta có thể so sánh bất kỳ hai chuỗi con nào có độ dài là lũy thừa của hai trong $O(1)$. Để làm điều này, ta so sánh các chuỗi theo lũy thừa của hai (từ cao nhất đến thấp nhất) và nếu các chuỗi con của độ dài này giống nhau, thì ta cộng độ dài bằng nhau vào đáp án và tiếp tục kiểm tra LCP ở bên phải của phần bằng nhau, tức là $i$ và $j$ được cộng thêm lũy thừa của hai hiện tại:

```
int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
    return ans;
}
```

Ở đây, log_n biểu thị một hằng số bằng logarit của $n$ cơ số 2 làm tròn xuống.

### Tiền tố chung dài nhất của hai chuỗi con không dùng bộ nhớ bổ sung

Ta có cùng nhiệm vụ như trong phần trước. Ta phải tính tiền tố chung dài nhất (LCP) cho hai hậu tố của một chuỗi $s$.

Không giống như phương pháp trước, phương pháp này chỉ sử dụng bộ nhớ $O(|s|)$. Kết quả của việc tiền xử lý sẽ là một mảng (mà bản thân nó là một nguồn thông tin quan trọng về chuỗi, và do đó cũng được sử dụng để giải các bài toán khác). Các truy vấn LCP có thể được trả lời bằng cách thực hiện các truy vấn RMQ (truy vấn cực tiểu trong khoảng) trong mảng này, vì vậy với các triển khai khác nhau, có thể đạt được thời gian truy vấn logarit hoặc thậm chí không đổi.

Cơ sở cho thuật toán này là ý tưởng sau: ta sẽ tính tiền tố chung dài nhất cho mỗi cặp hậu tố liền kề theo thứ tự đã sắp xếp. Nói cách khác, ta xây dựng một mảng $\text{lcp}[0 \dots n-2]$, trong đó $\text{lcp}[i]$ bằng độ dài của tiền tố chung dài nhất của các hậu tố bắt đầu tại $p[i]$ và $p[i+1]$. Mảng này sẽ cho ta câu trả lời cho bất kỳ hai hậu tố liền kề nào của chuỗi. Sau đó, câu trả lời cho hai hậu tố bất kỳ, không nhất thiết phải liền kề nhau, có thể được lấy từ mảng này. Thực tế, giả sử yêu cầu là tính LCP của các hậu tố $p[i]$ và $p[j]$. Khi đó câu trả lời cho truy vấn này sẽ là $\min(\text{lcp}[i],\ \text{lcp}[i+1],\ \dots,\ \text{lcp}[j-1])$.

Vậy nếu ta có mảng lcp như vậy, thì bài toán được rút gọn thành RMQ, vốn có nhiều giải pháp khác nhau với các độ phức tạp khác nhau.

Vì vậy, nhiệm vụ chính là xây dựng mảng lcp này. Ta sẽ sử dụng thuật toán của Kasai, có thể tính mảng này trong thời gian $O(n)$.

Hãy xem xét hai hậu tố liền kề trong thứ tự đã sắp xếp (thứ tự của mảng hậu tố). Giả sử vị trí bắt đầu của chúng là $i$ và $j$ và lcp của chúng bằng $k > 0$. Nếu ta loại bỏ chữ cái đầu tiên của cả hai hậu tố - tức là ta lấy các hậu tố $i+1$ và $j+1$ - thì rõ ràng lcp của hai hậu tố này là $k-1$. Tuy nhiên, ta không thể sử dụng giá trị này và ghi nó vào mảng lcp, vì hai hậu tố này có thể không liền kề nhau trong thứ tự đã sắp xếp. Hậu tố $i+1$ chắc chắn sẽ nhỏ hơn hậu tố $j+1$, nhưng có thể có một số hậu tố ở giữa chúng. Tuy nhiên, vì ta biết rằng LCP giữa hai hậu tố là giá trị nhỏ nhất của tất cả các chuyển tiếp, ta cũng biết rằng LCP giữa bất kỳ hai cặp nào trong khoảng đó phải ít nhất là $k-1$, đặc biệt là giữa $i+1$ và hậu tố tiếp theo. Và có thể nó còn lớn hơn.

Bây giờ ta đã có thể triển khai thuật toán. Ta sẽ lặp qua các hậu tố theo thứ tự độ dài của chúng. Bằng cách này, ta có thể tái sử dụng giá trị $k$ cuối cùng, vì việc chuyển từ hậu tố $i$ sang hậu tố $i+1$ chính là loại bỏ chữ cái đầu tiên. Ta sẽ cần một mảng bổ sung rank, sẽ cho ta vị trí của một hậu tố trong danh sách đã sắp xếp của các hậu tố:

```
vector<int> lcp_construction(string const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
```

```
        rank[p[i]] = i;

    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k])
            k++;
        lcp[rank[i]] = k;
        if (k)
            k--;
    }
    return lcp;
}
```

Dễ thấy rằng ta giảm $k$ tối đa $O(n)$ lần (mỗi lần lặp tối đa một lần, trừ khi $\text{rank}[i] == n-1$, nơi ta đặt lại trực tiếp về 0), và LCP giữa hai chuỗi tối đa là $n-1$, ta cũng sẽ tăng $k$ chỉ $O(n)$ lần. Do đó, thuật toán chạy trong thời gian $O(n)$.

### Số chuỗi con khác nhau

Ta tiền xử lý chuỗi $s$ bằng cách tính mảng hậu tố và mảng LCP. Sử dụng thông tin này, ta có thể tính số chuỗi con khác nhau trong chuỗi.

Để làm điều này, ta sẽ nghĩ về những chuỗi con mới nào bắt đầu tại vị trí $p[0]$, sau đó tại $p[1]$, v.v. Thực tế, ta lấy các hậu tố theo thứ tự đã sắp xếp và xem những tiền tố nào cho các chuỗi con mới. Như vậy, ta sẽ không bỏ sót bất kỳ chuỗi nào một cách tình cờ.

Vì các hậu tố đã được sắp xếp, rõ ràng hậu tố hiện tại $p[i]$ sẽ cho các chuỗi con mới cho tất cả các tiền tố của nó, trừ những tiền tố trùng với hậu tố $p[i-1]$. Do đó, tất cả các tiền tố của nó trừ $\text{lcp}[i-1]$ cái đầu tiên. Vì độ dài của hậu tố hiện tại là $n - p[i]$, $n - p[i] - \text{lcp}[i-1]$ tiền tố mới bắt đầu tại $p[i]$. Cộng dồn qua tất cả các hậu tố, ta nhận được đáp án cuối cùng:

$$\sum_{i=0}^{n-1}(n - p[i]) - \sum_{i=0}^{n-2}\text{lcp}[i] = \frac{n^2+n}{2} - \sum_{i=0}^{n-2}\text{lcp}[i]$$

# Chương 3

# Đồ thị

## 3.1 Tìm kiếm theo chiều rộng

Tìm kiếm theo chiều rộng là một trong những thuật toán tìm kiếm cơ bản và thiết yếu trên đồ thị.

Kết quả của cách hoạt động của thuật toán là đường đi được tìm thấy bởi tìm kiếm theo chiều rộng đến bất kỳ đỉnh nào là đường đi ngắn nhất đến đỉnh đó, tức là đường đi chứa số lượng cạnh nhỏ nhất trong đồ thị không có trọng số.

Thuật toán hoạt động trong thời gian $O(n+m)$, trong đó $n$ là số lượng đỉnh và $m$ là số lượng cạnh.

### 3.1.1 Mô tả thuật toán

Thuật toán nhận đầu vào là một đồ thị không có trọng số và mã định danh của đỉnh nguồn $s$. Đồ thị đầu vào có thể là đồ thị có hướng hoặc không có hướng, điều này không ảnh hưởng đến thuật toán.

Thuật toán có thể được hiểu như một đám cháy lan truyền trên đồ thị: ở bước thứ không, chỉ có đỉnh nguồn $s$ đang cháy. Ở mỗi bước, đám cháy tại mỗi đỉnh lan sang tất cả các đỉnh lân cận của nó. Trong một lần lặp của thuật toán, "vòng lửa" được mở rộng thêm một đơn vị về chiều rộng (do đó có tên gọi của thuật toán).

Chính xác hơn, thuật toán có thể được phát biểu như sau: Tạo một hàng đợi $q$ sẽ chứa các đỉnh cần xử lý và một mảng Boolean $used[]$ cho biết với mỗi đỉnh, liệu nó đã được đốt cháy (hoặc đã được thăm) hay chưa.

Ban đầu, đẩy đỉnh nguồn $s$ vào hàng đợi và đặt $used[s] = true$, và với tất cả các đỉnh khác $v$ đặt $used[v] = false$. Sau đó, lặp cho đến khi hàng đợi rỗng và trong mỗi lần lặp, lấy một đỉnh từ đầu hàng đợi. Duyệt qua tất cả các cạnh đi ra từ đỉnh này và nếu một số cạnh này dẫn đến các đỉnh chưa được đốt cháy, đốt cháy chúng và đặt chúng vào hàng đợi.

Kết quả là, khi hàng đợi rỗng, "vòng lửa" chứa tất cả các đỉnh có thể đến được từ đỉnh nguồn $s$, với mỗi đỉnh được tiếp cận theo cách ngắn nhất có thể. Bạn cũng có thể tính độ dài của các đường đi ngắn nhất (chỉ cần duy trì một mảng độ dài đường đi $d[]$) cũng như lưu thông tin để khôi phục tất cả các đường đi ngắn nhất này (đối với điều này, cần duy trì một mảng "cha" $p[]$, lưu trữ cho mỗi đỉnh đỉnh mà từ đó ta đã đến được nó).

### 3.1.2 Triển khai

Ta viết mã cho thuật toán được mô tả bằng C++:

```
vector<vector<int>> adj;  // biểu diễn danh sách kề
int n; // số lượng nút
int s; // đỉnh nguồn

queue<int> q;
vector<bool> used(n);
vector<int> d(n), p(n);

q.push(s);
used[s] = true;
p[s] = -1;
while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
            d[u] = d[v] + 1;
            p[u] = v;
        }
    }
}
```

Nếu ta phải khôi phục và hiển thị đường đi ngắn nhất từ nguồn đến một đỉnh nào đó $u$, điều này có thể được thực hiện theo cách sau:

```
if (!used[u]) {
    cout << "Không có đường đi!";
} else {
    vector<int> path;
```

```
    for (int v = u; v != -1; v = p[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    cout << "Đường đi: ";
    for (int v : path)
        cout << v << " ";
}
```

### 3.1.3 Ứng dụng của BFS

- **Tìm đường đi ngắn nhất từ một nguồn đến các đỉnh khác trong đồ thị không có trọng số.**

- **Tìm tất cả các thành phần liên thông trong một đồ thị không có hướng trong thời gian $O(n+m)$:** Để làm điều này, ta chỉ cần chạy BFS bắt đầu từ mỗi đỉnh, ngoại trừ các đỉnh đã được thăm từ các lần chạy trước. Do đó, ta thực hiện BFS thông thường từ mỗi đỉnh, nhưng không đặt lại mảng $used[]$ mỗi khi ta có một thành phần liên thông mới, và tổng thời gian chạy vẫn sẽ là $O(n+m)$ (thực hiện nhiều BFS trên đồ thị mà không đặt lại mảng $used[]$ được gọi là một chuỗi tìm kiếm theo chiều rộng).

- **Tìm giải pháp cho một bài toán hoặc trò chơi với số bước di chuyển ít nhất,** nếu mỗi trạng thái của trò chơi có thể được biểu diễn bằng một đỉnh của đồ thị, và các chuyển đổi từ trạng thái này sang trạng thái khác là các cạnh của đồ thị.

- **Tìm đường đi ngắn nhất trong đồ thị có trọng số 0 hoặc 1:** Điều này chỉ cần một chút sửa đổi so với tìm kiếm theo chiều rộng thông thường: Thay vì duy trì mảng $used[]$, giờ đây ta sẽ kiểm tra xem khoảng cách đến đỉnh có ngắn hơn khoảng cách đã tìm thấy hiện tại không, sau đó nếu cạnh hiện tại có trọng số bằng 0, ta thêm nó vào đầu hàng đợi, nếu không ta thêm vào cuối hàng đợi. Sự sửa đổi này được giải thích chi tiết hơn trong bài viết về BFS 0-1.

- **Tìm chu trình ngắn nhất trong một đồ thị có hướng không trọng số:** Bắt đầu tìm kiếm theo chiều rộng từ mỗi đỉnh. Ngay khi ta cố gắng đi từ đỉnh hiện tại trở lại đỉnh nguồn, ta đã tìm thấy chu trình ngắn nhất chứa đỉnh nguồn. Tại thời điểm này, ta có thể dừng BFS và bắt đầu một BFS mới từ đỉnh tiếp theo. Từ tất cả các chu trình như vậy (tối đa một từ mỗi BFS), chọn chu trình ngắn nhất.

- **Tìm tất cả các cạnh nằm trên bất kỳ đường đi ngắn nhất nào giữa một cặp đỉnh đã cho $(a, b)$:** Để làm điều này, chạy hai tìm kiếm theo chiều rộng: một từ $a$ và một từ $b$. Gọi $d_a[]$ là mảng chứa các khoảng cách ngắn nhất thu được từ BFS đầu tiên (từ $a$) và $d_b[]$ là mảng chứa các khoảng cách ngắn nhất thu được từ

BFS thứ hai từ $b$. Bây giờ, với mỗi cạnh $(u, v)$, dễ dàng kiểm tra xem cạnh đó có nằm trên bất kỳ đường đi ngắn nhất nào giữa $a$ và $b$ hay không: tiêu chí là điều kiện $d_a[u] + 1 + d_b[v] = d_a[b]$.

- **Tìm tất cả các đỉnh trên bất kỳ đường đi ngắn nhất nào giữa một cặp đỉnh đã cho $(a, b)$:** Để thực hiện điều này, chạy hai tìm kiếm theo chiều rộng: một từ $a$ và một từ $b$. Gọi $d_a[]$ là mảng chứa các khoảng cách ngắn nhất thu được từ BFS đầu tiên (từ $a$) và $d_b[]$ là mảng chứa các khoảng cách ngắn nhất thu được từ BFS thứ hai (từ $b$). Bây giờ, với mỗi đỉnh, dễ dàng kiểm tra xem nó có nằm trên bất kỳ đường đi ngắn nhất nào giữa $a$ và $b$ hay không: tiêu chí là điều kiện $d_a[v] + d_b[v] = d_a[b]$.

- **Tìm đường đi ngắn nhất có độ dài chẵn từ một đỉnh nguồn $s$ đến một đỉnh đích $t$ trong một đồ thị không trọng số:** Để làm điều này, ta phải xây dựng một đồ thị phụ trợ, trong đó các đỉnh là trạng thái $(v, c)$, với $v$ - đỉnh hiện tại, $c = 0$ hoặc $c = 1$ - tính chẵn lẻ hiện tại. Bất kỳ cạnh $(u, v)$ nào của đồ thị ban đầu trong đồ thị mới này sẽ biến thành hai cạnh $((u, 0), (v, 1))$ và $((u, 1), (v, 0))$. Sau đó, ta chạy BFS để tìm đường đi ngắn nhất từ đỉnh bắt đầu $(s, 0)$ đến đỉnh kết thúc $(t, 0)$.
  **Lưu ý:** Mục này sử dụng thuật ngữ "đường đi" thay vì "path" vì một lý do, vì các đỉnh có thể lặp lại trong đường đi được tìm thấy để làm cho độ dài của nó là số chẵn. Bài toán tìm đường đi ngắn nhất có độ dài chẵn là NP-Complete trong đồ thị có hướng, và có thể giải trong thời gian tuyến tính trong đồ thị không có hướng, nhưng với một cách tiếp cận phức tạp hơn nhiều.

## 3.2 Tìm kiếm theo chiều sâu

Tìm kiếm theo chiều sâu là một trong những thuật toán đồ thị chính.

Tìm kiếm theo chiều sâu tìm ra đường đi đầu tiên theo thứ tự từ điển trong đồ thị từ một đỉnh nguồn $u$ đến mỗi đỉnh. Tìm kiếm theo chiều sâu cũng sẽ tìm ra các đường đi ngắn nhất trong một cây (vì chỉ tồn tại một đường đi đơn giản), nhưng trên các đồ thị tổng quát thì không phải vậy.

Thuật toán hoạt động trong thời gian $O(m + n)$, trong đó $n$ là số lượng đỉnh và $m$ là số lượng cạnh.

### 3.2.1 Mô tả thuật toán

Ý tưởng đằng sau DFS là đi sâu vào đồ thị càng nhiều càng tốt, và quay lui khi bạn ở một đỉnh không còn đỉnh lân cận nào chưa được thăm.

Rất dễ để mô tả / triển khai thuật toán theo cách đệ quy: Ta bắt đầu tìm kiếm tại một đỉnh. Sau khi thăm một đỉnh, ta tiếp tục thực hiện DFS cho mỗi đỉnh lân cận mà

ta chưa thăm trước đó. Bằng cách này, ta thăm tất cả các đỉnh có thể đến được từ đỉnh bắt đầu.

Để biết thêm chi tiết, hãy xem phần triển khai.

### 3.2.2 Ứng dụng của Tìm kiếm theo chiều sâu

- **Tìm bất kỳ đường đi nào trong đồ thị từ đỉnh nguồn $u$ đến tất cả các đỉnh.**

- **Tìm đường đi đầu tiên theo thứ tự từ điển trong đồ thị từ nguồn $u$ đến tất cả các đỉnh.**

- **Kiểm tra xem một đỉnh trong cây có phải là tổ tiên của một đỉnh khác hay không:**
  Ở đầu và cuối mỗi lần gọi tìm kiếm, ta ghi nhớ thời gian "vào" và "ra" của mỗi đỉnh. Bây giờ, bạn có thể tìm câu trả lời cho bất kỳ cặp đỉnh nào $(i, j)$ trong $O(1)$: đỉnh $i$ là tổ tiên của đỉnh $j$ nếu và chỉ nếu $\text{entry}[i] < \text{entry}[j]$ và $\text{exit}[i] > \text{exit}[j]$.

- **Tìm tổ tiên chung thấp nhất (LCA) của hai đỉnh.**

- **Sắp xếp tô-pô:**
  Chạy một loạt các tìm kiếm theo chiều sâu để thăm mỗi đỉnh đúng một lần trong thời gian $O(n+m)$. Thứ tự tô-pô cần thiết sẽ là các đỉnh được sắp xếp theo thứ tự giảm dần của thời gian ra.

- **Kiểm tra xem một đồ thị đã cho có phải là không có chu trình hay không và tìm các chu trình trong đồ thị.** (Như đã đề cập ở trên bằng cách đếm các cạnh ngược trong mỗi thành phần liên thông).

- **Tìm các thành phần liên thông mạnh trong một đồ thị có hướng:**
  Đầu tiên thực hiện sắp xếp tô-pô của đồ thị. Sau đó, chuyển vị đồ thị và chạy một loạt các tìm kiếm theo chiều sâu khác theo thứ tự được định nghĩa bởi sắp xếp tô-pô. Với mỗi lần gọi DFS, thành phần được tạo ra bởi nó là một thành phần liên thông mạnh.

- **Tìm các cầu trong một đồ thị không có hướng:**
  Đầu tiên chuyển đổi đồ thị đã cho thành một đồ thị có hướng bằng cách chạy một loạt các tìm kiếm theo chiều sâu và làm cho mỗi cạnh có hướng khi ta đi qua nó, theo hướng mà ta đã đi. Thứ hai, tìm các thành phần liên thông mạnh trong đồ thị có hướng này. Các cầu là các cạnh có hai đầu thuộc về các thành phần liên thông mạnh khác nhau.

### 3.2.3 Phân loại các cạnh của đồ thị

Ta có thể phân loại các cạnh của một đồ thị, $G$, bằng cách sử dụng thời gian vào và ra của các nút cuối $u$ và $v$ của các cạnh $(u, v)$. Những phân loại này thường được sử dụng cho các bài toán như tìm cầu và tìm điểm phân cắt.

Ta thực hiện một DFS và phân loại các cạnh gặp phải bằng các quy tắc sau:

Nếu $v$ chưa được thăm:

- **Cạnh cây -** Nếu $v$ được thăm sau $u$ thì cạnh $(u, v)$ được gọi là cạnh cây. Nói cách khác, nếu $v$ được thăm lần đầu tiên và $u$ đang được thăm thì $(u, v)$ được gọi là cạnh cây. Những cạnh này tạo thành một cây DFS và do đó có tên là cạnh cây.

Nếu $v$ được thăm trước $u$:

- **Cạnh ngược -** Nếu $v$ là tổ tiên của $u$, thì cạnh $(u, v)$ là một cạnh ngược. $v$ là tổ tiên chính xác nếu ta đã vào $v$, nhưng chưa ra khỏi nó. Các cạnh ngược hoàn thành một chu trình vì có một đường đi từ tổ tiên $v$ đến hậu duệ $u$ (trong đệ quy của DFS) và một cạnh từ hậu duệ $u$ đến tổ tiên $v$ (cạnh ngược), do đó một chu trình được hình thành. Chu trình có thể được phát hiện bằng cách sử dụng các cạnh ngược.

- **Cạnh tiến -** Nếu $v$ là hậu duệ của $u$, thì cạnh $(u, v)$ là một cạnh tiến. Nói cách khác, nếu ta đã thăm và ra khỏi $v$ và $\text{entry}[u] < \text{entry}[v]$ thì cạnh $(u, v)$ tạo thành một cạnh tiến.

- **Cạnh chéo -** Nếu $v$ không phải là tổ tiên hay hậu duệ của $u$, thì cạnh $(u, v)$ là một cạnh chéo. Nói cách khác, nếu ta đã thăm và ra khỏi $v$ và $\text{entry}[u] > \text{entry}[v]$ thì $(u, v)$ là một cạnh chéo.

**Định lý.** Giả sử $G$ là một đồ thị không có hướng. Khi đó, thực hiện một DFS trên $G$ sẽ phân loại mỗi cạnh gặp phải thành cạnh cây hoặc cạnh ngược, tức là các cạnh tiến và cạnh chéo chỉ tồn tại trong đồ thị có hướng.

Giả sử $(u, v)$ là một cạnh bất kỳ của $G$ và không mất tính tổng quát, $u$ được thăm trước $v$, tức là $\text{entry}[u] < \text{entry}[v]$. Vì DFS chỉ xử lý các cạnh một lần, chỉ có hai cách để ta có thể xử lý cạnh $(u, v)$ và do đó phân loại nó:

- Lần đầu tiên ta khám phá cạnh $(u, v)$ là theo hướng từ $u$ đến $v$. Vì $\text{entry}[u] < \text{entry}[v]$, bản chất đệ quy của DFS có nghĩa là nút $v$ sẽ được khám phá hoàn toàn và do đó ra khỏi trước khi ta có thể "quay ngược lên ngăn xếp gọi" để ra khỏi nút $u$. Do đó, nút $v$ phải chưa được thăm khi DFS lần đầu tiên khám phá cạnh $(u, v)$ từ $u$ đến $v$ vì nếu không thì quá trình tìm kiếm sẽ đã khám phá $(u, v)$ từ $v$ đến $u$ trước khi ra khỏi nút $v$, vì các nút $u$ và $v$ là láng giềng. Do đó, cạnh $(u, v)$ là một cạnh cây.

- Lần đầu tiên ta khám phá cạnh $(u, v)$ là theo hướng từ $v$ đến $u$. Vì ta phát hiện nút $u$ trước khi phát hiện nút $v$, và ta chỉ xử lý các cạnh một lần, cách duy nhất để ta có thể khám phá cạnh $(u, v)$ theo hướng từ $v$ đến $u$ là nếu có một đường đi khác từ $u$ đến $v$ không liên quan đến cạnh $(u, v)$, do đó làm cho $u$ là tổ tiên của $v$. Cạnh $(u, v)$ do đó hoàn thành một chu trình vì nó đi từ hậu duệ, $v$, đến tổ tiên, $u$, mà ta chưa ra khỏi. Do đó, cạnh $(u, v)$ là một cạnh ngược.

Vì chỉ có hai cách để xử lý cạnh $(u, v)$, với hai trường hợp và phân loại kết quả của chúng được nêu ở trên, việc thực hiện một DFS trên $G$ do đó sẽ phân loại mỗi cạnh gặp phải thành cạnh cây hoặc cạnh ngược, tức là các cạnh tiến và cạnh chéo chỉ tồn tại trong đồ thị có hướng. Điều này hoàn thành chứng minh.

### 3.2.4 Triển khai

```
vector<vector<int>> adj; // đồ thị được biểu diễn dưới dạng danh sách kề
int n; // số lượng đỉnh

vector<bool> visited;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u])
            dfs(u);
    }
}
```

Đây là triển khai đơn giản nhất của Tìm kiếm theo chiều sâu. Như đã mô tả trong các ứng dụng, có thể hữu ích khi tính thêm thời gian vào và ra cũng như màu của đỉnh. Ta sẽ tô màu tất cả các đỉnh với màu 0 nếu ta chưa thăm chúng, với màu 1 nếu ta đã thăm chúng, và với màu 2 nếu ta đã ra khỏi đỉnh.

Dưới đây là một triển khai tổng quát hơn, ngoài ra còn tính toán những điều đó:

```
vector<vector<int>> adj; // đồ thị được biểu diễn dưới dạng danh sách kề
int n; // số lượng đỉnh

vector<int> color;

vector<int> time_in, time_out;
int dfs_timer = 0;
```

```
void dfs(int v) {
    time_in[v] = dfs_timer++;
    color[v] = 1;
    for (int u : adj[v])
        if (color[u] == 0)
            dfs(u);
    color[v] = 2;
    time_out[v] = dfs_timer++;
}
```

## 3.3 Tìm kiếm các thành phần liên thông trong đồ thị

Cho một đồ thị không có hướng $G$ với $n$ nút và $m$ cạnh. Ta được yêu cầu tìm tất cả các thành phần liên thông trong đó, tức là một số nhóm các đỉnh sao cho trong một nhóm, mỗi đỉnh có thể được tiếp cận từ một đỉnh khác và không tồn tại đường đi giữa các nhóm khác nhau.

### 3.3.1 Thuật toán giải bài toán

- Để giải bài toán, ta có thể sử dụng Tìm kiếm theo chiều sâu (DFS) hoặc Tìm kiếm theo chiều rộng (BFS).
- Thực tế, ta sẽ thực hiện một loạt các vòng DFS: Vòng đầu tiên sẽ bắt đầu từ nút đầu tiên và tất cả các nút trong thành phần liên thông đầu tiên sẽ được duyệt qua (tìm thấy). Sau đó, ta tìm nút chưa được thăm đầu tiên trong số các nút còn lại, và chạy Tìm kiếm theo chiều sâu trên nó, từ đó tìm ra thành phần liên thông thứ hai. Và cứ tiếp tục như vậy, cho đến khi tất cả các nút được thăm.
- Tổng thời gian chạy tiệm cận của thuật toán này là $O(n+m)$: Thực tế, thuật toán này sẽ không chạy lại trên cùng một đỉnh hai lần, điều này có nghĩa là mỗi cạnh sẽ được xem xét đúng hai lần (tại một đầu và tại đầu kia).

### 3.3.2 Triển khai

```
int n;
vector<vector<int>> adj;
vector<bool> used;
vector<int> comp;

void dfs(int v) {
    used[v] = true;
```

```
    comp.push_back(v);
    for (int u : adj[v]) {
        if (!used[u])
            dfs(u);
    }
}

void find_comps() {
    fill(used.begin(), used.end(), 0);
    for (int v = 0; v < n; ++v) {
        if (!used[v]) {
            comp.clear();
            dfs(v);
            cout << "Thành phần liên thông:";
            for (int u : comp)
                cout << ' ' << u;
            cout << endl;
        }
    }
}
```

- Hàm quan trọng nhất được sử dụng là find_comps(), hàm này tìm và hiển thị các thành phần liên thông của đồ thị.
- Đồ thị được lưu trữ dưới dạng biểu diễn danh sách kề, tức là adj[$v$] chứa danh sách các đỉnh có cạnh từ đỉnh $v$.
- Vector comp chứa danh sách các nút trong thành phần liên thông hiện tại.

### 3.3.3 Triển khai lặp

Các hàm đệ quy sâu thường không tốt. Mỗi lần gọi đệ quy sẽ yêu cầu một chút bộ nhớ trong ngăn xếp, và theo mặc định, các chương trình chỉ có một lượng không gian ngăn xếp giới hạn. Vì vậy, khi bạn thực hiện một DFS đệ quy trên một đồ thị liên thông với hàng triệu nút, bạn có thể gặp phải tràn ngăn xếp.

Luôn có thể chuyển đổi một chương trình đệ quy thành một chương trình lặp, bằng cách tự duy trì một cấu trúc dữ liệu ngăn xếp. Vì cấu trúc dữ liệu này được cấp phát trên heap, sẽ không xảy ra tràn ngăn xếp.

```
int n;
vector<vector<int>> adj;
vector<bool> used;
```

```
vector<int> comp;

void dfs(int v) {
    stack<int> st;
    st.push(v);

    while (!st.empty()) {
        int curr = st.top();
        st.pop();
        if (!used[curr]) {
            used[curr] = true;
            comp.push_back(curr);
            for (int i = adj[curr].size() - 1; i >= 0; i--) {
                st.push(adj[curr][i]);
            }
        }
    }
}

void find_comps() {
    fill(used.begin(), used.end(), 0);
    for (int v = 0; v < n; ++v) {
        if (!used[v]) {
            comp.clear();
            dfs(v);
            cout << "Thành phần liên thông:";
            for (int u : comp)
                cout << ' ' << u;
            cout << endl;
        }
    }
}
```

## 3.4 Tìm cầu trong đồ thị trong $O(N + M)$

Cho một đồ thị không có hướng. Một cầu được định nghĩa là một cạnh mà khi loại bỏ, làm cho đồ thị trở nên không liên thông (hoặc chính xác hơn, làm tăng số lượng thành phần liên thông trong đồ thị). Nhiệm vụ là tìm tất cả các cầu trong đồ thị đã cho.

Nói một cách không chính thức, bài toán được phát biểu như sau: cho một bản đồ các thành phố được kết nối bằng các con đường, tìm tất cả các con đường "quan trọng", tức

là các con đường mà khi loại bỏ, làm biến mất một đường đi giữa một số cặp thành phố.

Thuật toán được mô tả ở đây dựa trên tìm kiếm theo chiều sâu và có độ phức tạp $O(N + M)$, trong đó $N$ là số lượng đỉnh và $M$ là số lượng cạnh trong đồ thị.

Lưu ý rằng cũng có bài viết Tìm cầu Trực tuyến - không giống như thuật toán ngoại tuyến được mô tả ở đây, thuật toán trực tuyến có thể duy trì danh sách tất cả các cầu trong một đồ thị thay đổi (giả sử rằng loại thay đổi duy nhất là thêm cạnh mới).

### 3.4.1 Thuật toán

Chọn một đỉnh bất kỳ của đồ thị *root* và chạy tìm kiếm theo chiều sâu từ nó. Lưu ý sự thật sau (dễ chứng minh):

- Giả sử ta đang trong DFS, xem xét các cạnh bắt đầu từ đỉnh $v$. Cạnh hiện tại $(v, to)$ là một cầu nếu và chỉ nếu không có đỉnh nào trong số $to$ và các hậu duệ của nó trong cây duyệt DFS có một cạnh ngược đến đỉnh $v$ hoặc bất kỳ tổ tiên nào của nó. Thật vậy, điều kiện này có nghĩa là không có cách nào khác từ $v$ đến $to$ ngoại trừ cạnh $(v, to)$.

Bây giờ ta phải học cách kiểm tra sự thật này cho mỗi đỉnh một cách hiệu quả. Ta sẽ sử dụng "thời gian vào nút" được tính bởi tìm kiếm theo chiều sâu.

Vậy, gọi $\texttt{tin}[v]$ là thời gian vào của nút $v$. Ta giới thiệu một mảng `low` sẽ cho phép ta lưu trữ nút có thời gian vào sớm nhất được tìm thấy trong quá trình tìm kiếm DFS mà một nút $v$ có thể tiếp cận bằng một cạnh đơn từ chính nó hoặc từ các hậu duệ của nó. $\texttt{low}[v]$ là giá trị nhỏ nhất của $\texttt{tin}[v]$, thời gian vào $\texttt{tin}[p]$ cho mỗi nút $p$ được kết nối với nút $v$ qua một cạnh ngược $(v, p)$ và các giá trị của $\texttt{low}[to]$ cho mỗi đỉnh $to$ là hậu duệ trực tiếp của $v$ trong cây DFS:

$$\texttt{low}[v] = \min \begin{cases} \texttt{tin}[v] \\ \texttt{tin}[p], & \text{với mọi } p \text{ mà } (v, p) \text{ là cạnh ngược} \\ \texttt{low}[to], & \text{với mọi } to \text{ mà } (v, to) \text{ là cạnh cây} \end{cases}$$

Bây giờ, tồn tại một cạnh ngược từ đỉnh $v$ hoặc một trong các hậu duệ của nó đến một trong các tổ tiên của nó nếu và chỉ nếu đỉnh $v$ có một con $to$ sao cho $\texttt{low}[to] \leq \texttt{tin}[v]$. Nếu $\texttt{low}[to] = \texttt{tin}[v]$, cạnh ngược đi trực tiếp đến $v$, nếu không thì nó đi đến một trong các tổ tiên của $v$.

Do đó, cạnh hiện tại $(v, to)$ trong cây DFS là một cầu nếu và chỉ nếu $\texttt{low}[to] > \texttt{tin}[v]$.

### 3.4.2 Triển khai

Triển khai cần phân biệt ba trường hợp: khi ta đi xuống cạnh trong cây DFS, khi ta tìm thấy một cạnh ngược đến một tổ tiên của đỉnh và khi ta quay lại cha của đỉnh. Đây là các trường hợp:

- `visited`$[to] = false$ - cạnh là một phần của cây DFS;
- `visited`$[to] = true$ && $to \neq parent$ - cạnh là cạnh ngược đến một trong các tổ tiên;
- $to = parent$ - cạnh dẫn trở lại cha trong cây DFS.

Để triển khai điều này, ta cần một hàm tìm kiếm theo chiều sâu chấp nhận đỉnh cha của nút hiện tại.

Đối với trường hợp có nhiều cạnh, ta cần cẩn thận khi bỏ qua cạnh từ cha. Để giải quyết vấn đề này, ta có thể thêm một cờ parent_skipped sẽ đảm bảo ta chỉ bỏ qua cha một lần.

```
void IS_BRIDGE(int v, int to); // hàm nào đó để xử lý cầu được tìm thấy
int n; // số lượng nút
vector<vector<int>> adj; // danh sách kề của đồ thị

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
```

```
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

Hàm chính là find_bridges; nó thực hiện khởi tạo cần thiết và bắt đầu tìm kiếm theo chiều sâu trong mỗi thành phần liên thông của đồ thị.

Hàm IS_BRIDGE$(a, b)$ là một hàm nào đó sẽ xử lý việc cạnh $(a, b)$ là một cầu, ví dụ, in nó ra.

Lưu ý rằng triển khai này hoạt động không chính xác nếu đồ thị có nhiều cạnh, vì nó bỏ qua chúng. Tất nhiên, các cạnh nhiều lần sẽ không bao giờ là một phần của câu trả lời, vì vậy IS_BRIDGE có thể kiểm tra thêm rằng cầu được báo cáo không phải là một cạnh nhiều lần. Ngoài ra, có thể truyền cho dfs chỉ số của cạnh được sử dụng để vào đỉnh thay vì đỉnh cha (và lưu trữ các chỉ số của tất cả các đỉnh).

## 3.5 Tìm cầu trực tuyến

Cho một đồ thị không có hướng. Một cầu là một cạnh mà khi loại bỏ, làm cho đồ thị trở nên không liên thông (hoặc chính xác hơn, làm tăng số lượng thành phần liên thông). Nhiệm vụ của chúng ta là tìm tất cả các cầu trong đồ thị đã cho.

Nói một cách không chính thức, nhiệm vụ này có thể được diễn đạt như sau: ta phải tìm tất cả các con đường "quan trọng" trên bản đồ đường bộ đã cho, tức là những con đường mà việc loại bỏ bất kỳ con đường nào trong số đó sẽ dẫn đến một số thành phố không thể tiếp cận được từ những thành phố khác.

Đã có bài viết *Tìm cầu trong* $O(N + M)$ giải quyết nhiệm vụ này bằng cách duyệt theo chiều sâu (DFS). Thuật toán này sẽ phức tạp hơn nhiều, nhưng nó có một ưu điểm lớn: thuật toán được mô tả trong bài viết này hoạt động trực tuyến, nghĩa là đồ thị đầu vào không cần phải được biết trước. Các cạnh được thêm vào từng cái một, và sau mỗi lần thêm, thuật toán đếm lại tất cả các cầu trong đồ thị hiện tại. Nói cách khác, thuật toán được thiết kế để hoạt động hiệu quả trên một đồ thị động, thay đổi.

Chính xác hơn, phát biểu của bài toán như sau: Ban đầu đồ thị rỗng và bao gồm $n$ đỉnh. Sau đó, ta nhận được các cặp đỉnh $(a, b)$, biểu thị một cạnh được thêm vào đồ thị. Sau mỗi cạnh được nhận, tức là sau khi thêm mỗi cạnh, xuất ra số lượng cầu hiện tại trong đồ thị.

Cũng có thể duy trì một danh sách tất cả các cầu cũng như hỗ trợ rõ ràng các thành phần liên thông 2-cạnh.

Thuật toán được mô tả dưới đây hoạt động trong thời gian $O(n \log n + m)$, trong đó $m$ là số lượng cạnh. Thuật toán dựa trên cấu trúc dữ liệu Tập hợp không giao nhau (Disjoint Set Union - DSU). Tuy nhiên, triển khai trong bài viết này mất thời gian $O(n \log n + m \log n)$, vì nó sử dụng phiên bản đơn giản hóa của DSU mà không có Union by Rank.

### 3.5.1 Thuật toán

Đầu tiên, hãy định nghĩa một thành phần liên thông $k$-cạnh: đó là một thành phần liên thông vẫn giữ được tính liên thông khi bạn loại bỏ ít hơn $k$ cạnh.

Rất dễ thấy rằng các cầu chia đồ thị thành các thành phần liên thông 2-cạnh. Nếu ta nén mỗi thành phần liên thông 2-cạnh đó thành các đỉnh và chỉ giữ lại các cầu làm cạnh trong đồ thị nén, thì ta thu được một đồ thị không có chu trình, tức là một rừng (forest).

Thuật toán được mô tả dưới đây duy trì rừng này một cách rõ ràng cũng như các thành phần liên thông 2-cạnh.

Rõ ràng, ban đầu, khi đồ thị rỗng, nó chứa $n$ thành phần liên thông 2-cạnh, mà bản thân chúng không liên kết với nhau.

Khi thêm cạnh tiếp theo $(a, b)$, có thể xảy ra ba tình huống:

- Cả hai đỉnh $a$ và $b$ đều nằm trong cùng một thành phần liên thông 2-cạnh - khi đó cạnh này không phải là cầu, và không thay đổi gì trong cấu trúc rừng, vì vậy ta có thể bỏ qua cạnh này.
  Do đó, trong trường hợp này, số lượng cầu không thay đổi.

- Các đỉnh $a$ và $b$ nằm trong các thành phần liên thông hoàn toàn khác nhau, tức là mỗi đỉnh thuộc một cây khác nhau. Trong trường hợp này, cạnh $(a, b)$ trở thành một cầu mới, và hai cây này được kết hợp thành một (và tất cả các cầu cũ vẫn được giữ nguyên).
  Do đó, trong trường hợp này, số lượng cầu tăng thêm một.

- Các đỉnh $a$ và $b$ nằm trong một thành phần liên thông, nhưng trong các thành phần liên thông 2-cạnh khác nhau. Trong trường hợp này, cạnh này tạo thành một chu trình cùng với một số cầu cũ. Tất cả các cầu này không còn là cầu nữa, và chu trình kết quả phải được nén thành một thành phần liên thông 2-cạnh mới.
  Do đó, trong trường hợp này, số lượng cầu giảm đi hai hoặc nhiều hơn.

Do đó, toàn bộ nhiệm vụ được rút gọn về việc triển khai hiệu quả tất cả các thao tác này trên rừng của các thành phần liên thông 2-cạnh.

### 3.5.2 Cấu trúc dữ liệu để lưu trữ rừng

Cấu trúc dữ liệu duy nhất mà ta cần là Tập hợp không giao nhau (DSU). Thực tế, ta sẽ tạo hai bản sao của cấu trúc này: một để duy trì các thành phần liên thông, một để

duy trì các thành phần liên thông 2-cạnh. Ngoài ra, ta lưu trữ cấu trúc của các cây trong rừng của các thành phần liên thông 2-cạnh thông qua các con trỏ: Mỗi thành phần liên thông 2-cạnh sẽ lưu trữ chỉ số par[] của tổ tiên của nó trong cây.

Bây giờ ta sẽ phân tích từng thao tác mà ta cần học cách triển khai:

- **Kiểm tra xem hai đỉnh có nằm trong cùng một thành phần liên thông / thành phần liên thông 2-cạnh hay không.** Điều này được thực hiện với thuật toán DSU thông thường, ta chỉ cần tìm và so sánh các đại diện của các DSU.

- **Kết hợp hai cây cho một cạnh** $(a, b)$**.** Vì có thể xảy ra trường hợp cả đỉnh $a$ lẫn đỉnh $b$ đều không phải là gốc của cây của chúng, cách duy nhất để kết nối hai cây này là thay đổi gốc của một trong hai cây. Ví dụ, bạn có thể thay đổi gốc của cây của đỉnh $a$, rồi gắn nó vào cây kia bằng cách đặt tổ tiên của $a$ là $b$.

  Tuy nhiên, câu hỏi về hiệu quả của thao tác thay đổi gốc xuất hiện: để thay đổi gốc của cây với gốc $r$ thành đỉnh $v$, cần phải thăm tất cả các đỉnh trên đường đi giữa $v$ và $r$ và chuyển hướng các con trỏ par[] theo hướng ngược lại, đồng thời thay đổi các tham chiếu đến tổ tiên trong DSU chịu trách nhiệm cho các thành phần liên thông.

  Do đó, chi phí của việc thay đổi gốc là $O(h)$, trong đó $h$ là chiều cao của cây. Bạn có thể đưa ra một ước lượng tệ hơn bằng cách nói rằng chi phí là $O(\text{size})$, trong đó size là số lượng đỉnh trong cây. Độ phức tạp cuối cùng sẽ không khác biệt.

  Bây giờ ta áp dụng một kỹ thuật tiêu chuẩn: ta thay đổi gốc của cây có ít đỉnh hơn. Thì trực giác cho thấy trường hợp xấu nhất là khi hai cây có kích thước gần bằng nhau được kết hợp, nhưng sau đó kết quả là một cây có kích thước gấp đôi. Điều này không cho phép tình huống này xảy ra nhiều lần.

  Tổng chi phí tổng quát có thể được viết dưới dạng một công thức truy hồi:

  $$T(n) = \max_{k=1\ldots n-1} \{T(k) + T(n-k) + O(\min(k, n-k))\}$$

  $T(n)$ là số lượng thao tác cần thiết để có được một cây với $n$ đỉnh thông qua việc thay đổi gốc và hợp nhất các cây. Một cây có kích thước $n$ có thể được tạo ra bằng cách kết hợp hai cây nhỏ hơn có kích thước $k$ và $n-k$. Công thức truy hồi này có nghiệm là $T(n) = O(n \log n)$.

  Do đó, tổng thời gian dành cho tất cả các thao tác thay đổi gốc sẽ là $O(n \log n)$ nếu ta luôn thay đổi gốc của cây nhỏ hơn trong hai cây.

  Ta sẽ phải duy trì kích thước của mỗi thành phần liên thông, nhưng cấu trúc dữ liệu DSU cho phép thực hiện điều này mà không gặp khó khăn.

- **Tìm kiếm chu trình được hình thành bằng cách thêm một cạnh mới** $(a, b)$**.** Vì $a$ và $b$ đã được kết nối trong cây, ta cần tìm Tổ tiên chung thấp nhất (LCA) của

các đỉnh $a$ và $b$. Chu trình sẽ bao gồm các đường đi từ $b$ đến LCA, từ LCA đến $a$ và cạnh từ $a$ đến $b$.

Sau khi tìm thấy chu trình, ta nén tất cả các đỉnh của chu trình được phát hiện thành một đỉnh. Điều này có nghĩa là ta đã có độ phức tạp tỷ lệ với độ dài chu trình, nghĩa là ta cũng có thể sử dụng bất kỳ thuật toán LCA nào tỷ lệ với độ dài, và không cần phải sử dụng thuật toán nhanh.

Vì tất cả thông tin về cấu trúc của cây có sẵn trong mảng tổ tiên par[], thuật toán LCA hợp lý duy nhất là như sau: đánh dấu các đỉnh $a$ và $b$ là đã thăm, sau đó ta đi đến tổ tiên của chúng par[$a$] và par[$b$] và đánh dấu chúng, rồi tiếp tục tiến tới tổ tiên của chúng, cho đến khi ta đến một đỉnh đã được đánh dấu. Đỉnh này là LCA mà ta đang tìm kiếm, và ta có thể tìm các đỉnh trên chu trình bằng cách duyệt lại đường đi từ $a$ và $b$ đến LCA.

Rõ ràng, độ phức tạp của thuật toán này tỷ lệ với độ dài của chu trình mong muốn.

- **Nén chu trình bằng cách thêm một cạnh mới $(a, b)$ trong một cây.**
  Ta cần tạo một thành phần liên thông 2-cạnh mới, sẽ bao gồm tất cả các đỉnh của chu trình được phát hiện (chu trình được phát hiện cũng có thể bao gồm một số thành phần liên thông 2-cạnh, nhưng điều này không thay đổi gì). Ngoài ra, cần nén chúng sao cho cấu trúc của cây không bị xáo trộn, và tất cả các con trỏ par[] và hai DSU vẫn đúng.

  Cách dễ nhất để đạt được điều này là nén tất cả các đỉnh của chu trình về LCA của chúng. Thực tế, LCA là đỉnh cao nhất, tức là con trỏ tổ tiên par[] của nó vẫn không thay đổi. Đối với tất cả các đỉnh khác của vòng lặp, tổ tiên không cần được cập nhật, vì các đỉnh này chỉ đơn giản là không còn tồn tại. Nhưng trong DSU của các thành phần liên thông 2-cạnh, tất cả các đỉnh này sẽ chỉ đơn giản trỏ đến LCA.

  Ta sẽ triển khai DSU của các thành phần liên thông 2-cạnh mà không có tối ưu hóa Union by Rank, do đó ta sẽ có độ phức tạp $O(\log n)$ trung bình cho mỗi truy vấn. Để đạt được độ phức tạp $O(1)$ trung bình cho mỗi truy vấn, ta cần kết hợp các đỉnh của chu trình theo Union by Rank, rồi gán par[] tương ứng.

### 3.5.3 Triển khai

Dưới đây là triển khai cuối cùng của toàn bộ thuật toán.

Như đã đề cập trước đó, để đơn giản, DSU của các thành phần liên thông 2-cạnh được viết mà không có Union by Rank, do đó độ phức tạp kết quả sẽ là $O(\log n)$ trung bình.

Ngoài ra, trong triển khai này, các cầu không được lưu trữ, chỉ có số lượng của chúng bridges. Tuy nhiên, sẽ không khó để tạo một tập hợp tất cả các cầu.

Ban đầu, bạn gọi hàm init(), khởi tạo hai DSU (tạo một tập riêng cho mỗi đỉnh, và đặt kích thước bằng một), và đặt các tổ tiên par.

Hàm chính là add_edge$(a, b)$, xử lý và thêm một cạnh mới.

```
vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
int bridges;
int lca_iteration;
vector<int> last_visit;

void init(int n) {
    par.resize(n);
    dsu_2ecc.resize(n);
    dsu_cc.resize(n);
    dsu_cc_size.resize(n);
    lca_iteration = 0;
    last_visit.assign(n, 0);
    for (int i=0; i<n; ++i) {
        dsu_2ecc[i] = i;
        dsu_cc[i] = i;
        dsu_cc_size[i] = 1;
        par[i] = -1;
    }
    bridges = 0;
}

int find_2ecc(int v) {
    if (v == -1)
        return -1;
    return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
}

int find_cc(int v) {
    v = find_2ecc(v);
    return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
}

void make_root(int v) {
    int root = v;
    int child = -1;
    while (v != -1) {
        int p = find_2ecc(par[v]);
        par[v] = child;
        dsu_cc[v] = root;
        child = v;
```

```
        v = p;
    }
    dsu_cc_size[root] = dsu_cc_size[child];
}

void merge_path(int a, int b) {
    ++lca_iteration;
    vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (a != -1) {
            a = find_2ecc(a);
            path_a.push_back(a);
            if (last_visit[a] == lca_iteration){
                lca = a;
                break;
            }
            last_visit[a] = lca_iteration;
            a = par[a];
        }
        if (b != -1) {
            b = find_2ecc(b);
            path_b.push_back(b);
            if (last_visit[b] == lca_iteration){
                lca = b;
                break;
            }
            last_visit[b] = lca_iteration;
            b = par[b];
        }
    }

    for (int v : path_a) {
        dsu_2ecc[v] = lca;
        if (v == lca)
            break;
        --bridges;
    }
    for (int v : path_b) {
        dsu_2ecc[v] = lca;
```

```
        if (v == lca)
            break;
        --bridges;
    }
}

void add_edge(int a, int b) {
    a = find_2ecc(a);
    b = find_2ecc(b);
    if (a == b)
        return;

    int ca = find_cc(a);
    int cb = find_cc(b);

    if (ca != cb) {
        ++bridges;
        if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
            swap(a, b);
            swap(ca, cb);
        }
        make_root(a);
        par[a] = dsu_cc[a] = b;
        dsu_cc_size[cb] += dsu_cc_size[a];
    } else {
        merge_path(a, b);
    }
}
```

DSU cho các thành phần liên thông 2-cạnh được lưu trữ trong vector dsu_2ecc, và hàm trả về đại diện là find_2ecc($v$). Hàm này được sử dụng nhiều lần trong phần còn lại của mã, vì sau khi nén nhiều đỉnh thành một, tất cả các đỉnh này không còn tồn tại, và thay vào đó chỉ có đại diện có tổ tiên par chính xác trong rừng của các thành phần liên thông 2-cạnh.

DSU cho các thành phần liên thông được lưu trữ trong vector dsu_cc, và còn có một vector bổ sung dsu_cc_size để lưu trữ kích thước thành phần. Hàm find_cc($v$) trả về đại diện của thành phần liên thông (thực tế là gốc của cây).

Việc thay đổi gốc của một cây make_root($v$) hoạt động như đã mô tả ở trên: nó duyệt từ đỉnh $v$ qua các tổ tiên đến đỉnh gốc, mỗi lần chuyển hướng tổ tiên par theo hướng ngược lại. Liên kết đến đại diện của thành phần liên thông dsu_cc cũng được cập nhật, để trỏ đến đỉnh gốc mới. Sau khi thay đổi gốc, ta phải gán cho gốc mới kích thước chính

xác của thành phần liên thông. Ta cũng phải cẩn thận gọi find_2ecc() để lấy đại diện của thành phần liên thông 2-cạnh, thay vì một đỉnh khác đã bị nén.

Hàm tìm chu trình và nén merge_path$(a, b)$ cũng được triển khai như đã mô tả ở trên. Nó tìm kiếm LCA của $a$ và $b$ bằng cách nâng các nút này song song, cho đến khi gặp một đỉnh lần thứ hai. Vì mục đích hiệu quả, ta chọn một định danh duy nhất cho mỗi lần gọi tìm LCA, và đánh dấu các đỉnh đã duyệt bằng nó. Điều này hoạt động trong $O(1)$, trong khi các cách tiếp cận khác như sử dụng set hoạt động kém hơn. Các đường đi đã qua được lưu trữ trong các vector path_a và path_b, và ta sử dụng chúng để duyệt qua chúng lần thứ hai đến LCA, do đó thu được tất cả các đỉnh của chu trình. Tất cả các đỉnh của chu trình được nén bằng cách gắn chúng vào LCA, do đó độ phức tạp trung bình là $O(\log n)$ (vì ta không sử dụng Union by Rank). Tất cả các cạnh ta đi qua đều là cầu, vì vậy ta trừ 1 cho mỗi cạnh trong chu trình.

Cuối cùng, hàm truy vấn add_edge$(a, b)$ xác định các thành phần liên thông mà các đỉnh $a$ và $b$ nằm trong đó. Nếu chúng nằm trong các thành phần liên thông khác nhau, thì một cây nhỏ hơn được thay đổi gốc và sau đó gắn vào cây lớn hơn. Nếu không, nếu các đỉnh $a$ và $b$ nằm trong một cây, nhưng trong các thành phần liên thông 2-cạnh khác nhau, thì hàm merge_path$(a, b)$ được gọi, sẽ phát hiện chu trình và nén nó thành một thành phần liên thông 2-cạnh.

## 3.6 Tìm điểm phân cắt trong đồ thị trong $O(N + M)$

Cho một đồ thị không có hướng. Một điểm phân cắt (hoặc đỉnh cắt) được định nghĩa là một đỉnh mà khi loại bỏ cùng với các cạnh liên quan, làm cho đồ thị trở nên không liên thông (hoặc chính xác hơn, làm tăng số lượng thành phần liên thông trong đồ thị). Nhiệm vụ là tìm tất cả các điểm phân cắt trong đồ thị đã cho.

Thuật toán được mô tả ở đây dựa trên tìm kiếm theo chiều sâu và có độ phức tạp $O(N + M)$, trong đó $N$ là số lượng đỉnh và $M$ là số lượng cạnh trong đồ thị.

### 3.6.1 Thuật toán

Chọn một đỉnh bất kỳ của đồ thị *root* và chạy tìm kiếm theo chiều sâu từ nó. Lưu ý sự thật sau (dễ chứng minh):

- Giả sử ta đang trong DFS, xem xét các cạnh bắt đầu từ đỉnh $v \neq root$. Nếu cạnh hiện tại $(v, to)$ là sao cho không có đỉnh nào trong số *to* hoặc các hậu duệ của nó trong cây duyệt DFS có một cạnh ngược đến bất kỳ tổ tiên nào của $v$, thì $v$ là một điểm phân cắt. Nếu không, $v$ không phải là điểm phân cắt.

- Hãy xem xét trường hợp còn lại của $v = root$. Đỉnh này sẽ là điểm phân cắt nếu và chỉ nếu đỉnh này có nhiều hơn một con trong cây DFS.

Bây giờ ta phải học cách kiểm tra sự thật này cho mỗi đỉnh một cách hiệu quả. Ta sẽ sử dụng "thời gian vào nút" được tính bởi tìm kiếm theo chiều sâu.

Vậy, gọi $\text{tin}[v]$ là thời gian vào của nút $v$. Ta giới thiệu một mảng $\text{low}[v]$ sẽ cho phép ta kiểm tra sự thật này cho mỗi đỉnh $v$. $\text{low}[v]$ là giá trị nhỏ nhất của $\text{tin}[v]$, thời gian vào $\text{tin}[p]$ cho mỗi nút $p$ được kết nối với nút $v$ qua một cạnh ngược $(v, p)$ và các giá trị của $\text{low}[to]$ cho mỗi đỉnh $to$ là hậu duệ trực tiếp của $v$ trong cây DFS:

$$\text{low}[v] = \min \begin{cases} \text{tin}[v] & \\ \text{tin}[p] & \text{với mọi } p \text{ mà } (v, p) \text{ là cạnh ngược} \\ \text{low}[to] & \text{với mọi } to \text{ mà } (v, to) \text{ là cạnh cây} \end{cases}$$

Bây giờ, tồn tại một cạnh ngược từ đỉnh $v$ hoặc một trong các hậu duệ của nó đến một trong các tổ tiên của nó nếu và chỉ nếu đỉnh $v$ có một con $to$ sao cho $\text{low}[to] < \text{tin}[v]$. Nếu $\text{low}[to] = \text{tin}[v]$, cạnh ngược đi trực tiếp đến $v$, nếu không thì nó đi đến một trong các tổ tiên của $v$.

Do đó, đỉnh $v$ trong cây DFS là một điểm phân cắt nếu và chỉ nếu $\text{low}[to] \geq \text{tin}[v]$.

### 3.6.2 Triển khai

Triển khai cần phân biệt ba trường hợp: khi ta đi xuống cạnh trong cây DFS, khi ta tìm thấy một cạnh ngược đến một tổ tiên của đỉnh và khi ta quay lại cha của đỉnh. Đây là các trường hợp:

- $\text{visited}[to] = \text{false}$ - cạnh là một phần của cây DFS;
- $\text{visited}[to] = \text{true}$ && $to \neq \text{parent}$ - cạnh là cạnh ngược đến một trong các tổ tiên;
- $to = \text{parent}$ - cạnh dẫn trở lại cha trong cây DFS.

Để triển khai điều này, ta cần một hàm tìm kiếm theo chiều sâu chấp nhận đỉnh cha của nút hiện tại.

```
int n; // số lượng nút
vector<vector<int>> adj; // danh sách kề của đồ thị

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
```

```
    int children = 0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

Hàm chính là find_cutpoints; nó thực hiện khởi tạo cần thiết và bắt đầu tìm kiếm theo chiều sâu trong mỗi thành phần liên thông của đồ thị.

Hàm IS_CUTPOINT($a$) là một hàm nào đó sẽ xử lý việc đỉnh $a$ là một điểm phân cắt, ví dụ, in nó ra (Cẩn thận rằng hàm này có thể được gọi nhiều lần cho một đỉnh).

## 3.7 Thành phần liên thông mạnh và đồ thị nén

### 3.7.1 Định nghĩa

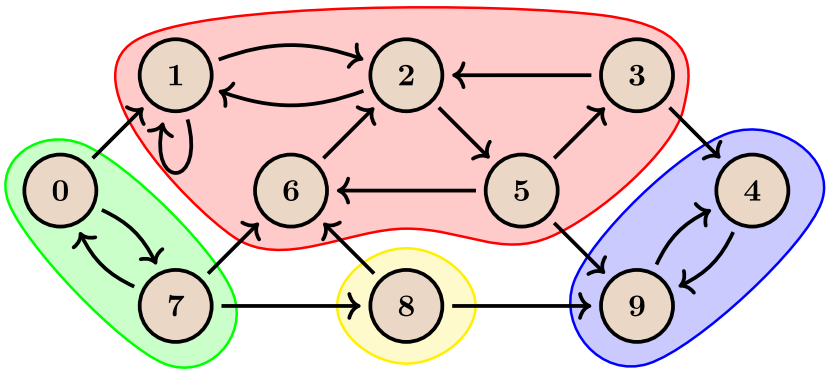
Cho một đồ thị có hướng $G = (V, E)$ với các đỉnh $V$ và các cạnh $E \subseteq V \times V$. Ta ký hiệu $n = |V|$ là số lượng đỉnh và $m = |E|$ là số lượng cạnh trong $G$. Có thể dễ dàng mở rộng tất cả các định nghĩa trong bài viết này cho đa đồ thị, nhưng chúng ta sẽ không tập trung vào điều đó.

Một tập con các đỉnh $C \subseteq V$ được gọi là một *thành phần liên thông mạnh* nếu thỏa mãn các điều kiện sau:

- Với mọi $u, v \in C$, nếu $u \neq v$ thì tồn tại một đường đi từ $u$ đến $v$ và một đường đi từ $v$ đến $u$,
- $C$ là tối đại, theo nghĩa không thể thêm đỉnh nào mà không vi phạm điều kiện trên.

Ta ký hiệu $\mathrm{SCC}(G)$ là tập hợp các thành phần liên thông mạnh của $G$. Các thành phần liên thông mạnh này không giao nhau và bao phủ toàn bộ các đỉnh trong đồ thị. Do đó, tập $\mathrm{SCC}(G)$ là một phân hoạch của $V$.

Xét đồ thị ví dụ $G_{\text{example}}$, trong đó các thành phần liên thông mạnh được làm nổi bật:




Hình 3.1: Graph

Ở đây ta có:

$$\mathrm{SCC}(G_{\text{example}}) = \{\{0, 7\}, \{1, 2, 3, 5, 6\}, \{4, 9\}, \{8\}\}.$$

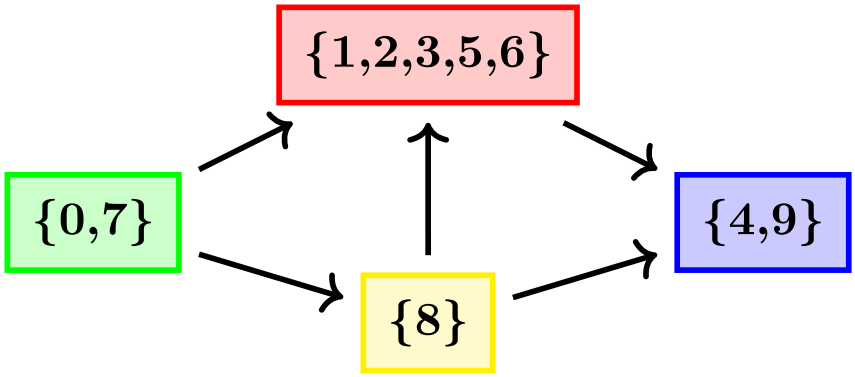
Ta có thể xác nhận rằng trong mỗi thành phần liên thông mạnh, tất cả các đỉnh đều có thể tiếp cận lẫn nhau.

Ta định nghĩa *đồ thị nén* $G^{\mathrm{SCC}} = (V^{\mathrm{SCC}}, E^{\mathrm{SCC}})$ như sau:

- Các đỉnh của $G^{\mathrm{SCC}}$ là các thành phần liên thông mạnh của $G$; tức là, $V^{\mathrm{SCC}} = \mathrm{SCC}(G)$,
- Với mọi đỉnh $C_i, C_j$ của đồ thị nén, tồn tại một cạnh từ $C_i$ đến $C_j$ nếu và chỉ nếu $C_i \neq C_j$ và tồn tại $a \in C_i$ và $b \in C_j$ sao cho có một cạnh từ $a$ đến $b$ trong $G$.

Đồ thị nén của $G_{\text{example}}$ trông như sau:

Đặc tính quan trọng nhất của đồ thị nén là nó không có chu trình. Thật vậy, theo định nghĩa, không có "vòng tự lặp" trong đồ thị nén, và nếu tồn tại một chu trình đi qua




Hình 3.2: Cond Graph

hai hoặc nhiều đỉnh (thành phần liên thông mạnh) trong đồ thị nén, thì do tính tiếp cận được, hợp của các thành phần liên thông mạnh này sẽ phải là một thành phần liên thông mạnh duy nhất: mâu thuẫn.

Thuật toán được mô tả trong phần tiếp theo sẽ tìm tất cả các thành phần liên thông mạnh trong một đồ thị đã cho. Sau đó, đồ thị nén có thể được xây dựng.

### 3.7.2 Mô tả thuật toán

Thuật toán được mô tả đã được Kosaraju và Sharir đề xuất độc lập vào khoảng năm 1980. Nó dựa trên hai loạt tìm kiếm theo chiều sâu, với thời gian chạy là $O(n+m)$.

Trong bước đầu tiên của thuật toán, ta thực hiện một loạt các tìm kiếm theo chiều sâu (DFS), thăm toàn bộ đồ thị. Tức là, miễn là vẫn còn các đỉnh chưa được thăm, ta chọn một trong số chúng và bắt đầu tìm kiếm theo chiều sâu từ đỉnh đó. Với mỗi đỉnh, ta theo dõi thời gian thoát $t_{\text{out}}[v]$. Đây là "dấu thời gian" tại đó việc thực thi DFS trên đỉnh $v$ kết thúc, tức là thời điểm mà tất cả các đỉnh có thể tiếp cận từ $v$ đã được thăm và thuật toán quay lại $v$. Bộ đếm dấu thời gian không nên được đặt lại giữa các lần gọi DFS liên tiếp. Thời gian thoát đóng vai trò quan trọng trong thuật toán, điều này sẽ trở nên rõ ràng khi ta thảo luận về định lý sau.

Trước tiên, ta định nghĩa thời gian thoát $t_{\text{out}}[C]$ của một thành phần liên thông mạnh $C$ là giá trị lớn nhất của $t_{\text{out}}[v]$ với mọi $v \in C$. Hơn nữa, trong chứng minh định lý, ta sẽ đề cập đến thời gian vào $t_{\text{in}}[v]$ cho mỗi đỉnh $v \in G$. Số $t_{\text{in}}[v]$ biểu thị "dấu thời gian" tại đó hàm đệ quy DFS được gọi trên đỉnh $v$ trong bước đầu tiên của thuật toán. Với một thành phần liên thông mạnh $C$, ta định nghĩa $t_{\text{in}}[C]$ là giá trị nhỏ nhất của $t_{\text{in}}[v]$ với mọi $v \in C$.

**Định lý.** Cho $C$ và $C'$ là hai thành phần liên thông mạnh khác nhau, và giả sử tồn tại một cạnh từ $C$ đến $C'$ trong đồ thị nén. Khi đó:

$$t_{\text{out}}[C] > t_{\text{out}}[C']$$

**Chứng minh.** Có hai trường hợp khác nhau, tùy thuộc vào thành phần nào được tìm kiếm theo chiều sâu tiếp cận trước:

- *Trường hợp 1:* Thành phần $C$ được tiếp cận trước (tức là $t_{\text{in}}[C] < t_{\text{in}}[C']$). Trong trường hợp này, tìm kiếm theo chiều sâu thăm một đỉnh $v \in C$ tại một thời điểm mà tất cả các đỉnh khác của các thành phần $C$ và $C'$ chưa được thăm. Vì có một cạnh từ $C$ đến $C'$ trong đồ thị nén, không chỉ tất cả các đỉnh khác trong $C$ có thể tiếp cận từ $v$ trong $G$, mà tất cả các đỉnh trong $C'$ cũng có thể tiếp cận. Điều này có nghĩa là lần thực thi DFS này, đang chạy từ đỉnh $v$, sẽ thăm tất cả các đỉnh khác của các thành phần $C$ và $C'$ trong tương lai, vì vậy các đỉnh này sẽ là hậu duệ của $v$ trong cây tìm kiếm theo chiều sâu. Điều này ngụ ý rằng với mỗi đỉnh $u \in (C \cup C') \setminus \{v\}$, ta có $t_{\text{out}}[v] > t_{\text{out}}[u]$. Do đó, $t_{\text{out}}[C] > t_{\text{out}}[C']$, hoàn tất trường hợp này của chứng minh.

- *Trường hợp 2:* Thành phần $C'$ được tiếp cận trước (tức là $t_{\text{in}}[C] > t_{\text{in}}[C']$). Trong trường hợp này, tìm kiếm theo chiều sâu thăm một đỉnh $v \in C'$ tại một thời điểm mà tất cả các đỉnh khác của các thành phần $C$ và $C'$ chưa được thăm. Vì có một cạnh từ $C$ đến $C'$ trong đồ thị nén, $C$ không thể tiếp cận từ $C'$, do đặc tính không chu trình. Do đó, lần thực thi DFS đang chạy từ đỉnh $v$ sẽ không tiếp cận bất kỳ đỉnh nào của $C$, nhưng sẽ thăm tất cả các đỉnh của $C'$. Các đỉnh của $C$ sẽ được thăm bởi một lần thực thi DFS khác sau đó trong bước này của thuật toán, vì vậy thực sự ta có $t_{\text{out}}[C] > t_{\text{out}}[C']$. Điều này hoàn tất chứng minh.

Định lý đã chứng minh rất quan trọng để tìm các thành phần liên thông mạnh. Nó có nghĩa là bất kỳ cạnh nào trong đồ thị nén đều đi từ một thành phần có giá trị $t_{\text{out}}$ lớn hơn đến một thành phần có giá trị nhỏ hơn.

Nếu ta sắp xếp tất cả các đỉnh $v \in V$ theo thứ tự giảm dần của thời gian thoát $t_{\text{out}}[v]$, thì đỉnh đầu tiên $u$ sẽ thuộc về thành phần liên thông mạnh "gốc", không có cạnh đi vào trong đồ thị nén. Bây giờ ta muốn chạy một loại tìm kiếm nào đó từ đỉnh $u$ này để nó sẽ thăm tất cả các đỉnh trong thành phần liên thông mạnh của nó, nhưng không thăm các đỉnh khác. Bằng cách lặp lại như vậy, ta có thể dần dần tìm tất cả các thành phần liên thông mạnh: ta loại bỏ tất cả các đỉnh thuộc thành phần đầu tiên tìm được, sau đó tìm đỉnh còn lại tiếp theo có giá trị $t_{\text{out}}$ lớn nhất, và chạy tìm kiếm này từ nó, và cứ tiếp tục. Cuối cùng, ta sẽ tìm được tất cả các thành phần liên thông mạnh. Để tìm một phương pháp tìm kiếm hoạt động như ta muốn, ta xem xét định lý sau:

**Định lý.** Gọi $G^T$ là đồ thị chuyển vị của $G$, thu được bằng cách đảo chiều các cạnh trong $G$. Khi đó, $\text{SCC}(G) = \text{SCC}(G^T)$. Hơn nữa, đồ thị nén của $G^T$ là chuyển vị của đồ thị nén của $G$.

Chứng minh bị bỏ qua (nhưng đơn giản). Hệ quả của định lý này là sẽ không có cạnh nào từ thành phần "gốc" đến các thành phần khác trong đồ thị nén của $G^T$. Do đó, để thăm toàn bộ thành phần liên thông mạnh "gốc", chứa đỉnh $v$, ta chỉ cần chạy tìm kiếm

theo chiều sâu từ đỉnh $v$ trong đồ thị chuyển vị $G^T$! Điều này sẽ thăm chính xác tất cả các đỉnh của thành phần liên thông mạnh này. Như đã đề cập trước đó, sau đó ta có thể loại bỏ các đỉnh này khỏi đồ thị. Tiếp theo, ta tìm đỉnh tiếp theo có giá trị $t_{\text{out}}[v]$ tối đa, và chạy tìm kiếm trong đồ thị chuyển vị bắt đầu từ đỉnh đó để tìm thành phần liên thông mạnh tiếp theo. Lặp lại điều này, ta tìm được tất cả các thành phần liên thông mạnh.

Tóm lại, ta đã thảo luận thuật toán sau để tìm các thành phần liên thông mạnh:

- *Bước 1.* Chạy một loạt các tìm kiếm theo chiều sâu trên $G$, sẽ tạo ra một danh sách (ví dụ: thứ tự) các đỉnh, được sắp xếp theo thời gian thoát $t_{\text{out}}$ tăng dần.
- *Bước 2.* Xây dựng đồ thị chuyển vị $G^T$, và chạy một loạt các tìm kiếm theo chiều sâu trên các đỉnh theo thứ tự ngược lại (tức là theo thứ tự giảm dần của thời gian thoát). Mỗi lần tìm kiếm theo chiều sâu sẽ cho ra một thành phần liên thông mạnh.
- *Bước 3 (tùy chọn).* Xây dựng đồ thị nén.

Độ phức tạp thời gian chạy của thuật toán là $O(n + m)$, vì tìm kiếm theo chiều sâu được thực hiện hai lần. Việc xây dựng đồ thị nén cũng là $O(n + m)$.

Cuối cùng, cần đề cập đến sắp xếp tô-pô ở đây. Trong bước 1, ta tìm các đỉnh theo thứ tự tăng dần của thời gian thoát. Nếu $G$ không có chu trình, điều này tương ứng với một sắp xếp tô-pô ngược của $G$. Trong bước 2, thuật toán tìm các thành phần liên thông mạnh theo thứ tự giảm dần của thời gian thoát của chúng. Do đó, nó tìm các thành phần - các đỉnh của đồ thị nén - theo thứ tự tương ứng với một sắp xếp tô-pô của đồ thị nén.

### 3.7.3 Triển khai

```
vector<bool> visited; // theo dõi các đỉnh đã được thăm

// chạy tìm kiếm theo chiều sâu bắt đầu từ đỉnh v.
// mỗi đỉnh được thăm sẽ được thêm vào vector đầu ra khi dfs rời khỏi nó.
void dfs(int v, vector<vector<int>> const& adj, vector<int> &output) {
    visited[v] = true;
    for (auto u : adj[v])
        if (!visited[u])
            dfs(u, adj, output);
    output.push_back(v);
}

// đầu vào: adj -- danh sách kề của G
// đầu ra: components -- các thành phần liên thông mạnh trong G
// đầu ra: adj_cond -- danh sách kề của G^SCC (theo các đỉnh gốc)
void strongly_connected_components(vector<vector<int>> const& adj,
```

```
                                vector<vector<int>> &components,
                                vector<vector<int>> &adj_cond) {
    int n = adj.size();
    components.clear(), adj_cond.clear();

    vector<int> order; // sẽ là danh sách các đỉnh của G được sắp xếp theo thời gian t

    visited.assign(n, false);

    // loạt tìm kiếm theo chiều sâu đầu tiên
    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs(i, adj, order);

    // tạo danh sách kề của G^T
    vector<vector<int>> adj_rev(n);
    for (int v = 0; v < n; v++)
        for (int u : adj[v])
            adj_rev[u].push_back(v);

    visited.assign(n, false);
    reverse(order.begin(), order.end());

    vector<int> roots(n, 0); // cho biết đỉnh gốc của SCC của một đỉnh

    // loạt tìm kiếm theo chiều sâu thứ hai
    for (auto v : order)
        if (!visited[v]) {
            std::vector<int> component;
            dfs(v, adj_rev, component);
            components.push_back(component);
            int root = *min_element(begin(component), end(component));
            for (auto u : component)
                roots[u] = root;
        }

    // thêm các cạnh vào đồ thị nén
    adj_cond.assign(n, {});
    for (int v = 0; v < n; v++)
        for (auto u : adj[v])
```

```
            if (roots[v] != roots[u])
                adj_cond[roots[v]].push_back(roots[u]);
}
```

Hàm dfs triển khai tìm kiếm theo chiều sâu. Nó nhận đầu vào là một danh sách kề và một đỉnh bắt đầu. Nó cũng nhận tham chiếu đến vector output: mỗi đỉnh được thăm sẽ được thêm vào output khi dfs rời khỏi đỉnh đó.

Lưu ý rằng ta sử dụng hàm dfs cả trong bước đầu tiên và bước thứ hai của thuật toán. Trong bước đầu tiên, ta truyền vào danh sách kề của $G$, và trong các lần gọi dfs liên tiếp, ta tiếp tục truyền vào cùng vector order "đầu ra", để cuối cùng ta thu được một danh sách các đỉnh theo thứ tự tăng dần của thời gian thoát. Trong bước thứ hai, ta truyền vào danh sách kề của $G^T$, và trong mỗi lần gọi, ta truyền vào một vector component "đầu ra" trống, sẽ cho ta một thành phần liên thông mạnh tại một thời điểm.

Khi xây dựng danh sách kề của đồ thị nén, ta chọn gốc của mỗi thành phần là đỉnh đầu tiên trong danh sách các đỉnh của nó (đây là lựa chọn tùy ý). Đỉnh gốc này đại diện cho toàn bộ SCC của nó. Với mỗi đỉnh $v$, giá trị roots[$v$] chỉ ra đỉnh gốc của SCC mà $v$ thuộc về.

Đồ thị nén của chúng ta giờ đây được biểu diễn bởi các đỉnh components (một thành phần liên thông mạnh tương ứng với một đỉnh trong đồ thị nén), và danh sách kề được cho bởi adj_cond, chỉ sử dụng các đỉnh gốc của các thành phần liên thông mạnh. Lưu ý rằng ta tạo một cạnh từ $C$ đến $C'$ trong $G^{\text{SCC}}$ cho mỗi cạnh từ một $a \in C$ đến một $b \in C'$ trong $G$ (nếu $C \neq C'$). Điều này ngụ ý rằng trong triển khai của chúng ta, có thể có nhiều cạnh giữa hai thành phần trong đồ thị nén.

### 3.7.4 Tài liệu tham khảo


- Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein. *Introduction to Algorithms* [2005].

- M. Sharir. *A strong-connectivity algorithm and its applications in data-flow analysis* [1979].


## 3.8 Định hướng mạnh

*Định hướng mạnh* của một đồ thị không có hướng là việc gán một hướng cho mỗi cạnh sao cho đồ thị trở thành một đồ thị liên thông mạnh. Tức là, sau khi định hướng, ta có thể thăm bất kỳ đỉnh nào từ bất kỳ đỉnh nào bằng cách đi theo các cạnh có hướng.

### 3.8.1 Giải pháp

Tất nhiên, điều này không thể thực hiện được với mọi đồ thị. Hãy xem xét một cây cầu trong đồ thị. Ta phải gán một hướng cho nó và bằng cách làm như vậy, ta khiến cây cầu này chỉ "có thể vượt qua" theo một hướng. Điều đó có nghĩa là ta không thể đi từ một đầu của cây cầu đến đầu kia, vì vậy ta không thể khiến đồ thị trở thành liên thông mạnh.

Bây giờ, hãy xem xét một DFS (tìm kiếm theo chiều sâu) qua một đồ thị liên thông không có cầu. Rõ ràng, ta sẽ thăm được mỗi đỉnh. Và vì không có cầu, ta có thể loại bỏ bất kỳ cạnh cây DFS nào và vẫn có thể đi từ dưới cạnh lên trên cạnh bằng cách sử dụng một đường đi chứa ít nhất một cạnh ngược. Từ đó suy ra rằng từ bất kỳ đỉnh nào, ta có thể đi đến gốc của cây DFS. Ngoài ra, từ gốc của cây DFS, ta có thể thăm bất kỳ đỉnh nào ta chọn. Ta đã tìm ra một định hướng mạnh!

Nói cách khác, để định hướng mạnh một đồ thị liên thông không có cầu, hãy chạy DFS trên nó và để các cạnh cây DFS hướng ra khỏi gốc DFS, còn tất cả các cạnh khác từ hậu duệ đến tổ tiên trong cây DFS.

Kết quả rằng các đồ thị liên thông không có cầu chính là các đồ thị có định hướng mạnh được gọi là *định lý Robbins*.

### 3.8.2 Mở rộng bài toán

Hãy xem xét bài toán tìm một định hướng của đồ thị sao cho số lượng thành phần liên thông mạnh (SCC) là nhỏ nhất.

Tất nhiên, mỗi thành phần của đồ thị có thể được xem xét riêng lẻ. Bây giờ, vì chỉ các đồ thị không có cầu mới có thể được định hướng mạnh, hãy tạm thời loại bỏ tất cả các cầu. Ta sẽ thu được một số thành phần không có cầu (chính xác là số lượng thành phần ban đầu cộng với số lượng cầu), và ta biết rằng ta có thể định hướng mạnh từng thành phần trong số chúng.

Ta chỉ được phép định hướng các cạnh, không được loại bỏ chúng, nhưng hóa ra ta có thể định hướng các cầu một cách tùy ý. Tất nhiên, cách dễ nhất để định hướng chúng là chạy thuật toán được mô tả ở trên mà không cần sửa đổi trên mỗi thành phần liên thông ban đầu.

### 3.8.3 Triển khai

Ở đây, đầu vào là $n$ — số lượng đỉnh, $m$ — số lượng cạnh, sau đó là $m$ dòng mô tả các cạnh.

Đầu ra là số lượng SCC tối thiểu trên dòng đầu tiên và trên dòng thứ hai là một chuỗi gồm $m$ ký tự, hoặc là $>$ — cho biết cạnh tương ứng từ đầu vào được định hướng từ đỉnh trái sang đỉnh phải (như trong đầu vào), hoặc $<$ — ngược lại.

Đây là một thuật toán tìm cầu được sửa đổi để đồng thời định hướng các cạnh, bạn cũng có thể định hướng các cạnh như một bước đầu tiên và đếm các SCC trên đồ thị đã

định hướng như một bước thứ hai.

```
vector<vector<pair<int, int>>> adj; // danh sách kề - cặp đỉnh và cạnh
vector<pair<int, int>> edges;

vector<int> tin, low;
int bridge_cnt;
string orient;
vector<bool> edge_used;
void find_bridges(int v) {
    static int time = 0;
    low[v] = tin[v] = time++;
    for (auto p : adj[v]) {
        if (edge_used[p.second]) continue;
        edge_used[p.second] = true;
        orient[p.second] = v == edges[p.second].first ? '>' : '<';
        int nv = p.first;
        if (tin[nv] == -1) { // nếu nv chưa được thăm
            find_bridges(nv);
            low[v] = min(low[v], low[nv]);
            if (low[nv] > tin[v]) {
                // một cây cầu giữa v và nv
                bridge_cnt++;
            }
        } else {
            low[v] = min(low[v], low[nv]);
        }
    }
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    adj.resize(n);
    tin.resize(n, -1);
    low.resize(n, -1);
    orient.resize(m);
    edges.resize(m);
    edge_used.resize(m);
    for (int i = 0; i < m; i++) {
        int a, b;
```

```
        scanf("%d %d", &a, &b);
        a--; b--;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
        edges[i] = {a, b};
    }
    int comp_cnt = 0;
    for (int v = 0; v < n; v++) {
        if (tin[v] == -1) {
            comp_cnt++;
            find_bridges(v);
        }
    }
    printf("%d\n%s\n", comp_cnt + bridge_cnt, orient.c_str());
}
```

## 3.9 Thuật toán Dijkstra

Cho một đồ thị có hướng hoặc không hướng có trọng số với $n$ đỉnh và $m$ cạnh. Trọng số của tất cả các cạnh là không âm. Bạn cũng được cho một đỉnh bắt đầu $s$. Bài viết này thảo luận về việc tìm độ dài của các đường đi ngắn nhất từ đỉnh bắt đầu $s$ đến tất cả các đỉnh khác, và xuất ra chính các đường đi ngắn nhất đó.

Bài toán này còn được gọi là bài toán đường đi ngắn nhất từ một nguồn.

### 3.9.1 Thuật toán

Dưới đây là thuật toán được mô tả bởi nhà khoa học máy tính người Hà Lan Edsger W. Dijkstra vào năm 1959.

Hãy tạo một mảng $d[]$ trong đó với mỗi đỉnh $v$, ta lưu độ dài hiện tại của đường đi ngắn nhất từ $s$ đến $v$ trong $d[v]$. Ban đầu, $d[s] = 0$, và với tất cả các đỉnh khác, độ dài này bằng vô cực. Trong triển khai, một số đủ lớn (được đảm bảo lớn hơn bất kỳ độ dài đường đi nào có thể xảy ra) được chọn làm vô cực:

$$d[v] = \infty,\ v \neq s$$

Ngoài ra, ta duy trì một mảng Boolean $u[]$ lưu trữ cho mỗi đỉnh $v$ liệu nó đã được đánh dấu hay chưa. Ban đầu, tất cả các đỉnh đều chưa được đánh dấu:

$$u[v] = \text{false}$$

Thuật toán Dijkstra chạy trong $n$ lần lặp. Ở mỗi lần lặp, một đỉnh $v$ được chọn là đỉnh chưa được đánh dấu có giá trị $d[v]$ nhỏ nhất:

Rõ ràng, trong lần lặp đầu tiên, đỉnh bắt đầu $s$ sẽ được chọn.

Đỉnh được chọn $v$ được đánh dấu. Tiếp theo, từ đỉnh $v$, các phép thư giãn (relaxation) được thực hiện: tất cả các cạnh dạng $(v, \text{to})$ được xem xét, và với mỗi đỉnh to, thuật toán cố gắng cải thiện giá trị $d[\text{to}]$. Nếu độ dài của cạnh hiện tại bằng $len$, mã cho phép thư giãn là:

$$d[\text{to}] = \min(d[\text{to}], d[v] + len)$$

Sau khi tất cả các cạnh như vậy được xem xét, lần lặp hiện tại kết thúc. Cuối cùng, sau $n$ lần lặp, tất cả các đỉnh sẽ được đánh dấu, và thuật toán kết thúc. Ta khẳng định rằng các giá trị $d[v]$ tìm được là độ dài của các đường đi ngắn nhất từ $s$ đến tất cả các đỉnh $v$.

Lưu ý rằng nếu một số đỉnh không thể tiếp cận từ đỉnh bắt đầu $s$, các giá trị $d[v]$ cho chúng sẽ vẫn là vô cực. Rõ ràng, vài lần lặp cuối cùng của thuật toán sẽ chọn những đỉnh này, nhưng không có công việc hữu ích nào được thực hiện cho chúng. Do đó, thuật toán có thể được dừng lại ngay khi đỉnh được chọn có khoảng cách vô cực đến nó.

### 3.9.2 Khôi phục đường đi ngắn nhất

Thường thì ta không chỉ cần biết độ dài của các đường đi ngắn nhất mà còn cần chính các đường đi ngắn nhất đó. Hãy xem cách duy trì đủ thông tin để khôi phục đường đi ngắn nhất từ $s$ đến bất kỳ đỉnh nào. Ta sẽ duy trì một mảng tiền tố $p[]$ trong đó với mỗi đỉnh $v \neq s$, $p[v]$ là đỉnh áp chót trong đường đi ngắn nhất từ $s$ đến $v$. Ở đây ta sử dụng thực tế rằng nếu ta lấy đường đi ngắn nhất đến một đỉnh $v$ nào đó và loại bỏ $v$ khỏi đường đi này, ta sẽ nhận được một đường đi kết thúc tại đỉnh $p[v]$, và đường đi này sẽ là ngắn nhất cho đỉnh $p[v]$. Mảng tiền tố này có thể được sử dụng để khôi phục đường đi ngắn nhất đến bất kỳ đỉnh nào: bắt đầu từ $v$, lặp lại việc lấy tiền tố của đỉnh hiện tại cho đến khi ta đến được đỉnh bắt đầu $s$ để nhận được đường đi ngắn nhất cần thiết với các đỉnh được liệt kê theo thứ tự ngược lại. Vì vậy, đường đi ngắn nhất $P$ đến đỉnh $v$ bằng:

$$P = (s, \ldots, p[p[p[v]]], p[p[v]], p[v], v)$$

Việc xây dựng mảng tiền tố này rất đơn giản: với mỗi lần thư giãn thành công, tức là khi với một đỉnh được chọn $v$, có sự cải thiện khoảng cách đến một đỉnh to, ta cập nhật đỉnh tiền tố cho to bằng đỉnh $v$:

$$p[\text{to}] = v$$

### 3.9.3 Chứng minh

Khẳng định chính mà tính đúng đắn của thuật toán Dijkstra dựa vào là như sau:

Sau khi bất kỳ đỉnh $v$ nào được đánh dấu, khoảng cách hiện tại đến nó $d[v]$ là ngắn nhất, và sẽ không thay đổi nữa.

Chứng minh được thực hiện bằng quy nạp. Đối với lần lặp đầu tiên, khẳng định này là hiển nhiên: đỉnh duy nhất được đánh dấu là $s$, và khoảng cách đến nó $d[s] = 0$ thực sự là độ dài của đường đi ngắn nhất đến $s$. Bây giờ, giả sử khẳng định này đúng cho tất cả các lần lặp trước đó, tức là cho tất cả các đỉnh đã được đánh dấu; hãy chứng minh rằng nó không bị vi phạm sau khi lần lặp hiện tại hoàn tất. Gọi $v$ là đỉnh được chọn trong lần lặp hiện tại, tức là $v$ là đỉnh mà thuật toán sẽ đánh dấu. Bây giờ ta phải chứng minh rằng $d[v]$ thực sự bằng độ dài của đường đi ngắn nhất đến nó $l[v]$.

Xét đường đi ngắn nhất $P$ đến đỉnh $v$. Đường đi này có thể được chia thành hai phần: $P_1$ chỉ bao gồm các nút đã được đánh dấu (ít nhất đỉnh bắt đầu $s$ là một phần của $P_1$), và phần còn lại của đường đi $P_2$ (nó có thể bao gồm một đỉnh đã được đánh dấu, nhưng luôn bắt đầu bằng một đỉnh chưa được đánh dấu). Gọi đỉnh đầu tiên của đường đi $P_2$ là $p$, và đỉnh cuối cùng của đường đi $P_1$ là $q$.

Trước tiên, ta chứng minh khẳng định của mình cho đỉnh $p$, tức là hãy chứng minh rằng $d[p] = l[p]$. Điều này gần như hiển nhiên: trong một trong các lần lặp trước, ta đã chọn đỉnh $q$ và thực hiện thư giãn từ nó. Vì (theo cách chọn đỉnh $p$) đường đi ngắn nhất đến $p$ là đường đi ngắn nhất đến $q$ cộng với cạnh $(p, q)$, phép thư giãn từ $q$ đã đặt giá trị của $d[p]$ thành độ dài của đường đi ngắn nhất $l[p]$.

Vì trọng số của các cạnh là không âm, độ dài của đường đi ngắn nhất $l[p]$ (mà ta vừa chứng minh bằng $d[p]$) không vượt quá độ dài $l[v]$ của đường đi ngắn nhất đến đỉnh $v$. Với điều kiện $l[v] \leq d[v]$ (vì thuật toán Dijkstra không thể tìm ra một cách ngắn hơn đường đi ngắn nhất có thể), ta có bất đẳng thức:

$$d[p] = l[p] \leq l[v] \leq d[v]$$

Mặt khác, vì cả hai đỉnh $p$ và $v$ đều chưa được đánh dấu, và lần lặp hiện tại đã chọn đỉnh $v$, không phải $p$, ta có một bất đẳng thức khác:

$$d[p] \geq d[v]$$

Từ hai bất đẳng thức này, ta kết luận rằng $d[p] = d[v]$, và sau đó từ các phương trình đã tìm được trước đó, ta có:

$$d[v] = l[v]$$

Điều này hoàn tất chứng minh.

### 3.9.4 Triển khai

Thuật toán Dijkstra thực hiện $n$ lần lặp. Ở mỗi lần lặp, nó chọn một đỉnh chưa được đánh dấu $v$ với giá trị $d[v]$ nhỏ nhất, đánh dấu nó và kiểm tra tất cả các cạnh $(v, \text{to})$ để cố gắng cải thiện giá trị $d[\text{to}]$.

Thời gian chạy của thuật toán bao gồm:

- $n$ lần tìm kiếm một đỉnh với giá trị $d[v]$ nhỏ nhất trong số $O(n)$ đỉnh chưa được đánh dấu,
- $m$ lần thử thư giãn.

Với triển khai đơn giản nhất của các thao tác này, ở mỗi lần lặp, việc tìm kiếm đỉnh yêu cầu $O(n)$ thao tác, và mỗi phép thư giãn có thể được thực hiện trong $O(1)$. Do đó, độ phức tạp tiệm cận kết quả của thuật toán là:

$$O(n^2 + m)$$

Độ phức tạp này là tối ưu cho đồ thị dày, tức là khi $m \approx n^2$. Tuy nhiên, trong các đồ thị thưa, khi $m$ nhỏ hơn nhiều so với số lượng cạnh tối đa $n^2$, bài toán có thể được giải trong độ phức tạp $O(n \log n + m)$. Thuật toán và triển khai có thể được tìm thấy trong bài viết *Dijkstra trên đồ thị thưa*.

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);
    vector<bool> u(n, false);

    d[s] = 0;
    for (int i = 0; i < n; i++) {
        int v = -1;
        for (int j = 0; j < n; j++) {
            if (!u[j] && (v == -1 || d[j] < d[v]))
                v = j;
        }

        if (d[v] == INF)
            break;

        u[v] = true;
        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
```

```
                d[to] = d[v] + len;
                p[to] = v;
            }
        }
    }
}
```

Ở đây, đồ thị adj được lưu trữ dưới dạng danh sách kề: với mỗi đỉnh $v$, adj[$v$] chứa danh sách các cạnh đi từ đỉnh này, tức là danh sách các cặp pair<int,int> trong đó phần tử đầu tiên trong cặp là đỉnh ở đầu kia của cạnh, và phần tử thứ hai là trọng số của cạnh.

Hàm nhận đỉnh bắt đầu $s$ và hai vector sẽ được sử dụng làm giá trị trả về.

Trước hết, mã khởi tạo các mảng: khoảng cách $d[]$, nhãn $u[]$ và tiền tố $p[]$. Sau đó, nó thực hiện $n$ lần lặp. Ở mỗi lần lặp, đỉnh $v$ được chọn có khoảng cách $d[v]$ nhỏ nhất trong số tất cả các đỉnh chưa được đánh dấu. Nếu khoảng cách đến đỉnh được chọn $v$ bằng vô cực, thuật toán dừng lại. Nếu không, đỉnh được đánh dấu, và tất cả các cạnh đi ra từ đỉnh này được kiểm tra. Nếu phép thư giãn dọc theo cạnh là khả thi (tức là khoảng cách $d$[to] có thể được cải thiện), khoảng cách $d$[to] và tiền tố $p$[to] được cập nhật.

Sau khi thực hiện tất cả các lần lặp, mảng $d[]$ lưu trữ độ dài của các đường đi ngắn nhất đến tất cả các đỉnh, và mảng $p[]$ lưu trữ tiền tố của tất cả các đỉnh (ngoại trừ đỉnh bắt đầu $s$). Đường đi đến bất kỳ đỉnh $t$ nào có thể được khôi phục theo cách sau:

```
vector<int> restore_path(int s, int t, vector<int> const& p) {
    vector<int> path;

    for (int v = t; v != s; v = p[v])
        path.push_back(v);
    path.push_back(s);

    reverse(path.begin(), path.end());
    return path;
}
```

### 3.9.5 Tài liệu tham khảo


- Edsger Dijkstra. *A note on two problems in connexion with graphs* [1959]

- Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein. *Introduction to Algorithms* [2005]

## 3.10 Dijkstra trên đồ thị thưa

Để biết phát biểu bài toán, thuật toán với triển khai và chứng minh, bạn có thể tham khảo bài viết về thuật toán Dijkstra.

### 3.10.1 Thuật toán

Ta nhớ lại trong quá trình suy ra độ phức tạp của thuật toán Dijkstra, ta đã sử dụng hai yếu tố: thời gian tìm đỉnh chưa được đánh dấu với khoảng cách nhỏ nhất $d[v]$, và thời gian thực hiện phép thư giãn, tức là thời gian thay đổi các giá trị $d[\text{to}]$.

Trong triển khai đơn giản nhất, các thao tác này yêu cầu thời gian $O(n)$ và $O(1)$. Do đó, vì ta thực hiện thao tác đầu tiên $O(n)$ lần, và thao tác thứ hai $O(m)$ lần, ta thu được độ phức tạp $O(n^2 + m)$.

Rõ ràng, độ phức tạp này là tối ưu cho đồ thị dày, tức là khi $m \approx n^2$. Tuy nhiên, trong các đồ thị thưa, khi $m$ nhỏ hơn nhiều so với số lượng cạnh tối đa $n^2$, độ phức tạp trở nên kém tối ưu do hạng tử đầu tiên. Vì vậy, cần cải thiện thời gian thực thi của thao tác đầu tiên (và tất nhiên không ảnh hưởng quá nhiều đến thao tác thứ hai).

Để đạt được điều đó, ta có thể sử dụng một biến thể của nhiều cấu trúc dữ liệu phụ trợ. Hiệu quả nhất là *Fibonacci heap*, cho phép thao tác đầu tiên chạy trong $O(\log n)$, và thao tác thứ hai trong $O(1)$. Do đó, ta sẽ thu được độ phức tạp $O(n \log n + m)$ cho thuật toán Dijkstra, đây cũng là mức tối thiểu lý thuyết cho bài toán tìm đường đi ngắn nhất. Vì vậy, thuật toán này hoạt động tối ưu, và Fibonacci heap là cấu trúc dữ liệu tối ưu. Không tồn tại cấu trúc dữ liệu nào có thể thực hiện cả hai thao tác trong $O(1)$, vì điều này cũng sẽ cho phép sắp xếp một danh sách các số ngẫu nhiên trong thời gian tuyến tính, điều không thể xảy ra. Thú vị là tồn tại một thuật toán của Thorup tìm đường đi ngắn nhất trong thời gian $O(m)$, tuy nhiên chỉ hoạt động với trọng số nguyên và sử dụng một ý tưởng hoàn toàn khác. Vì vậy, điều này không dẫn đến bất kỳ mâu thuẫn nào. Fibonacci heap cung cấp độ phức tạp tối ưu cho bài toán này. Tuy nhiên, chúng khá phức tạp để triển khai và cũng có một hằng số ẩn khá lớn.

Như một sự thỏa hiệp, bạn có thể sử dụng các cấu trúc dữ liệu thực hiện cả hai loại thao tác (trích xuất phần tử nhỏ nhất và cập nhật một phần tử) trong $O(\log n)$. Khi đó, độ phức tạp của thuật toán Dijkstra là:

$$O(n \log n + m \log n) = O(m \log n)$$

C++ cung cấp hai cấu trúc dữ liệu như vậy: set và priority_queue. Cấu trúc đầu tiên dựa trên cây đỏ-đen, và cấu trúc thứ hai dựa trên heap. Do đó, priority_queue có hằng số ẩn nhỏ hơn, nhưng cũng có nhược điểm: nó không hỗ trợ thao tác xóa một phần tử. Vì điều này, ta cần thực hiện một "giải pháp tạm thời", điều này thực tế dẫn đến một yếu tố hơi tệ hơn là $\log m$ thay vì $\log n$ (mặc dù về mặt độ phức tạp, chúng là tương đương).

### 3.10.2 Triển khai

**Sử dụng set**

Hãy bắt đầu với container set. Vì ta cần lưu trữ các đỉnh được sắp xếp theo giá trị $d[]$, việc lưu trữ các cặp thực tế là thuận tiện: khoảng cách và chỉ số của đỉnh. Kết quả là trong một set, các cặp được tự động sắp xếp theo khoảng cách của chúng.

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                q.erase({d[to], to});
                d[to] = d[v] + len;
                p[to] = v;
                q.insert({d[to], to});
            }
        }
    }
}
```

Ta không cần mảng $u[]$ từ triển khai thuật toán Dijkstra thông thường nữa. Ta sẽ sử dụng set để lưu trữ thông tin đó, và cũng tìm đỉnh có khoảng cách ngắn nhất với nó. Nó hoạt động giống như một hàng đợi. Vòng lặp chính thực thi cho đến khi không còn đỉnh nào trong set/$hng\emptyset i$. Một đỉnh với khoảng cách nhỏ nhất được trích xuất, và với mỗi phép thư giãn thành công, ta trước tiên xóa cặp cũ, sau đó sau khi thư giãn, thêm cặp mới vào hàng đợi.

**Sử dụng priority_queue**

Sự khác biệt chính so với triển khai bằng set là trong nhiều ngôn ngữ, bao gồm C++, ta không thể xóa phần tử khỏi priority_queue (mặc dù về lý thuyết heap có thể hỗ trợ thao tác đó). Do đó, ta phải sử dụng một giải pháp tạm thời: Ta đơn giản là không xóa cặp cũ khỏi hàng đợi. Kết quả là một đỉnh có thể xuất hiện nhiều lần với các khoảng cách khác nhau trong hàng đợi cùng một lúc. Trong số các cặp này, ta chỉ quan tâm đến các cặp mà phần tử đầu tiên bằng với giá trị tương ứng trong $d[]$, tất cả các cặp khác là cũ. Vì vậy, ta cần thực hiện một sửa đổi nhỏ: ở đầu mỗi lần lặp, sau khi trích xuất cặp tiếp theo, ta kiểm tra xem đó có phải là cặp quan trọng hay là cặp đã cũ và đã được xử lý. Kiểm tra này rất quan trọng, nếu không độ phức tạp có thể tăng lên đến $O(nm)$.

Theo mặc định, priority_queue sắp xếp các phần tử theo thứ tự giảm dần. Để khiến nó sắp xếp các phần tử theo thứ tự tăng dần, ta có thể lưu trữ các khoảng cách âm trong đó, hoặc truyền cho nó một hàm sắp xếp khác. Ta sẽ thực hiện tùy chọn thứ hai.

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>> q;
    q.push({0, s});
    while (!q.empty()) {
        int v = q.top().second;
        int d_v = q.top().first;
        q.pop();
        if (d_v != d[v])
            continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
```

```
                q.push({d[to], to});
            }
        }
    }
}
```

Trong thực tế, phiên bản priority_queue nhanh hơn một chút so với phiên bản dùng set.

Thú vị là, một báo cáo kỹ thuật năm 2007 kết luận rằng biến thể của thuật toán không sử dụng thao tác giảm khóa (decrease-key) chạy nhanh hơn biến thể dùng giảm khóa, với khoảng cách hiệu suất lớn hơn đối với đồ thị thưa.

#### Loại bỏ các cặp

Bạn có thể cải thiện hiệu suất thêm một chút nếu không lưu trữ các cặp trong các container, mà chỉ lưu trữ chỉ số của các đỉnh. Trong trường hợp này, ta phải nạp chồng toán tử so sánh: nó phải so sánh hai đỉnh bằng cách sử dụng các khoảng cách được lưu trong $d[]$.

Kết quả của phép thư giãn, khoảng cách của một số đỉnh sẽ thay đổi. Tuy nhiên, cấu trúc dữ liệu sẽ không tự động sắp xếp lại chính nó. Thực tế, việc thay đổi khoảng cách của các đỉnh trong hàng đợi có thể phá hủy cấu trúc dữ liệu. Như trước đây, ta cần xóa đỉnh trước khi thư giãn nó, và sau đó chèn lại nó.

Vì ta chỉ có thể xóa từ set, tối ưu hóa này chỉ áp dụng được cho phương pháp set, và không hoạt động với triển khai priority_queue. Trong thực tế, điều này tăng hiệu suất đáng kể, đặc biệt khi các kiểu dữ liệu lớn hơn được sử dụng để lưu trữ khoảng cách, như long long hoặc double.

## 3.11 Thuật toán Bellman-Ford

### 3.11.1 Đường đi ngắn nhất từ một nguồn với cạnh có trọng số âm

Giả sử ta được cho một đồ thị có hướng có trọng số $G$ với $n$ đỉnh và $m$ cạnh, và một đỉnh xác định $v$. Bạn muốn tìm độ dài của các đường đi ngắn nhất từ đỉnh $v$ đến mọi đỉnh khác.

Không giống như thuật toán Dijkstra, thuật toán này cũng có thể được áp dụng cho các đồ thị chứa các cạnh có trọng số âm. Tuy nhiên, nếu đồ thị chứa một chu trình có trọng số âm, thì rõ ràng, đường đi ngắn nhất đến một số đỉnh có thể không tồn tại (do thực tế rằng trọng số của đường đi ngắn nhất phải bằng âm vô cực); tuy nhiên, thuật toán này có thể được sửa đổi để báo hiệu sự hiện diện của một chu trình có trọng số âm, hoặc thậm chí suy ra chu trình này.

Thuật toán mang tên của hai nhà khoa học người Mỹ: Richard Bellman và Lester Ford. Ford thực sự đã phát minh ra thuật toán này vào năm 1956 trong khi nghiên cứu một bài toán toán học khác, cuối cùng được rút gọn thành một bài toán con của việc tìm đường đi ngắn nhất trong đồ thị, và Ford đã đưa ra một phác thảo của thuật toán để giải quyết bài toán này. Bellman vào năm 1958 đã xuất bản một bài báo dành riêng cho bài toán tìm đường đi ngắn nhất, và trong bài báo này, ông đã định hình rõ ràng thuật toán dưới dạng mà chúng ta biết ngày nay.

### 3.11.2 Mô tả thuật toán

Giả sử rằng đồ thị không chứa chu trình có trọng số âm. Trường hợp có chu trình có trọng số âm sẽ được thảo luận dưới đây trong một phần riêng.

Ta sẽ tạo một mảng khoảng cách $d[0 \dots n-1]$, mà sau khi thực thi thuật toán sẽ chứa câu trả lời cho bài toán. Ban đầu, ta điền nó như sau: $d[v] = 0$, và tất cả các phần tử khác của $d[]$ bằng vô cực $\infty$.

Thuật toán bao gồm nhiều giai đoạn. Mỗi giai đoạn quét qua tất cả các cạnh của đồ thị, và thuật toán cố gắng thực hiện phép thư giãn dọc theo mỗi cạnh $(a, b)$ có trọng số $c$. Phép thư giãn dọc theo các cạnh là một nỗ lực để cải thiện giá trị $d[b]$ bằng cách sử dụng giá trị $d[a] + c$. Thực tế, điều này có nghĩa là ta đang cố gắng cải thiện câu trả lời cho đỉnh này bằng cách sử dụng cạnh $(a, b)$ và câu trả lời hiện tại cho đỉnh $a$.

Ta khẳng định rằng $n-1$ giai đoạn của thuật toán là đủ để tính toán chính xác độ dài của tất cả các đường đi ngắn nhất trong đồ thị (một lần nữa, ta tin rằng các chu trình có trọng số âm không tồn tại). Đối với các đỉnh không thể tiếp cận, khoảng cách $d[]$ sẽ vẫn bằng vô cực $\infty$.

### 3.11.3 Triển khai

Không giống như nhiều thuật toán đồ thị khác, đối với thuật toán Bellman-Ford, việc biểu diễn đồ thị bằng một danh sách duy nhất của tất cả các cạnh (thay vì $n$ danh sách các cạnh - các cạnh từ mỗi đỉnh) là thuận tiện hơn. Ta bắt đầu triển khai với một cấu trúc Edge để biểu diễn các cạnh. Đầu vào của thuật toán là các số $n$, $m$, danh sách $e$ của các cạnh và đỉnh bắt đầu $v$. Tất cả các đỉnh được đánh số từ 0 đến $n-1$.

#### Triển khai đơn giản nhất

Hằng số INF biểu thị số "vô cực" — nó nên được chọn sao cho lớn hơn tất cả các độ dài đường đi có thể xảy ra.

```
struct Edge {
    int a, b, cost;
};
```

```
int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (Edge e : edges)
            if (d[e.a] < INF)
                d[e.b] = min(d[e.b], d[e.a] + e.cost);
    // hiển thị d, ví dụ, trên màn hình
}
```

Kiểm tra if (d[e.a] $<$ INF) chỉ cần thiết nếu đồ thị chứa các cạnh có trọng số âm: nếu không có kiểm tra này sẽ dẫn đến phép thư giãn từ các đỉnh mà đường đi chưa được tìm thấy, và khoảng cách không chính xác, kiểu như $\infty - 1$, $\infty - 2$, v.v., sẽ xuất hiện.

### Triển khai tốt hơn

Thuật toán này có thể được tăng tốc một chút: thường thì ta đã nhận được câu trả lời trong một vài giai đoạn và không có công việc hữu ích nào được thực hiện trong các giai đoạn còn lại, chỉ lãng phí thời gian thăm tất cả các cạnh. Vì vậy, hãy giữ một cờ để cho biết liệu có điều gì thay đổi trong giai đoạn hiện tại hay không, và nếu trong bất kỳ giai đoạn nào không có gì thay đổi, thuật toán có thể dừng lại. (Tối ưu hóa này không cải thiện hành vi tiệm cận, tức là một số đồ thị vẫn sẽ cần tất cả $n-1$ giai đoạn, nhưng tăng tốc đáng kể hành vi của thuật toán "trung bình", tức là trên các đồ thị ngẫu nhiên.)

Với tối ưu hóa này, thường không cần thiết phải giới hạn thủ công số giai đoạn của thuật toán ở $n - 1$ — thuật toán sẽ dừng lại sau số giai đoạn mong muốn.

```
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (;;) {
        bool any = false;

        for (Edge e : edges)
            if (d[e.a] < INF)
```

```
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    any = true;
                }

        if (!any)
            break;
    }
    // hiển thị d, ví dụ, trên màn hình
}
```

### Khôi phục đường đi

Bây giờ hãy xem xét cách sửa đổi thuật toán để nó không chỉ tìm độ dài của các đường đi ngắn nhất mà còn cho phép tái tạo các đường đi ngắn nhất.

Để làm điều đó, hãy tạo một mảng khác $p[0 \dots n-1]$, trong đó với mỗi đỉnh, ta lưu trữ "tiền tố" của nó, tức là đỉnh áp chót trong đường đi ngắn nhất dẫn đến nó. Thực tế, đường đi ngắn nhất đến bất kỳ đỉnh $a$ nào là đường đi ngắn nhất đến một đỉnh $p[a]$ nào đó, mà ta đã thêm $a$ vào cuối đường đi.

Lưu ý rằng thuật toán hoạt động trên cùng một logic: nó giả định rằng khoảng cách ngắn nhất đến một đỉnh đã được tính toán, và cố gắng cải thiện khoảng cách ngắn nhất đến các đỉnh khác từ đỉnh đó. Do đó, tại thời điểm cải thiện, ta chỉ cần nhớ $p[]$, tức là đỉnh mà từ đó sự cải thiện này đã xảy ra.

Dưới đây là triển khai của Bellman-Ford với việc khôi phục đường đi ngắn nhất đến một nút đã cho $t$:

```
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

    for (;;) {
        bool any = false;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    p[e.b] = e.a;
                    any = true;
                }
```

```
        if (!any)
            break;
    }

    if (d[t] == INF)
        cout << "Không có đường đi từ " << v << " đến " << t << ".";
    else {
        vector<int> path;
        for (int cur = t; cur != -1; cur = p[cur])
            path.push_back(cur);
        reverse(path.begin(), path.end());

        cout << "Đường đi từ " << v << " đến " << t << ": ";
        for (int u : path)
            cout << u << ' ';
    }
}
```

Ở đây, bắt đầu từ đỉnh $t$, ta đi qua các tiền tố cho đến khi đến được đỉnh bắt đầu không có tiền tố, và lưu tất cả các đỉnh trong đường đi vào danh sách path. Danh sách này là đường đi ngắn nhất từ $v$ đến $t$, nhưng theo thứ tự ngược lại, vì vậy ta gọi hàm reverse() trên path và sau đó xuất đường đi.

### 3.11.4 Chứng minh thuật toán

Đầu tiên, lưu ý rằng đối với tất cả các đỉnh không thể tiếp cận $u$, thuật toán sẽ hoạt động chính xác, nhãn $d[u]$ sẽ vẫn bằng vô cực (vì thuật toán Bellman-Ford sẽ tìm một số đường đi đến tất cả các đỉnh có thể tiếp cận từ đỉnh bắt đầu $v$, và phép thư giãn cho tất cả các đỉnh còn lại sẽ không bao giờ xảy ra).

Bây giờ ta chứng minh khẳng định sau: Sau khi thực thi giai đoạn thứ $i$, thuật toán Bellman-Ford tính toán chính xác tất cả các đường đi ngắn nhất có số cạnh không vượt quá $i$.

Nói cách khác, với bất kỳ đỉnh $a$ nào, gọi $k$ là số cạnh trong đường đi ngắn nhất đến nó (nếu có nhiều đường đi như vậy, bạn có thể chọn bất kỳ đường nào). Theo khẳng định này, thuật toán đảm bảo rằng sau giai đoạn thứ $k$, đường đi ngắn nhất cho đỉnh $a$ sẽ được tìm thấy.

**Chứng minh:** Xét một đỉnh bất kỳ $a$ mà có đường đi từ đỉnh bắt đầu $v$ đến nó, và xem xét một đường đi ngắn nhất đến nó ($p_0 = v, p_1, \ldots, p_k = a$). Trước giai đoạn đầu tiên, đường đi ngắn nhất đến đỉnh $p_0 = v$ đã được tìm thấy chính xác. Trong giai đoạn đầu tiên, cạnh $(p_0, p_1)$ đã được thuật toán kiểm tra, và do đó, khoảng cách đến đỉnh $p_1$ đã được tính toán chính xác sau giai đoạn đầu tiên. Lặp lại khẳng định này $k$ lần, ta thấy

rằng sau giai đoạn thứ $k$, khoảng cách đến đỉnh $p_k = a$ được tính toán chính xác, điều mà ta muốn chứng minh.

Điều cuối cùng cần lưu ý là bất kỳ đường đi ngắn nhất nào cũng không thể có quá $n-1$ cạnh. Do đó, thuật toán đủ đi đến giai đoạn thứ $(n-1)$. Sau đó, được đảm bảo rằng không có phép thư giãn nào sẽ cải thiện khoảng cách đến một số đỉnh.

### 3.11.5 Trường hợp có chu trình âm

Ở khắp nơi trên đây, ta đã xem xét rằng không có chu trình có trọng số âm trong đồ thị (cụ thể, ta quan tâm đến một chu trình có trọng số âm có thể tiếp cận từ đỉnh bắt đầu $v$, và đối với các chu trình không thể tiếp cận, không có gì thay đổi trong thuật toán trên). Khi có một hoặc nhiều chu trình âm, có thêm các phức tạp liên quan đến thực tế rằng khoảng cách đến tất cả các đỉnh trong chu trình này, cũng như các đỉnh có thể tiếp cận từ chu trình này, không được xác định — chúng nên bằng âm vô cực $(-\infty)$.

Dễ thấy rằng thuật toán Bellman-Ford có thể thực hiện phép thư giãn vô tận giữa tất cả các đỉnh của chu trình này và các đỉnh có thể tiếp cận từ nó. Do đó, nếu bạn không giới hạn số giai đoạn ở $n-1$, thuật toán sẽ chạy vô hạn, liên tục cải thiện khoảng cách từ các đỉnh này.

Vì vậy, ta thu được tiêu chí cho sự hiện diện của một chu trình có trọng số âm có thể tiếp cận từ đỉnh nguồn $v$: sau giai đoạn thứ $(n-1)$, nếu ta chạy thuật toán thêm một giai đoạn nữa, và nó thực hiện ít nhất một phép thư giãn nữa, thì đồ thị chứa một chu trình có trọng số âm có thể tiếp cận từ $v$; nếu không, chu trình như vậy không tồn tại.

Hơn nữa, nếu chu trình như vậy được tìm thấy, thuật toán Bellman-Ford có thể được sửa đổi để nó lấy lại chu trình này dưới dạng một chuỗi các đỉnh chứa trong đó. Để làm điều này, chỉ cần nhớ đỉnh cuối cùng $x$ mà có phép thư giãn trong giai đoạn thứ $n$. Đỉnh này sẽ nằm trên một chu trình có trọng số âm hoặc có thể tiếp cận từ nó. Để lấy các đỉnh chắc chắn nằm trên một chu trình âm, bắt đầu từ đỉnh $x$, đi qua các tiền tố $n$ lần. Bằng cách này, ta sẽ đến được đỉnh $y$, được đảm bảo nằm trên một chu trình âm. Ta phải đi từ đỉnh này, qua các tiền tố, cho đến khi quay lại cùng đỉnh $y$ (và điều này sẽ xảy ra, vì phép thư giãn trong một chu trình có trọng số âm diễn ra theo cách tuần hoàn).

#### Triển khai

```
void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);
    int x;
    for (int i = 0; i < n; ++i) {
        x = -1;
```

```
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = max(-INF, d[e.a] + e.cost);
                    p[e.b] = e.a;
                    x = e.b;
                }
    }

    if (x == -1)
        cout << "Không có chu trình âm từ " << v;
    else {
        int y = x;
        for (int i = 0; i < n; ++i)
            y = p[y];

        vector<int> path;
        for (int cur = y;; cur = p[cur]) {
            path.push_back(cur);
            if (cur == y && path.size() > 1)
                break;
        }
        reverse(path.begin(), path.end());

        cout << "Chu trình âm: ";
        for (int u : path)
            cout << u << ' ';
    }
}
```

Do sự hiện diện của một chu trình âm, trong $n$ lần lặp của thuật toán, các khoảng cách có thể đi xa vào phạm vi âm (đến các số âm bậc $-nmW$, trong đó $W$ là giá trị tuyệt đối tối đa của bất kỳ trọng số nào trong đồ thị). Do đó, trong mã, ta đã áp dụng các biện pháp bổ sung chống tràn số nguyên như sau:

$$d[e.b] = \max(-INF, d[e.a] + e.cost);$$

Triển khai trên tìm kiếm một chu trình âm có thể tiếp cận từ một đỉnh bắt đầu $v$ nào đó; tuy nhiên, thuật toán có thể được sửa đổi để chỉ tìm kiếm bất kỳ chu trình âm nào trong đồ thị. Để làm điều này, ta cần đặt tất cả khoảng cách $d[i]$ về 0 thay vì vô cực — như thể ta đang tìm đường đi ngắn nhất từ tất cả các đỉnh đồng thời; tính hợp lệ của việc phát hiện chu trình âm không bị ảnh hưởng.

Để biết thêm về chủ đề này — xem bài viết riêng, *Tìm chu trình âm trong đồ thị.*

### 3.11.6 Thuật toán đường đi ngắn nhất nhanh hơn (SPFA)

SPFA là một cải tiến của thuật toán Bellman-Ford, tận dụng thực tế rằng không phải tất cả các nỗ lực thư giãn đều sẽ hoạt động. Ý tưởng chính là tạo một hàng đợi chỉ chứa các đỉnh đã được thư giãn nhưng vẫn có thể tiếp tục thư giãn các đỉnh lân cận của chúng. Và bất cứ khi nào bạn có thể thư giãn một đỉnh lân cận nào đó, bạn nên đưa nó vào hàng đợi. Thuật toán này cũng có thể được sử dụng để phát hiện các chu trình âm như Bellman-Ford.

Trường hợp xấu nhất của thuật toán này bằng với $O(nm)$ của Bellman-Ford, nhưng trong thực tế, nó hoạt động nhanh hơn nhiều và một số người cho rằng nó hoạt động thậm chí trong $O(m)$ trung bình. Tuy nhiên, hãy cẩn thận, vì thuật toán này là xác định và dễ tạo ra các phản ví dụ khiến thuật toán chạy trong $O(nm)$.

Có một số lưu ý cần thực hiện trong triển khai, chẳng hạn như thực tế rằng thuật toán sẽ tiếp tục mãi mãi nếu có chu trình âm. Để tránh điều này, có thể tạo một bộ đếm lưu trữ số lần một đỉnh đã được thư giãn và dừng thuật toán ngay khi một đỉnh nào đó được thư giãn lần thứ $n$. Lưu ý, cũng không có lý do để đưa một đỉnh vào hàng đợi nếu nó đã có trong đó.

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inqueue(n, false);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
```

```
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = true;
                    cnt[to]++;
                    if (cnt[to] > n)
                        return false;  // chu trình âm
                }
            }
        }
    }
    return true;
}
```

## 3.12 BFS 0-1

Người ta biết rằng bạn có thể tìm các đường đi ngắn nhất từ một nguồn duy nhất đến tất cả các đỉnh khác trong $O(|E|)$ bằng cách sử dụng Tìm kiếm theo chiều rộng (Breadth First Search - BFS) trong một đồ thị không có trọng số, tức là khoảng cách là số lượng cạnh tối thiểu mà bạn cần đi qua từ nguồn đến một đỉnh khác. Ta cũng có thể diễn giải một đồ thị như vậy như một đồ thị có trọng số, trong đó mọi cạnh có trọng số 1. Nếu không phải tất cả các cạnh trong đồ thị đều có cùng trọng số, thì ta cần một thuật toán tổng quát hơn, như Dijkstra, chạy trong thời gian $O(|V|^2 + |E|)$ hoặc $O(|E| \log |V|)$.

Tuy nhiên, nếu các trọng số bị hạn chế chặt chẽ hơn, ta thường có thể làm tốt hơn. Trong bài viết này, ta sẽ chứng minh cách sử dụng BFS để giải bài toán SSSP (đường đi ngắn nhất từ một nguồn) trong $O(|E|)$, nếu trọng số của mỗi cạnh là 0 hoặc 1.

### 3.12.1 Thuật toán

Ta có thể phát triển thuật toán bằng cách nghiên cứu kỹ thuật toán Dijkstra và suy nghĩ về những hệ quả mà đồ thị đặc biệt của ta mang lại. Dạng tổng quát của thuật toán Dijkstra là (ở đây một set được sử dụng làm hàng đợi ưu tiên):

```
d.assign(n, INF);
d[s] = 0;
set<pair<int, int>> q;
q.insert({0, s});
while (!q.empty()) {
```

```
    int v = q.begin()->second;
    q.erase(q.begin());

    for (auto edge : adj[v]) {
        int u = edge.first;
        int w = edge.second;

        if (d[v] + w < d[u]) {
            q.erase({d[u], u});
            d[u] = d[v] + w;
            q.insert({d[u], u});
        }
    }
}
```

Ta có thể nhận thấy rằng sự khác biệt giữa các khoảng cách từ nguồn $s$ đến hai đỉnh khác trong hàng đợi chỉ chênh lệch tối đa một đơn vị. Đặc biệt, ta biết rằng $d[v] \leq d[u] \leq d[v] + 1$ với mỗi $u \in Q$. Lý do là vì trong mỗi lần lặp, ta chỉ thêm các đỉnh có khoảng cách bằng nhau hoặc lớn hơn một đơn vị vào hàng đợi. Giả sử tồn tại một $u$ trong hàng đợi với $d[u] - d[v] > 1$, thì $u$ phải được thêm vào hàng đợi qua một đỉnh khác $t$ với $d[t] \geq d[u] - 1 > d[v]$. Tuy nhiên, điều này là không thể, vì thuật toán Dijkstra lặp qua các đỉnh theo thứ tự tăng dần.

Điều này có nghĩa là thứ tự của hàng đợi trông như sau:

$$Q = \underbrace{v}_{d[v]}, \ldots, \underbrace{u}_{d[v]}, \underbrace{m}_{d[v]+1}, \ldots, \underbrace{n}_{d[v]+1}$$

Cấu trúc này đơn giản đến mức ta không cần một hàng đợi ưu tiên thực sự, tức là sử dụng một cây nhị phân cân bằng sẽ là quá mức cần thiết. Ta chỉ cần sử dụng một hàng đợi thông thường, và thêm các đỉnh mới vào đầu nếu cạnh tương ứng có trọng số 0, tức là nếu $d[u] = d[v]$, hoặc vào cuối nếu cạnh có trọng số 1, tức là nếu $d[u] = d[v] + 1$. Bằng cách này, hàng đợi vẫn luôn được sắp xếp.

```
vector<int> d(n, INF);
d[s] = 0;
deque<int> q;
q.push_front(s);
while (!q.empty()) {
    int v = q.front();
    q.pop_front();
    for (auto edge : adj[v]) {
        int u = edge.first;
```

```
        int w = edge.second;
        if (d[v] + w < d[u]) {
            d[u] = d[v] + w;
            if (w == 1)
                q.push_back(u);
            else
                q.push_front(u);
        }
    }
}
```

### 3.12.2 Thuật toán của Dial

Ta có thể mở rộng điều này thêm nếu cho phép trọng số của các cạnh lớn hơn. Nếu mọi cạnh trong đồ thị có trọng số $\leq k$, thì khoảng cách của các đỉnh trong hàng đợi sẽ chênh lệch tối đa $k$ so với khoảng cách từ $v$ đến nguồn. Vì vậy, ta có thể giữ $k+1$ thùng (buckets) cho các đỉnh trong hàng đợi, và bất cứ khi nào thùng tương ứng với khoảng cách nhỏ nhất trở nên trống, ta thực hiện một dịch chuyển vòng để lấy thùng với khoảng cách cao hơn tiếp theo. Phần mở rộng này được gọi là thuật toán của Dial.

## 3.13 Thuật toán D´Esopo-Pape

Cho một đồ thị với $n$ đỉnh và $m$ cạnh có trọng số $w_i$ và một đỉnh bắt đầu $v_0$. Nhiệm vụ là tìm đường đi ngắn nhất từ đỉnh $v_0$ đến mọi đỉnh khác.

Thuật toán D´Esopo-Pape sẽ hoạt động nhanh hơn thuật toán Dijkstra và thuật toán Bellman-Ford trong hầu hết các trường hợp, và cũng hoạt động được với các cạnh có trọng số âm. Tuy nhiên, không áp dụng được cho các chu trình âm.

### 3.13.1 Mô tả

Gọi mảng $d$ chứa độ dài của các đường đi ngắn nhất, tức là $d_i$ là độ dài hiện tại của đường đi ngắn nhất từ đỉnh $v_0$ đến đỉnh $i$. Ban đầu, mảng này được điền bằng vô cực cho mọi đỉnh, ngoại trừ $d_{v_0} = 0$. Sau khi thuật toán kết thúc, mảng này sẽ chứa các khoảng cách ngắn nhất.

Gọi mảng $p$ chứa các tổ tiên hiện tại, tức là $p_i$ là tổ tiên trực tiếp của đỉnh $i$ trên đường đi ngắn nhất hiện tại từ $v_0$ đến $i$. Giống như mảng $d$, mảng $p$ thay đổi dần trong suốt thuật toán và cuối cùng nhận các giá trị cuối cùng.

Bây giờ đến thuật toán. Ở mỗi bước, ba tập hợp đỉnh được duy trì:

- $M_0$ - các đỉnh mà khoảng cách đã được tính toán (mặc dù có thể chưa phải là khoảng cách cuối cùng),

- $M_1$ - các đỉnh mà khoảng cách hiện đang được tính toán,
- $M_2$ - các đỉnh mà khoảng cách chưa được tính toán.

Các đỉnh trong tập $M_1$ được lưu trữ trong một hàng đợi hai chiều (deque).

Ở mỗi bước của thuật toán, ta lấy một đỉnh từ tập $M_1$ (từ đầu hàng đợi). Gọi $u$ là đỉnh được chọn. Ta đặt đỉnh $u$ này vào tập $M_0$. Sau đó, ta lặp qua tất cả các cạnh đi ra từ đỉnh này. Gọi $v$ là đầu còn lại của cạnh hiện tại, và $w$ là trọng số của nó.

- Nếu $v$ thuộc $M_2$, thì $v$ được chèn vào tập $M_1$ bằng cách thêm nó vào cuối hàng đợi. $d_v$ được đặt thành $d_u + w$.
- Nếu $v$ thuộc $M_1$, thì ta cố gắng cải thiện giá trị của $d_v$: $d_v = \min(d_v, d_u + w)$. Vì $v$ đã ở trong $M_1$, ta không cần chèn nó vào $M_1$ và hàng đợi.
- Nếu $v$ thuộc $M_0$, và nếu $d_v$ có thể được cải thiện $d_v > d_u + w$, thì ta cải thiện $d_v$ và chèn đỉnh $v$ trở lại tập $M_1$, đặt nó ở đầu hàng đợi.

Và tất nhiên, với mỗi cập nhật trong mảng $d$, ta cũng phải cập nhật phần tử tương ứng trong mảng $p$.

### 3.13.2 Triển khai

Ta sẽ sử dụng một mảng $m$ để lưu trữ tập hợp mà mỗi đỉnh hiện đang thuộc về.

```
struct Edge {
    int to, w;
};

int n;
vector<vector<Edge>> adj;

const int INF = 1e9;

void shortest_paths(int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<int> m(n, 2);
    deque<int> q;
    q.push_back(v0);
    p.assign(n, -1);
```

```
    while (!q.empty()) {
        int u = q.front();
        q.pop_front();
        m[u] = 0;
        for (Edge e : adj[u]) {
            if (d[e.to] > d[u] + e.w) {
                d[e.to] = d[u] + e.w;
                p[e.to] = u;
                if (m[e.to] == 2) {
                    m[e.to] = 1;
                    q.push_back(e.to);
                } else if (m[e.to] == 0) {
                    m[e.to] = 1;
                    q.push_front(e.to);
                }
            }
        }
    }
}
```

### 3.13.3 Độ phức tạp

Thuật toán thường hoạt động khá nhanh - trong hầu hết các trường hợp, thậm chí nhanh hơn thuật toán Dijkstra. Tuy nhiên, tồn tại các trường hợp mà thuật toán mất thời gian cấp số mũ, khiến nó không phù hợp trong trường hợp xấu nhất. Xem các cuộc thảo luận trên Stack Overflow và Codeforces để tham khảo.

## 3.14 Thuật toán Floyd-Warshall

Cho một đồ thị có hướng hoặc không hướng có trọng số $G$ với $n$ đỉnh. Nhiệm vụ là tìm độ dài của đường đi ngắn nhất $d_{ij}$ giữa mỗi cặp đỉnh $i$ và $j$.

Đồ thị có thể có các cạnh có trọng số âm, nhưng không có chu trình trọng số âm.

Nếu tồn tại một chu trình âm như vậy, bạn có thể đi qua chu trình này lặp đi lặp lại, mỗi lần lặp làm giảm chi phí của đường đi. Vì vậy, bạn có thể làm cho một số đường đi nhỏ tùy ý, hay nói cách khác, đường đi ngắn nhất là không xác định. Điều này tự động có nghĩa là một đồ thị không hướng không thể có bất kỳ cạnh nào có trọng số âm, vì một cạnh như vậy đã tạo thành một chu trình âm khi bạn có thể di chuyển qua lại dọc theo cạnh đó bao lâu tùy thích.

Thuật toán này cũng có thể được sử dụng để phát hiện sự hiện diện của các chu trình âm. Đồ thị có một chu trình âm nếu vào cuối thuật toán, khoảng cách từ một đỉnh $v$ đến

chính nó là âm.

Thuật toán này đã được công bố đồng thời trong các bài báo của Robert Floyd và Stephen Warshall vào năm 1962. Tuy nhiên, vào năm 1959, Bernard Roy đã công bố một thuật toán tương tự, nhưng công bố của ông không được chú ý.

### 3.14.1 Mô tả thuật toán

Ý tưởng chính của thuật toán là chia quá trình tìm đường đi ngắn nhất giữa bất kỳ hai đỉnh nào thành nhiều giai đoạn tăng dần.

Hãy đánh số các đỉnh từ 1 đến $n$. Ma trận khoảng cách là $d[][]$.

Trước giai đoạn thứ $k$ ($k = 1 \dots n$), $d[i][j]$ cho bất kỳ đỉnh $i$ và $j$ nào lưu trữ độ dài của đường đi ngắn nhất giữa đỉnh $i$ và đỉnh $j$, mà chỉ chứa các đỉnh $\{1, 2, \dots, k-1\}$ làm các đỉnh nội tại trong đường đi.

Nói cách khác, trước giai đoạn thứ $k$, giá trị của $d[i][j]$ bằng độ dài của đường đi ngắn nhất từ đỉnh $i$ đến đỉnh $j$, nếu đường đi này chỉ được phép đi qua các đỉnh có số nhỏ hơn $k$ (đầu và cuối của đường đi không bị giới hạn bởi thuộc tính này).

Dễ dàng xác minh rằng thuộc tính này đúng cho giai đoạn đầu tiên. Với $k = 0$, ta có thể điền ma trận với $d[i][j] = w_{ij}$ nếu tồn tại một cạnh giữa $i$ và $j$ với trọng số $w_{ij}$ và $d[i][j] = \infty$ nếu không tồn tại cạnh. Trong thực tế, $\infty$ sẽ là một giá trị lớn nào đó. Như ta sẽ thấy sau, đây là yêu cầu cho thuật toán.

Giả sử bây giờ ta đang ở giai đoạn thứ $k$, và ta muốn tính toán ma trận $d[][]$ sao cho nó đáp ứng các yêu cầu cho giai đoạn thứ $(k+1)$. Ta phải sửa các khoảng cách cho một số cặp đỉnh $(i, j)$. Có hai trường hợp cơ bản khác nhau:

- Đường đi ngắn nhất từ đỉnh $i$ đến đỉnh $j$ với các đỉnh nội tại từ tập $\{1, 2, \dots, k\}$ trùng với đường đi ngắn nhất với các đỉnh nội tại từ tập $\{1, 2, \dots, k-1\}$. Trong trường hợp này, $d[i][j]$ sẽ không thay đổi trong quá trình chuyển đổi.

- Đường đi ngắn nhất với các đỉnh nội tại từ $\{1, 2, \dots, k\}$ ngắn hơn. Điều này có nghĩa là đường đi mới, ngắn hơn đi qua đỉnh $k$. Ta có thể chia đường đi ngắn nhất giữa $i$ và $j$ thành hai đường đi: đường đi giữa $i$ và $k$, và đường đi giữa $k$ và $j$. Rõ ràng cả hai đường đi này chỉ sử dụng các đỉnh nội tại của $\{1, 2, \dots, k-1\}$ và là các đường đi ngắn nhất theo nghĩa đó. Do đó, ta đã tính toán độ dài của các đường đi đó trước đó, và ta có thể tính độ dài của đường đi ngắn nhất giữa $i$ và $j$ là $d[i][k] + d[k][j]$.

Kết hợp hai trường hợp này, ta thấy rằng ta có thể tính toán lại độ dài của tất cả các cặp $(i, j)$ trong giai đoạn thứ $k$ theo cách sau:

$$d_{\text{new}}[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Vì vậy, tất cả công việc cần thiết trong giai đoạn thứ $k$ là lặp qua tất cả các cặp đỉnh và tính toán lại độ dài của đường đi ngắn nhất giữa chúng. Kết quả là, sau giai đoạn thứ

$n$, giá trị $d[i][j]$ trong ma trận khoảng cách là độ dài của đường đi ngắn nhất giữa $i$ và $j$, hoặc là $\infty$ nếu đường đi giữa các đỉnh $i$ và $j$ không tồn tại.

Một lưu ý cuối cùng - ta không cần tạo một ma trận khoảng cách riêng $d_{\text{new}}[][]$ để lưu trữ tạm thời các đường đi ngắn nhất của giai đoạn thứ $k$, tức là tất cả các thay đổi có thể được thực hiện trực tiếp trong ma trận $d[][]$ ở bất kỳ giai đoạn nào. Thực tế, ở bất kỳ giai đoạn thứ $k$ nào, ta tối đa chỉ cải thiện khoảng cách của bất kỳ đường đi nào trong ma trận khoảng cách, do đó ta không thể làm xấu đi độ dài của đường đi ngắn nhất cho bất kỳ cặp đỉnh nào sẽ được xử lý trong giai đoạn $(k+1)$ hoặc sau đó.

Độ phức tạp thời gian của thuật toán này rõ ràng là $O(n^3)$.

### 3.14.2 Triển khai

Gọi $d[][]$ là một mảng 2D kích thước $n \times n$, được điền theo giai đoạn thứ 0 như đã giải thích trước đó. Ta cũng sẽ đặt $d[i][i] = 0$ cho bất kỳ $i$ nào ở giai đoạn thứ 0.

Sau đó, thuật toán được triển khai như sau:

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

Giả định rằng nếu không có cạnh giữa bất kỳ hai đỉnh $i$ và $j$ nào, thì ma trận tại $d[i][j]$ chứa một số lớn (đủ lớn để lớn hơn độ dài của bất kỳ đường đi nào trong đồ thị này). Khi đó, cạnh này sẽ luôn không có lợi để chọn, và thuật toán sẽ hoạt động chính xác.

Tuy nhiên, nếu trong đồ thị có các cạnh có trọng số âm, cần thực hiện các biện pháp đặc biệt. Nếu không, các giá trị kết quả trong ma trận có thể có dạng $\infty - 1$, $\infty - 2$, v.v., điều này, tất nhiên, vẫn chỉ ra rằng giữa các đỉnh tương ứng không tồn tại đường đi. Do đó, nếu đồ thị có các cạnh có trọng số âm, tốt hơn là viết thuật toán Floyd-Warshall theo cách sau, để nó không thực hiện các chuyển đổi sử dụng các đường đi không tồn tại.

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

### 3.14.3 Khôi phục chuỗi đỉnh trong đường đi ngắn nhất

Việc duy trì thông tin bổ sung để có thể khôi phục đường đi ngắn nhất giữa bất kỳ hai đỉnh nào dưới dạng một chuỗi các đỉnh là dễ dàng.

Để làm điều này, ngoài ma trận khoảng cách $d[][]$, cần duy trì một ma trận tổ tiên $p[][]$, sẽ chứa số của giai đoạn mà khoảng cách ngắn nhất giữa hai đỉnh được sửa đổi lần cuối. Rõ ràng số của giai đoạn không gì khác ngoài một đỉnh ở giữa đường đi ngắn nhất mong muốn. Bây giờ ta chỉ cần tìm đường đi ngắn nhất giữa các đỉnh $i$ và $p[i][j]$, và giữa $p[i][j]$ và $j$. Điều này dẫn đến một thuật toán tái tạo đệ quy đơn giản của đường đi ngắn nhất.

### 3.14.4 Trường hợp trọng số thực

Nếu trọng số của các cạnh không phải là số nguyên mà là số thực, cần tính đến các sai số xảy ra khi làm việc với các kiểu số thực (float).

Thuật toán Floyd-Warshall có hiệu ứng không mong muốn là các sai số tích lũy rất nhanh. Thực tế, nếu có một sai số trong giai đoạn đầu tiên là $\delta$, sai số này có thể lan truyền đến lần lặp thứ hai là $2\delta$, đến lần lặp thứ ba là $4\delta$, v.v.

Để tránh điều này, thuật toán có thể được sửa đổi để tính đến sai số (EPS $= \delta$) bằng cách sử dụng so sánh sau:

```
if (d[i][k] + d[k][j] < d[i][j] - EPS)
    d[i][j] = d[i][k] + d[k][j];
```

### 3.14.5 Trường hợp chu trình âm

Về mặt chính thức, thuật toán Floyd-Warshall không áp dụng cho các đồ thị chứa chu trình trọng số âm. Nhưng đối với tất cả các cặp đỉnh $i$ và $j$ mà không tồn tại đường đi bắt đầu tại $i$, đi qua một chu trình âm, và kết thúc tại $j$, thuật toán vẫn sẽ hoạt động chính xác.

Đối với cặp đỉnh mà câu trả lời không tồn tại (do sự hiện diện của một chu trình âm trong đường đi giữa chúng), thuật toán Floyd sẽ lưu trữ bất kỳ số nào (có thể rất âm, nhưng không nhất thiết) trong ma trận khoảng cách. Tuy nhiên, có thể cải thiện thuật toán Floyd-Warshall để nó xử lý cẩn thận các cặp đỉnh như vậy, và xuất ra chúng, ví dụ như $-$INF.

Điều này có thể được thực hiện theo cách sau: hãy chạy thuật toán Floyd-Warshall thông thường cho một đồ thị đã cho. Sau đó, một đường đi ngắn nhất giữa các đỉnh $i$ và $j$ không tồn tại, nếu và chỉ nếu, tồn tại một đỉnh $t$ có thể tiếp cận từ $i$ và cũng từ $j$, mà tại đó $d[t][t] < 0$.

Ngoài ra, khi sử dụng thuật toán Floyd-Warshall cho các đồ thị có chu trình âm, ta nên lưu ý rằng có thể xảy ra các tình huống trong đó khoảng cách có thể giảm xuống rất

nhanh theo cấp số nhân vào vùng âm. Do đó, tràn số nguyên phải được xử lý bằng cách giới hạn khoảng cách tối thiểu bằng một giá trị nào đó (ví dụ: −INF).

Để biết thêm về việc tìm chu trình âm trong đồ thị, xem bài viết riêng *Tìm chu trình âm trong đồ thị*.

## 3.15 Số lượng đường đi có độ dài cố định / Đường đi ngắn nhất có độ dài cố định

Bài viết sau đây mô tả các giải pháp cho hai bài toán này dựa trên cùng một ý tưởng: quy bài toán về việc xây dựng ma trận và tính toán giải pháp bằng phép nhân ma trận thông thường hoặc bằng phép nhân đã được sửa đổi.

### 3.15.1 Số lượng đường đi có độ dài cố định

Cho một đồ thị có hướng, không trọng số $G$ với $n$ đỉnh và một số nguyên $k$. Nhiệm vụ là như sau: với mỗi cặp đỉnh $(i, j)$, ta phải tìm số lượng đường đi có độ dài $k$ giữa các đỉnh này. Các đường đi không cần phải đơn giản, tức là các đỉnh và cạnh có thể được thăm nhiều lần trong một đường đi.

Ta giả định rằng đồ thị được đặc tả bằng một ma trận kề, tức là ma trận $G[][]$ kích thước $n \times n$, trong đó mỗi phần tử $G[i][j]$ bằng 1 nếu đỉnh $i$ được nối với $j$ bằng một cạnh, và bằng 0 nếu chúng không được nối bằng cạnh. Thuật toán sau cũng hoạt động trong trường hợp có nhiều cạnh: nếu một cặp đỉnh $(i, j)$ được nối bằng $m$ cạnh, thì ta có thể ghi điều này vào ma trận kề bằng cách đặt $G[i][j] = m$. Thuật toán cũng hoạt động nếu đồ thị chứa các vòng lặp (vòng lặp là một cạnh nối một đỉnh với chính nó).

Rõ ràng ma trận kề đã xây dựng là câu trả lời cho bài toán trong trường hợp $k = 1$. Nó chứa số lượng đường đi có độ dài 1 giữa mỗi cặp đỉnh.

Ta sẽ xây dựng giải pháp một cách lặp: Giả sử ta biết câu trả lời cho một $k$ nào đó. Ở đây ta mô tả phương pháp để xây dựng câu trả lời cho $k + 1$. Ký hiệu $C_k$ là ma trận cho trường hợp $k$, và $C_{k+1}$ là ma trận ta muốn xây dựng. Với công thức sau, ta có thể tính mỗi phần tử của $C_{k+1}$:

$$C_{k+1}[i][j] = \sum_{p=1}^{n} C_k[i][p] \cdot G[p][j]$$

Dễ thấy rằng công thức này không tính gì khác ngoài tích của các ma trận $C_k$ và $G$:

$$C_{k+1} = C_k \cdot G$$

Do đó, giải pháp của bài toán có thể được biểu diễn như sau:

$$C_k = \underbrace{G \cdot G \cdots G}_{k \text{ lần}} = G^k$$

Cần lưu ý rằng các tích ma trận có thể được nâng lên lũy thừa cao một cách hiệu quả bằng cách sử dụng lũy thừa nhị phân (Binary exponentiation). Điều này cho ta một giải pháp với độ phức tạp $O(n^3 \log k)$.

### 3.15.2 Đường đi ngắn nhất có độ dài cố định

Cho một đồ thị có hướng có trọng số $G$ với $n$ đỉnh và một số nguyên $k$. Với mỗi cặp đỉnh $(i, j)$, ta phải tìm độ dài của đường đi ngắn nhất giữa $i$ và $j$ gồm đúng $k$ cạnh.

Ta giả định rằng đồ thị được đặc tả bởi một ma trận kề, tức là thông qua ma trận $G[][]$ kích thước $n \times n$, trong đó mỗi phần tử $G[i][j]$ chứa độ dài của cạnh từ đỉnh $i$ đến đỉnh $j$. Nếu không có cạnh giữa hai đỉnh, thì phần tử tương ứng của ma trận sẽ được gán giá trị vô cực $\infty$.

Rõ ràng trong dạng này, ma trận kề là câu trả lời cho bài toán với $k = 1$. Nó chứa độ dài của các đường đi ngắn nhất giữa mỗi cặp đỉnh, hoặc $\infty$ nếu đường đi gồm một cạnh không tồn tại.

Một lần nữa, ta có thể xây dựng giải pháp cho bài toán một cách lặp: Giả sử ta biết câu trả lời cho một $k$ nào đó. Ta sẽ chỉ ra cách tính câu trả lời cho $k + 1$. Ký hiệu $L_k$ là ma trận cho $k$ và $L_{k+1}$ là ma trận ta muốn xây dựng. Công thức sau tính mỗi phần tử của $L_{k+1}$:

$$L_{k+1}[i][j] = \min_{p=1\ldots n} (L_k[i][p] + G[p][j])$$

Khi xem xét kỹ công thức này, ta có thể rút ra một sự tương tự với phép nhân ma trận: thực tế ma trận $L_k$ được nhân với ma trận $G$, điểm khác biệt duy nhất là thay vì thao tác cộng trong phép nhân, ta lấy giá trị nhỏ nhất.

$$L_{k+1} = L_k \odot G,$$

trong đó phép toán $\odot$ được định nghĩa như sau:

$$A \odot B = C \iff C_{ij} = \min_{p=1\ldots n} (A_{ip} + B_{pj})$$

Do đó, giải pháp của bài toán có thể được biểu diễn bằng phép nhân đã sửa đổi:

$$L_k = \underbrace{G \odot \ldots \odot G}_{k \text{ lần}} = G^{\odot k}$$

Cần lưu ý rằng ta cũng có thể tính lũy thừa này một cách hiệu quả với lũy thừa nhị phân, vì phép nhân đã sửa đổi rõ ràng là kết hợp. Vì vậy, giải pháp này cũng có độ phức tạp $O(n^3 \log k)$.

### 3.15.3 Khái quát hóa các bài toán cho đường đi có độ dài tối đa $k$

Các giải pháp trên giải quyết các bài toán cho một $k$ cố định. Tuy nhiên, các giải pháp có thể được điều chỉnh để giải các bài toán mà các đường đi được phép chứa không quá $k$ cạnh.

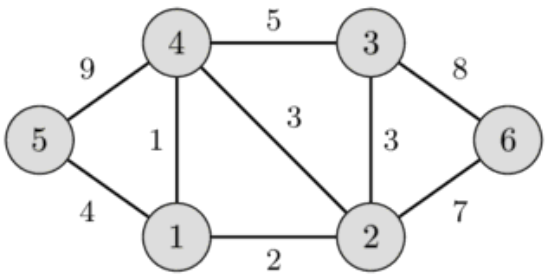
Điều này có thể được thực hiện bằng cách sửa đổi nhẹ đồ thị đầu vào.

Ta nhân đôi mỗi đỉnh: với mỗi đỉnh $v$, ta tạo thêm một đỉnh $v'$ và thêm cạnh $(v, v')$ và vòng lặp $(v', v')$. Số lượng đường đi giữa $i$ và $j$ với tối đa $k$ cạnh bằng với số lượng đường đi giữa $i$ và $j'$ với đúng $k+1$ cạnh, vì tồn tại một song ánh ánh xạ mọi đường đi $[p_0 = i, p_1, \ldots, p_{m-1}, p_m = j]$ có độ dài $m \leq k$ thành đường đi $[p_0 = i, p_1, \ldots, p_{m-1}, p_m = j, j', \ldots, j']$ có độ dài $k+1$.

Thủ thuật tương tự có thể được áp dụng để tính các đường đi ngắn nhất với tối đa $k$ cạnh. Ta lại nhân đôi mỗi đỉnh và thêm hai cạnh đã đề cập với trọng số 0.

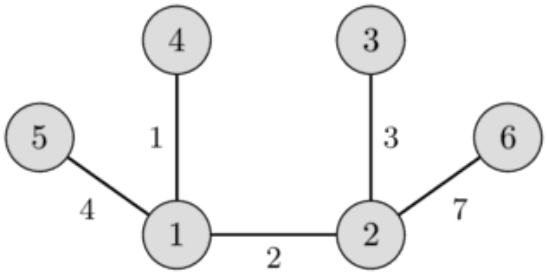
## 3.16 Cây bao trùm nhỏ nhất - Thuật toán Prim

Cho một đồ thị không hướng có trọng số $G$ với $n$ đỉnh và $m$ cạnh. Bạn muốn tìm một cây bao trùm của đồ thị này, cây này nối tất cả các đỉnh và có trọng số nhỏ nhất (tức là tổng trọng số của các cạnh là nhỏ nhất). Cây bao trùm là một tập hợp các cạnh sao cho bất kỳ đỉnh nào cũng có thể đến được bất kỳ đỉnh nào khác bằng đúng một đường đi đơn giản. Cây bao trùm có trọng số nhỏ nhất được gọi là cây bao trùm nhỏ nhất.




Hình 3.3: MST Before

Trong hình bên trái, bạn có thể thấy một đồ thị không hướng có trọng số, và trong hình bên phải, bạn có thể thấy cây bao trùm nhỏ nhất tương ứng.




Hình 3.4: MST After

Rõ ràng bất kỳ cây bao trùm nào cũng sẽ chứa đúng $n-1$ cạnh.

Bài toán này xuất hiện khá tự nhiên trong nhiều vấn đề. Chẳng hạn trong bài toán sau: có $n$ thành phố và với mỗi cặp thành phố, ta được cho chi phí để xây dựng một con đường giữa chúng (hoặc biết rằng không thể xây dựng đường giữa chúng về mặt vật lý). Ta phải xây dựng các con đường sao cho có thể đi từ mỗi thành phố đến mọi thành phố khác, và chi phí xây dựng tất cả các con đường là nhỏ nhất.

### 3.16.1 Thuật toán Prim

Thuật toán này ban đầu được nhà toán học người Séc Vojtěch Jarník phát hiện vào năm 1930. Tuy nhiên, thuật toán này chủ yếu được biết đến với tên gọi thuật toán Prim theo tên nhà toán học người Mỹ Robert Clay Prim, người đã tái khám phá và công bố lại nó vào năm 1957. Ngoài ra, Edsger Dijkstra cũng công bố thuật toán này vào năm 1959.

#### Mô tả thuật toán

Ở đây ta mô tả thuật toán dưới dạng đơn giản nhất. Cây bao trùm nhỏ nhất được xây dựng dần dần bằng cách thêm từng cạnh một. Ban đầu, cây bao trùm chỉ gồm một đỉnh duy nhất (được chọn tùy ý). Sau đó, cạnh có trọng số nhỏ nhất đi ra từ đỉnh này được chọn và thêm vào cây bao trùm. Sau bước đó, cây bao trùm đã gồm hai đỉnh. Tiếp theo, chọn và thêm cạnh có trọng số nhỏ nhất mà có một đầu nằm trong số các đỉnh đã chọn (tức là đỉnh đã là một phần của cây bao trùm), và đầu kia nằm trong số các đỉnh chưa được chọn. Và cứ thế, mỗi lần ta chọn và thêm cạnh có trọng số nhỏ nhất nối một đỉnh đã chọn với một đỉnh chưa được chọn. Quá trình được lặp lại cho đến khi cây bao trùm chứa tất cả các đỉnh (hoặc tương đương, cho đến khi ta có $n-1$ cạnh).

Cuối cùng, cây bao trùm được xây dựng sẽ là nhỏ nhất. Nếu đồ thị ban đầu không liên thông, thì không tồn tại cây bao trùm, do đó số lượng cạnh được chọn sẽ ít hơn $n-1$.

**Chứng minh**

Gọi đồ thị $G$ là liên thông, tức là đáp án tồn tại. Ta ký hiệu $T$ là đồ thị kết quả tìm được bởi thuật toán Prim, và $S$ là cây bao trùm nhỏ nhất. Rõ ràng $T$ thực sự là một cây bao trùm và là một đồ thị con của $G$. Ta chỉ cần chứng minh rằng trọng số của $S$ và $T$ là như nhau.

Xét thời điểm đầu tiên trong thuật toán khi ta thêm một cạnh vào $T$ mà không thuộc $S$. Gọi cạnh này là $e$, hai đầu của nó là $a$ và $b$, và tập hợp các đỉnh đã chọn là $V$ ($a \in V$ và $b \notin V$, hoặc ngược lại).

Trong cây bao trùm nhỏ nhất $S$, các đỉnh $a$ và $b$ được nối bằng một đường đi nào đó $P$. Trên đường đi này, ta có thể tìm một cạnh $f$ sao cho một đầu của $f$ nằm trong $V$ và đầu kia không nằm trong $V$. Vì thuật toán chọn $e$ thay vì $f$, điều này có nghĩa là trọng số của $f$ lớn hơn hoặc bằng trọng số của $e$.

Ta thêm cạnh $e$ vào cây bao trùm nhỏ nhất $S$ và loại bỏ cạnh $f$. Bằng cách thêm $e$, ta tạo ra một chu trình, và vì $f$ cũng là một phần của chu trình duy nhất, việc loại bỏ nó khiến đồ thị kết quả không còn chu trình. Và vì ta chỉ loại bỏ một cạnh từ một chu trình, đồ thị kết quả vẫn liên thông.

Cây bao trùm kết quả không thể có tổng trọng số lớn hơn, vì trọng số của $e$ không lớn hơn trọng số của $f$, và cũng không thể có trọng số nhỏ hơn vì $S$ là cây bao trùm nhỏ nhất. Điều này có nghĩa là bằng cách thay thế cạnh $f$ bằng $e$, ta đã tạo ra một cây bao trùm nhỏ nhất khác. Và $e$ phải có cùng trọng số với $f$.

Do đó, tất cả các cạnh ta chọn trong thuật toán Prim có cùng trọng số với các cạnh của bất kỳ cây bao trùm nhỏ nhất nào, nghĩa là thuật toán Prim thực sự tạo ra một cây bao trùm nhỏ nhất.

## 3.16.2 Triển khai

Độ phức tạp của thuật toán phụ thuộc vào cách ta tìm kiếm cạnh nhỏ nhất tiếp theo trong số các cạnh phù hợp. Có nhiều cách tiếp cận dẫn đến các độ phức tạp và triển khai khác nhau.

**Triển khai đơn giản: $O(nm)$ và $O(n^2 + m \log n)$**

Nếu ta tìm kiếm cạnh bằng cách lặp qua tất cả các cạnh có thể, thì mất thời gian $O(m)$ để tìm cạnh có trọng số nhỏ nhất. Tổng độ phức tạp sẽ là $O(nm)$. Trong trường hợp xấu nhất, điều này là $O(n^3)$, rất chậm.

Thuật toán này có thể được cải thiện nếu ta chỉ xem xét một cạnh từ mỗi đỉnh đã chọn. Chẳng hạn, ta có thể sắp xếp các cạnh từ mỗi đỉnh theo thứ tự tăng dần của trọng số, và lưu trữ một con trỏ đến cạnh hợp lệ đầu tiên (tức là cạnh dẫn đến một đỉnh chưa được chọn). Sau đó, sau khi tìm và chọn cạnh nhỏ nhất, ta cập nhật các con trỏ. Điều

này cho độ phức tạp $O(n^2 + m)$, và thêm $O(m \log n)$ để sắp xếp các cạnh, dẫn đến độ phức tạp $O(n^2 \log n)$ trong trường hợp xấu nhất.

Dưới đây ta xem xét hai thuật toán hơi khác nhau, một cho đồ thị dày và một cho đồ thị thưa, cả hai đều có độ phức tạp tốt hơn.

**Đồ thị dày:** $O(n^2)$

Ta tiếp cận bài toán này từ một góc độ khác: với mỗi đỉnh chưa được chọn, ta sẽ lưu trữ cạnh nhỏ nhất đến một đỉnh đã được chọn.

Sau đó, trong một bước, ta chỉ cần xem xét các cạnh có trọng số nhỏ nhất này, điều này sẽ có độ phức tạp $O(n)$.

Sau khi thêm một cạnh, một số con trỏ cạnh nhỏ nhất phải được tính toán lại. Lưu ý rằng các trọng số chỉ có thể giảm, tức là cạnh có trọng số nhỏ nhất của mỗi đỉnh chưa được chọn có thể giữ nguyên, hoặc sẽ được cập nhật bằng một cạnh đến đỉnh vừa được chọn. Do đó, giai đoạn này cũng có thể được thực hiện trong $O(n)$.

Vậy ta nhận được một phiên bản của thuật toán Prim với độ phức tạp $O(n^2)$.

Đặc biệt, triển khai này rất thuận tiện cho bài toán Cây bao trùm nhỏ nhất Euclidean: ta có $n$ điểm trên mặt phẳng và khoảng cách giữa mỗi cặp điểm là khoảng cách Euclidean giữa chúng, và ta muốn tìm một cây bao trùm nhỏ nhất cho đồ thị đầy đủ này. Bài toán này có thể được giải bằng thuật toán đã mô tả trong thời gian $O(n^2)$ và bộ nhớ $O(n)$, điều không thể thực hiện được với thuật toán Kruskal.

```
int n;
vector<vector<int>> adj; // ma trận kề của đồ thị
const int INF = 1000000000; // trọng số INF nghĩa là không có cạnh

struct Edge {
    int w = INF, to = -1;
};

void prim() {
    int total_weight = 0;
    vector<bool> selected(n, false);
    vector<Edge> min_e(n);
    min_e[0].w = 0;

    for (int i = 0; i < n; ++i) {
        int v = -1;
        for (int j = 0; j < n; ++j) {
            if (!selected[j] && (v == -1 || min_e[j].w < min_e[v].w))
                v = j;
```

```
        }

        if (min_e[v].w == INF) {
            cout << "Không có MST!" << endl;
            exit(0);
        }

        selected[v] = true;
        total_weight += min_e[v].w;
        if (min_e[v].to != -1)
            cout << v << " " << min_e[v].to << endl;

        for (int to = 0; to < n; ++to) {
            if (adj[v][to] < min_e[to].w)
                min_e[to] = {adj[v][to], v};
        }
    }

    cout << total_weight << endl;
}
```

Ma trận kề adj[][] kích thước $n \times n$ lưu trữ trọng số của các cạnh, và sử dụng trọng số INF nếu không tồn tại cạnh giữa hai đỉnh. Thuật toán sử dụng hai mảng: cờ selected[], cho biết đỉnh nào đã được chọn, và mảng min_e[] lưu trữ cạnh có trọng số nhỏ nhất đến một đỉnh đã chọn cho mỗi đỉnh chưa được chọn (nó lưu trọng số và đỉnh đích). Thuật toán thực hiện $n$ bước, trong mỗi lần lặp, đỉnh có trọng số cạnh nhỏ nhất được chọn, và min_e[] của tất cả các đỉnh khác được cập nhật.

**Đồ thị thưa:** $O(m \log n)$

Trong thuật toán đã mô tả ở trên, có thể diễn giải các thao tác tìm giá trị nhỏ nhất và sửa đổi một số giá trị như các thao tác trên tập hợp. Hai thao tác cổ điển này được hỗ trợ bởi nhiều cấu trúc dữ liệu, ví dụ như set trong C++ (được triển khai qua cây đỏ-đen).

Thuật toán chính vẫn giữ nguyên, nhưng giờ ta có thể tìm cạnh nhỏ nhất trong thời gian $O(\log n)$. Mặt khác, việc tính toán lại các con trỏ giờ sẽ mất $O(n \log n)$, điều này tệ hơn so với thuật toán trước.

Nhưng khi ta xem xét rằng ta chỉ cần cập nhật tổng cộng $O(m)$ lần, và thực hiện $O(n)$ tìm kiếm cạnh nhỏ nhất, thì tổng độ phức tạp sẽ là $O(m \log n)$. Với đồ thị thưa, điều này tốt hơn thuật toán trên, nhưng với đồ thị dày, điều này sẽ chậm hơn.

```
const int INF = 1000000000;
```

```
struct Edge {
    int w = INF, to = -1;
    bool operator<(Edge const& other) const {
        return make_pair(w, to) < make_pair(other.w, other.to);
    }
};

int n;
vector<vector<Edge>> adj;

void prim() {
    int total_weight = 0;
    vector<Edge> min_e(n);
    min_e[0].w = 0;
    set<Edge> q;
    q.insert({0, 0});
    vector<bool> selected(n, false);
    for (int i = 0; i < n; ++i) {
        if (q.empty()) {
            cout << "Không có MST!" << endl;
            exit(0);
        }

        int v = q.begin()->to;
        selected[v] = true;
        total_weight += q.begin()->w;
        q.erase(q.begin());

        if (min_e[v].to != -1)
            cout << v << " " << min_e[v].to << endl;

        for (Edge e : adj[v]) {
            if (!selected[e.to] && e.w < min_e[e.to].w) {
                q.erase({min_e[e.to].w, e.to});
                min_e[e.to] = {e.w, v};
                q.insert({e.w, e.to});
            }
        }
    }
```

```
    cout << total_weight << endl;
}
```

Ở đây, đồ thị được biểu diễn qua danh sách kề adj[], trong đó adj[$v$] chứa tất cả các cạnh (dưới dạng cặp trọng số và đích) cho đỉnh $v$. min_e[$v$] sẽ lưu trọng số của cạnh nhỏ nhất từ đỉnh $v$ đến một đỉnh đã chọn (lại dưới dạng cặp trọng số và đích). Ngoài ra, hàng đợi $q$ được điền với tất cả các đỉnh chưa được chọn theo thứ tự tăng dần của trọng số min_e. Thuật toán thực hiện $n$ bước, trong mỗi bước, nó chọn đỉnh $v$ có trọng số min_e nhỏ nhất (bằng cách trích xuất từ đầu hàng đợi), sau đó xem xét tất cả các cạnh từ đỉnh này và cập nhật các giá trị trong min_e (trong quá trình cập nhật, ta cũng cần xóa cạnh cũ khỏi hàng đợi $q$ và thêm cạnh mới vào).

## 3.17 Cây bao trùm nhỏ nhất - Thuật toán Kruskal

Cho một đồ thị không hướng có trọng số. Ta muốn tìm một cây con của đồ thị này nối tất cả các đỉnh (tức là một cây bao trùm) và có trọng số nhỏ nhất (tức là tổng trọng số của tất cả các cạnh là nhỏ nhất) trong số tất cả các cây bao trùm có thể có. Cây bao trùm này được gọi là cây bao trùm nhỏ nhất.

Trong hình bên trái, bạn có thể thấy một đồ thị không hướng có trọng số, và trong hình bên phải, bạn có thể thấy cây bao trùm nhỏ nhất tương ứng.

Bài viết này sẽ thảo luận về một số đặc điểm quan trọng liên quan đến cây bao trùm nhỏ nhất, và sau đó sẽ đưa ra triển khai đơn giản nhất của thuật toán Kruskal để tìm cây bao trùm nhỏ nhất.

### 3.17.1 Đặc điểm của cây bao trùm nhỏ nhất

- Cây bao trùm nhỏ nhất của một đồ thị là duy nhất nếu trọng số của tất cả các cạnh là khác nhau. Nếu không, có thể tồn tại nhiều cây bao trùm nhỏ nhất. (Các thuật toán cụ thể thường xuất ra một trong những cây bao trùm nhỏ nhất có thể).

- Cây bao trùm nhỏ nhất cũng là cây có tích trọng số của các cạnh nhỏ nhất. (Điều này có thể dễ dàng chứng minh bằng cách thay thế trọng số của tất cả các cạnh bằng lôgarit của chúng).

- Trong cây bao trùm nhỏ nhất của một đồ thị, trọng số lớn nhất của một cạnh là nhỏ nhất có thể trong số tất cả các cây bao trùm của đồ thị đó. (Điều này xuất phát từ tính hợp lệ của thuật toán Kruskal).

- Cây bao trùm lớn nhất (cây bao trùm với tổng trọng số của các cạnh là lớn nhất) của một đồ thị có thể được tìm thấy tương tự như cây bao trùm nhỏ nhất, bằng

cách đổi dấu của trọng số của tất cả các cạnh thành ngược lại và sau đó áp dụng bất kỳ thuật toán cây bao trùm nhỏ nhất nào.

### 3.17.2 Thuật toán Kruskal

Thuật toán này được Joseph Bernard Kruskal, Jr. mô tả vào năm 1956.

Thuật toán Kruskal ban đầu đặt tất cả các nút của đồ thị ban đầu tách biệt nhau, để tạo thành một rừng gồm các cây chỉ có một nút, và sau đó dần dần hợp nhất các cây này, kết hợp ở mỗi lần lặp bất kỳ hai cây nào trong số tất cả các cây với một cạnh nào đó của đồ thị ban đầu. Trước khi thực thi thuật toán, tất cả các cạnh được sắp xếp theo trọng số (theo thứ tự không giảm). Sau đó bắt đầu quá trình hợp nhất: chọn tất cả các cạnh từ đầu tiên đến cuối cùng (theo thứ tự đã sắp xếp), và nếu hai đầu của cạnh hiện tại thuộc về các cây con khác nhau, các cây con này được kết hợp, và cạnh được thêm vào đáp án. Sau khi lặp qua tất cả các cạnh, tất cả các đỉnh sẽ thuộc về cùng một cây con, và ta sẽ nhận được đáp án.

#### Triển khai đơn giản nhất

Mã sau đây triển khai trực tiếp thuật toán được mô tả ở trên, và có độ phức tạp thời gian $O(M \log M + N^2)$. Việc sắp xếp các cạnh yêu cầu $O(M \log N)$ (tương đương với $O(M \log M)$) thao tác. Thông tin liên quan đến cây con mà một đỉnh thuộc về được duy trì với sự trợ giúp của mảng tree_id[] - với mỗi đỉnh $v$, tree_id[$v$] lưu trữ số của cây mà $v$ thuộc về. Với mỗi cạnh, việc xác định liệu nó có thuộc về các đầu của các cây khác nhau hay không có thể được thực hiện trong $O(1)$. Cuối cùng, việc hợp nhất hai cây được thực hiện trong $O(N)$ bằng một lần duyệt đơn giản qua mảng tree_id[]. Vì tổng số thao tác hợp nhất là $N - 1$, ta thu được hành vi tiệm cận của $O(M \log N + N^2)$.

```
struct Edge {
    int u, v, weight;
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n;
vector<Edge> edges;

int cost = 0;
vector<int> tree_id(n);
vector<Edge> result;
for (int i = 0; i < n; i++)
```

```
    tree_id[i] = i;

sort(edges.begin(), edges.end());

for (Edge e : edges) {
    if (tree_id[e.u] != tree_id[e.v]) {
        cost += e.weight;
        result.push_back(e);

        int old_id = tree_id[e.u], new_id = tree_id[e.v];
        for (int i = 0; i < n; i++) {
            if (tree_id[i] == old_id)
                tree_id[i] = new_id;
        }
    }
}
```

### Chứng minh tính đúng đắn

Tại sao thuật toán Kruskal lại cho ta kết quả đúng?

Nếu đồ thị ban đầu là liên thông, thì đồ thị kết quả cũng sẽ liên thông. Bởi vì nếu không, sẽ có hai thành phần có thể được nối với ít nhất một cạnh. Tuy nhiên, điều này là không thể, vì Kruskal sẽ chọn một trong những cạnh này, do các ID của các thành phần là khác nhau. Đồ thị kết quả cũng không chứa bất kỳ chu trình nào, vì ta cấm điều này một cách rõ ràng trong thuật toán. Do đó, thuật toán tạo ra một cây bao trùm.

Vậy tại sao thuật toán này lại cho ta một cây bao trùm nhỏ nhất?

Ta có thể chứng minh đề xuất "nếu $F$ là tập hợp các cạnh được chọn bởi thuật toán ở bất kỳ giai đoạn nào trong thuật toán, thì tồn tại một MST chứa tất cả các cạnh của $F$" bằng quy nạp.

Đề xuất rõ ràng đúng ở lúc bắt đầu, tập rỗng là tập con của bất kỳ MST nào.

Bây giờ giả sử $F$ là một tập hợp cạnh ở bất kỳ giai đoạn nào của thuật toán, $T$ là một MST chứa $F$ và $e$ là cạnh mới ta muốn thêm bằng Kruskal.

Nếu $e$ tạo ra một chu trình, thì ta không thêm nó, và do đó đề xuất vẫn đúng sau bước này.

Trong trường hợp $T$ đã chứa $e$, đề xuất cũng đúng sau bước này.

Trong trường hợp $T$ không chứa cạnh $e$, thì $T + e$ sẽ chứa một chu trình $C$. Chu trình này sẽ chứa ít nhất một cạnh $f$ không nằm trong $F$. Tập hợp các cạnh $T - f + e$ cũng sẽ là một cây bao trùm. Lưu ý rằng trọng số của $f$ không thể nhỏ hơn trọng số của $e$, vì nếu không Kruskal sẽ chọn $f$ trước đó. Nó cũng không thể có trọng số lớn hơn, vì điều đó sẽ làm cho tổng trọng số của $T - f + e$ nhỏ hơn tổng trọng số của $T$, điều không thể vì $T$ đã là một MST. Điều này có nghĩa là trọng số của $e$ phải bằng trọng số của $f$. Do

đó, $T - f + e$ cũng là một MST, và nó chứa tất cả các cạnh từ $F + e$. Vì vậy, ở đây đề xuất vẫn được thỏa mãn sau bước này.

Điều này chứng minh đề xuất. Nghĩa là sau khi lặp qua tất cả các cạnh, tập hợp cạnh kết quả sẽ liên thông và sẽ được chứa trong một MST, nghĩa là nó đã là một MST.

**Triển khai cải tiến**

Ta có thể sử dụng cấu trúc dữ liệu Disjoint Set Union (DSU) để viết một triển khai nhanh hơn của thuật toán Kruskal với độ phức tạp thời gian khoảng $O(M \log N)$. Bài viết này chi tiết hóa cách tiếp cận như vậy.

## 3.18 Cây bao trùm nhỏ nhất - Kruskal với Disjoint Set Union

Để hiểu về bài toán cây bao trùm nhỏ nhất (MST) và thuật toán Kruskal, trước tiên hãy xem bài viết chính về thuật toán Kruskal.

Trong bài viết này, ta sẽ xem xét cấu trúc dữ liệu "Disjoint Set Union" (DSU) để triển khai thuật toán Kruskal, điều này sẽ cho phép thuật toán đạt được độ phức tạp thời gian $O(M \log N)$.

### 3.18.1 Mô tả

Giống như trong phiên bản đơn giản của thuật toán Kruskal, ta sắp xếp tất cả các cạnh của đồ thị theo thứ tự không giảm của trọng số. Sau đó, đặt mỗi đỉnh vào cây riêng của nó (tức là tập hợp của nó) thông qua các lời gọi hàm make_set - điều này sẽ mất tổng cộng $O(N)$. Ta lặp qua tất cả các cạnh (theo thứ tự đã sắp xếp) và với mỗi cạnh, xác định xem hai đầu của nó có thuộc về các cây khác nhau hay không (với hai lời gọi find_set trong $O(1)$ mỗi lần). Cuối cùng, ta cần thực hiện hợp nhất hai cây (tập hợp), điều này sẽ gọi hàm union_sets của DSU - cũng trong $O(1)$. Vì vậy, ta nhận được tổng độ phức tạp thời gian là $O(M \log N + N + M) = O(M \log N)$.

### 3.18.2 Triển khai

Dưới đây là triển khai của thuật toán Kruskal với Union by Rank.

```
vector<int> parent, rank;

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
```

```
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

struct Edge {
    int u, v, weight;
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n;
vector<Edge> edges;

int cost = 0;
vector<Edge> result;
parent.resize(n);
rank.resize(n);
for (int i = 0; i < n; i++)
    make_set(i);

sort(edges.begin(), edges.end());

for (Edge e : edges) {
```

```
    if (find_set(e.u) != find_set(e.v)) {
        cost += e.weight;
        result.push_back(e);
        union_sets(e.u, e.v);
    }
}
```

## 3.19 Cây bao trùm nhỏ nhất thứ hai

Cây bao trùm nhỏ nhất (MST) $T$ là một cây của đồ thị đã cho $G$, cây này bao trùm tất cả các đỉnh của đồ thị và có tổng trọng số của tất cả các cạnh là nhỏ nhất trong số tất cả các cây bao trùm có thể có. Cây bao trùm nhỏ nhất thứ hai $T'$ là một cây bao trùm có tổng trọng số của tất cả các cạnh là nhỏ thứ hai trong số tất cả các cây bao trùm có thể của đồ thị $G$.

### 3.19.1 Quan sát

Gọi $T$ là cây bao trùm nhỏ nhất của đồ thị $G$. Có thể quan sát thấy rằng cây bao trùm nhỏ nhất thứ hai chỉ khác $T$ bởi việc thay thế một cạnh duy nhất. (Để chứng minh điều này, tham khảo bài toán 23-1 tại đây).

Vì vậy, ta cần tìm một cạnh $e_{\text{new}}$ không nằm trong $T$, và thay thế nó bằng một cạnh trong $T$ (gọi là $e_{\text{old}}$) sao cho đồ thị mới $T' = (T \cup \{e_{\text{new}}\}) \setminus \{e_{\text{old}}\}$ là một cây bao trùm và sự chênh lệch trọng số $(e_{\text{new}} - e_{\text{old}})$ là nhỏ nhất.

### 3.19.2 Sử dụng thuật toán Kruskal

Ta có thể sử dụng thuật toán Kruskal để tìm MST trước, sau đó thử loại bỏ một cạnh duy nhất khỏi nó và thay thế bằng một cạnh khác.

- Sắp xếp các cạnh trong $O(E \log E)$, sau đó tìm một MST bằng Kruskal trong $O(E)$.
- Với mỗi cạnh trong MST (sẽ có $V - 1$ cạnh), tạm thời loại bỏ nó khỏi danh sách cạnh để nó không thể được chọn.
- Sau đó, thử tìm lại một MST trong $O(E)$ bằng các cạnh còn lại.
- Thực hiện điều này cho tất cả các cạnh trong MST, và chọn kết quả tốt nhất.

**Lưu ý:** Ta không cần sắp xếp lại các cạnh trong bước 3.
Vì vậy, tổng độ phức tạp thời gian sẽ là $O(E \log V + E + VE) = O(VE)$.

### 3.19.3 Mô hình hóa thành bài toán Tổ tiên chung thấp nhất (LCA)

Trong cách tiếp cận trước, ta đã thử tất cả các khả năng loại bỏ một cạnh của MST. Ở đây ta sẽ làm ngược lại hoàn toàn. Ta thử thêm mọi cạnh chưa nằm trong MST.

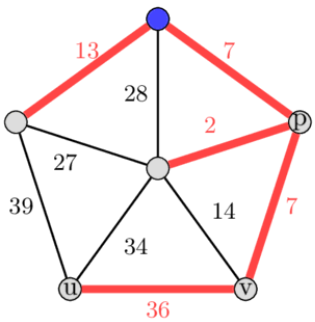
- Sắp xếp các cạnh trong $O(E \log E)$, sau đó tìm một MST bằng Kruskal trong $O(E)$.
- Với mỗi cạnh $e$ chưa nằm trong MST, tạm thời thêm nó vào MST, tạo ra một chu trình. Chu trình này sẽ đi qua LCA.
- Tìm cạnh $k$ có trọng số lớn nhất trong chu trình mà không bằng $e$, bằng cách theo dõi tổ tiên của các nút của cạnh $e$ lên đến LCA.
- Tạm thời loại bỏ $k$, tạo ra một cây bao trùm mới.
- Tính sự chênh lệch trọng số $\delta = \text{weight}(e) - \text{weight}(k)$, và ghi nhớ nó cùng với cạnh đã thay đổi.
- Lặp lại bước 2 cho tất cả các cạnh khác, và trả về cây bao trùm có sự chênh lệch trọng số nhỏ nhất so với MST.

Độ phức tạp thời gian của thuật toán phụ thuộc vào cách ta tính các $k$, tức là các cạnh có trọng số lớn nhất trong bước 2 của thuật toán này. Một cách để tính chúng hiệu quả trong $O(E \log V)$ là biến đổi bài toán thành bài toán Tổ tiên chung thấp nhất (LCA).

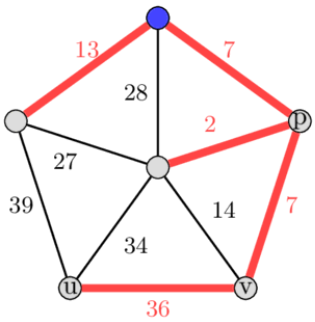
Ta sẽ tiền xử lý LCA bằng cách đặt gốc cho MST và cũng tính toán các trọng số cạnh lớn nhất cho mỗi nút trên đường đi đến tổ tiên của chúng. Điều này có thể được thực hiện bằng phương pháp Binary Lifting cho LCA.

Độ phức tạp thời gian cuối cùng của cách tiếp cận này là $O(E \log V)$.

**Ví dụ:**




Hình 3.5: In the image left is the MST and right is the second best MST.




Hình 3.6: In the image left is the MST and right is the second best MST.

Trong hình bên trái là MST và bên phải là cây bao trùm nhỏ nhất thứ hai.

Trong đồ thị đã cho, giả sử ta đặt gốc của MST tại đỉnh màu xanh ở trên cùng, sau đó chạy thuật toán bằng cách bắt đầu chọn các cạnh không nằm trong MST. Giả sử cạnh được chọn đầu tiên là cạnh $(u, v)$ với trọng số 36. Thêm cạnh này vào cây tạo ra chu trình 36 - 7 - 2 - 34.

Bây giờ ta sẽ tìm cạnh có trọng số lớn nhất trong chu trình này bằng cách tìm $\text{LCA}(u, v) = p$. Ta tính toán cạnh có trọng số lớn nhất trên đường đi từ $u$ đến $p$ và từ $v$ đến $p$. **Lưu ý:** $\text{LCA}(u, v)$ cũng có thể bằng $u$ hoặc $v$ trong một số trường hợp. Trong ví dụ này, ta sẽ nhận được cạnh có trọng số 34 là cạnh có trọng số lớn nhất trong chu trình. Bằng cách loại bỏ cạnh này, ta nhận được một cây bao trùm mới, có sự chênh lệch trọng số chỉ là 2.

Sau khi thực hiện điều này với tất cả các cạnh khác không thuộc MST ban đầu, ta có thể thấy rằng cây bao trùm này cũng là cây bao trùm nhỏ nhất thứ hai tổng thể. Chọn cạnh có trọng số 14 sẽ tăng trọng số của cây lên 7, chọn cạnh có trọng số 27 tăng lên 14, chọn cạnh có trọng số 28 tăng lên 21, và chọn cạnh có trọng số 39 sẽ tăng cây lên 5.

### 3.19.4 Triển khai

```
struct edge {
    int s, e, w, id;
    bool operator<(const struct edge& other) { return w < other.w; }
};
typedef struct edge Edge;

const int N = 2e5 + 5;
long long res = 0, ans = 1e18;
int n, m, a, b, w, id, l = 21;
vector<Edge> edges;
```

```
vector<int> h(N, 0), parent(N, -1), size(N, 0), present(N, 0);
vector<vector<pair<int, int>>> adj(N), dp(N, vector<pair<int, int>>(l));
vector<vector<int>> up(N, vector<int>(l, -1));

pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    vector<int> v = {a.first, a.second, b.first, b.second};
    int topTwo = -3, topOne = -2;
    for (int c : v) {
        if (c > topOne) {
            topTwo = topOne;
            topOne = c;
        } else if (c > topTwo && c < topOne) {
            topTwo = c;
        }
    }
    return {topOne, topTwo};
}

void dfs(int u, int par, int d) {
    h[u] = 1 + h[par];
    up[u][0] = par;
    dp[u][0] = {d, -1};
    for (auto v : adj[u]) {
        if (v.first != par) {
            dfs(v.first, u, v.second);
        }
    }
}

pair<int, int> lca(int u, int v) {
    pair<int, int> ans = {-2, -3};
    if (h[u] < h[v]) {
        swap(u, v);
    }
    for (int i = l - 1; i >= 0; i--) {
        if (h[u] - h[v] >= (1 << i)) {
            ans = combine(ans, dp[u][i]);
            u = up[u][i];
        }
    }
```

```
    if (u == v) {
        return ans;
    }
    for (int i = l - 1; i >= 0; i--) {
        if (up[u][i] != -1 && up[v][i] != -1 && up[u][i] != up[v][i]) {
            ans = combine(ans, combine(dp[u][i], dp[v][i]));
            u = up[u][i];
            v = up[v][i];
        }
    }
    ans = combine(ans, combine(dp[u][0], dp[v][0]));
    return ans;
}

int main(void) {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        size[i] = 1;
    }
    for (int i = 1; i <= m; i++) {
        cin >> a >> b >> w; // đánh số từ 1
        edges.push_back({a, b, w, i - 1});
    }
    sort(edges.begin(), edges.end());
    for (int i = 0; i <= m - 1; i++) {
        a = edges[i].s;
        b = edges[i].e;
        w = edges[i].w;
        id = edges[i].id;
        if (unite_set(a, b)) {
            adj[a].emplace_back(b, w);
            adj[b].emplace_back(a, w);
            present[id] = 1;
            res += w;
        }
    }
    dfs(1, 0, 0);
    for (int i = 1; i <= l - 1; i++) {
        for (int j = 1; j <= n; ++j) {
```

```
            if (up[j][i - 1] != -1) {
                int v = up[j][i - 1];
                up[j][i] = up[v][i - 1];
                dp[j][i] = combine(dp[j][i - 1], dp[v][i - 1]);
            }
        }
    }
    for (int i = 0; i <= m - 1; i++) {
        id = edges[i].id;
        w = edges[i].w;
        if (!present[id]) {
            auto rem = lca(edges[i].s, edges[i].e);
            if (rem.first != w) {
                if (ans > res + w - rem.first) {
                    ans = res + w - rem.first;
                }
            } else if (rem.second != -1) {
                if (ans > res + w - rem.second) {
                    ans = res + w - rem.second;
                }
            }
        }
    }
    cout << ans << "\n";
    return 0;
}
```

### 3.19.5 Tài liệu tham khảo

- Competitive Programming-3, bởi Steven Halim
- web.mit.edu

**Lưu ý:** Vì cây bao trùm nhỏ nhất sẽ chứa đúng $N - 1$ cạnh, ta có thể dừng vòng lặp for ngay khi đã tìm được số lượng đó.

## 3.20 Định lý Kirchhoff. Tìm số lượng cây bao trùm

**Bài toán:** Cho một đồ thị không hướng liên thông (có thể có nhiều cạnh giữa hai đỉnh) được biểu diễn bằng ma trận kề. Hãy tìm số lượng các cây bao trùm khác nhau của đồ thị này.

Công thức sau đã được Kirchhoff chứng minh vào năm 1847.

### 3.20.1 Định lý ma trận cây của Kirchhoff

Gọi $A$ là ma trận kề của đồ thị: $A_{u,v}$ là số lượng cạnh giữa $u$ và $v$. Gọi $D$ là ma trận bậc của đồ thị: một ma trận đường chéo với $D_{u,u}$ là bậc của đỉnh $u$ (bao gồm cả các cạnh nhiều lần và vòng lặp - các cạnh nối đỉnh $u$ với chính nó).

Ma trận Laplacian của đồ thị được định nghĩa là $L = D - A$. Theo định lý của Kirchhoff, tất cả các cofactor của ma trận này đều bằng nhau, và chúng bằng số lượng cây bao trùm của đồ thị. Cofactor $(i, j)$ của một ma trận là tích của $(-1)^{i+j}$ với định thức của ma trận thu được sau khi loại bỏ hàng $i$ và cột $j$. Vì vậy, bạn có thể, ví dụ, xóa hàng cuối cùng và cột cuối cùng của ma trận $L$, và giá trị tuyệt đối của định thức của ma trận kết quả sẽ cho bạn số lượng cây bao trùm.

Định thức của ma trận có thể được tìm trong $O(N^3)$ bằng phương pháp Gauss.

Chứng minh định lý này khá phức tạp và không được trình bày ở đây; để biết phác thảo chứng minh và các biến thể của định lý cho đồ thị không có cạnh nhiều lần và cho đồ thị có hướng, hãy tham khảo Wikipedia.

### 3.20.2 Mối liên hệ với định luật mạch điện của Kirchhoff

Định lý ma trận cây của Kirchhoff và các định luật của Kirchhoff cho mạch điện có mối liên hệ đẹp đẽ. Có thể chứng minh (sử dụng định luật Ohm và định luật thứ nhất của Kirchhoff) rằng điện trở $R_{ij}$ giữa hai điểm $i$ và $j$ của mạch là:

$$R_{ij} = \frac{\left|L^{(i,j)}\right|}{|L^j|}.$$

Ở đây, ma trận $L$ được thu từ ma trận nghịch đảo điện trở $A$ ($A_{i,j}$ là nghịch đảo của điện trở của dây dẫn giữa các điểm $i$ và $j$) bằng quy trình được mô tả trong định lý ma trận cây của Kirchhoff. $T^j$ là ma trận với hàng và cột $j$ bị xóa, $T^{(i,j)}$ là ma trận với hai hàng và hai cột $i$ và $j$ bị xóa.

Định lý của Kirchhoff mang lại ý nghĩa hình học cho công thức này.

## 3.21 Mã Prüfer

Trong bài viết này, chúng ta sẽ xem xét cái gọi là mã Prüfer (hoặc chuỗi Prüfer), một cách mã hóa một cây có gắn nhãn thành một chuỗi số theo cách duy nhất.

Với sự trợ giúp của mã Prüfer, chúng ta sẽ chứng minh công thức Cayley (công thức xác định số lượng cây bao trùm trong một đồ thị đầy đủ). Chúng ta cũng sẽ trình bày giải pháp cho bài toán đếm số cách thêm cạnh vào một đồ thị để làm cho nó liên thông.

**Lưu ý:** Chúng ta sẽ không xem xét các cây chỉ gồm một đỉnh - đây là một trường hợp đặc biệt mà nhiều phát biểu không áp dụng được.

### 3.21.1 Mã Prüfer

Mã Prüfer là một cách mã hóa một cây có gắn nhãn với $n$ đỉnh bằng một chuỗi gồm $n-2$ số nguyên trong khoảng $[0; n-1]$. Cách mã hóa này cũng đóng vai trò như một song ánh giữa tất cả các cây bao trùm của một đồ thị đầy đủ và các chuỗi số.

Mặc dù việc sử dụng mã Prüfer để lưu trữ và thao tác trên cây là không thực tế do đặc điểm của cách biểu diễn, mã Prüfer được sử dụng thường xuyên: chủ yếu trong việc giải quyết các bài toán tổ hợp.

Người phát minh - Heinz Prüfer - đã đề xuất mã này vào năm 1918 như một chứng minh cho công thức Cayley.

#### Xây dựng mã Prüfer cho một cây đã cho

Mã Prüfer được xây dựng như sau. Chúng ta sẽ lặp lại quy trình sau $n-2$ lần: chọn lá của cây có số nhỏ nhất, loại bỏ nó khỏi cây, và ghi lại số của đỉnh được nối với nó. Sau $n-2$ lần lặp, chỉ còn lại 2 đỉnh, và thuật toán kết thúc.

Do đó, mã Prüfer cho một cây đã cho là một chuỗi gồm $n-2$ số, trong đó mỗi số là số của đỉnh được nối, tức là số này nằm trong khoảng $[0, n-1]$.

Thuật toán tính mã Prüfer có thể được triển khai dễ dàng với độ phức tạp thời gian $O(n \log n)$, chỉ cần sử dụng một cấu trúc dữ liệu để trích xuất giá trị nhỏ nhất (ví dụ: set hoặc priority_queue trong C++), chứa danh sách tất cả các lá hiện tại.

```
vector<vector<int>> adj;

vector<int> pruefer_code() {
    int n = adj.size();
    set<int> leafs;
    vector<int> degree(n);
    vector<bool> killed(n, false);
    for (int i = 0; i < n; i++) {
        degree[i] = adj[i].size();
        if (degree[i] == 1)
            leafs.insert(i);
    }

    vector<int> code(n - 2);
    for (int i = 0; i < n - 2; i++) {
        int leaf = *leafs.begin();
```

```
        leafs.erase(leafs.begin());
        killed[leaf] = true;

        int v;
        for (int u : adj[leaf]) {
            if (!killed[u])
                v = u;
        }

        code[i] = v;
        if (--degree[v] == 1)
            leafs.insert(v);
    }

    return code;
}
```

Tuy nhiên, việc xây dựng này cũng có thể được triển khai trong thời gian tuyến tính. Cách tiếp cận như vậy được mô tả trong phần tiếp theo.

### Xây dựng mã Prüfer cho một cây đã cho trong thời gian tuyến tính

Bản chất của thuật toán là sử dụng một con trỏ di chuyển, con trỏ này sẽ luôn chỉ đến đỉnh lá hiện tại mà chúng ta muốn loại bỏ.

Thoạt nhìn, điều này dường như là không thể, vì trong quá trình xây dựng mã Prüfer, số lượng lá có thể tăng hoặc giảm. Tuy nhiên, sau khi xem xét kỹ hơn, điều này thực ra không đúng. Số lượng lá sẽ không tăng. Hoặc số lượng giảm đi một (chúng ta loại bỏ một đỉnh lá và không có thêm lá mới), hoặc số lượng giữ nguyên (chúng ta loại bỏ một đỉnh lá và có thêm một lá mới). Trong trường hợp đầu tiên, không có cách nào khác ngoài việc tìm kiếm đỉnh lá nhỏ nhất tiếp theo. Tuy nhiên, trong trường hợp thứ hai, chúng ta có thể quyết định trong $O(1)$ liệu có thể tiếp tục sử dụng đỉnh vừa trở thành lá mới hay phải tìm kiếm đỉnh lá nhỏ nhất tiếp theo. Và trong khá nhiều trường hợp, chúng ta có thể tiếp tục với đỉnh lá mới.

Để làm điều này, chúng ta sẽ sử dụng một biến ptr, biến này chỉ ra rằng trong tập hợp các đỉnh từ 0 đến ptr có tối đa một đỉnh lá, cụ thể là đỉnh lá hiện tại. Tất cả các đỉnh khác trong phạm vi đó hoặc đã bị loại khỏi cây, hoặc vẫn có nhiều hơn một đỉnh kề. Đồng thời, chúng ta nói rằng chưa loại bỏ bất kỳ đỉnh lá nào lớn hơn ptr.

Biến này đã rất hữu ích trong trường hợp đầu tiên. Sau khi loại bỏ đỉnh lá hiện tại, chúng ta biết rằng không thể có đỉnh lá nào giữa 0 và ptr, do đó chúng ta có thể bắt đầu tìm kiếm đỉnh tiếp theo trực tiếp từ $\text{ptr} + 1$, và không cần bắt đầu lại từ đỉnh 0. Trong trường hợp thứ hai, chúng ta có thể phân biệt thêm hai trường hợp: Hoặc đỉnh lá mới

nhỏ hơn ptr, thì đây phải là đỉnh lá tiếp theo, vì chúng ta biết không có đỉnh nào nhỏ hơn ptr. Hoặc đỉnh lá mới lớn hơn, nhưng chúng ta cũng biết rằng nó phải lớn hơn ptr, và có thể bắt đầu tìm kiếm lại từ $\text{ptr} + 1$.

Mặc dù chúng ta có thể phải thực hiện nhiều lần tìm kiếm tuyến tính cho đỉnh lá tiếp theo, con trỏ ptr chỉ tăng và do đó độ phức tạp thời gian tổng cộng là $O(n)$.

```
vector<vector<int>> adj;
vector<int> parent;

void dfs(int v) {
    for (int u : adj[v]) {
        if (u != parent[v]) {
            parent[u] = v;
            dfs(u);
        }
    }
}

vector<int> pruefer_code() {
    int n = adj.size();
    parent.resize(n);
    parent[n-1] = -1;
    dfs(n-1);

    int ptr = -1;
    vector<int> degree(n);
    for (int i = 0; i < n; i++) {
        degree[i] = adj[i].size();
        if (degree[i] == 1 && ptr == -1)
            ptr = i;
    }

    vector<int> code(n - 2);
    int leaf = ptr;
    for (int i = 0; i < n - 2; i++) {
        int next = parent[leaf];
        code[i] = next;
        if (--degree[next] == 1 && next < ptr) {
            leaf = next;
        } else {
            ptr++;
```

```
            while (degree[ptr] != 1)
                ptr++;
            leaf = ptr;
        }
    }

    return code;
}
```

Trong mã, chúng ta đầu tiên tìm tổ tiên parent[$i$] cho mỗi đỉnh, tức là tổ tiên mà đỉnh này sẽ có sau khi chúng ta loại bỏ nó khỏi cây. Chúng ta có thể tìm tổ tiên này bằng cách đặt gốc cho cây tại đỉnh $n-1$. Điều này khả thi vì đỉnh $n-1$ sẽ không bao giờ bị loại khỏi cây. Chúng ta cũng tính bậc cho mỗi đỉnh. ptr là con trỏ chỉ ra kích thước tối thiểu của các đỉnh lá còn lại (trừ đỉnh lá hiện tại). Chúng ta sẽ gán đỉnh lá hiện tại với next, nếu nó cũng là một đỉnh lá và nhỏ hơn ptr, hoặc chúng ta bắt đầu tìm kiếm tuyến tính cho đỉnh lá nhỏ nhất bằng cách tăng con trỏ.

Dễ thấy rằng mã này có độ phức tạp $O(n)$.

### Một số đặc tính của mã Prüfer

Sau khi xây dựng mã Prüfer, hai đỉnh sẽ còn lại. Một trong số đó là đỉnh cao nhất $n-1$, nhưng không thể nói gì thêm về đỉnh còn lại.

Mỗi đỉnh xuất hiện trong mã Prüfer đúng một số lần cố định - bậc của nó trừ đi một. Điều này có thể dễ dàng kiểm tra, vì bậc sẽ giảm mỗi lần chúng ta ghi lại nhãn của nó trong mã, và chúng ta loại bỏ nó khi bậc là 1. Với hai đỉnh còn lại, sự thật này cũng đúng.

### Khôi phục cây bằng mã Prüfer

Để khôi phục cây, chỉ cần tập trung vào đặc tính được thảo luận trong phần trước. Chúng ta đã biết bậc của tất cả các đỉnh trong cây mong muốn. Do đó, chúng ta có thể tìm tất cả các đỉnh lá, và cả đỉnh lá đầu tiên bị loại bỏ ở bước đầu tiên (nó phải là đỉnh lá nhỏ nhất). Đỉnh lá này được nối với đỉnh tương ứng với số trong ô đầu tiên của mã Prüfer.

Như vậy, chúng ta đã tìm ra cạnh đầu tiên bị loại bỏ khi mã Prüfer được tạo ra. Chúng ta có thể thêm cạnh này vào đáp án và giảm bậc ở cả hai đầu của cạnh.

Chúng ta sẽ lặp lại thao tác này cho đến khi sử dụng hết tất cả các số của mã Prüfer: tìm đỉnh nhỏ nhất có bậc bằng 1, nối nó với đỉnh tiếp theo từ mã Prüfer, và giảm bậc.

Cuối cùng, chúng ta chỉ còn hai đỉnh có bậc bằng 1. Đây là các đỉnh không bị loại bỏ bởi quá trình mã Prüfer. Chúng ta nối chúng để có được cạnh cuối cùng của cây. Một trong số chúng sẽ luôn là đỉnh $n-1$.

Thuật toán này có thể được triển khai dễ dàng trong $O(n \log n)$: chúng ta sử dụng một cấu trúc dữ liệu hỗ trợ trích xuất giá trị nhỏ nhất (ví dụ: set<> hoặc priority_queue<> trong C++) để lưu trữ tất cả các đỉnh lá.

Triển khai sau đây trả về danh sách các cạnh tương ứng với cây.

```
vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
    int n = code.size() + 2;
    vector<int> degree(n, 1);
    for (int i : code)
        degree[i]++;

    set<int> leaves;
    for (int i = 0; i < n; i++) {
        if (degree[i] == 1)
            leaves.insert(i);
    }

    vector<pair<int, int>> edges;
    for (int v : code) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());

        edges.emplace_back(leaf, v);
        if (--degree[v] == 1)
            leaves.insert(v);
    }
    edges.emplace_back(*leaves.begin(), n-1);
    return edges;
}
```

### Khôi phục cây bằng mã Prüfer trong thời gian tuyến tính

Để có được cây trong thời gian tuyến tính, chúng ta có thể áp dụng kỹ thuật tương tự được sử dụng để tạo mã Prüfer trong thời gian tuyến tính.

Chúng ta không cần một cấu trúc dữ liệu để trích xuất giá trị nhỏ nhất. Thay vào đó, chúng ta có thể nhận thấy rằng, sau khi xử lý cạnh hiện tại, chỉ một đỉnh trở thành lá. Do đó, chúng ta có thể tiếp tục với đỉnh này, hoặc tìm một đỉnh nhỏ hơn bằng tìm kiếm tuyến tính bằng cách di chuyển một con trỏ.

```
vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
    int n = code.size() + 2;
    vector<int> degree(n, 1);
    for (int i : code)
        degree[i]++;
```

```
    int ptr = 0;
    while (degree[ptr] != 1)
        ptr++;
    int leaf = ptr;

    vector<pair<int, int>> edges;
    for (int v : code) {
        edges.emplace_back(leaf, v);
        if (--degree[v] == 1 && v < ptr) {
            leaf = v;
        } else {
            ptr++;
            while (degree[ptr] != 1)
                ptr++;
            leaf = ptr;
        }
    }
    edges.emplace_back(leaf, n-1);
    return edges;
}
```

### 3.21.2 Song ánh giữa cây và mã Prüfer

Với mỗi cây, tồn tại một mã Prüfer tương ứng với nó. Và với mỗi mã Prüfer, chúng ta có thể khôi phục cây ban đầu.

Từ đó suy ra rằng mọi mã Prüfer (tức là một chuỗi gồm $n-2$ số trong khoảng $[0; n-1]$) cũng tương ứng với một cây.

Do đó, tất cả các cây và tất cả các mã Prüfer tạo thành một song ánh (một sự tương ứng một-một).

### 3.21.3 Công thức Cayley

Công thức Cayley phát biểu rằng số lượng cây bao trùm trong một đồ thị đầy đủ có gắn nhãn với $n$ đỉnh là:

$$n^{n-2}$$

Có nhiều chứng minh cho công thức này. Sử dụng khái niệm mã Prüfer, phát biểu này không còn gây bất ngờ.

Thực tế, bất kỳ mã Prüfer nào với $n-2$ số trong khoảng $[0; n-1]$ đều tương ứng với một cây nào đó với $n$ đỉnh. Vì vậy, chúng ta có $n^{n-2}$ mã Prüfer khác nhau như vậy. Vì mỗi cây như vậy là một cây bao trùm của một đồ thị đầy đủ với $n$ đỉnh, số lượng các cây bao trùm như vậy cũng là $n^{n-2}$.

### 3.21.4 Số cách làm cho một đồ thị liên thông

Khái niệm mã Prüfer còn mạnh mẽ hơn nữa. Nó cho phép tạo ra các công thức tổng quát hơn nhiều so với công thức Cayley.

Trong bài toán này, chúng ta được cho một đồ thị với $n$ đỉnh và $m$ cạnh. Đồ thị hiện tại có $k$ thành phần liên thông. Chúng ta muốn tính số cách thêm $k-1$ cạnh để đồ thị trở thành liên thông (rõ ràng $k-1$ là số lượng tối thiểu cần thiết để làm cho đồ thị liên thông).

Hãy suy ra một công thức để giải bài toán này.

Chúng ta sử dụng $s_1, \ldots, s_k$ để chỉ kích thước của các thành phần liên thông trong đồ thị. Chúng ta không thể thêm cạnh trong một thành phần liên thông. Do đó, bài toán này rất giống với việc tìm kiếm số lượng cây bao trùm của một đồ thị đầy đủ với $k$ đỉnh. Điểm khác biệt duy nhất là mỗi đỉnh thực sự có kích thước $s_i$: mỗi cạnh nối với đỉnh $i$ thực sự nhân đáp án với $s_i$.

Vì vậy, để tính số cách có thể, điều quan trọng là đếm xem mỗi đỉnh trong số $k$ đỉnh được sử dụng bao nhiêu lần trong cây kết nối. Để có được công thức cho bài toán, cần tính tổng đáp án qua tất cả các bậc có thể.

Gọi $d_1, \ldots, d_k$ là bậc của các đỉnh trong cây sau khi kết nối các đỉnh. Tổng các bậc bằng hai lần số cạnh:

$$\sum_{i=1}^{k} d_i = 2k-2$$

Nếu đỉnh $i$ có bậc $d_i$, thì nó xuất hiện $d_i - 1$ lần trong mã Prüfer. Mã Prüfer cho một cây với $k$ đỉnh có độ dài $k-2$. Vì vậy, số cách chọn một mã với $k-2$ số mà số $i$ xuất hiện đúng $d_i - 1$ lần bằng hệ số đa thức:

$$\binom{k-2}{d_1-1, d_2-1, \ldots, d_k-1} = \frac{(k-2)!}{(d_1-1)!(d_2-1)!\cdots(d_k-1)!}.$$

Thực tế là mỗi cạnh kề với đỉnh $i$ nhân đáp án với $s_i$, chúng ta nhận được đáp án, giả sử bậc của các đỉnh là $d_1, \ldots, d_k$:

$$s_1^{d_1} \cdot s_2^{d_2} \cdots s_k^{d_k} \cdot \binom{k-2}{d_1-1, d_2-1, \ldots, d_k-1}$$

Để có được đáp án cuối cùng, chúng ta cần tính tổng này cho tất cả các cách chọn bậc:

$$\sum_{\substack{d_i \geq 1 \\ \sum_{i=1}^{k} d_i = 2k-2}} s_1^{d_1} \cdot s_2^{d_2} \cdots s_k^{d_k} \cdot \binom{k-2}{d_1 - 1, d_2 - 1, \ldots, d_k - 1}$$

Hiện tại, điều này trông giống như một đáp án rất phức tạp, tuy nhiên chúng ta có thể sử dụng định lý đa thức, định lý này phát biểu:

$$(x_1 + \cdots + x_m)^p = \sum_{\substack{c_i \geq 0 \\ \sum_{i=1}^{m} c_i = p}} x_1^{c_1} \cdot x_2^{c_2} \cdots x_m^{c_m} \cdot \binom{p}{c_1, c_2, \ldots c_m}$$

Điều này đã khá giống. Để sử dụng nó, chúng ta chỉ cần thay thế bằng $e_i = d_i - 1$:

$$\sum_{\substack{e_i \geq 0 \\ \sum_{i=1}^{k} e_i = k-2}} s_1^{e_1+1} \cdot s_2^{e_2+1} \cdots s_k^{e_k+1} \cdot \binom{k-2}{e_1, e_2, \ldots, e_k}$$

Sau khi áp dụng định lý đa thức, chúng ta nhận được đáp án cho bài toán:

$$s_1 \cdot s_2 \cdots s_k \cdot (s_1 + s_2 + \cdots + s_k)^{k-2} = s_1 \cdot s_2 \cdots s_k \cdot n^{k-2}$$

Tình cờ, công thức này cũng đúng cho $k = 1$.

## 3.22 Kiểm tra đồ thị có chu trình và tìm chu trình trong $O(M)$

Xét một đồ thị có hướng hoặc không hướng không có vòng lặp và không có cạnh nhiều lần. Chúng ta cần kiểm tra xem nó có phải là không có chu trình hay không, và nếu không, thì tìm một chu trình bất kỳ.

Chúng ta có thể giải bài toán này bằng cách sử dụng Tìm kiếm theo chiều sâu (DFS) trong $O(M)$, trong đó $M$ là số lượng cạnh.

### 3.22.1 Thuật toán

Chúng ta sẽ chạy một loạt DFS trên đồ thị. Ban đầu, tất cả các đỉnh được tô màu trắng (0). Từ mỗi đỉnh chưa được thăm (màu trắng), bắt đầu DFS, đánh dấu nó là màu xám (1) khi vào và đánh dấu là màu đen (2) khi thoát. Nếu DFS di chuyển đến một đỉnh màu xám, thì chúng ta đã tìm thấy một chu trình (nếu đồ thị không có hướng, cạnh đến đỉnh cha không được xem xét). Chính chu trình đó có thể được xây dựng lại bằng cách sử dụng mảng cha.

### 3.22.2 Triển khai

Dưới đây là triển khai cho đồ thị có hướng.

```
int n;
vector<vector<int>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))
            break;
    }

    if (cycle_start == -1) {
        cout << "Không có chu trình" << endl;
    } else {
        vector<int> cycle;
```

```
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Tìm thấy chu trình: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

Dưới đây là triển khai cho đồ thị không hướng. Lưu ý rằng trong phiên bản không hướng, nếu một đỉnh $v$ được tô màu đen, nó sẽ không bao giờ được thăm lại bởi DFS. Điều này là do chúng ta đã khám phá tất cả các cạnh nối với $v$ khi lần đầu tiên thăm nó. Thành phần liên thông chứa $v$ (sau khi loại bỏ cạnh giữa $v$ và cha của nó) phải là một cây, nếu DFS đã hoàn tất xử lý $v$ mà không tìm thấy chu trình. Vì vậy, chúng ta thậm chí không cần phân biệt giữa trạng thái xám và đen. Do đó, chúng ta có thể biến vector color kiểu char thành vector visited kiểu bool.

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // truyền đỉnh và đỉnh cha của nó
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // bỏ qua cạnh đến đỉnh cha
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
```

```
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Không có chu trình" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Tìm thấy chu trình: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

## 3.23 Tìm chu trình âm trong đồ thị

Cho một đồ thị có hướng có trọng số $G$ với $N$ đỉnh và $M$ cạnh. Hãy tìm một chu trình có trọng số âm trong đó, nếu chu trình như vậy tồn tại.

Trong một cách diễn đạt khác của bài toán, bạn phải tìm tất cả các cặp đỉnh có đường đi giữa chúng với trọng số nhỏ tùy ý.

Việc sử dụng các thuật toán khác nhau để giải hai biến thể này của bài toán là thuận tiện, vì vậy chúng ta sẽ thảo luận cả hai cách ở đây.

### 3.23.1 Sử dụng thuật toán Bellman-Ford

Thuật toán Bellman-Ford cho phép kiểm tra xem có tồn tại chu trình có trọng số âm trong đồ thị hay không, và nếu có, tìm một trong những chu trình này.

Chi tiết của thuật toán được mô tả trong bài viết về thuật toán Bellman-Ford. Ở đây, chúng ta chỉ mô tả ứng dụng của nó cho bài toán này.

Triển khai chuẩn của Bellman-Ford tìm kiếm một chu trình âm có thể đến được từ một đỉnh bắt đầu $v$; tuy nhiên, thuật toán có thể được sửa đổi để chỉ tìm kiếm bất kỳ chu trình âm nào trong đồ thị. Để làm điều này, ta cần đặt tất cả khoảng cách $d[i]$ về 0 thay vì vô cực — như thể ta đang tìm đường đi ngắn nhất từ tất cả các đỉnh đồng thời; tính hợp lệ của việc phát hiện chu trình âm không bị ảnh hưởng.

Thực hiện $N$ lần lặp của thuật toán Bellman-Ford. Nếu không có thay đổi nào trong lần lặp cuối cùng, thì không có chu trình có trọng số âm trong đồ thị. Ngược lại, lấy một đỉnh mà khoảng cách đến nó đã thay đổi, và đi từ đó qua các tổ tiên của nó cho đến khi tìm thấy một chu trình. Chu trình này sẽ là chu trình có trọng số âm mong muốn.

**Triển khai**

```
struct Edge {
    int a, b, cost;
};

int n, m;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, INF);
    vector<int> p(n, -1);
    int x;

    d[0] = 0;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] < INF && d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
```

```
        }
    }

    if (x == -1) {
        cout << "Không tìm thấy chu trình âm.";
    } else {
        for (int i = 0; i < n; ++i)
            x = p[x];

        vector<int> cycle;
        for (int v = x;; v = p[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());

        cout << "Chu trình âm: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}
```

### 3.23.2 Sử dụng thuật toán Floyd-Warshall

Thuật toán Floyd-Warshall cho phép giải biến thể thứ hai của bài toán - tìm tất cả các cặp đỉnh $(i, j)$ không có đường đi ngắn nhất giữa chúng (tức là tồn tại đường đi có trọng số nhỏ tùy ý).

Một lần nữa, chi tiết có thể được tìm thấy trong bài viết về Floyd-Warshall, và ở đây chúng ta chỉ mô tả ứng dụng của nó.

Chạy thuật toán Floyd-Warshall trên đồ thị. Ban đầu $d[v][v] = 0$ với mỗi $v$. Nhưng sau khi chạy thuật toán, $d[v][v]$ sẽ nhỏ hơn 0 nếu tồn tại một đường đi có độ dài âm từ $v$ đến $v$. Chúng ta có thể sử dụng điều này để tìm tất cả các cặp đỉnh không có đường đi ngắn nhất giữa chúng. Chúng ta lặp qua tất cả các cặp đỉnh $(i, j)$ và với mỗi cặp, kiểm tra xem chúng có đường đi ngắn nhất giữa chúng hay không. Để làm điều này, thử tất cả các khả năng cho một đỉnh trung gian $t$. $(i, j)$ không có đường đi ngắn nhất, nếu một trong các đỉnh trung gian $t$ có $d[t][t] < 0$ (tức là $t$ là một phần của chu trình có trọng số âm), $t$ có thể đến được từ $i$ và $j$ có thể đến được từ $t$. Khi đó, đường đi từ $i$ đến $j$ có thể có trọng số nhỏ tùy ý. Chúng ta sẽ biểu thị điều này bằng -INF.

**Triển khai**

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        for (int t = 0; t < n; ++t) {
            if (d[i][t] < INF && d[t][t] < 0 && d[t][j] < INF)
                d[i][j] = -INF;
        }
    }
}
```

## 3.24 Tìm đường đi Euler trong $O(M)$

Đường đi Euler là một đường đi trong đồ thị đi qua tất cả các cạnh của nó đúng một lần. Chu trình Euler là một đường đi Euler đồng thời là một chu trình.

Bài toán là tìm đường đi Euler trong một đa đồ thị không hướng có vòng lặp.

### 3.24.1 Thuật toán

Trước tiên, chúng ta có thể kiểm tra xem có tồn tại đường đi Euler hay không. Chúng ta có thể sử dụng định lý sau. Một chu trình Euler tồn tại nếu và chỉ nếu bậc của tất cả các đỉnh là số chẵn. Và một đường đi Euler tồn tại nếu và chỉ nếu số lượng đỉnh có bậc lẻ là hai (hoặc bằng không, trong trường hợp tồn tại chu trình Euler). Ngoài ra, đồ thị đương nhiên phải đủ liên thông (tức là, nếu bạn loại bỏ tất cả các đỉnh cô lập khỏi nó, bạn sẽ thu được một đồ thị liên thông).

Để tìm đường đi Euler / chu trình Euler, chúng ta có thể sử dụng chiến lược sau: Tìm tất cả các chu trình đơn giản và kết hợp chúng thành một - đây sẽ là chu trình Euler. Nếu đồ thị là loại mà đường đi Euler không phải là chu trình, thì thêm cạnh còn thiếu, tìm chu trình Euler, sau đó loại bỏ cạnh thừa.

Việc tìm tất cả các chu trình và kết hợp chúng có thể được thực hiện bằng một thủ tục đệ quy đơn giản:

```
thủ tục FindEulerPath(V)
  1. lặp qua tất cả các cạnh xuất phát từ đỉnh V;
       loại bỏ cạnh này khỏi đồ thị,
       và gọi FindEulerPath từ đầu kia của cạnh này;
  2. thêm đỉnh V vào đáp án.
```

Độ phức tạp của thuật toán này rõ ràng là tuyến tính đối với số lượng cạnh.

Nhưng chúng ta cũng có thể viết cùng thuật toán này ở phiên bản không đệ quy:

```
ngăn xếp St;
đặt đỉnh bắt đầu vào St;
cho đến khi St rỗng
  cho V là giá trị ở đỉnh St;
  nếu bậc(V) = 0, thì
    thêm V vào đáp án;
    loại bỏ V khỏi đỉnh St;
  ngược lại
    tìm bất kỳ cạnh nào xuất phát từ V;
    loại bỏ nó khỏi đồ thị;
    đặt đầu kia của cạnh này vào St;
```

Dễ dàng kiểm tra tính tương đương của hai dạng thuật toán này. Tuy nhiên, dạng thứ hai rõ ràng nhanh hơn, và mã sẽ hiệu quả hơn nhiều.

### 3.24.2 Bài toán Domino

Chúng ta đưa ra ở đây một bài toán chu trình Euler cổ điển - bài toán Domino.

Có $N$ quân domino, như đã biết, ở cả hai đầu của domino có ghi một số (thường từ 1 đến 6, nhưng trong trường hợp của chúng ta điều này không quan trọng). Bạn muốn xếp tất cả các quân domino thành một hàng sao cho các số trên hai quân domino liền kề, được ghi ở mặt chung của chúng, trùng nhau. Domino được phép xoay.

Định dạng lại bài toán. Gọi các số được ghi ở mặt dưới là các đỉnh của đồ thị, và các quân domino là các cạnh của đồ thị này (mỗi quân domino với các số $(a, b)$ là các cạnh $(a, b)$ và $(b, a)$). Khi đó bài toán của chúng ta được rút gọn thành bài toán tìm đường đi Euler trong đồ thị này.

### 3.24.3 Triển khai

Chương trình dưới đây tìm kiếm và xuất ra một chu trình hoặc đường đi Euler trong đồ thị, hoặc xuất $-1$ nếu nó không tồn tại.

Đầu tiên, chương trình kiểm tra bậc của các đỉnh: nếu không có đỉnh nào có bậc lẻ, thì đồ thị có chu trình Euler; nếu có 2 đỉnh có bậc lẻ, thì trong đồ thị chỉ có đường đi Euler (nhưng không có chu trình Euler); nếu có nhiều hơn 2 đỉnh như vậy, thì trong đồ thị không có chu trình Euler hay đường đi Euler. Để tìm đường đi Euler (không phải chu trình), hãy làm như sau: nếu $V1$ và $V2$ là hai đỉnh có bậc lẻ, thì chỉ cần thêm một cạnh $(V1, V2)$, trong đồ thị kết quả tìm chu trình Euler (nó rõ ràng sẽ tồn tại), sau đó loại bỏ cạnh "giả" $(V1, V2)$ khỏi đáp án. Chúng ta sẽ tìm chu trình Euler đúng như mô tả ở trên (phiên bản không đệ quy), và đồng thời ở cuối thuật toán này, chúng ta sẽ kiểm tra xem đồ thị có liên thông hay không (nếu đồ thị không liên thông, thì ở cuối thuật toán sẽ còn

lại một số cạnh trong đồ thị, và trong trường hợp này chúng ta cần in $-1$). Cuối cùng, chương trình cũng tính đến việc có thể có các đỉnh cô lập trong đồ thị.

Lưu ý rằng chúng ta sử dụng ma trận kề trong bài toán này. Ngoài ra, triển khai này xử lý việc tìm cạnh tiếp theo bằng cách vét cạn, đòi hỏi phải lặp qua toàn bộ hàng trong ma trận nhiều lần. Một cách tốt hơn là lưu đồ thị dưới dạng danh sách kề, loại bỏ cạnh trong $O(1)$ và đánh dấu các cạnh ngược trong danh sách riêng. Bằng cách này, chúng ta có thể đạt được thuật toán $O(N)$.

```
int main() {
    int n;
    vector<vector<int>> g(n, vector<int>(n));
    // đọc đồ thị vào ma trận kề

    vector<int> deg(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];
    }

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1;
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1)
                v1 = i;
            else if (v2 == -1)
                v2 = i;
            else
                bad = true;
        }
    }

    if (v1 != -1)
```

```
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i])
                break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i) {
            if ((res[i] == v1 && res[i + 1] == v2) ||
                (res[i] == v2 && res[i + 1] == v1)) {
                vector<int> res2;
                for (size_t j = i + 1; j < res.size(); ++j)
                    res2.push_back(res[j]);
                for (size_t j = 1; j <= i; ++j)
                    res2.push_back(res[j]);
                res = res2;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (g[i][j])
                bad = true;
```

```
        }
    }

    if (bad) {
        cout << -1;
    } else {
        for (int x : res)
            cout << x << " ";
    }
}
```

## 3.25 Tổ tiên chung thấp nhất - $O(\sqrt{N})$ và $O(\log N)$ với tiền xử lý $O(N)$

Cho một cây $G$. Cho các truy vấn dạng $(v_1, v_2)$, với mỗi truy vấn bạn cần tìm tổ tiên chung thấp nhất (hoặc tổ tiên chung gần nhất), tức là một đỉnh $v$ nằm trên đường đi từ gốc đến $v_1$ và đường đi từ gốc đến $v_2$, và đỉnh này phải là thấp nhất. Nói cách khác, đỉnh mong muốn $v$ là tổ tiên thấp nhất của $v_1$ và $v_2$. Rõ ràng tổ tiên chung thấp nhất của chúng nằm trên đường đi ngắn nhất từ $v_1$ đến $v_2$. Ngoài ra, nếu $v_1$ là tổ tiên của $v_2$, thì $v_1$ chính là tổ tiên chung thấp nhất của chúng.

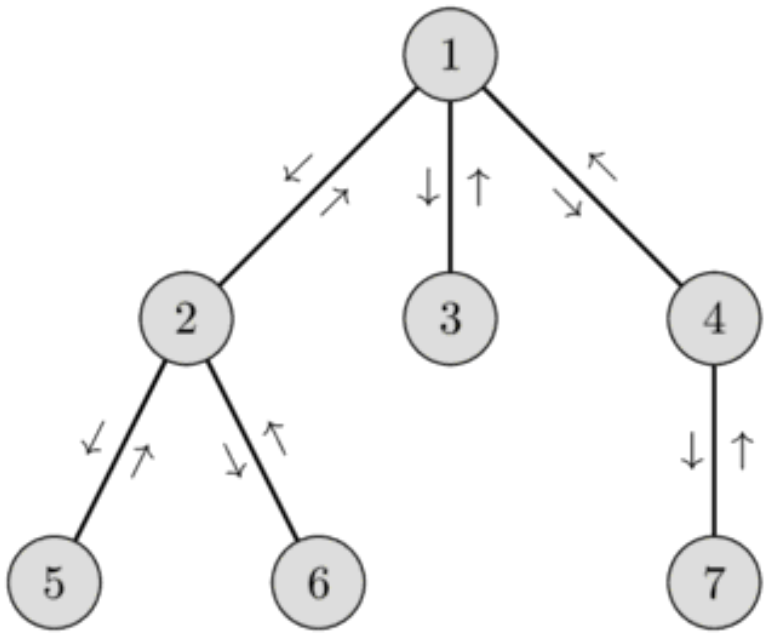
### 3.25.1 Ý tưởng của thuật toán

Trước khi trả lời các truy vấn, chúng ta cần tiền xử lý cây. Chúng ta thực hiện một lượt duyệt DFS bắt đầu từ gốc và xây dựng một danh sách euler lưu trữ thứ tự các đỉnh mà chúng ta thăm (một đỉnh được thêm vào danh sách khi chúng ta thăm nó lần đầu tiên, và sau khi DFS quay lại từ các con của nó). Điều này còn được gọi là hành trình Euler của cây. Rõ ràng kích thước của danh sách này sẽ là $O(N)$. Chúng ta cũng cần xây dựng một mảng first$[0..N-1]$ lưu trữ cho mỗi đỉnh $i$ lần xuất hiện đầu tiên của nó trong euler. Tức là, vị trí đầu tiên trong euler sao cho euler[first$[i]] = i$. Ngoài ra, bằng cách sử dụng DFS, chúng ta có thể tìm chiều cao của mỗi nút (khoảng cách từ gốc đến nó) và lưu trữ trong mảng height$[0..N-1]$.

Vậy làm thế nào để trả lời các truy vấn bằng hành trình Euler và hai mảng bổ sung này? Giả sử truy vấn là một cặp $v_1$ và $v_2$. Xét các đỉnh mà chúng ta thăm trong hành trình Euler giữa lần thăm đầu tiên của $v_1$ và lần thăm đầu tiên của $v_2$. Dễ thấy rằng LCA$(v_1, v_2)$ là đỉnh có chiều cao nhỏ nhất trên đường đi này. Chúng ta đã nhận thấy rằng LCA phải là một phần của đường đi ngắn nhất giữa $v_1$ và $v_2$. Rõ ràng nó cũng phải là đỉnh có chiều cao nhỏ nhất. Trong hành trình Euler, chúng ta thực chất sử dụng đường đi ngắn nhất, ngoại trừ việc chúng ta còn thăm thêm tất cả các cây con mà chúng ta tìm

thấy trên đường đi. Nhưng tất cả các đỉnh trong các cây con này đều thấp hơn trong cây so với LCA và do đó có chiều cao lớn hơn. Vì vậy, LCA$(v_1, v_2)$ có thể được xác định duy nhất bằng cách tìm đỉnh có chiều cao nhỏ nhất trong hành trình Euler giữa first$(v_1)$ và first$(v_2)$.

Hãy minh họa ý tưởng này. Xét đồ thị sau và hành trình Euler với các chiều cao tương ứng:




Hình 3.7: LCA

| Đỉnh: | 1 | 2 | 5 | 2 | 6 | 2 | 1 | 3 | 1 | 4 | 7 | 4 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Chiều cao: | 1 | 2 | 3 | 2 | 3 | 2 | 1 | 2 | 1 | 2 | 3 | 2 | 1 |

Hành trình bắt đầu từ đỉnh 6 và kết thúc tại 4, chúng ta thăm các đỉnh $[6, 2, 1, 3, 1, 4]$. Trong số các đỉnh này, đỉnh 1 có chiều cao nhỏ nhất, do đó LCA(6, 4) = 1.

Tóm lại: để trả lời một truy vấn, chúng ta chỉ cần tìm đỉnh có chiều cao nhỏ nhất trong mảng euler trong khoảng từ first$[v_1]$ đến first$[v_2]$. Như vậy, bài toán LCA được rút gọn thành bài toán RMQ (tìm giá trị nhỏ nhất trong một đoạn).

Sử dụng Phân tách căn bậc hai (Sqrt-Decomposition), có thể thu được giải pháp trả lời mỗi truy vấn trong $O(\sqrt{N})$ với thời gian tiền xử lý $O(N)$.

Sử dụng Cây phân đoạn (Segment Tree), bạn có thể trả lời mỗi truy vấn trong $O(\log N)$ với thời gian tiền xử lý $O(N)$.

Vì hầu như không bao giờ có cập nhật cho các giá trị được lưu trữ, Bảng thưa (Sparse Table) có thể là lựa chọn tốt hơn, cho phép trả lời truy vấn trong $O(1)$ với thời gian xây dựng $O(N \log N)$.

### 3.25.2 Triển khai

Trong triển khai sau của thuật toán LCA, một Cây phân đoạn được sử dụng.

```
struct LCA {
    vector<int> height, euler, first, segtree;
    vector<bool> visited;
    int n;

    LCA(vector<vector<int>> &adj, int root = 0) {
        n = adj.size();
        height.resize(n);
        first.resize(n);
        euler.reserve(n * 2);
        visited.assign(n, false);
        dfs(adj, root);
        int m = euler.size();
        segtree.resize(m * 4);
        build(1, 0, m - 1);
    }

    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
        visited[node] = true;
        height[node] = h;
        first[node] = euler.size();
        euler.push_back(node);
        for (auto to : adj[node]) {
            if (!visited[to]) {
                dfs(adj, to, h + 1);
                euler.push_back(node);
            }
        }
    }

    void build(int node, int b, int e) {
        if (b == e) {
            segtree[node] = euler[b];
```

```
        } else {
            int mid = (b + e) / 2;
            build(node << 1, b, mid);
            build(node << 1 | 1, mid + 1, e);
            int l = segtree[node << 1], r = segtree[node << 1 | 1];
            segtree[node] = (height[l] < height[r]) ? l : r;
        }
    }

    int query(int node, int b, int e, int L, int R) {
        if (b > R || e < L)
            return -1;
        if (b >= L && e <= R)
            return segtree[node];
        int mid = (b + e) >> 1;

        int left = query(node << 1, b, mid, L, R);
        int right = query(node << 1 | 1, mid + 1, e, L, R);
        if (left == -1) return right;
        if (right == -1) return left;
        return height[left] < height[right] ? left : right;
    }

    int lca(int u, int v) {
        int left = first[u], right = first[v];
        if (left > right)
            swap(left, right);
        return query(1, 0, euler.size() - 1, left, right);
    }
};
```

## 3.26 Tổ tiên chung thấp nhất - Nâng nhị phân

Cho một cây $G$. Với mỗi truy vấn dạng $(u, v)$, chúng ta muốn tìm tổ tiên chung thấp nhất của các nút $u$ và $v$, tức là chúng ta muốn tìm một nút $w$ nằm trên đường đi từ $u$ đến nút gốc, đồng thời nằm trên đường đi từ $v$ đến nút gốc, và nếu có nhiều nút thỏa mãn, chúng ta chọn nút xa nhất so với nút gốc. Nói cách khác, nút mong muốn $w$ là tổ tiên thấp nhất của $u$ và $v$. Đặc biệt, nếu $u$ là tổ tiên của $v$, thì $u$ chính là tổ tiên chung thấp nhất của chúng.

Thuật toán được mô tả trong bài viết này sẽ cần $O(N \log N)$ để tiền xử lý cây, và sau

đó $O(\log N)$ cho mỗi truy vấn LCA.

### 3.26.1 Thuật toán

Với mỗi nút, chúng ta sẽ tiền tính tổ tiên phía trên nó, tổ tiên cách nó hai nút phía trên, tổ tiên cách nó bốn nút phía trên, v.v. Hãy lưu trữ chúng trong mảng up, tức là up$[i][j]$ là tổ tiên thứ $2^j$ phía trên nút $i$ với $i = 1...N$, $j = 0...ceil(\log(N))$. Những thông tin này cho phép chúng ta nhảy từ bất kỳ nút nào đến bất kỳ tổ tiên nào phía trên nó trong thời gian $O(\log N)$. Chúng ta có thể tính mảng này bằng cách sử dụng một lượt duyệt DFS trên cây.

Với mỗi nút, chúng ta cũng sẽ ghi nhớ thời điểm thăm đầu tiên của nút này (tức là thời điểm DFS phát hiện ra nút), và thời điểm rời khỏi nó (tức là sau khi thăm tất cả các con và thoát khỏi hàm DFS). Chúng ta có thể sử dụng thông tin này để xác định trong thời gian hằng số liệu một nút có phải là tổ tiên của nút khác hay không.

Giả sử bây giờ chúng ta nhận được một truy vấn $(u, v)$. Chúng ta có thể kiểm tra ngay lập tức liệu một nút có phải là tổ tiên của nút kia hay không. Trong trường hợp này, nút đó đã là LCA. Nếu $u$ không phải là tổ tiên của $v$, và $v$ không phải là tổ tiên của $u$, chúng ta leo lên các tổ tiên của $u$ cho đến khi tìm thấy nút cao nhất (tức là gần gốc nhất) không phải là tổ tiên của $v$ (tức là một nút $x$, sao cho $x$ không phải là tổ tiên của $v$, nhưng up$[x][0]$ thì có). Chúng ta có thể tìm nút $x$ này trong thời gian $O(\log N)$ bằng cách sử dụng mảng up.

Chúng ta sẽ mô tả quá trình này chi tiết hơn. Gọi $L = ceil(\log(N))$. Giả sử ban đầu $i = L$. Nếu up$[u][i]$ không phải là tổ tiên của $v$, thì chúng ta có thể gán $u =$ up$[u][i]$ và giảm $i$. Nếu up$[u][i]$ là tổ tiên, thì chúng ta chỉ giảm $i$. Rõ ràng sau khi thực hiện điều này cho tất cả $i$ không âm, nút $u$ sẽ là nút mong muốn - tức là $u$ vẫn không phải là tổ tiên của $v$, nhưng up$[u][0]$ thì có.

Bây giờ, rõ ràng đáp án cho LCA sẽ là up$[u][0]$ - tức là, nút nhỏ nhất trong số các tổ tiên của nút $u$, đồng thời cũng là tổ tiên của $v$.

Vì vậy, để trả lời một truy vấn LCA, chúng ta lặp $i$ từ $ceil(\log(N))$ đến 0 và kiểm tra trong mỗi lần lặp liệu một nút có phải là tổ tiên của nút kia hay không. Do đó, mỗi truy vấn có thể được trả lời trong $O(\log N)$.

### 3.26.2 Triển khai

```
int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;
```

```
void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
```

```
}
```

## 3.27 Tổ tiên chung thấp nhất - Thuật toán Farach-Colton và Bender

Cho một cây $G$. Với mỗi truy vấn dạng $(u, v)$, chúng ta muốn tìm tổ tiên chung thấp nhất của các nút $u$ và $v$, tức là chúng ta muốn tìm một nút $w$ nằm trên đường đi từ $u$ đến nút gốc, đồng thời nằm trên đường đi từ $v$ đến nút gốc, và nếu có nhiều nút thỏa mãn, chúng ta chọn nút xa nhất so với nút gốc. Nói cách khác, nút mong muốn $w$ là tổ tiên thấp nhất của $u$ và $v$. Đặc biệt, nếu $u$ là tổ tiên của $v$, thì $u$ chính là tổ tiên chung thấp nhất của chúng.

Thuật toán được mô tả trong bài viết này được phát triển bởi Farach-Colton và Bender. Nó là tối ưu về mặt tiệm cận.

### 3.27.1 Thuật toán

Chúng ta sử dụng cách rút gọn cổ điển của bài toán LCA thành bài toán RMQ. Chúng ta duyệt qua tất cả các nút của cây bằng DFS và giữ một mảng chứa tất cả các nút đã thăm cùng với chiều cao của các nút này. Tổ tiên chung thấp nhất của hai nút $u$ và $v$ là nút nằm giữa các lần xuất hiện của $u$ và $v$ trong hành trình, có chiều cao nhỏ nhất.

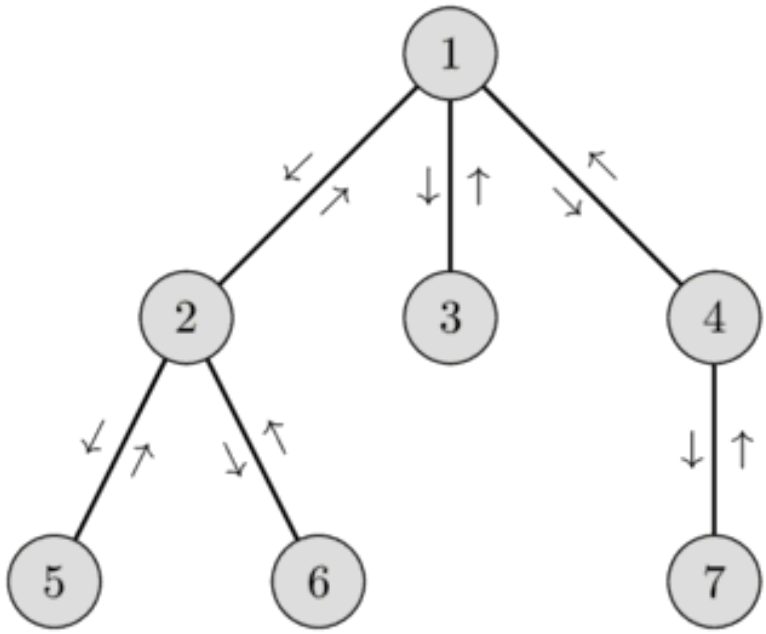
Trong hình sau, bạn có thể thấy một hành trình Euler khả thi của một đồ thị và trong danh sách dưới đây, bạn có thể thấy các nút đã thăm cùng với chiều cao của chúng.

| Nút: | 1 | 2 | 5 | 2 | 6 | 2 | 1 | 3 | 1 | 4 | 7 | 4 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Chiều cao: | 1 | 2 | 3 | 2 | 3 | 2 | 1 | 2 | 1 | 2 | 3 | 2 | 1 |

Bạn có thể đọc thêm về cách rút gọn này trong bài viết "Tổ tiên chung thấp nhất". Trong bài viết đó, giá trị nhỏ nhất của một đoạn được tìm bằng phân tách căn bậc hai trong $O(\sqrt{N})$ hoặc trong $O(\log N)$ bằng cách sử dụng Cây phân đoạn. Trong bài viết này, chúng ta xem xét cách giải quyết các truy vấn tìm giá trị nhỏ nhất trong đoạn đã cho trong thời gian $O(1)$, trong khi vẫn chỉ mất $O(N)$ thời gian để tiền xử lý.

Lưu ý rằng bài toán RMQ đã rút gọn rất đặc biệt: bất kỳ hai phần tử liền kề nào trong mảng chỉ khác nhau đúng một đơn vị (vì các phần tử của mảng không gì khác ngoài chiều cao của các nút được thăm theo thứ tự duyệt, và chúng ta hoặc đi đến một nút con, trong trường hợp đó phần tử tiếp theo lớn hơn một, hoặc quay lại tổ tiên, trong trường hợp đó phần tử tiếp theo nhỏ hơn một). Thuật toán Farach-Colton và Bender mô tả một giải pháp cho chính bài toán RMQ đặc biệt này.

Gọi $A$ là mảng mà chúng ta muốn thực hiện các truy vấn tìm giá trị nhỏ nhất trong đoạn. Và $N$ sẽ là kích thước của $A$.




Hình 3.8: LCA

Có một cấu trúc dữ liệu đơn giản mà chúng ta có thể sử dụng để giải bài toán RMQ với tiền xử lý $O(N \log N)$ và $O(1)$ cho mỗi truy vấn: Bảng thưa (Sparse Table). Chúng ta tạo một bảng $T$ mà mỗi phần tử $T[i][j]$ bằng giá trị nhỏ nhất của $A$ trong khoảng $[i, i + 2^j - 1]$. Rõ ràng $0 \leq j \leq \lceil \log N \rceil$, và do đó kích thước của Bảng thưa sẽ là $O(N \log N)$. Bạn có thể xây dựng bảng này dễ dàng trong $O(N \log N)$ bằng cách nhận thấy rằng $T[i][j] = \min(T[i][j-1], T[i + 2^{j-1}][j-1])$.

Làm thế nào để trả lời một truy vấn RMQ trong $O(1)$ bằng cách sử dụng cấu trúc dữ liệu này? Giả sử truy vấn nhận được là $[l, r]$, thì đáp án là $\min(T[l][\text{sz}], T[r - 2^{\text{sz}} + 1][\text{sz}])$, trong đó sz là lũy thừa lớn nhất sao cho $2^{\text{sz}}$ không lớn hơn độ dài đoạn $r - l + 1$. Thực tế, chúng ta có thể lấy đoạn $[l, r]$ và bao phủ nó bằng hai đoạn có độ dài $2^{\text{sz}}$ - một đoạn bắt đầu từ $l$ và đoạn kia kết thúc tại $r$. Các đoạn này chồng lấn, nhưng điều này không ảnh hưởng đến tính toán của chúng ta. Để thực sự đạt được độ phức tạp thời gian $O(1)$ cho mỗi truy vấn, chúng ta cần biết giá trị của sz cho tất cả các độ dài có thể từ 1 đến $N$. Nhưng điều này có thể được tính trước dễ dàng.

Bây giờ chúng ta muốn cải thiện độ phức tạp của việc tiền xử lý xuống còn $O(N)$.

Chúng ta chia mảng $A$ thành các khối có kích thước $K = 0.5 \log N$ với log là logarit cơ số 2. Với mỗi khối, chúng ta tính phần tử nhỏ nhất và lưu chúng vào một mảng $B$. $B$ có kích thước $\frac{N}{K}$. Chúng ta xây dựng một bảng thưa từ mảng $B$. Kích thước và độ phức tạp thời gian của nó sẽ là:

$$\frac{N}{K}\log\left(\frac{N}{K}\right) = \frac{2N}{\log(N)}\log\left(\frac{2N}{\log(N)}\right) =$$

$$= \frac{2N}{\log(N)}\left(1 + \log\left(\frac{N}{\log(N)}\right)\right) \leq \frac{2N}{\log(N)} + 2N = O(N)$$

Bây giờ chúng ta chỉ cần học cách trả lời nhanh các truy vấn tìm giá trị nhỏ nhất trong đoạn trong mỗi khối. Thực tế, nếu truy vấn tìm giá trị nhỏ nhất trong đoạn nhận được là $[l, r]$ và $l$ và $r$ nằm ở các khối khác nhau, thì đáp án là giá trị nhỏ nhất của ba giá trị sau: giá trị nhỏ nhất của hậu tố của khối chứa $l$ bắt đầu tại $l$, giá trị nhỏ nhất của tiền tố của khối chứa $r$ kết thúc tại $r$, và giá trị nhỏ nhất của các khối ở giữa chúng. Giá trị nhỏ nhất của các khối ở giữa có thể được trả lời trong $O(1)$ bằng cách sử dụng Bảng thưa. Vì vậy, điều này chỉ để lại cho chúng ta các truy vấn tìm giá trị nhỏ nhất trong đoạn bên trong các khối.

Ở đây chúng ta sẽ khai thác đặc tính của mảng. Hãy nhớ rằng các giá trị trong mảng - vốn chỉ là các giá trị chiều cao trong cây - sẽ luôn khác nhau một đơn vị. Nếu chúng ta loại bỏ phần tử đầu tiên của một khối và trừ nó khỏi mọi phần tử khác trong khối, mỗi khối có thể được xác định bằng một chuỗi có độ dài $K - 1$ gồm các số $+1$ và $-1$. Vì các khối này rất nhỏ, chỉ có một vài chuỗi khác nhau có thể xảy ra. Số lượng chuỗi có thể là:

$$2^{K-1} = 2^{0.5\log(N)-1} = 0.5\left(2^{\log(N)}\right)^{0.5} = 0.5\sqrt{N}$$

Do đó, số lượng khối khác nhau là $O(\sqrt{N})$, và vì vậy chúng ta có thể tính trước kết quả của các truy vấn tìm giá trị nhỏ nhất trong đoạn bên trong tất cả các khối khác nhau trong thời gian $O(\sqrt{N}K^2) = O(\sqrt{N}\log^2(N)) = O(N)$. Để triển khai, chúng ta có thể đặc trưng một khối bằng một mặt nạ bit có độ dài $K - 1$ (sẽ vừa với một kiểu int tiêu chuẩn) và lưu chỉ số của giá trị nhỏ nhất trong một mảng block[mask][$l$][$r$] có kích thước $O(\sqrt{N}\log^2(N))$.

Vậy chúng ta đã học cách tính trước các truy vấn tìm giá trị nhỏ nhất trong đoạn trong mỗi khối, cũng như các truy vấn tìm giá trị nhỏ nhất trong đoạn trên một dải các khối, tất cả trong $O(N)$. Với các tính toán trước này, chúng ta có thể trả lời mỗi truy vấn trong $O(1)$, bằng cách sử dụng tối đa bốn giá trị đã tính trước: giá trị nhỏ nhất của khối chứa $l$, giá trị nhỏ nhất của khối chứa $r$, và hai giá trị nhỏ nhất của các đoạn chồng lấn của các khối ở giữa chúng.

### 3.27.2 Triển khai

```
int n;
vector<vector<int>> adj;

int block_size, block_cnt;
```

```
vector<int> first_visit;
vector<int> euler_tour;
vector<int> height;
vector<int> log_2;
vector<vector<int>> st;
vector<vector<vector<int>>> blocks;
vector<int> block_mask;

void dfs(int v, int p, int h) {
    first_visit[v] = euler_tour.size();
    euler_tour.push_back(v);
    height[v] = h;

    for (int u : adj[v]) {
        if (u == p)
            continue;
        dfs(u, v, h + 1);
        euler_tour.push_back(v);
    }
}

int min_by_h(int i, int j) {
    return height[euler_tour[i]] < height[euler_tour[j]] ? i : j;
}

void precompute_lca(int root) {
    // lấy hành trình Euler & chỉ số của lần xuất hiện đầu tiên
    first_visit.assign(n, -1);
    height.assign(n, 0);
    euler_tour.reserve(2 * n);
    dfs(root, -1, 0);

    // tính trước tất cả giá trị log
    int m = euler_tour.size();
    log_2.reserve(m + 1);
    log_2.push_back(-1);
    for (int i = 1; i <= m; i++)
        log_2.push_back(log_2[i / 2] + 1);

    block_size = max(1, log_2[m] / 2);
```

```
    block_cnt = (m + block_size - 1) / block_size;

    // tính trước giá trị nhỏ nhất của mỗi khối và xây dựng bảng thưa
    st.assign(block_cnt, vector<int>(log_2[block_cnt] + 1));
    for (int i = 0, j = 0, b = 0; i < m; i++, j++) {
        if (j == block_size)
            j = 0, b++;
        if (j == 0 || min_by_h(i, st[b][0]) == i)
            st[b][0] = i;
    }
    for (int l = 1; l <= log_2[block_cnt]; l++) {
        for (int i = 0; i < block_cnt; i++) {
            int ni = i + (1 << (l - 1));
            if (ni >= block_cnt)
                st[i][l] = st[i][l-1];
            else
                st[i][l] = min_by_h(st[i][l-1], st[ni][l-1]);
        }
    }

    // tính trước mặt nạ cho mỗi khối
    block_mask.assign(block_cnt, 0);
    for (int i = 0, j = 0, b = 0; i < m; i++, j++) {
        if (j == block_size)
            j = 0, b++;
        if (j > 0 && (i >= m || min_by_h(i - 1, i) == i - 1))
            block_mask[b] += 1 << (j - 1);
    }

    // tính trước RMQ cho mỗi khối duy nhất
    int possibilities = 1 << (block_size - 1);
    blocks.resize(possibilities);
    for (int b = 0; b < block_cnt; b++) {
        int mask = block_mask[b];
        if (!blocks[mask].empty())
            continue;
        blocks[mask].assign(block_size, vector<int>(block_size));
        for (int l = 0; l < block_size; l++) {
            blocks[mask][l][l] = l;
            for (int r = l + 1; r < block_size; r++) {
```

```
                blocks[mask][l][r] = blocks[mask][l][r - 1];
                if (b * block_size + r < m)
                    blocks[mask][l][r] = min_by_h(b * block_size + blocks[mask][l][r]
                            b * block_size + r) - b * block_size;
            }
        }
    }
}

int lca_in_block(int b, int l, int r) {
    return blocks[block_mask[b]][l][r] + b * block_size;
}

int lca(int v, int u) {
    int l = first_visit[v];
    int r = first_visit[u];
    if (l > r)
        swap(l, r);
    int bl = l / block_size;
    int br = r / block_size;
    if (bl == br)
        return euler_tour[lca_in_block(bl, l % block_size, r % block_size)];
    int ans1 = lca_in_block(bl, l % block_size, block_size - 1);
    int ans2 = lca_in_block(br, 0, r % block_size);
    int ans = min_by_h(ans1, ans2);
    if (bl + 1 < br) {
        int l = log_2[br - bl - 1];
        int ans3 = st[bl+1][l];
        int ans4 = st[br - (1 << l)][l];
        ans = min_by_h(ans, min_by_h(ans3, ans4));
    }
    return euler_tour[ans];
}
```

## 3.28 Giải bài toán RMQ (Truy vấn giá trị nhỏ nhất trong đoạn) bằng cách tìm LCA (Tổ tiên chung thấp nhất)

Cho một mảng $A[0..N-1]$. Với mỗi truy vấn dạng $[L, R]$, chúng ta muốn tìm giá trị nhỏ nhất trong mảng $A$ bắt đầu từ vị trí $L$ và kết thúc tại vị trí $R$. Chúng ta sẽ giả định rằng mảng $A$ không thay đổi trong quá trình này, tức là bài viết này mô tả một giải pháp cho bài toán RMQ tĩnh.

Dưới đây là mô tả về một giải pháp tối ưu tiệm cận. Nó khác biệt so với các giải pháp khác cho bài toán RMQ, vì nó rất khác: nó rút gọn bài toán RMQ thành bài toán LCA, sau đó sử dụng thuật toán Farach-Colton và Bender, thuật toán này lại rút gọn bài toán LCA về một bài toán RMQ đặc biệt và giải quyết bài toán đó.

### 3.28.1 Thuật toán

Chúng ta xây dựng một cây Descartes từ mảng $A$. Cây Descartes của mảng $A$ là một cây nhị phân có tính chất min-heap (giá trị của nút cha phải nhỏ hơn hoặc bằng giá trị của các nút con) sao cho việc duyệt theo thứ tự giữa (in-order traversal) của cây sẽ thăm các nút theo cùng thứ tự như trong mảng $A$.

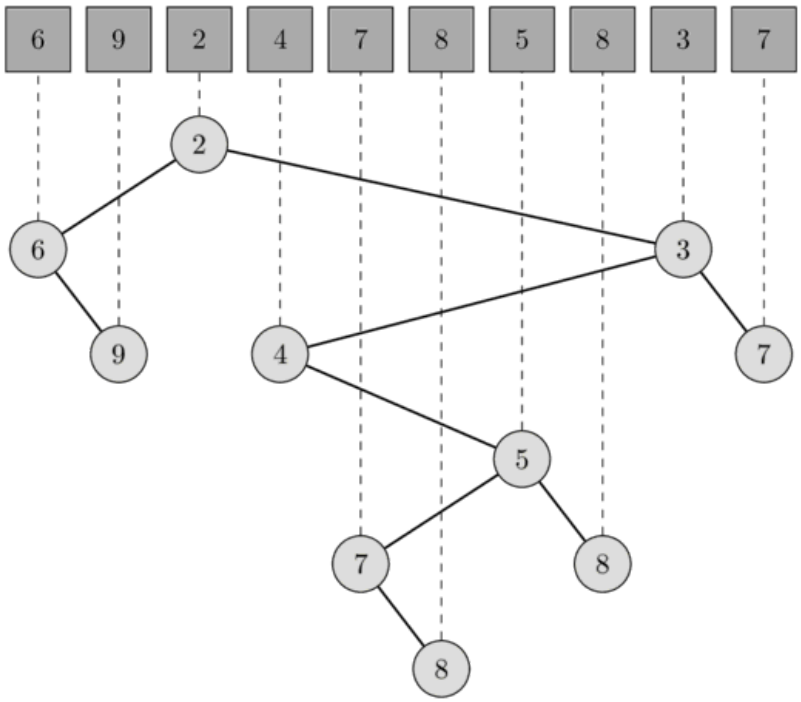
Nói cách khác, cây Descartes là một cấu trúc dữ liệu đệ quy. Mảng $A$ sẽ được chia thành 3 phần: tiền tố của mảng cho đến giá trị nhỏ nhất, giá trị nhỏ nhất, và hậu tố còn lại. Gốc của cây sẽ là một nút tương ứng với phần tử nhỏ nhất của mảng $A$, cây con bên trái sẽ là cây Descartes của tiền tố, và cây con bên phải sẽ là cây Descartes của hậu tố.

Trong hình sau, bạn có thể thấy một mảng có độ dài 10 và cây Descartes tương ứng.

Truy vấn giá trị nhỏ nhất trong đoạn $[l, r]$ tương đương với truy vấn tổ tiên chung thấp nhất $[l', r']$, trong đó $l'$ là nút tương ứng với phần tử $A[l]$ và $r'$ là nút tương ứng với phần tử $A[r]$. Thực tế, nút tương ứng với phần tử nhỏ nhất trong đoạn phải là tổ tiên của tất cả các nút trong đoạn, do đó cũng là tổ tiên của $l'$ và $r'$. Điều này tự động suy ra từ tính chất min-heap. Và nó cũng phải là tổ tiên thấp nhất, bởi nếu không, $l'$ và $r'$ sẽ cùng nằm trong cây con bên trái hoặc bên phải, dẫn đến mâu thuẫn vì trong trường hợp đó giá trị nhỏ nhất thậm chí sẽ không nằm trong đoạn.

Trong hình sau, bạn có thể thấy các truy vấn LCA cho các truy vấn RMQ $[1, 3]$ và $[5, 9]$. Trong truy vấn đầu tiên, LCA của các nút $A[1]$ và $A[3]$ là nút tương ứng với $A[2]$ có giá trị 2, và trong truy vấn thứ hai, LCA của $A[5]$ và $A[9]$ là nút tương ứng với $A[8]$ có giá trị 3.

Một cây như vậy có thể được xây dựng trong thời gian $O(N)$ và thuật toán Farach-Colton và Bender có thể tiền xử lý cây trong $O(N)$ và tìm LCA trong $O(1)$.



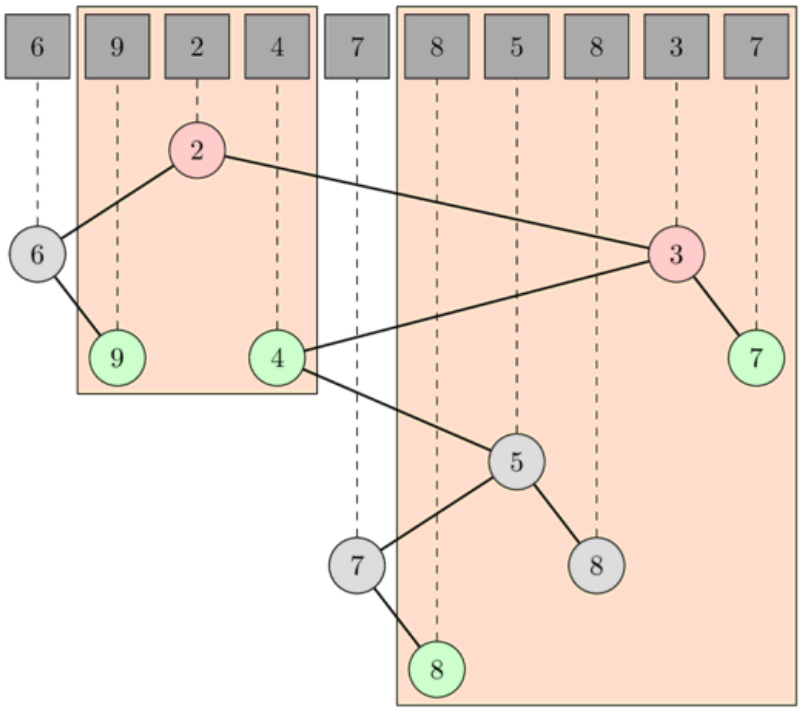

Hình 3.9: Hình ảnh của cây Descartes

### 3.28.2 Xây dựng cây Descartes

Chúng ta sẽ xây dựng cây Descartes bằng cách thêm các phần tử từng cái một. Ở mỗi bước, chúng ta duy trì một cây Descartes hợp lệ của tất cả các phần tử đã xử lý. Dễ thấy rằng việc thêm một phần tử $s[i]$ chỉ có thể thay đổi các nút trên đường đi bên phải nhất - bắt đầu từ gốc và liên tục lấy nút con bên phải - của cây. Cây con của nút có giá trị nhỏ nhất nhưng lớn hơn hoặc bằng $s[i]$ sẽ trở thành cây con bên trái của $s[i]$, và cây với gốc $s[i]$ sẽ trở thành cây con bên phải mới của nút có giá trị lớn nhất nhưng nhỏ hơn $s[i]$.

Điều này có thể được triển khai bằng cách sử dụng một ngăn xếp để lưu trữ các chỉ số của các nút bên phải nhất.

```
vector<int> parent(n, -1);
stack<int> s;
for (int i = 0; i < n; i++) {
    int last = -1;
    while (!s.empty() && A[s.top()] >= A[i]) {
        last = s.top();
        s.pop();
    }
```




Hình 3.10: Truy vấn LCA trong cây Descartes

```
    if (!s.empty())
        parent[i] = s.top();
    if (last >= 0)
        parent[last] = i;
    s.push(i);
}
```

## 3.29 Tổ tiên chung thấp nhất - Thuật toán ngoại tuyến của Tarjan

Chúng ta có một cây $G$ với $n$ nút và có $m$ truy vấn dạng $(u, v)$. Với mỗi truy vấn $(u, v)$, chúng ta muốn tìm tổ tiên chung thấp nhất của các đỉnh $u$ và $v$, tức là nút vừa là tổ tiên của cả $u$ và $v$ và có độ sâu lớn nhất trong cây. Nút $v$ cũng là tổ tiên của chính $v$, vì vậy LCA cũng có thể là một trong hai nút này.

Trong bài viết này, chúng ta sẽ giải bài toán theo cách ngoại tuyến, tức là giả định rằng tất cả các truy vấn đã được biết trước, và do đó chúng ta có thể trả lời các truy vấn theo bất kỳ thứ tự nào chúng ta muốn. Thuật toán sau cho phép trả lời tất cả $m$ truy

vấn trong tổng thời gian $O(n+m)$, tức là với $m$ đủ lớn, tương đương $O(1)$ cho mỗi truy vấn.

### 3.29.1 Thuật toán

Thuật toán được đặt theo tên của Robert Tarjan, người đã phát minh ra nó vào năm 1979 và cũng có nhiều đóng góp khác cho cấu trúc dữ liệu Disjoint Set Union (DSU), cấu trúc này sẽ được sử dụng nhiều trong thuật toán.

Thuật toán trả lời tất cả các truy vấn bằng một lần duyệt DFS trên cây. Cụ thể, một truy vấn $(u, v)$ được trả lời tại nút $u$, nếu nút $v$ đã được thăm trước đó, hoặc ngược lại.

Giả sử chúng ta hiện đang ở nút $v$, chúng ta đã thực hiện các lời gọi DFS đệ quy, và cũng đã thăm nút thứ hai $u$ từ truy vấn $(u, v)$. Hãy tìm hiểu cách tìm LCA của hai nút này.

Lưu ý rằng LCA$(u, v)$ hoặc là nút $v$ hoặc là một trong các tổ tiên của nó. Vì vậy, chúng ta cần tìm nút thấp nhất trong số các tổ tiên của $v$ (bao gồm cả $v$), mà tại đó nút $u$ là một hậu duệ. Cũng lưu ý rằng với một $v$ cố định, các nút đã thăm của cây được chia thành một tập hợp các tập hợp không giao nhau. Mỗi tổ tiên $p$ của nút $v$ có tập hợp riêng chứa nút này và tất cả các cây con có gốc là các con của nó không nằm trên đường đi từ $v$ đến gốc của cây. Tập hợp chứa nút $u$ sẽ xác định LCA$(u, v)$: LCA là đại diện của tập hợp, cụ thể là nút nằm trên đường đi giữa $v$ và gốc của cây.

Chúng ta chỉ cần học cách duy trì hiệu quả tất cả các tập hợp này. Để làm điều này, chúng ta áp dụng cấu trúc dữ liệu DSU. Để có thể áp dụng Union by rank, chúng ta lưu trữ đại diện thực sự (giá trị trên đường đi giữa $v$ và gốc của cây) của mỗi tập hợp trong mảng ancestor.

Hãy thảo luận về việc triển khai DFS. Giả sử chúng ta hiện đang thăm nút $v$. Chúng ta đặt nút này vào một tập hợp mới trong DSU, tức là ancestor$[v] = v$. Như thường lệ, chúng ta xử lý tất cả các con của $v$. Để làm điều này, trước tiên chúng ta phải gọi DFS đệ quy từ nút con đó, sau đó thêm nút này cùng toàn bộ cây con của nó vào tập hợp của $v$. Điều này có thể được thực hiện với hàm union_sets và phép gán sau:

$$\text{ancestor}[\text{find_set}(v)] = v$$

Điều này là cần thiết, vì union_sets có thể thay đổi đại diện của tập hợp.

Cuối cùng, sau khi xử lý tất cả các con, chúng ta có thể trả lời tất cả các truy vấn dạng $(u, v)$ mà tại đó $u$ đã được thăm. Đáp án cho truy vấn, tức là LCA của $u$ và $v$, sẽ là nút

$$\text{ancestor}[\text{find_set}(u)]$$

Dễ thấy rằng một truy vấn sẽ chỉ được trả lời một lần.

Hãy xác định độ phức tạp thời gian của thuật toán này. Thứ nhất, chúng ta có $O(n)$ do DFS. Thứ hai, chúng ta có các lời gọi hàm union_sets xảy ra $n$ lần, cũng dẫn đến

$O(n)$. Và thứ ba, chúng ta có các lời gọi find_set cho mỗi truy vấn, cho ra $O(m)$. Vì vậy, tổng độ phức tạp thời gian là $O(n+m)$, điều này có nghĩa là với $m$ đủ lớn, ta đạt được $O(1)$ cho mỗi truy vấn.

### 3.29.2 Triển khai

Dưới đây là triển khai của thuật toán này. Triển khai của DSU không được bao gồm, vì nó có thể được sử dụng mà không cần bất kỳ sửa đổi nào.

```
vector<vector<int>> adj;
vector<vector<int>> queries;
vector<int> ancestor;
vector<bool> visited;

void dfs(int v)
{
    visited[v] = true;
    ancestor[v] = v;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
            union_sets(v, u);
            ancestor[find_set(v)] = v;
        }
    }
    for (int other_node : queries[v]) {
        if (visited[other_node])
            cout << "LCA của " << v << " và " << other_node
                 << " là " << ancestor[find_set(other_node)] << ".\n";
    }
}

void compute_LCAs() {
    // khởi tạo n, adj và DSU
    // for (mỗi truy vấn (u, v)) {
    //    queries[u].push_back(v);
    //    queries[v].push_back(u);
    // }

    ancestor.resize(n);
    visited.assign(n, false);
```

```
    dfs(0);
}
```

## 3.30 Luồng cực đại - Ford-Fulkerson và Edmonds-Karp

Thuật toán Edmonds-Karp là một triển khai của phương pháp Ford-Fulkerson để tính luồng cực đại trong một mạng luồng.

### 3.30.1 Mạng luồng

Trước tiên, hãy định nghĩa mạng luồng, luồng và luồng cực đại là gì.

Một mạng là một đồ thị có hướng $G$ với các đỉnh $V$ và các cạnh $E$, kết hợp với một hàm $c$, gán cho mỗi cạnh $e \in E$ một giá trị nguyên không âm, gọi là dung lượng của $e$. Mạng như vậy được gọi là mạng luồng nếu chúng ta thêm nhãn cho hai đỉnh, một là nguồn (source) và một là đích (sink).

Luồng trong một mạng luồng là một hàm $f$, cũng gán cho mỗi cạnh $e$ một giá trị nguyên không âm, tức là luồng. Hàm này phải thỏa mãn hai điều kiện sau:

Luồng của một cạnh không được vượt quá dung lượng.

$$f(e) \leq c(e)$$

Và tổng luồng vào của một đỉnh $u$ phải bằng tổng luồng ra của $u$, trừ trường hợp ở đỉnh nguồn và đỉnh đích.

$$\sum_{(v,u)\in E} f((v,u)) = \sum_{(u,v)\in E} f((u,v))$$

Đỉnh nguồn $s$ chỉ có luồng ra, và đỉnh đích $t$ chỉ có luồng vào.

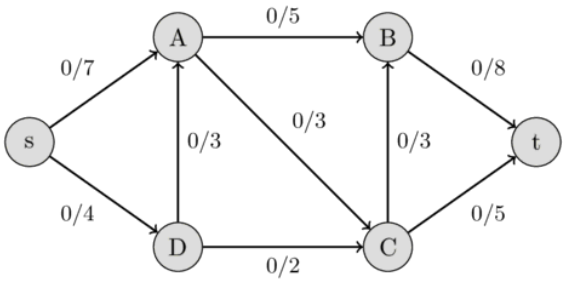
Dễ thấy rằng phương trình sau được thỏa mãn:

$$\sum_{(s,u)\in E} f((s,u)) = \sum_{(u,t)\in E} f((u,t))$$

Một cách hình dung tốt cho mạng luồng là hình dung sau: Chúng ta biểu diễn các cạnh như các ống nước, dung lượng của một cạnh là lượng nước tối đa có thể chảy qua ống mỗi giây, và luồng của một cạnh là lượng nước hiện đang chảy qua ống mỗi giây. Điều này giải thích điều kiện luồng đầu tiên: Không thể có nhiều nước chảy qua ống hơn dung lượng của nó. Các đỉnh hoạt động như các điểm nối, nơi nước chảy ra từ một số ống, sau đó các đỉnh này phân phối nước theo một cách nào đó sang các ống khác. Điều này cũng giải thích điều kiện luồng thứ hai: Tất cả nước vào phải được phân phối sang

các ống khác tại mỗi điểm nối, không thể tự nhiên biến mất hoặc xuất hiện. Nguồn $s$ là điểm xuất phát của tất cả nước, và nước chỉ có thể thoát ra ở đích $t$.

Hình sau cho thấy một mạng luồng. Giá trị đầu tiên của mỗi cạnh thể hiện luồng, ban đầu là 0, và giá trị thứ hai thể hiện dung lượng.

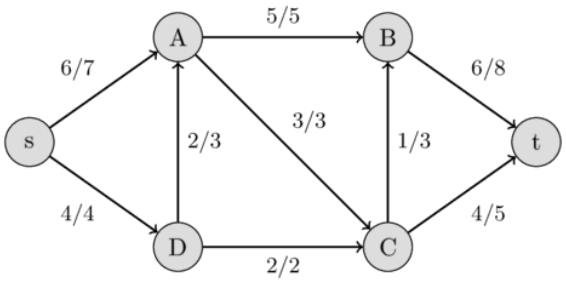



Hình 3.11: Mạng luồng

Giá trị của luồng của một mạng là tổng tất cả các luồng được tạo ra tại nguồn $s$, hoặc tương đương với tổng tất cả các luồng được tiêu thụ bởi đích $t$. Luồng cực đại là luồng có giá trị lớn nhất có thể. Tìm luồng cực đại của một mạng luồng là bài toán mà chúng ta muốn giải.

Trong hình dung với các ống nước, bài toán có thể được diễn đạt như sau: Chúng ta có thể đẩy bao nhiêu nước qua các ống từ nguồn đến đích?

Hình sau cho thấy luồng cực đại trong mạng luồng.




Hình 3.12: Luồng cực đại

### 3.30.2 Phương pháp Ford-Fulkerson

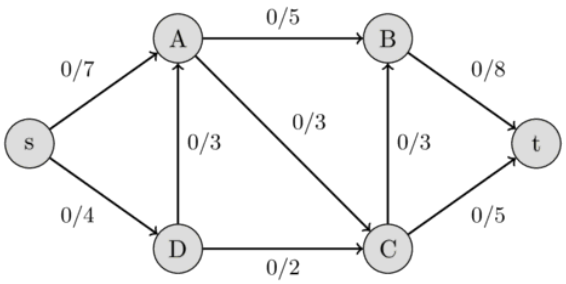
Hãy định nghĩa thêm một khái niệm nữa. Dung lượng còn lại của một cạnh có hướng là dung lượng trừ đi luồng. Cần lưu ý rằng nếu có luồng dọc theo một cạnh có hướng

$(u, v)$, thì cạnh ngược lại có dung lượng 0 và chúng ta có thể định nghĩa luồng của nó là $f((v, u)) = -f((u, v))$. Điều này cũng định nghĩa dung lượng còn lại cho tất cả các cạnh ngược. Chúng ta có thể tạo một mạng còn lại từ tất cả các cạnh này, chỉ là một mạng với cùng các đỉnh và cạnh, nhưng chúng ta sử dụng dung lượng còn lại làm dung lượng.

Phương pháp Ford-Fulkerson hoạt động như sau. Đầu tiên, chúng ta đặt luồng của mỗi cạnh bằng 0. Sau đó, chúng ta tìm một đường tăng luồng từ $s$ đến $t$. Đường tăng luồng là một đường đơn trong đồ thị còn lại mà dung lượng còn lại là dương cho tất cả các cạnh dọc theo đường đó. Nếu tìm thấy một đường như vậy, thì chúng ta có thể tăng luồng dọc theo các cạnh này. Chúng ta tiếp tục tìm các đường tăng luồng và tăng luồng. Khi không còn đường tăng luồng nào nữa, luồng đạt cực đại.

Hãy chỉ rõ chi tiết hơn về việc tăng luồng dọc theo một đường tăng luồng nghĩa là gì. Gọi $C$ là dung lượng còn lại nhỏ nhất của các cạnh trong đường đi. Sau đó, chúng ta tăng luồng theo cách sau: chúng ta cập nhật $f((u, v))$ += $C$ và $f((v, u))$ -= $C$ cho mọi cạnh $(u, v)$ trong đường đi.

Dưới đây là một ví dụ để minh họa phương pháp này. Chúng ta sử dụng cùng mạng luồng như trên. Ban đầu, chúng ta bắt đầu với luồng bằng 0.



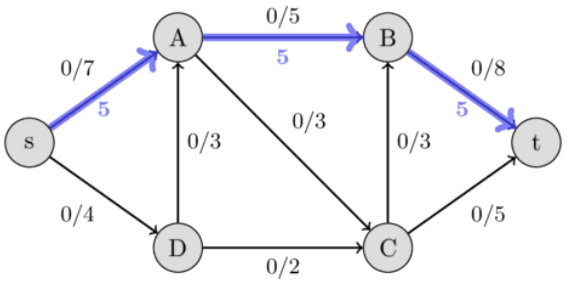

Hình 3.13: Mạng luồng

Chúng ta có thể tìm đường $s - A - B - t$ với dung lượng còn lại là 7, 5 và 8. Giá trị nhỏ nhất là 5, do đó chúng ta có thể tăng luồng dọc theo đường này lên 5. Điều này cho luồng của mạng là 5.

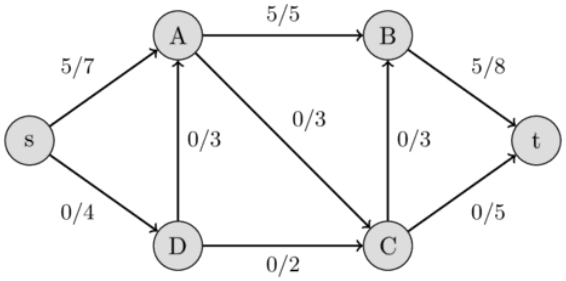
Lại tìm một đường tăng luồng, lần này chúng ta tìm thấy $s - D - A - C - t$ với dung lượng còn lại là 4, 3, 3 và 5. Do đó, chúng ta có thể tăng luồng thêm 3 và thu được luồng của mạng là 8.

Lần này, chúng ta tìm đường $s - D - C - B - t$ với dung lượng còn lại là 1, 2, 3 và 3, và do đó, chúng ta tăng luồng thêm 1.

Lần này, chúng ta tìm đường tăng luồng $s - A - D - C - t$ với dung lượng còn lại là 2, 3, 1 và 2. Chúng ta có thể tăng luồng thêm 1. Nhưng đường này rất thú vị. Nó bao gồm cạnh ngược $(A, D)$. Trong mạng luồng ban đầu, chúng ta không được phép gửi bất kỳ luồng nào từ $A$ đến $D$. Nhưng vì chúng ta đã có luồng 3 từ $D$ đến $A$, điều này là khả




Hình 3.14: Đường đầu tiên

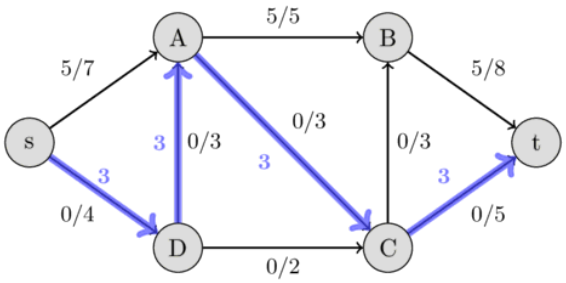



Hình 3.15: Mạng sau đường đầu tiên

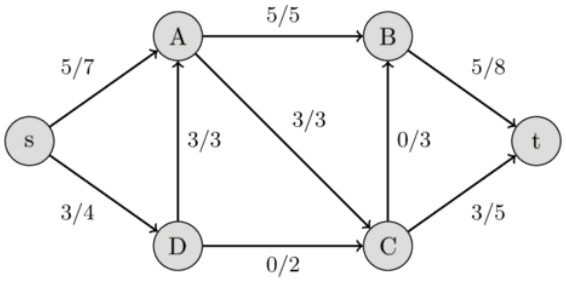
thi. Trực giác của nó như sau: Thay vì gửi luồng 3 từ $D$ đến $A$, chúng ta chỉ gửi 2 và bù đắp điều này bằng cách gửi thêm luồng 1 từ $s$ đến $A$, điều này cho phép chúng ta gửi thêm luồng 1 dọc theo đường $D - C - t$.

Bây giờ, không thể tìm thấy đường tăng luồng nào giữa $s$ và $t$, do đó luồng 10 này là cực đại có thể. Chúng ta đã tìm thấy luồng cực đại.

Cần lưu ý rằng phương pháp Ford-Fulkerson không chỉ định phương pháp tìm đường tăng luồng. Các cách tiếp cận có thể là sử dụng DFS hoặc BFS, cả hai đều hoạt động trong $O(E)$. Nếu tất cả dung lượng của mạng là số nguyên, thì với mỗi đường tăng luồng, luồng của mạng tăng ít nhất 1 (xem thêm định lý luồng nguyên). Do đó, độ phức tạp của Ford-Fulkerson là $O(EF)$, trong đó $F$ là luồng cực đại của mạng. Trong trường hợp dung lượng là số hữu tỉ, thuật toán cũng sẽ kết thúc, nhưng độ phức tạp không bị giới hạn. Trong trường hợp dung lượng là số vô tỉ, thuật toán có thể không bao giờ kết thúc, và thậm chí không hội tụ đến luồng cực đại.




Hình 3.16: Đường thứ hai




Hình 3.17: Mạng sau đường thứ hai

### 3.30.3 Thuật toán Edmonds-Karp

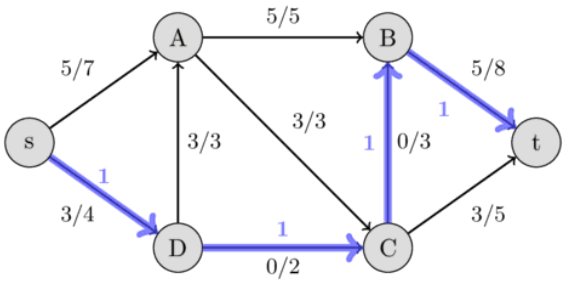
Thuật toán Edmonds-Karp chỉ là một triển khai của phương pháp Ford-Fulkerson sử dụng BFS để tìm đường tăng luồng. Thuật toán được công bố lần đầu tiên bởi Yefim Dinitz vào năm 1970, và sau đó được công bố độc lập bởi Jack Edmonds và Richard Karp vào năm 1972.

Độ phức tạp có thể được đưa ra độc lập với luồng cực đại. Thuật toán chạy trong thời gian $O(VE^2)$, ngay cả với dung lượng vô tỉ. Trực giác là, mỗi lần chúng ta tìm một đường tăng luồng, một trong các cạnh trở nên bão hòa, và khoảng cách từ cạnh đó đến $s$ sẽ dài hơn nếu nó xuất hiện lại sau này trong một đường tăng luồng. Độ dài của các đường đơn bị giới hạn bởi $V$.

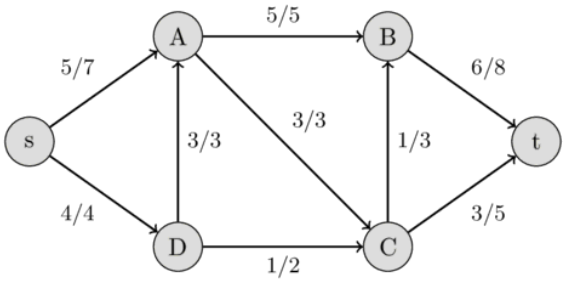
### 3.30.4 Triển khai

Ma trận capacity lưu trữ dung lượng cho mọi cặp đỉnh. adj là danh sách kề của đồ thị vô hướng, vì chúng ta cũng phải sử dụng các cạnh ngược của các cạnh có hướng khi tìm đường tăng luồng.

Hàm maxflow sẽ trả về giá trị của luồng cực đại. Trong quá trình thuật toán, ma trận




Hình 3.18: Đường thứ ba



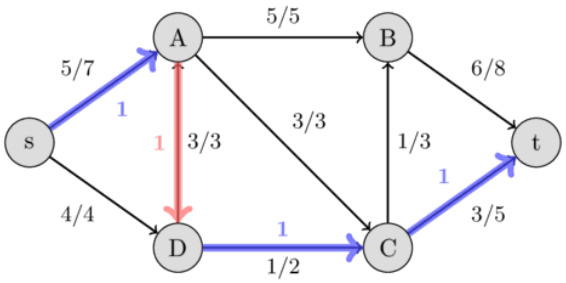

Hình 3.19: Mạng sau đường thứ ba

capacity thực sự sẽ lưu trữ dung lượng còn lại của mạng. Giá trị của luồng trong mỗi cạnh thực tế sẽ không được lưu trữ, nhưng dễ dàng mở rộng triển khai - bằng cách sử dụng một ma trận bổ sung - để cũng lưu trữ luồng và trả về nó.

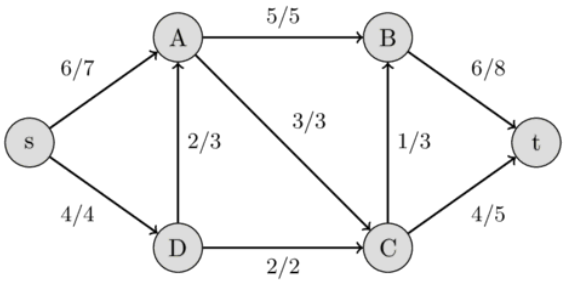
```
int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();
```




Hình 3.20: Đường thứ tư




Hình 3.21: Mạng sau đường thứ tư

```
        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }
    }

    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
```

```
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }

    return flow;
}
```

### 3.30.5 Định lý luồng nguyên

Định lý nói rằng, nếu mọi dung lượng trong mạng là số nguyên, thì kích thước của luồng cực đại là một số nguyên, và tồn tại một luồng cực đại sao cho luồng trong mỗi cạnh cũng là một số nguyên. Đặc biệt, phương pháp Ford-Fulkerson tìm được luồng như vậy.

### 3.30.6 Định lý luồng cực đại - cắt nhỏ nhất

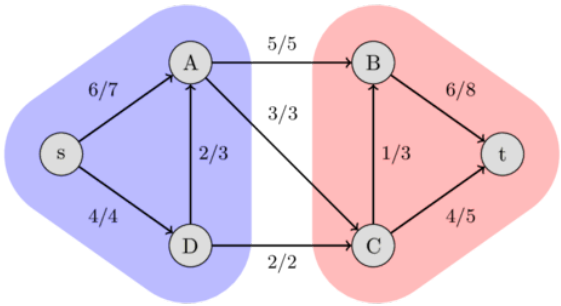
Một cắt $s$- $t$ là một phân vùng của các đỉnh của một mạng luồng thành hai tập, sao cho một tập bao gồm nguồn $s$ và tập kia bao gồm đích $t$. Dung lượng của một cắt $s$- $t$ được định nghĩa là tổng dung lượng của các cạnh từ phía nguồn sang phía đích.

Rõ ràng, chúng ta không thể gửi nhiều luồng hơn từ $s$ đến $t$ vượt quá dung lượng của bất kỳ cắt $s$- $t$ nào. Do đó, luồng cực đại bị giới hạn bởi dung lượng cắt nhỏ nhất.

Định lý luồng cực đại - cắt nhỏ nhất đi xa hơn. Nó nói rằng dung lượng của luồng cực đại phải bằng dung lượng của cắt nhỏ nhất.

Trong hình sau, bạn có thể thấy cắt nhỏ nhất của mạng luồng chúng ta đã sử dụng trước đó. Nó cho thấy dung lượng của cắt $\{s, A, D\}$ và $\{B, C, t\}$ là $5 + 3 + 2 = 10$, bằng với luồng cực đại mà chúng ta đã tìm thấy. Các cắt khác sẽ có dung lượng lớn hơn, như dung lượng giữa $\{s, A\}$ và $\{B, C, D, t\}$ là $4 + 3 + 5 = 12$.

Một cắt nhỏ nhất có thể được tìm thấy sau khi thực hiện tính toán luồng cực đại bằng phương pháp Ford-Fulkerson. Một cắt nhỏ nhất có thể là như sau: tập hợp tất cả các đỉnh có thể đến được từ $s$ trong đồ thị còn lại (sử dụng các cạnh có dung lượng còn lại dương), và tập hợp tất cả các đỉnh còn lại. Phân vùng này có thể dễ dàng được tìm thấy bằng cách sử dụng DFS bắt đầu từ $s$.




Hình 3.22: Cắt nhỏ nhất

## 3.31 Luồng cực đại - Thuật toán Push-relabel

Thuật toán push-relabel (còn được gọi là thuật toán preflow-push) là một thuật toán để tính luồng cực đại của một mạng luồng. Định nghĩa chính xác của bài toán mà chúng ta muốn giải có thể được tìm thấy trong bài viết "Luồng cực đại - Ford-Fulkerson và Edmonds-Karp".

Trong bài viết này, chúng ta sẽ xem xét việc giải bài toán bằng cách đẩy một preflow qua mạng, thuật toán này chạy trong thời gian $O(V^4)$, hoặc chính xác hơn là $O(V^2E)$. Thuật toán được thiết kế bởi Andrew Goldberg và Robert Tarjan vào năm 1985.

### 3.31.1 Định nghĩa

Trong quá trình thực hiện thuật toán, chúng ta sẽ phải xử lý một preflow - tức là một hàm $f$ tương tự như hàm luồng, nhưng không nhất thiết phải thỏa mãn ràng buộc bảo toàn luồng. Đối với nó, chỉ cần các ràng buộc sau được thỏa mãn:

$$0 \leq f(e) \leq c(e)$$

và

$$\sum_{(v,u)\in E} f((v,u)) \geq \sum_{(u,v)\in E} f((u,v))$$

Vì vậy, có thể một số đỉnh nhận được nhiều luồng hơn lượng nó phân phối. Chúng ta nói rằng đỉnh này có luồng dư thừa, và định nghĩa lượng dư thừa này bằng hàm dư thừa $x(u) = \sum_{(v,u)\in E} f((v,u)) - \sum_{(u,v)\in E} f((u,v))$.

Cũng giống như với hàm luồng, chúng ta có thể định nghĩa dung lượng còn lại và đồ thị còn lại với hàm preflow.

Thuật toán sẽ bắt đầu với một preflow ban đầu (một số đỉnh có luồng dư thừa), và trong quá trình thực thi, preflow sẽ được xử lý và sửa đổi. Tiết lộ trước một chút chi tiết,

thuật toán sẽ chọn một đỉnh có luồng dư thừa và đẩy luồng dư thừa sang các đỉnh lân cận. Nó sẽ lặp lại điều này cho đến khi tất cả các đỉnh, trừ nguồn và đích, không còn luồng dư thừa. Dễ thấy rằng một preflow không có luồng dư thừa là một luồng hợp lệ. Điều này khiến thuật toán kết thúc với một luồng thực tế.

Vẫn còn hai vấn đề mà chúng ta phải giải quyết. Thứ nhất, làm thế nào để đảm bảo rằng thuật toán này thực sự kết thúc? Và thứ hai, làm thế nào để đảm bảo rằng nó sẽ thực sự cho chúng ta một luồng cực đại, chứ không chỉ là một luồng ngẫu nhiên?

Để giải quyết các vấn đề này, chúng ta cần sự trợ giúp của một hàm khác, cụ thể là hàm gắn nhãn $h$, thường còn được gọi là hàm chiều cao, gán cho mỗi đỉnh một số nguyên. Chúng ta gọi một cách gắn nhãn là hợp lệ nếu $h(s) = |V|$, $h(t) = 0$, và $h(u) \leq h(v) + 1$ nếu có một cạnh $(u, v)$ trong đồ thị còn lại - tức là cạnh $(u, v)$ có dung lượng dương trong đồ thị còn lại. Nói cách khác, nếu có thể tăng luồng từ $u$ đến $v$, thì chiều cao của $v$ có thể nhỏ hơn chiều cao của $u$ tối đa một đơn vị, nhưng nó cũng có thể bằng hoặc thậm chí cao hơn.

Điều quan trọng cần lưu ý là nếu tồn tại một hàm gắn nhãn hợp lệ, thì không tồn tại một đường tăng luồng từ $s$ đến $t$ trong đồ thị còn lại. Bởi vì một đường như vậy sẽ có độ dài tối đa là $|V| - 1$ cạnh, và mỗi cạnh chỉ có thể giảm chiều cao tối đa một đơn vị, điều này là không thể nếu chiều cao đầu tiên là $h(s) = |V|$ và chiều cao cuối cùng là $h(t) = 0$.

Sử dụng hàm gắn nhãn này, chúng ta có thể nêu chiến lược của thuật toán push-relabel: Chúng ta bắt đầu với một preflow hợp lệ và một hàm gắn nhãn hợp lệ. Ở mỗi bước, chúng ta đẩy một phần luồng dư thừa giữa các đỉnh và cập nhật nhãn của các đỉnh. Chúng ta phải đảm bảo rằng sau mỗi bước, preflow và cách gắn nhãn vẫn hợp lệ. Nếu sau đó thuật toán xác định preflow là một luồng hợp lệ, và vì chúng ta cũng có một cách gắn nhãn hợp lệ, không tồn tại đường đi giữa $s$ và $t$ trong đồ thị còn lại, điều này có nghĩa là luồng thực sự là luồng cực đại.

Nếu so sánh thuật toán Ford-Fulkerson với thuật toán push-relabel, có vẻ như các thuật toán này là đối ngẫu của nhau. Thuật toán Ford-Fulkerson luôn giữ một luồng hợp lệ và cải thiện nó cho đến khi không còn đường tăng luồng nào nữa, trong khi trong thuật toán push-relabel, không tồn tại đường tăng luồng nào tại bất kỳ thời điểm nào, và chúng ta sẽ cải thiện preflow cho đến khi nó là một luồng hợp lệ.

### 3.31.2 Thuật toán

Đầu tiên, chúng ta phải khởi tạo đồ thị với một preflow hợp lệ và hàm gắn nhãn.

Sử dụng preflow rỗng - như được thực hiện trong thuật toán Ford-Fulkerson - là không thể, bởi vì khi đó sẽ tồn tại một đường tăng luồng và điều này ngụ ý rằng không tồn tại một cách gắn nhãn hợp lệ. Do đó, chúng ta sẽ khởi tạo mỗi cạnh xuất phát từ $s$ với dung lượng tối đa của nó: $f((s, u)) = c((s, u))$. Và tất cả các cạnh khác bằng 0. Trong trường hợp này, tồn tại một cách gắn nhãn hợp lệ, cụ thể là $h(s) = |V|$ cho đỉnh nguồn và $h(u) = 0$ cho tất cả các đỉnh khác.

Bây giờ hãy mô tả chi tiết hơn về hai thao tác.

Với thao tác đẩy (push), chúng ta cố gắng đẩy càng nhiều luồng dư thừa từ một đỉnh $u$ sang một đỉnh lân cận $v$. Chúng ta có một quy tắc: chúng ta chỉ được phép đẩy luồng từ $u$ đến $v$ nếu $h(u) = h(v) + 1$. Nói một cách đơn giản, luồng dư thừa phải chảy xuống dưới, nhưng không quá dốc. Tất nhiên, chúng ta chỉ có thể đẩy luồng $\min(x(u), c((u, v)) - f((u, v)))$.

Nếu một đỉnh có luồng dư thừa, nhưng không thể đẩy luồng dư thừa sang bất kỳ đỉnh lân cận nào, thì chúng ta cần tăng chiều cao của đỉnh này. Chúng ta gọi thao tác này là gắn nhãn lại (relabel). Chúng ta sẽ tăng nó lên nhiều nhất có thể, trong khi vẫn duy trì tính hợp lệ của cách gắn nhãn.

Tóm lại, thuật toán một cách ngắn gọn là: Chúng ta khởi tạo một preflow hợp lệ và một cách gắn nhãn hợp lệ. Trong khi chúng ta có thể thực hiện các thao tác đẩy hoặc gắn nhãn lại, chúng ta thực hiện chúng. Sau đó, preflow thực sự là một luồng và chúng ta trả về nó.

### 3.31.3 Độ phức tạp

Dễ dàng chứng minh rằng nhãn tối đa của một đỉnh là $2|V| - 1$. Tại thời điểm này, tất cả luồng dư thừa còn lại có thể và sẽ được đẩy ngược về nguồn. Điều này cho tối đa $O(V^2)$ thao tác gắn nhãn lại.

Cũng có thể chứng minh rằng sẽ có tối đa $O(VE)$ lần đẩy bão hòa (một lần đẩy sử dụng toàn bộ dung lượng của cạnh) và tối đa $O(V^2E)$ lần đẩy không bão hòa (một lần đẩy không sử dụng hết dung lượng của cạnh) được thực hiện. Nếu chúng ta chọn một cấu trúc dữ liệu cho phép tìm đỉnh tiếp theo có luồng dư thừa trong thời gian $O(1)$, thì tổng độ phức tạp của thuật toán là $O(V^2E)$.

### 3.31.4 Triển khai

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess, seen;
queue<int> excess_vertices;

void push(int u, int v) {
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
```

```
    if (d && excess[v] == d)
        excess_vertices.push(v);
}

void relabel(int u) {
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

void discharge(int u) {
    while (excess[u] > 0) {
        if (seen[u] < n) {
            int v = seen[u];
            if (capacity[u][v] - flow[u][v] > 0 && height[u] > height[v])
                push(u, v);
            else
                seen[u]++;
        } else {
            relabel(u);
            seen[u] = 0;
        }
    }
}

int max_flow(int s, int t) {
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }
    seen.assign(n, 0);
```

```
    while (!excess_vertices.empty()) {
        int u = excess_vertices.front();
        excess_vertices.pop();
        if (u != s && u != t)
            discharge(u);
    }

    int max_flow = 0;
    for (int i = 0; i < n; i++)
        max_flow += flow[i][t];
    return max_flow;
}
```

Ở đây, chúng ta sử dụng hàng đợi excess_vertices để lưu trữ tất cả các đỉnh hiện có luồng dư thừa. Bằng cách đó, chúng ta có thể chọn đỉnh tiếp theo cho một thao tác đẩy hoặc gắn nhãn lại trong thời gian hằng số.

Và để đảm bảo rằng chúng ta không mất quá nhiều thời gian để tìm đỉnh lân cận mà chúng ta có thể đẩy luồng tới, chúng ta sử dụng một cấu trúc dữ liệu gọi là current-arc. Về cơ bản, chúng ta sẽ lặp qua các cạnh theo thứ tự vòng tròn và luôn lưu trữ cạnh cuối cùng mà chúng ta đã sử dụng. Theo cách này, với một giá trị gắn nhãn nhất định, chúng ta sẽ chỉ chuyển đổi cạnh hiện tại $O(n)$ lần. Và vì việc gắn nhãn lại đã mất $O(n)$ thời gian, chúng ta không làm tăng độ phức tạp lên.

## 3.32 Luồng cực đại - Phương pháp Push-relabel cải tiến

Chúng ta sẽ sửa đổi phương pháp push-relabel để đạt được thời gian chạy tốt hơn.

### 3.32.1 Mô tả

Sự sửa đổi cực kỳ đơn giản: Trong bài viết trước, chúng ta đã chọn một đỉnh có luồng dư thừa mà không theo bất kỳ quy tắc cụ thể nào. Nhưng hóa ra, nếu chúng ta luôn chọn các đỉnh có chiều cao lớn nhất và áp dụng các thao tác đẩy (push) và gắn nhãn lại (relabel) lên chúng, thì độ phức tạp sẽ được cải thiện. Hơn nữa, để chọn các đỉnh có chiều cao lớn nhất, chúng ta thực sự không cần bất kỳ cấu trúc dữ liệu nào phức tạp, chỉ cần lưu trữ các đỉnh có chiều cao lớn nhất trong một danh sách, và tính toán lại danh sách khi tất cả chúng đã được xử lý (khi đó các đỉnh với chiều cao thấp hơn sẽ được thêm vào danh sách), hoặc bất cứ khi nào một đỉnh mới có luồng dư thừa và chiều cao lớn hơn xuất hiện (sau khi gắn nhãn lại một đỉnh).

Mặc dù đơn giản, nhưng sự sửa đổi này giảm độ phức tạp đi rất nhiều. Cụ thể, độ phức tạp của thuật toán kết quả là $O(VE+V^2\sqrt{E})$, trong trường hợp xấu nhất là $O(V^3)$.

Sự sửa đổi này được đề xuất bởi Cheriyan và Maheshwari vào năm 1989.

### 3.32.2 Triển khai

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess;

void push(int u, int v)
{
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
}

void relabel(int u)
{
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

vector<int> find_max_height_vertices(int s, int t) {
    vector<int> max_height;
    for (int i = 0; i < n; i++) {
        if (i != s && i != t && excess[i] > 0) {
            if (!max_height.empty() && height[i] > height[max_height[0]])
                max_height.clear();
            if (max_height.empty() || height[i] == height[max_height[0]])
                max_height.push_back(i);
        }
```

```
    }
    return max_height;
}

int max_flow(int s, int t)
{
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }

    vector<int> current;
    while (!(current = find_max_height_vertices(s, t)).empty()) {
        for (int i : current) {
            bool pushed = false;
            for (int j = 0; j < n && excess[i]; j++) {
                if (capacity[i][j] - flow[i][j] > 0 && height[i] == height[j] + 1) {
                    push(i, j);
                    pushed = true;
                }
            }
            if (!pushed) {
                relabel(i);
                break;
            }
        }
    }

    return excess[t];
}
```

## 3.33 Luồng cực đại - Thuật toán Dinic

Thuật toán Dinic giải quyết bài toán luồng cực đại trong thời gian $O(V^2E)$. Bài toán luồng cực đại được định nghĩa trong bài viết "Luồng cực đại - Ford-Fulkerson và Edmonds-

Karp". Thuật toán này được phát hiện bởi Yefim Dinitz vào năm 1970.

### 3.33.1 Định nghĩa

Mạng còn lại $G^R$ của mạng $G$ là một mạng chứa hai cạnh cho mỗi cạnh $(v, u) \in G$:
- $(v, u)$ với dung lượng $c^R_{vu} = c_{vu} - f_{vu}$ - $(u, v)$ với dung lượng $c^R_{uv} = f_{vu}$

Luồng chặn (blocking flow) của một mạng là một luồng sao cho mọi đường đi từ $s$ đến $t$ chứa ít nhất một cạnh bị bão hòa bởi luồng này. Lưu ý rằng luồng chặn không nhất thiết phải là luồng cực đại.

Mạng phân tầng (layered network) của một mạng $G$ là một mạng được xây dựng theo cách sau. Đầu tiên, với mỗi đỉnh $v$, chúng ta tính $level[v]$ - đường đi ngắn nhất (không có trọng số) từ $s$ đến đỉnh này chỉ sử dụng các cạnh có dung lượng dương. Sau đó, chúng ta chỉ giữ lại những cạnh $(v, u)$ mà $level[v] + 1 = level[u]$. Rõ ràng, mạng này không có chu trình.

### 3.33.2 Thuật toán

Thuật toán bao gồm nhiều giai đoạn. Ở mỗi giai đoạn, chúng ta xây dựng mạng phân tầng của mạng còn lại của $G$. Sau đó, chúng ta tìm một luồng chặn bất kỳ trong mạng phân tầng và thêm nó vào luồng hiện tại.

### 3.33.3 Chứng minh tính đúng đắn

Hãy chứng minh rằng nếu thuật toán kết thúc, nó tìm được luồng cực đại.

Nếu thuật toán kết thúc, nó không thể tìm thấy luồng chặn trong mạng phân tầng. Điều này có nghĩa là mạng phân tầng không có đường đi nào từ $s$ đến $t$. Điều đó cũng có nghĩa là mạng còn lại không có đường đi nào từ $s$ đến $t$. Do đó, luồng là cực đại.

### 3.33.4 Số lượng giai đoạn

Thuật toán kết thúc trong vòng chưa đến $V$ giai đoạn. Để chứng minh điều này, chúng ta cần chứng minh hai bổ đề sau.

**Bổ đề 1.** Khoảng cách từ $s$ đến mỗi đỉnh không giảm sau mỗi lần lặp, tức là $level_{i+1}[v] \geq level_i[v]$.

**Chứng minh.** Cố định một giai đoạn $i$ và một đỉnh $v$. Xét một đường đi ngắn nhất $P$ từ $s$ đến $v$ trong $G^R_{i+1}$. Độ dài của $P$ bằng $level_{i+1}[v]$. Lưu ý rằng $G^R_{i+1}$ chỉ có thể chứa các cạnh từ $G^R_i$ và các cạnh ngược của các cạnh từ $G^R_i$. Nếu $P$ không có cạnh ngược nào của $G^R_i$, thì $level_{i+1}[v] \geq level_i[v]$ vì $P$ cũng là một đường đi trong $G^R_i$. Bây giờ, giả sử $P$ có ít nhất một cạnh ngược. Gọi cạnh ngược đầu tiên là $(u, w)$. Khi đó $level_{i+1}[u] \geq level_i[u]$ (do trường hợp đầu tiên). Cạnh $(u, w)$ không thuộc $G^R_i$, nên cạnh $(w, u)$ đã bị ảnh hưởng bởi luồng chặn ở lần lặp trước. Điều này có nghĩa là $level_i[u] = level_i[w] + 1$. Ngoài ra,

$level_{i+1}[w] = level_{i+1}[u] + 1$. Từ hai phương trình này và $level_{i+1}[u] \geq level_i[u]$, ta suy ra $level_{i+1}[w] \geq level_i[w] + 2$. Bây giờ, chúng ta có thể áp dụng ý tưởng tương tự cho phần còn lại của đường đi.

**Bổ đề 2.** $level_{i+1}[t] > level_i[t]$

**Chứng minh.** Từ bổ đề trước, $level_{i+1}[t] \geq level_i[t]$. Giả sử $level_{i+1}[t] = level_i[t]$. Lưu ý rằng $G^R_{i+1}$ chỉ có thể chứa các cạnh từ $G^R_i$ và các cạnh ngược của các cạnh từ $G^R_i$. Điều này có nghĩa là tồn tại một đường đi ngắn nhất trong $G^R_i$ không bị chặn bởi luồng chặn. Đây là một mâu thuẫn.

Từ hai bổ đề này, chúng ta kết luận rằng có chưa đến $V$ giai đoạn vì $level[t]$ tăng, nhưng nó không thể lớn hơn $V - 1$.

### 3.33.5 Tìm luồng chặn

Để tìm luồng chặn ở mỗi lần lặp, chúng ta có thể đơn giản thử đẩy luồng bằng DFS từ $s$ đến $t$ trong mạng phân tầng trong khi vẫn có thể đẩy được. Để làm điều này nhanh hơn, chúng ta phải loại bỏ các cạnh không thể sử dụng để đẩy nữa. Để làm điều này, chúng ta có thể giữ một con trỏ ở mỗi đỉnh trỏ đến cạnh tiếp theo có thể sử dụng.

Một lần chạy DFS đơn lẻ mất thời gian $O(k + V)$, trong đó $k$ là số lần tiến con trỏ trong lần chạy này. Tổng hợp qua tất cả các lần chạy, số lần tiến con trỏ không thể vượt quá $E$. Mặt khác, tổng số lần chạy không vượt quá $E$, vì mỗi lần chạy bão hòa ít nhất một cạnh. Do đó, tổng thời gian chạy để tìm một luồng chặn là $O(VE)$.

### 3.33.6 Độ phức tạp

Có chưa đến $V$ giai đoạn, nên tổng độ phức tạp là $O(V^2E)$.

### 3.33.7 Mạng đơn vị

Mạng đơn vị là một mạng mà với bất kỳ đỉnh nào trừ $s$ và $t$, hoặc cạnh vào hoặc cạnh ra là duy nhất và có dung lượng đơn vị. Đây chính là trường hợp với mạng mà chúng ta xây dựng để giải bài toán ghép cực đại bằng luồng.

Trên mạng đơn vị, thuật toán Dinic hoạt động trong $O(E\sqrt{V})$. Hãy chứng minh điều này.

Đầu tiên, mỗi giai đoạn giờ đây hoạt động trong $O(E)$ vì mỗi cạnh sẽ được xem xét tối đa một lần.

Thứ hai, giả sử đã có $\sqrt{V}$ giai đoạn. Khi đó, tất cả các đường tăng luồng có độ dài $\leq \sqrt{V}$ đã được tìm thấy. Gọi $f$ là luồng hiện tại, $f'$ là luồng cực đại. Xét hiệu $f' - f$. Nó là một luồng trong $G^R$ có giá trị $|f'| - |f|$ và trên mỗi cạnh, nó là 0 hoặc 1. Nó có thể được phân tách thành $|f'| - |f|$ đường đi từ $s$ đến $t$ và có thể là các chu trình. Vì mạng là đơn vị, chúng không thể có đỉnh chung, nên tổng số đỉnh là $\geq (|f'| - |f|)\sqrt{V}$, nhưng cũng $\leq V$, nên trong $\sqrt{V}$ lần lặp nữa, chúng ta chắc chắn sẽ tìm được luồng cực đại.

### 3.33.8 Mạng có dung lượng đơn vị

Trong cài đặt tổng quát hơn khi tất cả các cạnh có dung lượng đơn vị, nhưng số lượng cạnh vào và ra không bị giới hạn, các đường đi không thể có cạnh chung thay vì đỉnh chung. Theo cách tương tự, điều này cho phép chứng minh giới hạn $\sqrt{E}$ về số lần lặp, do đó thời gian chạy của thuật toán Dinic trên các mạng như vậy là tối đa $O(E\sqrt{E})$.

Cuối cùng, cũng có thể chứng minh rằng số giai đoạn trên mạng có dung lượng đơn vị không vượt quá $O(V^{2/3})$, cung cấp một ước lượng thay thế là $O(EV^{2/3})$ trên các mạng có số lượng cạnh đặc biệt lớn.

### 3.33.9 Triển khai

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
```

```
bool bfs() {
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int id : adj[v]) {
            if (edges[id].cap == edges[id].flow)
                continue;
            if (level[edges[id].u] != -1)
                continue;
            level[edges[id].u] = level[v] + 1;
            q.push(edges[id].u);
        }
    }
    return level[t] != -1;
}

long long dfs(int v, long long pushed) {
    if (pushed == 0)
        return 0;
    if (v == t)
        return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u])
            continue;
        long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
```

```
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

## 3.34 Luồng cực đại - Thuật toán MPM

Thuật toán MPM (Malhotra, Pramodh-Kumar và Maheshwari) giải quyết bài toán luồng cực đại trong thời gian $O(V^3)$. Thuật toán này tương tự như thuật toán Dinic.

### 3.34.1 Thuật toán

Giống như thuật toán Dinic, MPM chạy theo các giai đoạn, trong mỗi giai đoạn chúng ta tìm luồng chặn trong mạng phân tầng của mạng còn lại của $G$. Điểm khác biệt chính so với Dinic là cách chúng ta tìm luồng chặn. Xét mạng phân tầng $L$. Với mỗi nút, chúng ta định nghĩa tiềm năng bên trong và tiềm năng bên ngoài như sau:

$$p_{\text{in}}(v) = \sum_{(u,v)\in L} (c(u,v) - f(u,v))$$
$$p_{\text{out}}(v) = \sum_{(v,u)\in L} (c(v,u) - f(v,u))$$

Ngoài ra, chúng ta đặt $p_{in}(s) = p_{out}(t) = \infty$. Với $p_{in}$ và $p_{out}$, chúng ta định nghĩa tiềm năng là $p(v) = \min(p_{in}(v), p_{out}(v))$. Chúng ta gọi một nút $r$ là nút tham chiếu nếu $p(r) = \min\{p(v)\}$. Xét một nút tham chiếu $r$. Chúng ta khẳng định rằng luồng có thể được tăng thêm $p(r)$ theo cách mà $p(r)$ trở thành 0. Điều này đúng vì $L$ không có chu trình, nên chúng ta có thể đẩy luồng ra khỏi $r$ qua các cạnh ra và nó sẽ đến $t$ vì mỗi nút có đủ tiềm năng bên ngoài để đẩy luồng ra khi luồng đến nó. Tương tự, chúng ta có thể kéo luồng từ $s$. Việc xây dựng luồng chặn dựa trên thực tế này. Ở mỗi lần lặp, chúng ta tìm một nút tham chiếu và đẩy luồng từ $s$ đến $t$ qua $r$. Quá trình này có thể được mô phỏng bằng BFS. Tất cả các cung hoàn toàn bão hòa có thể được xóa khỏi $L$ vì chúng sẽ

không được sử dụng sau này trong giai đoạn này. Tương tự, tất cả các nút khác với $s$ và $t$ mà không có cung ra hoặc cung vào cũng có thể được xóa.

Mỗi giai đoạn hoạt động trong $O(V^2)$ vì có tối đa $V$ lần lặp (vì ít nhất nút tham chiếu được chọn sẽ bị xóa), và ở mỗi lần lặp, chúng ta xóa tất cả các cạnh đã đi qua trừ tối đa $V$ cạnh. Tổng hợp lại, ta được $O(V^2 + E) = O(V^2)$. Vì có chưa đến $V$ giai đoạn (xem chứng minh tại đây), MPM hoạt động trong tổng thời gian $O(V^3)$.

### 3.34.2 Triển khai

```
struct MPM{
    struct FlowEdge{
        int v, u;
        long long cap, flow;
        FlowEdge(){}
        FlowEdge(int _v, int _u, long long _cap, long long _flow)
            : v(_v), u(_u), cap(_cap), flow(_flow){}
        FlowEdge(int _v, int _u, long long _cap)
            : v(_v), u(_u), cap(_cap), flow(0ll){}
    };
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<char> alive;
    vector<long long> pin, pout;
    vector<list<int> > in, out;
    vector<vector<int> > adj;
    vector<long long> ex;
    int n, m = 0;
    int s, t;
    vector<int> level;
    vector<int> q;
    int qh, qt;
    void resize(int _n){
        n = _n;
        ex.resize(n);
        q.resize(n);
        pin.resize(n);
        pout.resize(n);
        adj.resize(n);
        level.resize(n);
        in.resize(n);
        out.resize(n);
```

```
    }
    MPM(){}
    MPM(int _n, int _s, int _t){resize(_n); s = _s; t = _t;}
    void add_edge(int v, int u, long long cap){
        edges.push_back(FlowEdge(v, u, cap));
        edges.push_back(FlowEdge(u, v, 0));
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
    bool bfs(){
        while(qh < qt){
            int v = q[qh++];
            for(int id : adj[v]){
                if(edges[id].cap - edges[id].flow < 1)continue;
                if(level[edges[id].u] != -1)continue;
                level[edges[id].u] = level[v] + 1;
                q[qt++] = edges[id].u;
            }
        }
        return level[t] != -1;
    }
    long long pot(int v){
        return min(pin[v], pout[v]);
    }
    void remove_node(int v){
        for(int i : in[v]){
            int u = edges[i].v;
            auto it = find(out[u].begin(), out[u].end(), i);
            out[u].erase(it);
            pout[u] -= edges[i].cap - edges[i].flow;
        }
        for(int i : out[v]){
            int u = edges[i].u;
            auto it = find(in[u].begin(), in[u].end(), i);
            in[u].erase(it);
            pin[u] -= edges[i].cap - edges[i].flow;
        }
    }
    void push(int from, int to, long long f, bool forw){
```

```
    qh = qt = 0;
    ex.assign(n, 0);
    ex[from] = f;
    q[qt++] = from;
    while(qh < qt){
        int v = q[qh++];
        if(v == to)
            break;
        long long must = ex[v];
        auto it = forw ? out[v].begin() : in[v].begin();
        while(true){
            int u = forw ? edges[*it].u : edges[*it].v;
            long long pushed = min(must, edges[*it].cap - edges[*it].flow);
            if(pushed == 0)break;
            if(forw){
                pout[v] -= pushed;
                pin[u] -= pushed;
            }
            else{
                pin[v] -= pushed;
                pout[u] -= pushed;
            }
            if(ex[u] == 0)
                q[qt++] = u;
            ex[u] += pushed;
            edges[*it].flow += pushed;
            edges[(*it)^1].flow -= pushed;
            must -= pushed;
            if(edges[*it].cap - edges[*it].flow == 0){
                auto jt = it;
                ++jt;
                if(forw){
                    in[u].erase(find(in[u].begin(), in[u].end(), *it));
                    out[v].erase(it);
                }
                else{
                    out[u].erase(find(out[u].begin(), out[u].end(), *it));
                    in[v].erase(it);
                }
                it = jt;
```

```
                }
                else break;
                if(!must)break;
            }
        }
    }
    long long flow(){
        long long ans = 0;
        while(true){
            pin.assign(n, 0);
            pout.assign(n, 0);
            level.assign(n, -1);
            alive.assign(n, true);
            level[s] = 0;
            qh = 0; qt = 1;
            q[0] = s;
            if(!bfs())
                break;
            for(int i = 0; i < n; i++){
                out[i].clear();
                in[i].clear();
            }
            for(int i = 0; i < m; i++){
                if(edges[i].cap - edges[i].flow == 0)
                    continue;
                int v = edges[i].v, u = edges[i].u;
                if(level[v] + 1 == level[u] && (level[u] < level[t] || u == t)){
                    in[u].push_back(i);
                    out[v].push_back(i);
                    pin[u] += edges[i].cap - edges[i].flow;
                    pout[v] += edges[i].cap - edges[i].flow;
                }
            }
            pin[s] = pout[t] = flow_inf;
            while(true){
                int v = -1;
                for(int i = 0; i < n; i++){
                    if(!alive[i])continue;
                    if(v == -1 || pot(i) < pot(v))
                        v = i;
```

```
                }
                if(v == -1)
                    break;
                if(pot(v) == 0){
                    alive[v] = false;
                    remove_node(v);
                    continue;
                }
                long long f = pot(v);
                ans += f;
                push(v, s, f, false);
                push(v, t, f, true);
                alive[v] = false;
                remove_node(v);
            }
        }
        return ans;
    }
};
```

## 3.35 Luồng với Yêu Cầu

Trong một mạng luồng thông thường, luồng trên một cạnh chỉ bị giới hạn trên bởi khả năng chuyên chở $c(e)$ và giới hạn dưới bởi 0. Trong bài viết này, chúng ta sẽ thảo luận về các mạng luồng, trong đó chúng ta yêu cầu thêm rằng luồng trên mỗi cạnh phải có một lượng nhất định, tức là chúng ta chặn luồng từ dưới bởi một hàm yêu cầu $d(e)$:

$$d(e) \leq f(e) \leq c(e)$$

Như vậy, mỗi cạnh có một giá trị luồng tối thiểu mà chúng ta phải truyền qua cạnh đó.

Đây là một sự tổng quát hóa của bài toán luồng thông thường, vì việc đặt $d(e) = 0$ cho tất cả các cạnh $e$ sẽ cho ta một mạng luồng thông thường. Lưu ý rằng, trong mạng luồng thông thường, việc tìm một luồng hợp lệ là cực kỳ đơn giản, chỉ cần đặt $f(e) = 0$ là đã có một luồng hợp lệ. Tuy nhiên, nếu luồng trên mỗi cạnh phải thỏa mãn một yêu cầu, thì việc tìm một luồng hợp lệ trở nên khá phức tạp.

Chúng ta sẽ xem xét hai bài toán:

- Tìm một luồng bất kỳ thỏa mãn tất cả các ràng buộc
- Tìm một luồng tối thiểu thỏa mãn tất cả các ràng buộc

### 3.35.1 Tìm một luồng bất kỳ

Chúng ta thực hiện các thay đổi sau trong mạng. Chúng ta thêm một nguồn mới $s'$ và một đích mới $t'$, một cạnh mới từ nguồn $s'$ đến mọi đỉnh khác, một cạnh mới từ mọi đỉnh đến đích $t'$, và một cạnh từ $t$ đến $s$. Ngoài ra, chúng ta định nghĩa hàm khả năng chuyên chở mới $c'$ như sau:

$$c'((s', v)) = \sum_{u \in V} d((u, v)) \quad \text{cho mỗi cạnh } (s', v).$$

$$c'((v, t')) = \sum_{w \in V} d((v, w)) \quad \text{cho mỗi cạnh } (v, t').$$

$$c'((u, v)) = c((u, v)) - d((u, v)) \quad \text{cho mỗi cạnh } (u, v) \text{ trong mạng cũ.}$$

$$c'((t, s)) = \infty$$

Nếu mạng mới có một luồng bão hòa (một luồng mà mọi cạnh đi ra từ $s'$ đều được lấp đầy hoàn toàn, tương đương với việc mọi cạnh đi vào $t'$ đều được lấp đầy hoàn toàn), thì mạng với các yêu cầu có một luồng hợp lệ, và luồng thực tế có thể dễ dàng được tái cấu trúc từ mạng mới. Nếu không, thì không tồn tại một luồng nào thỏa mãn tất cả các điều kiện. Vì một luồng bão hòa phải là một luồng cực đại, nó có thể được tìm thấy bằng bất kỳ thuật toán luồng cực đại nào, chẳng hạn như thuật toán Edmonds-Karp hoặc thuật toán Push-relabel.

Tính đúng đắn của các phép biến đổi này khó hiểu hơn. Chúng ta có thể nghĩ về nó theo cách sau: Mỗi cạnh $e = (u, v)$ với $d(e) > 0$ ban đầu được thay thế bằng hai cạnh: một cạnh với khả năng chuyên chở $d(e)$, và cạnh còn lại với $c(e) - d(e)$. Chúng ta muốn tìm một luồng bão hòa cạnh đầu tiên (tức là luồng dọc theo cạnh này phải bằng khả năng chuyên chở của nó). Cạnh thứ hai ít quan trọng hơn - luồng dọc theo nó có thể là bất cứ điều gì, giả sử rằng nó không vượt quá khả năng chuyên chở của nó. Xét mỗi cạnh phải được bão hòa, và chúng ta thực hiện thao tác sau: chúng ta vẽ cạnh từ nguồn mới $s'$ đến điểm cuối của nó $v$, vẽ cạnh từ điểm bắt đầu của nó $u$ đến đích mới $t'$, loại bỏ cạnh đó, và từ đích cũ $t$ đến nguồn cũ $s$ chúng ta vẽ một cạnh có khả năng vô hạn. Bằng những hành động này, chúng ta mô phỏng thực tế rằng cạnh này đã bão hòa - từ $v$ sẽ có thêm một luồng $d(e)$ đi ra (chúng ta mô phỏng nó bằng một nguồn mới cung cấp lượng luồng phù hợp cho $v$), và $u$ cũng sẽ đẩy thêm một luồng $d(e)$ (nhưng thay vì dọc theo cạnh cũ, luồng này sẽ đi trực tiếp đến đích mới $t'$). Một luồng có giá trị $d(e)$, ban đầu chảy dọc theo đường dẫn $s - \cdots - u - v - \ldots t$ giờ có thể đi theo đường dẫn mới $s' - v - \cdots - t - s - \cdots - u - t'$. Điều duy nhất được đơn giản hóa trong định nghĩa của mạng mới, là nếu quy trình tạo ra nhiều cạnh giữa cùng một cặp đỉnh, thì chúng được kết hợp thành một cạnh duy nhất với khả năng chuyên chở được cộng lại.

### 3.35.2 Luồng tối thiểu

Lưu ý rằng dọc theo cạnh $(t, s)$ (từ đích cũ đến nguồn cũ) với khả năng chuyên chở $\infty$ chảy toàn bộ luồng của mạng cũ tương ứng. Tức là, khả năng chuyên chở của cạnh này ảnh hưởng đến giá trị luồng của mạng cũ. Bằng cách cho cạnh này một khả năng chuyên chở đủ lớn (tức là $\infty$), luồng của mạng cũ là không giới hạn. Bằng cách giới hạn cạnh này bằng các khả năng chuyên chở nhỏ hơn, giá trị luồng sẽ giảm. Tuy nhiên, nếu chúng ta giới hạn cạnh này bằng một giá trị quá nhỏ, thì mạng sẽ không có một nghiệm bão hòa, ví dụ: nghiệm tương ứng cho mạng gốc sẽ không thỏa mãn yêu cầu của các cạnh. Rõ ràng ở đây có thể sử dụng tìm kiếm nhị phân để tìm giá trị thấp nhất mà tất cả các ràng buộc vẫn được thỏa mãn. Điều này cho ta luồng tối thiểu của mạng gốc.

## 3.36 Luồng Cực Tiểu Chi Phí - Thuật Toán Đường Đi Ngắn Nhất Liên Tiếp

Cho một mạng $G$ bao gồm $n$ đỉnh và $m$ cạnh. Với mỗi cạnh (nói chung là các cạnh có hướng, nhưng xem bên dưới), cho trước khả năng chuyên chở (một số nguyên không âm) và chi phí trên mỗi đơn vị luồng dọc theo cạnh này (một số nguyên). Ngoài ra, nguồn $s$ và đích $t$ được đánh dấu.

Với một giá trị $K$ cho trước, chúng ta phải tìm một luồng có giá trị này, và trong số tất cả các luồng có giá trị này, chúng ta phải chọn luồng có chi phí thấp nhất. Nhiệm vụ này được gọi là bài toán luồng cực tiểu chi phí.

Đôi khi nhiệm vụ được đưa ra hơi khác một chút: bạn muốn tìm luồng cực đại, và trong số tất cả các luồng cực đại, chúng ta muốn tìm luồng có chi phí thấp nhất. Điều này được gọi là bài toán luồng cực đại chi phí cực tiểu.

Cả hai bài toán này đều có thể được giải quyết hiệu quả bằng thuật toán đường đi ngắn nhất liên tiếp.

### 3.36.1 Thuật Toán

Thuật toán này rất giống với Edmonds-Karp để tính toán luồng cực đại.

#### Trường hợp đơn giản nhất

Đầu tiên, chúng ta chỉ xem xét trường hợp đơn giản nhất, trong đó đồ thị có hướng và có tối đa một cạnh giữa bất kỳ cặp đỉnh nào (ví dụ: nếu $(i, j)$ là một cạnh trong đồ thị, thì $(j, i)$ cũng không thể là một phần của nó).

Gọi $U_{ij}$ là khả năng chuyên chở của một cạnh $(i, j)$ nếu cạnh này tồn tại. Và gọi $C_{ij}$ là chi phí trên mỗi đơn vị luồng dọc theo cạnh $(i, j)$. Và cuối cùng, gọi $F_{i,j}$ là luồng dọc theo cạnh $(i, j)$. Ban đầu, tất cả các giá trị luồng đều bằng không.

Chúng ta sửa đổi mạng như sau: đối với mỗi cạnh $(i, j)$, chúng ta thêm cạnh ngược $(j, i)$ vào mạng với khả năng chuyên chở $U_{ji} = 0$ và chi phí $C_{ji} = -C_{ij}$. Vì, theo các hạn chế của chúng ta, cạnh $(j, i)$ không có trong mạng trước đó, chúng ta vẫn có một mạng không phải là đa đồ thị (đồ thị có nhiều cạnh). Ngoài ra, chúng ta sẽ luôn giữ điều kiện $F_{ji} = -F_{ij}$ đúng trong các bước của thuật toán.

Chúng ta định nghĩa mạng thặng dư cho một số luồng cố định $F$ như sau (giống như trong thuật toán Ford-Fulkerson): mạng thặng dư chỉ chứa các cạnh chưa bão hòa (tức là các cạnh trong đó $F_{ij} < U_{ij}$), và khả năng chuyên chở thặng dư của mỗi cạnh như vậy là $R_{ij} = U_{ij} - F_{ij}$.

Bây giờ chúng ta có thể nói về các thuật toán để tính toán luồng cực tiểu chi phí. Tại mỗi lần lặp của thuật toán, chúng ta tìm đường đi ngắn nhất trong đồ thị thặng dư từ $s$ đến $t$. Ngược lại với Edmonds-Karp, chúng ta tìm kiếm đường đi ngắn nhất theo chi phí của đường đi thay vì số lượng cạnh. Nếu không còn đường đi nào nữa, thì thuật toán kết thúc và luồng $F$ là luồng mong muốn. Nếu một đường đi được tìm thấy, chúng ta tăng luồng dọc theo nó càng nhiều càng tốt (tức là chúng ta tìm khả năng chuyên chở thặng dư tối thiểu $R$ của đường đi và tăng luồng theo nó, và giảm các cạnh ngược lại một lượng tương tự). Nếu tại một thời điểm nào đó, luồng đạt đến giá trị $K$, thì chúng ta dừng thuật toán (lưu ý rằng trong lần lặp cuối cùng của thuật toán, cần thiết phải tăng luồng chỉ một lượng sao cho giá trị luồng cuối cùng không vượt quá $K$).

Không khó để thấy rằng, nếu chúng ta đặt $K$ thành vô cực, thì thuật toán sẽ tìm thấy luồng cực đại chi phí cực tiểu. Vì vậy, cả hai biến thể của bài toán đều có thể được giải quyết bằng cùng một thuật toán.

### Đồ thị vô hướng / Đa đồ thị

Trường hợp đồ thị vô hướng hoặc đa đồ thị không khác về mặt khái niệm so với thuật toán trên. Thuật toán cũng sẽ hoạt động trên các đồ thị này. Tuy nhiên, việc triển khai nó trở nên khó khăn hơn một chút.

Một cạnh vô hướng $(i, j)$ thực sự giống như hai cạnh có hướng $(i, j)$ và $(j, i)$ với cùng khả năng chuyên chở và giá trị. Vì thuật toán luồng cực tiểu chi phí được mô tả ở trên tạo ra một cạnh ngược cho mỗi cạnh có hướng, nên nó chia cạnh vô hướng thành 4 cạnh có hướng và chúng ta thực sự có được một đa đồ thị.

Làm thế nào để chúng ta đối phó với nhiều cạnh? Đầu tiên, luồng cho mỗi cạnh trong số nhiều cạnh phải được giữ riêng biệt. Thứ hai, khi tìm kiếm đường đi ngắn nhất, cần phải tính đến rằng điều quan trọng là cạnh nào trong số nhiều cạnh được sử dụng trong đường đi. Do đó, thay vì mảng tổ tiên thông thường, chúng ta phải lưu trữ thêm số cạnh mà chúng ta đến từ đó cùng với tổ tiên. Thứ ba, khi luồng tăng dọc theo một cạnh nhất định, cần thiết phải giảm luồng dọc theo cạnh ngược lại. Vì chúng ta có nhiều cạnh, chúng ta phải lưu trữ số cạnh cho cạnh đảo ngược cho mỗi cạnh.

Không có trở ngại nào khác với đồ thị vô hướng hoặc đa đồ thị.

### 3.36.2 Độ Phức Tạp

Thuật toán ở đây thường có độ phức tạp hàm mũ theo kích thước đầu vào. Cụ thể hơn, trong trường hợp xấu nhất, nó có thể chỉ đẩy 1 đơn vị luồng trên mỗi lần lặp, mất $O(F)$ lần lặp để tìm một luồng cực tiểu chi phí có kích thước $F$, làm cho tổng thời gian chạy là $O(F \cdot T)$, trong đó $T$ là thời gian cần thiết để tìm đường đi ngắn nhất từ nguồn đến đích.

Nếu thuật toán Bellman-Ford được sử dụng cho việc này, nó sẽ làm cho thời gian chạy là $O(Fmn)$. Cũng có thể sửa đổi thuật toán Dijkstra, sao cho nó cần $O(nm)$ tiền xử lý như một bước ban đầu và sau đó hoạt động trong $O(m \log n)$ trên mỗi lần lặp, làm cho tổng thời gian chạy là $O(mn + Fm \log n)$. Đây là một trình tạo đồ thị, trên đó thuật toán như vậy sẽ yêu cầu $O(2^{n/2} n^2 \log n)$ thời gian.

Thuật toán Dijkstra đã sửa đổi sử dụng cái gọi là thế năng từ thuật toán Johnson. Có thể kết hợp các ý tưởng của thuật toán này và thuật toán Dinic để giảm số lần lặp từ $F$ xuống $\min(F, nC)$, trong đó $C$ là chi phí tối đa được tìm thấy giữa các cạnh. Bạn có thể đọc thêm về thế năng và sự kết hợp của chúng với thuật toán Dinic tại đây.

### 3.36.3 Triển Khai

Dưới đây là một triển khai sử dụng thuật toán SPFA cho trường hợp đơn giản nhất.

```
struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
```

```
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]) {
                d[v] = d[u] + cost[u][v];
                p[v] = u;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
}

int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }

    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;

        // find max flow on that path
        int f = K - flow;
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]][cur]);
            cur = p[cur];
        }
```

```
        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }
    }

    if (flow < K)
        return -1;
    else
        return cost;
}
```

## 3.37 Giải Bài Toán Gán Việc Sử Dụng Luồng Cực Tiểu Chi Phí

Bài toán gán việc có hai phát biểu tương đương:

1. Cho một ma trận vuông $A[1..N, 1..N]$, bạn cần chọn $N$ phần tử trong đó sao cho có đúng một phần tử được chọn trong mỗi hàng và cột, và tổng các giá trị của các phần tử này là nhỏ nhất.

2. Có $N$ đơn hàng và $N$ máy móc. Chi phí sản xuất trên mỗi máy là đã biết cho mỗi đơn hàng. Chỉ một đơn hàng có thể được thực hiện trên mỗi máy. Yêu cầu là gán tất cả các đơn hàng cho các máy sao cho tổng chi phí là tối thiểu.

Ở đây chúng ta sẽ xem xét giải pháp của bài toán dựa trên thuật toán tìm luồng cực tiểu chi phí (min-cost-flow), giải bài toán gán việc trong $\mathcal{O}(N^3)$.

### 3.37.1 Mô Tả

Hãy xây dựng một mạng hai phía: có một nguồn $S$, một đích $T$, trong phần đầu tiên có $N$ đỉnh (tương ứng với các hàng của ma trận, hoặc các đơn hàng), trong phần thứ hai cũng có $N$ đỉnh (tương ứng với các cột của ma trận, hoặc các máy móc). Giữa mỗi đỉnh $i$ của tập đầu tiên và mỗi đỉnh $j$ của tập thứ hai, chúng ta vẽ một cạnh với băng thông 1

và chi phí $A_{ij}$. Từ nguồn $S$ chúng ta vẽ các cạnh đến tất cả các đỉnh $i$ của tập đầu tiên với băng thông 1 và chi phí 0. Chúng ta vẽ một cạnh với băng thông 1 và chi phí 0 từ mỗi đỉnh của tập thứ hai $j$ đến đích $T$.

Chúng ta tìm trong mạng kết quả luồng cực đại với chi phí tối thiểu. Rõ ràng, giá trị của luồng sẽ là $N$. Hơn nữa, đối với mỗi đỉnh $i$ của đoạn đầu tiên, có chính xác một đỉnh $j$ của đoạn thứ hai, sao cho luồng $F_{ij} = 1$. Cuối cùng, đây là một sự tương ứng một-một giữa các đỉnh của đoạn đầu tiên và các đỉnh của phần thứ hai, đó là giải pháp cho bài toán (vì luồng tìm được có chi phí tối thiểu, thì tổng chi phí của các cạnh được chọn sẽ là thấp nhất có thể, đó là tiêu chí tối ưu).

Độ phức tạp của giải pháp này của bài toán gán việc phụ thuộc vào thuật toán mà việc tìm kiếm luồng cực đại với chi phí tối thiểu được thực hiện. Độ phức tạp sẽ là $\mathcal{O}(N^3)$ sử dụng Dijkstra hoặc $\mathcal{O}(N^4)$ sử dụng Bellman-Ford. Điều này là do thực tế là luồng có kích thước $O(N)$ và mỗi lần lặp của thuật toán Dijkstra có thể được thực hiện trong $O(N^2)$, trong khi nó là $O(N^3)$ đối với Bellman-Ford.

### 3.37.2 Triển Khai

Việc triển khai được đưa ra ở đây là dài, nó có thể được giảm đáng kể. Nó sử dụng thuật toán SPFA để tìm đường đi ngắn nhất.

```
const int INF = 1000 * 1000 * 1000;

vector<int> assignment(vector<vector<int>> a) {
    int n = a.size();
    int m = n * 2 + 2;
    vector<vector<int>> f(m, vector<int>(m));
    int s = m - 2, t = m - 1;
    int cost = 0;
    while (true) {
        vector<int> dist(m, INF);
        vector<int> p(m);
        vector<bool> inq(m, false);
        queue<int> q;
        dist[s] = 0;
        p[s] = -1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            inq[v] = false;
            if (v == s) {
```

```
            for (int i = 0; i < n; ++i) {
                if (f[s][i] == 0) {
                    dist[i] = 0;
                    p[i] = s;
                    inq[i] = true;
                    q.push(i);
                }
            }
        } else {
            if (v < n) {
                for (int j = n; j < n + n; ++j) {
                    if (f[v][j] < 1 && dist[j] > dist[v] + a[v][j - n]) {
                        dist[j] = dist[v] + a[v][j - n];
                        p[j] = v;
                        if (!inq[j]) {
                            q.push(j);
                            inq[j] = true;
                        }
                    }
                }
            } else {
                for (int j = 0; j < n; ++j) {
                    if (f[v][j] < 0 && dist[j] > dist[v] - a[j][v - n]) {
                        dist[j] = dist[v] - a[j][v - n];
                        p[j] = v;
                        if (!inq[j]) {
                            q.push(j);
                            inq[j] = true;
                        }
                    }
                }
            }
        }
    }

    int curcost = INF;
    for (int i = n; i < n + n; ++i) {
        if (f[i][t] == 0 && dist[i] < curcost) {
            curcost = dist[i];
            p[t] = i;
```

```
            }
        }
        if (curcost == INF)
            break;
        cost += curcost;
        for (int cur = t; cur != -1; cur = p[cur]) {
            int prev = p[cur];
            if (prev != -1)
                f[cur][prev] = -(f[prev][cur] = 1);
        }
    }

    vector<int> answer(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (f[i][j + n] == 1)
                answer[i] = j;
        }
    }
    return answer;
}
```

## 3.38 Kiểm Tra Xem Một Đồ Thị Có Phải Là Đồ Thị Hai Phía Hay Không

Một đồ thị hai phía là một đồ thị mà các đỉnh của nó có thể được chia thành hai tập hợp rời nhau sao cho mọi cạnh kết nối hai đỉnh từ các tập hợp khác nhau (tức là không có cạnh nào kết nối các đỉnh từ cùng một tập hợp). Các tập hợp này thường được gọi là các phía.

Bạn được cho một đồ thị vô hướng. Kiểm tra xem nó có phải là đồ thị hai phía hay không, và nếu có, hãy xuất ra các phía của nó.

### 3.38.1 Thuật Toán

Có một định lý khẳng định rằng một đồ thị là hai phía khi và chỉ khi tất cả các chu trình của nó có độ dài chẵn. Tuy nhiên, trong thực tế, việc sử dụng một công thức khác của định nghĩa sẽ thuận tiện hơn: một đồ thị là hai phía khi và chỉ khi nó có thể được tô hai màu.

Hãy sử dụng một loạt các tìm kiếm theo chiều rộng, bắt đầu từ mỗi đỉnh chưa được truy cập. Trong mỗi tìm kiếm, gán đỉnh mà chúng ta bắt đầu cho phía 1. Mỗi khi chúng ta

truy cập một hàng xóm chưa được truy cập của một đỉnh được gán cho một phía, chúng ta gán nó cho phía kia. Khi chúng ta cố gắng đi đến một hàng xóm của một đỉnh được gán cho một phía đã được truy cập, chúng ta kiểm tra xem nó đã được gán cho phía kia hay chưa; nếu nó đã được gán cho cùng một phía, chúng ta kết luận rằng đồ thị không phải là hai phía. Khi chúng ta đã truy cập tất cả các đỉnh và gán chúng thành công cho các phía, chúng ta biết rằng đồ thị là hai phía và chúng ta đã xây dựng được sự phân vùng của nó.

### 3.38.2 Triển Khai

```
int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}

cout << (is_bipartite ? "YES" : "NO") << endl;
```

# 3.39 Thuật Toán Kuhn cho Tìm Kiếm Ghép Cặp Lớn Nhất trong Đồ Thị Hai Phía

## 3.39.1 Bài Toán

Cho một đồ thị hai phía $G$ chứa $n$ đỉnh và $m$ cạnh. Tìm kiếm ghép cặp lớn nhất, tức là chọn càng nhiều cạnh càng tốt sao cho không có cạnh nào được chọn chung đỉnh với bất kỳ cạnh được chọn nào khác.

## 3.39.2 Mô Tả Thuật Toán

### Các Định Nghĩa Cần Thiết

Một phép ghép cặp $M$ là một tập hợp các cạnh không kề nhau theo cặp của một đồ thị (nói cách khác, không quá một cạnh từ tập hợp được liên kết với bất kỳ đỉnh nào của đồ thị $M$). Bản số của một phép ghép cặp là số lượng cạnh trong đó. Tất cả các đỉnh có một cạnh liền kề từ phép ghép cặp (tức là có bậc chính xác là một trong đồ thị con được tạo bởi $M$) được gọi là bão hòa bởi phép ghép cặp này.

Một phép ghép cặp cực đại là một phép ghép cặp $M$ của một đồ thị $G$ mà không phải là một tập hợp con của bất kỳ phép ghép cặp nào khác.

Một phép ghép cặp lớn nhất (còn được gọi là phép ghép cặp có bản số lớn nhất) là một phép ghép cặp chứa số lượng cạnh lớn nhất có thể. Mọi phép ghép cặp lớn nhất đều là một phép ghép cặp cực đại.

Một đường đi có độ dài $k$ ở đây có nghĩa là một đường đi đơn giản (tức là không chứa các đỉnh hoặc cạnh lặp lại) chứa $k$ cạnh, trừ khi được chỉ định khác.

Một đường đi xen kẽ (trong một đồ thị hai phía, liên quan đến một số phép ghép cặp) là một đường đi trong đó các cạnh luân phiên thuộc / không thuộc phép ghép cặp.

Một đường đi tăng (trong một đồ thị hai phía, liên quan đến một số phép ghép cặp) là một đường đi xen kẽ có các đỉnh đầu và cuối không bão hòa, tức là chúng không thuộc phép ghép cặp.

Hiệu đối xứng (còn được gọi là hợp không giao nhau) của các tập hợp $A$ và $B$, được biểu diễn bằng $A \oplus B$, là tập hợp của tất cả các phần tử thuộc chính xác một trong $A$ hoặc $B$, nhưng không thuộc cả hai. Đó là, $A \oplus B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$.

### Bổ Đề Berge

Bổ đề này được chứng minh bởi nhà toán học người Pháp Claude Berge vào năm 1957, mặc dù nó đã được quan sát bởi nhà toán học người Đan Mạch Julius Petersen vào năm 1891 và nhà toán học người Hungary Denés Kőnig vào năm 1931.

**Phát biểu**

Một phép ghép cặp $M$ là lớn nhất $\Leftrightarrow$ không có đường đi tăng tương ứng với phép ghép cặp $M$.

**Chứng minh**

Cả hai phía của phép kéo theo hai chiều sẽ được chứng minh bằng phản chứng.

**Một phép ghép cặp $M$ là lớn nhất $\Rightarrow$ không có đường đi tăng tương ứng với phép ghép cặp $M$.**

Giả sử có một đường đi tăng $P$ tương ứng với phép ghép cặp lớn nhất đã cho $M$. Đường đi tăng $P$ này nhất thiết sẽ có độ dài lẻ, có nhiều hơn một cạnh không thuộc $M$ so với số lượng cạnh mà nó có cũng thuộc $M$. Chúng ta tạo một phép ghép cặp mới $M'$ bằng cách bao gồm tất cả các cạnh trong phép ghép cặp ban đầu $M$ ngoại trừ những cạnh cũng nằm trong $P$, và các cạnh trong $P$ không nằm trong $M$. Đây là một phép ghép cặp hợp lệ vì các đỉnh đầu và cuối của $P$ không bão hòa bởi $M$, và các đỉnh còn lại chỉ bão hòa bởi phép ghép cặp $P \cap M$. Phép ghép cặp mới này $M'$ sẽ có nhiều hơn một cạnh so với $M$, và do đó $M$ không thể là lớn nhất.

Một cách hình thức, cho một đường đi tăng $P$ đối với một số phép ghép cặp lớn nhất $M$, phép ghép cặp $M' = P \oplus M$ sao cho $|M'| = |M| + 1$, một mâu thuẫn.

**Một phép ghép cặp $M$ là lớn nhất $\Leftarrow$ không có đường đi tăng tương ứng với phép ghép cặp $M$.**

Giả sử có một phép ghép cặp $M'$ có bản số lớn hơn $M$. Chúng ta xem xét hiệu đối xứng $Q = M \oplus M'$. Đồ thị con $Q$ không còn nhất thiết là một phép ghép cặp. Bất kỳ đỉnh nào trong $Q$ có bậc tối đa là 2, có nghĩa là tất cả các thành phần liên thông trong đó là một trong ba -

- một đỉnh cô lập
- một đường đi (đơn giản) có các cạnh luân phiên từ $M$ và $M'$
- một chu trình có độ dài chẵn có các cạnh luân phiên từ $M$ và $M'$

Vì $M'$ có bản số lớn hơn $M$, $Q$ có nhiều cạnh từ $M'$ hơn $M$. Theo nguyên lý Pigeonhole, ít nhất một thành phần liên thông sẽ là một đường đi có nhiều cạnh từ $M'$ hơn $M$. Bởi vì bất kỳ đường đi nào như vậy là xen kẽ, nó sẽ có các đỉnh đầu và cuối không bão hòa bởi $M$, làm cho nó trở thành một đường đi tăng cho $M$, điều này mâu thuẫn với tiền đề.

### Thuật Toán Kuhn

Thuật toán Kuhn là một ứng dụng trực tiếp của bổ đề Berge. Về cơ bản nó được mô tả như sau:

Đầu tiên, chúng ta lấy một phép ghép cặp trống. Sau đó, trong khi thuật toán có thể tìm thấy một đường đi tăng, chúng ta cập nhật phép ghép cặp bằng cách luân phiên nó dọc theo đường đi này và lặp lại quá trình tìm kiếm đường đi tăng. Ngay sau khi không

thể tìm thấy một đường đi như vậy, chúng ta dừng quá trình - phép ghép cặp hiện tại là lớn nhất.

Vẫn còn phải chi tiết cách tìm đường đi tăng. Thuật toán Kuhn chỉ đơn giản là tìm kiếm bất kỳ đường đi nào trong số này bằng cách sử dụng duyệt theo chiều sâu hoặc duyệt theo chiều rộng. Thuật toán xem xét tất cả các đỉnh của đồ thị lần lượt, bắt đầu mỗi lần duyệt từ nó, cố gắng tìm một đường đi tăng bắt đầu từ đỉnh này.

Thuật toán thuận tiện hơn để mô tả nếu chúng ta giả định rằng đồ thị đầu vào đã được chia thành hai phần (mặc dù, trên thực tế, thuật toán có thể được triển khai theo cách mà đồ thị đầu vào không được chia rõ ràng thành hai phần).

Thuật toán xem xét tất cả các đỉnh $v$ của phần đầu tiên của đồ thị: $v = 1 \dots n_1$. Nếu đỉnh hiện tại $v$ đã bão hòa với phép ghép cặp hiện tại (tức là một số cạnh liền kề với nó đã được chọn), thì bỏ qua đỉnh này. Nếu không, thuật toán cố gắng bão hòa đỉnh này, vì nó bắt đầu tìm kiếm một đường đi tăng bắt đầu từ đỉnh này.

Việc tìm kiếm một đường đi tăng được thực hiện bằng cách sử dụng một duyệt theo chiều sâu hoặc duyệt theo chiều rộng đặc biệt (thường thì duyệt theo chiều sâu được sử dụng để dễ triển khai). Ban đầu, việc duyệt theo chiều sâu ở đỉnh không bão hòa hiện tại $v$ của phần đầu tiên. Hãy xem xét tất cả các cạnh từ đỉnh này. Cho cạnh hiện tại là một cạnh $(v, to)$. Nếu đỉnh $to$ chưa bão hòa với phép ghép cặp, thì chúng ta đã thành công trong việc tìm kiếm một đường đi tăng: nó bao gồm một cạnh duy nhất $(v, to)$; trong trường hợp này, chúng ta chỉ cần bao gồm cạnh này trong phép ghép cặp và ngừng tìm kiếm đường đi tăng từ đỉnh $v$. Nếu không, nếu $to$ đã bão hòa với một số cạnh $(to, p)$, thì sẽ đi dọc theo cạnh này: do đó, chúng ta sẽ cố gắng tìm một đường đi tăng đi qua các cạnh $(v, to), (to, p), \dots$. Để làm điều này, chỉ cần đi đến đỉnh $p$ trong quá trình duyệt của chúng ta - bây giờ chúng ta cố gắng tìm một đường đi tăng từ đỉnh này.

Vì vậy, quá trình duyệt này, được khởi chạy từ đỉnh $v$, sẽ tìm thấy một đường đi tăng và do đó bão hòa đỉnh $v$, hoặc nó sẽ không tìm thấy một đường đi tăng như vậy (và do đó, đỉnh này $v$ không thể bão hòa).

Sau khi tất cả các đỉnh $v = 1 \dots n_1$ đã được quét, phép ghép cặp hiện tại sẽ là lớn nhất.

### Thời Gian Chạy

Thuật toán Kuhn có thể được coi là một loạt các lần chạy duyệt theo chiều sâu/chiều rộng $n$ trên toàn bộ đồ thị. Do đó, toàn bộ thuật toán được thực thi trong thời gian $O(nm)$, trong trường hợp xấu nhất là $O(n^3)$.

Tuy nhiên, ước tính này có thể được cải thiện một chút. Hóa ra đối với thuật toán Kuhn, điều quan trọng là phần nào của đồ thị được chọn làm phần đầu tiên và phần nào là phần thứ hai. Thật vậy, trong cách triển khai được mô tả ở trên, quá trình duyệt theo chiều sâu/chiều rộng chỉ bắt đầu từ các đỉnh của phần đầu tiên, vì vậy toàn bộ thuật toán được thực thi trong thời gian $O(n_1 m)$, trong đó $n_1$ là số lượng đỉnh của phần đầu tiên. Trong trường hợp xấu nhất, đây là $O(n_1^2 n_2)$ (trong đó $n_2$ là số lượng đỉnh của phần thứ

hai). Điều này cho thấy rằng sẽ có lợi hơn khi phần đầu tiên chứa ít đỉnh hơn phần thứ hai. Trên các đồ thị rất không cân bằng (khi $n_1$ và $n_2$ rất khác nhau), điều này chuyển thành một sự khác biệt đáng kể về thời gian chạy.

### 3.39.3 Triển Khai

#### Triển Khai Tiêu Chuẩn

Hãy trình bày ở đây một triển khai của thuật toán trên dựa trên duyệt theo chiều sâu và chấp nhận một đồ thị hai phía dưới dạng một đồ thị được chia rõ ràng thành hai phần. Việc triển khai này rất ngắn gọn và có lẽ nó nên được ghi nhớ ở dạng này.

Ở đây $n$ là số lượng đỉnh trong phần đầu tiên, $k$ - trong phần thứ hai, $g[v]$ là danh sách các cạnh từ đỉnh của phần đầu tiên (tức là danh sách các số của các đỉnh mà các cạnh này dẫn đến từ $v$). Các đỉnh ở cả hai phần được đánh số độc lập, tức là các đỉnh ở phần đầu tiên được đánh số $1 \dots n$, và các đỉnh ở phần thứ hai được đánh số $1 \dots k$.

Sau đó, có hai mảng phụ trợ: mt và used. Mảng đầu tiên - mt - chứa thông tin về phép ghép cặp hiện tại. Để thuận tiện cho việc lập trình, thông tin này chỉ được chứa cho các đỉnh của phần thứ hai: mt[$i$] - đây là số của đỉnh của phần đầu tiên được kết nối bằng một cạnh với đỉnh $i$ của phần thứ hai (hoặc $-1$, nếu không có cạnh ghép cặp nào đi ra từ nó). Mảng thứ hai là used: mảng "lượt truy cập" thông thường đến các đỉnh trong quá trình duyệt theo chiều sâu (nó chỉ cần thiết để quá trình duyệt theo chiều sâu không vào cùng một đỉnh hai lần).

Một hàm try_kuhn là một quá trình duyệt theo chiều sâu. Nó trả về true nếu có thể tìm thấy một đường đi tăng từ đỉnh $v$, và khi đó, hàm này được coi là đã thực hiện luân phiên phép ghép cặp dọc theo chuỗi tìm thấy.

Bên trong hàm, tất cả các cạnh đi ra từ đỉnh $v$ của phần đầu tiên được quét, và sau đó điều sau được kiểm tra: nếu cạnh này dẫn đến một đỉnh không bão hòa $to$, hoặc nếu đỉnh này $to$ đã bão hòa, nhưng có thể tìm thấy một chuỗi tăng bằng cách bắt đầu đệ quy từ mt[$to$], thì chúng ta nói rằng chúng ta đã tìm thấy một đường đi tăng, và trước khi trả về từ hàm với kết quả true, chúng ta luân phiên cạnh hiện tại: chúng ta chuyển hướng cạnh liền kề với $to$ đến đỉnh $v$.

Chương trình chính trước tiên chỉ ra rằng phép ghép cặp hiện tại là trống (danh sách mt được điền bằng các số $-1$). Sau đó, đỉnh $v$ của phần đầu tiên được tìm kiếm bởi try_kuhn, và một quá trình duyệt theo chiều sâu được bắt đầu từ đó, với mảng used được đặt về 0 trước đó.

Điều đáng chú ý là kích thước của phép ghép cặp rất dễ có được, đó là số lượng các lệnh gọi try_kuhn trong chương trình chính đã trả về kết quả true. Bản thân phép ghép cặp lớn nhất mong muốn được lưu trong mảng mt.

```
int n, k;
vector<vector<int>> g;
```

```
vector<int> mt;
vector<bool> used;

bool try_kuhn(int v) {
    if (used[v])
        return false;
    used[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || try_kuhn(mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    //... reading the graph ...

    mt.assign(k, -1);
    for (int v = 0; v < n; ++v) {
        used.assign(n, false);
        try_kuhn(v);
    }

    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

Chúng ta lặp lại một lần nữa rằng thuật toán Kuhn rất dễ triển khai theo cách mà nó hoạt động trên các đồ thị được biết là hai phía, nhưng việc chia rõ ràng chúng thành hai phần chưa được đưa ra. Trong trường hợp này, cần phải từ bỏ sự phân chia thuận tiện thành hai phần và lưu trữ tất cả thông tin cho tất cả các đỉnh của đồ thị. Đối với điều này, một mảng danh sách $g$ hiện được chỉ định không chỉ cho các đỉnh của phần đầu tiên, mà cho tất cả các đỉnh của đồ thị (tất nhiên, bây giờ các đỉnh của cả hai phần được đánh số theo một cách đánh số chung - từ 1 đến $n$). Các mảng mt và used hiện cũng được xác định cho các đỉnh của cả hai phần và, theo đó, chúng cần được giữ ở trạng thái này.

### Triển Khai Cải Tiến

Hãy sửa đổi thuật toán như sau. Trước vòng lặp chính của thuật toán, chúng ta sẽ tìm một phép ghép cặp tùy ý bằng một số thuật toán đơn giản (một thuật toán heuristic đơn giản), và chỉ sau đó chúng ta sẽ thực hiện một vòng lặp với các lệnh gọi đến hàm try_kuhn(), hàm này sẽ cải thiện phép ghép cặp này.

Do đó, thuật toán sẽ hoạt động nhanh hơn đáng kể trên các đồ thị ngẫu nhiên—vì trong hầu hết các đồ thị, ta có thể dễ dàng tìm thấy một phép ghép cặp có kích thước đủ lớn bằng cách sử dụng heuristic, và sau đó cải thiện phép ghép cặp tìm thấy thành lớn nhất bằng thuật toán Kuhn thông thường.

Nhờ vậy, chúng ta sẽ tiết kiệm được chi phí khi khởi chạy một quá trình duyệt theo chiều sâu từ những đỉnh đã được bao gồm vào phép ghép cặp hiện tại bằng heuristic.

Ví dụ: bạn có thể chỉ cần lặp lại trên tất cả các đỉnh của phần đầu tiên và cho mỗi đỉnh trong số chúng, tìm một cạnh tùy ý có thể được thêm vào phép ghép cặp và thêm nó. Ngay cả một heuristic đơn giản như vậy cũng có thể tăng tốc thuật toán Kuhn lên nhiều lần.

Xin lưu ý rằng vòng lặp chính sẽ phải được sửa đổi một chút. Vì khi gọi hàm try_kuhn trong vòng lặp chính, ta giả định rằng đỉnh hiện tại chưa được bao gồm trong phép ghép cặp, do đó cần thêm một kiểm tra thích hợp.

Trong quá trình triển khai, chỉ mã trong hàm main() sẽ thay đổi:

```
int main() {
    // ... reading the graph ...

    mt.assign(k, -1);
    vector<bool> used1(n, false);
    for (int v = 0; v < n; ++v) {
        for (int to : g[v]) {
            if (mt[to] == -1) {
                mt[to] = v;
                used1[v] = true;
                break;
            }
        }
    }
    for (int v = 0; v < n; ++v) {
        if (used1[v])
            continue;
        used.assign(n, false);
        try_kuhn(v);
    }
```

```
    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

Một heuristic tốt khác như sau. Tại mỗi bước, nó sẽ tìm kiếm đỉnh có bậc nhỏ nhất (nhưng không cô lập), chọn bất kỳ cạnh nào từ nó và thêm nó vào phép ghép cặp, sau đó loại bỏ cả hai đỉnh này với tất cả các cạnh liên kết khỏi đồ thị. Lòng tham như vậy hoạt động rất tốt trên các đồ thị ngẫu nhiên; trong nhiều trường hợp, nó thậm chí còn xây dựng phép ghép cặp lớn nhất (mặc dù có một trường hợp thử nghiệm chống lại nó, trên đó nó sẽ tìm thấy một phép ghép cặp nhỏ hơn nhiều so với lớn nhất).

### 3.39.4 Ghi Chú

- Thuật toán Kuhn là một chương trình con trong thuật toán Hungary, còn được gọi là thuật toán Kuhn-Munkres.
- Thuật toán Kuhn chạy trong thời gian $O(nm)$. Nói chung, nó rất đơn giản để triển khai, tuy nhiên, các thuật toán hiệu quả hơn tồn tại cho bài toán ghép cặp hai phía lớn nhất - chẳng hạn như thuật toán Hopcroft-Karp-Karzanov, chạy trong thời gian $O(\sqrt{n}m)$.
- Bài toán phủ đỉnh tối thiểu là NP-khó đối với các đồ thị tổng quát. Tuy nhiên, định lý Kőnig cho rằng, đối với các đồ thị hai phía, bản số của phép ghép cặp lớn nhất bằng bản số của phủ đỉnh tối thiểu. Do đó, chúng ta có thể sử dụng các thuật toán ghép cặp hai phía lớn nhất để giải bài toán phủ đỉnh tối thiểu trong thời gian đa thức cho các đồ thị hai phía.

## 3.40 Thuật Toán Hungary để Giải Bài Toán Gán Việc

### 3.40.1 Phát Biểu Bài Toán Gán Việc

Có một số công thức tiêu chuẩn của bài toán gán việc (tất cả đều về cơ bản là tương đương). Dưới đây là một số trong số chúng:

- Có $n$ công việc và $n$ công nhân. Mỗi công nhân chỉ định số tiền họ mong đợi cho một công việc cụ thể. Mỗi công nhân chỉ có thể được giao cho một công việc. Mục tiêu là giao công việc cho công nhân theo cách giảm thiểu tổng chi phí.
- Cho một ma trận $n \times n$ $A$, nhiệm vụ là chọn một số từ mỗi hàng sao cho chính xác một số được chọn từ mỗi cột và tổng của các số được chọn là nhỏ nhất.

- Cho một ma trận $n \times n$ $A$, nhiệm vụ là tìm một hoán vị $p$ có độ dài $n$ sao cho giá trị $\sum A[i]\,[p[i]]$ là nhỏ nhất.
- Xét một đồ thị hai phía đầy đủ với $n$ đỉnh trên mỗi phần, trong đó mỗi cạnh được gán một trọng số. Mục tiêu là tìm một phép ghép cặp hoàn hảo với tổng trọng số tối thiểu.

Điều quan trọng cần lưu ý là tất cả các kịch bản trên đều là các bài toán "vuông", có nghĩa là cả hai chiều luôn bằng $n$. Trong thực tế, các công thức "chữ nhật" tương tự thường gặp phải, trong đó $n$ không bằng $m$ và nhiệm vụ là chọn $\min(n, m)$ phần tử. Tuy nhiên, có thể quan sát thấy rằng một bài toán "chữ nhật" luôn có thể được chuyển đổi thành một bài toán "vuông" bằng cách thêm các hàng hoặc cột với các giá trị tương ứng là không hoặc vô cùng.

Chúng ta cũng lưu ý rằng bằng cách tương tự với việc tìm kiếm một giải pháp tối thiểu, người ta cũng có thể đặt ra bài toán tìm kiếm một giải pháp tối đa. Tuy nhiên, hai bài toán này tương đương với nhau: chỉ cần nhân tất cả các trọng số với $-1$.

## 3.40.2 Thuật Toán Hungary

### Tham Khảo Lịch Sử

Thuật toán được phát triển và công bố bởi Harold Kuhn vào năm 1955. Bản thân Kuhn đã đặt cho nó cái tên "Hungary" vì nó dựa trên công trình trước đó của các nhà toán học người Hungary Dénes Kőnig và Jenő Egerváry. Năm 1957, James Munkres đã chỉ ra rằng thuật toán này chạy trong thời gian đa thức (một cách chặt chẽ), độc lập với chi phí. Do đó, trong văn học, thuật toán này được biết đến không chỉ là "Hungary", mà còn là "thuật toán Kuhn-Mankres" hoặc "thuật toán Mankres". Tuy nhiên, gần đây người ta đã phát hiện ra vào năm 2006 rằng thuật toán tương tự đã được phát minh một thế kỷ trước Kuhn bởi nhà toán học người Đức Carl Gustav Jacobi. Công trình của ông, Về nghiên cứu thứ tự của một hệ phương trình vi phân thường tùy ý, được xuất bản truy tặng vào năm 1890, chứa, trong số các phát hiện khác, một thuật toán đa thức để giải bài toán gán việc. Thật không may, vì ấn phẩm bằng tiếng Latinh, nó đã không được chú ý trong giới toán học.

Cũng đáng chú ý rằng thuật toán ban đầu của Kuhn có độ phức tạp tiệm cận là $\mathcal{O}(n^4)$ và chỉ sau đó Jack Edmonds và Richard Karp (và độc lập Tomizawa) đã chỉ ra cách cải thiện nó thành độ phức tạp tiệm cận là $\mathcal{O}(n^3)$.

### Thuật Toán $\mathcal{O}(n^4)$

Để tránh sự mơ hồ, chúng ta lưu ý ngay rằng chúng ta chủ yếu quan tâm đến bài toán gán việc trong một công thức ma trận (tức là, cho một ma trận $A$, bạn cần chọn $n$ ô từ

đó nằm trên các hàng và cột khác nhau). Chúng ta lập chỉ mục các mảng bắt đầu từ 1, tức là, ví dụ: một ma trận $A$ có các chỉ số $A[1 \ldots n][1 \ldots n]$.

Chúng ta cũng sẽ giả định rằng tất cả các số trong ma trận A là không âm (nếu không phải như vậy, bạn luôn có thể làm cho ma trận không âm bằng cách thêm một hằng số vào tất cả các số).

Hãy gọi một thế năng hai mảng số tùy ý $u[1 \ldots n]$ và $v[1 \ldots n]$, sao cho điều kiện sau được thỏa mãn:

$$u[i] + v[j] \leq A[i][j], \quad i = 1 \ldots n,\ j = 1 \ldots n$$

(Như bạn có thể thấy, $u[i]$ tương ứng với hàng thứ $i$ và $v[j]$ tương ứng với cột thứ $j$ của ma trận).

Hãy gọi giá trị $f$ của thế năng là tổng các phần tử của nó:

$$f = \sum_{i=1}^{n} u[i] + \sum_{j=1}^{n} v[j].$$

Một mặt, rất dễ thấy rằng chi phí của giải pháp mong muốn *sol* không nhỏ hơn giá trị của bất kỳ thế năng nào.

**Bổ đề.** $sol \geq f$.

**Chứng minh** Giải pháp mong muốn của bài toán bao gồm $n$ ô của ma trận $A$, do đó $u[i] + v[j] \leq A[i][j]$ cho mỗi ô trong số chúng. Vì tất cả các phần tử trong *sol* nằm trên các hàng và cột khác nhau, nên việc tổng hợp các bất đẳng thức này trên tất cả các $A[i][j]$ đã chọn, bạn sẽ nhận được $f$ ở phía bên trái của bất đẳng thức và *sol* ở phía bên phải.

Mặt khác, hóa ra luôn có một giải pháp và một thế năng biến bất đẳng thức này thành đẳng thức. Thuật toán Hungary được mô tả dưới đây sẽ là một chứng minh mang tính xây dựng của thực tế này. Hiện tại, chúng ta hãy chú ý đến thực tế là nếu bất kỳ giải pháp nào có chi phí bằng bất kỳ thế năng nào, thì giải pháp này là tối ưu.

Hãy cố định một số thế năng. Hãy gọi một cạnh $(i, j)$ là cứng nếu $u[i] + v[j] = A[i][j]$.

Nhắc lại một công thức thay thế của bài toán gán việc, sử dụng một đồ thị hai phía. Ký hiệu bằng $H$ một đồ thị hai phía chỉ bao gồm các cạnh cứng. Thuật toán Hungary sẽ duy trì, cho thế năng hiện tại, phép ghép cặp số lượng cạnh tối đa $M$ của đồ thị $H$. Ngay sau khi $M$ chứa $n$ cạnh, thì giải pháp cho bài toán sẽ chỉ là $M$ (sau tất cả, nó sẽ là một giải pháp có chi phí trùng với giá trị của một thế năng).

Hãy tiến hành trực tiếp đến mô tả của thuật toán.

**Bước 1.** Lúc đầu, thế năng được giả định là bằng không ($u[i] = v[i] = 0$ cho tất cả $i$), và phép ghép cặp $M$ được giả định là trống.

**Bước 2.** Hơn nữa, tại mỗi bước của thuật toán, chúng ta cố gắng, mà không thay đổi thế năng, để tăng bản số của phép ghép cặp hiện tại $M$ thêm một (hãy nhớ rằng phép ghép cặp được tìm kiếm trong đồ thị các cạnh cứng $H$). Để làm điều này, Thuật Toán

Kuhn thông thường để tìm phép ghép cặp lớn nhất trong đồ thị hai phía được sử dụng. Chúng ta hãy nhắc lại thuật toán ở đây. Tất cả các cạnh của phép ghép cặp $M$ được định hướng theo hướng từ phần bên phải sang phần bên trái và tất cả các cạnh khác của đồ thị $H$ được định hướng theo hướng ngược lại.

Nhắc lại (từ thuật ngữ tìm kiếm các phép ghép cặp) rằng một đỉnh được gọi là bão hòa nếu một cạnh của phép ghép cặp hiện tại liền kề với nó. Một đỉnh không liền kề với bất kỳ cạnh nào của phép ghép cặp hiện tại được gọi là không bão hòa. Một đường đi có độ dài lẻ, trong đó cạnh đầu tiên không thuộc phép ghép cặp và đối với tất cả các cạnh tiếp theo có một sự luân phiên thuộc về phép ghép cặp (thuộc/không thuộc) - được gọi là một đường đi tăng. Từ tất cả các đỉnh không bão hòa trong phần bên trái, một quá trình duyệt theo chiều sâu hoặc chiều rộng được bắt đầu. Nếu, do kết quả tìm kiếm, có thể đạt đến một đỉnh không bão hòa của phần bên phải, chúng ta đã tìm thấy một đường đi tăng từ phần bên trái sang phần bên phải. Nếu chúng ta bao gồm các cạnh lẻ của đường đi và loại bỏ các cạnh chẵn trong phép ghép cặp (tức là bao gồm cạnh đầu tiên trong phép ghép cặp, loại trừ cạnh thứ hai, bao gồm cạnh thứ ba, v.v.), thì chúng ta sẽ tăng bản số ghép cặp thêm một.

Nếu không có đường đi tăng, thì phép ghép cặp hiện tại $M$ là cực đại trong đồ thị $H$.

**Bước 3.** Nếu tại bước hiện tại, không thể tăng bản số của phép ghép cặp hiện tại, thì một phép tính lại thế năng được thực hiện theo cách mà, tại các bước tiếp theo, sẽ có nhiều cơ hội hơn để tăng phép ghép cặp.

Ký hiệu bằng $Z_1$ tập hợp các đỉnh của phần bên trái đã được truy cập trong quá trình duyệt cuối cùng của thuật toán Kuhn và thông qua $Z_2$ tập hợp các đỉnh đã truy cập của phần bên phải.

Hãy tính giá trị $\Delta$:

$$\Delta = \min_{i \in Z_1,\ j \notin Z_2} A[i][j] - u[i] - v[j].$$

**Bổ đề.** $\Delta > 0$.

**Chứng minh** Bây giờ hãy tính lại thế năng theo cách này:
cho tất cả các đỉnh $i \in Z_1$, thực hiện $u[i] \leftarrow u[i] + \Delta$,
cho tất cả các đỉnh $j \in Z_2$, thực hiện $v[j] \leftarrow v[j] - \Delta$.

**Bổ đề.** Thế năng kết quả vẫn là một thế năng chính xác.

**Chứng minh Bổ đề.** Phép ghép cặp cũ $M$ của các cạnh cứng là hợp lệ, tức là tất cả các cạnh của phép ghép cặp sẽ vẫn cứng.

**Chứng minh Bổ đề.** Sau mỗi lần tính lại thế năng, số lượng đỉnh có thể truy cập bằng quá trình duyệt, tức là $|Z_1| + |Z_2|$, tăng lên một cách nghiêm ngặt.

**Chứng minh** Đầu tiên, lưu ý rằng bất kỳ đỉnh nào có thể truy cập trước khi tính lại, vẫn có thể truy cập được. Thật vậy, nếu một số đỉnh có thể truy cập được, thì có một số đường đi từ các đỉnh có thể truy cập đến nó, bắt đầu từ đỉnh không bão hòa của phần bên trái; vì đối với các cạnh có dạng $(i, j)$, $i \in Z_1$, $j \in Z_2$ tổng $u[i] + v[j]$ không thay đổi, toàn bộ đường đi này sẽ được bảo toàn sau khi thay đổi thế năng. Thứ hai, chúng ta

chỉ ra rằng sau khi tính lại, ít nhất một đỉnh mới sẽ có thể truy cập được. Điều này tuân theo định nghĩa của $\Delta$: cạnh $(i, j)$ mà $\Delta$ đề cập đến sẽ trở nên cứng, vì vậy đỉnh $j$ sẽ có thể truy cập được từ đỉnh $i$.

Do bổ đề cuối cùng, không quá $n$ lần tính lại thế năng có thể xảy ra trước khi một đường đi tăng được tìm thấy và bản số ghép cặp của $M$ được tăng lên. Do đó, sớm hay muộn, một thế năng tương ứng với một phép ghép cặp hoàn hảo $M^*$ sẽ được tìm thấy và $M^*$ sẽ là câu trả lời cho bài toán. Nếu chúng ta nói về độ phức tạp của thuật toán, thì nó là $\mathcal{O}(n^4)$: tổng cộng sẽ có nhiều nhất $n$ lần tăng trong phép ghép cặp, trước mỗi lần trong số đó có không quá $n$ lần tính lại thế năng, mỗi lần trong số đó được thực hiện trong thời gian $\mathcal{O}(n^2)$.

Chúng ta sẽ không đưa ra cách triển khai cho thuật toán $\mathcal{O}(n^4)$ ở đây, vì nó sẽ không ngắn hơn cách triển khai cho thuật toán $\mathcal{O}(n^3)$, được mô tả bên dưới.

### Thuật Toán $\mathcal{O}(n^3)$

Bây giờ chúng ta hãy học cách triển khai cùng một thuật toán trong $\mathcal{O}(n^3)$ (đối với các bài toán hình chữ nhật $n \times m$, $\mathcal{O}(n^2 m)$).

Ý tưởng chính là xem xét các hàng ma trận từng hàng một và không phải tất cả cùng một lúc. Do đó, thuật toán được mô tả ở trên sẽ có dạng sau:

1. Xem xét hàng tiếp theo của ma trận $A$.

2. Trong khi không có đường đi tăng bắt đầu từ hàng này, hãy tính lại thế năng.

3. Ngay sau khi một đường đi tăng được tìm thấy, hãy truyền phép ghép cặp dọc theo nó (do đó bao gồm cạnh cuối cùng trong phép ghép cặp) và khởi động lại từ bước 1 (để xem xét dòng tiếp theo).

Để đạt được độ phức tạp yêu cầu, cần thiết phải triển khai các bước 2-3, được thực hiện cho mỗi hàng của ma trận, trong thời gian $\mathcal{O}(n^2)$ (đối với các bài toán hình chữ nhật trong $\mathcal{O}(nm)$).

Để làm điều này, hãy nhớ lại hai sự kiện đã được chứng minh ở trên:

- Với sự thay đổi về thế năng, các đỉnh có thể truy cập được bằng quá trình duyệt của Kuhn sẽ vẫn có thể truy cập được.

- Tổng cộng, chỉ có thể xảy ra $\mathcal{O}(n)$ lần tính lại thế năng trước khi một đường đi tăng được tìm thấy.

Từ đó tuân theo những ý tưởng chính này cho phép chúng ta đạt được độ phức tạp yêu cầu:

- Để kiểm tra sự hiện diện của một đường đi tăng, không cần phải bắt đầu lại quá trình duyệt của Kuhn sau mỗi lần tính lại thế năng. Thay vào đó, bạn có thể thực hiện quá trình duyệt của Kuhn ở dạng lặp: sau mỗi lần tính lại thế năng, hãy xem xét các cạnh cứng được thêm vào và, nếu các đầu bên trái của chúng có thể truy cập được, hãy đánh dấu các đầu bên phải của chúng cũng có thể truy cập được và tiếp tục quá trình duyệt từ chúng.

Phát triển thêm ý tưởng này, chúng ta có thể trình bày thuật toán như sau: tại mỗi bước của vòng lặp, thế năng được tính lại. Sau đó, một cột đã trở nên có thể truy cập được được xác định (cột này sẽ luôn tồn tại khi các đỉnh có thể truy cập mới xuất hiện sau mỗi lần tính lại thế năng). Nếu cột không bão hòa, một chuỗi tăng được phát hiện. Ngược lại, nếu cột bão hòa, hàng ghép cặp cũng trở nên có thể truy cập được.

Để tính lại nhanh chóng thế năng (nhanh hơn phiên bản ngây thơ $\mathcal{O}(n^2)$), bạn cần duy trì các giá trị cực tiểu phụ trợ cho mỗi cột:

$$minv[j] = \min_{i \in Z_1} A[i][j] - u[i] - v[j].$$

Rất dễ thấy rằng giá trị mong muốn $\Delta$ được biểu thị theo chúng như sau:

$$\Delta = \min_{j \notin Z_2} minv[j].$$

Do đó, việc tìm $\Delta$ bây giờ có thể được thực hiện trong $\mathcal{O}(n)$.

Cần thiết phải cập nhật mảng $minv$ khi các hàng đã truy cập mới xuất hiện. Điều này có thể được thực hiện trong $\mathcal{O}(n)$ cho hàng được thêm vào (điều này cộng lại trên tất cả các hàng thành $\mathcal{O}(n^2)$). Cũng cần thiết phải cập nhật mảng $minv$ khi tính lại thế năng, điều này cũng được thực hiện trong thời gian $\mathcal{O}(n)$ ($minv$ chỉ thay đổi đối với các cột chưa đạt được: cụ thể là, nó giảm đi $\Delta$).

Do đó, thuật toán có dạng sau: trong vòng lặp bên ngoài, chúng ta xem xét các hàng ma trận từng hàng một. Mỗi hàng được xử lý trong thời gian $\mathcal{O}(n^2)$, vì chỉ có thể xảy ra $\mathcal{O}(n)$ lần tính lại thế năng (mỗi lần trong thời gian $\mathcal{O}(n)$) và mảng $minv$ được duy trì trong thời gian $\mathcal{O}(n^2)$; Thuật toán Kuhn sẽ hoạt động trong thời gian $\mathcal{O}(n^2)$ (vì nó được trình bày ở dạng $\mathcal{O}(n)$ lần lặp, mỗi lần trong số đó truy cập một cột mới).

Độ phức tạp kết quả là $\mathcal{O}(n^3)$ hoặc, nếu bài toán là hình chữ nhật, $\mathcal{O}(n^2 m)$.

### 3.40.3 Triển Khai Thuật Toán Hungary

Việc triển khai dưới đây được phát triển bởi Andrey Lopatin vài năm trước. Nó được phân biệt bởi sự ngắn gọn đáng kinh ngạc: toàn bộ thuật toán bao gồm 30 dòng mã.

Việc triển khai tìm một giải pháp cho ma trận hình chữ nhật $A[1 \dots n][1 \dots m]$, trong đó $n \leq m$. Ma trận này là 1-based để thuận tiện và ngắn gọn mã: việc triển khai này giới

thiệu một hàng số không giả và một cột số không, cho phép chúng ta viết nhiều chu kỳ ở dạng chung, mà không cần kiểm tra thêm.

Các mảng $u[0 \ldots n]$ và $v[0 \ldots m]$ lưu trữ thế năng. Ban đầu, chúng được đặt thành không, điều này phù hợp với một ma trận các hàng số không (Lưu ý rằng không quan trọng đối với việc triển khai này liệu ma trận $A$ có chứa các số âm hay không).

Mảng $p[0 \ldots m]$ chứa một phép ghép cặp: đối với mỗi cột $j = 1 \ldots m$, nó lưu trữ số $p[j]$ của hàng được chọn (hoặc 0 nếu chưa có gì được chọn). Để thuận tiện cho việc triển khai, $p[0]$ được giả định là bằng số của hàng hiện tại.

Mảng $minv[1 \ldots m]$ chứa, cho mỗi cột $j$, các giá trị cực tiểu phụ trợ cần thiết để tính lại nhanh chóng thế năng, như được mô tả ở trên.

Mảng $way[1 \ldots m]$ chứa thông tin về nơi các giá trị cực tiểu này đạt được để chúng ta có thể xây dựng lại đường đi tăng sau này. Lưu ý rằng, để xây dựng lại đường đi, chỉ cần lưu trữ các giá trị cột, vì số hàng có thể được lấy từ phép ghép cặp (tức là từ mảng $p$). Do đó, $way[j]$, cho mỗi cột $j$, chứa số của cột trước đó trong đường đi (hoặc 0 nếu không có).

Bản thân thuật toán là một vòng lặp bên ngoài thông qua các hàng của ma trận, bên trong đó hàng thứ $i$ của ma trận được xem xét. Vòng lặp do-while đầu tiên chạy cho đến khi một cột trống $j0$ được tìm thấy. Mỗi lần lặp của vòng lặp đánh dấu đã truy cập một cột mới với số $j0$ (được tính ở lần lặp cuối cùng; và ban đầu bằng không - tức là chúng ta bắt đầu từ một cột giả), cũng như một hàng mới $i0$ - liền kề với nó trong phép ghép cặp (tức là $p[j0]$; và ban đầu khi $j0 = 0$ hàng thứ $i$ được lấy). Do sự xuất hiện của một hàng đã truy cập mới $i0$, bạn cần tính lại mảng $minv$ và $\Delta$ cho phù hợp. Nếu $\Delta$ được cập nhật, thì cột $j1$ trở thành giá trị cực tiểu đã đạt được (lưu ý rằng với một triển khai như vậy, $\Delta$ có thể hóa ra bằng không, có nghĩa là không thể thay đổi thế năng ở bước hiện tại: đã có một cột có thể truy cập mới). Sau đó, thế năng và mảng $minv$ được tính lại. Vào cuối vòng lặp "do-while", chúng ta tìm thấy một đường đi tăng kết thúc ở một cột $j0$ có thể được "mở ra" bằng cách sử dụng mảng tổ tiên $way$.

Hằng số INF là "vô cùng", tức là một số nào đó, rõ ràng lớn hơn tất cả các số có thể có trong ma trận đầu vào $A$.

```
vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
for (int i=1; i<=n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<int> minv (m+1, INF);
    vector<bool> used (m+1, false);
    do {
        used[j0] = true;
        int i0 = p[j0],  delta = INF,  j1;
        for (int j=1; j<=m; ++j)
            if (!used[j]) {
```

```
                int cur = A[i0][j]-u[i0]-v[j];
                if (cur < minv[j])
                    minv[j] = cur,  way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j],  j1 = j;
            }
        for (int j=0; j<=m; ++j)
            if (used[j])
                u[p[j]] += delta,  v[j] -= delta;
            else
                minv[j] -= delta;
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    } while (j0);
}
```

Để khôi phục câu trả lời ở dạng quen thuộc hơn, tức là tìm cho mỗi hàng $i = 1 \dots n$ số $ans[i]$ của cột được chọn trong đó, có thể được thực hiện như sau:

```
vector<int> ans (n+1);
for (int j=1; j<=m; ++j)
    ans[p[j]] = j;
```

Chi phí của phép ghép cặp có thể đơn giản được lấy làm thế năng của cột số không (lấy với dấu ngược lại). Thật vậy, như bạn có thể thấy từ mã, $-v[0]$ chứa tổng của tất cả các giá trị của $\Delta$, tức là tổng thay đổi về thế năng. Mặc dù một số giá trị $u[i]$ và $v[j]$ có thể thay đổi cùng một lúc, nhưng tổng thay đổi về thế năng chính xác bằng $\Delta$, vì cho đến khi có một đường đi tăng, số lượng hàng có thể truy cập chính xác nhiều hơn một so với số lượng cột có thể truy cập (chỉ hàng hiện tại $i$ không có "cặp" ở dạng một cột đã truy cập):

```
int cost = -v[0];
```

### 3.40.4 Kết Nối với Thuật Toán Đường Đi Ngắn Nhất Liên Tiếp

Thuật toán Hungary có thể được xem như là Thuật Toán Đường Đi Ngắn Nhất Liên Tiếp, được điều chỉnh cho bài toán gán việc. Không đi sâu vào chi tiết, hãy cung cấp một trực giác liên quan đến kết nối giữa chúng.

Thuật toán Đường Đi Liên Tiếp sử dụng một phiên bản sửa đổi của thuật toán Johnson làm kỹ thuật đánh giá lại. Cái này được chia thành bốn bước:

1. Sử dụng thuật toán Bellman-Ford, bắt đầu từ đích $s$ và, cho mỗi nút, tìm trọng số tối thiểu $h(v)$ của một đường đi từ $s$ đến $v$.

2. Đối với mọi bước của thuật toán chính:

   (a) Đánh giá lại các cạnh của đồ thị gốc theo cách này: $w(u, v) \leftarrow w(u, v) + h(u) - h(v)$.

   (b) Sử dụng thuật toán Dijkstra để tìm đồ thị con đường đi ngắn nhất của mạng gốc.

   (c) Cập nhật thế năng cho lần lặp tiếp theo.

Với mô tả này, chúng ta có thể quan sát thấy rằng có một sự tương đồng mạnh mẽ giữa $h(v)$ và thế năng: có thể kiểm tra rằng chúng bằng nhau với một độ lệch không đổi. Ngoài ra, có thể chỉ ra rằng, sau khi đánh giá lại, tập hợp tất cả các cạnh có trọng số bằng không đại diện cho đồ thị con đường đi ngắn nhất nơi thuật toán chính cố gắng tăng luồng. Điều này cũng xảy ra trong thuật toán Hungary: chúng ta tạo một đồ thị con được tạo thành từ các cạnh cứng (những cạnh mà số lượng $A[i][j] - u[i] - v[j]$ bằng không) và chúng ta cố gắng tăng kích thước của phép ghép cặp.

Trong bước 4, tất cả $h(v)$ được cập nhật: mỗi khi chúng ta sửa đổi mạng luồng, chúng ta nên đảm bảo rằng khoảng cách từ nguồn là chính xác (nếu không, trong lần lặp tiếp theo, thuật toán Dijkstra có thể không thành công). Điều này nghe có vẻ giống như bản cập nhật được thực hiện trên thế năng, nhưng trong trường hợp này, chúng không được tăng lên một cách đồng đều.

Để làm sâu sắc thêm sự hiểu biết về thế năng, hãy tham khảo bài viết này.

### 3.40.5 Ví Dụ Bài Toán

Dưới đây là một vài ví dụ liên quan đến bài toán gán việc, từ rất tầm thường đến các nhiệm vụ ít rõ ràng hơn:

- Cho một đồ thị hai phía, cần tìm trong đó phép ghép cặp lớn nhất với trọng số tối thiểu (tức là, trước hết, kích thước của phép ghép cặp được tối đa hóa và thứ hai, chi phí của nó được giảm thiểu). Để giải quyết nó, chúng ta chỉ cần xây dựng một bài toán gán việc, đặt số "vô cùng" vào vị trí của các cạnh bị thiếu. Sau đó, chúng ta giải bài toán bằng thuật toán Hungary và loại bỏ các cạnh có trọng số vô cùng khỏi câu trả lời (chúng có thể nhập câu trả lời nếu bài toán không có giải pháp ở dạng một phép ghép cặp hoàn hảo).

- Cho một đồ thị hai phía, cần tìm trong đó phép ghép cặp lớn nhất với trọng số tối đa. Giải pháp lại rõ ràng, tất cả các trọng số phải được nhân với trừ một.

- Nhiệm vụ phát hiện các đối tượng chuyển động trong hình ảnh: hai hình ảnh đã được chụp, do đó thu được hai tập hợp tọa độ. Cần thiết để tương quan các đối tượng trong hình ảnh thứ nhất và thứ hai, tức là xác định cho mỗi điểm của hình ảnh thứ hai, điểm nào của hình ảnh thứ nhất mà nó tương ứng. Trong trường hợp này, cần giảm thiểu tổng khoảng cách giữa các điểm được so sánh (tức là chúng ta đang tìm kiếm một giải pháp trong đó các đối tượng đã đi theo con đường ngắn nhất trong tổng số). Để giải quyết, chúng ta chỉ cần xây dựng và giải một bài toán gán việc, trong đó trọng số của các cạnh là khoảng cách Euclid giữa các điểm.

- Nhiệm vụ phát hiện các đối tượng chuyển động bằng bộ định vị: có hai bộ định vị không thể xác định vị trí của một đối tượng trong không gian, mà chỉ có thể xác định hướng của nó. Cả hai bộ định vị (nằm ở các điểm khác nhau) nhận được thông tin dưới dạng $n$ hướng như vậy. Nhiệm vụ đặt ra là xác định vị trí của các đối tượng, tức là tìm vị trí dự kiến của các đối tượng và các cặp hướng tương ứng của chúng sao cho tổng khoảng cách từ các đối tượng đến các tia hướng được giảm thiểu.

  Giải pháp: một lần nữa, chúng ta chỉ cần xây dựng và giải bài toán gán việc, trong đó các đỉnh của phần bên trái là $n$ hướng từ bộ định vị thứ nhất, các đỉnh của phần bên phải là $n$ hướng từ bộ định vị thứ hai và trọng số của các cạnh là khoảng cách giữa các tia tương ứng.

- Che phủ một đồ thị có hướng không chu trình bằng các đường đi: cho một đồ thị có hướng không chu trình, cần tìm số lượng đường đi nhỏ nhất (nếu bằng nhau, với tổng trọng số nhỏ nhất) sao cho mỗi đỉnh của đồ thị nằm trên chính xác một đường đi. Giải pháp là xây dựng đồ thị hai phía tương ứng từ đồ thị đã cho và tìm phép ghép cặp lớn nhất có trọng số tối thiểu trong đó. Xem bài viết riêng để biết thêm chi tiết.

- Sách tô màu cây. Cho một cây trong đó mỗi đỉnh, ngoại trừ lá, có chính xác $k-1$ con. Cần thiết để chọn cho mỗi đỉnh một trong $k$ màu có sẵn sao cho không có hai đỉnh liền kề nào có cùng màu. Ngoài ra, đối với mỗi đỉnh và mỗi màu, chi phí tô màu đỉnh này bằng màu này được biết và cần thiết để giảm thiểu tổng chi phí. Để giải quyết bài toán này, chúng ta sử dụng quy hoạch động. Cụ thể, hãy học cách tính giá trị $d[v][c]$, trong đó $v$ là số đỉnh, $c$ là số màu và bản thân giá trị $d[v][c]$ là chi phí tối thiểu cần thiết để tô màu tất cả các đỉnh trong cây con gốc tại $v$ và bản thân đỉnh $v$ bằng màu $c$. Để tính một giá trị như vậy $d[v][c]$, cần thiết phải phân phối $k-1$ màu còn lại cho các con của đỉnh $v$ và vì điều này, cần thiết phải xây dựng và giải bài toán gán việc (trong đó các đỉnh của phần bên trái là màu sắc, các đỉnh của phần bên phải là con và trọng số của các cạnh là các giá trị tương ứng của

$d$). Do đó, mỗi giá trị $d[v][c]$ được tính bằng cách sử dụng giải pháp của bài toán gán việc, cuối cùng cho tiệm cận $\mathcal{O}(nk^4)$.

- Nếu, trong bài toán gán việc, trọng số không nằm trên các cạnh, mà trên các đỉnh và chỉ trên các đỉnh của cùng một phần, thì không cần thiết phải sử dụng thuật toán Hungary: chỉ cần sắp xếp các đỉnh theo trọng số và chạy thuật toán Kuhn thông thường (để biết thêm chi tiết, hãy xem một bài viết riêng).

- Xem xét trường hợp đặc biệt sau. Cho mỗi đỉnh của phần bên trái được gán một số $\alpha[i]$ và mỗi đỉnh của phần bên phải $\beta[j]$. Cho trọng số của bất kỳ cạnh nào $(i, j)$ bằng $\alpha[i] \cdot \beta[j]$ (các số $\alpha[i]$ và $\beta[j]$ đã biết). Giải bài toán gán việc. Để giải quyết nó mà không cần thuật toán Hungary, trước tiên chúng ta xem xét trường hợp khi cả hai phần có hai đỉnh. Trong trường hợp này, như bạn có thể dễ dàng thấy, tốt hơn là kết nối các đỉnh theo thứ tự ngược lại: kết nối đỉnh có $\alpha[i]$ nhỏ hơn với đỉnh có $\beta[j]$ lớn hơn. Quy tắc này có thể dễ dàng được tổng quát hóa cho một số lượng đỉnh tùy ý: bạn cần sắp xếp các đỉnh của phần đầu tiên theo thứ tự tăng dần của các giá trị $\alpha[i]$, phần thứ hai theo thứ tự giảm dần của các giá trị $\beta[j]$ và kết nối các đỉnh theo cặp theo thứ tự đó. Do đó, chúng ta thu được một giải pháp với độ phức tạp là $\mathcal{O}(n \log n)$.

- Bài Toán Thế Năng. Cho một ma trận $A[1 \ldots n][1 \ldots m]$, cần tìm hai mảng $u[1 \ldots n]$ và $v[1 \ldots m]$ sao cho, đối với bất kỳ $i$ và $j$ nào, $u[i] + v[j] \leq a[i][j]$ và tổng các phần tử của mảng $u$ và $v$ là lớn nhất. Biết thuật toán Hungary, giải pháp cho bài toán này sẽ không khó: thuật toán Hungary chỉ tìm một thế năng $u, v$ thỏa mãn điều kiện của bài toán. Mặt khác, nếu không có kiến thức về thuật toán Hungary, thì dường như gần như không thể giải quyết bài toán này.

**Nhận xét**

Nhiệm vụ này còn được gọi là bài toán đối ngẫu của bài toán gán việc: giảm thiểu tổng chi phí của việc gán tương đương với việc tối đa hóa tổng các thế năng.

### 3.40.6 Tài Liệu Tham Khảo


- Ravindra Ahuja, Thomas Magnanti, James Orlin. Network Flows [1993]

- Harold Kuhn. The Hungarian Method for the Assignment Problem [1955]

- James Munkres. Algorithms for Assignment and Transportation Problems [1957]

# 3.41 Sắp Xếp Tô-pô (Topological Sorting)

## 3.41.1 Bài Toán

Bạn được cho một đồ thị có hướng với $n$ đỉnh và $m$ cạnh. Bạn cần tìm một thứ tự của các đỉnh sao cho mọi cạnh dẫn từ đỉnh có chỉ số nhỏ hơn đến đỉnh có chỉ số lớn hơn.

Nói cách khác, bạn muốn tìm một hoán vị của các đỉnh (thứ tự tô-pô) tương ứng với thứ tự được xác định bởi tất cả các cạnh của đồ thị.

**Ví dụ:** Một đồ thị có hướng cùng với một thứ tự tô-pô của nó:

- Thứ tự tô-pô có thể không duy nhất (ví dụ, nếu tồn tại ba đỉnh $a$, $b$, $c$ mà có các đường đi từ $a$ đến $b$ và từ $a$ đến $c$, nhưng không có đường đi từ $b$ đến $c$ hoặc từ $c$ đến $b$). Đồ thị ví dụ cũng có nhiều thứ tự tô-pô, một thứ tự tô-pô khác là:

**Lưu ý:** Một thứ tự tô-pô có thể không tồn tại. Nó chỉ tồn tại nếu đồ thị có hướng không chứa chu trình. Nếu không, sẽ có một mâu thuẫn: nếu có một chu trình chứa các đỉnh $a$ và $b$, thì $a$ cần có chỉ số nhỏ hơn $b$ (vì bạn có thể đi từ $a$ đến $b$) và cũng lớn hơn (vì bạn có thể đi từ $b$ đến $a$). Thuật toán được mô tả trong bài viết này cũng cho thấy bằng cách xây dựng rằng mọi đồ thị có hướng không chu trình đều chứa ít nhất một thứ tự tô-pô.

Một bài toán phổ biến trong đó sắp xếp tô-pô xuất hiện là như sau. Có $n$ biến với các giá trị chưa biết. Đối với một số biến, chúng ta biết rằng một biến nhỏ hơn biến khác. Bạn cần kiểm tra xem các ràng buộc này có mâu thuẫn hay không, và nếu không, hãy xuất các biến theo thứ tự tăng dần (nếu có nhiều câu trả lời, hãy xuất bất kỳ câu trả lời nào). Dễ dàng nhận thấy rằng đây chính là bài toán tìm thứ tự tô-pô của một đồ thị với $n$ đỉnh.

## 3.41.2 Thuật Toán

Để giải bài toán này, chúng ta sẽ sử dụng thuật toán tìm kiếm theo chiều sâu (DFS).

Giả sử rằng đồ thị là không chu trình. Tìm kiếm theo chiều sâu làm gì?

Khi bắt đầu từ một đỉnh $v$, DFS cố gắng duyệt qua tất cả các cạnh đi ra từ $v$. Nó dừng lại ở các cạnh mà các đỉnh cuối đã được truy cập trước đó và tiếp tục duyệt qua các cạnh còn lại một cách đệ quy tại các đỉnh cuối của chúng.

Do đó, tại thời điểm lời gọi hàm dfs($v$) kết thúc, tất cả các đỉnh có thể truy cập từ $v$ đã được truy cập trực tiếp (qua một cạnh) hoặc gián tiếp bởi tìm kiếm.

Hãy thêm đỉnh $v$ vào một danh sách khi chúng ta hoàn thành dfs($v$). Vì tất cả các đỉnh có thể truy cập đã được truy cập, chúng sẽ đã có trong danh sách khi chúng ta thêm $v$. Hãy làm điều này cho mọi đỉnh trong đồ thị, với một hoặc nhiều lần chạy tìm kiếm theo chiều sâu. Đối với mọi cạnh có hướng $v \to u$ trong đồ thị, $u$ sẽ xuất hiện sớm hơn trong danh sách này so với $v$, vì $u$ có thể truy cập từ $v$. Vì vậy, nếu chúng ta chỉ cần gán

nhãn các đỉnh trong danh sách này với $n-1, n-2, \ldots, 1, 0$, chúng ta đã tìm thấy một thứ tự tô-pô của đồ thị. Nói cách khác, danh sách này đại diện cho thứ tự tô-pô ngược.

Những giải thích này cũng có thể được trình bày dưới dạng thời gian thoát của thuật toán DFS. Thời gian thoát cho đỉnh $v$ là thời gian tại đó lời gọi hàm dfs($v$) kết thúc (các thời gian có thể được đánh số từ 0 đến $n-1$). Dễ dàng hiểu rằng thời gian thoát của bất kỳ đỉnh $v$ nào luôn lớn hơn thời gian thoát của bất kỳ đỉnh nào có thể truy cập từ nó (vì chúng được truy cập trước khi gọi dfs($v$) hoặc trong khi gọi). Do đó, thứ tự tô-pô mong muốn là các đỉnh theo thứ tự giảm dần của thời gian thoát.

### 3.41.3 Triển Khai

Dưới đây là một triển khai giả định rằng đồ thị là không chu trình, tức là thứ tự tô-pô mong muốn tồn tại. Nếu cần, bạn có thể dễ dàng kiểm tra rằng đồ thị là không chu trình, như được mô tả trong bài viết về tìm kiếm theo chiều sâu.

```
int n; // số lượng đỉnh
vector<vector<int>> adj; // danh sách kề của đồ thị
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

Hàm chính của giải pháp là `topological_sort`, khởi tạo các biến DFS, khởi chạy DFS và nhận câu trả lời trong vector `ans`. Đáng chú ý rằng khi đồ thị không phải là không chu trình, kết quả của `topological_sort` vẫn sẽ có ý nghĩa ở một mức độ nào đó, theo nghĩa rằng nếu một đỉnh $u$ có thể truy cập từ đỉnh $v$, nhưng không ngược lại, thì đỉnh $v$ sẽ luôn xuất hiện trước trong mảng kết quả. Tính chất này của triển khai được cung cấp được sử dụng trong thuật toán Kosaraju để trích xuất các thành phần liên thông mạnh và sắp xếp tô-pô của chúng trong một đồ thị có hướng chứa chu trình.

## 3.42 Kết Nối Cạnh / Kết Nối Đỉnh

### 3.42.1 Định Nghĩa

Cho một đồ thị vô hướng $G$ với $n$ đỉnh và $m$ cạnh. Cả kết nối cạnh và kết nối đỉnh đều là các đặc trưng mô tả đồ thị.

#### Kết Nối Cạnh

Kết nối cạnh $\lambda$ của đồ thị $G$ là số lượng cạnh tối thiểu cần xóa để đồ thị $G$ bị ngắt kết nối.

Ví dụ, một đồ thị đã bị ngắt kết nối có kết nối cạnh bằng 0, một đồ thị liên thông với ít nhất một cầu có kết nối cạnh bằng 1, và một đồ thị liên thông không có cầu có kết nối cạnh ít nhất là 2.

Chúng ta nói rằng một tập hợp $S$ các cạnh tách rời các đỉnh $s$ và $t$, nếu, sau khi xóa tất cả các cạnh trong $S$ khỏi đồ thị $G$, các đỉnh $s$ và $t$ nằm trong các thành phần liên thông khác nhau.

Rõ ràng, kết nối cạnh của một đồ thị bằng kích thước nhỏ nhất của một tập hợp như vậy tách rời hai đỉnh $s$ và $t$, được lấy trong tất cả các cặp $(s, t)$ có thể.

#### Kết Nối Đỉnh

Kết nối đỉnh $\kappa$ của đồ thị $G$ là số lượng đỉnh tối thiểu cần xóa để đồ thị $G$ bị ngắt kết nối.

Ví dụ, một đồ thị đã bị ngắt kết nối có kết nối đỉnh bằng 0, và một đồ thị liên thông với một điểm khớp có kết nối đỉnh bằng 1. Chúng ta định nghĩa rằng một đồ thị đầy đủ có kết nối đỉnh bằng $n - 1$. Đối với tất cả các đồ thị khác, kết nối đỉnh không vượt quá $n - 2$, vì bạn có thể tìm một cặp đỉnh không được kết nối bởi một cạnh và xóa tất cả $n - 2$ đỉnh còn lại.

Chúng ta nói rằng một tập hợp $T$ các đỉnh tách rời các đỉnh $s$ và $t$, nếu, sau khi xóa tất cả các đỉnh trong $T$ khỏi đồ thị $G$, các đỉnh nằm trong các thành phần liên thông khác nhau.

Rõ ràng, kết nối đỉnh của một đồ thị bằng kích thước nhỏ nhất của một tập hợp như vậy tách rời hai đỉnh $s$ và $t$, được lấy trong tất cả các cặp $(s, t)$ có thể.

### 3.42.2 Tính Chất

#### Bất Đẳng Thức Whitney

Bất đẳng thức Whitney (1932) đưa ra mối quan hệ giữa kết nối cạnh $\lambda$, kết nối đỉnh $\kappa$, và bậc nhỏ nhất của bất kỳ đỉnh nào trong đồ thị $\delta$:

$$\kappa \leq \lambda \leq \delta$$

Trực quan, nếu chúng ta có một tập hợp các cạnh có kích thước $\lambda$ làm đồ thị bị ngắt kết nối, chúng ta có thể chọn một trong mỗi điểm cuối và tạo một tập hợp các đỉnh cũng làm đồ thị bị ngắt kết nối. Và tập hợp này có kích thước $\leq \lambda$.

Nếu chúng ta chọn một đỉnh và bậc nhỏ nhất $\delta$, và xóa tất cả các cạnh kết nối với nó, thì chúng ta cũng sẽ có một đồ thị bị ngắt kết nối. Do đó, bất đẳng thức thứ hai $\lambda \leq \delta$.

Điều thú vị là, bất đẳng thức Whitney không thể được cải thiện: tức là, đối với bất kỳ bộ ba số nào thỏa mãn bất đẳng thức này, tồn tại ít nhất một đồ thị tương ứng. Một đồ thị như vậy có thể được xây dựng theo cách sau: Đồ thị sẽ bao gồm $2(\delta + 1)$ đỉnh, $\delta + 1$ đỉnh đầu tiên tạo thành một clique (tất cả các cặp đỉnh được kết nối qua một cạnh), và $\delta + 1$ đỉnh thứ hai tạo thành một clique thứ hai. Ngoài ra, chúng ta kết nối hai clique với $\lambda$ cạnh, sao cho nó sử dụng $\lambda$ đỉnh khác nhau trong clique đầu tiên và chỉ $\kappa$ đỉnh trong clique thứ hai. Đồ thị kết quả sẽ có ba đặc trưng này.

#### Định Lý Ford-Fulkerson

Định lý Ford-Fulkerson ngụ ý rằng số lượng lớn nhất các đường đi không giao nhau kết nối hai đỉnh bằng với số lượng nhỏ nhất các cạnh tách rời hai đỉnh này.

### 3.42.3 Tính Toán Các Giá Trị

#### Kết Nối Cạnh Sử Dụng Luồng Cực Đại

Phương pháp này dựa trên định lý Ford-Fulkerson.

Chúng ta lặp qua tất cả các cặp đỉnh $(s, t)$ và giữa mỗi cặp, chúng ta tìm số lượng lớn nhất các đường đi không giao nhau giữa chúng. Giá trị này có thể được tìm bằng thuật toán luồng cực đại: chúng ta sử dụng $s$ làm nguồn, $t$ làm đích, và gán cho mỗi cạnh một dung lượng bằng 1. Sau đó, luồng cực đại chính là số lượng các đường đi không giao nhau.

Độ phức tạp của thuật toán sử dụng Edmonds-Karp là $O(V^2VE^2) = O(V^3E^2)$. Nhưng cần lưu ý rằng điều này bao gồm một yếu tố ẩn, vì thực tế không thể tạo ra một đồ thị sao cho thuật toán luồng cực đại sẽ chậm đối với tất cả các nguồn và đích. Đặc biệt, thuật toán sẽ chạy khá nhanh đối với các đồ thị ngẫu nhiên.

**Thuật Toán Đặc Biệt cho Kết Nối Cạnh**

Nhiệm vụ tìm kết nối cạnh tương đương với nhiệm vụ tìm cắt nhỏ nhất toàn cục.

Các thuật toán đặc biệt đã được phát triển cho nhiệm vụ này. Một trong số đó là thuật toán Stoer-Wagner, hoạt động trong thời gian $O(V^3)$ hoặc $O(VE)$.

**Kết Nối Đỉnh**

Một lần nữa, chúng ta lặp qua tất cả các cặp đỉnh $s$ và $t$, và đối với mỗi cặp, chúng ta tìm số lượng nhỏ nhất các đỉnh tách rời $s$ và $t$.

Bằng cách làm điều này, chúng ta có thể áp dụng cùng một cách tiếp cận luồng cực đại như đã mô tả trong các phần trước.

Chúng ta chia mỗi đỉnh $x$ với $x \neq s$ và $x \neq t$ thành hai đỉnh $x_1$ và $x_2$. Chúng ta kết nối hai đỉnh này với một cạnh có hướng $(x_1, x_2)$ với dung lượng 1, và thay thế tất cả các cạnh $(u, v)$ bằng hai cạnh có hướng $(u_2, v_1)$ và $(v_2, u_1)$, cả hai đều có dung lượng 1. Theo cấu trúc này, giá trị của luồng cực đại sẽ bằng với số lượng nhỏ nhất các đỉnh cần thiết để tách rời $s$ và $t$.

Cách tiếp cận này có cùng độ phức tạp như cách tiếp cận luồng để tìm kết nối cạnh.

## 3.43 Phân Rã Heavy-Light

Phân rã Heavy-Light là một kỹ thuật khá tổng quát cho phép chúng ta giải quyết hiệu quả nhiều bài toán liên quan đến các truy vấn trên cây.

### 3.43.1 Mô tả

Cho một cây $G$ có $n$ đỉnh, với một gốc tùy ý.

Bản chất của phân rã cây này là chia cây thành một số đường đi sao cho chúng ta có thể đến đỉnh gốc từ bất kỳ $v$ nào bằng cách đi qua tối đa $\log n$ đường đi. Ngoài ra, không có đường đi nào trong số này giao nhau với đường đi khác.

Rõ ràng, nếu chúng ta tìm thấy một phân rã như vậy cho bất kỳ cây nào, nó sẽ cho phép chúng ta giảm một số truy vấn đơn lẻ có dạng "tính toán một cái gì đó trên đường đi từ $a$ đến $b$" thành một số truy vấn có dạng "tính toán một cái gì đó trên đoạn $[l, r]$ của đường đi thứ $k^{\text{th}}$".

### 3.43.2 Thuật toán xây dựng

1. Chúng ta tính toán cho mỗi đỉnh $v$ kích thước của cây con của nó $s(v)$, tức là số lượng đỉnh trong cây con của đỉnh $v$ bao gồm cả chính nó.

2. Tiếp theo, xem xét tất cả các cạnh dẫn đến các con của một đỉnh $v$. Chúng ta gọi một cạnh là **heavy** nếu nó dẫn đến một đỉnh $c$ sao cho:

$$s(c) \geq \frac{s(v)}{2} \iff \text{cạnh } (v, c) \text{ là heavy}$$

Tất cả các cạnh khác được gắn nhãn là **light**.

3. Rõ ràng, nhiều nhất một cạnh heavy có thể xuất phát từ một đỉnh xuống dưới, bởi vì nếu không thì đỉnh $v$ sẽ có ít nhất hai con có kích thước $\geq \frac{s(v)}{2}$, và do đó kích thước của cây con của $v$ sẽ quá lớn:

$$s(v) \geq 1 + 2\frac{s(v)}{2} > s(v),$$

điều này dẫn đến một mâu thuẫn.

4. Bây giờ chúng ta sẽ phân rã cây thành các đường đi rời nhau. Xem xét tất cả các đỉnh mà từ đó không có cạnh heavy nào đi xuống. Chúng ta sẽ đi lên từ mỗi đỉnh như vậy cho đến khi chúng ta đến gốc của cây hoặc đi qua một cạnh light. Kết quả là, chúng ta sẽ nhận được một số đường đi được tạo thành từ không hoặc nhiều cạnh heavy cộng với một cạnh light. Đường đi có một đầu ở gốc là một ngoại lệ đối với điều này và sẽ không có cạnh light. Gọi đây là các đường đi heavy – đây là các đường đi mong muốn của phân rã heavy-light.

### 3.43.3 Chứng minh tính đúng đắn

1. Đầu tiên, chúng ta lưu ý rằng các đường đi heavy thu được bởi thuật toán sẽ **rời nhau**. Trên thực tế, nếu hai đường đi như vậy có một cạnh chung, nó sẽ ngụ ý rằng có hai cạnh heavy đi ra từ một đỉnh, điều này là không thể.

2. Thứ hai, chúng ta sẽ chỉ ra rằng đi xuống từ gốc của cây đến một đỉnh tùy ý, chúng ta sẽ thay đổi không quá

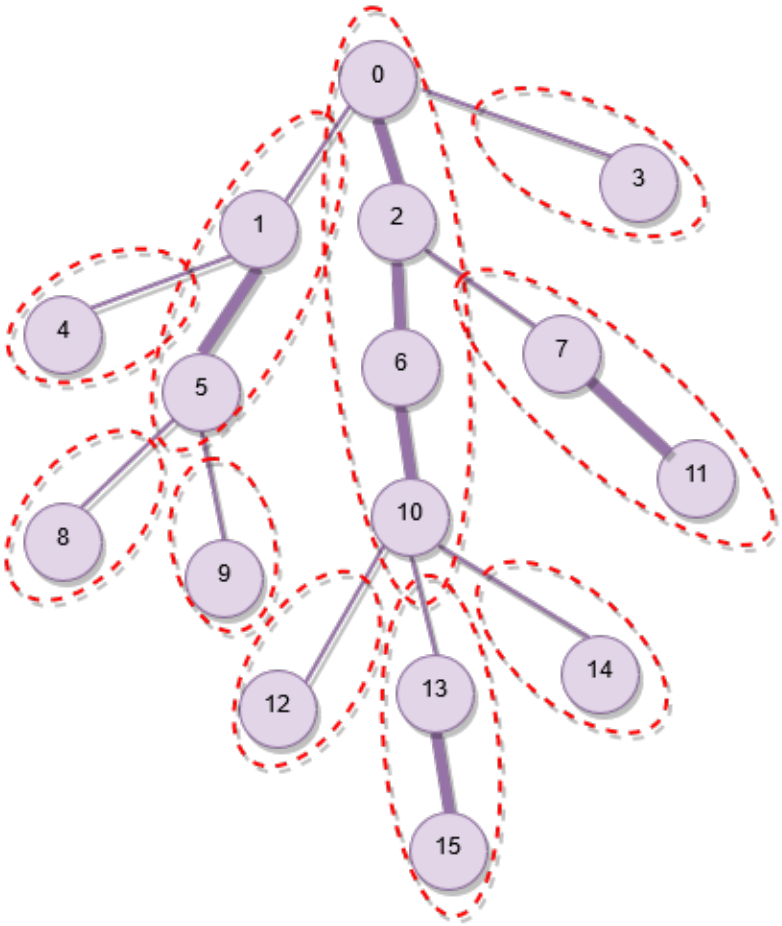
$\log n$ đường đi heavy trên đường đi. Di chuyển xuống một cạnh light làm giảm kích thước của cây con hiện tại xuống một nửa hoặc thấp hơn:

$$s(c) < \frac{s(v)}{2} \iff \text{cạnh } (v, c) \text{ là light}$$

Do đó, chúng ta có thể đi qua tối đa $\log n$ cạnh light trước khi kích thước cây con giảm xuống còn một.

Vì chúng ta có thể di chuyển từ một đường đi heavy sang một đường đi heavy khác chỉ thông qua một cạnh light (mỗi đường đi heavy, ngoại trừ đường đi bắt đầu từ gốc, có một cạnh light), chúng ta không thể thay đổi các đường đi heavy nhiều hơn $\log n$ lần dọc theo đường đi từ gốc đến bất kỳ đỉnh nào, như yêu cầu.

Hình ảnh sau đây minh họa sự phân rã của một cây mẫu. Các cạnh heavy dày hơn các cạnh light. Các đường đi heavy được đánh dấu bằng các ranh giới chấm chấm.




Hình 3.23: HLD

### 3.43.4 Các bài toán mẫu

Khi giải quyết các bài toán, đôi khi thuận tiện hơn khi xem xét phân rã heavy-light như một tập hợp các đường đi rời nhau về đỉnh (thay vì các đường đi rời nhau về cạnh). Để làm điều này, chỉ cần loại trừ cạnh cuối cùng khỏi mỗi đường đi heavy nếu nó là một cạnh

light, thì không có thuộc tính nào bị vi phạm, nhưng bây giờ mỗi đỉnh thuộc về chính xác một đường đi heavy.

Dưới đây chúng ta sẽ xem xét một số nhiệm vụ điển hình có thể được giải quyết với sự trợ giúp của phân rã heavy-light.

Riêng biệt, đáng chú ý là bài toán về tổng các số trên đường đi, vì đây là một ví dụ về một bài toán có thể được giải quyết bằng các kỹ thuật đơn giản hơn.

### Giá trị lớn nhất trên đường đi giữa hai đỉnh

Cho một cây, mỗi đỉnh được gán một giá trị. Có các truy vấn có dạng $(a, b)$, trong đó $a$ và $b$ là hai đỉnh trong cây và cần tìm giá trị lớn nhất trên đường đi giữa các đỉnh $a$ và $b$.

Chúng ta xây dựng trước một phân rã heavy-light của cây. Trên mỗi đường đi heavy, chúng ta sẽ xây dựng một cây đoạn, cho phép chúng ta tìm kiếm một đỉnh có giá trị được gán tối đa trong đoạn được chỉ định của đường đi heavy được chỉ định trong $\mathcal{O}(\log n)$. Mặc dù số lượng đường đi heavy trong phân rã heavy-light có thể đạt tới $n-1$, tổng kích thước của tất cả các đường đi bị giới hạn bởi $\mathcal{O}(n)$, do đó tổng kích thước của các cây đoạn cũng sẽ là tuyến tính.

Để trả lời một truy vấn $(a, b)$, chúng ta tìm tổ tiên chung thấp nhất của $a$ và $b$ là $l$, bằng bất kỳ phương pháp ưa thích nào. Bây giờ nhiệm vụ đã được giảm xuống thành hai truy vấn $(a, l)$ và $(b, l)$, cho mỗi truy vấn mà chúng ta có thể làm như sau: tìm đường đi heavy mà đỉnh dưới nằm trong đó, thực hiện một truy vấn trên đường đi này, di chuyển lên trên cùng của đường đi này, lại xác định đường đi heavy mà chúng ta đang ở và thực hiện một truy vấn trên nó, v.v., cho đến khi chúng ta đến đường đi chứa $l$.

Người ta nên cẩn thận với trường hợp khi, ví dụ, $a$ và $l$ nằm trên cùng một đường đi heavy - thì truy vấn tối đa trên đường đi này không nên được thực hiện trên bất kỳ tiền tố nào, mà trên phần bên trong giữa $a$ và $l$.

Trả lời các truy vấn con $(a, l)$ và $(b, l)$ mỗi truy vấn yêu cầu đi qua $\mathcal{O}(\log n)$ đường đi heavy và cho mỗi đường đi, một truy vấn tối đa được thực hiện trên một số phần của đường đi, điều này lại yêu cầu $\mathcal{O}(\log n)$ thao tác trong cây đoạn. Do đó, một truy vấn $(a, b)$ mất $\mathcal{O}(\log^2 n)$ thời gian.

Nếu bạn tính toán và lưu trữ thêm các giá trị tối đa của tất cả các tiền tố cho mỗi đường đi heavy, thì bạn sẽ nhận được một giải pháp $\mathcal{O}(\log n)$ vì tất cả các truy vấn tối đa đều nằm trên các tiền tố ngoại trừ nhiều nhất một lần khi chúng ta đến tổ tiên $l$.

### Tổng các số trên đường đi giữa hai đỉnh

Cho một cây, mỗi đỉnh được gán một giá trị. Có các truy vấn có dạng $(a, b)$, trong đó $a$ và $b$ là hai đỉnh trong cây và cần tìm tổng các giá trị trên đường đi giữa các đỉnh $a$ và $b$. Một biến thể của nhiệm vụ này có thể xảy ra khi ngoài ra còn có các thao tác cập nhật thay đổi số được gán cho một hoặc nhiều đỉnh.

Nhiệm vụ này có thể được giải quyết tương tự như bài toán trước về các giá trị tối đa với sự trợ giúp của phân rã heavy-light bằng cách xây dựng các cây đoạn trên các đường đi heavy. Các tổng tiền tố có thể được sử dụng thay thế nếu không có cập nhật. Tuy nhiên, bài toán này cũng có thể được giải quyết bằng các kỹ thuật đơn giản hơn.

Nếu không có cập nhật, thì có thể tìm ra tổng trên đường đi giữa hai đỉnh song song với tìm kiếm LCA của hai đỉnh bằng cách nâng nhị phân — vì điều này, cùng với tổ tiên thứ $2^k$ của mỗi đỉnh, cũng cần thiết phải lưu trữ tổng trên các đường đi lên đến những tổ tiên đó trong quá trình tiền xử lý.

Có một cách tiếp cận cơ bản khác đối với bài toán này - xem xét Euler tour của cây và xây dựng một cây đoạn trên nó. Thuật toán này được xem xét trong một bài viết về một bài toán tương tự. Một lần nữa, nếu không có cập nhật, việc lưu trữ các tổng tiền tố là đủ và không cần cây đoạn.

Cả hai phương pháp này đều cung cấp các giải pháp tương đối đơn giản mất $\mathcal{O}(\log n)$ cho một truy vấn.

#### Tô lại màu các cạnh của đường đi giữa hai đỉnh

Cho một cây, mỗi cạnh ban đầu được tô màu trắng. Có các cập nhật có dạng $(a, b, c)$, trong đó $a$ và $b$ là hai đỉnh và $c$ là một màu, hướng dẫn rằng tất cả các cạnh trên đường đi từ $a$ đến $b$ phải được tô lại bằng màu $c$. Sau tất cả các lần tô lại, cần báo cáo có bao nhiêu cạnh của mỗi màu đã thu được.

Tương tự như các bài toán trên, giải pháp là chỉ cần áp dụng phân rã heavy-light và tạo một cây đoạn trên mỗi đường đi heavy.

Mỗi lần tô lại trên đường đi $(a, b)$ sẽ biến thành hai cập nhật $(a, l)$ và $(b, l)$, trong đó $l$ là tổ tiên chung thấp nhất của các đỉnh $a$ và $b$.

$\mathcal{O}(\log n)$ trên mỗi đường đi cho $\mathcal{O}(\log n)$ đường đi dẫn đến độ phức tạp $\mathcal{O}(\log^2 n)$ trên mỗi cập nhật.

### 3.43.5 Triển khai

Một số phần nhất định của phương pháp được thảo luận ở trên có thể được sửa đổi để giúp việc triển khai dễ dàng hơn mà không làm mất hiệu quả.

Định nghĩa về cạnh heavy có thể được thay đổi thành cạnh dẫn đến con có cây con lớn nhất, với các mối quan hệ được phá vỡ một cách tùy ý. Điều này có thể dẫn đến một số cạnh light được chuyển đổi thành heavy, có nghĩa là một số đường đi heavy sẽ kết hợp để tạo thành một đường đi duy nhất, nhưng tất cả các đường đi heavy sẽ vẫn rời nhau. Nó cũng vẫn được đảm bảo rằng việc đi xuống một cạnh light làm giảm kích thước cây con xuống một nửa hoặc ít hơn. Thay vì xây dựng cây đoạn trên mọi đường đi heavy, một cây đoạn duy nhất có thể được sử dụng với các đoạn rời nhau được phân bổ cho mỗi đường đi heavy. Đã đề cập rằng việc trả lời các truy vấn yêu cầu tính toán LCA. Mặc

dù LCA có thể được tính toán riêng biệt, nhưng cũng có thể tích hợp tính toán LCA vào quá trình trả lời các truy vấn. Để thực hiện phân rã heavy-light:

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
```

```
    decompose(0, 0, adj);
}
```

Danh sách kề của cây phải được chuyển đến hàm `init` và phân rã được thực hiện giả sử đỉnh 0 là gốc.

Hàm `dfs` được sử dụng để tính toán `heavy[v]`, con ở đầu kia của cạnh heavy từ v, cho mọi đỉnh v. Ngoài ra, `dfs` cũng lưu trữ cha và độ sâu của mỗi đỉnh, điều này sẽ hữu ích sau này trong các truy vấn.

Hàm `decompose` gán cho mỗi đỉnh v các giá trị `head[v]` và `pos[v]`, tương ứng là đầu của đường đi heavy mà v thuộc về và vị trí của v trên cây đoạn duy nhất bao phủ tất cả các đỉnh.

Để trả lời các truy vấn trên các đường đi, ví dụ như truy vấn tối đa đã thảo luận, chúng ta có thể làm một cái gì đó như thế này:

```
int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = segment_tree_query(pos[head[b]], pos[b]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

# Chương 4

# Cấu trúc dữ liệu

## 4.1 Stack Tối Thiểu / Queue Tối Thiểu

Trong bài viết này, chúng ta sẽ xem xét ba bài toán: đầu tiên, chúng ta sẽ sửa đổi một stack theo cách cho phép chúng ta tìm phần tử nhỏ nhất của stack trong $O(1)$, sau đó chúng ta sẽ làm điều tương tự với một queue, và cuối cùng chúng ta sẽ sử dụng các cấu trúc dữ liệu này để tìm giá trị nhỏ nhất trong tất cả các mảng con có độ dài cố định trong một mảng trong $O(n)$.

### 4.1.1 Sửa đổi Stack

Chúng ta muốn sửa đổi cấu trúc dữ liệu stack theo cách mà có thể tìm thấy phần tử nhỏ nhất trong stack trong thời gian $O(1)$, đồng thời duy trì hành vi tiệm cận tương tự cho việc thêm và xóa các phần tử khỏi stack. Nhắc lại nhanh chóng, trên một stack, chúng ta chỉ thêm và xóa các phần tử ở một đầu.

Để làm điều này, chúng ta sẽ không chỉ lưu trữ các phần tử trong stack, mà chúng ta sẽ lưu trữ chúng theo cặp: chính phần tử đó và giá trị nhỏ nhất trong stack bắt đầu từ phần tử này trở xuống.

```
stack<pair<int, int>> st;
```

Rõ ràng là việc tìm giá trị nhỏ nhất trong toàn bộ stack chỉ bao gồm việc xem xét giá trị `stack.top().second`.

Cũng rõ ràng là việc thêm hoặc xóa một phần tử mới vào stack có thể được thực hiện trong thời gian không đổi.

**Triển khai:**

**Thêm một phần tử:**

```
int new_min = st.empty() ? new_elem : min(new_elem, st.top().second);
st.push({new_elem, new_min});
```

**Xóa một phần tử:**

```
int removed_element = st.top().first;
st.pop();
```

**Tìm giá trị nhỏ nhất:**

```
int minimum = st.top().second;
```

### 4.1.2 Sửa đổi Queue (phương pháp 1)

Bây giờ chúng ta muốn đạt được các thao tác tương tự với một queue, tức là chúng ta muốn thêm các phần tử ở cuối và xóa chúng từ phía trước.

Ở đây chúng ta xem xét một phương pháp đơn giản để sửa đổi một queue. Tuy nhiên, nó có một nhược điểm lớn, bởi vì queue đã sửa đổi sẽ thực sự không lưu trữ tất cả các phần tử.

Ý tưởng chính là chỉ lưu trữ các mục trong queue cần thiết để xác định giá trị nhỏ nhất. Cụ thể, chúng ta sẽ giữ queue theo thứ tự không giảm (tức là giá trị nhỏ nhất sẽ được lưu trữ ở đầu), và tất nhiên không phải theo bất kỳ cách tùy ý nào, giá trị nhỏ nhất thực tế phải luôn được chứa trong queue. Bằng cách này, phần tử nhỏ nhất sẽ luôn ở đầu queue. Trước khi thêm một phần tử mới vào queue, chỉ cần thực hiện một "cắt": chúng ta sẽ xóa tất cả các phần tử cuối của queue lớn hơn phần tử mới và sau đó thêm phần tử mới vào queue. Bằng cách này, chúng ta không phá vỡ thứ tự của queue và chúng ta cũng sẽ không mất phần tử hiện tại nếu nó là giá trị nhỏ nhất ở bất kỳ bước tiếp theo nào. Tất cả các phần tử mà chúng ta đã xóa không bao giờ có thể là giá trị nhỏ nhất, vì vậy thao tác này được cho phép. Khi chúng ta muốn trích xuất một phần tử từ đầu, nó thực sự có thể không ở đó (vì chúng ta đã xóa nó trước đó trong khi thêm một phần tử nhỏ hơn). Do đó, khi xóa một phần tử khỏi queue, chúng ta cần biết giá trị của phần tử đó. Nếu đầu của queue có cùng giá trị, chúng ta có thể xóa nó một cách an toàn, nếu không thì chúng ta không làm gì cả.

Xem xét các triển khai của các thao tác trên:

```
deque<int> q;
```

**Tìm giá trị nhỏ nhất:**

```
int minimum = q.front();
```

**Thêm một phần tử:**

```
while (!q.empty() && q.back() > new_element)
    q.pop_back();
q.push_back(new_element);
```

**Xóa một phần tử:**

```
if (!q.empty() && q.front() == remove_element)
    q.pop_front();
```

Rõ ràng là trung bình tất cả các thao tác này chỉ mất thời gian $O(1)$ (vì mỗi phần tử chỉ có thể được đẩy và bật một lần).

### 4.1.3 Sửa đổi Queue (phương pháp 2)

Đây là một sửa đổi của phương pháp 1. Chúng ta muốn có thể xóa các phần tử mà không cần biết phần tử nào chúng ta phải xóa. Chúng ta có thể thực hiện điều đó bằng cách lưu trữ chỉ mục cho mỗi phần tử trong queue. Và chúng ta cũng nhớ có bao nhiêu phần tử chúng ta đã thêm và xóa.

```
deque<pair<int, int>> q;
int cnt_added = 0;
int cnt_removed = 0;
```

**Tìm giá trị nhỏ nhất:**

```
int minimum = q.front().first;
```

**Thêm một phần tử:**

```
while (!q.empty() && q.back().first > new_element)
    q.pop_back();
q.push_back({new_element, cnt_added});
cnt_added++;
```

**Xóa một phần tử:**

```
if (!q.empty() && q.front().second == cnt_removed)
    q.pop_front();
cnt_removed++;
```

### 4.1.4 Sửa đổi Queue (phương pháp 3)

Ở đây chúng ta xem xét một cách khác để sửa đổi một queue để tìm giá trị nhỏ nhất trong $O(1)$. Cách này có phần phức tạp hơn để triển khai, nhưng lần này chúng ta thực sự lưu trữ tất cả các phần tử. Và chúng ta cũng có thể xóa một phần tử từ phía trước mà không cần biết giá trị của nó.

Ý tưởng là giảm bài toán về bài toán stack, mà chúng ta đã giải quyết. Vì vậy, chúng ta chỉ cần học cách mô phỏng một queue bằng hai stack.

Chúng ta tạo hai stack, `s1` và `s2`. Tất nhiên các stack này sẽ có dạng đã sửa đổi, để chúng ta có thể tìm giá trị nhỏ nhất trong $O(1)$. Chúng ta sẽ thêm các phần tử mới vào stack `s1` và xóa các phần tử khỏi stack `s2`. Nếu tại bất kỳ thời điểm nào stack `s2` trống, chúng ta sẽ di chuyển tất cả các phần tử từ `s1` sang `s2` (về cơ bản đảo ngược thứ tự của các phần tử đó). Cuối cùng, việc tìm giá trị nhỏ nhất trong một queue chỉ liên quan đến việc tìm giá trị nhỏ nhất của cả hai stack.

Do đó, chúng ta thực hiện tất cả các thao tác trong $O(1)$ trung bình (mỗi phần tử sẽ được thêm một lần vào stack `s1`, một lần được chuyển sang `s2` và một lần được bật khỏi `s2`)

**Triển khai:**

```
stack<pair<int, int>> s1, s2;
```

**Tìm giá trị nhỏ nhất:**

```
if (s1.empty() || s2.empty())
    minimum = s1.empty() ? s2.top().second : s1.top().second;
else
    minimum = min(s1.top().second, s2.top().second);
```

**Thêm phần tử:**

```
int minimum = s1.empty() ? new_element : min(new_element, s1.top().second);
s1.push({new_element, minimum});
```

**Xóa một phần tử:**

```
if (s2.empty()) {
    while (!s1.empty()) {
        int element = s1.top().first;
        s1.pop();
        int minimum = s2.empty() ? element : min(element, s2.top().second);
        s2.push({element, minimum});
    }
}
int remove_element = s2.top().first;
s2.pop();
```

### 4.1.5 Tìm giá trị nhỏ nhất cho tất cả các mảng con có độ dài cố định

Giả sử chúng ta có một mảng $A$ có độ dài $N$ và một $M \leq N$ đã cho. Chúng ta phải tìm giá trị nhỏ nhất của mỗi mảng con có độ dài $M$ trong mảng này, tức là chúng ta phải tìm:

$$\min_{0 \leq i \leq M-1} A[i], \min_{1 \leq i \leq M} A[i], \min_{2 \leq i \leq M+1} A[i], \ \ldots \ , \min_{N-M \leq i \leq N-1} A[i]$$

Chúng ta phải giải quyết bài toán này trong thời gian tuyến tính, tức là $O(n)$.

Chúng ta có thể sử dụng bất kỳ một trong ba queue đã sửa đổi để giải quyết bài toán. Các giải pháp phải rõ ràng: chúng ta thêm $M$ phần tử đầu tiên của mảng, tìm và xuất giá trị nhỏ nhất của nó, sau đó thêm phần tử tiếp theo vào queue và xóa phần tử đầu tiên của mảng, tìm và xuất giá trị nhỏ nhất của nó, v.v. Vì tất cả các thao tác với queue được thực hiện trong thời gian không đổi trung bình, nên độ phức tạp của toàn bộ thuật toán sẽ là $O(n)$.

## 4.2 Bảng Thưa (Sparse Table)

Bảng thưa là một cấu trúc dữ liệu cho phép trả lời các truy vấn phạm vi. Nó có thể trả lời hầu hết các truy vấn phạm vi trong $O(\log n)$, nhưng sức mạnh thực sự của nó là trả lời các truy vấn phạm vi tối thiểu (hoặc các truy vấn phạm vi tối đa tương đương). Đối với những truy vấn này, nó có thể tính toán câu trả lời trong thời gian $O(1)$.

Nhược điểm duy nhất của cấu trúc dữ liệu này là nó chỉ có thể được sử dụng trên các mảng bất biến. Điều này có nghĩa là mảng không thể thay đổi giữa hai truy vấn. Nếu bất kỳ phần tử nào trong mảng thay đổi, toàn bộ cấu trúc dữ liệu phải được tính toán lại.

### 4.2.1 Trực giác

Bất kỳ số không âm nào cũng có thể được biểu diễn duy nhất dưới dạng tổng của các lũy thừa giảm dần của hai. Đây chỉ là một biến thể của biểu diễn nhị phân của một số. Ví dụ: $13 = (1101)_2 = 8 + 4 + 1$. Đối với một số $x$ có thể có tối đa $\lceil \log_2 x \rceil$ số hạng.

Bằng lý luận tương tự, bất kỳ khoảng nào cũng có thể được biểu diễn duy nhất dưới dạng hợp của các khoảng có độ dài là các lũy thừa giảm dần của hai. Ví dụ: $[2, 14] = [2, 9] \cup [10, 13] \cup [14, 14]$, trong đó khoảng hoàn chỉnh có độ dài 13 và các khoảng riêng lẻ có độ dài lần lượt là 8, 4 và 1. Và ở đây hợp cũng bao gồm tối đa $\lceil \log_2(\text{độ dài của khoảng}) \rceil$ nhiều khoảng.

Ý tưởng chính đằng sau Bảng thưa là tính toán trước tất cả các câu trả lời cho các truy vấn phạm vi có độ dài là lũy thừa của hai. Sau đó, một truy vấn phạm vi khác có thể được trả lời bằng cách chia phạm vi thành các phạm vi có độ dài là lũy thừa của hai,

tra cứu các câu trả lời đã tính toán trước và kết hợp chúng để nhận được một câu trả lời hoàn chỉnh.

### 4.2.2 Tính toán trước

Chúng ta sẽ sử dụng một mảng 2 chiều để lưu trữ các câu trả lời cho các truy vấn đã tính toán trước. st$[i][j]$ sẽ lưu trữ câu trả lời cho phạm vi $[j, j + 2^i - 1]$ có độ dài $2^i$. Kích thước của mảng 2 chiều sẽ là $(K + 1) \times \mathrm{MAXN}$, trong đó MAXN là độ dài mảng lớn nhất có thể. K phải thỏa mãn $\mathrm{K} \geq \lfloor \log_2 \mathrm{MAXN} \rfloor$, vì $2^{\lfloor \log_2 \mathrm{MAXN} \rfloor}$ là phạm vi lũy thừa của hai lớn nhất mà chúng ta phải hỗ trợ. Đối với các mảng có độ dài hợp lý ( $\leq 10^7$ phần tử), $K = 25$ là một giá trị tốt.

Chiều MAXN là thứ hai để cho phép truy cập bộ nhớ liên tiếp (thân thiện với bộ nhớ cache).

```
int st[K + 1][MAXN];
```

Vì phạm vi $[j, j + 2^i - 1]$ có độ dài $2^i$ chia thành các phạm vi $[j, j + 2^{i-1} - 1]$ và $[j + 2^{i-1}, j + 2^i - 1]$, cả hai đều có độ dài $2^{i-1}$, chúng ta có thể tạo bảng một cách hiệu quả bằng cách sử dụng quy hoạch động:

```
std::copy(array.begin(), array.end(), st[0]);

for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = f(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

Hàm $f$ sẽ phụ thuộc vào loại truy vấn. Đối với các truy vấn tổng phạm vi, nó sẽ tính tổng, đối với các truy vấn phạm vi tối thiểu, nó sẽ tính giá trị nhỏ nhất.

Độ phức tạp thời gian của việc tính toán trước là $O(\mathrm{N} \log \mathrm{N})$.

### 4.2.3 Truy vấn Tổng Phạm vi

Đối với loại truy vấn này, chúng ta muốn tìm tổng của tất cả các giá trị trong một phạm vi. Do đó, định nghĩa tự nhiên của hàm $f$ là $f(x, y) = x + y$. Chúng ta có thể xây dựng cấu trúc dữ liệu với:

```
long long st[K + 1][MAXN];

std::copy(array.begin(), array.end(), st[0]);

for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = st[i - 1][j] + st[i - 1][j + (1 << (i - 1))];
```

Để trả lời truy vấn tổng cho phạm vi $[L, R]$, chúng ta lặp lại tất cả các lũy thừa của hai, bắt đầu từ lũy thừa lớn nhất. Ngay sau khi một lũy thừa của hai $2^i$ nhỏ hơn hoặc bằng độ dài của phạm vi ( $= R - L + 1$), chúng ta xử lý phần đầu tiên của phạm vi $[L, L + 2^i - 1]$ và tiếp tục với phạm vi còn lại $[L + 2^i, R]$.

```
long long sum = 0;
for (int i = K; i >= 0; i--) {
    if ((1 << i) <= R - L + 1) {
        sum += st[i][L];
        L += 1 << i;
    }
}
```

Độ phức tạp thời gian cho một Truy vấn Tổng Phạm vi là $O(K) = O(\log \text{MAXN})$.

### 4.2.4 Truy vấn Phạm vi Tối thiểu (RMQ)

Đây là những truy vấn mà Bảng thưa tỏa sáng. Khi tính giá trị nhỏ nhất của một phạm vi, không quan trọng nếu chúng ta xử lý một giá trị trong phạm vi một hoặc hai lần. Do đó, thay vì chia một phạm vi thành nhiều phạm vi, chúng ta cũng có thể chia phạm vi thành chỉ hai phạm vi chồng lên nhau có độ dài là lũy thừa của hai. Ví dụ: chúng ta có thể chia phạm vi $[1, 6]$ thành các phạm vi $[1, 4]$ và $[3, 6]$. Giá trị nhỏ nhất của phạm vi $[1, 6]$ rõ ràng là giống như giá trị nhỏ nhất của giá trị nhỏ nhất của phạm vi $[1, 4]$ và giá trị nhỏ nhất của phạm vi $[3, 6]$. Vì vậy, chúng ta có thể tính giá trị nhỏ nhất của phạm vi $[L, R]$ với:

$$\min(\text{st}[i][L], \text{st}[i][R - 2^i + 1]) \quad \text{trong đó } i = \log_2(R - L + 1)$$

Điều này đòi hỏi chúng ta phải có thể tính toán $\log_2(R - L + 1)$ nhanh chóng. Bạn có thể thực hiện điều đó bằng cách tính toán trước tất cả các logarit:

```
int lg[MAXN+1];
lg[1] = 0;
for (int i = 2; i <= MAXN; i++)
    lg[i] = lg[i/2] + 1;
```

Ngoài ra, log có thể được tính toán nhanh chóng trong không gian và thời gian không đổi:

```
// C++20
#include <bit>
int log2_floor(unsigned long i) {
```

```
    return std::bit_width(i) - 1;
}

// pre C++20
int log2_floor(unsigned long long i) {
    return i ? __builtin_clzll(1) - __builtin_clzll(i) : -1;
}
```

Điểm chuẩn này cho thấy rằng việc sử dụng mảng lg chậm hơn vì bỏ lỡ bộ nhớ cache. Sau đó, chúng ta cần tính toán trước cấu trúc Bảng thưa. Lần này chúng ta định nghĩa $f$ với $f(x, y) = \min(x, y)$.

```
int st[K + 1][MAXN];

std::copy(array.begin(), array.end(), st[0]);

for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

Và giá trị nhỏ nhất của một phạm vi $[L, R]$ có thể được tính toán với:

```
int i = lg[R - L + 1];
int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);
```

Độ phức tạp thời gian cho một Truy vấn Phạm vi Tối thiểu là $O(1)$.

### 4.2.5 Các cấu trúc dữ liệu tương tự hỗ trợ nhiều loại truy vấn hơn

Một trong những điểm yếu chính của phương pháp $O(1)$ được thảo luận trong phần trước là phương pháp này chỉ hỗ trợ các truy vấn của các hàm lũy đẳng. Tức là, nó hoạt động tuyệt vời cho các truy vấn phạm vi tối thiểu, nhưng không thể trả lời các truy vấn tổng phạm vi bằng phương pháp này.

Có các cấu trúc dữ liệu tương tự có thể xử lý bất kỳ loại hàm kết hợp nào và trả lời các truy vấn phạm vi trong $O(1)$. Một trong số đó được gọi là Bảng thưa rời nhau. Một cái khác sẽ là Cây Sqrt.

## 4.3 Tập Hợp Không Giao Nhau (Disjoint Set Union)

Bài viết này thảo luận về cấu trúc dữ liệu Tập Hợp Không Giao Nhau (Disjoint Set Union - DSU). Nó thường được gọi là Union Find vì hai thao tác chính của nó.

Cấu trúc dữ liệu này cung cấp các khả năng sau. Chúng ta được cho một số phần tử, mỗi phần tử ban đầu là một tập hợp riêng biệt. DSU sẽ có thao tác để kết hợp bất kỳ hai tập hợp nào, và nó có thể cho biết một phần tử cụ thể thuộc tập hợp nào. Phiên bản cổ điển cũng giới thiệu thao tác thứ ba, đó là tạo một tập hợp từ một phần tử mới.

Do đó, giao diện cơ bản của cấu trúc dữ liệu này chỉ bao gồm ba thao tác:

- `make_set(v)` - tạo một tập hợp mới chỉ gồm phần tử mới `v`.
- `union_sets(a, b)` - hợp nhất hai tập hợp được chỉ định (tập hợp chứa phần tử `a` và tập hợp chứa phần tử `b`).
- `find_set(v)` - trả về đại diện (còn gọi là leader) của tập hợp chứa phần tử `v`. Đại diện này là một phần tử của tập hợp tương ứng. Nó được cấu trúc dữ liệu tự chọn trong mỗi tập hợp (và có thể thay đổi theo thời gian, đặc biệt sau các lệnh gọi `union_sets`). Đại diện này có thể được dùng để kiểm tra xem hai phần tử có thuộc cùng một tập hợp hay không. `a` và `b` cùng thuộc một tập hợp nếu `find_set(a) == find_set(b)`. Ngược lại, chúng thuộc các tập hợp khác nhau.

Như được mô tả chi tiết hơn sau này, cấu trúc dữ liệu cho phép thực hiện mỗi thao tác này trong thời gian gần như $O(1)$ trung bình.

Ngoài ra, trong một tiểu mục, một cấu trúc thay thế của DSU sẽ được giải thích, đạt độ phức tạp trung bình chậm hơn là $O(\log n)$, nhưng có thể mạnh mẽ hơn cấu trúc DSU thông thường.

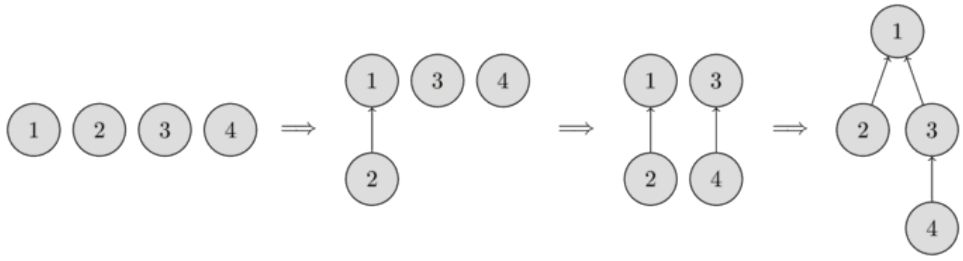
### 4.3.1 Xây dựng một cấu trúc dữ liệu hiệu quả

Chúng ta sẽ lưu trữ các tập hợp dưới dạng cây: mỗi cây sẽ tương ứng với một tập hợp. Và gốc của cây sẽ là đại diện/leader của tập hợp.

Trong hình ảnh sau, bạn có thể thấy cách biểu diễn của các cây như vậy.

Ban đầu, mỗi phần tử bắt đầu là một tập hợp riêng lẻ, do đó mỗi đỉnh là một cây riêng. Sau đó, chúng ta kết hợp tập hợp chứa phần tử 1 và tập hợp chứa phần tử 2. Tiếp theo, chúng ta kết hợp tập hợp chứa phần tử 3 và tập hợp chứa phần tử 4. Và ở bước cuối cùng, chúng ta kết hợp tập hợp chứa phần tử 1 và tập hợp chứa phần tử 3.

Về mặt triển khai, điều này có nghĩa là chúng ta sẽ phải duy trì một mảng `parent` lưu trữ tham chiếu đến tổ tiên trực tiếp của nó trong cây.




Hình 4.1: Ví dụ về biểu diễn tập hợp bằng cây

### 4.3.2 Triển khai ngây thơ

Chúng ta đã có thể viết triển khai đầu tiên của cấu trúc dữ liệu DSU. Ban đầu nó sẽ khá kém hiệu quả, nhưng sau này chúng ta có thể cải thiện nó bằng hai tối ưu hóa, để mỗi lệnh gọi hàm mất thời gian gần như hằng số.

Như đã nói, tất cả thông tin về các tập hợp của các phần tử sẽ được giữ trong một mảng `parent`.

Để tạo một tập hợp mới (thao tác `make_set(v)`), chúng ta chỉ cần tạo một cây với gốc tại đỉnh `v`, nghĩa là nó là tổ tiên của chính nó.

Để kết hợp hai tập hợp (thao tác `union_sets(a, b)`), chúng ta trước tiên tìm đại diện của tập hợp chứa `a` và đại diện của tập hợp chứa `b`. Nếu các đại diện giống nhau, chúng ta không cần làm gì, các tập hợp đã được hợp nhất. Ngược lại, chúng ta có thể chỉ định rằng một trong hai đại diện là cha của đại diện kia - qua đó kết hợp hai cây.

Cuối cùng là triển khai hàm tìm đại diện (thao tác `find_set(v)`): chúng ta chỉ cần leo lên các tổ tiên của đỉnh `v` cho đến khi đến gốc, tức là một đỉnh mà tham chiếu đến tổ tiên dẫn đến chính nó. Thao tác này dễ dàng được triển khai đệ quy.

```
void make_set(int v) {
    parent[v] = v;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
```

```
    if (a != b)
        parent[b] = a;
}
```

Tuy nhiên, triển khai này không hiệu quả. Dễ dàng xây dựng một ví dụ khiến các cây thoái hóa thành các chuỗi dài. Trong trường hợp đó, mỗi lệnh gọi `find_set(v)` có thể mất thời gian $O(n)$.

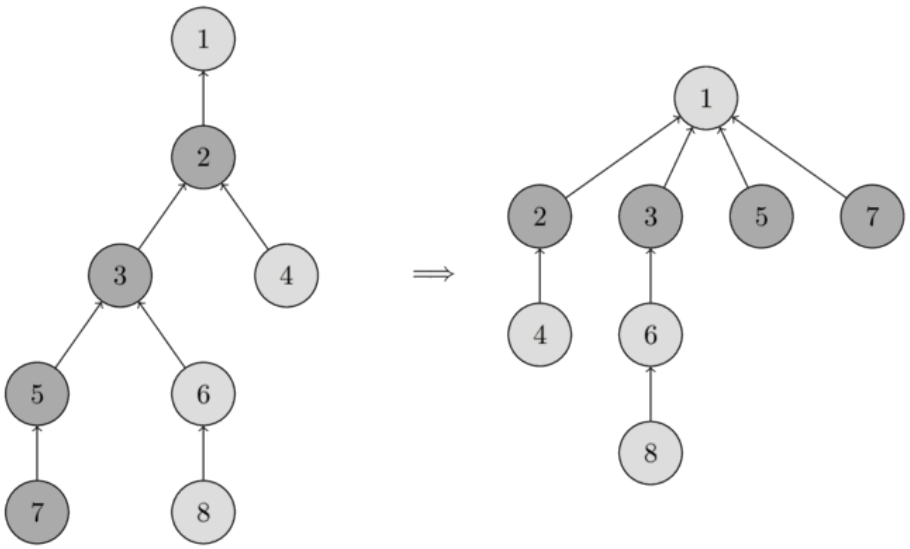
Điều này còn xa so với độ phức tạp mà chúng ta mong muốn (gần như thời gian hằng số). Do đó, chúng ta sẽ xem xét hai tối ưu hóa cho phép tăng tốc đáng kể công việc.

### 4.3.3 Tối ưu hóa nén đường đi

Tối ưu hóa này được thiết kế để tăng tốc `find_set`.

Nếu chúng ta gọi `find_set(v)` cho một đỉnh `v` nào đó, chúng ta thực sự tìm đại diện `p` cho tất cả các đỉnh mà chúng ta đi qua trên đường từ `v` đến đại diện thực sự `p`. Mẹo là làm ngắn các đường đi cho tất cả các nút đó, bằng cách đặt cha của mỗi đỉnh được thăm trực tiếp là `p`.

Bạn có thể thấy thao tác này trong hình ảnh sau. Bên trái là một cây, và bên phải là cây đã nén sau khi gọi `find_set(7)`, làm ngắn các đường đi cho các nút được thăm 7, 5, 3 và 2.




Hình 4.2: Nén đường đi của lệnh gọi `find_set(7)`

Triển khai mới của `find_set` như sau:

```
int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}
```

Triển khai đơn giản này thực hiện đúng ý định: đầu tiên tìm đại diện của tập hợp (đỉnh gốc), sau đó trong quá trình tháo gỡ ngăn xếp, các nút được thăm được gắn trực tiếp vào đại diện.

Sự sửa đổi đơn giản này của thao tác đã đạt được độ phức tạp thời gian $O(\log n)$ trung bình cho mỗi lệnh gọi (ở đây không có chứng minh). Có một sửa đổi thứ hai sẽ làm nó nhanh hơn nữa.

### 4.3.4 Hợp nhất theo kích thước / cấp bậc

Trong tối ưu hóa này, chúng ta sẽ thay đổi thao tác `union_sets`. Cụ thể, chúng ta sẽ thay đổi cây nào được gắn vào cây kia. Trong triển khai ngây thơ, cây thứ hai luôn được gắn vào cây thứ nhất. Trong thực tế, điều này có thể dẫn đến các cây chứa các chuỗi dài $O(n)$. Với tối ưu hóa này, chúng ta sẽ tránh điều này bằng cách chọn rất cẩn thận cây nào được gắn.

Có nhiều heuristic có thể được sử dụng. Hai cách tiếp cận phổ biến nhất là: Trong cách tiếp cận đầu tiên, chúng ta sử dụng kích thước của các cây làm cấp bậc, và trong cách thứ hai, chúng ta sử dụng độ sâu của cây (chính xác hơn, giới hạn trên của độ sâu cây, vì độ sâu sẽ nhỏ hơn khi áp dụng nén đường đi).

Ở cả hai cách tiếp cận, bản chất của tối ưu hóa là như nhau: chúng ta gắn cây có cấp bậc thấp hơn vào cây có cấp bậc lớn hơn.

Dưới đây là triển khai của hợp nhất theo kích thước:

```
void make_set(int v) {
    parent[v] = v;
    size[v] = 1;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (size[a] < size[b])
            swap(a, b);
        parent[b] = a;
        size[a] += size[b];
```

```
    }
}
```

Và đây là triển khai của hợp nhất theo cấp bậc dựa trên độ sâu của các cây:

```
void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}
```

Cả hai tối ưu hóa đều tương đương về độ phức tạp thời gian và không gian. Vì vậy, trong thực tế, bạn có thể sử dụng bất kỳ cái nào trong số chúng.

### 4.3.5 Độ phức tạp thời gian

Như đã đề cập trước đó, nếu chúng ta kết hợp cả hai tối ưu hóa - nén đường đi với hợp nhất theo kích thước/cấp bậc - chúng ta sẽ đạt được các truy vấn gần như thời gian hằng số. Hóa ra, độ phức tạp thời gian phân bổ cuối cùng là $O(\alpha(n))$, trong đó $\alpha(n)$ là hàm Ackermann ngược, hàm này tăng trưởng rất chậm. Thực tế, nó tăng chậm đến mức không vượt quá 4 cho tất cả $n$ hợp lý (xấp xỉ $n < 10^{600}$).

Độ phức tạp phân bổ là tổng thời gian cho mỗi thao tác, được đánh giá qua một chuỗi nhiều thao tác. Ý tưởng là đảm bảo tổng thời gian của toàn bộ chuỗi, trong khi cho phép các thao tác đơn lẻ chậm hơn nhiều so với thời gian phân bổ. Ví dụ, trong trường hợp của chúng ta, một lệnh gọi duy nhất có thể mất $O(\log n)$ trong trường hợp xấu nhất, nhưng nếu chúng ta thực hiện $m$ lệnh gọi liên tiếp, chúng ta sẽ có thời gian trung bình là $O(\alpha(n))$.

Chúng ta cũng sẽ không trình bày chứng minh cho độ phức tạp thời gian này, vì nó khá dài và phức tạp.

Ngoài ra, đáng nhắc đến là DSU với hợp nhất theo kích thước/cấp bậc, nhưng không có nén đường đi hoạt động trong thời gian $O(\log n)$ cho mỗi truy vấn.

### 4.3.6 Liên kết theo chỉ số / Liên kết tung đồng xu

Cả hợp nhất theo cấp bậc và hợp nhất theo kích thước đều yêu cầu bạn lưu trữ dữ liệu bổ sung cho mỗi tập hợp và duy trì các giá trị này trong mỗi thao tác hợp nhất. Cũng tồn tại một thuật toán ngẫu nhiên, đơn giản hóa thao tác hợp nhất một chút: liên kết theo chỉ số.

Chúng ta gán cho mỗi tập hợp một giá trị ngẫu nhiên gọi là chỉ số, và chúng ta gắn tập hợp có chỉ số nhỏ hơn vào tập hợp có chỉ số lớn hơn. Có khả năng một tập hợp lớn hơn sẽ có chỉ số lớn hơn tập hợp nhỏ hơn, do đó thao tác này liên quan chặt chẽ với hợp nhất theo kích thước. Thực tế, có thể chứng minh rằng thao tác này có cùng độ phức tạp thời gian với hợp nhất theo kích thước. Tuy nhiên, trong thực tế, nó chậm hơn một chút so với hợp nhất theo kích thước.

Bạn có thể tìm thấy chứng minh về độ phức tạp và nhiều kỹ thuật hợp nhất hơn tại đây.

```
void make_set(int v) {
    parent[v] = v;
    index[v] = rand();
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (index[a] < index[b])
            swap(a, b);
        parent[b] = a;
    }
}
```

Có một quan niệm sai lầm phổ biến rằng chỉ cần tung đồng xu để quyết định tập hợp nào gắn vào tập hợp kia sẽ có cùng độ phức tạp. Tuy nhiên, điều đó không đúng. Tài liệu được liên kết ở trên phỏng đoán rằng liên kết tung đồng xu kết hợp với nén đường đi có độ phức tạp $\Omega\left(n\frac{\log n}{\log\log n}\right)$. Và trong các bài kiểm tra, nó hoạt động tệ hơn nhiều so với hợp nhất theo kích thước/cấp bậc hoặc liên kết theo chỉ số.

```
void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rand() % 2)
```

```
            swap(a, b);
        parent[b] = a;
    }
}
```

### 4.3.7 Ứng dụng và các cải tiến khác nhau

Trong phần này, chúng ta xem xét một số ứng dụng của cấu trúc dữ liệu, cả những cách sử dụng đơn giản và một số cải tiến cho cấu trúc dữ liệu.

#### Thành phần liên thông trong đồ thị

Đây là một trong những ứng dụng rõ ràng của DSU.

Về mặt hình thức, bài toán được định nghĩa như sau: Ban đầu chúng ta có một đồ thị rỗng. Chúng ta phải thêm các đỉnh và các cạnh vô hướng, và trả lời các truy vấn dạng $(a, b)$ - "các đỉnh $a$ và $b$ có nằm trong cùng một thành phần liên thông của đồ thị không?"

Ở đây, chúng ta có thể áp dụng trực tiếp cấu trúc dữ liệu và nhận được giải pháp xử lý việc thêm một đỉnh hoặc một cạnh và một truy vấn trong thời gian gần như hằng số trung bình.

Ứng dụng này khá quan trọng, vì gần như cùng bài toán xuất hiện trong thuật toán Kruskal để tìm cây khung nhỏ nhất. Sử dụng DSU, chúng ta có thể cải thiện độ phức tạp từ $O(m \log n + n^2)$ xuống $O(m \log n)$.

#### Tìm kiếm thành phần liên thông trong ảnh

Một trong những ứng dụng của DSU là bài toán sau: có một ảnh kích thước $n \times m$ pixel. Ban đầu tất cả đều là màu trắng, nhưng sau đó một vài pixel đen được vẽ. Bạn muốn xác định kích thước của mỗi thành phần liên thông màu trắng trong ảnh cuối cùng.

Để giải quyết, chúng ta chỉ cần lặp qua tất cả các pixel trắng trong ảnh, với mỗi ô lặp qua bốn láng giềng của nó, và nếu láng giềng là màu trắng thì gọi `union_sets`. Do đó, chúng ta sẽ có một DSU với $nm$ nút tương ứng với các pixel ảnh. Các cây kết quả trong DSU là các thành phần liên thông mong muốn.

Bài toán cũng có thể được giải bằng DFS hoặc BFS, nhưng phương pháp được mô tả ở đây có ưu điểm: nó có thể xử lý ma trận theo từng hàng (tức là để xử lý một hàng, chúng ta chỉ cần hàng trước đó và hàng hiện tại, và chỉ cần một DSU được xây dựng cho các phần tử của một hàng) trong bộ nhớ $O(\min(n, m))$.

#### Lưu trữ thông tin bổ sung cho mỗi tập hợp

DSU cho phép dễ dàng lưu trữ thông tin bổ sung trong các tập hợp.

Một ví dụ đơn giản là kích thước của các tập hợp: việc lưu trữ kích thước đã được mô tả trong phần Hợp nhất theo kích thước (thông tin được lưu trữ bởi đại diện hiện tại của tập hợp).

Theo cách tương tự - bằng cách lưu trữ tại các nút đại diện - bạn cũng có thể lưu trữ bất kỳ thông tin nào khác về các tập hợp.

### Nén các bước nhảy dọc theo một đoạn / Tô màu các mảng con ngoại tuyến

Một ứng dụng phổ biến của DSU là bài toán sau: Có một tập hợp các đỉnh, và mỗi đỉnh có một cạnh ra đến một đỉnh khác. Với DSU, bạn có thể tìm điểm cuối mà chúng ta đến sau khi đi qua tất cả các cạnh từ một điểm xuất phát cho trước, trong thời gian gần như hằng số.

Một ví dụ tốt của ứng dụng này là bài toán tô màu các mảng con. Chúng ta có một đoạn độ dài $L$, mỗi phần tử ban đầu có màu 0. Chúng ta phải tô lại mảng con $[l, r]$ với màu $c$ cho mỗi truy vấn $(l, r, c)$. Cuối cùng, chúng ta muốn tìm màu cuối cùng của mỗi ô. Chúng ta giả định rằng chúng ta biết trước tất cả các truy vấn, tức là bài toán là ngoại tuyến.

Để giải quyết bài toán này, chúng ta có thể tạo một DSU, trong đó mỗi ô lưu trữ một liên kết đến ô chưa được tô tiếp theo. Ban đầu, mỗi ô trỏ đến chính nó. Sau khi tô một đoạn được yêu cầu, tất cả các ô từ đoạn đó sẽ trỏ đến ô ngay sau đoạn.

Bây giờ để giải bài toán này, chúng ta xem xét các truy vấn theo thứ tự ngược lại: từ cuối đến đầu. Bằng cách này, khi thực hiện một truy vấn, chúng ta chỉ cần tô chính xác các ô chưa được tô trong mảng con $[l, r]$. Tất cả các ô khác đã chứa màu cuối cùng của chúng. Để nhanh chóng lặp qua tất cả các ô chưa được tô, chúng ta sử dụng DSU. Chúng ta tìm ô chưa được tô ở bên trái nhất trong một đoạn, tô lại nó, và với con trỏ, chúng ta di chuyển đến ô trống tiếp theo ở bên phải.

Ở đây, chúng ta có thể sử dụng DSU với nén đường đi, nhưng không thể sử dụng hợp nhất theo cấp bậc/kích thước (vì điều quan trọng là ai trở thành leader sau khi hợp nhất). Do đó, độ phức tạp sẽ là $O(\log n)$ cho mỗi hợp nhất (cũng khá nhanh).

Triển khai:

```
for (int i = 0; i <= L; i++) {
    make_set(i);
}

for (int i = m-1; i >= 0; i--) {
    int l = query[i].l;
    int r = query[i].r;
    int c = query[i].c;
    for (int v = find_set(l); v <= r; v = find_set(v)) {
        answer[v] = c;
```

```
        parent[v] = v + 1;
    }
}
```

Có một tối ưu hóa: Chúng ta có thể sử dụng hợp nhất theo cấp bậc, nếu chúng ta lưu ô chưa được tô tiếp theo trong một mảng bổ sung `end[]`. Sau đó, chúng ta có thể hợp nhất hai tập hợp thành một theo heuristic của chúng, và chúng ta nhận được giải pháp trong $O(\alpha(n))$.

### Hỗ trợ khoảng cách đến đại diện

Đôi khi trong các ứng dụng cụ thể của DSU, bạn cần duy trì khoảng cách giữa một đỉnh và đại diện của tập hợp của nó (tức là độ dài đường đi trong cây từ nút hiện tại đến gốc của cây).

Nếu chúng ta không sử dụng nén đường đi, khoảng cách chỉ là số lần gọi đệ quy. Nhưng điều này sẽ không hiệu quả.

Tuy nhiên, có thể thực hiện nén đường đi, nếu chúng ta lưu trữ khoảng cách đến cha dưới dạng thông tin bổ sung cho mỗi nút.

Trong triển khai, việc sử dụng một mảng các cặp cho `parent[]` là thuận tiện, và hàm `find_set` giờ đây trả về hai số: đại diện của tập hợp và khoảng cách đến nó.

```
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int len = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second += len;
    }
    return parent[v];
}

void union_sets(int a, int b) {
    a = find_set(a).first;
    b = find_set(b).first;
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
```

```
        parent[b] = make_pair(a, 1);
        if (rank[a] == rank[b])
            rank[a]++;
    }
}
```

**Hỗ trợ tính chẵn lẻ của độ dài đường đi / Kiểm tra tính hai phía trực tuyến**

Cũng giống như tính toán độ dài đường đi đến leader, có thể duy trì tính chẵn lẻ của độ dài đường đi trước nó. Tại sao ứng dụng này lại ở một đoạn riêng?

Yêu cầu bất thường về lưu trữ tính chẵn lẻ của đường đi xuất hiện trong bài toán sau: ban đầu chúng ta được cho một đồ thị rỗng, có thể thêm các cạnh, và chúng ta phải trả lời các truy vấn dạng "thành phần liên thông chứa đỉnh này có phải là hai phía không?".

Để giải bài toán này, chúng ta tạo một DSU để lưu trữ các thành phần và lưu tính chẵn lẻ của đường đi đến đại diện cho mỗi đỉnh. Do đó, chúng ta có thể nhanh chóng kiểm tra xem việc thêm một cạnh có dẫn đến vi phạm tính hai phía hay không: cụ thể, nếu hai đầu của cạnh nằm trong cùng một thành phần liên thông và có cùng tính chẵn lẻ độ dài đến leader, thì việc thêm cạnh này sẽ tạo ra một chu trình độ dài lẻ, và thành phần sẽ mất tính chất hai phía.

Khó khăn duy nhất mà chúng ta đối mặt là tính toán tính chẵn lẻ trong phương thức `union_find`.

Nếu chúng ta thêm một cạnh $(a, b)$ nối hai thành phần liên thông thành một, thì khi gắn một cây vào cây kia, chúng ta cần điều chỉnh tính chẵn lẻ.

Hãy suy ra một công thức tính toán tính chẵn lẻ được gán cho leader của tập hợp sẽ được gắn vào tập hợp khác. Gọi $x$ là tính chẵn lẻ của độ dài đường đi từ đỉnh $a$ đến leader của nó $A$, và $y$ là tính chẵn lẻ của độ dài đường đi từ đỉnh $b$ đến leader của nó $B$, và $t$ là tính chẵn lẻ mong muốn mà chúng ta phải gán cho $B$ sau khi hợp nhất. Đường đi bao gồm ba phần: từ $B$ đến $b$, từ $b$ đến $a$, được kết nối bởi một cạnh và do đó có tính chẵn lẻ 1, và từ $a$ đến $A$. Do đó, chúng ta nhận được công thức ($\oplus$ biểu thị phép XOR):

$$t = x \oplus y \oplus 1$$

Do đó, bất kể chúng ta thực hiện bao nhiêu lần hợp nhất, tính chẵn lẻ của các cạnh được chuyển từ leader này sang leader khác.

Chúng ta đưa ra triển khai của DSU hỗ trợ tính chẵn lẻ. Như trong phần trước, chúng ta sử dụng một cặp để lưu trữ tổ tiên và tính chẵn lẻ. Ngoài ra, cho mỗi tập hợp, chúng ta lưu trong mảng `bipartite[]` liệu nó có vẫn là hai phía hay không.

```
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
    bipartite[v] = true;
```

```
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int parity = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second ^= parity;
    }
    return parent[v];
}

void add_edge(int a, int b) {
    pair<int, int> pa = find_set(a);
    a = pa.first;
    int x = pa.second;

    pair<int, int> pb = find_set(b);
    b = pb.first;
    int y = pb.second;

    if (a == b) {
        if (x == y)
            bipartite[a] = false;
    } else {
        if (rank[a] < rank[b])
            swap (a, b);
        parent[b] = make_pair(a, x^y^1);
        bipartite[a] &= bipartite[b];
        if (rank[a] == rank[b])
            ++rank[a];
    }
}

bool is_bipartite(int v) {
    return bipartite[find_set(v).first];
}
```

### RMQ ngoại tuyến (truy vấn tối thiểu trong đoạn) trong $O(\alpha(n))$ trung bình / Mẹo của Arpa

Chúng ta được cho một mảng `a[]` và phải tính toán một số giá trị nhỏ nhất trong các đoạn đã cho của mảng.

Ý tưởng để giải bài toán này với DSU như sau: Chúng ta sẽ lặp qua mảng và khi ở phần tử thứ `i`, chúng ta sẽ trả lời tất cả các truy vấn `(L, R)` với `R == i`. Để làm điều này hiệu quả, chúng ta sẽ giữ một DSU sử dụng `i` phần tử đầu tiên với cấu trúc sau: cha của một phần tử là phần tử nhỏ hơn tiếp theo bên phải của nó. Sau đó, sử dụng cấu trúc này, câu trả lời cho một truy vấn sẽ là `a[find_set(L)]`, số nhỏ nhất bên phải của `L`.

Cách tiếp cận này rõ ràng chỉ hoạt động ngoại tuyến, tức là nếu chúng ta biết trước tất cả các truy vấn.

Dễ thấy rằng chúng ta có thể áp dụng nén đường đi. Và chúng ta cũng có thể sử dụng Hợp nhất theo cấp bậc, nếu chúng ta lưu trữ leader thực sự trong một mảng riêng.

```
struct Query {
    int L, R, idx;
};

vector<int> answer;
vector<vector<Query>> container;
// container[i] chứa tất cả các truy vấn với R == i.

stack<int> s;
for (int i = 0; i < n; i++) {
    while (!s.empty() && a[s.top()] > a[i]) {
        parent[s.top()] = i;
        s.pop();
    }
    s.push(i);
    for (Query q : container[i]) {
        answer[q.idx] = a[find_set(q.L)];
    }
}
```

Ngày nay, thuật toán này được biết đến với tên gọi Mẹo của Arpa. Nó được đặt theo tên của AmirReza Poorakhavan, người đã độc lập phát hiện và phổ biến kỹ thuật này. Mặc dù thuật toán này đã tồn tại trước khi anh ấy phát hiện ra nó.

### LCA ngoại tuyến (tổ tiên chung thấp nhất trong cây) trong $O(\alpha(n))$ trung bình

Thuật toán để tìm LCA được thảo luận trong bài viết "Tổ tiên chung thấp nhất - Thuật toán ngoại tuyến của Tarjan". Thuật toán này so sánh thuận lợi với các thuật toán khác để tìm LCA nhờ sự đơn giản của nó (đặc biệt so với một thuật toán tối ưu như thuật toán của Farach-Colton và Bender).

### Lưu trữ DSU rõ ràng trong danh sách tập hợp / Ứng dụng của ý tưởng này khi hợp nhất các cấu trúc dữ liệu khác nhau

Một trong những cách thay thế để lưu trữ DSU là giữ mỗi tập hợp dưới dạng một danh sách được lưu trữ rõ ràng của các phần tử của nó. Đồng thời, mỗi phần tử cũng lưu trữ tham chiếu đến đại diện của tập hợp của nó.

Thoạt nhìn, điều này trông giống như một cấu trúc dữ liệu không hiệu quả: bằng cách kết hợp hai tập hợp, chúng ta sẽ phải thêm một danh sách vào cuối danh sách kia và phải cập nhật leadership trong tất cả các phần tử của một trong hai danh sách.

Tuy nhiên, hóa ra việc sử dụng một heuristic trọng số (tương tự như Hợp nhất theo kích thước) có thể giảm đáng kể độ phức tạp tiệm cận: $O(m + n \log n)$ để thực hiện $m$ truy vấn trên $n$ phần tử.

Dưới heuristic trọng số, chúng ta hiểu rằng chúng ta sẽ luôn thêm tập hợp nhỏ hơn vào tập hợp lớn hơn. Việc thêm một tập hợp vào tập hợp khác dễ triển khai trong `union_sets` và sẽ mất thời gian tỷ lệ với kích thước của tập hợp được thêm. Và việc tìm kiếm leader trong `find_set` sẽ mất $O(1)$ với cách lưu trữ này.

Hãy chứng minh độ phức tạp thời gian $O(m + n \log n)$ cho việc thực hiện $m$ truy vấn. Chúng ta sẽ cố định một phần tử bất kỳ $x$ và đếm xem nó được chạm đến bao nhiêu lần trong thao tác hợp nhất `union_sets`. Khi phần tử $x$ được chạm lần đầu tiên, kích thước của tập hợp mới sẽ ít nhất là 2. Khi nó được chạm lần thứ hai, tập hợp kết quả sẽ có kích thước ít nhất là 4, vì tập hợp nhỏ hơn được thêm vào tập hợp lớn hơn. Và cứ thế. Điều này có nghĩa là $x$ chỉ có thể được di chuyển trong tối đa $\log n$ thao tác hợp nhất. Do đó, tổng hợp trên tất cả các đỉnh cho $O(n \log n)$ cộng với $O(1)$ cho mỗi yêu cầu.

Dưới đây là triển khai:

```
vector<int> lst[MAXN];
int parent[MAXN];

void make_set(int v) {
    lst[v] = vector<int>(1, v);
    parent[v] = v;
}

int find_set(int v) {
```

```
    return parent[v];
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (lst[a].size() < lst[b].size())
            swap(a, b);
        while (!lst[b].empty()) {
            int v = lst[b].back();
            lst[b].pop_back();
            parent[v] = a;
            lst[a].push_back (v);
        }
    }
}
```

Ý tưởng thêm phần nhỏ hơn vào phần lớn hơn cũng có thể được sử dụng trong nhiều giải pháp không liên quan đến DSU.

Ví dụ, xét bài toán sau: chúng ta được cho một cây, mỗi lá có một số được gán (cùng một số có thể xuất hiện nhiều lần trên các lá khác nhau). Chúng ta muốn tính số lượng các số khác nhau trong cây con cho mỗi nút của cây.

Áp dụng ý tưởng tương tự cho bài toán này, có thể nhận được giải pháp sau: chúng ta có thể triển khai một DFS, sẽ trả về một con trỏ đến một tập hợp các số nguyên - danh sách các số trong cây con đó. Sau đó, để lấy câu trả lời cho nút hiện tại (trừ khi đó là lá), chúng ta gọi DFS cho tất cả các con của nút đó và hợp nhất tất cả các tập hợp nhận được lại với nhau. Kích thước của tập hợp kết quả sẽ là câu trả lời cho nút hiện tại. Để kết hợp hiệu quả nhiều tập hợp, chúng ta chỉ cần áp dụng công thức được mô tả ở trên: chúng ta hợp nhất các tập hợp bằng cách thêm các tập hợp nhỏ hơn vào các tập hợp lớn hơn. Cuối cùng, chúng ta nhận được giải pháp $O(n \log^2 n)$, vì một số chỉ được thêm vào một tập hợp tối đa $O(\log n)$ lần.

### Lưu trữ DSU bằng cách duy trì cấu trúc cây rõ ràng / Tìm cầu trực tuyến trong $O(\alpha(n))$ trung bình

Một trong những ứng dụng mạnh mẽ nhất của DSU là nó cho phép lưu trữ cả dưới dạng cây nén và không nén. Dạng nén có thể được sử dụng để hợp nhất các cây và để xác minh xem hai đỉnh có trong cùng một cây hay không, còn dạng không nén có thể được sử dụng - ví dụ - để tìm kiếm đường đi giữa hai đỉnh cho trước, hoặc các lượt đi qua khác của cấu trúc cây.

Trong triển khai, điều này có nghĩa là ngoài mảng tổ tiên nén `parent[]`, chúng ta sẽ cần giữ mảng tổ tiên không nén `real_parent[]`. Rõ ràng việc duy trì mảng bổ sung này sẽ không làm xấu đi độ phức tạp: các thay đổi trong đó chỉ xảy ra khi chúng ta hợp nhất hai cây, và chỉ trong một phần tử.

Mặt khác, khi áp dụng trong thực tế, chúng ta thường cần kết nối các cây bằng một cạnh được chỉ định thay vì sử dụng hai nút gốc. Điều này có nghĩa là chúng ta không có lựa chọn nào khác ngoài việc đổi gốc của một trong hai cây (làm cho các đầu của cạnh trở thành gốc mới của cây).

Thoạt nhìn, có vẻ như việc đổi gốc này rất tốn kém và sẽ làm xấu đi đáng kể độ phức tạp thời gian. Thật vậy, để đặt lại gốc cho một cây tại đỉnh $v$, chúng ta phải đi từ đỉnh đến gốc cũ và thay đổi hướng trong `parent[]` và `real_parent[]` cho tất cả các nút trên đường đi đó.

Tuy nhiên, trong thực tế, nó không tệ đến vậy, chúng ta có thể chỉ đổi gốc của cây nhỏ hơn trong hai cây tương tự như các ý tưởng trong các phần trước, và nhận được $O(\log n)$ trung bình.

Chi tiết hơn (bao gồm chứng minh độ phức tạp thời gian) có thể được tìm thấy trong bài viết "Tìm Cầu Trực Tuyến".

### 4.3.8 Hồi cố lịch sử

Cấu trúc dữ liệu DSU đã được biết đến từ lâu.

Cách lưu trữ cấu trúc này dưới dạng rừng cây dường như được mô tả lần đầu tiên bởi Galler và Fisher vào năm 1964 (Galler, Fisher, "An Improved Equivalence Algorithm"), tuy nhiên phân tích đầy đủ về độ phức tạp thời gian được tiến hành muộn hơn nhiều.

Các tối ưu hóa nén đường đi và Hợp nhất theo cấp bậc đã được phát triển bởi McIlroy và Morris, và độc lập với họ cũng bởi Tritter.

Hopcroft và Ullman đã chỉ ra vào năm 1973 độ phức tạp thời gian $O(\log^\star n)$ (Hopcroft, Ullman "Set-merging algorithms") - ở đây $\log^\star$ là logarit lặp (đây là một hàm tăng trưởng chậm, nhưng vẫn không chậm như hàm Ackermann ngược).

Lần đầu tiên đánh giá $O(\alpha(n))$ được chỉ ra vào năm 1975 (Tarjan "Efficiency of a Good But Not Linear Set Union Algorithm"). Sau đó vào năm 1985, ông cùng với Leeuwen đã công bố nhiều phân tích độ phức tạp cho một số heuristic cấp bậc khác nhau và cách nén đường đi (Tarjan, Leeuwen "Worst-case Analysis of Set Union Algorithms").

Cuối cùng, vào năm 1989, Fredman và Sachs đã chứng minh rằng trong mô hình tính toán được áp dụng, bất kỳ thuật toán nào cho bài toán tập hợp không giao nhau phải hoạt động trong ít nhất thời gian $O(\alpha(n))$ trung bình (Fredman, Saks, "The cell probe complexity of dynamic data structures").

## 4.4 Cây Fenwick

Cho $f$ là một phép toán nhóm nào đó (một hàm kết hợp nhị phân trên một tập hợp với một phần tử đơn vị và các phần tử nghịch đảo) và $A$ là một mảng các số nguyên có độ dài $N$. Ký hiệu phép toán infix của $f$ là $*$; tức là, $f(x, y) = x * y$ cho các số nguyên tùy ý $x, y$. (Vì đây là phép kết hợp, chúng ta sẽ bỏ qua dấu ngoặc đơn cho thứ tự áp dụng của $f$ khi sử dụng ký hiệu infix.)

Cây Fenwick là một cấu trúc dữ liệu mà:

- Tính giá trị của hàm $f$ trong phạm vi đã cho $[l, r]$ (tức là $A_l * A_{l+1} * \cdots * A_r$) trong thời gian $O(\log N)$
- Cập nhật giá trị của một phần tử của $A$ trong thời gian $O(\log N)$
- Yêu cầu bộ nhớ $O(N)$ (cùng lượng bộ nhớ cần thiết cho $A$)
- Dễ sử dụng và mã hóa, đặc biệt trong trường hợp mảng đa chiều

Ứng dụng phổ biến nhất của cây Fenwick là tính tổng của một phạm vi. Ví dụ: sử dụng phép cộng trên tập hợp các số nguyên làm phép toán nhóm, tức là $f(x, y) = x + y$: phép toán nhị phân, $*$, là $+$ trong trường hợp này, vì vậy $A_l * A_{l+1} * \cdots * A_r = A_l + A_{l+1} + \cdots + A_r$.

Cây Fenwick còn được gọi là Cây chỉ số nhị phân (Binary Indexed Tree - BIT). Nó được mô tả lần đầu tiên trong một bài báo có tựa đề "Một cấu trúc dữ liệu mới cho bảng tần số tích lũy" (Peter M. Fenwick, 1994).

## 4.5 Mô tả

### 4.5.1 Tổng quan

Để đơn giản, chúng ta sẽ giả định rằng hàm $f$ được định nghĩa là $f(x, y) = x + y$ trên các số nguyên.

Giả sử chúng ta có một mảng các số nguyên, $A[0 \ldots N-1]$. (Lưu ý rằng chúng ta đang sử dụng chỉ số bắt đầu từ 0.) Một cây Fenwick chỉ là một mảng, $T[0 \ldots N-1]$, trong đó mỗi phần tử bằng tổng của các phần tử của $A$ trong một phạm vi nào đó, $[g(i), i]$:

$$T_i = \sum_{j=g(i)}^{i} A_j$$

trong đó $g$ là một hàm nào đó thỏa mãn $0 \leq g(i) \leq i$. Chúng ta sẽ định nghĩa $g$ trong một vài đoạn tiếp theo.

Cấu trúc dữ liệu này được gọi là cây vì có một biểu diễn đẹp của nó dưới dạng một cây, mặc dù chúng ta không cần mô hình hóa một cây thực tế với các nút và cạnh. Chúng ta chỉ cần duy trì mảng $T$ để xử lý tất cả các truy vấn.

Lưu ý: Cây Fenwick được trình bày ở đây sử dụng chỉ số bắt đầu từ 0. Nhiều người sử dụng một phiên bản của cây Fenwick sử dụng chỉ số bắt đầu từ 1. Do đó, bạn cũng sẽ tìm thấy một triển khai thay thế sử dụng chỉ số bắt đầu từ 1 trong phần triển khai. Cả hai phiên bản đều tương đương về độ phức tạp thời gian và bộ nhớ.

Bây giờ chúng ta có thể viết một số mã giả cho hai thao tác đã đề cập ở trên. Dưới đây, chúng ta lấy tổng của các phần tử của $A$ trong phạm vi $[0, r]$ và cập nhật (tăng) một số phần tử $A_i$:

```
def sum(int r):
    res = 0
    while (r >= 0):
        res += t[r]
        r = g(r) - 1
    return res

def increase(int i, int delta):
    for all j with g(j) <= i <= j:
        t[j] += delta
```

Hàm sum hoạt động như sau:

- Đầu tiên, nó cộng tổng của phạm vi $[g(r), r]$ (tức là $T[r]$) vào kết quả.
- Sau đó, nó "nhảy" đến phạm vi $[g(g(r)-1), g(r)-1]$ và cộng tổng của phạm vi này vào kết quả.
- Điều này tiếp tục cho đến khi nó "nhảy" từ $[0, g(g(\ldots g(r) - 1 \cdots - 1) - 1)]$ đến $[g(-1), -1]$; đây là nơi hàm sum ngừng nhảy.

Hàm increase hoạt động tương tự, nhưng nó "nhảy" theo hướng tăng chỉ số:

- Tổng cho mỗi phạm vi có dạng $[g(j), j]$ thỏa mãn điều kiện $g(j) \leq i \leq j$ được tăng thêm delta; tức là, t[j] += delta. Do đó, nó cập nhật tất cả các phần tử trong $T$ tương ứng với các phạm vi mà $A_i$ nằm trong đó.

Độ phức tạp của cả sum và increase phụ thuộc vào hàm $g$. Có nhiều cách để chọn hàm $g$ sao cho $0 \leq g(i) \leq i$ cho tất cả $i$. Ví dụ: hàm $g(i) = i$ hoạt động, tạo ra $T = A$ (trong trường hợp này, các truy vấn tổng chậm). Chúng ta cũng có thể lấy hàm $g(i) = 0$. Điều này sẽ tương ứng với mảng tổng tiền tố (trong trường hợp này, việc tìm tổng của phạm vi $[0, i]$ sẽ chỉ mất thời gian không đổi; tuy nhiên, việc cập nhật chậm). Phần thông minh của thuật toán cho cây Fenwick là cách nó sử dụng một định nghĩa đặc biệt của hàm $g$ có thể xử lý cả hai thao tác trong thời gian $O(\log N)$.

### 4.5.2 Định nghĩa của $g(i)$

Việc tính toán $g(i)$ được định nghĩa bằng thao tác đơn giản sau: chúng ta thay thế tất cả các bit 1 ở cuối trong biểu diễn nhị phân của $i$ bằng các bit 0.

Nói cách khác, nếu chữ số có nghĩa ít nhất của $i$ trong hệ nhị phân là 0, thì $g(i) = i$. Và nếu không, chữ số có nghĩa ít nhất là 1, và chúng ta lấy 1 này và tất cả các 1 khác ở cuối và lật chúng.

Ví dụ: chúng ta có

$$\begin{aligned}
g(11) &= g(1011_2) = 1000_2 = 8 \\
g(12) &= g(1100_2) = 1100_2 = 12 \\
g(13) &= g(1101_2) = 1100_2 = 12 \\
g(14) &= g(1110_2) = 1110_2 = 14 \\
g(15) &= g(1111_2) = 0000_2 = 0
\end{aligned}$$

Có một triển khai đơn giản sử dụng các thao tác bitwise cho thao tác không tầm thường được mô tả ở trên:

$$g(i) = i \ \& \ (i + 1),$$

trong đó & là toán tử AND bitwise. Không khó để tự thuyết phục rằng giải pháp này thực hiện cùng một việc với thao tác được mô tả ở trên.

Bây giờ, chúng ta chỉ cần tìm một cách để lặp lại tất cả các $j$, sao cho $g(j) \leq i \leq j$.

Rất dễ thấy rằng chúng ta có thể tìm thấy tất cả các $j$ như vậy bằng cách bắt đầu với $i$ và lật bit chưa được đặt cuối cùng. Chúng ta sẽ gọi thao tác này là $h(j)$. Ví dụ: đối với $i = 10$, chúng ta có:

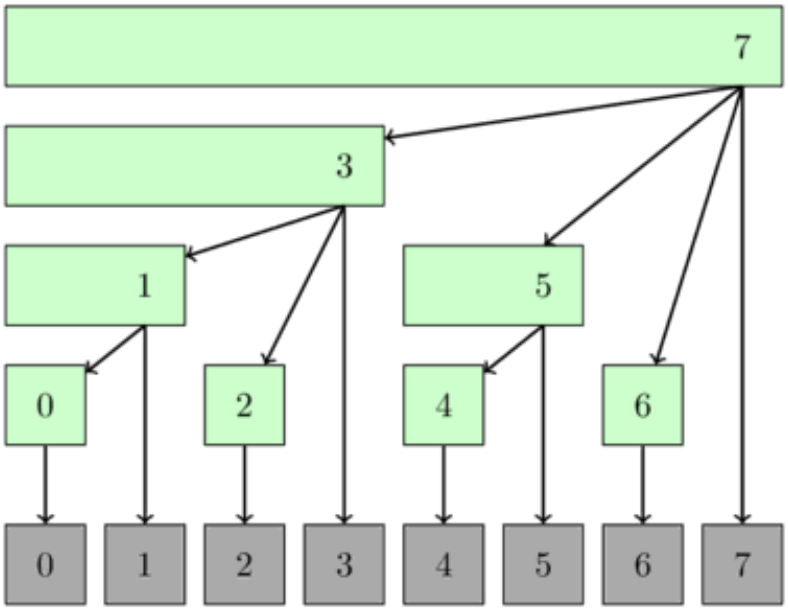
$$\begin{aligned}
10 &= 0001010_2 \\
h(10) = 11 &= 0001011_2 \\
h(11) = 15 &= 0001111_2 \\
h(15) = 31 &= 0011111_2 \\
h(31) = 63 &= 0111111_2 \\
&\vdots
\end{aligned}$$

Không có gì đáng ngạc nhiên, cũng có một cách đơn giản để thực hiện $h$ bằng cách sử dụng các thao tác bitwise:

$$h(j) = j \mid (j + 1),$$

trong đó | là toán tử OR bitwise.

Hình ảnh sau đây cho thấy một cách giải thích có thể có của cây Fenwick dưới dạng cây. Các nút của cây hiển thị các phạm vi mà chúng bao phủ.




## 4.6 Triển khai

### 4.6.1 Tìm tổng trong mảng một chiều

Ở đây chúng tôi trình bày một triển khai của cây Fenwick cho các truy vấn tổng và các cập nhật đơn lẻ.

Cây Fenwick thông thường chỉ có thể trả lời các truy vấn tổng thuộc loại $[0, r]$ bằng cách sử dụng sum(int r), tuy nhiên chúng ta cũng có thể trả lời các truy vấn khác thuộc loại $[l, r]$ bằng cách tính hai tổng $[0, r]$ và $[0, l-1]$ và trừ chúng. Điều này được xử lý trong phương thức sum(int l, int r).

Ngoài ra, triển khai này hỗ trợ hai hàm tạo. Bạn có thể tạo một cây Fenwick được khởi tạo bằng các số không, hoặc bạn có thể chuyển đổi một mảng hiện có thành dạng Fenwick.

```
struct FenwickTree {
    vector<int> bit;  // binary indexed tree
    int n;

    FenwickTree(int n) {
        this->n = n;
        bit.assign(n, 0);
    }
```

```
    FenwickTree(vector<int> const &a) : FenwickTree(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add(i, a[i]);
    }

    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret += bit[r];
        return ret;
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] += delta;
    }
};
```

### 4.6.2 Xây dựng tuyến tính

Việc triển khai trên yêu cầu thời gian $O(N \log N)$. Có thể cải thiện điều đó thành thời gian $O(N)$.

Ý tưởng là, số $a[i]$ tại chỉ số $i$ sẽ đóng góp vào phạm vi được lưu trữ trong $bit[i]$, và cho tất cả các phạm vi mà chỉ số $i|(i+1)$ đóng góp vào. Vì vậy, bằng cách cộng các số theo thứ tự, bạn chỉ cần đẩy tổng hiện tại xa hơn đến phạm vi tiếp theo, nơi nó sẽ được đẩy xa hơn đến phạm vi tiếp theo, v.v.

```
FenwickTree(vector<int> const &a) : FenwickTree(a.size()){
    for (int i = 0; i < n; i++) {
        bit[i] += a[i];
        int r = i | (i + 1);
        if (r < n) bit[r] += bit[i];
    }
}
```

### 4.6.3 Tìm giá trị nhỏ nhất của $[0, r]$ trong mảng một chiều

Rõ ràng là không có cách dễ dàng để tìm giá trị nhỏ nhất của phạm vi $[l, r]$ bằng cách sử dụng cây Fenwick, vì cây Fenwick chỉ có thể trả lời các truy vấn thuộc loại $[0, r]$. Ngoài ra, mỗi khi một giá trị được cập nhật, giá trị mới phải nhỏ hơn giá trị hiện tại. Cả hai hạn chế đáng kể là do thao tác *min* cùng với tập hợp các số nguyên không tạo thành một nhóm, vì không có các phần tử nghịch đảo.

```
struct FenwickTreeMin {
    vector<int> bit;
    int n;
    const int INF = (int)1e9;

    FenwickTreeMin(int n) {
        this->n = n;
        bit.assign(n, INF);
    }

    FenwickTreeMin(vector<int> a) : FenwickTreeMin(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            update(i, a[i]);
    }

    int getmin(int r) {
        int ret = INF;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret = min(ret, bit[r]);
        return ret;
    }

    void update(int idx, int val) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] = min(bit[idx], val);
    }
};
```

Lưu ý: có thể triển khai một cây Fenwick có thể xử lý các truy vấn phạm vi tối thiểu tùy ý và các cập nhật tùy ý. Bài báo Truy vấn phạm vi tối thiểu hiệu quả bằng cách sử dụng Cây chỉ số nhị phân mô tả một phương pháp như vậy. Tuy nhiên, với phương pháp đó, bạn cần duy trì một cây chỉ số nhị phân thứ hai trên dữ liệu, với một cấu trúc hơi khác, vì một cây không đủ để lưu trữ các giá trị của tất cả các phần tử trong mảng. Việc triển khai cũng khó hơn nhiều so với việc triển khai thông thường cho các tổng.

### 4.6.4 Tìm tổng trong mảng hai chiều

Như đã tuyên bố trước đó, rất dễ triển khai Cây Fenwick cho mảng đa chiều.

```
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;

    // init(...) { ... }

    int sum(int x, int y) {
        int ret = 0;
        for (int i = x; i >= 0; i = (i & (i + 1)) - 1)
            for (int j = y; j >= 0; j = (j & (j + 1)) - 1)
                ret += bit[i][j];
        return ret;
    }

    void add(int x, int y, int delta) {
        for (int i = x; i < n; i = i | (i + 1))
            for (int j = y; j < m; j = j | (j + 1))
                bit[i][j] += delta;
    }
};
```

### 4.6.5 Phương pháp lập chỉ mục dựa trên một

Đối với phương pháp này, chúng ta thay đổi các yêu cầu và định nghĩa cho $T[]$ và $g()$ một chút. Chúng ta muốn $T[i]$ lưu trữ tổng của $[g(i) + 1; i]$. Điều này thay đổi một chút việc triển khai và cho phép một định nghĩa tương tự tốt cho $g(i)$:

```
def sum(int r):
    res = 0
    while (r > 0):
        res += t[r]
        r = g(r)
    return res

def increase(int i, int delta):
    for all j with g(j) < i <= j:
        t[j] += delta
```

Việc tính toán $g(i)$ được định nghĩa là: chuyển đổi bit 1 được đặt cuối cùng trong biểu diễn nhị phân của $i$.

$$g(7) = g(111_2) = 110_2 = 6$$
$$g(6) = g(110_2) = 100_2 = 4$$
$$g(4) = g(100_2) = 000_2 = 0$$

Bit được đặt cuối cùng có thể được trích xuất bằng cách sử dụng $i \ \& \ (-i)$, vì vậy thao tác có thể được biểu thị là:

$$g(i) = i - (i \ \& \ (-i)).$$

Và không khó để thấy rằng bạn cần thay đổi tất cả các giá trị $T[j]$ trong chuỗi $i$, $h(i)$, $h(h(i))$, ... khi bạn muốn cập nhật $A[j]$, trong đó $h(i)$ được định nghĩa là:

$$h(i) = i + (i \ \& \ (-i)).$$

Như bạn có thể thấy, lợi ích chính của phương pháp này là các thao tác nhị phân bổ sung cho nhau rất tốt.

Việc triển khai sau đây có thể được sử dụng như các triển khai khác, tuy nhiên nó sử dụng lập chỉ mục dựa trên một cách nội bộ.

```
struct FenwickTreeOneBasedIndexing {
    vector<int> bit;  // binary indexed tree
    int n;

    FenwickTreeOneBasedIndexing(int n) {
        this->n = n + 1;
        bit.assign(n + 1, 0);
    }

    FenwickTreeOneBasedIndexing(vector<int> a)
        : FenwickTreeOneBasedIndexing(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add(i, a[i]);
    }

    int sum(int idx) {
        int ret = 0;
        for (++idx; idx > 0; idx -= idx & -idx)
            ret += bit[idx];
        return ret;
```

```
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void add(int idx, int delta) {
        for (++idx; idx < n; idx += idx & -idx)
            bit[idx] += delta;
    }
};
```

### 4.6.6 Các thao tác phạm vi

Một cây Fenwick có thể hỗ trợ các thao tác phạm vi sau:

1. Cập nhật điểm và truy vấn phạm vi
2. Cập nhật phạm vi và truy vấn điểm
3. Cập nhật phạm vi và truy vấn phạm vi

#### 1. Cập nhật điểm và truy vấn phạm vi

Đây chỉ là cây Fenwick thông thường như đã giải thích ở trên.

#### 2. Cập nhật phạm vi và truy vấn điểm

Sử dụng các thủ thuật đơn giản, chúng ta cũng có thể thực hiện các thao tác ngược lại: tăng phạm vi và truy vấn cho các giá trị đơn lẻ.

Giả sử cây Fenwick được khởi tạo bằng các số không. Giả sử rằng chúng ta muốn tăng khoảng $[l, r]$ lên $x$. Chúng ta thực hiện hai thao tác cập nhật điểm trên cây Fenwick là add(l, x) và add(r+1, -x).

Nếu chúng ta muốn lấy giá trị của $A[i]$, chúng ta chỉ cần lấy tổng tiền tố bằng phương pháp tổng phạm vi thông thường. Để xem tại sao điều này đúng, chúng ta chỉ cần tập trung lại vào thao tác tăng trước đó. Nếu $i < l$, thì hai thao tác cập nhật không có tác dụng đối với truy vấn và chúng ta nhận được tổng 0. Nếu $i \in [l, r]$, thì chúng ta nhận được câu trả lời $x$ vì thao tác cập nhật đầu tiên. Và nếu $i > r$, thì thao tác cập nhật thứ hai sẽ hủy bỏ tác dụng của thao tác đầu tiên.

Việc triển khai sau đây sử dụng lập chỉ mục dựa trên một.

```
void add(int idx, int val) {
    for (++idx; idx < n; idx += idx & -idx)
        bit[idx] += val;
}

void range_add(int l, int r, int val) {
    add(l, val);
    add(r + 1, -val);
}

int point_query(int idx) {
    int ret = 0;
    for (++idx; idx > 0; idx -= idx & -idx)
        ret += bit[idx];
    return ret;
}
```

Lưu ý: tất nhiên, cũng có thể tăng một điểm duy nhất $A[i]$ với range_add(i, i, val).

### 3. Cập nhật phạm vi và truy vấn phạm vi

Để hỗ trợ cả cập nhật phạm vi và truy vấn phạm vi, chúng ta sẽ sử dụng hai BIT, cụ thể là $B_1[]$ và $B_2[]$, được khởi tạo bằng các số không.

Giả sử rằng chúng ta muốn tăng khoảng $[l, r]$ theo giá trị $x$. Tương tự như trong phương pháp trước, chúng ta thực hiện hai cập nhật điểm trên $B_1$: add(B1, l, x) và add(B1, r+1, -x). Và chúng ta cũng cập nhật $B_2$. Các chi tiết sẽ được giải thích sau.

```
def range_add(l, r, x):
    add(B1, l, x)
    add(B1, r+1, -x)
    add(B2, l, x*(l-1))
    add(B2, r+1, -x*r))
```

Sau khi cập nhật phạm vi $(l, r, x)$, truy vấn tổng phạm vi sẽ trả về các giá trị sau:

$$sum[0, i] = \begin{cases} 0 & i < l \\ x \cdot (i - (l-1)) & l \leq i \leq r \\ x \cdot (r - l + 1) & i > r \end{cases}$$

Chúng ta có thể viết tổng phạm vi dưới dạng hiệu của hai số hạng, trong đó chúng ta sử dụng $B_1$ cho số hạng đầu tiên và $B_2$ cho số hạng thứ hai. Hiệu của các truy vấn sẽ cho chúng ta tổng tiền tố trên $[0, i]$.

$$
\begin{aligned}
sum[0,i] &= sum(B_1, i) \cdot i - sum(B_2, i) \\
&= \begin{cases} 0 \cdot i - 0 & \text{nếu } i < l, \\ x \cdot i - x \cdot (l-1) & \text{nếu } l \le i \le r, \\ 0 \cdot i - (x \cdot (l-1) - x \cdot r) & \text{nếu } i > r. \end{cases}
\end{aligned}
$$

Biểu thức cuối cùng chính xác bằng các số hạng bắt buộc. Do đó, chúng ta có thể sử dụng $B_2$ để loại bỏ các số hạng thừa khi chúng ta nhân $B_1[i] \times i$.

Chúng ta có thể tìm thấy các tổng phạm vi tùy ý bằng cách tính tổng tiền tố cho $l-1$ và $r$ và lấy hiệu của chúng một lần nữa.

```
def add(b, idx, x):
    while idx <= N:
        b[idx] += x
        idx += idx & -idx

def range_add(l,r,x):
    add(B1, l, x)
    add(B1, r+1, -x)
    add(B2, l, x*(l-1))
    add(B2, r+1, -x*r)

def sum(b, idx):
    total = 0
    while idx > 0:
        total += b[idx]
        idx -= idx & -idx
    return total

def prefix_sum(idx):
    return sum(B1, idx)*idx -  sum(B2, idx)

def range_sum(l, r):
    return prefix_sum(r) - prefix_sum(l-1)
```

## 4.7 Phân Rã Căn Bậc Hai (Sqrt Decomposition)

Phân rã căn bậc hai là một phương pháp (hoặc một cấu trúc dữ liệu) cho phép bạn thực hiện một số thao tác phổ biến (tìm tổng các phần tử của mảng con, tìm phần tử nhỏ nhất/lớn nhất, v.v.) trong $O(\sqrt{n})$ thao tác, nhanh hơn nhiều so với $O(n)$ của thuật toán đơn giản.

Đầu tiên, chúng ta sẽ mô tả cấu trúc dữ liệu cho một trong những ứng dụng đơn giản nhất của ý tưởng này, sau đó chỉ ra cách tổng quát hóa nó để giải quyết một số vấn đề khác và cuối cùng xem xét một cách sử dụng hơi khác của ý tưởng này: chia các yêu cầu đầu vào thành các khối sqrt.

## 4.8 Cấu trúc dữ liệu dựa trên phân rã Sqrt

Cho một mảng $a[0 \dots n-1]$, hãy triển khai một cấu trúc dữ liệu cho phép tìm tổng các phần tử $a[l \dots r]$ cho các $l$ và $r$ tùy ý trong $O(\sqrt{n})$ thao tác.

### 4.8.1 Mô tả

Ý tưởng cơ bản của phân rã sqrt là tiền xử lý. Chúng ta sẽ chia mảng $a$ thành các khối có độ dài xấp xỉ $\sqrt{n}$ và đối với mỗi khối $i$, chúng ta sẽ tính toán trước tổng các phần tử trong đó $b[i]$.

Chúng ta có thể giả định rằng cả kích thước của khối và số lượng khối đều bằng $\sqrt{n}$ được làm tròn lên:

$$s = \lceil \sqrt{n} \rceil$$

Sau đó, mảng $a$ được chia thành các khối theo cách sau:

$$\underbrace{a[0], a[1], \dots, a[s-1]}_{\text{b[0]}}, \underbrace{a[s], \dots, a[2s-1]}_{\text{b[1]}}, \dots, \underbrace{a[(s-1) \cdot s], \dots, a[n-1]}_{\text{b[s-1]}}$$

Khối cuối cùng có thể có ít phần tử hơn các khối khác (nếu $n$ không phải là bội số của $s$), điều đó không quan trọng đối với cuộc thảo luận (vì nó có thể được xử lý dễ dàng). Do đó, đối với mỗi khối $k$, chúng ta biết tổng các phần tử trên đó $b[k]$:

$$b[k] = \sum_{i=k \cdot s}^{\min(n-1, (k+1) \cdot s - 1)} a[i]$$

Vì vậy, chúng ta đã tính toán các giá trị của $b[k]$ (điều này đòi hỏi $O(n)$ thao tác). Làm thế nào chúng có thể giúp chúng ta trả lời mỗi truy vấn $[l, r]$? Lưu ý rằng nếu khoảng $[l, r]$ đủ dài, nó sẽ chứa một số khối nguyên và đối với những khối đó, chúng ta có thể tìm

thấy tổng các phần tử trong chúng trong một thao tác duy nhất. Do đó, khoảng $[l, r]$ sẽ chỉ chứa các phần của hai khối và chúng ta sẽ phải tính tổng các phần tử trong các phần này một cách đơn giản.

Do đó, để tính tổng các phần tử trên khoảng $[l, r]$, chúng ta chỉ cần tính tổng các phần tử của hai "đuôi": $[l \ldots (k+1) \cdot s - 1]$ và $[p \cdot s \ldots r]$, và tính tổng các giá trị $b[i]$ trong tất cả các khối từ $k+1$ đến $p-1$:

$$\sum_{i=l}^{r} a[i] = \sum_{i=l}^{(k+1)\cdot s-1} a[i] + \sum_{i=k+1}^{p-1} b[i] + \sum_{i=p\cdot s}^{r} a[i]$$

Lưu ý: Khi $k = p$, tức là $l$ và $r$ thuộc cùng một khối, công thức không thể được áp dụng và tổng phải được tính một cách đơn giản.

Cách tiếp cận này cho phép chúng ta giảm đáng kể số lượng thao tác. Thật vậy, kích thước của mỗi "đuôi" không vượt quá độ dài khối $s$ và số lượng khối trong tổng không vượt quá $s$. Vì chúng ta đã chọn $s \approx \sqrt{n}$, tổng số thao tác cần thiết để tìm tổng các phần tử trên khoảng $[l, r]$ là $O(\sqrt{n})$.

### 4.8.2 Triển khai

Hãy bắt đầu với việc triển khai đơn giản nhất:

```
// dữ liệu đầu vào
int n;
vector<int> a (n);

// tiền xử lý
int len = (int) sqrt (n + .0) + 1; // kích thước của khối và số lượng khối
vector<int> b (len);
for (int i=0; i<n; ++i)
    b[i / len] += a[i];

// trả lời các truy vấn
for (;;) {
    int l, r;
  // đọc dữ liệu đầu vào cho truy vấn tiếp theo
    int sum = 0;
    for (int i=l; i<=r; )
        if (i % len == 0 && i + len - 1 <= r) {
            // nếu toàn bộ khối bắt đầu từ i thuộc [l, r]
            sum += b[i / len];
            i += len;
```

```
        }
        else {
            sum += a[i];
            ++i;
        }
}
```

Việc triển khai này có quá nhiều phép chia (chậm hơn nhiều so với các phép toán số học khác). Thay vào đó, chúng ta có thể tính toán các chỉ số của các khối $c_l$ và $c_r$ chứa các chỉ số $l$ và $r$ và lặp qua các khối $c_l + 1 \ldots c_r - 1$ với xử lý riêng các "đuôi" trong các khối $c_l$ và $c_r$. Cách tiếp cận này tương ứng với công thức cuối cùng trong mô tả và làm cho trường hợp $c_l = c_r$ trở thành một trường hợp đặc biệt.

```
int sum = 0;
int c_l = l / len,   c_r = r / len;
if (c_l == c_r)
    for (int i=l; i<=r; ++i)
        sum += a[i];
else {
    for (int i=l, end=(c_l+1)*len-1; i<=end; ++i)
        sum += a[i];
    for (int i=c_l+1; i<=c_r-1; ++i)
        sum += b[i];
    for (int i=c_r*len; i<=r; ++i)
        sum += a[i];
}
```

## 4.9 Các vấn đề khác

Cho đến nay, chúng ta đã thảo luận về vấn đề tìm tổng các phần tử của một mảng con liên tục. Vấn đề này có thể được mở rộng để cho phép cập nhật các phần tử mảng riêng lẻ. Nếu một phần tử $a[i]$ thay đổi, chỉ cần cập nhật giá trị của $b[k]$ cho khối mà phần tử này thuộc về ($k = i/s$) trong một thao tác:

$$b[k] += a_{new}[i] - a_{old}[i]$$

Mặt khác, nhiệm vụ tìm tổng các phần tử có thể được thay thế bằng nhiệm vụ tìm phần tử nhỏ nhất/lớn nhất của một mảng con. Nếu vấn đề này cũng phải giải quyết các cập nhật phần tử riêng lẻ, thì việc cập nhật giá trị của $b[k]$ cũng có thể thực hiện được, nhưng nó sẽ yêu cầu lặp lại tất cả các giá trị của khối $k$ trong $O(s) = O(\sqrt{n})$ thao tác.

Phân rã Sqrt có thể được áp dụng theo cách tương tự cho cả một lớp các vấn đề khác: tìm số lượng phần tử bằng không, tìm phần tử khác không đầu tiên, đếm các phần tử thỏa mãn một thuộc tính nhất định, v.v.

Một lớp vấn đề khác xuất hiện khi chúng ta cần cập nhật các phần tử mảng trên các khoảng: tăng các phần tử hiện có hoặc thay thế chúng bằng một giá trị đã cho.

Ví dụ: giả sử chúng ta có thể thực hiện hai loại thao tác trên một mảng: thêm một giá trị đã cho $\delta$ vào tất cả các phần tử mảng trên khoảng $[l, r]$ hoặc truy vấn giá trị của phần tử $a[i]$. Hãy lưu trữ giá trị cần được thêm vào tất cả các phần tử của khối $k$ trong $b[k]$ (ban đầu tất cả $b[k] = 0$). Trong mỗi thao tác "add", chúng ta cần thêm $\delta$ vào $b[k]$ cho tất cả các khối thuộc khoảng $[l, r]$ và thêm $\delta$ vào $a[i]$ cho tất cả các phần tử thuộc "đuôi" của khoảng. Câu trả lời cho truy vấn $i$ chỉ đơn giản là $a[i] + b[i/s]$. Bằng cách này, thao tác "add" có độ phức tạp $O(\sqrt{n})$ và trả lời một truy vấn có độ phức tạp $O(1)$.

Cuối cùng, hai lớp vấn đề đó có thể được kết hợp nếu nhiệm vụ yêu cầu thực hiện cả cập nhật phần tử trên một khoảng và truy vấn trên một khoảng. Cả hai thao tác có thể được thực hiện với độ phức tạp $O(\sqrt{n})$. Điều này sẽ yêu cầu hai mảng khối $b$ và $c$: một để theo dõi các cập nhật phần tử và một để theo dõi các câu trả lời cho truy vấn.

Có những vấn đề khác có thể được giải quyết bằng cách sử dụng phân rã sqrt, ví dụ: một vấn đề về duy trì một tập hợp các số cho phép thêm/xóa số, kiểm tra xem một số có thuộc tập hợp hay không và tìm số lớn thứ $k$. Để giải quyết nó, người ta phải lưu trữ các số theo thứ tự tăng dần, chia thành nhiều khối với $\sqrt{n}$ số trong mỗi khối. Mỗi khi một số được thêm/xóa, các khối phải được cân bằng lại bằng cách di chuyển các số giữa phần đầu và phần cuối của các khối liền kề.

## 4.10 Thuật toán Mo

Một ý tưởng tương tự, dựa trên phân rã sqrt, có thể được sử dụng để trả lời các truy vấn phạm vi ($Q$) ngoại tuyến trong $O((N + Q)\sqrt{N})$. Điều này có vẻ tệ hơn nhiều so với các phương pháp trong phần trước, vì đây là độ phức tạp hơi tệ hơn so với chúng ta đã có trước đó và không thể cập nhật các giá trị giữa hai truy vấn. Nhưng trong nhiều tình huống, phương pháp này có những ưu điểm. Trong quá trình phân rã sqrt thông thường, chúng ta phải tính toán trước các câu trả lời cho mỗi khối và hợp nhất chúng trong quá trình trả lời các truy vấn. Trong một số vấn đề, bước hợp nhất này có thể khá khó khăn. Ví dụ: khi mỗi truy vấn yêu cầu tìm mode của phạm vi của nó (số xuất hiện thường xuyên nhất). Đối với điều này, mỗi khối sẽ phải lưu trữ số lượng của mỗi số trong đó trong một số loại cấu trúc dữ liệu và chúng ta không còn có thể thực hiện bước hợp nhất đủ nhanh nữa. Thuật toán Mo sử dụng một cách tiếp cận hoàn toàn khác, có thể trả lời nhanh chóng các loại truy vấn này, vì nó chỉ theo dõi một cấu trúc dữ liệu và các thao tác duy nhất với nó rất dễ dàng và nhanh chóng.

Ý tưởng là trả lời các truy vấn theo một thứ tự đặc biệt dựa trên các chỉ số. Trước tiên, chúng ta sẽ trả lời tất cả các truy vấn có chỉ số bên trái trong khối 0, sau đó trả lời

tất cả các truy vấn có chỉ số bên trái trong khối 1, v.v. Và chúng ta cũng sẽ phải trả lời các truy vấn của một khối theo một thứ tự đặc biệt, cụ thể là được sắp xếp theo chỉ số bên phải của các truy vấn.

Như đã nói, chúng ta sẽ sử dụng một cấu trúc dữ liệu duy nhất. Cấu trúc dữ liệu này sẽ lưu trữ thông tin về phạm vi. Lúc đầu, phạm vi này sẽ trống. Khi chúng ta muốn trả lời truy vấn tiếp theo (theo thứ tự đặc biệt), chúng ta chỉ cần mở rộng hoặc giảm phạm vi, bằng cách thêm/xóa các phần tử ở cả hai phía của phạm vi hiện tại, cho đến khi chúng ta biến nó thành phạm vi truy vấn. Bằng cách này, chúng ta chỉ cần thêm hoặc xóa một phần tử duy nhất một lần, đó phải là các thao tác khá dễ dàng trong cấu trúc dữ liệu của chúng ta.

Vì chúng ta thay đổi thứ tự trả lời các truy vấn, điều này chỉ có thể thực hiện được khi chúng ta được phép trả lời các truy vấn ở chế độ ngoại tuyến.

### 4.10.1 Triển khai

Trong thuật toán Mo, chúng ta sử dụng hai hàm để thêm một chỉ số và để xóa một chỉ số khỏi phạm vi mà chúng ta hiện đang duy trì.

```
void remove(idx);  // TODO: xóa giá trị tại idx khỏi cấu trúc dữ liệu
void add(idx);     // TODO: thêm giá trị tại idx vào cấu trúc dữ liệu
int get_answer();  // TODO: trích xuất câu trả lời hiện tại của cấu trúc dữ liệu

int block_size;

struct Query {
    int l, r, idx;
    bool operator<(Query other) const
    {
        return make_pair(l / block_size, r) <
               make_pair(other.l / block_size, other.r);
    }
};

vector<int> mo_s_algorithm(vector<Query> queries) {
    vector<int> answers(queries.size());
    sort(queries.begin(), queries.end());

    // TODO: khởi tạo cấu trúc dữ liệu

    int cur_l = 0;
    int cur_r = -1;
```

```
    // bất biến: cấu trúc dữ liệu sẽ luôn phản ánh phạm vi [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < q.r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < q.l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}
```

Dựa trên vấn đề, chúng ta có thể sử dụng một cấu trúc dữ liệu khác và sửa đổi các hàm add/remove/get_answer cho phù hợp. Ví dụ: nếu chúng ta được yêu cầu tìm các truy vấn tổng phạm vi, thì chúng ta sử dụng một số nguyên đơn giản làm cấu trúc dữ liệu, là 0 lúc ban đầu. Hàm add sẽ chỉ cần thêm giá trị của vị trí và sau đó cập nhật biến answer. Mặt khác, hàm remove sẽ trừ giá trị tại vị trí và sau đó cập nhật biến answer. Và get_answer chỉ trả về số nguyên.

Để trả lời các truy vấn mode, chúng ta có thể sử dụng cây tìm kiếm nhị phân (ví dụ: map<int, int>) để lưu trữ tần suất mỗi số xuất hiện trong phạm vi hiện tại và một cây tìm kiếm nhị phân thứ hai (ví dụ: set<pair<int, int») để giữ số lượng của các số (ví dụ: dưới dạng các cặp số lượng-số) theo thứ tự. Phương thức add loại bỏ số hiện tại khỏi BST thứ hai, tăng số lượng trong phương thức đầu tiên và chèn số trở lại vào phương thức thứ hai. remove thực hiện điều tương tự, nó chỉ giảm số lượng. Và get_answer chỉ nhìn vào cây thứ hai và trả về giá trị tốt nhất trong $O(1)$.

### 4.10.2 Độ phức tạp

Sắp xếp tất cả các truy vấn sẽ mất $O(Q \log Q)$.

Còn các thao tác khác thì sao? Hàm add và remove sẽ được gọi bao nhiêu lần?

Giả sử kích thước khối là $S$.

Nếu chúng ta chỉ xem xét tất cả các truy vấn có chỉ số bên trái trong cùng một khối, các truy vấn được sắp xếp theo chỉ số bên phải. Do đó, chúng ta sẽ chỉ gọi add(cur_r) và remove(cur_r) $O(N)$ lần cho tất cả các truy vấn này kết hợp. Điều này cho $O(\frac{N}{S}N)$ cuộc gọi cho tất cả các khối.

Giá trị của cur_l có thể thay đổi tối đa $O(S)$ trong khoảng giữa hai truy vấn. Do đó, chúng ta có thêm $O(SQ)$ cuộc gọi của add(cur_l) và remove(cur_l).

Đối với $S \approx \sqrt{N}$, điều này cho $O((N+Q)\sqrt{N})$ thao tác trong tổng số. Do đó, độ phức tạp là $O((N+Q)F\sqrt{N})$ trong đó $O(F)$ là độ phức tạp của hàm add và remove.

### 4.10.3 Lời khuyên để cải thiện thời gian chạy

Kích thước khối chính xác là $\sqrt{N}$ không phải lúc nào cũng mang lại thời gian chạy tốt nhất. Ví dụ: nếu $\sqrt{N} = 750$ thì có thể xảy ra trường hợp kích thước khối là 700 hoặc 800 có thể chạy tốt hơn. Quan trọng hơn, đừng tính toán kích thước khối tại thời gian chạy - hãy đặt nó là const. Phép chia cho các hằng số được trình biên dịch tối ưu hóa tốt. Trong các khối lẻ, hãy sắp xếp chỉ số bên phải theo thứ tự tăng dần và trong các khối chẵn, hãy sắp xếp nó theo thứ tự giảm dần. Điều này sẽ giảm thiểu chuyển động của con trỏ bên phải, vì việc sắp xếp thông thường sẽ di chuyển con trỏ bên phải từ cuối trở lại đầu ở đầu mỗi khối. Với phiên bản cải tiến, việc đặt lại này không còn cần thiết nữa.

```
bool cmp(pair<int, int> p, pair<int, int> q) {
    if (p.first / BLOCK_SIZE != q.first / BLOCK_SIZE)
        return p < q;
    return (p.first / BLOCK_SIZE & 1) ? (p.second < q.second) : (p.second > q.second)
}
```

Bạn có thể đọc về phương pháp sắp xếp nhanh hơn nữa tại đây.

## 4.11 Cây Đoạn (Segment Tree)

Cây Đoạn là một cấu trúc dữ liệu lưu trữ thông tin về các khoảng của mảng dưới dạng một cây. Điều này cho phép trả lời các truy vấn phạm vi trên một mảng một cách hiệu quả, đồng thời vẫn đủ linh hoạt để cho phép sửa đổi nhanh chóng mảng. Điều này bao gồm tìm tổng của các phần tử mảng liên tiếp $a[l \dots r]$, hoặc tìm phần tử tối thiểu trong một phạm vi như vậy trong thời gian $O(\log n)$. Giữa việc trả lời các truy vấn như vậy, Cây Đoạn cho phép sửa đổi mảng bằng cách thay thế một phần tử, hoặc thậm chí thay đổi các phần tử của toàn bộ đoạn con (ví dụ: gán tất cả các phần tử $a[l \dots r]$ cho bất kỳ giá trị nào, hoặc thêm một giá trị vào tất cả các phần tử trong đoạn con).

Nói chung, Cây Đoạn là một cấu trúc dữ liệu rất linh hoạt và một số lượng lớn các vấn đề có thể được giải quyết bằng nó. Ngoài ra, cũng có thể áp dụng các thao tác phức tạp

hơn và trả lời các truy vấn phức tạp hơn (xem Các phiên bản nâng cao của Cây Đoạn). Đặc biệt, Cây Đoạn có thể dễ dàng được tổng quát hóa cho các chiều lớn hơn. Ví dụ: với Cây Đoạn hai chiều, bạn có thể trả lời các truy vấn tổng hoặc tối thiểu trên một hình chữ nhật con của một ma trận đã cho chỉ trong thời gian $O(\log^2 n)$.

Một thuộc tính quan trọng của Cây Đoạn là chúng chỉ yêu cầu một lượng bộ nhớ tuyến tính. Cây Đoạn tiêu chuẩn yêu cầu $4n$ đỉnh để làm việc trên một mảng có kích thước $n$.

## 4.12 Dạng đơn giản nhất của Cây Đoạn

Để bắt đầu dễ dàng, chúng ta xem xét dạng đơn giản nhất của Cây Đoạn. Chúng ta muốn trả lời các truy vấn tổng một cách hiệu quả. Định nghĩa chính thức của nhiệm vụ của chúng ta là: Cho một mảng $a[0 \dots n-1]$, Cây Đoạn phải có khả năng tìm tổng các phần tử giữa các chỉ số $l$ và $r$ (tức là tính tổng $\sum_{i=l}^{r} a[i]$), và cũng xử lý việc thay đổi các giá trị của các phần tử trong mảng (tức là thực hiện các phép gán có dạng $a[i] = x$). Cây Đoạn phải có khả năng xử lý cả hai truy vấn trong thời gian $O(\log n)$.

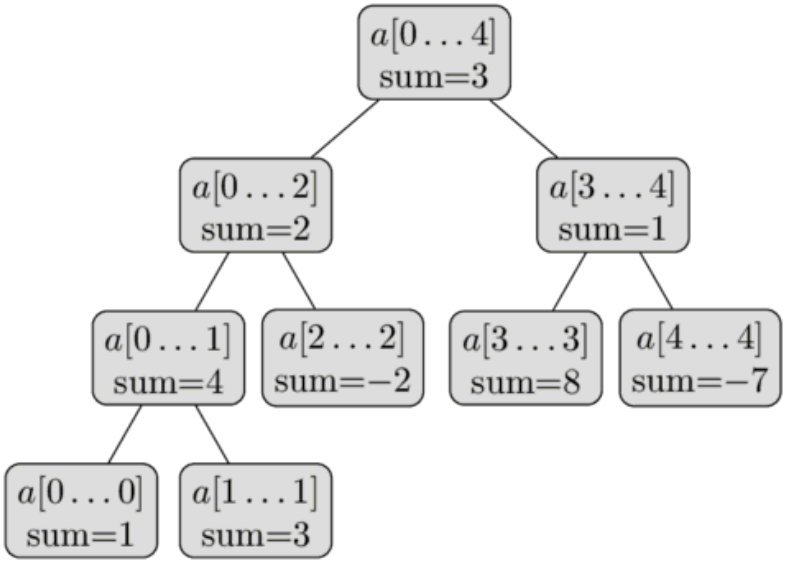
Đây là một cải tiến so với các phương pháp đơn giản hơn. Một triển khai mảng đơn giản - chỉ sử dụng một mảng đơn giản - có thể cập nhật các phần tử trong $O(1)$, nhưng yêu cầu $O(n)$ để tính toán mỗi truy vấn tổng. Và tổng tiền tố được tính toán trước có thể tính toán các truy vấn tổng trong $O(1)$, nhưng việc cập nhật một phần tử mảng yêu cầu $O(n)$ thay đổi đối với tổng tiền tố.

### 4.12.1 Cấu trúc của Cây Đoạn

Chúng ta có thể sử dụng phương pháp chia để trị khi nói đến các đoạn mảng. Chúng ta tính toán và lưu trữ tổng của các phần tử của toàn bộ mảng, tức là tổng của đoạn $a[0 \dots n-1]$. Sau đó, chúng ta chia mảng thành hai nửa $a[0 \dots n/2-1]$ và $a[n/2 \dots n-1]$ và tính tổng của mỗi nửa và lưu trữ chúng. Mỗi một trong hai nửa này lần lượt được chia đôi, v.v. cho đến khi tất cả các đoạn đạt đến kích thước 1.

Chúng ta có thể xem các đoạn này như tạo thành một cây nhị phân: gốc của cây này là đoạn $a[0 \dots n-1]$, và mỗi đỉnh (ngoại trừ các đỉnh lá) có chính xác hai đỉnh con. Đây là lý do tại sao cấu trúc dữ liệu được gọi là "Cây Đoạn", mặc dù trong hầu hết các triển khai, cây không được xây dựng một cách rõ ràng (xem Triển khai).

Đây là một biểu diễn trực quan của một Cây Đoạn như vậy trên mảng $a = [1, 3, -2, 8, -7]$:




Từ mô tả ngắn gọn này về cấu trúc dữ liệu, chúng ta đã có thể kết luận rằng một Cây Đoạn chỉ yêu cầu một số lượng đỉnh tuyến tính. Cấp độ đầu tiên của cây chứa một nút duy nhất (gốc), cấp độ thứ hai sẽ chứa hai đỉnh, trong cấp độ thứ ba nó sẽ chứa bốn đỉnh, cho đến khi số lượng đỉnh đạt đến $n$. Do đó, số lượng đỉnh trong trường hợp xấu nhất có thể được ước tính bằng tổng:

$$1 + 2 + 4 + \cdots + 2^{\lceil \log_2 n \rceil} < 2^{\lceil \log_2 n \rceil + 1} < 4n.$$

Điều đáng chú ý là bất cứ khi nào $n$ không phải là lũy thừa của hai, không phải tất cả các cấp độ của Cây Đoạn sẽ được lấp đầy hoàn toàn. Chúng ta có thể thấy hành vi đó trong hình ảnh. Hiện tại, chúng ta có thể quên đi thực tế này, nhưng nó sẽ trở nên quan trọng sau này trong quá trình triển khai.

Chiều cao của Cây Đoạn là $O(\log n)$, bởi vì khi đi xuống từ gốc đến các lá, kích thước của các đoạn giảm đi khoảng một nửa.

### 4.12.2 Xây dựng

Trước khi xây dựng cây đoạn, chúng ta cần quyết định:

- giá trị được lưu trữ tại mỗi nút của cây đoạn. Ví dụ: trong một cây đoạn tổng, một nút sẽ lưu trữ tổng của các phần tử trong phạm vi của nó $[l, r]$.
- thao tác hợp nhất hợp nhất hai nút anh em trong một cây đoạn. Ví dụ: trong một cây đoạn tổng, hai nút tương ứng với các phạm vi $a[l_1 \ldots r_1]$ và $a[l_2 \ldots r_2]$ sẽ được hợp nhất thành một nút tương ứng với phạm vi $a[l_1 \ldots r_2]$ bằng cách cộng các giá trị của hai nút.

Lưu ý rằng một đỉnh là một "đỉnh lá", nếu đoạn tương ứng của nó chỉ bao gồm một giá trị trong mảng ban đầu. Nó có mặt ở cấp độ thấp nhất của một cây đoạn. Giá trị của nó sẽ bằng phần tử (tương ứng) $a[i]$.

Bây giờ, để xây dựng cây đoạn, chúng ta bắt đầu ở cấp độ dưới cùng (các đỉnh lá) và gán cho chúng các giá trị tương ứng. Trên cơ sở các giá trị này, chúng ta có thể tính toán các giá trị của cấp độ trước đó, sử dụng hàm hợp nhất. Và trên cơ sở đó, chúng ta có thể tính toán các giá trị của cấp độ trước đó và lặp lại quy trình cho đến khi chúng ta đạt đến đỉnh gốc.

Thật thuận tiện để mô tả thao tác này một cách đệ quy theo hướng khác, tức là từ đỉnh gốc đến các đỉnh lá. Quy trình xây dựng, nếu được gọi trên một đỉnh không phải là lá, sẽ thực hiện như sau:

- xây dựng đệ quy các giá trị của hai đỉnh con
- hợp nhất các giá trị đã tính toán của các con này.

Chúng ta bắt đầu xây dựng tại đỉnh gốc và do đó, chúng ta có thể tính toán toàn bộ cây đoạn.

Độ phức tạp thời gian của việc xây dựng này là $O(n)$, giả sử rằng thao tác hợp nhất là thời gian không đổi (thao tác hợp nhất được gọi $n$ lần, bằng với số lượng nút bên trong trong cây đoạn).

### 4.12.3 Truy vấn tổng

Hiện tại, chúng ta sẽ trả lời các truy vấn tổng. Là một đầu vào, chúng ta nhận được hai số nguyên $l$ và $r$ và chúng ta phải tính tổng của đoạn $a[l \dots r]$ trong thời gian $O(\log n)$.

Để làm điều này, chúng ta sẽ duyệt Cây Đoạn và sử dụng các tổng được tính toán trước của các đoạn. Giả sử rằng chúng ta hiện đang ở đỉnh bao phủ đoạn $a[tl \dots tr]$. Có ba trường hợp có thể xảy ra.

Trường hợp dễ nhất là khi đoạn $a[l \dots r]$ bằng với đoạn tương ứng của đỉnh hiện tại (tức là $a[l \dots r] = a[tl \dots tr]$), thì chúng ta đã hoàn thành và có thể trả về tổng được tính toán trước được lưu trữ trong đỉnh.

Ngoài ra, đoạn của truy vấn có thể rơi hoàn toàn vào miền của con bên trái hoặc con bên phải. Nhớ lại rằng con bên trái bao phủ đoạn $a[tl \dots tm]$ và đỉnh bên phải bao phủ đoạn $a[tm + 1 \dots tr]$ với $tm = (tl + tr)/2$. Trong trường hợp này, chúng ta có thể chỉ cần đi đến đỉnh con, đoạn tương ứng bao phủ đoạn truy vấn và thực hiện thuật toán được mô tả ở đây với đỉnh đó.

Và sau đó có trường hợp cuối cùng, đoạn truy vấn giao với cả hai con. Trong trường hợp này, chúng ta không có lựa chọn nào khác ngoài việc thực hiện hai cuộc gọi đệ quy, một cho mỗi con. Đầu tiên, chúng ta đi đến con bên trái, tính toán một câu trả lời một phần cho đỉnh này (tức là tổng các giá trị của giao điểm giữa đoạn của truy vấn và

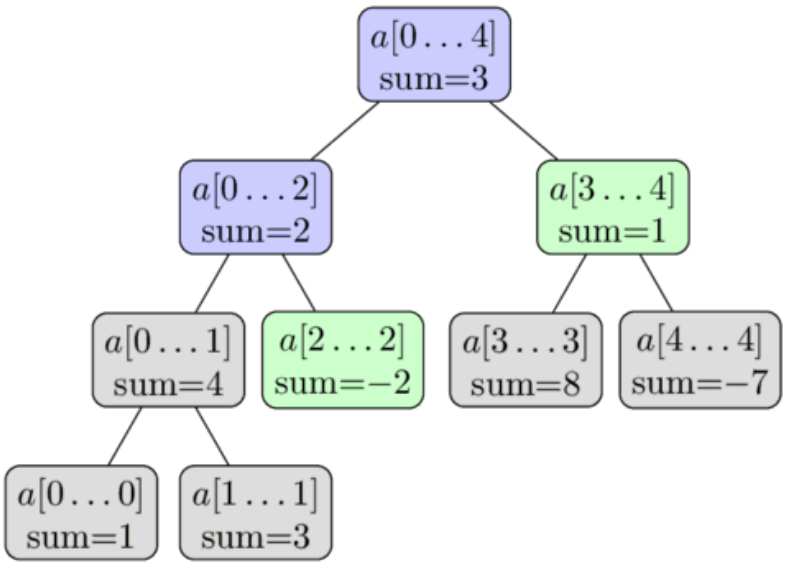
đoạn của con bên trái), sau đó đi đến con bên phải, tính toán câu trả lời một phần bằng cách sử dụng đỉnh đó và sau đó kết hợp các câu trả lời bằng cách thêm chúng. Nói cách khác, vì con bên trái đại diện cho đoạn $a[tl \dots tm]$ và con bên phải đại diện cho đoạn $a[tm+1 \dots tr]$, chúng ta tính toán truy vấn tổng $a[l \dots tm]$ bằng cách sử dụng con bên trái và truy vấn tổng $a[tm+1 \dots r]$ bằng cách sử dụng con bên phải.

Vì vậy, xử lý một truy vấn tổng là một hàm đệ quy gọi chính nó một lần với con bên trái hoặc con bên phải (mà không thay đổi ranh giới truy vấn), hoặc hai lần, một lần cho bên trái và một lần cho con bên phải (bằng cách chia truy vấn thành hai truy vấn con). Và đệ quy kết thúc, bất cứ khi nào ranh giới của đoạn truy vấn hiện tại trùng với ranh giới của đoạn của đỉnh hiện tại. Trong trường hợp đó, câu trả lời sẽ là giá trị được tính toán trước của tổng của đoạn này, được lưu trữ trong cây.

Nói cách khác, việc tính toán truy vấn là một sự duyệt cây, lan rộng qua tất cả các nhánh cần thiết của cây và sử dụng các giá trị tổng được tính toán trước của các đoạn trong cây.

Rõ ràng chúng ta sẽ bắt đầu duyệt từ đỉnh gốc của Cây Đoạn.

Quy trình được minh họa trong hình ảnh sau. Một lần nữa, mảng $a = [1, 3, -2, 8, -7]$ được sử dụng và ở đây chúng ta muốn tính tổng $\sum_{i=2}^{4} a[i]$. Các đỉnh màu sẽ được truy cập và chúng ta sẽ sử dụng các giá trị được tính toán trước của các đỉnh màu xanh lá cây. Điều này cho chúng ta kết quả $-2 + 1 = -1$.




Tại sao độ phức tạp của thuật toán này là $O(\log n)$? Để hiển thị độ phức tạp này, chúng ta xem xét từng cấp độ của cây. Hóa ra, đối với mỗi cấp độ, chúng ta chỉ truy cập không quá bốn đỉnh. Và vì chiều cao của cây là $O(\log n)$, chúng ta nhận được thời gian chạy mong muốn.

Chúng ta có thể chứng minh rằng mệnh đề này (tối đa bốn đỉnh mỗi cấp độ) là đúng bằng quy nạp. Ở cấp độ đầu tiên, chúng ta chỉ truy cập một đỉnh, đỉnh gốc, vì vậy ở đây chúng ta truy cập ít hơn bốn đỉnh. Bây giờ, hãy xem xét một cấp độ tùy ý. Theo giả thuyết quy nạp, chúng ta truy cập tối đa bốn đỉnh. Nếu chúng ta chỉ truy cập tối đa hai đỉnh, cấp độ tiếp theo có tối đa bốn đỉnh. Điều đó là tầm thường, bởi vì mỗi đỉnh chỉ có thể gây ra tối đa hai cuộc gọi đệ quy. Vì vậy, hãy giả sử rằng chúng ta truy cập ba hoặc bốn đỉnh ở cấp độ hiện tại. Từ những đỉnh đó, chúng ta sẽ phân tích các đỉnh ở giữa cẩn thận hơn. Vì truy vấn tổng yêu cầu tổng của một mảng con liên tục, chúng ta biết rằng các đoạn tương ứng với các đỉnh được truy cập ở giữa sẽ được bao phủ hoàn toàn bởi đoạn của truy vấn tổng. Do đó, các đỉnh này sẽ không thực hiện bất kỳ cuộc gọi đệ quy nào. Vì vậy, chỉ có đỉnh ngoài cùng bên trái và đỉnh ngoài cùng bên phải mới có khả năng thực hiện các cuộc gọi đệ quy. Và những điều đó sẽ chỉ tạo ra tối đa bốn cuộc gọi đệ quy, vì vậy cấp độ tiếp theo cũng sẽ thỏa mãn khẳng định. Chúng ta có thể nói rằng một nhánh tiếp cận ranh giới bên trái của truy vấn và nhánh thứ hai tiếp cận ranh giới bên phải.

Do đó, chúng ta truy cập tối đa $4 \log n$ đỉnh trong tổng số và điều đó bằng với thời gian chạy là $O(\log n)$.

Tóm lại, truy vấn hoạt động bằng cách chia đoạn đầu vào thành một số đoạn con mà tất cả các tổng đã được tính toán trước và lưu trữ trong cây. Và nếu chúng ta ngừng phân vùng bất cứ khi nào đoạn truy vấn trùng với đoạn đỉnh, thì chúng ta chỉ cần $O(\log n)$ các đoạn như vậy, điều này mang lại hiệu quả của Cây Đoạn.

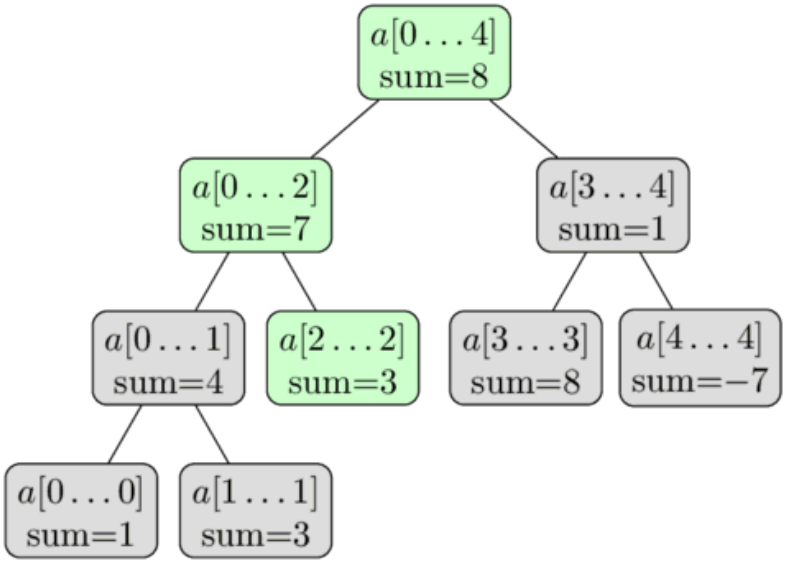
### 4.12.4 Truy vấn cập nhật

Bây giờ chúng ta muốn sửa đổi một phần tử cụ thể trong mảng, giả sử chúng ta muốn thực hiện phép gán $a[i] = x$. Và chúng ta phải xây dựng lại Cây Đoạn, sao cho nó tương ứng với mảng mới, đã sửa đổi.

Truy vấn này dễ hơn truy vấn tổng. Mỗi cấp độ của Cây Đoạn tạo thành một phân vùng của mảng. Do đó, một phần tử $a[i]$ chỉ đóng góp vào một đoạn từ mỗi cấp độ. Do đó, chỉ cần cập nhật $O(\log n)$ đỉnh.

Rất dễ thấy, yêu cầu cập nhật có thể được triển khai bằng một hàm đệ quy. Hàm được truyền đỉnh cây hiện tại và nó đệ quy gọi chính nó với một trong hai đỉnh con (đỉnh chứa $a[i]$ trong đoạn của nó) và sau đó tính toán lại giá trị tổng của nó, tương tự như cách nó được thực hiện trong phương thức xây dựng (tức là tổng của hai con của nó).

Một lần nữa, đây là một hình ảnh trực quan sử dụng cùng một mảng. Ở đây chúng ta thực hiện cập nhật $a[2] = 3$. Các đỉnh màu xanh lá cây là các đỉnh mà chúng ta truy cập và cập nhật.




## 4.13 Triển khai

Cân nhắc chính là cách lưu trữ Cây Đoạn. Tất nhiên, chúng ta có thể xác định một cấu trúc Vertex và tạo các đối tượng, lưu trữ ranh giới của đoạn, tổng của nó và ngoài ra còn có các con trỏ đến các đỉnh con của nó. Tuy nhiên, điều này đòi hỏi phải lưu trữ rất nhiều thông tin dư thừa dưới dạng con trỏ. Chúng ta sẽ sử dụng một thủ thuật đơn giản để làm cho điều này hiệu quả hơn nhiều bằng cách sử dụng một cấu trúc dữ liệu ngầm định: Chỉ lưu trữ các tổng trong một mảng. (Một phương pháp tương tự được sử dụng cho các heap nhị phân). Tổng của đỉnh gốc tại chỉ số 1, tổng của hai đỉnh con của nó tại các chỉ số 2 và 3, tổng của các con của hai đỉnh đó tại các chỉ số 4 đến 7, v.v. Với lập chỉ mục 1, một cách thuận tiện, con bên trái của một đỉnh tại chỉ số $i$ được lưu trữ tại chỉ số $2i$ và con bên phải tại chỉ số $2i + 1$. Tương đương, cha của một đỉnh tại chỉ số $i$ được lưu trữ tại $i/2$ (phép chia số nguyên).

Điều này đơn giản hóa rất nhiều việc triển khai. Chúng ta không cần lưu trữ cấu trúc của cây trong bộ nhớ. Nó được xác định một cách ngầm định. Chúng ta chỉ cần một mảng chứa tổng của tất cả các đoạn.

Như đã lưu ý trước đó, chúng ta cần lưu trữ tối đa $4n$ đỉnh. Nó có thể ít hơn, nhưng để thuận tiện, chúng ta luôn phân bổ một mảng có kích thước $4n$. Sẽ có một số phần tử trong mảng tổng, sẽ không tương ứng với bất kỳ đỉnh nào trong cây thực tế, nhưng điều này không làm phức tạp việc triển khai.

Vì vậy, chúng ta lưu trữ Cây Đoạn đơn giản là một mảng $t[]$ với kích thước gấp bốn lần kích thước đầu vào $n$:

```
int n, t[4*MAXN];
```

Quy trình xây dựng Cây Đoạn từ một mảng đã cho $a[]$ trông như thế này: nó là một hàm đệ quy với các tham số $a[]$ (mảng đầu vào), $v$ (chỉ số của đỉnh hiện tại) và các ranh giới $tl$ và $tr$ của đoạn hiện tại. Trong chương trình chính, hàm này sẽ được gọi với các tham số của đỉnh gốc: $v = 1$, $tl = 0$ và $tr = n - 1$.

```
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}
```

Hơn nữa, hàm trả lời các truy vấn tổng cũng là một hàm đệ quy, nhận làm tham số thông tin về đỉnh/đoạn hiện tại (tức là chỉ số $v$ và các ranh giới $tl$ và $tr$) và cả thông tin về ranh giới của truy vấn, $l$ và $r$. Để đơn giản hóa mã, hàm này luôn thực hiện hai cuộc gọi đệ quy, ngay cả khi chỉ cần một - trong trường hợp đó, cuộc gọi đệ quy thừa sẽ có $l > r$ và điều này có thể dễ dàng bị bắt bằng cách sử dụng một kiểm tra bổ sung ở đầu hàm.

```
int sum(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
           + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}
```

Cuối cùng là truy vấn cập nhật. Hàm này cũng sẽ nhận thông tin về đỉnh/đoạn hiện tại và ngoài ra còn có tham số của truy vấn cập nhật (tức là vị trí của phần tử và giá trị mới của nó).

```
void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
```

```
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}
```

## 4.14 Triển khai tiết kiệm bộ nhớ

Hầu hết mọi người sử dụng cách triển khai từ phần trước. Nếu bạn nhìn vào mảng t, bạn có thể thấy rằng nó tuân theo số thứ tự của các nút cây theo thứ tự duyệt BFS (duyệt theo cấp độ). Sử dụng phép duyệt này, các con của đỉnh $v$ lần lượt là $2v$ và $2v + 1$. Tuy nhiên, nếu $n$ không phải là lũy thừa của hai, phương pháp này sẽ bỏ qua một số chỉ số và để lại một số phần của mảng t không được sử dụng. Mức tiêu thụ bộ nhớ bị giới hạn bởi $4n$, mặc dù Cây Đoạn của một mảng gồm $n$ phần tử chỉ yêu cầu $2n - 1$ đỉnh.

Tuy nhiên, nó có thể được giảm bớt. Chúng ta đánh số lại các đỉnh của cây theo thứ tự duyệt Euler (duyệt trước), và chúng ta viết tất cả các đỉnh này cạnh nhau.

Hãy xem xét một đỉnh tại chỉ số $v$ và giả sử nó chịu trách nhiệm cho đoạn $[l, r]$ và giả sử $mid = \dfrac{l+r}{2}$. Rõ ràng là con bên trái sẽ có chỉ số $v + 1$. Con bên trái chịu trách nhiệm cho đoạn $[l, mid]$, tức là tổng cộng sẽ có $2 * (mid - l + 1) - 1$ đỉnh trong cây con của con bên trái. Do đó, chúng ta có thể tính toán chỉ số của con bên phải của $v$. Chỉ số sẽ là $v + 2 * (mid - l + 1)$. Bằng cách đánh số này, chúng ta đạt được mức giảm bộ nhớ cần thiết xuống còn $2n$.

## 4.15 Các phiên bản nâng cao của Cây Đoạn

Cây Đoạn là một cấu trúc dữ liệu rất linh hoạt, và nó cho phép nhiều biến thể và mở rộng theo nhiều hướng khác nhau. Hãy cùng thử phân loại chúng dưới đây.

### 4.15.1 Các truy vấn phức tạp hơn

Việc thay đổi Cây Đoạn theo một hướng nào đó để tính toán các truy vấn khác nhau (ví dụ: tính giá trị nhỏ nhất / lớn nhất thay vì tổng) có thể khá dễ dàng, nhưng đôi khi cũng rất không đơn giản.

### Tìm giá trị lớn nhất

Hãy thay đổi một chút điều kiện của bài toán đã mô tả trước đó: thay vì truy vấn tổng, chúng ta sẽ thực hiện các truy vấn tìm giá trị lớn nhất.

Cây sẽ có cấu trúc hoàn toàn giống với cây đã mô tả trước đó. Chúng ta chỉ cần thay đổi cách tính toán giá trị $t[v]$ trong các hàm build và update. Bây giờ $t[v]$ sẽ lưu trữ giá trị lớn nhất của đoạn tương ứng. Ngoài ra, chúng ta cũng cần thay đổi cách tính giá trị trả về của hàm sum (thay phép cộng bằng phép lấy giá trị lớn nhất).

Tất nhiên, bài toán này có thể dễ dàng được chuyển thành tính giá trị nhỏ nhất thay vì giá trị lớn nhất.

Thay vì đưa ra triển khai cho bài toán này, chúng ta sẽ cung cấp triển khai cho một phiên bản phức tạp hơn của bài toán này trong phần tiếp theo.

### Tìm giá trị lớn nhất và số lần xuất hiện của nó

Bài toán này rất giống với bài trước. Ngoài việc tìm giá trị lớn nhất, chúng ta còn phải tìm số lần xuất hiện của giá trị lớn nhất đó.

Để giải bài toán này, chúng ta lưu một cặp số tại mỗi đỉnh trong cây: Ngoài giá trị lớn nhất, chúng ta còn lưu số lần xuất hiện của nó trong đoạn tương ứng. Việc xác định cặp giá trị chính xác để lưu tại $t[v]$ vẫn có thể được thực hiện trong thời gian hằng số bằng cách sử dụng thông tin từ các cặp giá trị được lưu tại các đỉnh con. Việc kết hợp hai cặp giá trị như vậy nên được thực hiện trong một hàm riêng, vì đây sẽ là thao tác mà chúng ta thực hiện khi xây dựng cây, khi trả lời các truy vấn lớn nhất và khi thực hiện các sửa đổi.

```
pair<int, int> t[4*MAXN];

pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    if (a.first > b.first)
        return a;
    if (b.first > a.first)
        return b;
    return make_pair(a.first, a.second + b.second);
}

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = make_pair(a[tl], 1);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
```

```
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

pair<int, int> get_max(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return make_pair(-INF, 0);
    if (l == tl && r == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    return combine(get_max(v*2, tl, tm, l, min(r, tm)),
                   get_max(v*2+1, tm+1, tr, max(l, tm+1), r));
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = make_pair(new_val, 1);
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}
```

### Tính ước số chung lớn nhất / bội số chung nhỏ nhất

Trong bài toán này, chúng ta muốn tính ước số chung lớn nhất (GCD) / bội số chung nhỏ nhất (LCM) của tất cả các số trong các phạm vi đã cho của mảng.

Biến thể thú vị này của Cây Đoạn có thể được giải quyết theo cách hoàn toàn giống với các Cây Đoạn mà chúng ta đã suy ra cho các truy vấn tổng / nhỏ nhất / lớn nhất: chỉ cần lưu trữ GCD / LCM của đỉnh tương ứng trong mỗi đỉnh của cây. Việc kết hợp hai đỉnh có thể được thực hiện bằng cách tính GCD / LCM của cả hai đỉnh.

### Đếm số lượng số không, tìm số không thứ $k$

Trong bài toán này, chúng ta muốn tìm số lượng số không trong một phạm vi đã cho, và ngoài ra còn tìm chỉ số của số không thứ $k$ bằng một hàm thứ hai.

Một lần nữa, chúng ta phải thay đổi giá trị lưu trữ của cây một chút: Lần này chúng

ta sẽ lưu số lượng số không trong mỗi đoạn trong $t[]$. Rõ ràng là cách triển khai các hàm build, update và count_zero khá dễ dàng, chúng ta chỉ cần sử dụng ý tưởng từ bài toán truy vấn tổng. Như vậy, chúng ta đã giải quyết phần đầu tiên của bài toán.

Bây giờ chúng ta học cách giải bài toán tìm số không thứ $k$ trong mảng $a[]$. Để thực hiện nhiệm vụ này, chúng ta sẽ duyệt xuống Cây Đoạn, bắt đầu từ đỉnh gốc, và mỗi lần di chuyển sang con bên trái hoặc con bên phải, tùy thuộc vào đoạn nào chứa số không thứ $k$. Để quyết định xem cần đi đến con nào, chỉ cần nhìn vào số lượng số không xuất hiện trong đoạn tương ứng với đỉnh bên trái. Nếu số lượng được tính trước này lớn hơn hoặc bằng $k$, cần phải đi xuống con bên trái, nếu không thì đi xuống con bên phải. Lưu ý, nếu chúng ta chọn con bên phải, chúng ta phải trừ số lượng số không của con bên trái khỏi $k$.

Trong triển khai, chúng ta có thể xử lý trường hợp đặc biệt, khi $a[]$ chứa ít hơn $k$ số không, bằng cách trả về -1.

```
int find_kth(int v, int tl, int tr, int k) {
    if (k > t[v])
        return -1;
    if (tl == tr)
        return tl;
    int tm = (tl + tr) / 2;
    if (t[v*2] >= k)
        return find_kth(v*2, tl, tm, k);
    else
        return find_kth(v*2+1, tm+1, tr, k - t[v*2]);
}
```

### Tìm tiền tố mảng với tổng đã cho

Bài toán như sau: cho một giá trị $x$, chúng ta phải nhanh chóng tìm chỉ số nhỏ nhất $i$ sao cho tổng của $i$ phần tử đầu tiên của mảng $a[]$ lớn hơn hoặc bằng $x$ (giả sử mảng $a[]$ chỉ chứa các giá trị không âm).

Bài toán này có thể được giải bằng tìm kiếm nhị phân, tính tổng các tiền tố bằng Cây Đoạn. Tuy nhiên, điều này sẽ dẫn đến giải pháp có độ phức tạp $O(\log^2 n)$.

Thay vào đó, chúng ta có thể sử dụng cùng ý tưởng như trong phần trước, và tìm vị trí bằng cách duyệt xuống cây: mỗi lần di chuyển sang trái hoặc sang phải, tùy thuộc vào tổng của con bên trái. Do đó, tìm được câu trả lời trong thời gian $O(\log n)$.

### Tìm phần tử đầu tiên lớn hơn một giá trị đã cho

Bài toán như sau: cho một giá trị $x$ và một phạm vi $a[l \dots r]$, tìm $i$ nhỏ nhất trong phạm vi $a[l \dots r]$, sao cho $a[i]$ lớn hơn $x$.

Bài toán này có thể được giải bằng tìm kiếm nhị phân trên các truy vấn tiền tố lớn nhất với Cây Đoạn. Tuy nhiên, điều này sẽ dẫn đến giải pháp có độ phức tạp $O(\log^2 n)$.

Thay vào đó, chúng ta có thể sử dụng cùng ý tưởng như trong các phần trước, và tìm vị trí bằng cách duyệt xuống cây: mỗi lần di chuyển sang trái hoặc sang phải, tùy thuộc vào giá trị lớn nhất của con bên trái. Do đó, tìm được câu trả lời trong thời gian $O(\log n)$.

```
int get_first(int v, int tl, int tr, int l, int r, int x) {
    if(tl > r || tr < l) return -1;
    if(t[v] <= x) return -1;

    if (tl == tr) return tl;

    int tm = tl + (tr-tl)/2;
    int left = get_first(2*v, tl, tm, l, r, x);
    if(left != -1) return left;
    return get_first(2*v+1, tm+1, tr, l ,r, x);
}
```

### Tìm đoạn con có tổng lớn nhất

Ở đây, chúng ta lại nhận một phạm vi $a[l \dots r]$ cho mỗi truy vấn, lần này chúng ta phải tìm một đoạn con $a[l' \dots r']$ sao cho $l \leq l'$ và $r' \leq r$ và tổng các phần tử của đoạn này là lớn nhất. Như trước đây, chúng ta cũng muốn có khả năng sửa đổi từng phần tử riêng lẻ của mảng. Các phần tử của mảng có thể âm, và đoạn con tối ưu có thể rỗng (ví dụ: nếu tất cả các phần tử đều âm).

Bài toán này là một cách sử dụng không tầm thường của Cây Đoạn. Lần này, chúng ta sẽ lưu bốn giá trị cho mỗi đỉnh: tổng của đoạn, tổng tiền tố lớn nhất, tổng hậu tố lớn nhất, và tổng của đoạn con tối ưu trong đó. Nói cách khác, với mỗi đoạn của Cây Đoạn, câu trả lời đã được tính toán trước cũng như các câu trả lời cho các đoạn chạm vào ranh giới trái và phải của đoạn.

Làm thế nào để xây dựng một cây với dữ liệu như vậy? Một lần nữa, chúng ta tính toán nó theo cách đệ quy: đầu tiên chúng ta tính toán tất cả bốn giá trị cho con bên trái và con bên phải, sau đó kết hợp chúng để đạt được bốn giá trị cho đỉnh hiện tại. Lưu ý rằng câu trả lời cho đỉnh hiện tại là một trong các giá trị sau:

- Câu trả lời của con bên trái, nghĩa là đoạn con tối ưu hoàn toàn nằm trong đoạn của con bên trái. - Câu trả lời của con bên phải, nghĩa là đoạn con tối ưu hoàn toàn nằm trong đoạn của con bên phải. - Tổng của tổng hậu tố lớn nhất của con bên trái và tổng tiền tố lớn nhất của con bên phải, nghĩa là đoạn con tối ủy giao với cả hai con.

Do đó, câu trả lời cho đỉnh hiện tại là giá trị lớn nhất trong ba giá trị này. Việc tính toán tổng tiền tố / hậu tố lớn nhất thậm chí còn dễ hơn. Dưới đây là triển khai của hàm

combine, hàm này chỉ nhận dữ liệu từ con bên trái và con bên phải, và trả về dữ liệu của đỉnh hiện tại.

```
struct data {
    int sum, pref, suff, ans;
};

data combine(data l, data r) {
    data res;
    res.sum = l.sum + r.sum;
    res.pref = max(l.pref, l.sum + r.pref);
    res.suff = max(r.suff, r.sum + l.suff);
    res.ans = max(max(l.ans, r.ans), l.suff + r.pref);
    return res;
}
```

Sử dụng hàm combine, việc xây dựng Cây Đoạn trở nên dễ dàng. Chúng ta có thể triển khai nó theo cách hoàn toàn giống với các triển khai trước đó. Để khởi tạo các đỉnh lá, chúng ta thêm vào hàm phụ trợ make_data, hàm này sẽ trả về một đối tượng data chứa thông tin của một giá trị đơn lẻ.

```
data make_data(int val) {
    data res;
    res.sum = val;
    res.pref = res.suff = res.ans = max(0, val);
    return res;
}

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = make_data(a[tl]);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
```

```
        t[v] = make_data(new_val);
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}
```

Chỉ còn lại việc tính toán câu trả lời cho một truy vấn. Để trả lời, chúng ta đi xuống cây như trước, chia truy vấn thành nhiều đoạn con trùng với các đoạn của Cây Đoạn, và kết hợp các câu trả lời trong chúng thành một câu trả lời duy nhất cho truy vấn. Sau đó, rõ ràng là công việc hoàn toàn giống với Cây Đoạn đơn giản, nhưng thay vì cộng / tìm nhỏ nhất / tìm lớn nhất các giá trị, chúng ta sử dụng hàm combine.

```
data query(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return make_data(0);
    if (l == tl && r == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    return combine(query(v*2, tl, tm, l, min(r, tm)),
                   query(v*2+1, tm+1, tr, max(l, tm+1), r));
}
```

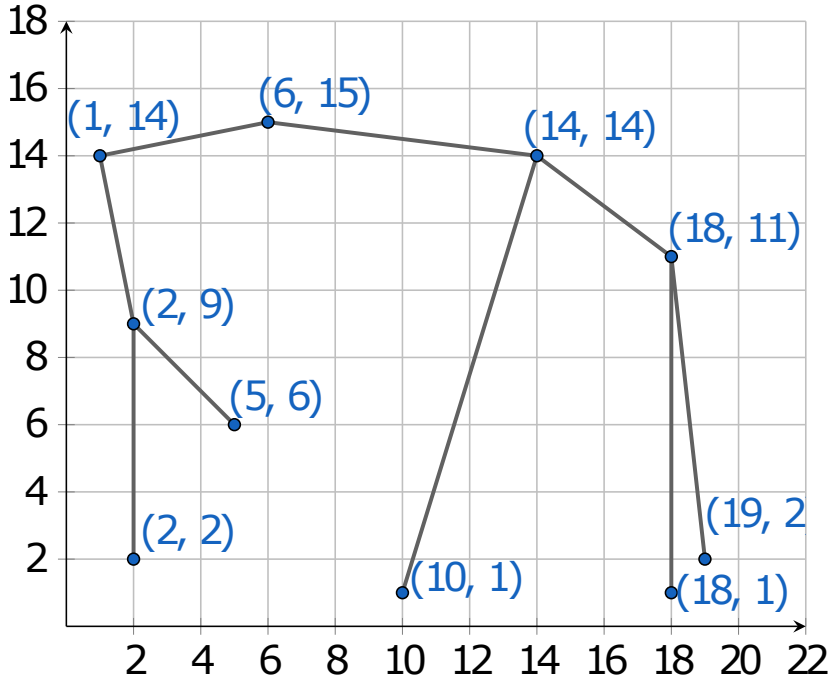
## 4.16 Treap (Cây Descartes)

Treap là một cấu trúc dữ liệu kết hợp giữa cây nhị phân và đống nhị phân (tên gọi bắt nguồn từ: tree + heap ⇒ Treap).

Cụ thể hơn, treap là một cấu trúc dữ liệu lưu trữ các cặp $(X, Y)$ trong một cây nhị phân sao cho nó vừa là một cây tìm kiếm nhị phân theo $X$ vừa là một đống nhị phân theo $Y$. Nếu một nút của cây chứa giá trị $(X_0, Y_0)$, tất cả các nút trong cây con bên trái có $X \leq X_0$, tất cả các nút trong cây con bên phải có $X_0 \leq X$, và tất cả các nút trong cả hai cây con trái và phải có $Y \leq Y_0$.

Treap còn thường được gọi là "cây Descartes", vì nó dễ dàng được nhúng vào một mặt phẳng Descartes.

Treap được đề xuất bởi Raimund Siedel và Cecilia Aragon vào năm 1989.




Hình 4.3: Treap

### 4.16.1 Lợi ích của cách tổ chức dữ liệu này

Trong cách triển khai này, các giá trị $X$ là các khóa (và đồng thời là các giá trị được lưu trong treap), còn các giá trị $Y$ được gọi là độ ưu tiên. Nếu không có độ ưu tiên, treap sẽ chỉ là một cây tìm kiếm nhị phân thông thường theo $X$, và một tập hợp các giá trị $X$ có thể tương ứng với nhiều cây khác nhau, một số trong đó có thể bị thoái hóa (ví dụ, dưới dạng danh sách liên kết), dẫn đến hiệu suất cực kỳ chậm (các thao tác chính sẽ có độ phức tạp $O(N)$).

Đồng thời, các độ ưu tiên (khi chúng là duy nhất) cho phép xác định duy nhất cây sẽ được xây dựng (tất nhiên, điều này không phụ thuộc vào thứ tự thêm các giá trị), điều này có thể được chứng minh bằng định lý tương ứng. Rõ ràng, nếu bạn chọn các độ ưu tiên một cách ngẫu nhiên, trung bình bạn sẽ nhận được các cây không thoái hóa, đảm bảo độ phức tạp $O(\log N)$ cho các thao tác chính. Do đó, treap còn có tên gọi khác là cây tìm kiếm nhị phân ngẫu nhiên.

### 4.16.2 Các thao tác

Treap cung cấp các thao tác sau:

- **Chèn (X, Y)** trong $O(\log N)$. Thêm một nút mới vào cây. Một biến thể khả thi là

chỉ truyền $X$ và tạo $Y$ ngẫu nhiên bên trong thao tác.

- **Tìm kiếm (X)** trong $O(\log N)$. Tìm một nút với giá trị khóa đã chỉ định $X$. Cách triển khai giống như đối với cây tìm kiếm nhị phân thông thường.
- **Xóa (X)** trong $O(\log N)$. Tìm một nút với giá trị khóa đã chỉ định $X$ và xóa nó khỏi cây.
- **Xây dựng** $(X_1, ..., X_N)$ trong $O(N)$. Xây dựng một cây từ danh sách các giá trị. Điều này có thể được thực hiện trong thời gian tuyến tính (giả sử $X_1, ..., X_N$ đã được sắp xếp).
- **Hợp** $(T_1, T_2)$ trong $O(M \log(N/M))$. Gộp hai cây, giả sử tất cả các phần tử là khác nhau. Có thể đạt được độ phức tạp tương tự nếu các phần tử trùng lặp cần được loại bỏ trong quá trình gộp.
- **Giao** $(T_1, T_2)$ trong $O(M \log(N/M))$. Tìm giao của hai cây (tức là các phần tử chung của chúng). Chúng ta sẽ không xem xét việc triển khai thao tác này ở đây.

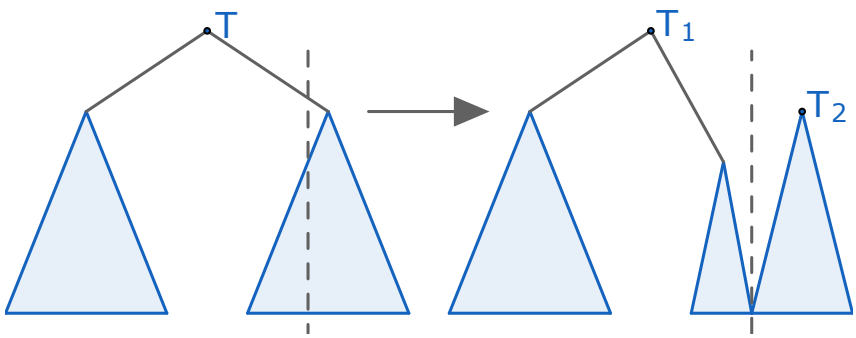
Ngoài ra, do treap là một cây tìm kiếm nhị phân, nó có thể thực hiện các thao tác khác, chẳng hạn như tìm phần tử lớn thứ $K$ hoặc tìm chỉ số của một phần tử.

### 4.16.3 Mô tả triển khai

Về mặt triển khai, mỗi nút chứa $X$, $Y$ và các con trỏ đến cây con bên trái ($L$) và bên phải ($R$).

Chúng ta sẽ triển khai tất cả các thao tác cần thiết chỉ bằng hai thao tác phụ trợ: *Split* và *Merge*.

**Split (Chia)**




Hình 4.4: Treap Split

*Split* $(T, X)$ tách cây $T$ thành 2 cây con $L$ và $R$ (là giá trị trả về của *split*) sao cho $L$ chứa tất cả các phần tử có khóa $X_L \leq X$, và $R$ chứa tất cả các phần tử có khóa $X_R > X$. Thao tác này có độ phức tạp $O(\log N)$ và được triển khai bằng đệ quy đơn giản:

- Nếu giá trị của nút gốc $(R) \leq X$, thì $L$ sẽ bao gồm ít nhất $R \to L$ và $R$. Sau đó, gọi *split* trên $R \to R$, ghi nhận kết quả chia là $L'$ và $R'$. Cuối cùng, $L$ cũng chứa $L'$, còn $R = R'$.

- Nếu giá trị của nút gốc $(R) > X$, thì $R$ sẽ bao gồm ít nhất $R$ và $R \to R$. Sau đó, gọi *split* trên $R \to L$, ghi nhận kết quả chia là $L'$ và $R'$. Cuối cùng, $L = L'$, còn $R$ cũng chứa $R'$.

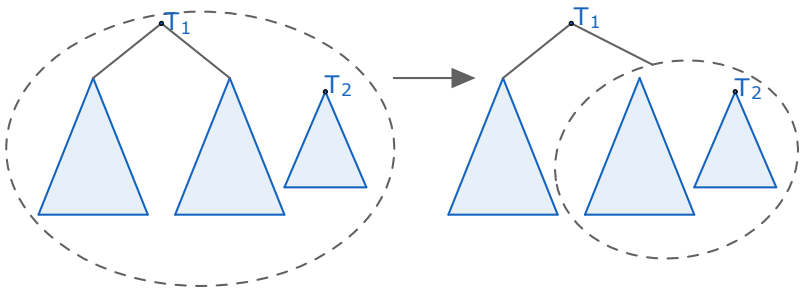
Do đó, thuật toán *split* là:

- Quyết định nút gốc thuộc về cây con nào (trái hay phải).

- Gọi đệ quy *split* trên một trong các cây con của nó.

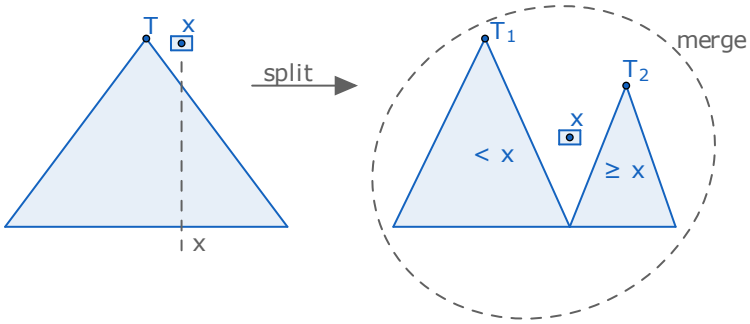
- Tạo kết quả cuối cùng bằng cách tái sử dụng lời gọi *split* đệ quy.

**Merge (Gộp)**




Hình 4.5: Treap Merge

*Merge* $(T_1, T_2)$ kết hợp hai cây con $T_1$ và $T_2$ và trả về cây mới. Thao tác này cũng có độ phức tạp $O(\log N)$. Nó hoạt động với giả định rằng $T_1$ và $T_2$ được sắp xếp (tất cả các khóa $X$ trong $T_1$ nhỏ hơn các khóa trong $T_2$). Do đó, chúng ta cần kết hợp các cây này mà không vi phạm thứ tự ưu tiên $Y$. Để làm điều này, chúng ta chọn làm gốc cây có độ ưu tiên $Y$ cao hơn tại nút gốc, và gọi đệ quy *Merge* cho cây còn lại và cây con tương ứng của nút gốc được chọn.



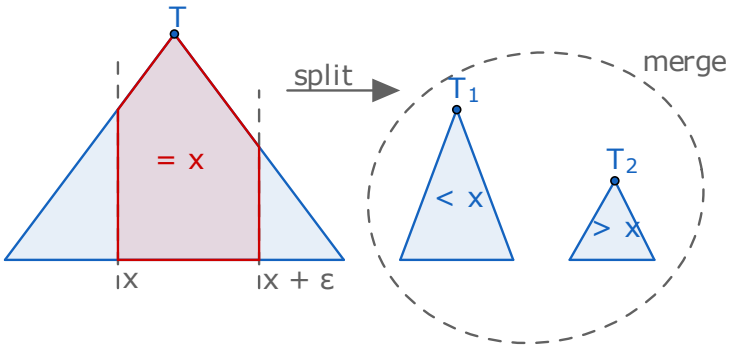

Hình 4.6: Treap Insert

### Chèn

Việc triển khai *Insert* $(X, Y)$ giờ đây trở nên rõ ràng. Đầu tiên, chúng ta đi xuống cây (như trong cây tìm kiếm nhị phân thông thường theo $X$), và dừng lại tại nút đầu tiên mà giá trị ưu tiên nhỏ hơn $Y$. Chúng ta đã tìm thấy vị trí để chèn phần tử mới. Tiếp theo, gọi *Split (T, X)* trên cây con bắt đầu từ nút được tìm thấy, và sử dụng các cây con trả về $L$ và $R$ làm cây con trái và phải của nút mới.

Ngoài ra, việc chèn có thể được thực hiện bằng cách chia treap ban đầu theo $X$ và thực hiện 2 lần *merge* với nút mới.

### Xóa




Hình 4.7: Treap Erase

Việc triển khai *Erase* $(X)$ cũng rõ ràng. Đầu tiên, chúng ta đi xuống cây (như trong cây tìm kiếm nhị phân thông thường theo $X$), tìm phần tử cần xóa. Khi nút được tìm thấy, gọi *Merge* trên các cây con của nó và đặt giá trị trả về của thao tác vào vị trí của phần tử đang xóa.

Ngoài ra, chúng ta có thể tách cây con chứa $X$ bằng 2 thao tác *split* và gộp các treap còn lại.

**Xây dựng**

Chúng ta triển khai thao tác *Build* với độ phức tạp $O(N \log N)$ bằng cách sử dụng $N$ lời gọi *Insert*.

**Hợp**

*Union* $(T_1, T_2)$ có độ phức tạp lý thuyết $O(M \log(N/M))$, nhưng trong thực tế, nó hoạt động rất tốt, có lẽ với một hằng số ẩn rất nhỏ. Giả sử không mất tính tổng quát rằng $T_1 \to Y > T_2 \to Y$, tức là gốc của $T_1$ sẽ là gốc của kết quả. Để có được kết quả, chúng ta cần gộp các cây $T_1 \to L$, $T_1 \to R$ và $T_2$ thành hai cây có thể là cây con của gốc $T_1$. Để làm điều này, gọi *Split* $(T_2, T_1 \to X)$, từ đó chia $T_2$ thành hai phần $L$ và $R$, sau đó kết hợp đệ quy với các cây con của $T_1$: *Union* $(T_1 \to L, L)$ và *Union* $(T_1 \to R, R)$, từ đó nhận được các cây con trái và phải của kết quả.

### 4.16.4 Triển khai

```
struct item {
    int key, prior;
    item *l, *r;
    item () { }
    item (int key) : key(key), prior(rand()), l(NULL), r(NULL) { }
    item (int key, int prior) : key(key), prior(prior), l(NULL), r(NULL) { }
};
typedef item* pitem;

void split (pitem t, int key, pitem & l, pitem & r) {
    if (!t)
        l = r = NULL;
    else if (t->key <= key)
        split (t->r, key, t->r, r),  l = t;
    else
        split (t->l, key, l, t->l),  r = t;
}

void insert (pitem & t, pitem it) {
    if (!t)
        t = it;
```

```
    else if (it->prior > t->prior)
        split (t, it->key, it->l, it->r),  t = it;
    else
        insert (t->key <= it->key ? t->r : t->l, it);
}

void merge (pitem & t, pitem l, pitem r) {
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r),  t = l;
    else
        merge (r->l, l, r->l),  t = r;
}

void erase (pitem & t, int key) {
    if (t->key == key) {
        pitem th = t;
        merge (t, t->l, t->r);
        delete th;
    }
    else
        erase (key < t->key ? t->l : t->r, key);
}

pitem unite (pitem l, pitem r) {
    if (!l || !r)  return l ? l : r;
    if (l->prior < r->prior)  swap (l, r);
    pitem lt, rt;
    split (r, l->key, lt, rt);
    l->l = unite (l->l, lt);
    l->r = unite (l->r, rt);
    return l;
}
```

## 4.17 Treap Ngầm

Treap ngầm là một biến thể đơn giản của treap thông thường, một cấu trúc dữ liệu rất mạnh mẽ. Trên thực tế, treap ngầm có thể được xem như một mảng với các thao tác sau được triển khai (tất cả đều trong $O(\log N)$ ở chế độ trực tuyến):

- Chèn một phần tử vào mảng ở bất kỳ vị trí nào.
- Xóa một phần tử bất kỳ.
- Tìm tổng, phần tử nhỏ nhất / lớn nhất, v.v. trên một khoảng bất kỳ.
- Cộng, tô màu trên một khoảng bất kỳ.
- Đảo ngược các phần tử trên một khoảng bất kỳ.

Ý tưởng là các khóa (key) sẽ là các chỉ số dựa trên số 0 của các phần tử trong mảng. Tuy nhiên, chúng ta sẽ không lưu trữ các giá trị này một cách rõ ràng (nếu không, ví dụ, việc chèn một phần tử sẽ gây ra thay đổi khóa trong $O(N)$ nút của cây).

Lưu ý rằng khóa của một nút là số lượng các nút nhỏ hơn nó (các nút này có thể xuất hiện không chỉ trong cây con bên trái của nó mà còn trong các cây con bên trái của các tổ tiên của nó). Cụ thể hơn, khóa ngầm cho một nút $T$ là số lượng đỉnh $cnt(T \to L)$ trong cây con bên trái của nút này cộng với các giá trị tương tự $cnt(P \to L) + 1$ cho mỗi tổ tiên $P$ của nút $T$, nếu $T$ nằm trong cây con bên phải của $P$.

Giờ đây, ta có thể thấy cách tính khóa ngầm của nút hiện tại một cách nhanh chóng. Vì trong tất cả các thao tác, chúng ta đến bất kỳ nút nào bằng cách đi xuống cây, ta chỉ cần tích lũy tổng này và truyền nó vào hàm. Nếu ta đi vào cây con bên trái, tổng tích lũy không thay đổi; nếu ta đi vào cây con bên phải, nó tăng thêm $cnt(T \to L) + 1$.

Dưới đây là triển khai mới của các hàm `Split` và `Merge`:

```
void merge (pitem & t, pitem l, pitem r) {
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r),  t = l;
    else
        merge (r->l, l, r->l),  t = r;
    upd_cnt (t);
}

void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
    if (!t)
        return void( l = r = 0 );
    int cur_key = add + cnt(t->l); // khóa ngầm
    if (key <= cur_key)
        split (t->l, l, t->l, key, add),  r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)),  l = t;
```

```
    upd_cnt (t);
}
```

Trong triển khai trên, sau khi gọi $split(T, T_1, T_2, k)$, cây $T_1$ sẽ chứa $k$ phần tử đầu tiên của $T$ (tức là các phần tử có khóa ngầm nhỏ hơn $k$), và $T_2$ sẽ chứa tất cả các phần tử còn lại.

Bây giờ, hãy xem xét triển khai các thao tác khác nhau trên treap ngầm:

- **Chèn phần tử.** Giả sử chúng ta cần chèn một phần tử tại vị trí $pos$. Ta chia treap thành hai phần, tương ứng với các mảng $[0..pos-1]$ và $[pos..sz]$; để làm điều này, ta gọi $split(T, T_1, T_2, pos)$. Sau đó, ta có thể kết hợp cây $T_1$ với nút mới bằng cách gọi $merge(T_1, T_1, \text{new item})$ (dễ thấy rằng tất cả các điều kiện tiên quyết đều được đáp ứng). Cuối cùng, ta kết hợp lại các cây $T_1$ và $T_2$ thành $T$ bằng cách gọi $merge(T, T_1, T_2)$.

- **Xóa phần tử.** Thao tác này thậm chí còn đơn giản hơn: tìm phần tử cần xóa $T$, thực hiện hợp nhất các con của nó $L$ và $R$, và thay thế phần tử $T$ bằng kết quả của việc hợp nhất. Thực tế, việc xóa phần tử trong treap ngầm hoàn toàn giống với treap thông thường.

- **Tìm tổng / nhỏ nhất, v.v. trên khoảng.** Đầu tiên, tạo một trường bổ sung $F$ trong cấu trúc mục để lưu giá trị của hàm mục tiêu cho cây con của nút này. Trường này dễ duy trì tương tự như duy trì kích thước của các cây con: tạo một hàm tính giá trị này cho một nút dựa trên các giá trị cho các con của nó và thêm các lệnh gọi hàm này vào cuối tất cả các hàm sửa đổi cây. Thứ hai, ta cần biết cách xử lý truy vấn cho một khoảng bất kỳ $[A; B]$. Để lấy một phần của cây tương ứng với khoảng $[A; B]$, ta cần gọi $split(T, T_2, T_3, B+1)$, rồi $split(T_2, T_1, T_2, A)$: sau đó $T_2$ sẽ chứa tất cả các phần tử trong khoảng $[A; B]$, và chỉ có chúng. Do đó, câu trả lời cho truy vấn sẽ được lưu trong trường $F$ của gốc của $T_2$. Sau khi trả lời truy vấn, cây phải được khôi phục bằng cách gọi $merge(T, T_1, T_2)$ và $merge(T, T, T_3)$.

- **Cộng / tô màu trên khoảng.** Ta thực hiện tương tự như đoạn trước, nhưng thay vì trường $F$, ta sẽ lưu một trường `add` chứa giá trị được cộng thêm cho cây con (hoặc giá trị mà cây con được tô). Trước khi thực hiện bất kỳ thao tác nào, ta phải "đẩy" giá trị này một cách chính xác - tức là thay đổi $T \to L \to add$ và $T \to R \to add$, đồng thời xóa trường `add` trong nút cha. Bằng cách này, sau bất kỳ thay đổi nào trên cây, thông tin sẽ không bị mất.

- **Đảo ngược trên khoảng.** Điều này cũng tương tự như thao tác trước: ta phải thêm cờ boolean `rev` và đặt nó thành `true` khi cây con của nút hiện tại cần được đảo ngược. Việc "đẩy" giá trị này hơi phức tạp - ta hoán đổi các con của nút này và đặt cờ này thành `true` cho chúng.

Dưới đây là một triển khai ví dụ của treap ngầm với đảo ngược trên khoảng. Với mỗi nút, ta lưu trường `value` là giá trị thực tế của phần tử mảng tại vị trí hiện tại. Ta cũng cung cấp triển khai của hàm `output()`, hàm này xuất ra mảng tương ứng với trạng thái hiện tại của treap ngầm.

```
typedef struct item * pitem;
struct item {
    int prior, value, cnt;
    bool rev;
    pitem l, r;
};

int cnt (pitem it) {
    return it ? it->cnt : 0;
}

void upd_cnt (pitem it) {
    if (it)
        it->cnt = cnt(it->l) + cnt(it->r) + 1;
}

void push (pitem it) {
    if (it && it->rev) {
        it->rev = false;
        swap (it->l, it->r);
        if (it->l)  it->l->rev ^= true;
        if (it->r)  it->r->rev ^= true;
    }
}

void merge (pitem & t, pitem l, pitem r) {
    push (l);
    push (r);
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r),  t = l;
    else
        merge (r->l, l, r->l),  t = r;
    upd_cnt (t);
}
```

```
void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
    if (!t)
        return void( l = r = 0 );
    push (t);
    int cur_key = add + cnt(t->l);
    if (key <= cur_key)
        split (t->l, l, t->l, key, add),  r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)),  l = t;
    upd_cnt (t);
}

void reverse (pitem t, int l, int r) {
    pitem t1, t2, t3;
    split (t, t1, t2, l);
    split (t2, t2, t3, r-l+1);
    t2->rev ^= true;
    merge (t, t1, t2);
    merge (t, t, t3);
}

void output (pitem t) {
    if (!t)  return;
    push (t);
    output (t->l);
    printf ("%d ", t->value);
    output (t->r);
}
```

## 4.18 Tài liệu tham khảo

- Blelloch, Reid-Miller "Fast Set Operations Using Treaps"

## 4.19 Cây Căn Bậc Hai

Cho một mảng $a$ chứa $n$ phần tử và một phép toán $\circ$ thỏa mãn tính chất kết hợp: $(x \circ y) \circ z = x \circ (y \circ z)$ đúng với mọi $x$, $y$, $z$.

Vì vậy, các phép toán như gcd, min, max, +, and, or, xor, v.v. đều thỏa mãn các điều

kiện này.

Ngoài ra, chúng ta có một số truy vấn $q(l, r)$. Với mỗi truy vấn, ta cần tính $a_l \circ a_{l+1} \circ \cdots \circ a_r$.

Cây Căn Bậc Hai có thể xử lý các truy vấn này trong thời gian $O(1)$ với thời gian tiền xử lý $O(n \cdot \log\log n)$ và bộ nhớ $O(n \cdot \log\log n)$.

### 4.19.1 Mô tả

**Xây dựng phân chia căn bậc hai**

Hãy thực hiện phân chia căn bậc hai. Chúng ta chia mảng thành $\sqrt{n}$ khối, mỗi khối có kích thước $\sqrt{n}$. Với mỗi khối, ta tính:

- Câu trả lời cho các truy vấn nằm trong khối và bắt đầu từ đầu khối (prefixOp).
- Câu trả lời cho các truy vấn nằm trong khối và kết thúc ở cuối khối (suffixOp).

Và ta sẽ tính thêm một mảng:

between$_{i,j}$ (với $i \le j$) - câu trả lời cho truy vấn bắt đầu từ đầu khối $i$ và kết thúc ở cuối khối $j$. Lưu ý rằng ta có $\sqrt{n}$ khối, nên kích thước của mảng này sẽ là $O(\sqrt{n}^2) = O(n)$.

Hãy xem ví dụ.

Giả sử $\circ$ là $+$ (ta tính tổng trên một đoạn) và ta có mảng $a$ sau:

{1, 2, 3, 4, 5, 6, 7, 8, 9}

Nó sẽ được chia thành ba khối: {1, 2, 3}, {4, 5, 6} và {7, 8, 9}.

Đối với khối đầu tiên, prefixOp là {1, 3, 6} và suffixOp là {6, 5, 3}.

Đối với khối thứ hai, prefixOp là {4, 9, 15} và suffixOp là {15, 11, 6}.

Đối với khối thứ ba, prefixOp là {7, 15, 24} và suffixOp là {24, 17, 9}.

Mảng between là:

$$\{6, 21, 45\},$$
$$\{0, 15, 39\},$$
$$\{0, 0, 24\}$$

(chúng ta giả sử các phần tử không hợp lệ khi $i > j$ được điền bằng số 0).

Rõ ràng, các mảng này có thể được tính dễ dàng trong thời gian và bộ nhớ $O(n)$.

Chúng ta đã có thể trả lời một số truy vấn bằng cách sử dụng các mảng này. Nếu truy vấn không nằm gọn trong một khối, ta có thể chia nó thành ba phần: hậu tố của một khối, sau đó một đoạn gồm các khối liền kề, và cuối cùng là tiền tố của một khối nào đó. Ta có thể trả lời truy vấn bằng cách chia nó thành ba phần và lấy phép toán của một giá trị từ suffixOp, rồi một giá trị từ between, rồi một giá trị từ prefixOp.

Nhưng nếu ta có các truy vấn hoàn toàn nằm trong một khối, ta không thể xử lý chúng bằng ba mảng này. Vì vậy, ta cần làm thêm điều gì đó.

### Tạo một cây

Chúng ta không thể trả lời các truy vấn hoàn toàn nằm trong một khối. Nhưng nếu ta xây dựng cùng một cấu trúc như đã mô tả ở trên cho mỗi khối thì sao? Đúng vậy, ta có thể làm điều đó. Và ta thực hiện nó một cách đệ quy, cho đến khi đạt kích thước khối là 1 hoặc 2. Câu trả lời cho các khối như vậy có thể được tính dễ dàng trong $O(1)$.

Vậy là ta có một cây. Mỗi nút của cây đại diện cho một đoạn của mảng. Nút đại diện cho đoạn mảng có kích thước $k$ sẽ có $\sqrt{k}$ con - cho mỗi khối. Ngoài ra, mỗi nút chứa ba mảng đã mô tả ở trên cho đoạn mà nó chứa. Gốc của cây đại diện cho toàn bộ mảng. Các nút có độ dài đoạn 1 hoặc 2 là các lá.

Cũng rõ ràng rằng chiều cao của cây này là $O(\log\log n)$, vì nếu một đỉnh của cây đại diện cho mảng có độ dài $k$, thì các con của nó có độ dài $\sqrt{k}$. $\log(\sqrt{k}) = \frac{\log k}{2}$, nên $\log k$ giảm đi hai lần qua mỗi tầng của cây và do đó chiều cao của nó là $O(\log\log n)$. Thời gian xây dựng và sử dụng bộ nhớ sẽ là $O(n \cdot \log\log n)$, vì mỗi phần tử của mảng xuất hiện chính xác một lần trên mỗi tầng của cây.

Giờ đây, ta có thể trả lời các truy vấn trong $O(\log\log n)$. Ta có thể đi xuống cây cho đến khi gặp một đoạn có độ dài 1 hoặc 2 (câu trả lời cho nó có thể được tính trong thời gian $O(1)$) hoặc gặp đoạn đầu tiên mà truy vấn của ta không nằm hoàn toàn trong một khối. Xem phần đầu tiên để biết cách trả lời truy vấn trong trường hợp này.

OK, giờ ta có thể thực hiện $O(\log\log n)$ cho mỗi truy vấn. Liệu có thể làm nhanh hơn không?

### Tối ưu hóa độ phức tạp truy vấn

Một trong những tối ưu hóa rõ ràng nhất là sử dụng tìm kiếm nhị phân để tìm nút cây mà ta cần. Sử dụng tìm kiếm nhị phân, ta có thể đạt độ phức tạp $O(\log\log\log n)$ cho mỗi truy vấn. Liệu có thể làm nhanh hơn nữa không?

Câu trả lời là có. Hãy giả sử hai điều sau:

- Mỗi kích thước khối là một lũy thừa của hai.
- Tất cả các khối đều bằng nhau trên mỗi tầng.

Để đạt được điều này, ta có thể thêm một số phần tử 0 vào mảng để kích thước của nó trở thành lũy thừa của hai.

Khi sử dụng điều này, một số kích thước khối có thể lớn gấp đôi để trở thành lũy thừa của hai, nhưng nó vẫn là $O(\sqrt{k})$ về kích thước và ta giữ được độ phức tạp tuyến tính để xây dựng các mảng trong một đoạn.

Giờ đây, ta có thể dễ dàng kiểm tra xem truy vấn có nằm hoàn toàn trong một khối có kích thước $2^k$ hay không. Hãy viết các phạm vi của truy vấn, $l$ và $r$ (ta sử dụng chỉ số bắt đầu từ 0) dưới dạng nhị phân. Ví dụ, giả sử $k = 4, l = 39, r = 46$. Biểu diễn nhị phân của $l$ và $r$ là:

$l = 39_{10} = 100111_2$
$r = 46_{10} = 101110_2$

Hãy nhớ rằng một tầng chứa các đoạn có kích thước bằng nhau, và các khối trên một tầng cũng có kích thước bằng nhau (trong trường hợp của ta, kích thước của chúng là $2^k = 2^4 = 16$). Các khối bao phủ toàn bộ mảng, nên khối đầu tiên bao gồm các phần tử $(0-15)$ $((000000_2 - 001111_2)$ trong nhị phân), khối thứ hai bao gồm các phần tử $(16-31)$ $((010000_2 - 011111_2)$ trong nhị phân), v.v. Ta thấy rằng các chỉ số của các vị trí được bao phủ bởi một khối chỉ có thể khác nhau ở $k$ (trong trường hợp của ta là 4) bit cuối cùng. Trong trường hợp của ta, $l$ và $r$ có các bit bằng nhau ngoại trừ bốn bit thấp nhất, nên chúng nằm trong một khối.

Vì vậy, ta cần kiểm tra xem không có gì hơn $k$ bit nhỏ nhất khác nhau (hoặc $l$ xor $r$ không vượt quá $2^k - 1$).

Sử dụng quan sát này, ta có thể nhanh chóng tìm ra tầng phù hợp để trả lời truy vấn. Cách làm như sau:

- Với mỗi $i$ không vượt quá kích thước mảng, ta tìm bit cao nhất bằng 1. Để làm nhanh, ta sử dụng DP và một mảng tính trước.
- Với mỗi $q(l, r)$, ta tìm bit cao nhất của $l$ xor $r$ và, sử dụng thông tin này, dễ dàng chọn tầng mà ta có thể xử lý truy vấn dễ dàng. Ta cũng có thể sử dụng một mảng tính trước ở đây.

Xem chi tiết trong mã dưới đây.

Vậy là, sử dụng điều này, ta có thể trả lời các truy vấn trong $O(1)$ mỗi truy vấn. Hoan hô! :)

## 4.19.2 Cập nhật phần tử

Chúng ta cũng có thể cập nhật các phần tử trong Cây Căn Bậc Hai. Cả cập nhật một phần tử đơn lẻ và cập nhật trên một đoạn đều được hỗ trợ.

### Cập nhật một phần tử đơn lẻ

Xem xét một truy vấn update$(x, val)$ thực hiện gán $a_x = val$. Ta cần thực hiện truy vấn này đủ nhanh.

**Cách tiếp cận ngây thơ**

Đầu tiên, hãy xem điều gì thay đổi trong cây khi một phần tử đơn lẻ thay đổi. Xem xét một nút cây có độ dài $l$ và các mảng của nó: prefixOp, suffixOp và between. Dễ thấy rằng chỉ $O(\sqrt{l})$ phần tử từ prefixOp và suffixOp thay đổi (chỉ bên trong khối chứa phần tử thay đổi). $O(l)$ phần tử được thay đổi trong between. Do đó, $O(l)$ phần tử trong nút cây được cập nhật.

Ta nhớ rằng bất kỳ phần tử $x$ nào cũng xuất hiện chính xác một lần ở mỗi tầng của cây. Nút gốc (tầng 0) có độ dài $O(n)$, các nút ở tầng 1 có độ dài $O(\sqrt{n})$, các nút ở tầng 2 có độ dài $O(\sqrt{\sqrt{n}})$, v.v. Nên độ phức tạp thời gian cho mỗi lần cập nhật là $O(n + \sqrt{n} + \sqrt{\sqrt{n}} + \dots) = O(n)$.

Nhưng như vậy là quá chậm. Liệu có thể làm nhanh hơn không?

**Một cây căn bậc hai bên trong cây căn bậc hai**

Lưu ý rằng điểm nghẽn của việc cập nhật là xây dựng lại between của nút gốc. Để tối ưu hóa cây, hãy loại bỏ mảng này! Thay vì mảng between, ta lưu trữ một cây căn bậc hai khác cho nút gốc. Gọi nó là index. Nó đóng vai trò giống như between—trả lời các truy vấn trên các đoạn của các khối. Lưu ý rằng các nút còn lại của cây không có index, chúng giữ các mảng between của mình.

Một cây căn bậc hai được lập chỉ mục nếu nút gốc của nó có index. Một cây căn bậc hai với mảng between trong nút gốc của nó là không được lập chỉ mục. Lưu ý rằng index bản thân nó là không được lập chỉ mục.

Vậy, ta có thuật toán sau để cập nhật một cây được lập chỉ mục:

- Cập nhật prefixOp và suffixOp trong $O(\sqrt{n})$.
- Cập nhật index. Nó có độ dài $O(\sqrt{n})$ và ta chỉ cần cập nhật một mục trong đó (đại diện cho khối đã thay đổi). Nên độ phức tạp thời gian cho bước này là $O(\sqrt{n})$. Ta có thể sử dụng thuật toán đã mô tả ở phần đầu của mục này (cách "chậm") để thực hiện.
- Đi vào nút con đại diện cho khối đã thay đổi và cập nhật nó trong $O(\sqrt{n})$ với thuật toán "chậm".

Lưu ý rằng độ phức tạp truy vấn vẫn là $O(1)$: ta cần sử dụng index trong truy vấn không quá một lần, và điều này sẽ mất $O(1)$ thời gian.

Vậy, tổng độ phức tạp thời gian để cập nhật một phần tử đơn lẻ là $O(\sqrt{n})$. Hoan hô! :)

### Cập nhật một đoạn

Cây căn bậc hai cũng có thể thực hiện các thao tác như gán một giá trị trên một đoạn. massUpdate$(x, l, r)$ nghĩa là $a_i = x$ với mọi $l \le i \le r$.

Có hai cách tiếp cận để thực hiện điều này: một cách thực hiện massUpdate trong $O(\sqrt{n} \cdot \log\log n)$, giữ $O(1)$ cho mỗi truy vấn. Cách thứ hai thực hiện massUpdate trong $O(\sqrt{n})$, nhưng độ phức tạp truy vấn trở thành $O(\log\log n)$.

Ta sẽ thực hiện lan truyền lười (lazy propagation) theo cách tương tự như trong cây đoạn: ta đánh dấu một số nút là lười, nghĩa là ta sẽ đẩy chúng khi cần thiết. Nhưng có một điều khác biệt so với cây đoạn: việc đẩy một nút là tốn kém, nên nó không thể thực hiện trong các truy vấn. Ở tầng 0, việc đẩy một nút mất $O(\sqrt{n})$ thời gian. Vì vậy, ta

không đẩy các nút bên trong các truy vấn, ta chỉ xem liệu nút hiện tại hoặc cha của nó có lười không, và chỉ cần tính đến điều đó khi thực hiện các truy vấn.

**Cách tiếp cận đầu tiên**

Trong cách tiếp cận đầu tiên, ta nói rằng chỉ các nút ở tầng 1 (có độ dài $O(\sqrt{n})$ có thể là lười. Khi đẩy nút như vậy, nó cập nhật toàn bộ cây con của nó bao gồm chính nó trong $O(\sqrt{n} \cdot \log\log n)$. Quá trình massUpdate được thực hiện như sau:

- Xem xét các nút ở tầng 1 và các khối tương ứng với chúng.
- Một số khối được bao phủ hoàn toàn bởi massUpdate. Đánh dấu chúng là lười trong $O(\sqrt{n})$.
- Một số khối được bao phủ một phần. Lưu ý rằng không có quá hai khối loại này. Xây dựng lại chúng trong $O(\sqrt{n} \cdot \log\log n)$. Nếu chúng đã lười, hãy tính đến điều đó.
- Cập nhật prefixOp và suffixOp cho các khối được bao phủ một phần trong $O(\sqrt{n})$ (vì chỉ có hai khối như vậy).
- Xây dựng lại index trong $O(\sqrt{n} \cdot \log\log n)$.

Vậy là ta có thể thực hiện massUpdate nhanh. Nhưng lan truyền lười ảnh hưởng đến các truy vấn như thế nào? Chúng sẽ có các sửa đổi sau:

- Nếu truy vấn của ta nằm hoàn toàn trong một khối lười, tính toán nó và tính đến trạng thái lười. $O(1)$.
- Nếu truy vấn của ta gồm nhiều khối, một số trong đó là lười, ta chỉ cần quan tâm đến trạng thái lười ở khối cực trái và cực phải. Các khối còn lại được tính bằng index, vốn đã biết câu trả lời trên khối lười (vì nó được xây dựng lại sau mỗi sửa đổi). $O(1)$.

Độ phức tạp truy vấn vẫn giữ nguyên là $O(1)$.

**Cách tiếp cận thứ hai**

Trong cách tiếp cận này, mỗi nút có thể là lười (ngoại trừ gốc). Ngay cả các nút trong index cũng có thể là lười. Vì vậy, khi xử lý một truy vấn, ta phải tìm kiếm các thẻ lười trong tất cả các nút cha, tức là độ phức tạp truy vấn sẽ là $O(\log\log n)$.

Nhưng massUpdate trở nên nhanh hơn. Nó trông như sau:

- Một số khối được bao phủ hoàn toàn bởi massUpdate. Vì vậy, các thẻ lười được thêm vào chúng. Điều này là $O(\sqrt{n})$.
- Cập nhật prefixOp và suffixOp cho các khối được bao phủ một phần trong $O(\sqrt{n})$ (vì chỉ có hai khối như vậy).

- Đừng quên cập nhật index. Nó là $O(\sqrt{n})$ (ta sử dụng cùng thuật toán massUpdate).
- Cập nhật mảng between cho các cây con không được lập chỉ mục.
- Đi vào các nút đại diện cho các khối được bao phủ một phần và gọi massUpdate một cách đệ quy.

Lưu ý rằng khi thực hiện lệnh gọi đệ quy, ta thực hiện massUpdate tiền tố hoặc hậu tố. Nhưng với các cập nhật tiền tố và hậu tố, ta có thể có không quá một con được bao phủ một phần. Vì vậy, ta thăm một nút ở tầng 1, hai nút ở tầng 2 và hai nút ở bất kỳ tầng sâu hơn nào. Do đó, độ phức tạp thời gian là $O(\sqrt{n} + \sqrt{\sqrt{n}} + \dots) = O(\sqrt{n})$. Cách tiếp cận này tương tự như cập nhật hàng loạt trong cây đoạn.

### 4.19.3 Triển khai

Triển khai sau của Cây Căn Bậc Hai có thể thực hiện các thao tác sau: xây dựng trong $O(n \cdot \log\log n)$, trả lời truy vấn trong $O(1)$ và cập nhật một phần tử trong $O(\sqrt{n})$.

```
SqrtTreeItem op(const SqrtTreeItem &a, const SqrtTreeItem &b);

inline int log2Up(int n) {
    int res = 0;
    while ((1 << res) < n) {
        res++;
    }
    return res;
}

class SqrtTree {
private:
    int n, lg, indexSz;
    vector<SqrtTreeItem> v;
    vector<int> clz, layers, onLayer;
    vector< vector<SqrtTreeItem> > pref, suf, between;

    inline void buildBlock(int layer, int l, int r) {
        pref[layer][l] = v[l];
        for (int i = l+1; i < r; i++) {
            pref[layer][i] = op(pref[layer][i-1], v[i]);
        }
        suf[layer][r-1] = v[r-1];
        for (int i = r-2; i >= l; i--) {
```

```
        suf[layer][i] = op(v[i], suf[layer][i+1]);
    }
}

inline void buildBetween(int layer, int lBound, int rBound, int betweenOffs) {
    int bSzLog = (layers[layer]+1) >> 1;
    int bCntLog = layers[layer] >> 1;
    int bSz = 1 << bSzLog;
    int bCnt = (rBound - lBound + bSz - 1) >> bSzLog;
    for (int i = 0; i < bCnt; i++) {
        SqrtTreeItem ans;
        for (int j = i; j < bCnt; j++) {
            SqrtTreeItem add = suf[layer][lBound + (j << bSzLog)];
            ans = (i == j) ? add : op(ans, add);
            between[layer-1][betweenOffs + lBound + (i << bCntLog) + j] = ans;
        }
    }
}

inline void buildBetweenZero() {
    int bSzLog = (lg+1) >> 1;
    for (int i = 0; i < indexSz; i++) {
        v[n+i] = suf[0][i << bSzLog];
    }
    build(1, n, n + indexSz, (1 << lg) - n);
}

inline void updateBetweenZero(int bid) {
    int bSzLog = (lg+1) >> 1;
    v[n+bid] = suf[0][bid << bSzLog];
    update(1, n, n + indexSz, (1 << lg) - n, n+bid);
}

void build(int layer, int lBound, int rBound, int betweenOffs) {
    if (layer >= (int)layers.size()) {
        return;
    }
    int bSz = 1 << ((layers[layer]+1) >> 1);
    for (int l = lBound; l < rBound; l += bSz) {
        int r = min(l + bSz, rBound);
```

```
            buildBlock(layer, l, r);
            build(layer+1, l, r, betweenOffs);
        }
        if (layer == 0) {
            buildBetweenZero();
        } else {
            buildBetween(layer, lBound, rBound, betweenOffs);
        }
    }

    void update(int layer, int lBound, int rBound, int betweenOffs, int x) {
        if (layer >= (int)layers.size()) {
            return;
        }
        int bSzLog = (layers[layer]+1) >> 1;
        int bSz = 1 << bSzLog;
        int blockIdx = (x - lBound) >> bSzLog;
        int l = lBound + (blockIdx << bSzLog);
        int r = min(l + bSz, rBound);
        buildBlock(layer, l, r);
        if (layer == 0) {
            updateBetweenZero(blockIdx);
        } else {
            buildBetween(layer, lBound, rBound, betweenOffs);
        }
        update(layer+1, l, r, betweenOffs, x);
    }

    inline SqrtTreeItem query(int l, int r, int betweenOffs, int base) {
        if (l == r) {
            return v[l];
        }
        if (l + 1 == r) {
            return op(v[l], v[r]);
        }
        int layer = onLayer[clz[(l - base) ^ (r - base)]];
        int bSzLog = (layers[layer]+1) >> 1;
        int bCntLog = layers[layer] >> 1;
        int lBound = (((l - base) >> layers[layer]) << layers[layer]) + base;
        int lBlock = ((l - lBound) >> bSzLog) + 1;
```

```
        int rBlock = ((r - lBound) >> bSzLog) - 1;
        SqrtTreeItem ans = suf[layer][l];
        if (lBlock <= rBlock) {
            SqrtTreeItem add = (layer == 0) ? (
                query(n + lBlock, n + rBlock, (1 << lg) - n, n)
            ) : (
                between[layer-1][betweenOffs + lBound + (lBlock << bCntLog) + rBlock]
            );
            ans = op(ans, add);
        }
        ans = op(ans, pref[layer][r]);
        return ans;
    }
public:
    inline SqrtTreeItem query(int l, int r) {
        return query(l, r, 0, 0);
    }

    inline void update(int x, const SqrtTreeItem &item) {
        v[x] = item;
        update(0, 0, n, 0, x);
    }

    SqrtTree(const vector<SqrtTreeItem>& a)
        : n((int)a.size()), lg(log2Up(n)), v(a), clz(1 << lg), onLayer(lg+1) {
        clz[0] = 0;
        for (int i = 1; i < (int)clz.size(); i++) {
            clz[i] = clz[i >> 1] + 1;
        }
        int tlg = lg;
        while (tlg > 1) {
            onLayer[tlg] = (int)layers.size();
            layers.push_back(tlg);
            tlg = (tlg+1) >> 1;
        }
        for (int i = lg-1; i >= 0; i--) {
            onLayer[i] = max(onLayer[i], onLayer[i+1]);
        }
        int betweenLayers = max(0, (int)layers.size() - 1);
        int bSzLog = (lg+1) >> 1;
```

```
        int bSz = 1 << bSzLog;
        indexSz = (n + bSz - 1) >> bSzLog;
        v.resize(n + indexSz);
        pref.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
        suf.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
        between.assign(betweenLayers, vector<SqrtTreeItem>((1 << lg) + bSz));
        build(0, 0, n, 0);
    }
};
```

## 4.20 Heap Ngẫu Nhiên

Heap ngẫu nhiên là một heap mà nhờ sử dụng tính ngẫu nhiên, cho phép thực hiện tất cả các thao tác trong thời gian kỳ vọng là logarithmic.

Heap nhỏ nhất (min heap) là một cây nhị phân trong đó giá trị của mỗi đỉnh nhỏ hơn hoặc bằng giá trị của các con của nó. Do đó, giá trị nhỏ nhất của cây luôn nằm ở đỉnh gốc.

Heap lớn nhất (max heap) có thể được định nghĩa tương tự: bằng cách thay "nhỏ hơn" bằng "lớn hơn".

Các thao tác mặc định của heap bao gồm:

- Thêm một giá trị.
- Trích xuất giá trị nhỏ nhất.
- Xóa giá trị nhỏ nhất.
- Hợp nhất hai heap (không xóa các phần tử trùng lặp).
- Xóa một phần tử bất kỳ (nếu biết vị trí của nó trong cây).

Heap ngẫu nhiên có thể thực hiện tất cả các thao tác này trong thời gian kỳ vọng $O(\log n)$ với một triển khai rất đơn giản.

### 4.20.1 Cấu trúc dữ liệu

Chúng ta có thể mô tả ngay cấu trúc của heap nhị phân:

```
struct Tree {
    int value;
    Tree * l = nullptr;
    Tree * r = nullptr;
};
```

Trong mỗi đỉnh, ta lưu một giá trị. Ngoài ra, ta có các con trỏ tới con trái và con phải, được đặt là `nullptr` nếu con tương ứng không tồn tại.

### 4.20.2 Thao tác

Không khó để thấy rằng tất cả các thao tác có thể được quy về một thao tác duy nhất: hợp nhất hai heap thành một. Thật vậy:

- Thêm một giá trị mới vào heap tương đương với việc hợp nhất heap với một heap chỉ gồm một đỉnh có giá trị đó.
- Tìm giá trị nhỏ nhất không cần thao tác nào - giá trị nhỏ nhất chỉ đơn giản là giá trị tại gốc.
- Xóa giá trị nhỏ nhất tương đương với kết quả của việc hợp nhất con trái và con phải của đỉnh gốc.
- Xóa một phần tử bất kỳ cũng tương tự: ta hợp nhất các con của đỉnh đó và thay thế đỉnh bằng kết quả của việc hợp nhất.

Vì vậy, ta thực sự chỉ cần triển khai thao tác hợp nhất hai heap. Tất cả các thao tác khác đều có thể được quy về thao tác này một cách đơn giản.

Giả sử có hai heap $T_1$ và $T_2$. Rõ ràng, gốc của mỗi heap chứa giá trị nhỏ nhất của nó. Do đó, gốc của heap kết quả sẽ là giá trị nhỏ nhất trong hai giá trị này. Ta so sánh cả hai giá trị và chọn giá trị nhỏ hơn làm gốc mới. Bây giờ, ta phải kết hợp các con của đỉnh được chọn với heap còn lại. Để làm điều này, ta chọn một trong hai con và hợp nhất nó với heap còn lại. Như vậy, ta lại có thao tác hợp nhất hai heap. Quá trình này sẽ sớm kết thúc (số bước bị giới hạn bởi tổng chiều cao của hai heap).

Để đạt được độ phức tạp logarithmic trung bình, ta cần chỉ định một phương pháp chọn một trong hai con sao cho độ dài đường đi trung bình là logarithmic. Không khó để đoán rằng ta sẽ đưa ra quyết định này một cách ngẫu nhiên. Do đó, triển khai thao tác hợp nhất như sau:

```
Tree* merge(Tree* t1, Tree* t2) {
    if (!t1 || !t2)
        return t1 ? t1 : t2;
    if (t2->value < t1->value)
        swap(t1, t2);
    if (rand() & 1)
        swap(t1->l, t1->r);
    t1->l = merge(t1->l, t2);
    return t1;
}
```

Ở đây, trước tiên ta kiểm tra xem một trong hai heap có rỗng không; nếu có, ta không cần thực hiện hành động hợp nhất nào. Nếu không, ta đảm bảo heap `t1` là heap có giá trị nhỏ hơn (bằng cách hoán đổi `t1` và `t2` nếu cần). Ta muốn hợp nhất con trái của `t1` với `t2`, do đó ta ngẫu nhiên hoán đổi các con của `t1`, rồi thực hiện hợp nhất.

### 4.20.3 Độ phức tạp

Ta giới thiệu biến ngẫu nhiên $h(T)$, biểu thị độ dài của đường đi ngẫu nhiên từ gốc đến lá (độ dài tính bằng số cạnh). Rõ ràng, thuật toán `merge` thực hiện $O(h(T_1) + h(T_2))$ bước. Do đó, để hiểu độ phức tạp của các thao tác, ta phải xem xét biến ngẫu nhiên $h(T)$.

#### Giá trị kỳ vọng

Ta giả định rằng kỳ vọng $h(T)$ có thể được ước lượng từ trên bởi logarit của số đỉnh trong heap:

$$\mathbf{E}h(T) \leq \log(n+1)$$

Điều này có thể được chứng minh dễ dàng bằng quy nạp. Gọi $L$ và $R$ là các cây con trái và phải của gốc $T$, và $n_L$ cùng $n_R$ là số đỉnh trong chúng ($n = n_L + n_R + 1$). Bước quy nạp được thể hiện như sau:

$$\begin{aligned}
\mathbf{E}h(T) &= 1 + \frac{\mathbf{E}h(L) + \mathbf{E}h(R)}{2} \leq 1 + \frac{\log(n_L+1) + \log(n_R+1)}{2} \\
&= 1 + \log\sqrt{(n_L+1)(n_R+1)} = \log 2\sqrt{(n_L+1)(n_R+1)} \\
&\leq \log\frac{2\left((n_L+1)+(n_R+1)\right)}{2} = \log(n_L+n_R+2) = \log(n+1)
\end{aligned}$$

#### Vượt quá giá trị kỳ vọng

Dĩ nhiên, ta vẫn chưa hài lòng. Giá trị kỳ vọng của $h(T)$ không nói gì về trường hợp xấu nhất. Vẫn có khả năng các đường đi từ gốc đến các đỉnh trung bình lớn hơn nhiều so với $\log(n+1)$ cho một cây cụ thể.

Hãy chứng minh rằng xác suất vượt quá giá trị kỳ vọng là rất nhỏ:

$$P\{h(T) > (c+1)\log n\} < \frac{1}{n^c}$$

với bất kỳ hằng số dương $c$ nào.

Ở đây, ta ký hiệu $P$ là tập hợp các đường đi từ gốc của heap đến các lá mà độ dài vượt quá $(c+1)\log n$. Lưu ý rằng với bất kỳ đường đi $p$ nào có độ dài $|p|$, xác suất nó được chọn làm đường đi ngẫu nhiên là $2^{-|p|}$. Do đó, ta có:

$$P\{h(T) > (c+1)\log n\} = \sum_{p\in P} 2^{-|p|} < \sum_{p\in P} 2^{-(c+1)\log n} = |P|n^{-(c+1)} \leq n^{-c}$$

### 4.20.4 Độ phức tạp của thuật toán

Như vậy, thuật toán `merge`, và do đó tất cả các thao tác khác được biểu diễn qua nó, có thể được thực hiện trong $O(\log n)$ trung bình.

Hơn nữa, với bất kỳ hằng số dương $\epsilon$ nào, tồn tại một hằng số dương $c$ sao cho xác suất thao tác yêu cầu hơn $c \log n$ bước là nhỏ hơn $n^{-\epsilon}$ (theo một nghĩa nào đó, điều này mô tả hành vi trường hợp xấu nhất của thuật toán).

## 4.21 Xóa khỏi cấu trúc dữ liệu trong $O(T(n) \log n)$

Giả sử bạn có một cấu trúc dữ liệu cho phép thêm các phần tử trong thời gian thực sự $O(T(n))$. Bài viết này sẽ mô tả một kỹ thuật cho phép xóa trong $O(T(n) \log n)$ theo cách ngoại tuyến.

### 4.21.1 Thuật toán

Mỗi phần tử tồn tại trong cấu trúc dữ liệu trong một số đoạn thời gian giữa các thao tác thêm và xóa. Hãy xây dựng một cây phân đoạn (segment tree) trên các truy vấn. Mỗi đoạn thời gian mà một phần tử còn "sống" sẽ được chia thành $O(\log n)$ nút của cây. Hãy đặt mỗi truy vấn mà ta muốn biết điều gì đó về cấu trúc vào lá tương ứng. Bây giờ, để xử lý tất cả các truy vấn, ta sẽ thực hiện duyệt DFS trên cây phân đoạn. Khi vào một nút, ta sẽ thêm tất cả các phần tử có trong nút này. Sau đó, ta sẽ tiếp tục đi sâu vào các con của nút này hoặc trả lời các truy vấn (nếu nút là một lá). Khi rời khỏi nút, ta phải hoàn tác các thao tác thêm. Lưu ý rằng nếu ta thay đổi cấu trúc trong $O(T(n))$, ta có thể hoàn tác các thay đổi trong $O(T(n))$ bằng cách giữ một ngăn xếp (stack) các thay đổi. Lưu ý rằng các thao tác hoàn tác sẽ phá vỡ độ phức tạp phân bổ (amortized complexity).

### 4.21.2 Ghi chú

Ý tưởng tạo một cây phân đoạn trên các đoạn thời gian mà một thứ gì đó còn "sống" có thể được sử dụng không chỉ cho các bài toán về cấu trúc dữ liệu. Xem một số bài toán bên dưới.

### 4.21.3 Triển khai

Phần triển khai này dành cho bài toán kết nối động (dynamic connectivity). Nó có thể thêm cạnh, xóa cạnh và đếm số thành phần liên thông.

```
struct dsu_save {
    int v, rnkv, u, rnku;
```

```
    dsu_save() {}

    dsu_save(int _v, int _rnkv, int _u, int _rnku)
        : v(_v), rnkv(_rnkv), u(_u), rnku(_rnku) {}
};

struct dsu_with_rollbacks {
    vector<int> p, rnk;
    int comps;
    stack<dsu_save> op;

    dsu_with_rollbacks() {}

    dsu_with_rollbacks(int n) {
        p.resize(n);
        rnk.resize(n);
        for (int i = 0; i < n; i++) {
            p[i] = i;
            rnk[i] = 0;
        }
        comps = n;
    }

    int find_set(int v) {
        return (v == p[v]) ? v : find_set(p[v]);
    }

    bool unite(int v, int u) {
        v = find_set(v);
        u = find_set(u);
        if (v == u)
            return false;
        comps--;
        if (rnk[v] > rnk[u])
            swap(v, u);
        op.push(dsu_save(v, rnk[v], u, rnk[u]));
        p[v] = u;
        if (rnk[u] == rnk[v])
            rnk[u]++;
        return true;
```

```
    }

    void rollback() {
        if (op.empty())
            return;
        dsu_save x = op.top();
        op.pop();
        comps++;
        p[x.v] = x.v;
        rnk[x.v] = x.rnkv;
        p[x.u] = x.u;
        rnk[x.u] = x.rnku;
    }
};

struct query {
    int v, u;
    bool united;
    query(int _v, int _u) : v(_v), u(_u) {
    }
};

struct QueryTree {
    vector<vector<query>> t;
    dsu_with_rollbacks dsu;
    int T;

    QueryTree() {}

    QueryTree(int _T, int n) : T(_T) {
        dsu = dsu_with_rollbacks(n);
        t.resize(4 * T + 4);
    }

    void add_to_tree(int v, int l, int r, int ul, int ur, query& q) {
        if (ul > ur)
            return;
        if (l == ul && r == ur) {
            t[v].push_back(q);
            return;
```

```
        }
        int mid = (l + r) / 2;
        add_to_tree(2 * v, l, mid, ul, min(ur, mid), q);
        add_to_tree(2 * v + 1, mid + 1, r, max(ul, mid + 1), ur, q);
    }

    void add_query(query q, int l, int r) {
        add_to_tree(1, 0, T - 1, l, r, q);
    }

    void dfs(int v, int l, int r, vector<int>& ans) {
        for (query& q : t[v]) {
            q.united = dsu.unite(q.v, q.u);
        }
        if (l == r)
            ans[l] = dsu.comps;
        else {
            int mid = (l + r) / 2;
            dfs(2 * v, l, mid, ans);
            dfs(2 * v + 1, mid + 1, r, ans);
        }
        for (query q : t[v]) {
            if (q.united)
                dsu.rollback();
        }
    }

    vector<int> solve() {
        vector<int> ans(T);
        dfs(1, 0, T - 1, ans);
        return ans;
    }
};
```

# Chương 5

# Đại Số

## 5.1 Lũy thừa nhị phân

Lũy thừa nhị phân (còn được gọi là lũy thừa bằng cách bình phương) là một kỹ thuật cho phép tính $a^n$ chỉ sử dụng $O(\log n)$ phép nhân (thay vì $O(n)$ phép nhân cần thiết theo cách tiếp cận đơn giản).

Nó cũng có các ứng dụng quan trọng trong nhiều bài toán không liên quan đến số học, vì nó có thể được sử dụng với bất kỳ phép toán nào có tính chất kết hợp:

$$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$$

Rõ ràng nhất, điều này áp dụng cho phép nhân modulo, phép nhân ma trận và các bài toán khác mà chúng ta sẽ thảo luận bên dưới.

### 5.1.1 Thuật toán

Việc nâng $a$ lên lũy thừa $n$ theo cách đơn giản được biểu diễn bằng phép nhân với $a$ thực hiện $n-1$ lần: $a^n = a \cdot a \cdot \ldots \cdot a$. Tuy nhiên, cách tiếp cận này không thực tế với $a$ hoặc $n$ lớn.

$$a^{b+c} = a^b \cdot a^c \quad \text{và} \quad a^{2b} = a^b \cdot a^b = (a^b)^2$$

Ý tưởng của lũy thừa nhị phân là chia nhỏ công việc bằng cách sử dụng biểu diễn nhị phân của số mũ.

Hãy viết $n$ dưới dạng cơ số 2, ví dụ:

$$3^{13} = 3^{1101_2} = 3^8 \cdot 3^4 \cdot 3^1$$

Vì số $n$ có chính xác $\lfloor \log_2 n \rfloor + 1$ chữ số trong cơ số 2, ta chỉ cần thực hiện $O(\log n)$ phép nhân, nếu ta biết các lũy thừa $a^1, a^2, a^4, a^8, \ldots, a^{2^{\lfloor \log n \rfloor}}$.

Vậy ta chỉ cần tìm cách tính nhanh các lũy thừa này. May mắn thay, điều này rất dễ, vì một phần tử trong dãy chỉ là bình phương của phần tử trước đó:

$$\begin{aligned}
3^1 &= 3 \\
3^2 &= (3^1)^2 = 9 \\
3^4 &= (3^2)^2 = 81 \\
3^8 &= (3^4)^2 = 6561
\end{aligned}$$

Để có kết quả cuối cùng cho $3^{13}$, ta chỉ cần nhân ba trong số đó (bỏ qua $3^2$ vì bit tương ứng trong $n$ không được đặt): $3^{13} = 6561 \cdot 81 \cdot 3 = 1594323$.

Độ phức tạp cuối cùng của thuật toán này là $O(\log n)$: ta phải tính $\log n$ lũy thừa của $a$, sau đó thực hiện tối đa $\log n$ phép nhân để có kết quả cuối cùng.

Cách tiếp cận đệ quy sau đây biểu diễn cùng ý tưởng:

$$a^n = \begin{cases} 1 & \text{nếu } n = 0 \\ \left(a^{\frac{n}{2}}\right)^2 & \text{nếu } n > 0 \text{ và } n \text{ chẵn} \\ \left(a^{\frac{n-1}{2}}\right)^2 \cdot a & \text{nếu } n > 0 \text{ và } n \text{ lẻ} \end{cases}$$

### 5.1.2 Triển khai

Cách tiếp cận đệ quy đầu tiên, là bản dịch trực tiếp của công thức đệ quy:

```
long long binpow(long long a, long long b) {
    if (b == 0)
        return 1;
    long long res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}
```

Cách tiếp cận thứ hai thực hiện cùng nhiệm vụ mà không dùng đệ quy. Nó tính tất cả các lũy thừa trong một vòng lặp và nhân những lũy thừa có bit được đặt tương ứng trong $n$. Mặc dù độ phức tạp của cả hai cách là như nhau, cách này sẽ nhanh hơn trong thực tế vì không có chi phí của các lời gọi đệ quy.

```
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
```

```
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

### 5.1.3 Ứng dụng

**Tính toán hiệu quả các lũy thừa lớn theo modulo một số**

**Bài toán:** Tính $x^n \mod m$. Đây là một phép toán rất phổ biến, ví dụ được dùng trong tính nghịch đảo nhân modulo.

**Giải pháp:** Vì ta biết rằng toán tử modulo không ảnh hưởng đến phép nhân ($a \cdot b \equiv (a \mod m) \cdot (b \mod m) \pmod m$), ta có thể sử dụng trực tiếp mã trên, chỉ thay mỗi phép nhân bằng phép nhân modulo:

```
long long binpow(long long a, long long b, long long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

**Lưu ý:** Có thể tăng tốc thuật toán này khi $b >> m$. Nếu $m$ là số nguyên tố thì $x^n \equiv x^{n \mod (m-1)} \pmod m$, còn với $m$ hợp số thì $x^n \equiv x^{n \mod \phi(m)} \pmod m$. Điều này xuất phát trực tiếp từ định lý nhỏ của Fermat và định lý Euler.

**Tính toán hiệu quả số Fibonacci**

**Bài toán:** Tính số Fibonacci thứ $n$, $F_n$.

**Giải pháp:** Để tính số Fibonacci tiếp theo, chỉ cần hai số trước đó: $F_n = F_{n-1} + F_{n-2}$. Ta có thể xây dựng ma trận $2 \times 2$ mô tả phép biến đổi này. Nâng ma trận này lên lũy thừa $n$ để tìm $F_n$ trong độ phức tạp $O(\log n)$.

### Áp dụng một phép hoán vị $k$ lần

**Bài toán:** Cho một dãy độ dài $n$, áp dụng một phép hoán vị cho nó $k$ lần.

**Giải pháp:** Nâng phép hoán vị lên lũy thừa $k$ bằng lũy thừa nhị phân, rồi áp dụng nó vào dãy. Độ phức tạp là $O(n \log k)$.

```
vector<int> applyPermutation(vector<int> sequence, vector<int> permutation) {
    vector<int> newSequence(sequence.size());
    for(int i = 0; i < sequence.size(); i++) {
        newSequence[i] = sequence[permutation[i]];
    }
    return newSequence;
}

vector<int> permute(vector<int> sequence, vector<int> permutation, long long k) {
    while (k > 0) {
        if (k & 1) {
            sequence = applyPermutation(sequence, permutation);
        }
        permutation = applyPermutation(permutation, permutation);
        k >>= 1;
    }
    return sequence;
}
```

**Lưu ý:** Bài toán này có thể được giải hiệu quả hơn trong thời gian tuyến tính bằng cách xây dựng đồ thị hoán vị và xem xét từng chu trình độc lập.

### Áp dụng nhanh một tập hợp phép toán hình học lên tập hợp các điểm

**Bài toán:** Cho $n$ điểm $p_i$, áp dụng $m$ phép biến đổi (dịch chuyển, tỷ lệ, xoay) lên mỗi điểm. Có thao tác "lặp" áp dụng một danh sách biến đổi $k$ lần. Cần thực hiện nhanh hơn $O(n \cdot length)$.

**Giải pháp:** Mỗi phép biến đổi được biểu diễn dưới dạng ma trận $4 \times 4$. Tính ma trận tổng hợp trong $O(m \log k)$, sau đó áp dụng cho $n$ điểm trong $O(n)$, tổng độ phức tạp là $O(n + m \log k)$.

### Số đường đi độ dài $k$ trong đồ thị

**Bài toán:** Cho đồ thị có hướng không trọng số với $n$ đỉnh, tìm số đường đi độ dài $k$ từ đỉnh $u$ đến $v$.

**Giải pháp:** Nâng ma trận kề lên lũy thừa $k$. Độ phức tạp là $O(n^3 \log k)$.

**Biến thể của lũy thừa nhị phân: Nhân hai số modulo $m$**

**Bài toán:** Nhân hai số $a$ và $b$ modulo $m$, khi tích của chúng vượt quá kiểu 64-bit.

**Giải pháp:** Áp dụng thuật toán lũy thừa nhị phân, thay phép nhân bằng phép cộng, với độ phức tạp $O(\log m)$.

## 5.2 Thuật toán Euclid để tính ước chung lớn nhất

Cho hai số nguyên không âm $a$ và $b$, ta cần tìm ước chung lớn nhất (UCLN) của chúng, tức là số lớn nhất là ước của cả $a$ và $b$. Nó thường được ký hiệu là $\gcd(a, b)$. Về mặt toán học, nó được định nghĩa như sau:

$$\gcd(a, b) = \max\{k > 0 : (k \mid a) \text{ và } (k \mid b)\}$$

(Ở đây ký hiệu "|" biểu thị tính chia hết, tức là "$k \mid a$" có nghĩa là "$k$ chia hết cho $a$").

Khi một trong hai số bằng 0, trong khi số kia khác 0, ước chung lớn nhất của chúng, theo định nghĩa, là số thứ hai. Khi cả hai số đều bằng 0, ước chung lớn nhất không được xác định (nó có thể là bất kỳ số lớn tùy ý nào), nhưng để tiện lợi và giữ tính kết hợp của gcd, ta thường định nghĩa nó là 0. Điều này cho ta một quy tắc đơn giản: nếu một trong hai số là 0, ước chung lớn nhất là số còn lại.

Thuật toán Euclid, được thảo luận dưới đây, cho phép tìm ước chung lớn nhất của hai số $a$ và $b$ trong $O(\log \min(a, b))$. Vì hàm này có tính kết hợp, để tìm UCLN của nhiều hơn hai số, ta có thể làm $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$ và tiếp tục như vậy.

Thuật toán này được mô tả lần đầu trong "Các yếu tố" của Euclid (khoảng 300 TCN), nhưng có thể thuật toán này có nguồn gốc sớm hơn.

### 5.2.1 Thuật toán

Ban đầu, thuật toán Euclid được diễn đạt như sau: trừ số nhỏ hơn từ số lớn hơn cho đến khi một trong hai số bằng 0. Thật vậy, nếu $g$ chia hết cho $a$ và $b$, thì nó cũng chia hết cho $a - b$. Ngược lại, nếu $g$ chia hết cho $a - b$ và $b$, thì nó cũng chia hết cho $a = b + (a - b)$, nghĩa là tập hợp các ước chung của $\{a, b\}$ và $\{b, a - b\}$ trùng nhau.

Lưu ý rằng $a$ vẫn là số lớn hơn cho đến khi $b$ được trừ khỏi nó ít nhất $\left\lfloor \frac{a}{b} \right\rfloor$ lần. Do đó, để tăng tốc, $a - b$ được thay bằng $a - \left\lfloor \frac{a}{b} \right\rfloor b = a \mod b$. Sau đó, thuật toán được diễn đạt một cách cực kỳ đơn giản:

$$\gcd(a, b) = \begin{cases} a, & \text{nếu } b = 0 \\ \gcd(b, a \mod b), & \text{ngược lại.} \end{cases}$$

### 5.2.2 Triển khai

```
int gcd(int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}
```

Sử dụng toán tử ba ngôi trong C++, ta có thể viết gọn thành một dòng:

```
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}
```

Cuối cùng, đây là phiên bản không đệ quy:

```
int gcd(int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

Lưu ý rằng từ C++17, hàm gcd đã được triển khai như một hàm chuẩn trong C++.

### 5.2.3 Độ phức tạp thời gian

Thời gian chạy của thuật toán được ước lượng bởi định lý Lamé, định lý này thiết lập một mối liên hệ bất ngờ giữa thuật toán Euclid và dãy Fibonacci:

Nếu $a > b \geq 1$ và $b < F_n$ với một số $n$, thuật toán Euclid thực hiện tối đa $n - 2$ lời gọi đệ quy.

Hơn nữa, có thể chứng minh rằng giới hạn trên của định lý này là tối ưu. Khi $a = F_n$ và $b = F_{n-1}$, $\gcd(a, b)$ sẽ thực hiện chính xác $n - 2$ lời gọi đệ quy. Nói cách khác, các số Fibonacci liên tiếp là trường hợp xấu nhất cho thuật toán Euclid.

Vì các số Fibonacci tăng trưởng theo cấp số nhân, ta có thuật toán Euclid hoạt động trong $O(\log \min(a, b))$.

Một cách khác để ước lượng độ phức tạp là nhận thấy rằng $a \mod b$ trong trường hợp $a \geq b$ nhỏ hơn $a$ ít nhất 2 lần, do đó số lớn hơn được giảm ít nhất một nửa trong mỗi lần lặp của thuật toán. Áp dụng lý luận này cho trường hợp tính UCLN của tập hợp các số $a_1, \ldots, a_n \leq C$, điều này cũng cho phép ước lượng tổng thời gian chạy là $O(n + \log C)$, thay vì $O(n \log C)$, vì mỗi lần lặp không tầm thường của thuật toán giảm ứng cử viên UCLN hiện tại ít nhất một hệ số 2.

### 5.2.4 Bội chung nhỏ nhất

Việc tính bội chung nhỏ nhất (thường ký hiệu là BCNN) có thể được quy về tính UCLN bằng công thức đơn giản sau:

$$\operatorname{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Do đó, BCNN có thể được tính bằng thuật toán Euclid với cùng độ phức tạp thời gian:

Dưới đây là một triển khai thông minh tránh tràn số nguyên bằng cách chia $a$ cho UCLN trước:

```
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

### 5.2.5 UCLN nhị phân

Thuật toán UCLN nhị phân là một tối ưu hóa cho thuật toán Euclid thông thường.

Phần chậm của thuật toán thông thường là các phép toán modulo. Phép toán modulo, mặc dù ta coi là $O(1)$, nhưng chậm hơn nhiều so với các phép toán đơn giản như cộng, trừ hoặc thao tác bit. Vì vậy, sẽ tốt hơn nếu tránh được chúng.

Hóa ra, ta có thể thiết kế một thuật toán UCLN nhanh không dùng phép toán modulo. Nó dựa trên một số tính chất:

- Nếu cả hai số đều chẵn, ta có thể phân tích thừa số 2 từ cả hai và tính UCLN của các số còn lại: $\gcd(2a, 2b) = 2\gcd(a, b)$.
- Nếu một số chẵn và số kia lẻ, ta có thể loại bỏ thừa số 2 từ số chẵn: $\gcd(2a, b) = \gcd(a, b)$ nếu $b$ lẻ.
- Nếu cả hai số đều lẻ, thì việc trừ một số từ số kia không thay đổi UCLN: $\gcd(a, b) = \gcd(b, a - b)$.

Sử dụng chỉ các tính chất này và một số hàm bit nhanh từ GCC, ta có thể triển khai phiên bản nhanh:

```
int gcd(int a, int b) {
    if (!a || !b)
        return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do {
```

```
        b >>= __builtin_ctz(b);
        if (a > b)
            swap(a, b);
        b -= a;
    } while (b);
    return a << shift;
}
```

Lưu ý rằng tối ưu hóa như vậy thường không cần thiết, và hầu hết các ngôn ngữ lập trình đã có hàm UCLN trong thư viện chuẩn của chúng, ví dụ C++17 có hàm `std::gcd` trong tiêu đề `numeric`.

## 5.3 Thuật toán Euclid mở rộng

Trong khi thuật toán Euclid chỉ tính ước chung lớn nhất (UCLN) của hai số nguyên $a$ và $b$, phiên bản mở rộng còn tìm cách biểu diễn UCLN dưới dạng tổ hợp tuyến tính của $a$ và $b$, tức là tìm các hệ số $x$ và $y$ sao cho:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

Điều quan trọng cần lưu ý là theo định lý Bézout, ta luôn có thể tìm được biểu diễn như vậy. Ví dụ, $\gcd(55, 80) = 5$, do đó ta có thể biểu diễn 5 dưới dạng tổ hợp tuyến tính của 55 và 80:

$$55 \cdot 3 + 80 \cdot (-2) = 5$$

Dạng tổng quát hơn của bài toán này được thảo luận trong bài viết về phương trình Diophantine tuyến tính. Nó sẽ dựa trên thuật toán này.

### 5.3.1 Thuật toán

Ta sẽ ký hiệu UCLN của $a$ và $b$ là $g$ trong phần này.

Những thay đổi so với thuật toán gốc rất đơn giản. Nếu nhớ lại thuật toán Euclid, ta thấy rằng thuật toán kết thúc khi $b = 0$ và $a = g$. Với các tham số này, ta dễ dàng tìm được các hệ số, cụ thể là:

$$g \cdot 1 + 0 \cdot 0 = g$$

Bắt đầu từ các hệ số $(x, y) = (1, 0)$, ta có thể đi ngược lên qua các lời gọi đệ quy. Tất cả những gì cần làm là tìm ra cách các hệ số $x$ và $y$ thay đổi trong quá trình chuyển từ $(a, b)$ sang $(b, a \mod b)$.

Giả sử ta đã tìm được các hệ số $(x_1, y_1)$ cho $(b, a \mod b)$:

$$b \cdot x_1 + (a \mod b) \cdot y_1 = g$$

và ta muốn tìm cặp $(x, y)$ cho $(a, b)$:

$$a \cdot x + b \cdot y = g$$

Ta có thể biểu diễn $a \mod b$ như sau:

$$a \mod b = a - \left\lfloor \frac{a}{b} \right\rfloor \cdot b$$

Thay biểu thức này vào phương trình hệ số của $(x_1, y_1)$, ta được:

$$g = b \cdot x_1 + (a \mod b) \cdot y_1 = b \cdot x_1 + \left(a - \left\lfloor \frac{a}{b} \right\rfloor \cdot b\right) \cdot y_1$$

Sau khi sắp xếp lại các hạng:

$$g = a \cdot y_1 + b \cdot \left(x_1 - y_1 \cdot \left\lfloor \frac{a}{b} \right\rfloor\right)$$

Ta tìm được giá trị của $x$ và $y$:

$$\begin{cases} x = y_1 \\ y = x_1 - y_1 \cdot \left\lfloor \frac{a}{b} \right\rfloor \end{cases}$$

### 5.3.2 Triển khai

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

Hàm đệ quy trên trả về UCLN và các giá trị hệ số cho $x$ và $y$ (được truyền bằng tham chiếu vào hàm).

Triển khai này của thuật toán Euclid mở rộng cũng cho kết quả chính xác với các số nguyên âm.

### 5.3.3 Phiển bản không đệ quy

Cũng có thể viết thuật toán Euclid mở rộng theo cách không đệ quy. Vì tránh được đệ quy, mã sẽ chạy nhanh hơn một chút so với phiên bản đệ quy.

```
int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}
```

Nếu quan sát kỹ các biến $a1$ và $b1$, ta sẽ nhận thấy chúng nhận chính xác các giá trị giống như trong phiên bản không đệ quy của thuật toán Euclid thông thường. Vì vậy, thuật toán ít nhất sẽ tính đúng UCLN.

Để hiểu tại sao thuật toán tính đúng các hệ số, hãy xem xét rằng các bất biến sau luôn đúng tại bất kỳ thời điểm nào (trước khi vòng lặp `while` bắt đầu và ở cuối mỗi lần lặp):

$$x \cdot a + y \cdot b = a_1$$

$$x_1 \cdot a + y_1 \cdot b = b_1$$

Gọi các giá trị ở cuối một lần lặp là được ký hiệu bằng dấu nháy đơn ($'$), và giả sử $q = \frac{a_1}{b_1}$. Theo thuật toán Euclid, ta có:

$$a'_1 = b_1$$

$$b'_1 = a_1 - q \cdot b_1$$

Để bất biến đầu tiên đúng, điều sau phải đúng:

$$x' \cdot a + y' \cdot b = a'_1 = b_1$$

$$x' \cdot a + y' \cdot b = x_1 \cdot a + y_1 \cdot b$$

Tương tự, để bất biến thứ hai đúng, điều sau phải đúng:

$$x'_1 \cdot a + y'_1 \cdot b = a_1 - q \cdot b_1$$

$$x'_1 \cdot a + y'_1 \cdot b = (x - q \cdot x_1) \cdot a + (y - q \cdot y_1) \cdot b$$

Bằng cách so sánh các hệ số của $a$ và $b$, ta có thể suy ra các phương trình cập nhật cho từng biến, đảm bảo rằng các bất biến được duy trì suốt thuật toán.

Cuối cùng, ta biết rằng $a_1$ chứa UCLN, nên $x \cdot a + y \cdot b = g$. Điều này có nghĩa là ta đã tìm được các hệ số cần thiết.

Bạn thậm chí có thể tối ưu mã hơn nữa, loại bỏ các biến $a_1$ và $b_1$ khỏi mã và chỉ tái sử dụng $a$ và $b$. Tuy nhiên, nếu làm vậy, bạn sẽ mất khả năng lập luận về các bất biến.

## 5.4 Phương trình Diophantine tuyến tính

Phương trình Diophantine tuyến tính (với hai biến) là một phương trình có dạng tổng quát:

$$ax + by = c$$

trong đó $a$, $b$, $c$ là các số nguyên đã cho, và $x$, $y$ là các số nguyên chưa biết.

Trong bài viết này, ta xem xét một số bài toán kinh điển về các phương trình này:

- Tìm một nghiệm
- Tìm tất cả các nghiệm
- Tìm số lượng nghiệm và các nghiệm trong một khoảng cho trước
- Tìm nghiệm có giá trị $x + y$ nhỏ nhất

### 5.4.1 Trường hợp thoái hóa

Một trường hợp thoái hóa cần được xem xét là khi $a = b = 0$. Dễ thấy rằng ta hoặc không có nghiệm hoặc có vô số nghiệm, tùy thuộc vào việc $c = 0$ hay không. Trong phần còn lại của bài viết, ta sẽ bỏ qua trường hợp này.

### 5.4.2 Nghiệm phân tích

Khi $a \neq 0$ và $b \neq 0$, phương trình $ax + by = c$ có thể được xử lý tương đương như một trong hai phương trình sau:

$$ax \equiv c \pmod{b}$$
$$by \equiv c \pmod{a}$$

Không mất tính tổng quát, giả sử $b \neq 0$ và xem xét phương trình đầu tiên. Khi $a$ và $b$ là nguyên tố cùng nhau, nghiệm của nó được cho bởi:

$$x \equiv ca^{-1} \pmod{b},$$

trong đó $a^{-1}$ là nghịch đảo modulo của $a$ theo modulo $b$.

Khi $a$ và $b$ không nguyên tố cùng nhau, các giá trị của $ax$ modulo $b$ với mọi số nguyên $x$ đều chia hết cho $g = \gcd(a, b)$, nên nghiệm chỉ tồn tại khi $c$ chia hết cho $g$. Trong trường hợp này, một nghiệm có thể được tìm bằng cách rút gọn phương trình theo $g$:

$$(a/g)x \equiv (c/g) \pmod{b/g}.$$

Theo định nghĩa của $g$, các số $a/g$ và $b/g$ là nguyên tố cùng nhau, nên nghiệm được cho một cách rõ ràng như sau:

$$\begin{cases} x \equiv (c/g)(a/g)^{-1} \pmod{b/g}, \\ y = \frac{c-ax}{b}. \end{cases}$$

### 5.4.3 Nghiệm thuật toán

Định lý Bézout (còn gọi là đẳng thức Bézout) là một kết quả hữu ích để hiểu giải pháp sau:

- Gọi $g = \gcd(a, b)$. Khi đó tồn tại các số nguyên $x, y$ sao cho $ax + by = g$.
- Hơn nữa, $g$ là số nguyên dương nhỏ nhất có thể được viết dưới dạng $ax + by$; tất cả các số nguyên dạng $ax + by$ đều là bội của $g$.

Để tìm một nghiệm của phương trình Diophantine với 2 ẩn, ta có thể dùng thuật toán Euclid mở rộng. Trước tiên, giả sử $a$ và $b$ không âm. Khi áp dụng thuật toán Euclid mở rộng cho $a$ và $b$, ta có thể tìm ước chung lớn nhất $g$ và hai số $x_g$ và $y_g$ sao cho:

$$ax_g + by_g = g$$

Nếu $c$ chia hết cho $g = \gcd(a, b)$, thì phương trình Diophantine đã cho có nghiệm, ngược lại thì không. Chứng minh rất đơn giản: một tổ hợp tuyến tính của hai số chia hết cho ước chung của chúng.

Bây giờ giả sử $c$ chia hết cho $g$, ta có:

$$a \cdot x_g \cdot \frac{c}{g} + b \cdot y_g \cdot \frac{c}{g} = c$$

Do đó, một trong các nghiệm của phương trình Diophantine là:

$$x_0 = x_g \cdot \frac{c}{g},$$
$$y_0 = y_g \cdot \frac{c}{g}.$$

Ý tưởng trên vẫn hoạt động khi $a$ hoặc $b$ hoặc cả hai âm. Ta chỉ cần thay đổi dấu của $x_0$ và $y_0$ khi cần thiết.

Cuối cùng, ta có thể triển khai ý tưởng này như sau (lưu ý mã này không xem xét trường hợp $a = b = 0$):

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

### 5.4.4 Lấy tất cả các nghiệm

Từ một nghiệm $(x_0, y_0)$, ta có thể thu được tất cả các nghiệm của phương trình đã cho.

Gọi $g = \gcd(a, b)$ và $x_0, y_0$ là các số nguyên thỏa mãn:

$$a \cdot x_0 + b \cdot y_0 = c$$

Bây giờ, ta thấy rằng việc cộng $b/g$ vào $x_0$ và đồng thời trừ $a/g$ khỏi $y_0$ sẽ không làm phá vỡ đẳng thức:

$$a \cdot \left(x_0 + \frac{b}{g}\right) + b \cdot \left(y_0 - \frac{a}{g}\right) = a \cdot x_0 + b \cdot y_0 + a \cdot \frac{b}{g} - b \cdot \frac{a}{g} = c$$

Rõ ràng, quá trình này có thể được lặp lại, nên tất cả các số dạng:

$$x = x_0 + k \cdot \frac{b}{g}$$

$$y = y_0 - k \cdot \frac{a}{g}$$

là nghiệm của phương trình Diophantine đã cho.

Vì phương trình là tuyến tính, tất cả các nghiệm nằm trên cùng một đường thẳng, và theo định nghĩa của $g$, đây là tập hợp tất cả các nghiệm có thể của phương trình.

### 5.4.5 Tìm số lượng nghiệm và các nghiệm trong một khoảng cho trước

Từ phần trước, rõ ràng nếu không đặt bất kỳ hạn chế nào lên nghiệm, sẽ có vô số nghiệm. Vì vậy, trong phần này, ta thêm một số hạn chế về khoảng của $x$ và $y$, và sẽ đếm cũng như liệt kê tất cả các nghiệm.

Giả sử có hai khoảng: $[min_x; max_x]$ và $[min_y; max_y]$, và ta chỉ muốn tìm các nghiệm trong hai khoảng này.

Lưu ý rằng nếu $a$ hoặc $b$ bằng 0, thì bài toán chỉ có một nghiệm. Ta không xem xét trường hợp này ở đây.

Đầu tiên, ta có thể tìm một nghiệm có giá trị nhỏ nhất của $x$, sao cho $x \geq min_x$. Để làm điều này, ta tìm bất kỳ nghiệm nào của phương trình Diophantine, sau đó dịch chuyển nghiệm này để có $x \geq min_x$ (sử dụng kiến thức về tập hợp tất cả các nghiệm từ phần trước). Điều này có thể được thực hiện trong $O(1)$. Ký hiệu giá trị nhỏ nhất của $x$ này là $l_{x1}$.

Tương tự, ta có thể tìm giá trị lớn nhất của $x$ thỏa mãn $x \leq max_x$. Ký hiệu giá trị lớn nhất của $x$ này là $r_{x1}$.

Tương tự, ta có thể tìm giá trị nhỏ nhất của $y$ ($y \geq min_y$) và giá trị lớn nhất của $y$ ($y \leq max_y$). Ký hiệu các giá trị tương ứng của $x$ là $l_{x2}$ và $r_{x2}$.

Nghiệm cuối cùng là tất cả các nghiệm với $x$ trong giao của $[l_{x1}, r_{x1}]$ và $[l_{x2}, r_{x2}]$. Gọi giao này là $[l_x, r_x]$.

Dưới đây là mã triển khai ý tưởng này. Lưu ý rằng ta chia $a$ và $b$ cho $g$ ngay từ đầu. Vì phương trình $ax + by = c$ tương đương với $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$, ta có thể dùng phương trình này thay thế và có $\gcd(\frac{a}{g}, \frac{b}{g}) = 1$, điều này đơn giản hóa các công thức.

```
void shift_solution(int & x, int & y, int a, int b, int cnt) {
    x += cnt * b;
    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
```

```
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}
```

Khi đã có $l_x$ và $r_x$, việc liệt kê tất cả các nghiệm cũng đơn giản. Chỉ cần lặp qua

$x = l_x + k \cdot \frac{b}{g}$ với mọi $k \geq 0$ cho đến khi $x = r_x$, và tìm các giá trị $y$ tương ứng bằng phương trình $ax + by = c$.

### 5.4.6 Tìm nghiệm có giá trị $x + y$ nhỏ nhất

Ở đây, $x$ và $y$ cũng cần được giới hạn, nếu không câu trả lời có thể là âm vô cực.

Ý tưởng tương tự như phần trước: Ta tìm bất kỳ nghiệm nào của phương trình Diophantine, sau đó dịch chuyển nghiệm để thỏa mãn một số điều kiện.

Cuối cùng, sử dụng kiến thức về tập hợp tất cả các nghiệm để tìm giá trị nhỏ nhất:

$$x' = x + k \cdot \frac{b}{g},$$

$$y' = y - k \cdot \frac{a}{g}.$$

Lưu ý rằng $x + y$ thay đổi như sau:

$$x' + y' = x + y + k \cdot \left(\frac{b}{g} - \frac{a}{g}\right) = x + y + k \cdot \frac{b - a}{g}$$

Nếu $a < b$, ta cần chọn giá trị nhỏ nhất có thể của $k$. Nếu $a > b$, ta cần chọn giá trị lớn nhất có thể của $k$. Nếu $a = b$, tất cả các nghiệm sẽ có cùng tổng $x + y$.

## 5.5 Số Fibonacci

Dãy Fibonacci được định nghĩa như sau:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

Các phần tử đầu tiên của dãy (OEIS A000045) là:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...$$

### 5.5.1 Tính chất

Số Fibonacci sở hữu nhiều tính chất thú vị. Dưới đây là một số trong đó:

- Đẳng thức Cassini:
$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n$$
- Quy tắc "cộng":
$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

- Áp dụng đẳng thức trên với $k = n$, ta được:

$$F_{2n} = F_n(F_{n+1} + F_{n-1})$$

- Từ đây, ta có thể chứng minh bằng quy nạp rằng với mọi số nguyên dương $k$, $F_{nk}$ là bội của $F_n$.

- Điều ngược lại cũng đúng: nếu $F_m$ là bội của $F_n$, thì $m$ là bội của $n$.

- Đẳng thức UCLN:

$$\gcd(F_m, F_n) = F_{\gcd(m,n)}$$

- Số Fibonacci là đầu vào tệ nhất cho thuật toán Euclid (xem định lý Lamé trong thuật toán Euclid).

### 5.5.2 Mã hóa Fibonacci

Ta có thể dùng dãy để mã hóa các số nguyên dương thành các từ mã nhị phân. Theo định lý Zeckendorf, bất kỳ số tự nhiên $n$ nào cũng có thể được biểu diễn duy nhất dưới dạng tổng của các số Fibonacci:

$$N = F_{k_1} + F_{k_2} + \ldots + F_{k_r}$$

sao cho $k_1 \geq k_2 + 2,\ k_2 \geq k_3 + 2,\ \ldots,\ k_r \geq 2$ (tức là biểu diễn không thể dùng hai số Fibonacci liên tiếp).

Từ đó, bất kỳ số nào cũng có thể được mã hóa duy nhất trong mã hóa Fibonacci. Ta có thể mô tả biểu diễn này bằng các mã nhị phân $d_0 d_1 d_2 \ldots d_s 1$, trong đó $d_i$ là 1 nếu $F_{i+2}$ được dùng trong biểu diễn. Mã sẽ được thêm một 1 ở cuối để chỉ ra kết thúc của từ mã. Lưu ý rằng đây là lần duy nhất xuất hiện hai bit 1 liên tiếp:

$$\begin{aligned}
1 &= 1 = F_2 = (11)_F \\
2 &= 2 = F_3 = (011)_F \\
6 &= 5 + 1 = F_5 + F_2 = (10011)_F \\
8 &= 8 = F_6 = (000011)_F \\
9 &= 8 + 1 = F_6 + F_2 = (100011)_F \\
19 &= 13 + 5 + 1 = F_7 + F_5 + F_2 = (1001011)_F
\end{aligned}$$

Việc mã hóa một số nguyên $n$ có thể được thực hiện bằng thuật toán tham lam đơn giản:

- Duyệt qua các số Fibonacci từ lớn nhất đến nhỏ nhất cho đến khi tìm được số nhỏ hơn hoặc bằng $n$.

- Giả sử số này là $F_i$. Trừ $F_i$ khỏi $n$ và đặt một 1 vào vị trí $i-2$ của từ mã (đánh chỉ số từ 0 từ bit trái nhất sang phải nhất).
- Lặp lại cho đến khi không còn số dư.
- Thêm một 1 cuối cùng vào từ mã để chỉ ra kết thúc.

Để giải mã một từ mã, trước tiên loại bỏ 1 cuối cùng. Sau đó, nếu bit thứ $i$ được đặt (đánh chỉ số từ 0 từ trái sang phải), cộng $F_{i+2}$ vào số.

### 5.5.3 Công thức cho số Fibonacci thứ $n$

#### Biểu thức dạng đóng

Có một công thức được gọi là "công thức Binet", mặc dù nó đã được Moivre biết đến từ trước:

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

Công thức này dễ chứng minh bằng quy nạp, nhưng cũng có thể suy ra bằng khái niệm hàm sinh hoặc giải phương trình hàm.

Ta có thể nhận thấy ngay rằng giá trị tuyệt đối của hạng thứ hai luôn nhỏ hơn 1 và giảm rất nhanh (theo cấp số nhân). Do đó, giá trị của hạng đầu tiên gần giống $F_n$. Điều này có thể được viết chặt chẽ như sau:

$$F_n = \left[\frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}}\right]$$

trong đó dấu ngoặc vuông biểu thị làm tròn đến số nguyên gần nhất.

Vì hai công thức này yêu cầu độ chính xác rất cao khi làm việc với số thập phân, chúng ít hữu dụng trong tính toán thực tế.

#### Fibonacci trong thời gian tuyến tính

Số Fibonacci thứ $n$ có thể dễ dàng được tìm trong $O(n)$ bằng cách tính từng số một đến $n$. Tuy nhiên, cũng có cách nhanh hơn, như ta sẽ thấy.

Ta bắt đầu từ cách tiếp cận lặp, tận dụng công thức $F_n = F_{n-1} + F_{n-2}$, do đó ta sẽ tính trước các giá trị này trong một mảng. Xét đến các trường hợp cơ sở cho $F_0$ và $F_1$:

```
int fib(int n) {
    int a = 0;
    int b = 1;
```

```
    for (int i = 0; i < n; i++) {
        int tmp = a + b;
        a = b;
        b = tmp;
    }
    return a;
}
```

Bằng cách này, ta có giải pháp tuyến tính, thời gian $O(n)$, lưu tất cả các giá trị trước $n$ trong dãy.

### Dạng ma trận

Dễ chứng minh quan hệ sau:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Do đó, để tìm $F_n$ trong $O(\log n)$, ta phải nâng ma trận lên lũy thừa $n$. (Xem Lũy thừa nhị phân)

```
struct matrix {
    long long mat[2][2];
    matrix friend operator *(const matrix &a, const matrix &b){
        matrix c;
        for (int i = 0; i < 2; i++) {
          for (int j = 0; j < 2; j++) {
              c.mat[i][j] = 0;
              for (int k = 0; k < 2; k++) {
                  c.mat[i][j] += a.mat[i][k] * b.mat[k][j];
              }
          }
        }
        return c;
    }
};

matrix matpow(matrix base, long long n) {
    matrix ans{ {
      {1, 0},
      {0, 1}
    } };
```

```
    while (n) {
        if(n&1)
            ans = ans*base;
        base = base*base;
        n >>= 1;
    }
    return ans;
}

long long fib(int n) {
    matrix base{ {
      {1, 1},
      {1, 0}
    } };
    return matpow(base, n).mat[0][1];
}
```

**Phương pháp nhân đôi nhanh**

Bằng cách mở rộng biểu thức ma trận trên với $n = 2 \cdot k$:

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2k} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}^2$$

ta tìm được các phương trình đơn giản hơn:

$$\begin{aligned} F_{2k+1} &= F_{k+1}^2 + F_k^2 \\ F_{2k} &= F_k(F_{k+1} + F_{k-1}) = F_k(2F_{k+1} - F_k) \end{aligned}$$

Do đó, dùng hai phương trình trên, số Fibonacci có thể được tính dễ dàng bằng mã sau:

```
pair<int, int> fib (int n) {
    if (n == 0)
        return {0, 1};

    auto p = fib(n >> 1);
    int c = p.first * (2 * p.second - p.first);
    int d = p.first * p.first + p.second * p.second;
    if (n & 1)
        return {d, c + d};
    else
```

```
        return {c, d};
}
```

Mã trên trả về $F_n$ và $F_{n+1}$ dưới dạng một cặp.

**Tính chu kỳ modulo $p$**

Xem xét dãy Fibonacci modulo $p$. Ta sẽ chứng minh dãy này có chu kỳ.

Ta chứng minh bằng phản chứng. Xem xét $p^2+1$ cặp số Fibonacci đầu tiên lấy modulo $p$:

$$(F_0,\ F_1),\ (F_1,\ F_2),\ \ldots,\ (F_{p^2},\ F_{p^2+1})$$

Chỉ có thể có $p$ dư khác nhau modulo $p$, và tối đa $p^2$ cặp dư khác nhau, nên ít nhất có hai cặp giống nhau trong số đó. Điều này đủ để chứng minh dãy có chu kỳ, vì một số Fibonacci chỉ được xác định bởi hai số trước nó. Do đó, nếu hai cặp số liên tiếp lặp lại, các số sau cặp đó cũng sẽ lặp lại theo cùng cách.

Giờ ta chọn hai cặp dư giống nhau có chỉ số nhỏ nhất trong dãy. Gọi các cặp là $(F_a,\ F_{a+1})$ và $(F_b,\ F_{b+1})$. Ta sẽ chứng minh $a = 0$. Nếu điều này sai, sẽ có hai cặp trước đó $(F_{a-1},\ F_a)$ và $(F_{b-1},\ F_b)$, theo tính chất của số Fibonacci, cũng sẽ bằng nhau. Tuy nhiên, điều này mâu thuẫn với việc ta đã chọn các cặp có chỉ số nhỏ nhất, hoàn thành chứng minh rằng không có tiền chu kỳ (tức là các số có chu kỳ bắt đầu từ $F_0$).

## 5.6 Sàng Eratosthenes

Sàng Eratosthenes là một thuật toán để tìm tất cả các số nguyên tố trong đoạn $[1; n]$ sử dụng $O(n \log \log n)$ phép toán.

Thuật toán rất đơn giản: đầu tiên, ta ghi lại tất cả các số từ 2 đến $n$. Ta đánh dấu tất cả các bội thực sự của 2 (vì 2 là số nguyên tố nhỏ nhất) là hợp số. Bội thực sự của một số $x$ là số lớn hơn $x$ và chia hết cho $x$. Sau đó, ta tìm số tiếp theo chưa bị đánh dấu là hợp số, trong trường hợp này là 3. Điều này có nghĩa là 3 là số nguyên tố, và ta đánh dấu tất cả các bội thực sự của 3 là hợp số. Số tiếp theo chưa bị đánh dấu là 5, là số nguyên tố tiếp theo, và ta đánh dấu tất cả các bội thực sự của nó. Ta tiếp tục quy trình này cho đến khi xử lý hết tất cả các số trong dãy.

Trong hình ảnh sau, bạn có thể thấy hình dung của thuật toán để tính tất cả các số nguyên tố trong khoảng $[1; 16]$. Có thể thấy rằng khá thường xuyên ta đánh dấu các số là hợp số nhiều lần.

**Sàng Eratosthenes**

Ý tưởng đằng sau là: Một số là nguyên tố nếu không có số nguyên tố nhỏ hơn nào chia hết cho nó. Vì ta duyệt qua các số nguyên tố theo thứ tự, ta đã đánh dấu tất cả các số chia hết cho ít nhất một số nguyên tố là hợp số. Do đó, nếu ta đến một ô và nó chưa bị đánh dấu, thì nó không chia hết cho bất kỳ số nguyên tố nhỏ hơn nào và do đó phải là số nguyên tố.

### 5.6.1 Triển khai

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; i++) {
    if (is_prime[i] && (long long)i * i <= n) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

Mã này đầu tiên đánh dấu tất cả các số trừ 0 và 1 là các số nguyên tố tiềm năng, sau đó bắt đầu quá trình loại bỏ các số hợp số. Để làm điều này, nó duyệt qua tất cả các số từ 2 đến $n$. Nếu số hiện tại $i$ là số nguyên tố, nó đánh dấu tất cả các bội của $i$ là số hợp số, bắt đầu từ $i^2$. Đây đã là một tối ưu so với cách triển khai ngây thơ, và được phép vì tất cả các số nhỏ hơn là bội của $i$ chắc chắn cũng có một thừa số nguyên tố nhỏ hơn $i$, nên chúng đã được sàng lọc trước đó. Vì $i^2$ dễ dàng vượt quá kiểu int, nên có thêm kiểm tra sử dụng kiểu long long trước vòng lặp lồng thứ hai.

Với triển khai này, thuật toán tiêu tốn $O(n)$ bộ nhớ (rõ ràng) và thực hiện $O(n \log \log n)$ phép toán (xem phần tiếp theo).

### 5.6.2 Phân tích tiệm cận

Dễ chứng minh thời gian chạy là $O(n \log n)$ mà không cần biết gì về phân bố số nguyên tố – bỏ qua kiểm tra `is_prime`, vòng lặp trong chạy tối đa $n/i$ lần với $i = 2, 3, 4, \ldots$, dẫn đến tổng số phép toán trong vòng lặp trong là một tổng điều hòa như $n(1/2+1/3+1/4+\cdots)$, bị chặn bởi $O(n \log n)$.

Hãy chứng minh rằng thời gian chạy của thuật toán là $O(n \log \log n)$. Thuật toán sẽ thực hiện $\frac{n}{p}$ phép toán cho mỗi số nguyên tố $p \le n$ trong vòng lặp trong. Do đó, ta cần đánh giá biểu thức sau:

$$\sum_{\substack{p \le n, \\ p \text{ nguyên tố}}} \frac{n}{p} = n \cdot \sum_{\substack{p \le n, \\ p \text{ nguyên tố}}} \frac{1}{p}.$$

Hãy nhớ lại hai sự thật đã biết:

- Số lượng số nguyên tố nhỏ hơn hoặc bằng $n$ xấp xỉ $\frac{n}{\ln n}$.
- Số nguyên tố thứ $k$ xấp xỉ bằng $k \ln k$ (điều này suy ra từ sự thật trước).

Do đó, ta có thể viết tổng theo cách sau:

$$\sum_{\substack{p \le n, \\ p \text{ nguyên tố}}} \frac{1}{p} \approx \frac{1}{2} + \sum_{k=2}^{\frac{n}{\ln n}} \frac{1}{k \ln k}.$$

Ở đây, ta tách số nguyên tố đầu tiên 2 ra khỏi tổng, vì $k = 1$ trong xấp xỉ $k \ln k$ là 0 và gây chia cho 0.

Bây giờ, hãy đánh giá tổng này bằng tích phân của cùng hàm theo $k$ từ 2 đến $\frac{n}{\ln n}$ (ta có thể thực hiện xấp xỉ này vì tổng thực tế liên quan đến tích phân như một xấp xỉ bằng phương pháp hình chữ nhật):

$$\sum_{k=2}^{\frac{n}{\ln n}} \frac{1}{k \ln k} \approx \int_2^{\frac{n}{\ln n}} \frac{1}{k \ln k}\, dk.$$

Nguyên hàm của hàm bị tích là $\ln \ln k$. Sử dụng phép thay thế và bỏ các hạng bậc thấp, ta được kết quả:

$$\int_2^{\frac{n}{\ln n}} \frac{1}{k \ln k}\, dk = \ln \ln \frac{n}{\ln n} - \ln \ln 2 = \ln(\ln n - \ln \ln n) - \ln \ln 2 \approx \ln \ln n.$$

Quay lại tổng ban đầu, ta được đánh giá xấp xỉ:

$$\sum_{\substack{p \le n \\ p \text{ nguyên tố}}} \frac{n}{p} \approx n \ln \ln n + o(n).$$

Bạn có thể tìm chứng minh chặt chẽ hơn (cho đánh giá chính xác hơn trong phạm vi các hệ số không đổi) trong sách của Hardy & Wright Än Introduction to the Theory of Numbers(trang 349).

### 5.6.3 Các tối ưu khác nhau của Sàng Eratosthenes

Điểm yếu lớn nhất của thuật toán là nó "đi qua" bộ nhớ nhiều lần, chỉ thao tác trên từng phần tử riêng lẻ. Điều này không thân thiện với bộ nhớ cache. Vì lý do đó, hằng số ẩn trong $O(n \log \log n)$ khá lớn.

Ngoài ra, bộ nhớ tiêu thụ là một nút thắt cổ chai với $n$ lớn.

Các phương pháp dưới đây cho phép giảm số lượng phép toán thực hiện, cũng như giảm đáng kể bộ nhớ tiêu thụ.

### Sàng đến căn

Rõ ràng, để tìm tất cả số nguyên tố đến $n$, chỉ cần thực hiện sàng lọc bởi các số nguyên tố không vượt quá căn của $n$ là đủ:

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i * i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

Tối ưu này không ảnh hưởng đến độ phức tạp (thực tế, lặp lại chứng minh trên ta được đánh giá $n \ln \ln \sqrt{n} + o(n)$, là tương đương tiệm cận theo tính chất của logarit), dù số lượng phép toán sẽ giảm đáng kể.

### Sàng chỉ với số lẻ

Vì tất cả các số chẵn (trừ 2) là hợp số, ta có thể ngừng kiểm tra số chẵn hoàn toàn. Thay vào đó, ta chỉ cần thao tác với số lẻ.

Đầu tiên, điều này cho phép giảm một nửa bộ nhớ cần thiết. Thứ hai, nó giảm khoảng một nửa số phép toán thực hiện bởi thuật toán.

### Tiêu thụ bộ nhớ và tốc độ phép toán

Ta nên nhận thấy rằng hai triển khai này của Sàng Eratosthenes sử dụng $n$ bit bộ nhớ bằng cách dùng cấu trúc dữ liệu vector<bool>. vector<bool> không phải là một container thông thường lưu trữ một chuỗi bool (vì trong hầu hết kiến trúc máy tính, một bool chiếm một byte bộ nhớ). Nó là một tối ưu hóa bộ nhớ của vector<T>, chỉ tiêu tốn $\frac{N}{8}$ byte bộ nhớ.

Kiến trúc bộ xử lý hiện đại hoạt động hiệu quả hơn nhiều với byte so với bit vì chúng thường không thể truy cập trực tiếp bit. Vì vậy, bên dưới vector<bool> lưu trữ các bit trong một bộ nhớ liền mạch lớn, truy cập bộ nhớ theo khối vài byte, và trích xuất/đặt các bit bằng các thao tác bit như che bit và dịch bit.

Do đó, có một chi phí nhất định khi đọc hoặc ghi bit với vector<bool>, và thường sử dụng vector<char> (dùng 1 byte cho mỗi phần tử, tức là gấp 8 lần bộ nhớ) lại nhanh hơn.

Tuy nhiên, với các triển khai đơn giản của Sàng Eratosthenes, dùng vector<bool> nhanh hơn. Bạn bị giới hạn bởi tốc độ tải dữ liệu vào cache, và do đó dùng ít bộ nhớ hơn

mang lại lợi thế lớn. Một bài kiểm tra (link) cho thấy dùng vector<bool> nhanh hơn từ 1.4x đến 1.7x so với vector<char>.

Những cân nhắc tương tự cũng áp dụng cho bitset. Nó cũng là cách lưu trữ bit hiệu quả, tương tự vector<bool>, nên chỉ chiếm $\frac{N}{8}$ byte bộ nhớ, nhưng chậm hơn một chút khi truy cập phần tử. Trong bài kiểm tra trên, bitset hoạt động kém hơn một chút so với vector<bool>. Một nhược điểm khác của bitset là bạn cần biết kích thước tại thời điểm biên dịch.

### Sàng phân đoạn

Từ tối ưu "sàng đến căn", ta thấy không cần giữ toàn bộ mảng `is_prime`$[1 \dots n]$ mọi lúc. Để sàng, chỉ cần giữ các số nguyên tố đến căn của $n$, tức là `prime`$[1 \dots \sqrt{n}]$, chia toàn bộ phạm vi thành các khối, và sàng từng khối riêng lẻ.

Gọi $s$ là một hằng số xác định kích thước khối, ta có $\lceil \frac{n}{s} \rceil$ khối tất cả, và khối $k$ ($k = 0 \dots \lfloor \frac{n}{s} \rfloor$) chứa các số trong đoạn $[ks; ks + s - 1]$. Ta có thể xử lý từng khối lần lượt, tức là với mỗi khối $k$, ta sẽ duyệt qua tất cả các số nguyên tố (từ 1 đến $\sqrt{n}$) và thực hiện sàng bằng chúng. Cần lưu ý rằng ta phải điều chỉnh chiến lược một chút khi xử lý các số đầu tiên: đầu tiên, tất cả các số nguyên tố từ $[1; \sqrt{n}]$ không nên tự xóa chính chúng; thứ hai, các số 0 và 1 nên được đánh dấu là không nguyên tố. Khi xử lý khối cuối, không nên quên rằng số cuối cần $n$ không nhất thiết nằm ở cuối khối.

Như đã thảo luận trước đó, triển khai điển hình của Sàng Eratosthenes bị giới hạn bởi tốc độ tải dữ liệu vào cache CPU. Bằng cách chia phạm vi các số nguyên tố tiềm năng $[1; n]$ thành các khối nhỏ hơn, ta không bao giờ phải giữ nhiều khối trong bộ nhớ cùng lúc, và tất cả các thao tác đều thân thiện với cache hơn. Vì giờ ta không còn bị giới hạn bởi tốc độ cache, ta có thể thay vector<bool> bằng vector<char>, và tăng thêm hiệu suất vì bộ xử lý có thể xử lý đọc và ghi trực tiếp với byte mà không cần dựa vào thao tác bit để trích xuất từng bit. Bài kiểm tra (link) cho thấy dùng vector<char> nhanh hơn khoảng 3x so với vector<bool> trong tình huống này. Một lời cảnh báo: các con số này có thể khác nhau tùy thuộc vào kiến trúc, trình biên dịch, và mức tối ưu hóa.

Dưới đây là triển khai đếm số lượng số nguyên tố nhỏ hơn hoặc bằng $n$ dùng sàng phân đoạn:

```
int count_primes(int n) {
    const int S = 10000;

    vector<int> primes;
    int nsqrt = sqrt(n);
    vector<char> is_prime(nsqrt + 2, true);
    for (int i = 2; i <= nsqrt; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
```

```
            for (int j = i * i; j <= nsqrt; j += i)
                is_prime[j] = false;
        }
    }

    int result = 0;
    vector<char> block(S);
    for (int k = 0; k * S <= n; k++) {
        fill(block.begin(), block.end(), true);
        int start = k * S;
        for (int p : primes) {
            int start_idx = (start + p - 1) / p;
            int j = max(start_idx, p) * p - start;
            for (; j < S; j += p)
                block[j] = false;
        }
        if (k == 0)
            block[0] = block[1] = false;
        for (int i = 0; i < S && start + i <= n; i++) {
            if (block[i])
                result++;
        }
    }
    return result;
}
```

Thời gian chạy của sàng phân đoạn tương đương với sàng Eratosthenes thông thường (trừ khi kích thước khối rất nhỏ), nhưng bộ nhớ cần thiết giảm xuống $O(\sqrt{n} + S)$ và ta có kết quả cache tốt hơn. Mặt khác, sẽ có một phép chia cho mỗi cặp khối và số nguyên tố từ $[1; \sqrt{n}]$, và điều này sẽ tệ hơn nhiều với kích thước khối nhỏ. Do đó, cần giữ cân bằng khi chọn hằng số $S$. Ta đạt kết quả tốt nhất với kích thước khối từ $10^4$ đến $10^5$.

### Tìm số nguyên tố trong khoảng

Đôi khi ta cần tìm tất cả số nguyên tố trong khoảng $[L, R]$ có kích thước nhỏ (ví dụ $R - L + 1 \approx 1e7$), trong đó $R$ có thể rất lớn (ví dụ $1e12$).

Để giải bài toán này, ta có thể dùng ý tưởng của sàng phân đoạn. Ta tạo trước tất cả các số nguyên tố đến $\sqrt{R}$, và dùng các số nguyên tố đó để đánh dấu tất cả số hợp số trong đoạn $[L, R]$.

```
vector<char> segmentedSieve(long long L, long long R) {
```

```
    // tạo tất cả số nguyên tố đến sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes)
        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}
```

Độ phức tạp thời gian của cách tiếp cận này là $O((R-L+1)\log\log(R)+\sqrt{R}\log\log\sqrt{R})$. Cũng có thể không tạo trước tất cả số nguyên tố:

```
vector<char> segmentedSieveNoPreGen(long long L, long long R) {
    vector<char> isPrime(R - L + 1, true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}
```

Rõ ràng, độ phức tạp tệ hơn, là $O((R-L+1)\log(R)+\sqrt{R})$. Tuy nhiên, nó vẫn chạy rất nhanh trong thực tế.

### Sửa đổi thời gian tuyến tính

Ta có thể sửa đổi thuật toán sao cho chỉ có độ phức tạp thời gian tuyến tính. Cách tiếp cận này được mô tả trong bài viết Sàng tuyến tính. Tuy nhiên, thuật toán này cũng có những điểm yếu riêng.

## 5.7 Sàng tuyến tính

Cho một số $n$, tìm tất cả các số nguyên tố trong đoạn $[2; n]$.

Cách tiêu chuẩn để giải bài toán này là sử dụng sàng Eratosthenes. Thuật toán này rất đơn giản, nhưng có thời gian chạy $O(n \log \log n)$.

Mặc dù có nhiều thuật toán đã biết với thời gian chạy dưới tuyến tính (tức là $o(n)$), thuật toán được mô tả dưới đây thú vị bởi sự đơn giản của nó: nó không phức tạp hơn sàng Eratosthenes cổ điển.

Ngoài ra, thuật toán được đưa ra ở đây tính toán phân tích thừa số nguyên tố của tất cả các số trong đoạn $[2; n]$ như một hiệu ứng phụ, điều này có thể hữu ích trong nhiều ứng dụng thực tế.

Điểm yếu của thuật toán này là sử dụng nhiều bộ nhớ hơn so với sàng Eratosthenes cổ điển: nó yêu cầu một mảng gồm $n$ số, trong khi với sàng Eratosthenes cổ điển, chỉ cần $n$ bit bộ nhớ (ít hơn 32 lần).

Do đó, chỉ có ý nghĩa khi sử dụng thuật toán được mô tả cho các số có bậc khoảng $10^7$ và không lớn hơn.

Thuật toán này được phát triển bởi Paul Pritchard. Nó là một biến thể của Thuật toán 3.3 trong (Pritchard, 1987: xem tài liệu tham khảo ở cuối bài viết).

### 5.7.1 Thuật toán

Mục tiêu của chúng ta là tính thừa số nguyên tố nhỏ nhất $lp[i]$ cho mỗi số $i$ trong đoạn $[2; n]$.

Ngoài ra, ta cần lưu danh sách tất cả các số nguyên tố đã tìm thấy - gọi nó là $pr[]$.

Ta sẽ khởi tạo các giá trị $lp[i]$ bằng 0, nghĩa là ta giả sử tất cả các số đều là nguyên tố. Trong quá trình thực hiện thuật toán, mảng này sẽ được điền dần.

Bây giờ, ta sẽ duyệt qua các số từ 2 đến $n$. Có hai trường hợp cho số hiện tại $i$:

- $lp[i] = 0$ - điều này có nghĩa là $i$ là số nguyên tố, tức là ta chưa tìm thấy bất kỳ thừa số nhỏ hơn nào cho nó. Do đó, ta gán $lp[i] = i$ và thêm $i$ vào cuối danh sách $pr[]$.
- $lp[i] \neq 0$ - điều này có nghĩa là $i$ là hợp số, và thừa số nguyên tố nhỏ nhất của nó là $lp[i]$.

Trong cả hai trường hợp, ta cập nhật giá trị của $lp[]$ cho các số chia hết cho $i$. Tuy nhiên, mục tiêu của ta là học cách làm sao để đặt giá trị $lp[]$ tối đa một lần cho mỗi số. Ta có thể làm điều này như sau:

Hãy xem xét các số $x_j = i \cdot p_j$, trong đó $p_j$ là tất cả các số nguyên tố nhỏ hơn hoặc bằng $lp[i]$ (đây là lý do ta cần lưu danh sách tất cả các số nguyên tố).

Ta sẽ đặt giá trị mới $lp[x_j] = p_j$ cho tất cả các số dạng này.

Chứng minh tính đúng đắn của thuật toán và thời gian chạy của nó sẽ được trình bày sau phần triển khai.

### 5.7.2 Triển khai

```
const int N = 10000000;
vector<int> lp(N+1);
vector<int> pr;

for (int i=2; i <= N; ++i) {
    if (lp[i] == 0) {
        lp[i] = i;
        pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) {
            break;
        }
    }
}
```

### 5.7.3 Chứng minh tính đúng đắn

Ta cần chứng minh rằng thuật toán đặt tất cả giá trị $lp[]$ một cách chính xác, và mỗi giá trị sẽ được đặt đúng một lần. Do đó, thuật toán sẽ có thời gian chạy tuyến tính, vì tất cả các hành động còn lại của thuật toán rõ ràng hoạt động trong $O(n)$.

Chú ý rằng mỗi số $i$ có đúng một biểu diễn dưới dạng:

$$i = lp[i] \cdot x,$$

trong đó $lp[i]$ là thừa số nguyên tố nhỏ nhất của $i$, và số $x$ không có bất kỳ thừa số nguyên tố nào nhỏ hơn $lp[i]$, tức là:

$$lp[i] \leq lp[x].$$

Bây giờ, hãy so sánh điều này với các hành động của thuật toán: thực tế, với mỗi $x$, nó duyệt qua tất cả các số nguyên tố mà nó có thể được nhân với, tức là tất cả các số nguyên tố đến $lp[x]$ bao gồm cả nó, để tạo ra các số dưới dạng đã cho ở trên.

Do đó, thuật toán sẽ duyệt qua mỗi số hợp số đúng một lần, đặt các giá trị $lp[]$ chính xác tại đó. Q.E.D.

### 5.7.4 Thời gian chạy và Bộ nhớ

Mặc dù thời gian chạy $O(n)$ tốt hơn $O(n \log \log n)$ của sàng Eratosthenes cổ điển, sự khác biệt giữa chúng không quá lớn. Trong thực tế, sàng tuyến tính chạy nhanh gần tương đương với một triển khai điển hình của sàng Eratosthenes.

So với các phiên bản tối ưu của sàng Eratosthenes, ví dụ như sàng phân đoạn, nó chậm hơn nhiều.

Xét đến yêu cầu bộ nhớ của thuật toán này - một mảng $lp[]$ độ dài $n$, và một mảng $pr[]$ độ dài $\frac{n}{\ln n}$, thuật toán này dường như tệ hơn sàng cổ điển về mọi mặt.

Tuy nhiên, điểm mạnh của nó là thuật toán này tính toán mảng $lp[]$, cho phép ta tìm phân tích thừa số nguyên tố của bất kỳ số nào trong đoạn $[2; n]$ trong thời gian tỷ lệ với kích thước của phân tích này. Hơn nữa, chỉ cần dùng thêm một mảng phụ sẽ cho phép ta tránh phép chia khi tìm phân tích thừa số.

Việc biết phân tích thừa số của tất cả các số rất hữu ích cho một số bài toán, và thuật toán này là một trong số ít cho phép tìm chúng trong thời gian tuyến tính.

### 5.7.5 Tài liệu tham khảo


- Paul Pritchard, Linear Prime-Number Sieves: a Family Tree, Science of Computer Programming, vol. 9 (1987), pp.17-35.


## 5.8 Kiểm tra số nguyên tố

Bài viết này mô tả nhiều thuật toán để xác định một số có phải là số nguyên tố hay không.

### 5.8.1 Thử chia

Theo định nghĩa, một số nguyên tố không có ước nào khác ngoài 1 và chính nó. Một số hợp số có ít nhất một ước khác, gọi là $d$. Tự nhiên $\frac{n}{d}$ cũng là một ước của $n$. Dễ thấy rằng hoặc $d \leq \sqrt{n}$ hoặc $\frac{n}{d} \leq \sqrt{n}$, do đó một trong hai ước $d$ và $\frac{n}{d}$ là $\leq \sqrt{n}$. Ta có thể dùng thông tin này để kiểm tra tính nguyên tố.

Ta thử tìm một ước không tầm thường bằng cách kiểm tra xem có số nào từ 2 đến $\sqrt{n}$ là ước của $n$ không. Nếu có, thì $n$ chắc chắn không phải số nguyên tố, ngược lại thì là số nguyên tố.

```
bool isPrime(int x) {
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0)
            return false;
```

```
    }
    return x >= 2;
}
```

Đây là dạng đơn giản nhất của kiểm tra số nguyên tố. Bạn có thể tối ưu hàm này khá nhiều, ví dụ chỉ kiểm tra các số lẻ trong vòng lặp, vì số nguyên tố chẵn duy nhất là 2. Nhiều tối ưu như vậy được mô tả trong bài viết về phân tích thừa số nguyên.

### 5.8.2 Kiểm tra nguyên tố Fermat

Đây là một kiểm tra xác suất.

Định lý nhỏ Fermat (xem thêm hàm totient của Euler) phát biểu rằng, với một số nguyên tố $p$ và một số nguyên $a$ nguyên tố cùng nhau với $p$, phương trình sau đúng:

$$a^{p-1} \equiv 1 \pmod{p}$$

Nói chung, định lý này không đúng với số hợp số.

Điều này có thể được dùng để tạo một kiểm tra tính nguyên tố. Ta chọn một số nguyên $2 \le a \le p-2$, và kiểm tra xem phương trình có đúng hay không. Nếu không đúng, ví dụ $a^{p-1} \not\equiv 1 \pmod{p}$, ta biết rằng $p$ không thể là số nguyên tố. Trong trường hợp này, ta gọi cơ số $a$ là nhân chứng Fermat cho tính hợp số của $p$.

Tuy nhiên, cũng có thể phương trình đúng với một số hợp số. Vì vậy, nếu phương trình đúng, ta không có bằng chứng cho tính nguyên tố. Ta chỉ có thể nói rằng $p$ có lẽ là nguyên tố. Nếu hóa ra số đó thực sự là hợp số, ta gọi cơ số $a$ là kẻ nói dối Fermat.

Bằng cách chạy kiểm tra với tất cả các cơ số $a$ có thể, ta thực sự có thể chứng minh một số là nguyên tố. Tuy nhiên, điều này không được thực hiện trong thực tế, vì tốn nhiều công sức hơn so với thử chia. Thay vào đó, kiểm tra sẽ được lặp lại nhiều lần với các lựa chọn ngẫu nhiên cho $a$. Nếu không tìm thấy nhân chứng cho tính hợp số, rất có khả năng số đó thực sự là nguyên tố.

```
bool probablyPrimeFermat(int n, int iter=5) {
    if (n < 4)
        return n == 2 || n == 3;

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (binpower(a, n - 1, n) != 1)
            return false;
    }
    return true;
}
```

Ta dùng Lũy thừa nhị phân để tính lũy thừa $a^{p-1}$ một cách hiệu quả.

Tuy nhiên, có một tin xấu: tồn tại một số hợp số mà $a^{n-1} \equiv 1 \pmod{n}$ đúng với mọi $a$ nguyên tố cùng nhau với $n$, ví dụ số $561 = 3 \cdot 11 \cdot 17$. Những số như vậy được gọi là số Carmichael. Kiểm tra nguyên tố Fermat chỉ có thể phát hiện các số này nếu ta cực kỳ may mắn và chọn một cơ số $a$ với $\gcd(a, n) \neq 1$.

Kiểm tra Fermat vẫn được sử dụng trong thực tế, vì nó rất nhanh và số Carmichael rất hiếm. Ví dụ, chỉ có 646 số như vậy dưới $10^9$.

### 5.8.3 Kiểm tra nguyên tố Miller-Rabin

Kiểm tra Miller-Rabin mở rộng ý tưởng từ kiểm tra Fermat.

Với một số lẻ $n$, $n-1$ là số chẵn và ta có thể phân tích hết các lũy thừa của 2. Ta có thể viết:

$$n - 1 = 2^s \cdot d, \text{ với } d \text{ là số lẻ.}$$

Điều này cho phép ta phân tích phương trình của định lý nhỏ Fermat:

$$\begin{aligned}
a^{n-1} \equiv 1 \pmod{n} &\iff a^{2^s d} - 1 \equiv 0 \pmod{n} \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-1}d} - 1) \equiv 0 \pmod{n} \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1)(a^{2^{s-2}d} - 1) \equiv 0 \pmod{n} \\
&\quad \vdots \\
&\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1) \cdots (a^d + 1)(a^d - 1) \equiv 0 \pmod{n}
\end{aligned}$$

Nếu $n$ là nguyên tố, thì $n$ phải chia hết cho một trong các thừa số này. Và trong kiểm tra nguyên tố Miller-Rabin, ta kiểm tra chính xác phát biểu đó, là phiên bản nghiêm ngặt hơn của kiểm tra Fermat. Với một cơ số $2 \leq a \leq n-2$, ta kiểm tra xem liệu:

$$a^d \equiv 1 \pmod{n}$$

có đúng không, hoặc:

$$a^{2^r d} \equiv -1 \pmod{n}$$

có đúng với một số $0 \leq r \leq s-1$ không.

Nếu ta tìm thấy cơ số $a$ không thỏa mãn bất kỳ đẳng thức nào trên, thì ta đã tìm thấy nhân chứng cho tính hợp số của $n$. Trong trường hợp này, ta đã chứng minh $n$ không phải là số nguyên tố.

Tương tự kiểm tra Fermat, cũng có thể tập hợp các phương trình được thỏa mãn với một số hợp số. Trong trường hợp đó, cơ số $a$ được gọi là kẻ nói dối mạnh. Nếu cơ số $a$ thỏa mãn các phương trình (một trong số chúng), $n$ chỉ là nguyên tố có lẽ mạnh. Tuy nhiên, không tồn tại số như số Carmichael mà mọi cơ số không tầm thường đều nói dối. Thực tế, có thể chứng minh rằng tối đa $\frac{1}{4}$ cơ số có thể là kẻ nói dối mạnh. Nếu $n$ là hợp số, ta có xác suất $\geq 75\%$ rằng một cơ số ngẫu nhiên sẽ cho biết nó là hợp số. Bằng cách

thực hiện nhiều lần lặp với các cơ số ngẫu nhiên khác nhau, ta có thể xác định với xác suất rất cao liệu số đó thực sự là nguyên tố hay hợp số.

Dưới đây là triển khai cho số nguyên 64 bit:

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};

bool MillerRabin(u64 n, int iter=5) { // trả về true nếu n có lẽ là nguyên tố, ngược l
    if (n < 4)
        return n == 2 || n == 3;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }
```

```
    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

Trước khi kiểm tra Miller-Rabin, bạn có thể kiểm tra thêm xem một trong những số nguyên tố đầu tiên có phải là ước không. Điều này có thể tăng tốc kiểm tra rất nhiều, vì hầu hết số hợp số có ước nguyên tố rất nhỏ. Ví dụ, 88% số có thừa số nguyên tố nhỏ hơn 100.

### 5.8.4 Phiên bản xác định

Miller đã chỉ ra rằng có thể làm thuật toán này thành xác định bằng cách chỉ kiểm tra tất cả các cơ số $\leq O((\ln n)^2)$. Bach sau đó đưa ra một giới hạn cụ thể, chỉ cần kiểm tra các cơ số $a \leq 2\ln(n)^2$.

Đây vẫn là một số lượng cơ số khá lớn. Vì vậy, người ta đã đầu tư khá nhiều sức mạnh tính toán để tìm các giới hạn thấp hơn. Hóa ra, để kiểm tra một số nguyên 32 bit, chỉ cần kiểm tra 4 cơ số nguyên tố đầu tiên: 2, 3, 5 và 7. Số hợp số nhỏ nhất thất bại kiểm tra này là $3,215,031,751 = 151 \cdot 751 \cdot 28351$. Và để kiểm tra số nguyên 64 bit, đủ để kiểm tra 12 cơ số nguyên tố đầu tiên: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31 và 37.

Điều này dẫn đến triển khai xác định sau:

```
bool MillerRabin(u64 n) { // trả về true nếu n là nguyên tố, ngược lại trả về false.
    if (n < 2)
        return false;

    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
```

```
            return false;
    }
    return true;
}
```

Cũng có thể thực hiện kiểm tra chỉ với 7 cơ số: 2, 325, 9375, 28178, 450775, 9780504 và 1795265022. Tuy nhiên, vì các số này (trừ 2) không phải là nguyên tố, bạn cần kiểm tra thêm xem số đang kiểm tra có bằng bất kỳ ước nguyên tố nào của các cơ số đó không: 2, 3, 5, 13, 19, 73, 193, 407521, 299210837.

## 5.9 Phân tích số nguyên

Trong bài viết này, chúng tôi liệt kê một số thuật toán để phân tích số nguyên thành thừa số, mỗi thuật toán có thể nhanh hoặc chậm ở các mức độ khác nhau tùy thuộc vào dữ liệu đầu vào.

Lưu ý rằng, nếu số mà bạn muốn phân tích thực sự là một số nguyên tố, phần lớn các thuật toán sẽ chạy rất chậm. Điều này đặc biệt đúng với các thuật toán phân tích của Fermat, Pollard $p-1$ và Pollard rho. Do đó, việc thực hiện một bài kiểm tra nguyên tố xác suất (hoặc một bài kiểm tra nguyên tố nhanh xác định) trước khi cố gắng phân tích số là hợp lý nhất.

### 5.9.1 Phân tích thử

Đây là thuật toán cơ bản nhất để tìm phân tích nguyên tố.

Chúng ta chia số cho từng ước số có thể có $d$. Có thể thấy rằng không thể nào tất cả các thừa số nguyên tố của một số hợp số $n$ đều lớn hơn $\sqrt{n}$. Do đó, chúng ta chỉ cần kiểm tra các ước số $2 \le d \le \sqrt{n}$, điều này cho chúng ta phân tích nguyên tố trong thời gian $O(\sqrt{n})$. (Đây là thời gian giả đa thức, tức là đa thức theo giá trị của đầu vào nhưng là hàm mũ theo số bit của đầu vào.)

Ước số nhỏ nhất phải là một số nguyên tố. Chúng ta loại bỏ số đã phân tích, và tiếp tục quá trình. Nếu không tìm thấy ước số nào trong khoảng $[2; \sqrt{n}]$, thì số đó phải là số nguyên tố.

```
vector<long long> trial_division1(long long n) {
    vector<long long> factorization;
    for (long long d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
```

```
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

### 5.9.2 Phân tích bánh xe

Đây là một tối ưu hóa của phương pháp phân tích thử. Khi chúng ta biết số không chia hết cho 2, chúng ta không cần kiểm tra các số chẵn khác. Điều này giảm còn 50% số lượng số cần kiểm tra. Sau khi phân tích hết các thừa số 2 và nhận được một số lẻ, chúng ta chỉ cần bắt đầu từ 3 và chỉ đếm các số lẻ khác.

```
vector<long long> trial_division2(long long n) {
    vector<long long> factorization;
    while (n % 2 == 0) {
        factorization.push_back(2);
        n /= 2;
    }
    for (long long d = 3; d * d <= n; d += 2) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

Phương pháp này có thể được mở rộng thêm. Nếu số không chia hết cho 3, chúng ta cũng có thể bỏ qua các bội số của 3 trong các phép tính sau này. Vì vậy, chúng ta chỉ cần kiểm tra các số $5, 7, 11, 13, 17, 19, 23, \ldots$. Chúng ta có thể nhận thấy một mẫu của các số còn lại này. Chúng ta cần kiểm tra tất cả các số với $d \mod 6 = 1$ và $d \mod 6 = 5$. Điều này chỉ để lại $33,3\%$ số cần kiểm tra. Chúng ta có thể triển khai điều này bằng cách phân tích trước các số nguyên tố 2 và 3, sau đó bắt đầu từ 5 và chỉ đếm các số dư 1 và 5 theo modulo 6.

Dưới đây là một triển khai cho các số nguyên tố 2, 3 và 5. Việc lưu các bước nhảy trong một mảng là thuận tiện.

```
vector<long long> trial_division3(long long n) {
    vector<long long> factorization;
```

```
    for (int d : {2, 3, 5}) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    static array<int, 8> increments = {4, 2, 4, 2, 4, 6, 2, 6};
    int i = 0;
    for (long long d = 7; d * d <= n; d += increments[i++]) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
        if (i == 8)
            i = 0;
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

Nếu chúng ta tiếp tục mở rộng phương pháp này để bao gồm nhiều số nguyên tố hơn nữa, tỷ lệ phần trăm tốt hơn có thể đạt được, nhưng danh sách các bước nhảy sẽ trở nên lớn hơn.

### 5.9.3 Số nguyên tố tính trước

Mở rộng phương pháp phân tích bánh xe vô hạn, chúng ta sẽ chỉ còn lại các số nguyên tố để kiểm tra. Một cách tốt để thực hiện việc này là tính trước tất cả các số nguyên tố bằng Sàng Eratosthenes cho đến $\sqrt{n}$, và kiểm tra từng số một.

```
vector<long long> primes;

vector<long long> trial_division4(long long n) {
    vector<long long> factorization;
    for (long long d : primes) {
        if (d * d > n)
            break;
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
```

```
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

### 5.9.4 Phương pháp phân tích của Fermat

Chúng ta có thể viết một số hợp số lẻ $n = p \cdot q$ dưới dạng hiệu của hai bình phương $n = a^2 - b^2$:

$$n = \left(\frac{p+q}{2}\right)^2 - \left(\frac{p-q}{2}\right)^2$$

Phương pháp phân tích của Fermat cố gắng khai thác điều này bằng cách đoán bình phương đầu tiên $a^2$, và kiểm tra xem phần còn lại, $b^2 = a^2 - n$, có phải là một số bình phương hay không. Nếu đúng, thì chúng ta đã tìm thấy các thừa số $a - b$ và $a + b$ của $n$.

```
int fermat(int n) {
    int a = ceil(sqrt(n));
    int b2 = a*a - n;
    int b = round(sqrt(b2));
    while (b * b != b2) {
        a = a + 1;
        b2 = a*a - n;
        b = round(sqrt(b2));
    }
    return a - b;
}
```

Phương pháp phân tích này có thể rất nhanh nếu hiệu giữa hai thừa số $p$ và $q$ nhỏ. Thuật toán chạy trong thời gian $O(|p - q|)$. Tuy nhiên, trong thực tế, phương pháp này hiếm khi được sử dụng. Khi các thừa số cách xa nhau hơn, nó cực kỳ chậm.

Tuy nhiên, vẫn còn rất nhiều tùy chọn tối ưu hóa liên quan đến phương pháp này. Bằng cách xem xét các bình phương $a^2$ theo modulo một số nhỏ cố định, có thể thấy rằng một số giá trị $a$ không cần được xem xét, vì chúng không thể tạo ra một số bình phương $a^2 - n$.

### 5.9.5 Phương pháp Pollard $p - 1$

Rất có khả năng ít nhất một thừa số của một số là $B$-powersmooth với $B$ nhỏ. $B$-powersmooth nghĩa là mọi lũy thừa nguyên tố $d^k$ chia hết cho $p - 1$ đều nhỏ hơn hoặc

bằng $B$. Ví dụ, phân tích nguyên tố của 4817191 là $1303 \cdot 3697$. Và các thừa số lần lượt là 31-powersmooth và 16-powersmooth, vì $1303 - 1 = 2 \cdot 3 \cdot 7 \cdot 31$ và $3697 - 1 = 2^4 \cdot 3 \cdot 7 \cdot 11$. Năm 1974, John Pollard đã phát minh ra một phương pháp để trích xuất các thừa số $B$-powersmooth từ một số hợp số.

Ý tưởng đến từ định lý nhỏ của Fermat. Giả sử phân tích của $n$ là $n = p \cdot q$. Định lý nói rằng nếu $a$ nguyên tố cùng nhau với $p$, thì phát biểu sau đúng:

$$a^{p-1} \equiv 1 \pmod{p}$$

Điều này cũng có nghĩa là:

$$\left(a^{(p-1)}\right)^k \equiv a^{k\cdot(p-1)} \equiv 1 \pmod{p}.$$

Vậy với bất kỳ $M$ nào mà $p - 1 \mid M$, ta biết rằng $a^M \equiv 1$. Điều này có nghĩa là $a^M - 1 = p \cdot r$, và do đó $p \mid \gcd(a^M - 1, n)$.

Do đó, nếu $p - 1$ cho một thừa số $p$ của $n$ chia hết cho $M$, ta có thể trích xuất một thừa số bằng thuật toán Euclid.

Rõ ràng, $M$ nhỏ nhất là bội số của mọi số $B$-powersmooth là $\operatorname{lcm}(1,\ 2\ ,3\ ,4\ ,\ \ldots,\ B)$. Hoặc thay vào đó:

$$M = \prod_{\text{số nguyên tố } q \le B} q^{\lfloor \log_q B \rfloor}$$

Lưu ý, nếu $p-1$ chia hết cho $M$ với mọi thừa số nguyên tố $p$ của $n$, thì $\gcd(a^M - 1, n)$ sẽ chỉ là $n$. Trong trường hợp này, chúng ta không nhận được thừa số. Do đó, chúng ta sẽ cố gắng thực hiện gcd nhiều lần, trong khi tính toán $M$.

Một số số hợp số không có thừa số $B$-powersmooth với $B$ nhỏ. Ví dụ, các thừa số của số hợp số $100\,000\,000\,000\,000\,493 = 763\,013 \cdot 131\,059\,365\,961$ là 190 753-powersmooth và 1 092 161 383-powersmooth. Chúng ta sẽ phải chọn $B >= 190\,753$ để phân tích số này.

Trong triển khai sau, chúng ta bắt đầu với $B = 10$ và tăng $B$ sau mỗi lần lặp.

```
long long pollards_p_minus_1(long long n) {
    int B = 10;
    long long g = 1;
    while (B <= 1000000 && g < n) {
        long long a = 2 + rand() %  (n - 3);
        g = gcd(a, n);
        if (g > 1)
            return g;

        // tính a^M
        for (int p : primes) {
            if (p >= B)
```

```
                continue;
            long long p_power = 1;
            while (p_power * p <= B)
                p_power *= p;
            a = power(a, p_power, n);

            g = gcd(a - 1, n);
            if (g > 1 && g < n)
                return g;
        }
        B *= 2;
    }
    return 1;
}
```

Quan sát rằng đây là một thuật toán xác suất. Hệ quả là có khả năng thuật toán không thể tìm thấy thừa số nào cả.

Độ phức tạp là $O(B \log B \log^2 n)$ cho mỗi lần lặp.

### 5.9.6 Thuật toán rho của Pollard

Thuật toán Rho của Pollard là một thuật toán phân tích khác từ John Pollard.

Giả sử phân tích nguyên tố của một số là $n = pq$. Thuật toán xem xét một chuỗi giả ngẫu nhiên $\{x_i\} = \{x_0,\ f(x_0),\ f(f(x_0)),\ \ldots\}$ trong đó $f$ là một hàm đa thức, thường chọn $f(x) = (x^2 + c) \mod n$ với $c = 1$.

Trong trường hợp này, chúng ta không quan tâm đến chuỗi $\{x_i\}$. Chúng ta quan tâm hơn đến chuỗi $\{x_i \mod p\}$. Vì $f$ là hàm đa thức, và tất cả các giá trị nằm trong khoảng $[0;\ p)$, chuỗi này cuối cùng sẽ hội tụ thành một vòng lặp. Nghịch lý sinh nhật thực sự cho thấy số phần tử kỳ vọng là $O(\sqrt{p})$ cho đến khi lặp lại bắt đầu. Nếu $p$ nhỏ hơn $\sqrt{n}$, thì sự lặp lại có thể sẽ bắt đầu trong $O(\sqrt[4]{n})$.

Dưới đây là hình dung của một chuỗi $\{x_i \mod p\}$ với $n = 2206637$, $p = 317$, $x_0 = 2$ và $f(x) = x^2 + 1$. Từ dạng của chuỗi, bạn có thể thấy rất rõ tại sao thuật toán được gọi là thuật toán $\rho$ của Pollard.

#### Phát hiện vòng lặp của Floyd

Thuật toán này tìm một vòng lặp bằng cách sử dụng hai con trỏ di chuyển qua chuỗi với tốc độ khác nhau. Trong mỗi lần lặp, con trỏ đầu tiên sẽ tiến lên một phần tử, trong khi con trỏ thứ hai tiến đến mọi phần tử thứ hai. Sử dụng ý tưởng này, dễ dàng nhận thấy rằng nếu có một vòng lặp, tại một số điểm, con trỏ thứ hai sẽ quay lại gặp con trỏ đầu

tiên trong các vòng lặp. Nếu độ dài vòng lặp là $\lambda$ và $\mu$ là chỉ số đầu tiên mà vòng lặp bắt đầu, thì thuật toán sẽ chạy trong thời gian $O(\lambda + \mu)$.

Thuật toán này còn được gọi là thuật toán Rùa và Thỏ, dựa trên câu chuyện trong đó một con rùa (con trỏ chậm) và một con thỏ (con trỏ nhanh) đua nhau.

```
function floyd(f, x0):
    tortoise = x0
    hare = f(x0)
    while tortoise != hare:
        tortoise = f(tortoise)
        hare = f(f(hare))
    return true
```

### Triển khai

Dưới đây là một triển khai sử dụng thuật toán phát hiện vòng lặp của Floyd. Thuật toán thường chạy trong thời gian $O(\sqrt[4]{n}\log(n))$.

```
long long mult(long long a, long long b, long long mod) {
    return (__int128)a * b % mod;
}

long long f(long long x, long long c, long long mod) {
    return (mult(x, x, mod) + c) % mod;
}

long long rho(long long n, long long x0=2, long long c=1) {
    long long x = x0;
    long long y = x0;
    long long g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(y, c, n);
        y = f(y, c, n);
        g = gcd(abs(x - y), n);
    }
    return g;
}
```

## 5.10 Hàm Totient của Euler

Hàm totient của Euler, còn được gọi là hàm phi $\phi(n)$, đếm số lượng số nguyên từ 1 đến $n$ (bao gồm cả $n$), mà nguyên tố cùng nhau với $n$. Hai số được gọi là nguyên tố cùng nhau nếu ước chung lớn nhất của chúng bằng 1 (1 được coi là nguyên tố cùng nhau với bất kỳ số nào).

Dưới đây là giá trị của $\phi(n)$ cho vài số nguyên dương đầu tiên:

| $n$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\phi(n)$ | 1 | 1 | 2 | 2 | 4 | 2 | 6 | 4 | 6 | 4 | 10 | 4 | 12 | 6 | 8 | 8 | 16 | 6 | 18 | 8 | 12 |

### 5.10.1 Tính chất

Các tính chất sau của hàm totient Euler đủ để tính toán nó cho bất kỳ số nào:

- Nếu $p$ là một số nguyên tố, thì $\gcd(p, q) = 1$ với mọi $1 \leq q < p$. Do đó ta có:

$$\phi(p) = p - 1.$$

- Nếu $p$ là một số nguyên tố và $k \geq 1$, thì có chính xác $p^k/p$ số từ 1 đến $p^k$ chia hết cho $p$. Điều này cho ta:

$$\phi(p^k) = p^k - p^{k-1}.$$

- Nếu $a$ và $b$ nguyên tố cùng nhau, thì:

$$\phi(ab) = \phi(a) \cdot \phi(b).$$

Quan hệ này không dễ thấy ngay lập tức. Nó xuất phát từ định lý thặng dư Trung Hoa. Định lý thặng dư Trung Hoa đảm bảo rằng, với mỗi $0 \leq x < a$ và mỗi $0 \leq y < b$, tồn tại duy nhất một $0 \leq z < ab$ sao cho $z \equiv x \pmod{a}$ và $z \equiv y \pmod{b}$. Không khó để chỉ ra rằng $z$ nguyên tố cùng nhau với $ab$ nếu và chỉ nếu $x$ nguyên tố cùng nhau với $a$ và $y$ nguyên tố cùng nhau với $b$. Do đó, số lượng số nguyên tố cùng nhau với $ab$ bằng tích của số lượng số nguyên tố cùng nhau với $a$ và $b$.

Nói chung, với $a$ và $b$ không nguyên tố cùng nhau, ta có công thức:

$$\phi(ab) = \phi(a) \cdot \phi(b) \cdot \frac{d}{\phi(d)},$$

với $d = \gcd(a, b)$.

Như vậy, sử dụng ba tính chất đầu tiên, ta có thể tính $\phi(n)$ thông qua phân tích $n$ thành thừa số nguyên tố. Nếu $n = {p_1}^{a_1} \cdot {p_2}^{a_2} \cdots {p_k}^{a_k}$, trong đó $p_i$ là các thừa số nguyên tố của $n$, thì:

$$\begin{aligned}
\phi(n) &= \phi({p_1}^{a_1}) \cdot \phi({p_2}^{a_2}) \cdots \phi({p_k}^{a_k}) \\
&= \left({p_1}^{a_1} - {p_1}^{a_1-1}\right) \cdot \left({p_2}^{a_2} - {p_2}^{a_2-1}\right) \cdots \left({p_k}^{a_k} - {p_k}^{a_k-1}\right) \\
&= p_1^{a_1} \cdot \left(1 - \frac{1}{p_1}\right) \cdot p_2^{a_2} \cdot \left(1 - \frac{1}{p_2}\right) \cdots p_k^{a_k} \cdot \left(1 - \frac{1}{p_k}\right) \\
&= n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right).
\end{aligned}$$

### 5.10.2 Triển khai

Dưới đây là một triển khai sử dụng phân tích trong $O(\sqrt{n})$:

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

### 5.10.3 Hàm totient từ 1 đến $n$ trong $O(n \log \log n)$

Nếu ta cần tính totient của tất cả các số từ 1 đến $n$, thì việc phân tích từng số $n$ là không hiệu quả. Ta có thể sử dụng ý tưởng tương tự như Sàng Eratosthenes. Nó vẫn dựa trên tính chất đã nêu ở trên, nhưng thay vì cập nhật kết quả tạm thời cho mỗi thừa số nguyên tố cho từng số, ta tìm tất cả các số nguyên tố và với mỗi số nguyên tố, cập nhật kết quả tạm thời của tất cả các số chia hết cho số nguyên tố đó.

Vì cách tiếp cận này về cơ bản giống với Sàng Eratosthenes, độ phức tạp cũng sẽ tương tự: $O(n \log \log n)$.

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
```

```
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

### 5.10.4 Tính chất tổng ước

Tính chất thú vị này được Gauss thiết lập:

$$\sum_{d|n} \phi(d) = n,$$

trong đó tổng được tính trên tất cả các ước dương $d$ của $n$.

Ví dụ, các ước của 10 là 1, 2, 5 và 10. Do đó, $\phi(1)+\phi(2)+\phi(5)+\phi(10) = 1+1+4+4 = 10$.

### 5.10.5 Tính totient từ 1 đến $n$ bằng tính chất tổng ước

Tính chất tổng ước cũng cho phép tính totient của tất cả các số từ 1 đến $n$. Triển khai này đơn giản hơn một chút so với triển khai trước dựa trên Sàng Eratosthenes, tuy nhiên có độ phức tạp hơi tệ hơn: $O(n \log n)$.

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    phi[0] = 0;
    phi[1] = 1;
    for (int i = 2; i <= n; i++)
        phi[i] = i - 1;

    for (int i = 2; i <= n; i++)
        for (int j = 2 * i; j <= n; j += i)
              phi[j] -= phi[i];
}
```

### 5.10.6 Ứng dụng trong định lý Euler

Tính chất nổi tiếng và quan trọng nhất của hàm totient Euler được thể hiện trong định lý Euler:

$$a^{\phi(m)} \equiv 1 \pmod{m} \quad \text{nếu } a \text{ và } m \text{ nguyên tố cùng nhau.}$$

Trong trường hợp đặc biệt khi $m$ là số nguyên tố, định lý Euler biến thành định lý nhỏ của Fermat:

$$a^{m-1} \equiv 1 \pmod{m}.$$

Định lý Euler và hàm totient Euler xuất hiện khá thường xuyên trong các ứng dụng thực tế, ví dụ cả hai đều được sử dụng để tính nghịch đảo nhân modulo.

Hệ quả tức thì là ta cũng có đẳng thức:

$$a^n \equiv a^{n \bmod \phi(m)} \pmod{m}.$$

Điều này cho phép tính $x^n \bmod m$ với $n$ rất lớn, đặc biệt nếu $n$ là kết quả của một phép tính khác, vì nó cho phép tính $n$ dưới một modulo.

### 5.10.7 Lý thuyết nhóm

$$\phi(n) \text{ là bậc của nhóm nhân modulo } n, \text{ tức là nhóm các đơn vị } (\mathbb{Z}/n\mathbb{Z})^\times.$$

Các phần tử trong nhóm này là những số nguyên dương nhỏ hơn $n$ và nguyên tố cùng nhau với $n$.

Bậc nhân của một phần tử $a$ modulo $n$, ký hiệu $\mathrm{ord}_n(a)$, là số $k > 0$ nhỏ nhất sao cho $a^k \equiv 1 \pmod{m}$. $\mathrm{ord}_n(a)$ là kích thước của nhóm con được tạo ra bởi $a$, nên theo định lý Lagrange, bậc nhân của bất kỳ $a$ nào cũng phải chia hết cho $\phi(n)$. Nếu bậc nhân của $a$ là $\phi(n)$, giá trị lớn nhất có thể, thì $a$ là một căn nguyên thủy và nhóm là nhóm cyclic theo định nghĩa.

### 5.10.8 Khái quát hóa

Có một phiên bản ít được biết đến của đẳng thức cuối cùng, cho phép tính $x^n \bmod m$ một cách hiệu quả ngay cả khi $x$ và $m$ không nguyên tố cùng nhau. Với $x, m$ tùy ý và $n \geq \log_2 m$:

$$x^n \equiv x^{\phi(m)+[n \bmod \phi(m)]} \mod m.$$

**Chứng minh:**

Gọi $p_1, \ldots, p_t$ là các thừa số nguyên tố chung của $x$ và $m$, và $k_i$ là số mũ của chúng trong $m$. Với những giá trị này, ta định nghĩa $a = p_1^{k_1} \ldots p_t^{k_t}$, điều này khiến $\frac{m}{a}$ nguyên tố

cùng nhau với $x$. Gọi $k$ là số nhỏ nhất sao cho $a$ chia hết cho $x^k$. Giả sử $n \geq k$, ta có thể viết:

$$\begin{aligned} x^n \bmod m &= \frac{x^k}{a} a x^{n-k} \bmod m \\ &= \frac{x^k}{a} \left(a x^{n-k} \bmod m\right) \bmod m \\ &= \frac{x^k}{a} \left(a x^{n-k} \bmod a\frac{m}{a}\right) \bmod m \\ &= \frac{x^k}{a} a \left(x^{n-k} \bmod \frac{m}{a}\right) \bmod m \\ &= x^k \left(x^{n-k} \bmod \frac{m}{a}\right) \bmod m. \end{aligned}$$

Đẳng thức giữa dòng thứ ba và thứ tư xuất phát từ thực tế rằng $ab \bmod ac = a(b \bmod c)$. Thật vậy, nếu $b = cd + r$ với $r < c$, thì $ab = acd + ar$ với $ar < ac$.

Vì $x$ và $\frac{m}{a}$ nguyên tố cùng nhau, ta có thể áp dụng định lý Euler và thu được công thức hiệu quả (vì $k$ rất nhỏ; thực tế $k \leq \log_2 m$):

$$x^n \bmod m = x^k \left(x^{n-k \bmod \phi\left(\frac{m}{a}\right)} \bmod \frac{m}{a}\right) \bmod m.$$

Công thức này khó áp dụng, nhưng ta có thể dùng nó để phân tích hành vi của $x^n \bmod m$. Ta thấy rằng chuỗi các lũy thừa $(x^1 \bmod m, x^2 \bmod m, x^3 \bmod m, \dots)$ đi vào một chu kỳ có độ dài $\phi\left(\frac{m}{a}\right)$ sau $k$ phần tử đầu tiên (hoặc ít hơn). $\phi\left(\frac{m}{a}\right)$ chia hết cho $\phi(m)$ (vì $a$ và $\frac{m}{a}$ nguyên tố cùng nhau nên $\phi(a) \cdot \phi\left(\frac{m}{a}\right) = \phi(m)$), do đó ta cũng có thể nói rằng chu kỳ có độ dài $\phi(m)$. Và vì $\phi(m) \geq \log_2 m \geq k$, ta kết luận được công thức đơn giản hơn, như mong muốn:

$$x^n \equiv x^{\phi(m)} x^{(n-\phi(m)) \bmod \phi(m)} \bmod m \equiv x^{\phi(m)+[n \bmod \phi(m)]} \bmod m.$$

## 5.11 Số lượng ước số / Tổng các ước số

Trong bài viết này, chúng tôi thảo luận cách tính số lượng ước số $d(n)$ và tổng các ước số $\sigma(n)$ của một số cho trước $n$.

### 5.11.1 Số lượng ước số

Rõ ràng rằng phân tích nguyên tố của một ước số $d$ phải là một tập con của phân tích nguyên tố của $n$, ví dụ $6 = 2 \cdot 3$ là một ước số của $60 = 2^2 \cdot 3 \cdot 5$. Vì vậy, ta chỉ cần tìm tất cả các tập con khác nhau của phân tích nguyên tố của $n$.

Thông thường, số lượng tập con là $2^x$ với một tập hợp có $x$ phần tử. Tuy nhiên, điều này không còn đúng nếu có các phần tử lặp lại trong tập hợp. Trong trường hợp của

chúng ta, một số thừa số nguyên tố có thể xuất hiện nhiều lần trong phân tích nguyên tố của $n$.

Nếu một thừa số nguyên tố $p$ xuất hiện $e$ lần trong phân tích nguyên tố của $n$, thì ta có thể sử dụng thừa số $p$ tối đa $e$ lần trong tập con. Điều này có nghĩa là ta có $e+1$ lựa chọn.

Do đó, nếu phân tích nguyên tố của $n$ là $p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, trong đó $p_i$ là các số nguyên tố khác nhau, thì số lượng ước số là:

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdots (e_k + 1).$$

Một cách để hình dung điều này là như sau:

- Nếu chỉ có một thừa số nguyên tố duy nhất $n = p_1^{e_1}$, thì rõ ràng có $e_1 + 1$ ước số $(1, p_1, p_1^2, \ldots, p_1^{e_1})$.
- Nếu có hai thừa số nguyên tố khác nhau $n = p_1^{e_1} \cdot p_2^{e_2}$, thì bạn có thể sắp xếp tất cả các ước số dưới dạng một bảng:

| | $1$ | $p_2$ | $p_2^2$ | $\ldots$ | $p_2^{e_2}$ |
|---|---|---|---|---|---|
| $1$ | $1$ | $p_2$ | $p_2^2$ | $\ldots$ | $p_2^{e_2}$ |
| $p_1$ | $p_1$ | $p_1 \cdot p_2$ | $p_1 \cdot p_2^2$ | $\ldots$ | $p_1 \cdot p_2^{e_2}$ |
| $p_1^2$ | $p_1^2$ | $p_1^2 \cdot p_2$ | $p_1^2 \cdot p_2^2$ | $\ldots$ | $p_1^2 \cdot p_2^{e_2}$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ |
| $p_1^{e_1}$ | $p_1^{e_1}$ | $p_1^{e_1} \cdot p_2$ | $p_1^{e_1} \cdot p_2^2$ | $\ldots$ | $p_1^{e_1} \cdot p_2^{e_2}$ |

  Vậy số lượng ước số rõ ràng là $(e_1 + 1) \cdot (e_2 + 1)$.

- Lập luận tương tự có thể được áp dụng nếu có nhiều hơn hai thừa số nguyên tố khác nhau.

```
long long numberOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);
            total *= e + 1;
        }
    }
```

```
    if (num > 1) {
        total *= 2;
    }
    return total;
}
```

### 5.11.2 Tổng các ước số

Ta có thể sử dụng lập luận tương tự như phần trước.

- Nếu chỉ có một thừa số nguyên tố duy nhất $n = p_1^{e_1}$, thì tổng là:

$$1 + p_1 + p_1^2 + \cdots + p_1^{e_1} = \frac{p_1^{e_1+1} - 1}{p_1 - 1}.$$

- Nếu có hai thừa số nguyên tố khác nhau $n = p_1^{e_1} \cdot p_2^{e_2}$, thì ta có thể lập bảng tương tự như trước. Điểm khác biệt là giờ đây ta muốn tính tổng thay vì đếm số phần tử. Dễ thấy rằng tổng của mỗi tổ hợp có thể được biểu diễn như sau:

$$\begin{aligned}&\left(1 + p_1 + p_1^2 + \cdots + p_1^{e_1}\right) \cdot \left(1 + p_2 + p_2^2 + \cdots + p_2^{e_2}\right)\\ &= \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1}.\end{aligned}$$

- Nói chung, với $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, ta nhận được công thức:

$$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \cdots \frac{p_k^{e_k+1} - 1}{p_k - 1}.$$

```
long long SumOfDivisors(long long num) {
    long long total = 1;

    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);

            long long sum = 0, pow = 1;
            do {
```

```
                sum += pow;
                pow *= i;
            } while (e-- > 0);
            total *= sum;
        }
    }
    if (num > 1) {
        total *= (1 + num);
    }
    return total;
}
```

### 5.11.3 Hàm nhân tính

Hàm nhân tính là một hàm $f(x)$ thỏa mãn:

$$f(a \cdot b) = f(a) \cdot f(b),$$

nếu $a$ và $b$ nguyên tố cùng nhau.

Cả $d(n)$ và $\sigma(n)$ đều là các hàm nhân tính.

Các hàm nhân tính có rất nhiều tính chất thú vị, có thể rất hữu ích trong các bài toán lý thuyết số. Ví dụ, phép chập Dirichlet của hai hàm nhân tính cũng là một hàm nhân tính.

## 5.12 Nghịch đảo nhân modulo

### 5.12.1 Định nghĩa

Nghịch đảo nhân modulo của một số nguyên $a$ là một số nguyên $x$ sao cho $a \cdot x$ đồng dư với 1 theo modulo $m$. Nói cách khác, ta muốn tìm một số nguyên $x$ thỏa mãn:

$$a \cdot x \equiv 1 \mod m.$$

Ta cũng sẽ ký hiệu $x$ đơn giản là $a^{-1}$.

Cần lưu ý rằng nghịch đảo modulo không phải lúc nào cũng tồn tại. Ví dụ, lấy $m = 4$, $a = 2$. Bằng cách kiểm tra tất cả các giá trị có thể theo modulo $m$, ta sẽ thấy không thể tìm được $a^{-1}$ thỏa mãn phương trình trên. Có thể chứng minh rằng nghịch đảo modulo tồn tại nếu và chỉ nếu $a$ và $m$ nguyên tố cùng nhau (tức là $\gcd(a, m) = 1$).

Trong bài viết này, chúng tôi trình bày hai phương pháp để tìm nghịch đảo modulo trong trường hợp nó tồn tại, và một phương pháp để tìm nghịch đảo modulo cho tất cả các số trong thời gian tuyến tính.

### 5.12.2 Tìm nghịch đảo modulo bằng thuật toán Euclid mở rộng

Xem xét phương trình sau (với $x$ và $y$ là ẩn số):

$$a \cdot x + m \cdot y = 1.$$

Đây là một phương trình Diophantine tuyến tính với hai biến. Như đã chỉ ra trong bài viết liên quan, khi $\gcd(a, m) = 1$, phương trình có nghiệm và có thể được tìm bằng thuật toán Euclid mở rộng. Lưu ý rằng $\gcd(a, m) = 1$ cũng là điều kiện để nghịch đảo modulo tồn tại.

Bây giờ, nếu ta lấy modulo $m$ của cả hai vế, ta có thể loại bỏ $m \cdot y$, và phương trình trở thành:

$$a \cdot x \equiv 1 \mod m.$$

Do đó, nghịch đảo modulo của $a$ là $x$.

Triển khai như sau:

```
int x, y;
int g = extended_euclidean(a, m, x, y);
if (g != 1) {
    cout << "Không có nghiệm!";
}
else {
    x = (x % m + m) % m;
    cout << x << endl;
}
```

Lưu ý cách ta điều chỉnh $x$. Kết quả $x$ từ thuật toán Euclid mở rộng có thể âm, nên $x\%m$ cũng có thể âm, và ta cần cộng thêm $m$ để biến nó thành dương.

### 5.12.3 Tìm nghịch đảo modulo bằng lũy thừa nhị phân

Một phương pháp khác để tìm nghịch đảo modulo là sử dụng định lý Euler, định lý này phát biểu rằng nếu $a$ và $m$ nguyên tố cùng nhau, thì đẳng thức sau là đúng:

$$a^{\phi(m)} \equiv 1 \mod m,$$

trong đó $\phi$ là hàm totient của Euler. Một lần nữa, lưu ý rằng $a$ và $m$ nguyên tố cùng nhau cũng là điều kiện để nghịch đảo modulo tồn tại.

Nếu $m$ là một số nguyên tố, điều này đơn giản hóa thành định lý nhỏ của Fermat:

$$a^{m-1} \equiv 1 \mod m.$$

Nhân cả hai vế của các phương trình trên với $a^{-1}$, ta được:

- Với một modulo $m$ bất kỳ (nhưng nguyên tố cùng nhau với $a$): $a^{\phi(m)-1} \equiv a^{-1} \mod m$.
- Với một modulo $m$ là số nguyên tố: $a^{m-2} \equiv a^{-1} \mod m$.

Từ các kết quả này, ta có thể dễ dàng tìm nghịch đảo modulo bằng thuật toán lũy thừa nhị phân, hoạt động trong thời gian $O(\log m)$.

Mặc dù phương pháp này dễ hiểu hơn phương pháp được mô tả trong đoạn trước, nhưng trong trường hợp $m$ không phải là số nguyên tố, ta cần tính hàm phi Euler, điều này liên quan đến việc phân tích $m$ thành thừa số nguyên tố, có thể rất khó. Nếu phân tích nguyên tố của $m$ đã biết, độ phức tạp của phương pháp này là $O(\log m)$.

### 5.12.4 Tìm nghịch đảo modulo cho modulo nguyên tố bằng phép chia Euclid

Cho một modulo nguyên tố $m > a$ (hoặc ta có thể áp dụng modulo để làm nó nhỏ hơn trong 1 bước), theo phép chia Euclid:

$$m = k \cdot a + r,$$

trong đó $k = \left\lfloor \frac{m}{a} \right\rfloor$ và $r = m \mod a$, thì:

$$\begin{aligned}
\implies \quad & 0 \equiv k \cdot a + r && \mod m \\
\iff \quad & r \equiv -k \cdot a && \mod m \\
\iff \quad & r \cdot a^{-1} \equiv -k && \mod m \\
\iff \quad & a^{-1} \equiv -k \cdot r^{-1} && \mod m.
\end{aligned}$$

Lưu ý rằng lập luận này không đúng nếu $m$ không phải là số nguyên tố, vì sự tồn tại của $a^{-1}$ không đảm bảo sự tồn tại của $r^{-1}$ trong trường hợp tổng quát. Để thấy điều này, hãy thử tính $5^{-1}$ modulo 12 với công thức trên. Ta muốn đạt được 5, vì $5 \cdot 5 \equiv 1 \mod 12$. Tuy nhiên, $12 = 2 \cdot 5 + 2$, và ta có $k = 2$, $r = 2$, với 2 không khả nghịch modulo 12.

Tuy nhiên, nếu modulo là nguyên tố, tất cả $a$ với $0 < a < m$ đều khả nghịch modulo $m$, và ta có thể dùng hàm đệ quy sau (bằng C++) để tính nghịch đảo modulo cho số $a$ theo $m$:

```
int inv(int a) {
  return a <= 1 ? a : m - (long long)(m/a) * inv(m % a) % m;
}
```

Độ phức tạp chính xác của đệ quy này chưa được biết rõ. Nó nằm trong khoảng từ $O\left(\frac{\log m}{\log \log m}\right)$ đến $O(m^{\frac{1}{3}-\frac{2}{177}+\epsilon})$. Trong thực tế, triển khai này nhanh, ví dụ với modulo $10^9 + 7$, nó luôn hoàn thành trong dưới 50 lần lặp.

Áp dụng công thức này, ta cũng có thể tính trước nghịch đảo modulo cho mọi số trong khoảng $[1, m-1]$ trong $O(m)$:

```
inv[1] = 1;
for(int a = 2; a < m; ++a)
    inv[a] = m - (long long)(m/a) * inv[m%a] % m;
```

### 5.12.5 Tìm nghịch đảo modulo cho mảng số modulo $m$

Giả sử ta có một mảng và muốn tìm nghịch đảo modulo cho tất cả các số trong đó (tất cả đều khả nghịch). Thay vì tính nghịch đảo cho từng số, ta có thể mở rộng phân số bằng tích tiền tố (không bao gồm chính nó) và tích hậu tố (không bao gồm chính nó), và chỉ cần tính một nghịch đảo duy nhất:

$$\begin{aligned} x_i^{-1} = \frac{1}{x_i} &= \frac{\overbrace{x_1 \cdot x_2 \cdots x_{i-1}}^{\text{prefix}_{i-1}} \cdot 1 \cdot \overbrace{x_{i+1} \cdot x_{i+2} \cdots x_n}^{\text{suffix}_{i+1}}}{x_1 \cdot x_2 \cdots x_{i-1} \cdot x_i \cdot x_{i+1} \cdot x_{i+2} \cdots x_n} \\ &= \text{prefix}_{i-1} \cdot \text{suffix}_{i+1} \cdot (x_1 \cdot x_2 \cdots x_n)^{-1}. \end{aligned}$$

Trong mã, ta chỉ cần tạo một mảng tích tiền tố (không bao gồm chính nó, bắt đầu từ phần tử đơn vị), tính nghịch đảo modulo cho tích của tất cả các số, sau đó nhân nó với tích tiền tố và tích hậu tố (không bao gồm chính nó). Tích hậu tố được tính bằng cách lặp từ sau ra trước:

```
std::vector<int> invs(const std::vector<int> &a, int m) {
    int n = a.size();
    if (n == 0) return {};
    std::vector<int> b(n);
    int v = 1;
    for (int i = 0; i != n; ++i) {
        b[i] = v;
        v = static_cast<long long>(v) * a[i] % m;
    }
    int x, y;
    extended_euclidean(v, m, x, y);
    x = (x % m + m) % m;
    for (int i = n - 1; i >= 0; --i) {
        b[i] = static_cast<long long>(x) * b[i] % m;
        x = static_cast<long long>(x) * a[i] % m;
    }
    return b;
}
```

## 5.13 Phương trình đồng dư tuyến tính

Phương trình này có dạng:

$$a \cdot x \equiv b \pmod{n},$$

trong đó $a$, $b$ và $n$ là các số nguyên đã cho, còn $x$ là một số nguyên chưa biết.

Yêu cầu là tìm giá trị $x$ trong khoảng $[0, n-1]$ (rõ ràng, trên toàn bộ trục số có thể có vô số nghiệm khác nhau theo $n \cdot k$, trong đó $k$ là bất kỳ số nguyên nào). Nếu nghiệm không duy nhất, ta sẽ xem xét cách tìm tất cả các nghiệm.

### 5.13.1 Giải bằng cách tìm phần tử nghịch đảo

Trước tiên, hãy xem xét trường hợp đơn giản hơn khi $a$ và $n$ nguyên tố cùng nhau ($\gcd(a, n) = 1$). Khi đó, ta có thể tìm nghịch đảo của $a$, và nhân cả hai vế của phương trình với nghịch đảo, ta thu được một nghiệm duy nhất:

$$x \equiv b \cdot a^{-1} \pmod{n}.$$

Bây giờ, xem xét trường hợp $a$ và $n$ không nguyên tố cùng nhau ($\gcd(a, n) \neq 1$). Khi đó, nghiệm không phải lúc nào cũng tồn tại (ví dụ, $2 \cdot x \equiv 1 \pmod{4}$ không có nghiệm).

Gọi $g = \gcd(a, n)$, tức là ước chung lớn nhất của $a$ và $n$ (trong trường hợp này lớn hơn 1).

Nếu $b$ không chia hết cho $g$, thì không có nghiệm. Thực tế, với bất kỳ $x$ nào, vế trái của phương trình $a \cdot x \pmod{n}$ luôn chia hết cho $g$, trong khi vế phải không chia hết cho $g$, do đó suy ra không có nghiệm.

Nếu $g$ chia hết cho $b$, thì bằng cách chia cả hai vế của phương trình cho $g$ (tức là chia $a$, $b$ và $n$ cho $g$), ta thu được một phương trình mới:

$$a' \cdot x \equiv b' \pmod{n'},$$

trong đó $a'$ và $n'$ giờ đây nguyên tố cùng nhau, và ta đã biết cách xử lý phương trình như vậy. Ta tìm được $x'$ là nghiệm cho $x$.

Rõ ràng, $x'$ này cũng sẽ là một nghiệm của phương trình ban đầu. Tuy nhiên, nó không phải là nghiệm duy nhất. Có thể chứng minh rằng phương trình ban đầu có chính xác $g$ nghiệm, và chúng sẽ có dạng:

$$x_i \equiv (x' + i \cdot n') \pmod{n} \quad \text{với } i = 0 \ldots g-1.$$

Tóm lại, ta có thể nói rằng số lượng nghiệm của phương trình đồng dư tuyến tính bằng $g = \gcd(a, n)$ hoặc bằng 0.

### 5.13.2 Giải bằng thuật toán Euclid mở rộng

Ta có thể viết lại phương trình đồng dư tuyến tính thành phương trình Diophantine sau:

$$a \cdot x + n \cdot k = b,$$

trong đó $x$ và $k$ là các số nguyên chưa biết.

Phương pháp giải phương trình này được mô tả trong bài viết tương ứng về "Phương trình Diophantine tuyến tính" và bao gồm việc áp dụng thuật toán Euclid mở rộng.

Bài viết đó cũng mô tả phương pháp thu được tất cả các nghiệm của phương trình này từ một nghiệm đã tìm thấy, và ngẫu nhiên, phương pháp này, khi được xem xét kỹ lưỡng, hoàn toàn tương đương với phương pháp được mô tả trong phần trước.

## 5.14 Định lý Thặng dư Trung Hoa

Định lý Thặng dư Trung Hoa (sẽ được gọi là CRT trong phần còn lại của bài viết này) được phát hiện bởi nhà toán học Trung Quốc Tôn Tử.

### 5.14.1 Công thức

Cho $m = m_1 \cdot m_2 \cdots m_k$, trong đó $m_i$ đôi một nguyên tố cùng nhau. Ngoài $m_i$, ta còn được cho một hệ phương trình đồng dư:

$$\begin{cases} a & \equiv & a_1 & \pmod{m_1} \\ a & \equiv & a_2 & \pmod{m_2} \\ & \vdots & & \\ a & \equiv & a_k & \pmod{m_k} \end{cases}$$

trong đó $a_i$ là các hằng số đã cho. Dạng nguyên bản của CRT phát biểu rằng hệ phương trình đồng dư đã cho luôn có một và chỉ một nghiệm modulo $m$.

Ví dụ, hệ phương trình đồng dư:

$$\begin{cases} a & \equiv & 2 & \pmod{3} \\ a & \equiv & 3 & \pmod{5} \\ a & \equiv & 2 & \pmod{7} \end{cases}$$

có nghiệm 23 modulo 105, vì $23 \mod 3 = 2$, $23 \mod 5 = 3$, và $23 \mod 7 = 2$. Ta có thể viết mọi nghiệm dưới dạng $23 + 105 \cdot k$ với $k \in \mathbb{Z}$.

### 5.14.2 Hệ quả

Hệ quả của CRT là phương trình:

$$x \equiv a \pmod{m}$$

tương đương với hệ phương trình:

$$\begin{cases} x & \equiv & a_1 & \pmod{m_1} \\ & \vdots & & \\ x & \equiv & a_k & \pmod{m_k} \end{cases}$$

(Như trên, giả sử $m = m_1 m_2 \cdots m_k$ và $m_i$ đôi một nguyên tố cùng nhau).

### 5.14.3 Nghiệm cho hai modulo

Xem xét một hệ gồm hai phương trình với $m_1, m_2$ nguyên tố cùng nhau:

$$\begin{cases} a \equiv a_1 \pmod{m_1} \\ a \equiv a_2 \pmod{m_2} \end{cases}$$

Ta muốn tìm nghiệm cho $a \pmod{m_1 m_2}$. Sử dụng thuật toán Euclid mở rộng, ta có thể tìm các hệ số Bézout $n_1, n_2$ sao cho:

$$n_1 m_1 + n_2 m_2 = 1.$$

Thực tế, $n_1$ và $n_2$ chính là nghịch đảo modulo của $m_1$ và $m_2$ theo $m_2$ và $m_1$. Ta có $n_1 m_1 \equiv 1 \pmod{m_2}$, nên $n_1 \equiv m_1^{-1} \pmod{m_2}$, và ngược lại $n_2 \equiv m_2^{-1} \pmod{m_1}$.

Với hai hệ số này, ta có thể định nghĩa một nghiệm:

$$a = a_1 n_2 m_2 + a_2 n_1 m_1 \mod m_1 m_2.$$

Dễ dàng kiểm tra rằng đây thực sự là một nghiệm bằng cách tính $a \mod m_1$ và $a \mod m_2$:

$$\begin{aligned} a &\equiv a_1 n_2 m_2 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1(1 - n_1 m_1) + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 - a_1 n_1 m_1 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 && \pmod{m_1}. \end{aligned}$$

Lưu ý rằng định lý Thặng dư Trung Hoa cũng đảm bảo rằng chỉ có một nghiệm duy nhất modulo $m_1 m_2$. Điều này cũng dễ chứng minh.

Giả sử ta có hai nghiệm khác nhau $x$ và $y$. Vì $x \equiv a_i \pmod{m_i}$ và $y \equiv a_i \pmod{m_i}$, suy ra $x - y \equiv 0 \pmod{m_i}$, và do đó $x - y \equiv 0 \pmod{m_1 m_2}$, hoặc tương đương $x \equiv y \pmod{m_1 m_2}$. Vậy $x$ và $y$ thực chất là cùng một nghiệm.

### 5.14.4 Nghiệm cho trường hợp tổng quát

#### Nghiệm quy nạp

Vì $m_1m_2$ nguyên tố cùng nhau với $m_3$, ta có thể áp dụng quy nạp lặp lại nghiệm cho hai modulo cho bất kỳ số lượng modulo nào. Đầu tiên, ta tính $b_2 := a \pmod{m_1m_2}$ bằng hai đồng dư đầu tiên, sau đó tính $b_3 := a \pmod{m_1m_2m_3}$ bằng các đồng dư $a \equiv b_2 \pmod{m_1m_2}$ và $a \equiv a_3 \pmod{m_3}$, v.v.

#### Xây dựng trực tiếp

Một cách xây dựng trực tiếp tương tự như nội suy Lagrange cũng khả thi.

Gọi $M_i := \prod_{i \neq j} m_j$, là tích của tất cả các modulo trừ $m_i$, và $N_i$ là nghịch đảo modulo $N_i := M_i^{-1} \mod m_i$. Khi đó, nghiệm của hệ phương trình đồng dư là:

$$a \equiv \sum_{i=1}^{k} a_i M_i N_i \pmod{m_1 m_2 \cdots m_k}.$$

Ta có thể kiểm tra đây thực sự là một nghiệm bằng cách tính $a \mod m_i$ cho mọi $i$. Vì $M_j$ là bội số của $m_i$ với $i \neq j$, ta có:

$$\begin{aligned} a &\equiv \textstyle\sum_{j=1}^{k} a_j M_j N_j && \pmod{m_i} \\ &\equiv a_i M_i N_i && \pmod{m_i} \\ &\equiv a_i M_i M_i^{-1} && \pmod{m_i} \\ &\equiv a_i && \pmod{m_i}. \end{aligned}$$

### 5.14.5 Triển khai

```
struct Congruence {
    long long a, m;
};

long long chinese_remainder_theorem(vector<Congruence> const& congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
```

```
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}
```

### 5.14.6 Nghiệm cho các modulo không nguyên tố cùng nhau

Như đã đề cập, thuật toán trên chỉ hoạt động với các modulo $m_1, m_2, \ldots, m_k$ đôi một nguyên tố cùng nhau.

Trong trường hợp không nguyên tố cùng nhau, một hệ phương trình đồng dư có chính xác một nghiệm modulo $\operatorname{lcm}(m_1, m_2, \ldots, m_k)$, hoặc không có nghiệm nào cả.

Ví dụ, trong hệ sau, đồng dư đầu tiên ngụ ý nghiệm là số lẻ, còn đồng dư thứ hai ngụ ý nghiệm là số chẵn. Không thể có số nào vừa lẻ vừa chẵn, do đó rõ ràng không có nghiệm:

$$\begin{cases} a \equiv 1 \pmod 4 \\ a \equiv 2 \pmod 6 \end{cases}$$

Việc xác định một hệ có nghiệm hay không khá đơn giản. Và nếu có nghiệm, ta có thể dùng thuật toán ban đầu để giải một hệ đồng dư hơi sửa đổi.

Một đồng dư đơn $a \equiv a_i \pmod{m_i}$ tương đương với hệ đồng dư $a \equiv a_i \pmod{p_j^{n_j}}$, trong đó $p_1^{n_1} p_2^{n_2} \cdots p_k^{n_k}$ là phân tích nguyên tố của $m_i$.

Với thực tế này, ta có thể sửa đổi hệ đồng dư thành một hệ chỉ có các modulo là lũy thừa nguyên tố. Ví dụ, hệ đồng dư trên tương đương với:

$$\begin{cases} a \equiv 1 & \pmod 4 \\ a \equiv 2 \equiv 0 & \pmod 2 \\ a \equiv 2 & \pmod 3 \end{cases}$$

Vì ban đầu một số modulo có thừa số chung, ta sẽ nhận được một số đồng dư dựa trên cùng số nguyên tố, nhưng có thể với các lũy thừa nguyên tố khác nhau.

Ta có thể quan sát rằng đồng dư với modulo lũy thừa nguyên tố cao nhất sẽ là đồng dư mạnh nhất trong tất cả các đồng dư dựa trên cùng số nguyên tố. Nó hoặc sẽ mâu thuẫn với một đồng dư khác, hoặc đã bao hàm tất cả các đồng dư khác.

Trong trường hợp của ta, đồng dư đầu tiên $a \equiv 1 \pmod 4$ ngụ ý $a \equiv 1 \pmod 2$, và do đó mâu thuẫn với đồng dư thứ hai $a \equiv 0 \pmod 2$. Vì vậy, hệ đồng dư này không có nghiệm.

Nếu không có mâu thuẫn, thì hệ phương trình có nghiệm. Ta có thể bỏ qua tất cả các đồng dư trừ những đồng dư có modulo lũy thừa nguyên tố cao nhất. Các modulo này giờ đây nguyên tố cùng nhau, và do đó ta có thể giải bằng thuật toán đã thảo luận ở các phần trên.

Ví dụ, hệ sau có nghiệm modulo $\text{lcm}(10, 12) = 60$:

$$\begin{cases} a \equiv 3 \pmod{10} \\ a \equiv 5 \pmod{12} \end{cases}$$

Hệ đồng dư tương đương với:

$$\begin{cases} a \equiv 3 \equiv 1 \pmod{2} \\ a \equiv 3 \equiv 3 \pmod{5} \\ a \equiv 5 \equiv 1 \pmod{4} \\ a \equiv 5 \equiv 2 \pmod{3} \end{cases}$$

Các đồng dư duy nhất có cùng modulo nguyên tố là $a \equiv 1 \pmod{4}$ và $a \equiv 1 \pmod{2}$. Đồng dư đầu tiên đã bao hàm đồng dư thứ hai, nên ta có thể bỏ qua đồng dư thứ hai, và giải hệ sau với các modulo nguyên tố cùng nhau:

$$\begin{cases} a \equiv 3 \equiv 3 \pmod{5} \\ a \equiv 5 \equiv 1 \pmod{4} \\ a \equiv 5 \equiv 2 \pmod{3} \end{cases}$$

Nó có nghiệm $53 \pmod{60}$, và thực tế $53 \mod 10 = 3$ và $53 \mod 12 = 5$.

### 5.14.7 Thuật toán Garner

Một hệ quả khác của CRT là ta có thể biểu diễn các số lớn bằng một mảng các số nguyên nhỏ.

Thay vì thực hiện nhiều phép tính với các số rất lớn, điều này có thể tốn kém (hãy nghĩ đến việc chia các số có 1000 chữ số), bạn có thể chọn một vài modulo nguyên tố cùng nhau và biểu diễn số lớn dưới dạng một hệ đồng dư, rồi thực hiện tất cả các phép toán trên hệ phương trình. Bất kỳ số $a$ nào nhỏ hơn $m_1 m_2 \cdots m_k$ đều có thể được biểu diễn dưới dạng mảng $a_1, \ldots, a_k$, trong đó $a \equiv a_i \pmod{m_i}$.

Bằng cách sử dụng thuật toán trên, bạn có thể tái tạo lại số lớn bất cứ khi nào cần.

Ngoài ra, bạn cũng có thể biểu diễn số trong biểu diễn cơ số hỗn hợp:

$$a = x_1 + x_2 m_1 + x_3 m_1 m_2 + \ldots + x_k m_1 \cdots m_{k-1} \text{ với } x_i \in [0, m_i).$$

Thuật toán Garner, được thảo luận trong bài viết chuyên biệt "Thuật toán Garner", tính các hệ số $x_i$. Và với các hệ số này, bạn có thể khôi phục toàn bộ số.

## 5.15 Thuật toán Garner

Một hệ quả của Định lý Thặng dư Trung Hoa là chúng ta có thể biểu diễn các số lớn bằng một mảng các số nguyên nhỏ. Ví dụ, giả sử $p$ là tích của 1000 số nguyên tố đầu tiên. $p$ có khoảng 3000 chữ số.

Bất kỳ số nào $a$ nhỏ hơn $p$ đều có thể được biểu diễn dưới dạng một mảng $a_1, \ldots, a_k$, trong đó $a_i \equiv a \pmod{p_i}$. Nhưng để làm điều này, rõ ràng chúng ta cần biết cách lấy lại số $a$ từ biểu diễn của nó. Một cách được thảo luận trong bài viết về Định lý Thặng dư Trung Hoa.

Trong bài viết này, chúng ta thảo luận về một phương pháp thay thế, thuật toán Garner, cũng có thể được sử dụng cho mục đích này.

### 5.15.1 Biểu diễn cơ số hỗn hợp

Chúng ta có thể biểu diễn số $a$ trong biểu diễn cơ số hỗn hợp:

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \ldots + x_k p_1 \cdots p_{k-1} \text{ với } x_i \in [0, p_i)$$

Biểu diễn cơ số hỗn hợp là một hệ thống số vị trí, là sự khái quát hóa của các hệ thống số thông thường, như hệ thống số nhị phân hoặc hệ thống số thập phân. Ví dụ, hệ thống số thập phân là một hệ thống số vị trí với cơ số (hay base) là 10. Mọi số được biểu diễn dưới dạng một chuỗi các chữ số $d_1 d_2 d_3 \ldots d_n$ nằm trong khoảng từ 0 đến 9. Chẳng hạn, chuỗi 415 biểu diễn số $4 \cdot 10^2 + 1 \cdot 10^1 + 5 \cdot 10^0$. Nói chung, chuỗi chữ số $d_1 d_2 d_3 \ldots d_n$ biểu diễn số $d_1 b^{n-1} + d_2 b^{n-2} + \cdots + d_n b^0$ trong hệ thống số vị trí với cơ số $b$.

Trong hệ thống cơ số hỗn hợp, chúng ta không còn chỉ có một cơ số duy nhất nữa. Cơ số thay đổi theo từng vị trí.

## 5.16 Thuật toán Garner

Thuật toán Garner tính toán các chữ số $x_1, \ldots, x_k$. Lưu ý rằng các chữ số này tương đối nhỏ. Chữ số $x_i$ là một số nguyên nằm trong khoảng từ 0 đến $p_i - 1$.

Gọi $r_{ij}$ là nghịch đảo của $p_i$ modulo $p_j$:

$$r_{ij} = (p_i)^{-1} \pmod{p_j}$$

Nghịch đảo này có thể được tìm bằng thuật toán mô tả trong phần Nghịch đảo Modulo.

Thay $a$ từ biểu diễn cơ số hỗn hợp vào phương trình đồng dư đầu tiên, ta được:

$$a_1 \equiv x_1 \pmod{p_1}.$$

Thay vào phương trình thứ hai, ta có:

$$a_2 \equiv x_1 + x_2 p_1 \pmod{p_2},$$

phương trình này có thể được viết lại bằng cách trừ $x_1$ và chia cho $p_1$ để được:

$$\begin{aligned} a_2 - x_1 &\equiv x_2 p_1 && \pmod{p_2} \\ (a_2 - x_1) r_{12} &\equiv x_2 && \pmod{p_2} \\ x_2 &\equiv (a_2 - x_1) r_{12} && \pmod{p_2} \end{aligned}$$

Tương tự, ta có:

$$x_3 \equiv ((a_3 - x_1) r_{13} - x_2) r_{23} \pmod{p_3}.$$

Bây giờ, ta có thể thấy rõ một mẫu hình xuất hiện, điều này có thể được biểu diễn bằng đoạn mã sau:

```
for (int i = 0; i < k; ++i) {
    x[i] = a[i];
    for (int j = 0; j < i; ++j) {
        x[i] = r[j][i] * (x[i] - x[j]);
        x[i] = x[i] % p[i];
        if (x[i] < 0)
            x[i] += p[i];
    }
}
```

Vậy là chúng ta đã học cách tính các chữ số $x_i$ trong thời gian $O(k^2)$. Số $a$ giờ đây có thể được tính bằng công thức đã đề cập trước đó:

$$a = x_1 + x_2 \cdot p_1 + x_3 \cdot p_1 \cdot p_2 + \ldots + x_k \cdot p_1 \cdots p_{k-1}$$

Đáng chú ý là trong thực tế, chúng ta hầu như chắc chắn cần tính kết quả $a$ bằng Số học Độ chính xác Tùy ý, nhưng các chữ số $x_i$ (vì chúng nhỏ) thường có thể được tính bằng các kiểu dữ liệu có sẵn, do đó thuật toán Garner rất hiệu quả.

## 5.17 Triển khai thuật toán Garner

Việc triển khai thuật toán này bằng Java rất thuận tiện, vì nó có hỗ trợ sẵn cho các số lớn thông qua lớp `BigInteger`.

Dưới đây, chúng tôi trình bày một triển khai có thể lưu trữ các số lớn dưới dạng tập hợp các phương trình đồng dư. Nó hỗ trợ phép cộng, trừ và nhân. Và với thuật toán Garner, chúng ta có thể chuyển tập hợp các phương trình này thành một số nguyên duy nhất. Trong mã này, chúng ta lấy 100 số nguyên tố lớn hơn $10^9$, cho phép biểu diễn các số lớn tới khoảng $10^{900}$.

```
final int SZ = 100;
int pr[] = new int[SZ];
int r[][] = new int[SZ][SZ];

void init() {
    for (int x = 1000 * 1000 * 1000, i = 0; i < SZ; ++x)
        if (BigInteger.valueOf(x).isProbablePrime(100))
            pr[i++] = x;

    for (int i = 0; i < SZ; ++i)
        for (int j = i + 1; j < SZ; ++j)
            r[i][j] =
                BigInteger.valueOf(pr[i]).modInverse(BigInteger.valueOf(pr[j])).intVal
}

class Number {
    int a[] = new int[SZ];

    public Number() {
    }

    public Number(int n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n % pr[i];
    }

    public Number(BigInteger n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n.mod(BigInteger.valueOf(pr[i])).intValue();
    }

    public Number add(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] + n.a[i]) % pr[i];
        return result;
    }

    public Number subtract(Number n) {
        Number result = new Number();
```

```
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] - n.a[i] + pr[i]) % pr[i];
        return result;
    }

    public Number multiply(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (int)((a[i] * 1l * n.a[i]) % pr[i]);
        return result;
    }

    public BigInteger bigIntegerValue(boolean can_be_negative) {
        BigInteger result = BigInteger.ZERO, mult = BigInteger.ONE;
        int x[] = new int[SZ];
        for (int i = 0; i < SZ; ++i) {
            x[i] = a[i];
            for (int j = 0; j < i; ++j) {
                long cur = (x[i] - x[j]) * 1l * r[j][i];
                x[i] = (int)((cur % pr[i] + pr[i]) % pr[i]);
            }
            result = result.add(mult.multiply(BigInteger.valueOf(x[i])));
            mult = mult.multiply(BigInteger.valueOf(pr[i]));
        }

        if (can_be_negative)
            if (result.compareTo(mult.shiftRight(1)) >= 0)
                result = result.subtract(mult);

        return result;
    }
}
```

## 5.18 Giai thừa modulo $p$

Trong một số trường hợp, cần xem xét các công thức phức tạp modulo một số nguyên tố $p$, trong đó có các giai thừa ở cả tử số và mẫu số, chẳng hạn như trong công thức tính hệ số nhị thức. Chúng ta xem xét trường hợp khi $p$ tương đối nhỏ. Bài toán này chỉ có ý nghĩa khi các giai thừa xuất hiện ở cả tử số và mẫu số của phân số. Nếu không, $p!$ và các số hạng tiếp theo sẽ rút gọn về 0. Nhưng trong các phân số, các thừa số của $p$ có thể

triệt tiêu, và biểu thức kết quả sẽ khác 0 modulo $p$.

Vậy, nhiệm vụ chính thức là: Bạn muốn tính $n!$ mod $p$, mà không tính đến tất cả các thừa số bội của $p$ xuất hiện trong giai thừa. Hãy tưởng tượng bạn viết ra phân tích thừa số nguyên tố của $n!$, loại bỏ tất cả các thừa số $p$, rồi tính tích modulo $p$. Chúng ta sẽ ký hiệu giai thừa đã được điều chỉnh này là $n!_{\%p}$. Ví dụ:

$$7!_{\%p} \equiv 1 \cdot 2 \cdot \underbrace{1}_{3} \cdot 4 \cdot 5 \cdot \underbrace{2}_{6} \cdot 7 \equiv 2 \pmod 3.$$

Việc học cách tính hiệu quả giai thừa đã điều chỉnh này cho phép chúng ta nhanh chóng tính giá trị của các công thức tổ hợp khác nhau (ví dụ, hệ số nhị thức).

### 5.18.1 Thuật toán

Hãy viết rõ giai thừa đã điều chỉnh này:

$$\begin{aligned} n!_{\%p} &= 1 \cdot 2 \cdot 3 \cdot \ldots \cdot (p-2) \cdot (p-1) \cdot \underbrace{1}_{p} \cdot (p+1) \cdot (p+2) \cdot \ldots \cdot (2p-1) \cdot \underbrace{2}_{2p} \\ &\quad \cdot (2p+1) \cdot \ldots \cdot (p^2-1) \cdot \underbrace{1}_{p^2} \cdot (p^2+1) \cdot \ldots \cdot n \pmod p \\ &= 1 \cdot 2 \cdot 3 \cdot \ldots \cdot (p-2) \cdot (p-1) \cdot \underbrace{1}_{p} \cdot 1 \cdot 2 \cdot \ldots \cdot (p-1) \cdot \underbrace{2}_{2p} \cdot 1 \cdot 2 \\ &\quad \cdot \ldots \cdot (p-1) \cdot \underbrace{1}_{p^2} \cdot 1 \cdot 2 \cdot \ldots \cdot (n \bmod p) \pmod p \end{aligned}$$

Có thể thấy rõ rằng giai thừa được chia thành nhiều khối có độ dài giống nhau, trừ khối cuối cùng:

$$\begin{aligned} n!_{\%p} = &\underbrace{1 \cdot 2 \cdot 3 \cdot \ldots \cdot (p-2) \cdot (p-1) \cdot 1}_{\text{khối 1}} \cdot \underbrace{1 \cdot 2 \cdot 3 \cdot \ldots \cdot (p-2) \cdot (p-1) \cdot 2}_{\text{khối 2}} \cdot \ldots \\ &\cdot \underbrace{1 \cdot 2 \cdot 3 \cdot \ldots \cdot (p-2) \cdot (p-1) \cdot 1}_{\text{khối } p} \cdot \ldots \cdot \underbrace{1 \cdot 2 \cdot \ldots \cdot (n \bmod p)}_{\text{phần đuôi}} \pmod p. \end{aligned}$$

Phần chính của các khối dễ tính — đó chỉ là $(p-1)!$ mod $p$. Chúng ta có thể tính điều này bằng lập trình hoặc áp dụng định lý Wilson, định lý này phát biểu rằng $(p-1)!$ mod $p = -1$ với bất kỳ số nguyên tố $p$ nào.

Chúng ta có chính xác $\lfloor \frac{n}{p} \rfloor$ khối như vậy, do đó cần nâng $-1$ lên lũy thừa $\lfloor \frac{n}{p} \rfloor$. Điều này có thể thực hiện trong thời gian logarithmic bằng Lũy thừa Nhị phân; tuy nhiên, bạn cũng có thể nhận thấy kết quả sẽ luân phiên giữa $-1$ và 1, nên chỉ cần xem tính chẵn lẻ của lũy thừa và nhân với $-1$ nếu lũy thừa là số lẻ. Thay vì nhân, chúng ta cũng có thể trừ kết quả hiện tại khỏi $p$.

Giá trị của khối cuối cùng có thể được tính riêng trong $O(p)$.

Điều này chỉ còn lại phần tử cuối của mỗi khối. Nếu ẩn các phần tử đã xử lý, ta thấy mẫu hình sau:

$$n!_{\%p} = \underbrace{\ldots \cdot 1} \cdot \underbrace{\ldots \cdot 2} \cdot \ldots \cdot \underbrace{\ldots \cdot (p-1)} \cdot \underbrace{\ldots \cdot 1} \cdot \underbrace{\ldots \cdot 1} \cdot \underbrace{\ldots \cdot 2} \cdots$$

Đây lại là một giai thừa đã điều chỉnh, nhưng với kích thước nhỏ hơn nhiều. Đó là $\lfloor n/p \rfloor !_{\%p}$.

Do đó, trong quá trình tính giai thừa đã điều chỉnh $n!_{\%p}$, chúng ta thực hiện $O(p)$ phép toán và còn lại việc tính $\lfloor n/p \rfloor !_{\%p}$. Ta có một công thức đệ quy. Độ sâu đệ quy là $O(\log_p n)$, và do đó độ phức tạp tiệm cận tổng thể của thuật toán là $O(p \log_p n)$.

Lưu ý, nếu bạn tính trước các giai thừa $0!$, $1!$, $2!$, $\ldots$, $(p-1)!$ modulo $p$, thì độ phức tạp sẽ chỉ còn $O(\log_p n)$.

### 5.18.2 Triển khai

Chúng ta không cần đệ quy vì đây là trường hợp đệ quy đuôi và do đó có thể dễ dàng triển khai bằng lặp. Trong triển khai sau, chúng ta tính trước các giai thừa $0!$, $1!$, $\ldots$, $(p-1)!$, và do đó có thời gian chạy $O(p + \log_p n)$. Nếu bạn cần gọi hàm nhiều lần, thì có thể thực hiện việc tính trước bên ngoài hàm và tính $n!_{\%p}$ trong thời gian $O(\log_p n)$.

```
int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i-1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n/p) % 2)
            res = p - res;
        res = res * f[n%p] % p;
        n /= p;
    }
    return res;
}
```

Ngoài ra, nếu bạn chỉ có bộ nhớ hạn chế và không thể lưu trữ tất cả các giai thừa, bạn cũng có thể chỉ nhớ các giai thừa cần thiết, sắp xếp chúng, rồi tính chúng trong một lần duyệt bằng cách tính các giai thừa $0!$, $1!$, $2!$, $\ldots$, $(p-1)!$ trong một vòng lặp mà không lưu trữ chúng rõ ràng.

## 5.19 Bội số của $p$

Nếu muốn tính hệ số nhị thức modulo $p$, thì chúng ta còn cần thêm bội số của $p$ trong $n$, tức là số lần $p$ xuất hiện trong phân tích thừa số nguyên tố của $n$, hoặc số lần chúng ta xóa $p$ trong quá trình tính giai thừa đã điều chỉnh.

Công thức Legendre cho chúng ta cách tính điều này trong thời gian $O(\log_p n)$. Công thức cho bội số $\nu_p$ là:

$$\nu_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

Do đó, ta có triển khai:

```
int multiplicity_factorial(int n, int p) {
    int count = 0;
    do {
        n /= p;
        count += n;
    } while (n);
    return count;
}
```

Công thức này có thể được chứng minh dễ dàng bằng cách sử dụng các ý tưởng tương tự như trong các phần trước. Loại bỏ tất cả các phần tử không chứa thừa số $p$. Điều này để lại $\lfloor n/p \rfloor$ phần tử. Nếu ta loại bỏ thừa số $p$ khỏi mỗi phần tử, ta được tích $1 \cdot 2 \cdots \lfloor n/p \rfloor = \lfloor n/p \rfloor!$, và ta lại có một đệ quy.

## 5.20 Logarit rời rạc

Logarit rời rạc là một số nguyên $x$ thỏa mãn phương trình:

$$a^x \equiv b \pmod{m}$$

với các số nguyên cho trước $a$, $b$ và $m$.

Logarit rời rạc không phải lúc nào cũng tồn tại, chẳng hạn không có nghiệm cho $2^x \equiv 3 \pmod{7}$. Không có điều kiện đơn giản nào để xác định xem logarit rời rạc có tồn tại hay không.

Trong bài viết này, chúng ta mô tả thuật toán Baby-step Giant-step, một thuật toán để tính logarit rời rạc do Shanks đề xuất vào năm 1971, với độ phức tạp thời gian $O(\sqrt{m})$. Đây là một thuật toán gặp nhau ở giữa (meet-in-the-middle) vì nó sử dụng kỹ thuật chia đôi nhiệm vụ.

### 5.20.1 Thuật toán

Xét phương trình:

$$a^x \equiv b \pmod{m},$$

trong đó $a$ và $m$ nguyên tố cùng nhau.

Đặt $x = np - q$, trong đó $n$ là một hằng số được chọn trước (chúng ta sẽ mô tả cách chọn $n$ sau). $p$ được gọi là bước lớn (giant step), vì tăng $p$ thêm 1 sẽ tăng $x$ thêm $n$. Tương tự, $q$ được gọi là bước nhỏ (baby step).

Rõ ràng, bất kỳ số $x$ nào trong khoảng $[0; m)$ đều có thể được biểu diễn dưới dạng này, với $p \in [1; \lceil \frac{m}{n} \rceil]$ và $q \in [0; n]$.

Khi đó, phương trình trở thành:

$$a^{np-q} \equiv b \pmod{m}.$$

Sử dụng tính chất $a$ và $m$ nguyên tố cùng nhau, ta được:

$$a^{np} \equiv ba^q \pmod{m}.$$

Phương trình mới này có thể được viết lại dưới dạng đơn giản:

$$f_1(p) = f_2(q).$$

Bài toán này có thể được giải bằng phương pháp gặp nhau ở giữa như sau:

- Tính $f_1$ cho tất cả các đối số $p$ có thể có. Sắp xếp mảng các cặp giá trị-đối số.
- Với tất cả các đối số $q$ có thể có, tính $f_2$ và tìm $p$ tương ứng trong mảng đã sắp xếp bằng tìm kiếm nhị phân.

### 5.20.2 Độ phức tạp

Chúng ta có thể tính $f_1(p)$ trong $O(\log m)$ bằng thuật toán lũy thừa nhị phân. Tương tự cho $f_2(q)$.

Trong bước đầu tiên của thuật toán, chúng ta cần tính $f_1$ cho mọi đối số $p$ có thể có và sau đó sắp xếp các giá trị. Do đó, bước này có độ phức tạp:

$$O\left(\left\lceil \frac{m}{n} \right\rceil \left(\log m + \log \left\lceil \frac{m}{n} \right\rceil\right)\right) = O\left(\left\lceil \frac{m}{n} \right\rceil \log m\right).$$

Trong bước thứ hai, chúng ta cần tính $f_2(q)$ cho mọi đối số $q$ có thể có và thực hiện tìm kiếm nhị phân trên mảng giá trị của $f_1$, do đó bước này có độ phức tạp:

$$O\left(n\left(\log m + \log \frac{m}{n}\right)\right) = O\left(n \log m\right).$$

Khi cộng hai độ phức tạp này, ta được $\log m$ nhân với tổng của $n$ và $m/n$, tổng này nhỏ nhất khi $n = m/n$, nghĩa là để đạt hiệu suất tối ưu, $n$ nên được chọn sao cho:

$$n = \sqrt{m}.$$

Khi đó, độ phức tạp của thuật toán trở thành:

$$O(\sqrt{m} \log m).$$

### 5.20.3 Triển khai

#### Triển khai đơn giản nhất

Trong đoạn mã sau, hàm `powmod` tính $a^b \pmod m$ và hàm `solve` đưa ra một nghiệm phù hợp cho bài toán. Nó trả về $-1$ nếu không có nghiệm và trả về một trong các nghiệm có thể nếu tồn tại.

```
int powmod(int a, int b, int m) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = (res * 1ll * a) % m;
        }
        a = (a * 1ll * a) % m;
        b >>= 1;
    }
    return res;
}

int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;
    map<int, int> vals;
    for (int p = 1; p <= n; ++p)
        vals[powmod(a, p * n, m)] = p;
    for (int q = 0; q <= n; ++q) {
        int cur = (powmod(a, q, m) * 1ll * b) % m;
        if (vals.count(cur)) {
            int ans = vals[cur] * n - q;
            return ans;
        }
    }
```

```
    return -1;
}
```

Trong mã này, chúng ta sử dụng `map` từ thư viện chuẩn C++ để lưu các giá trị của $f_1$. Bên trong, `map` sử dụng cây đỏ-đen để lưu trữ giá trị. Do đó, mã này hơi chậm hơn so với việc dùng mảng và tìm kiếm nhị phân, nhưng dễ viết hơn nhiều.

Lưu ý rằng mã của chúng ta giả định $0^0 = 1$, tức là mã sẽ tính 0 là nghiệm cho phương trình $0^x \equiv 1 \pmod{m}$ và cũng cho $0^x \equiv 0 \pmod{1}$. Đây là quy ước thường dùng trong đại số, nhưng không được chấp nhận phổ biến trong mọi lĩnh vực. Đôi khi $0^0$ chỉ đơn giản là không xác định. Nếu bạn không thích quy ước này, cần xử lý trường hợp $a = 0$ riêng:

```
if (a == 0)
    return b == 0 ? 1 : -1;
```

Một điều cần lưu ý nữa là nếu có nhiều đối số $p$ ánh xạ đến cùng giá trị $f_1$, chúng ta chỉ lưu một đối số như vậy. Điều này hoạt động trong trường hợp này vì chúng ta chỉ muốn trả về một nghiệm có thể. Nếu cần trả về tất cả các nghiệm, cần thay `map<int, int>` thành, ví dụ, `map<int, vector<int>>`. Chúng ta cũng cần điều chỉnh bước thứ hai tương ứng.

### Triển khai cải tiến

Một cải tiến có thể là loại bỏ lũy thừa nhị phân. Điều này có thể thực hiện bằng cách giữ một biến được nhân với $a$ mỗi khi tăng $q$ và một biến được nhân với $a^n$ mỗi khi tăng $p$. Với thay đổi này, độ phức tạp của thuật toán vẫn giữ nguyên, nhưng giờ đây yếu tố log chỉ dành cho `map`. Thay vì `map`, chúng ta cũng có thể dùng bảng băm (`unordered_map` trong C++) với độ phức tạp thời gian trung bình $O(1)$ cho việc chèn và tìm kiếm.

Các bài toán thường yêu cầu $x$ nhỏ nhất thỏa mãn nghiệm. Có thể lấy tất cả các nghiệm và chọn giá trị nhỏ nhất, hoặc giảm nghiệm đầu tiên tìm được bằng định lý Euler, nhưng ta có thể thông minh trong cách tính giá trị để đảm bảo nghiệm đầu tiên tìm được là nhỏ nhất.

```
// Trả về x nhỏ nhất sao cho a ^ x % m = b % m, a và m nguyên tố cùng nhau.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
```

```
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}
```

Độ phức tạp là $O(\sqrt{m})$ khi sử dụng `unordered_map`.

### 5.20.4 Khi $a$ và $m$ không nguyên tố cùng nhau

Gọi $g = \gcd(a, m)$, và $g > 1$. Rõ ràng $a^x \bmod m$ với mọi $x \geq 1$ sẽ chia hết cho $g$.

Nếu $g \nmid b$, không có nghiệm cho $x$.

Nếu $g \mid b$, đặt $a = g\alpha$, $b = g\beta$, $m = g\nu$:

$$\begin{aligned} a^x &\equiv b \pmod{m} \\ (g\alpha)a^{x-1} &\equiv g\beta \pmod{g\nu} \\ \alpha a^{x-1} &\equiv \beta \pmod{\nu} \end{aligned}$$

Thuật toán Baby-step Giant-step có thể dễ dàng mở rộng để giải $ka^x \equiv b \pmod{m}$ cho $x$.

```
// Trả về x nhỏ nhất sao cho a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
```

```
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}
```

Độ phức tạp thời gian vẫn là $O(\sqrt{m})$ như trước, vì việc giảm ban đầu xuống $a$ và $m$ nguyên tố cùng nhau được thực hiện trong $O(\log^2 m)$.

## 5.21 Căn nguyên thủy

### 5.21.1 Định nghĩa

Trong số học modulo, một số $g$ được gọi là căn nguyên thủy modulo $n$ nếu mọi số nguyên tố cùng nhau với $n$ đều đồng dư với một lũy thừa của $g$ modulo $n$. Về mặt toán học, $g$ là một căn nguyên thủy modulo $n$ nếu và chỉ nếu với bất kỳ số nguyên $a$ nào sao cho $\gcd(a, n) = 1$, tồn tại một số nguyên $k$ thỏa mãn:

$$g^k \equiv a \pmod{n}.$$

$k$ khi đó được gọi là chỉ số hoặc logarit rời rạc của $a$ với cơ số $g$ modulo $n$. $g$ cũng được gọi là phần tử sinh của nhóm nhân các số nguyên modulo $n$.

Đặc biệt, trong trường hợp $n$ là một số nguyên tố, các lũy thừa của căn nguyên thủy chạy qua tất cả các số từ 1 đến $n - 1$.

### 5.21.2 Tính tồn tại

Căn nguyên thủy modulo $n$ tồn tại nếu và chỉ nếu:

- $n$ là 1, 2, 4, hoặc
- $n$ là lũy thừa của một số nguyên tố lẻ ($n = p^k$), hoặc
- $n$ là gấp đôi lũy thừa của một số nguyên tố lẻ ($n = 2 \cdot p^k$).

Định lý này được Gauss chứng minh vào năm 1801.

### 5.21.3 Mối quan hệ với hàm Euler

Gọi $g$ là một căn nguyên thủy modulo $n$. Khi đó, ta có thể chứng minh rằng số nhỏ nhất $k$ sao cho $g^k \equiv 1 \pmod{n}$ bằng $\phi(n)$. Hơn nữa, điều ngược lại cũng đúng, và thực tế này sẽ được sử dụng trong bài viết này để tìm căn nguyên thủy.

Ngoài ra, số lượng căn nguyên thủy modulo $n$, nếu có, bằng $\phi(\phi(n))$.

### 5.21.4 Thuật toán tìm căn nguyên thủy

Một thuật toán ngây thơ là xem xét tất cả các số trong khoảng $[1, n-1]$. Sau đó, kiểm tra xem mỗi số có phải là căn nguyên thủy hay không bằng cách tính tất cả các lũy thừa của nó để xem chúng có khác nhau hết không. Thuật toán này có độ phức tạp $O(g \cdot n)$, quá chậm. Trong phần này, chúng ta đề xuất một thuật toán nhanh hơn bằng cách sử dụng một số định lý nổi tiếng.

Từ phần trước, ta biết rằng nếu số nhỏ nhất $k$ sao cho $g^k \equiv 1 \pmod{n}$ là $\phi(n)$, thì $g$ là một căn nguyên thủy. Vì với bất kỳ số $a$ nào nguyên tố cùng nhau với $n$, ta biết từ định lý Euler rằng $a^{\phi(n)} \equiv 1 \pmod{n}$, nên để kiểm tra xem $g$ có phải là căn nguyên thủy hay không, chỉ cần kiểm tra rằng với mọi $d$ nhỏ hơn $\phi(n)$, $g^d \not\equiv 1 \pmod{n}$. Tuy nhiên, thuật toán này vẫn quá chậm.

Từ định lý Lagrange, ta biết rằng chỉ số của 1 của bất kỳ số nào modulo $n$ phải là ước của $\phi(n)$. Do đó, chỉ cần kiểm tra với tất cả các ước thực sự $d \mid \phi(n)$ rằng $g^d \not\equiv 1 \pmod{n}$. Đây đã là một thuật toán nhanh hơn nhiều, nhưng ta vẫn có thể làm tốt hơn.

Phân tích $\phi(n) = p_1^{a_1} \cdots p_s^{a_s}$. Ta chứng minh rằng trong thuật toán trước, chỉ cần xem xét các giá trị $d$ có dạng $\frac{\phi(n)}{p_j}$. Thật vậy, giả sử $d$ là một ước thực sự bất kỳ của $\phi(n)$. Rõ ràng, tồn tại $j$ sao cho $d \mid \frac{\phi(n)}{p_j}$, tức là $d \cdot k = \frac{\phi(n)}{p_j}$. Tuy nhiên, nếu $g^d \equiv 1 \pmod{n}$, ta sẽ có:

$$g^{\frac{\phi(n)}{p_j}} \equiv g^{d \cdot k} \equiv (g^d)^k \equiv 1^k \equiv 1 \pmod{n},$$

tức là trong số các số dạng $\frac{\phi(n)}{p_i}$, sẽ có ít nhất một số không thỏa mãn điều kiện.

Bây giờ ta có một thuật toán hoàn chỉnh để tìm căn nguyên thủy:

- Đầu tiên, tính $\phi(n)$ và phân tích nó ra thừa số.
- Sau đó, duyệt qua tất cả các số $g \in [1, n]$, và với mỗi số, để kiểm tra xem nó có phải là căn nguyên thủy hay không, ta làm như sau:
  - Tính tất cả $g^{\frac{\phi(n)}{p_i}} \pmod n$.
  - Nếu tất cả các giá trị tính được đều khác 1, thì $g$ là một căn nguyên thủy.

Thời gian chạy của thuật toán này là $O(Ans \cdot \log \phi(n) \cdot \log n)$ (giả sử $\phi(n)$ có $\log \phi(n)$ ước).

Shoup (1990, 1992) đã chứng minh, với giả thuyết Riemann tổng quát, rằng $g$ là $O(\log^6 p)$.

### 5.21.5 Triển khai

Đoạn mã sau giả định rằng modulo $p$ là một số nguyên tố. Để áp dụng cho bất kỳ giá trị nào của $p$, ta phải thêm tính toán $\phi(p)$.

```
int powmod(int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int(res * 1ll * a % p), --b;
        else
            a = int(a * 1ll * a % p), b >>= 1;
    return res;
}

int generator(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back(n);

    for (int res = 2; res <= p; ++res) {
```

```
        bool ok = true;
        for (size_t i = 0; i < fact.size() && ok; ++i)
            ok &= powmod(res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}
```

## 5.22 Căn rời rạc

Bài toán tìm căn rời rạc được định nghĩa như sau. Cho một số nguyên tố $n$ và hai số nguyên $a$ và $k$, tìm tất cả các $x$ sao cho:

$$x^k \equiv a \pmod{n}.$$

### 5.22.1 Thuật toán

Chúng ta sẽ giải bài toán này bằng cách quy về bài toán logarit rời rạc.

Hãy áp dụng khái niệm căn nguyên thủy modulo $n$. Gọi $g$ là một căn nguyên thủy modulo $n$. Lưu ý rằng vì $n$ là số nguyên tố, nó chắc chắn tồn tại và có thể được tìm trong thời gian $O(Ans \cdot \log \phi(n) \cdot \log n) = O(Ans \cdot \log^2 n)$ cộng với thời gian phân tích $\phi(n)$.

Ta có thể dễ dàng loại bỏ trường hợp $a = 0$. Trong trường hợp này, rõ ràng chỉ có một đáp án: $x = 0$.

Vì ta biết $n$ là số nguyên tố và bất kỳ số nào từ 1 đến $n-1$ đều có thể được biểu diễn dưới dạng lũy thừa của căn nguyên thủy, ta có thể biểu diễn bài toán căn rời rạc như sau:

$$(g^y)^k \equiv a \pmod{n},$$

trong đó:

$$x \equiv g^y \pmod{n}.$$

Điều này có thể được viết lại thành:

$$(g^k)^y \equiv a \pmod{n}.$$

Bây giờ ta có một ẩn số $y$, đây là một bài toán logarit rời rạc. Nghiệm có thể được tìm bằng thuật toán Baby-step Giant-step của Shanks trong $O(\sqrt{n} \log n)$ (hoặc ta có thể xác minh rằng không có nghiệm).

Khi đã tìm được một nghiệm $y_0$, một trong những nghiệm của bài toán căn rời rạc sẽ là:

$$x_0 = g^{y_0} \pmod{n}.$$

### 5.22.2 Tìm tất cả các nghiệm từ một nghiệm đã biết

Để giải bài toán này một cách đầy đủ, ta cần tìm tất cả các nghiệm khi biết một nghiệm: $x_0 = g^{y_0} \pmod n$.

Hãy nhớ lại rằng căn nguyên thủy luôn có bậc $\phi(n)$, tức là lũy thừa nhỏ nhất của $g$ cho ra 1 là $\phi(n)$. Do đó, nếu ta thêm số hạng $\phi(n)$ vào số mũ, ta vẫn nhận được cùng giá trị:

$$x^k \equiv g^{y_0 \cdot k + l \cdot \phi(n)} \equiv a \pmod n \quad \forall l \in \mathbb{Z}.$$

Vậy, tất cả các nghiệm có dạng:

$$x = g^{y_0 + \frac{l \cdot \phi(n)}{k}} \pmod n \quad \forall l \in \mathbb{Z},$$

trong đó $l$ được chọn sao cho phân số phải là một số nguyên. Để điều này đúng, tử số phải chia hết cho bội số chung nhỏ nhất của $\phi(n)$ và $k$. Nhớ rằng bội số chung nhỏ nhất của hai số $lcm(a, b) = \frac{a \cdot b}{\gcd(a,b)}$; ta sẽ được:

$$x = g^{y_0 + i \frac{\phi(n)}{\gcd(k, \phi(n))}} \pmod n \quad \forall i \in \mathbb{Z}.$$

Đây là công thức cuối cùng cho tất cả các nghiệm của bài toán căn rời rạc.

### 5.22.3 Triển khai

Dưới đây là triển khai đầy đủ, bao gồm các thủ tục tìm căn nguyên thủy, logarit rời rạc và tìm, in ra tất cả các nghiệm.

```
int gcd(int a, int b) {
    return a ? gcd(b % a, a) : b;
}

int powmod(int a, int b, int p) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = res * a % p;
        }
        a = a * a % p;
        b >>= 1;
    }
    return res;
}
```

```
// Tìm căn nguyên thủy modulo p
int generator(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    }
    if (n > 1)
        fact.push_back(n);

    for (int res = 2; res <= p; ++res) {
        bool ok = true;
        for (int factor : fact) {
            if (powmod(res, phi / factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
}

// Chương trình tìm tất cả các số x sao cho x^k = a (mod n)
int main() {
    int n, k, a;
    scanf("%d %d %d", &n, &k, &a);
    if (a == 0) {
        puts("1\n0");
        return 0;
    }

    int g = generator(n);

    // Thuật toán Baby-step Giant-step cho logarit rời rạc
    int sq = (int) sqrt(n + .0) + 1;
```

```
    vector<pair<int, int>> dec(sq);
    for (int i = 1; i <= sq; ++i)
        dec[i-1] = {powmod(g, i * sq * k % (n - 1), n), i};
    sort(dec.begin(), dec.end());
    int any_ans = -1;
    for (int i = 0; i < sq; ++i) {
        int my = powmod(g, i * k % (n - 1), n) * a % n;
        auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
        if (it != dec.end() && it->first == my) {
            any_ans = it->second * sq - i;
            break;
        }
    }
    if (any_ans == -1) {
        puts("0");
        return 0;
    }

    // In tất cả các đáp án có thể
    int delta = (n - 1) / gcd(k, n - 1);
    vector<int> ans;
    for (int cur = any_ans % delta; cur < n - 1; cur += delta)
        ans.push_back(powmod(g, cur, n));
    sort(ans.begin(), ans.end());
    printf("%d\n", ans.size());
    for (int answer : ans)
        printf("%d ", answer);
}
```

## 5.23 Phép nhân Montgomery

Nhiều thuật toán trong lý thuyết số, như kiểm tra số nguyên tố hoặc phân tích thừa số nguyên, và trong mật mã học, như RSA, đòi hỏi nhiều phép toán modulo một số lớn. Một phép nhân như $xy \bmod n$ khá chậm khi tính toán bằng các thuật toán thông thường, vì nó yêu cầu một phép chia để biết bao nhiêu lần $n$ cần được trừ khỏi tích. Và phép chia là một thao tác rất tốn kém, đặc biệt với các số lớn.

Phép nhân Montgomery (modular) là một phương pháp cho phép tính các phép nhân như vậy nhanh hơn. Thay vì chia tích và trừ $n$ nhiều lần, nó cộng các bội số của $n$ để triệt tiêu các bit thấp và sau đó chỉ bỏ qua các bit thấp.

### 5.23.1 Biểu diễn Montgomery

Tuy nhiên, phép nhân Montgomery không miễn phí. Thuật toán chỉ hoạt động trong không gian Montgomery. Và ta cần chuyển đổi các số của mình vào không gian đó trước khi có thể bắt đầu nhân.

Cho không gian này, ta cần một số nguyên dương $r \geq n$ nguyên tố cùng nhau với $n$, tức là $\gcd(n, r) = 1$. Trong thực tế, ta luôn chọn $r$ là $2^m$ với $m$ là một số nguyên dương, vì các phép nhân, chia và modulo $r$ sau đó có thể được thực hiện hiệu quả bằng cách sử dụng dịch bit và các thao tác bit khác. $n$ sẽ là một số lẻ trong hầu hết các ứng dụng, vì không khó để phân tích một số chẵn. Vậy nên mọi lũy thừa của 2 sẽ nguyên tố cùng nhau với $n$.

Đại diện $\bar{x}$ của một số $x$ trong không gian Montgomery được định nghĩa là:

$$\bar{x} := x \cdot r \bmod n.$$

Lưu ý, việc chuyển đổi thực chất là một phép nhân mà ta muốn tối ưu hóa. Vậy nên đây vẫn là một thao tác tốn kém. Tuy nhiên, bạn chỉ cần chuyển đổi một số vào không gian một lần. Khi đã ở trong không gian Montgomery, bạn có thể thực hiện bao nhiêu thao tác tùy ý một cách hiệu quả. Và cuối cùng, bạn chuyển đổi kết quả cuối cùng trở lại. Vì vậy, miễn là bạn thực hiện nhiều thao tác modulo $n$, điều này sẽ không thành vấn đề.

Trong không gian Montgomery, bạn vẫn có thể thực hiện hầu hết các thao tác như bình thường. Bạn có thể cộng hai phần tử ($x \cdot r + y \cdot r \equiv (x + y) \cdot r \bmod n$), trừ, kiểm tra bằng nhau, và thậm chí tính ước chung lớn nhất của một số với $n$ (vì $\gcd(n, r) = 1$). Tất cả đều dùng các thuật toán thông thường.

Tuy nhiên, điều này không đúng với phép nhân.

Ta kỳ vọng kết quả là:

$$\bar{x} * \bar{y} = \overline{x \cdot y} = (x \cdot y) \cdot r \bmod n.$$

Nhưng phép nhân thông thường sẽ cho ta:

$$\bar{x} \cdot \bar{y} = (x \cdot y) \cdot r \cdot r \bmod n.$$

Do đó, phép nhân trong không gian Montgomery được định nghĩa là:

$$\bar{x} * \bar{y} := \bar{x} \cdot \bar{y} \cdot r^{-1} \bmod n.$$

### 5.23.2 Giảm Montgomery

Phép nhân của hai số trong không gian Montgomery yêu cầu tính toán hiệu quả $x \cdot r^{-1} \bmod n$. Thao tác này được gọi là giảm Montgomery, và còn được biết đến với tên thuật toán REDC.

Vì $\gcd(n, r) = 1$, ta biết rằng tồn tại hai số $r^{-1}$ và $n'$ với $0 < r^{-1}, n' < n$ sao cho:

$$r \cdot r^{-1} + n \cdot n' = 1.$$

Cả $r^{-1}$ và $n'$ đều có thể được tính bằng thuật toán Euclid mở rộng.

Sử dụng đẳng thức này, ta có thể viết $x \cdot r^{-1}$ như sau:

$$\begin{aligned} x \cdot r^{-1} &= x \cdot r \cdot r^{-1}/r = x \cdot (-n \cdot n' + 1)/r \\ &= (-x \cdot n \cdot n' + x)/r \equiv (-x \cdot n \cdot n' + l \cdot r \cdot n + x)/r \bmod n \\ &\equiv ((-x \cdot n' + l \cdot r) \cdot n + x)/r \bmod n \end{aligned}$$

Các đồng dư này đúng với bất kỳ số nguyên $l$ nào. Điều này có nghĩa là ta có thể cộng hoặc trừ một bội số bất kỳ của $r$ vào $x \cdot n'$, hoặc nói cách khác, ta có thể tính $q := x \cdot n'$ modulo $r$.

Điều này cho ta thuật toán sau để tính $x \cdot r^{-1} \bmod n$:

```
function reduce(x):
    q = (x mod r) * n' mod r
    a = (x - q * n) / r
    if a < 0:
        a += n
    return a
```

Vì $x < n \cdot n < r \cdot n$ (ngay cả khi $x$ là tích của một phép nhân) và $q \cdot n < r \cdot n$, ta biết rằng $-n < (x - q \cdot n)/r < n$. Do đó, thao tác modulo cuối cùng được thực hiện bằng một lần kiểm tra và một phép cộng.

Như ta thấy, ta có thể thực hiện giảm Montgomery mà không cần bất kỳ phép toán modulo nặng nào. Nếu ta chọn $r$ là một lũy thừa của 2, các phép toán modulo và chia trong thuật toán có thể được tính bằng cách che bit và dịch chuyển.

Một ứng dụng thứ hai của giảm Montgomery là chuyển một số từ không gian Montgomery trở lại không gian thông thường.

### 5.23.3 Mẹo tính nghịch đảo nhanh

Để tính nghịch đảo $n' := n^{-1} \bmod r$ một cách hiệu quả, ta có thể dùng mẹo sau (lấy cảm hứng từ phương pháp Newton):

$$a \cdot x \equiv 1 \bmod 2^k \Longrightarrow a \cdot x \cdot (2 - a \cdot x) \equiv 1 \bmod 2^{2k}.$$

Điều này dễ dàng được chứng minh. Nếu ta có $a \cdot x = 1 + m \cdot 2^k$, thì ta có:

$$\begin{aligned} a \cdot x \cdot (2 - a \cdot x) &= 2 \cdot a \cdot x - (a \cdot x)^2 \\ &= 2 \cdot (1 + m \cdot 2^k) - (1 + m \cdot 2^k)^2 \\ &= 2 + 2 \cdot m \cdot 2^k - 1 - 2 \cdot m \cdot 2^k - m^2 \cdot 2^{2k} \\ &= 1 - m^2 \cdot 2^{2k} \\ &\equiv 1 \bmod 2^{2k}. \end{aligned}$$

Điều này có nghĩa là ta có thể bắt đầu với $x = 1$ là nghịch đảo của $a$ modulo $2^1$, áp dụng mẹo này vài lần và trong mỗi lần lặp, ta tăng gấp đôi số bit chính xác của $x$.

### 5.23.4 Triển khai

Sử dụng trình biên dịch GCC, ta vẫn có thể tính $x \cdot y \bmod n$ hiệu quả khi cả ba số đều là số nguyên 64 bit, vì trình biên dịch hỗ trợ số nguyên 128 bit với các kiểu `__int128` và `__uint128`.

```
long long result = (__int128)x * y % n;
```

Tuy nhiên, không có kiểu nào cho số nguyên 256 bit. Do đó, ta sẽ trình bày một triển khai cho phép nhân 128 bit.

```
using u64 = uint64_t;
using u128 = __uint128_t;
using i128 = __int128_t;

struct u256 {
    u128 high, low;

    static u256 mult(u128 x, u128 y) {
        u64 a = x >> 64, b = x;
        u64 c = y >> 64, d = y;
        // (a*2^64 + b) * (c*2^64 + d) =
        // (a*c) * 2^128 + (a*d + b*c)*2^64 + (b*d)
        u128 ac = (u128)a * c;
        u128 ad = (u128)a * d;
        u128 bc = (u128)b * c;
        u128 bd = (u128)b * d;
        u128 carry = (u128)(u64)ad + (u128)(u64)bc + (bd >> 64u);
        u128 high = ac + (ad >> 64u) + (bc >> 64u) + (carry >> 64u);
        u128 low = (ad << 64u) + (bc << 64u) + bd;
```

```
        return {high, low};
    }
};

struct Montgomery {
    Montgomery(u128 n) : mod(n), inv(1) {
        for (int i = 0; i < 7; i++)
            inv *= 2 - n * inv;
    }

    u128 init(u128 x) {
        x %= mod;
        for (int i = 0; i < 128; i++) {
            x <<= 1;
            if (x >= mod)
                x -= mod;
        }
        return x;
    }

    u128 reduce(u256 x) {
        u128 q = x.low * inv;
        i128 a = x.high - u256::mult(q, mod).high;
        if (a < 0)
            a += mod;
        return a;
    }

    u128 mult(u128 a, u128 b) {
        return reduce(u256::mult(a, b));
    }

    u128 mod, inv;
};
```

### 5.23.5 Chuyển đổi nhanh

Phương pháp hiện tại để chuyển đổi một số vào không gian Montgomery khá chậm. Có những cách nhanh hơn.

Bạn có thể nhận thấy mối quan hệ sau:

$$\bar{x} := x \cdot r \bmod n = x \cdot r^2/r = x * r^2.$$

Việc chuyển đổi một số vào không gian chỉ là một phép nhân trong không gian của số đó với $r^2$. Do đó, ta có thể tính trước $r^2 \bmod n$ và chỉ thực hiện một phép nhân thay vì dịch số 128 lần.

Trong đoạn mã sau, ta khởi tạo `r2` bằng $-n\%n$, tương đương với $r - n \equiv r \bmod n$, dịch nó 4 lần để được $r \cdot 2^4 \bmod n$. Số này có thể được hiểu là $2^4$ trong không gian Montgomery. Nếu ta bình phương nó 5 lần, ta được $(2^4)^{2^5} = (2^4)^{32} = 2^{128} = r$ trong không gian Montgomery, chính là $r^2 \bmod n$.

```
struct Montgomery {
    Montgomery(u128 n) : mod(n), inv(1), r2(-n % n) {
        for (int i = 0; i < 7; i++)
            inv *= 2 - n * inv;

        for (int i = 0; i < 4; i++) {
            r2 <<= 1;
            if (r2 >= mod)
                r2 -= mod;
        }
        for (int i = 0; i < 5; i++)
            r2 = mul(r2, r2);
    }

    u128 init(u128 x) {
        return mult(x, r2);
    }

    u128 mod, inv, r2;
};
```

## 5.24 Hệ tam phân cân bằng

"Máy tính Setun sử dụng hệ tam phân cân bằng"

Đây là một hệ thống số vị trí không chuẩn nhưng vẫn mang tính chất vị trí. Đặc điểm của nó là các chữ số có thể mang một trong ba giá trị: $-1$, 0 và 1. Tuy nhiên, cơ số của nó vẫn là 3 (vì có ba giá trị có thể). Vì không tiện viết $-1$ dưới dạng chữ số, ta sẽ dùng chữ cái $Z$ để biểu thị giá trị này từ đây trở đi. Nếu bạn nghĩ đây là một hệ thống khá kỳ lạ - hãy nhìn vào hình ảnh - đây là một trong những máy tính sử dụng nó.

Dưới đây là một vài số đầu tiên được viết trong hệ tam phân cân bằng:

- 0 0
- 1 1
- 2 $1Z$
- 3 10
- 4 11
- 5 $1ZZ$
- 6 $1Z0$
- 7 $1Z1$
- 8 $10Z$
- 9 100

Hệ thống này cho phép bạn viết các giá trị âm mà không cần dấu trừ phía trước: bạn chỉ cần đảo ngược các chữ số trong bất kỳ số dương nào.

- $-1$ $Z$
- $-2$ $Z1$
- $-3$ $Z0$
- $-4$ $ZZ$
- $-5$ $Z11$

Lưu ý rằng số âm bắt đầu bằng $Z$ và số dương bắt đầu bằng 1.

### 5.24.1 Thuật toán chuyển đổi

Việc biểu diễn một số cho trước trong hệ tam phân cân bằng khá dễ dàng thông qua việc tạm thời biểu diễn nó trong hệ tam phân thông thường. Khi giá trị ở dạng tam phân chuẩn, các chữ số của nó là 0, 1 hoặc 2. Duyệt từ chữ số thấp nhất, ta có thể bỏ qua các số 0 và 1 một cách an toàn, tuy nhiên, 2 cần được chuyển thành $Z$ đồng thời cộng 1 vào chữ số bên trái nó. Các chữ số 3 cần được chuyển thành 0 với điều kiện tương tự - các chữ số này không có trong số ban đầu nhưng có thể xuất hiện sau khi tăng một số chữ số 2.

**Ví dụ 1:** Hãy chuyển 64 sang hệ tam phân cân bằng. Đầu tiên, ta dùng hệ tam phân thông thường để viết lại số:

$$64_{10} = 02101_3.$$

Hãy xử lý từ chữ số thấp nhất (bên phải nhất):

- 1, 0 và 1 được giữ nguyên vì chúng được phép trong hệ tam phân cân bằng.
- 2 được chuyển thành $Z$, tăng chữ số bên trái nó, nên ta được $1Z101$.

Kết quả cuối cùng là $1Z101$.
Hãy chuyển ngược lại hệ thập phân bằng cách cộng các giá trị vị trí có trọng số:

$$1Z101 = 81 \cdot 1 + 27 \cdot (-1) + 9 \cdot 1 + 3 \cdot 0 + 1 \cdot 1 = 64_{10}.$$

**Ví dụ 2:** Hãy chuyển 237 sang hệ tam phân cân bằng. Đầu tiên, ta dùng hệ tam phân thông thường để viết lại số:

$$237_{10} = 22210_3.$$

Hãy xử lý từ chữ số thấp nhất (bên phải nhất):

- 0 và 1 được giữ nguyên vì chúng được phép trong hệ tam phân cân bằng.
- 2 được chuyển thành $Z$, tăng chữ số bên trái nó, nên ta được $23Z10$.
- 3 được chuyển thành 0, tăng chữ số bên trái nó, nên ta được $30Z10$.
- 3 được chuyển thành 0, tăng chữ số bên trái nó (mặc định là 0), nên ta được $100Z10$.

Kết quả cuối cùng là $100Z10$.
Hãy chuyển ngược lại hệ thập phân bằng cách cộng các giá trị vị trí có trọng số:

$$100Z10 = 243 \cdot 1 + 81 \cdot 0 + 27 \cdot 0 + 9 \cdot (-1) + 3 \cdot 1 + 1 \cdot 0 = 237_{10}.$$

## 5.25 Mã Gray

Mã Gray là một hệ thống số nhị phân trong đó hai giá trị liên tiếp chỉ khác nhau ở một bit.

Ví dụ, dãy mã Gray cho các số 3 bit là: 000, 001, 011, 010, 110, 111, 101, 100, vậy nên $G(4) = 6$.

Mã này được Frank Gray phát minh vào năm 1953.

### 5.25.1 Tìm mã Gray

Hãy xem xét các bit của số $n$ và các bit của số $G(n)$. Chú ý rằng bit thứ $i$ của $G(n)$ bằng 1 chỉ khi bit thứ $i$ của $n$ bằng 1 và bit thứ $i+1$ bằng 0, hoặc ngược lại (bit thứ $i$ bằng 0 và bit thứ $i+1$ bằng 1). Do đó, $G(n) = n \oplus (n >> 1)$:

```
int g(int n) {
    return n ^ (n >> 1);
}
```

### 5.25.2 Tìm mã Gray ngược

Cho mã Gray $g$, khôi phục lại số ban đầu $n$.

Ta sẽ di chuyển từ các bit quan trọng nhất đến các bit ít quan trọng nhất (bit ít quan trọng nhất có chỉ số 1 và bit quan trọng nhất có chỉ số $k$). Quan hệ giữa các bit $n_i$ của số $n$ và các bit $g_i$ của số $g$:

$$\begin{aligned}
n_k &= g_k, \\
n_{k-1} &= g_{k-1} \oplus n_k = g_k \oplus g_{k-1}, \\
n_{k-2} &= g_{k-2} \oplus n_{k-1} = g_k \oplus g_{k-1} \oplus g_{k-2}, \\
n_{k-3} &= g_{k-3} \oplus n_{k-2} = g_k \oplus g_{k-1} \oplus g_{k-2} \oplus g_{k-3}, \\
&\vdots
\end{aligned}$$

Cách dễ nhất để viết nó trong mã là:

```
int rev_g(int g) {
    int n = 0;
    for (; g; g >>= 1)
        n ^= g;
    return n;
}
```

### 5.25.3 Ứng dụng thực tiễn

Mã Gray có một số ứng dụng hữu ích, đôi khi khá bất ngờ:

- Mã Gray của $n$ bit tạo thành một chu trình Hamilton trên một siêu lập phương, trong đó mỗi bit tương ứng với một chiều.
- Mã Gray được sử dụng để giảm thiểu sai số trong quá trình chuyển đổi tín hiệu số sang tương tự (ví dụ, trong các cảm biến).

- Mã Gray có thể được dùng để giải bài toán Tháp Hà Nội. Gọi $n$ là số đĩa. Bắt đầu với mã Gray độ dài $n$ gồm toàn số không ($G(0)$) và di chuyển giữa các mã Gray liên tiếp (từ $G(i)$ đến $G(i+1)$). Gọi bit thứ $i$ của mã Gray hiện tại biểu thị đĩa thứ $n$ (bit ít quan trọng nhất tương ứng với đĩa nhỏ nhất và bit quan trọng nhất tương ứng với đĩa lớn nhất). Vì mỗi bước chỉ thay đổi đúng một bit, ta có thể coi việc thay đổi bit thứ $i$ là di chuyển đĩa thứ $i$. Chú ý rằng ở mỗi bước (trừ vị trí bắt đầu và kết thúc), chỉ có đúng một lựa chọn di chuyển cho mỗi đĩa (trừ đĩa nhỏ nhất). Luôn có hai lựa chọn di chuyển cho đĩa nhỏ nhất, nhưng có một chiến lược sẽ luôn dẫn đến đáp án: nếu $n$ là số lẻ thì dãy di chuyển của đĩa nhỏ nhất trông như $f \to t \to r \to f \to t \to r \to ...$ (trong đó $f$ là cột ban đầu, $t$ là cột đích và $r$ là cột còn lại), và nếu $n$ là số chẵn: $f \to r \to t \to f \to r \to t \to ....$

- Mã Gray cũng được sử dụng trong lý thuyết thuật toán di truyền.

## 5.26 Thao tác bit

### 5.26.1 Số nhị phân

Số nhị phân là một số được biểu diễn trong hệ số cơ số 2, hay còn gọi là hệ nhị phân, đây là một phương pháp biểu diễn toán học chỉ sử dụng hai ký hiệu: thường là "0" (không) và "1" (một).

Ta nói rằng một bit nào đó được đặt (set) nếu nó là 1, và được xóa (cleared) nếu nó là 0.

Số nhị phân $(a_k a_{k-1} \dots a_1 a_0)_2$ biểu thị số:

$$(a_k a_{k-1} \dots a_1 a_0)_2 = a_k \cdot 2^k + a_{k-1} \cdot 2^{k-1} + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0.$$

Ví dụ, số nhị phân $1101_2$ biểu thị số 13:

$$\begin{aligned} 1101_2 &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13. \end{aligned}$$

Máy tính biểu diễn số nguyên dưới dạng số nhị phân. Các số nguyên dương (cả có dấu và không dấu) chỉ được biểu diễn bằng các chữ số nhị phân của chúng, còn các số nguyên âm có dấu (có thể là dương hoặc âm) thường được biểu diễn bằng bù hai (Two's complement).

```
unsigned int unsigned_number = 13;
assert(unsigned_number == 0b1101);

int positive_signed_number = 13;
assert(positive_signed_number == 0b1101);
```

```
int negative_signed_number = -13;
assert(negative_signed_number == 0b1111'1111'1111'1111'1111'1111'1111'0011);
```

CPU xử lý các bit này rất nhanh với các thao tác cụ thể. Đối với một số bài toán, ta có thể tận dụng biểu diễn số nhị phân này để tăng tốc độ thực thi. Và đối với một số bài toán (thường trong tổ hợp hoặc lập trình động) khi ta muốn theo dõi các đối tượng đã chọn từ một tập hợp cho trước, ta có thể sử dụng một số nguyên đủ lớn mà mỗi chữ số biểu thị một đối tượng, tùy thuộc vào việc chọn hay bỏ đối tượng mà ta đặt hoặc xóa chữ số đó.

### 5.26.2 Các toán tử bit

Tất cả các toán tử được giới thiệu dưới đây đều thực hiện tức thì (tốc độ tương đương phép cộng) trên CPU đối với các số nguyên có độ dài cố định.

#### Toán tử bitwise

- &: Toán tử AND bitwise so sánh từng bit của toán hạng đầu tiên với bit tương ứng của toán hạng thứ hai. Nếu cả hai bit đều là 1, bit kết quả tương ứng được đặt thành 1. Nếu không, bit kết quả được đặt thành 0.
- |: Toán tử OR bao hàm bitwise so sánh từng bit của toán hạng đầu tiên với bit tương ứng của toán hạng thứ hai. Nếu một trong hai bit là 1, bit kết quả tương ứng được đặt thành 1. Nếu không, bit kết quả được đặt thành 0.
- $\wedge$: Toán tử XOR bitwise so sánh từng bit của toán hạng đầu tiên với bit tương ứng của toán hạng thứ hai. Nếu một bit là 0 và bit kia là 1, bit kết quả tương ứng được đặt thành 1. Nếu không, bit kết quả được đặt thành 0.
- $\sim$: Toán tử NOT bitwise đảo ngược từng bit của một số, nếu bit được đặt thì toán tử sẽ xóa nó, nếu bit bị xóa thì toán tử sẽ đặt nó.

Ví dụ:

```
n         = 01011000
n-1       = 01010111
--------------------
n & (n-1) = 01010000

n         = 01011000
n-1       = 01010111
```

```
--------------------
n | (n-1) = 01011111

n         = 01011000
n-1       = 01010111
--------------------
n ^ (n-1) = 00001111

n         = 01011000
--------------------
~n        = 10100111
```

**Toán tử dịch bit**

Có hai toán tử để dịch các bit:

- $\gg$: Dịch số sang phải bằng cách loại bỏ vài chữ số nhị phân cuối cùng của số. Mỗi lần dịch một lần tương đương với phép chia lấy nguyên cho 2, nên dịch phải $k$ lần tương đương với phép chia lấy nguyên cho $2^k$.
  Ví dụ: $5 \gg 2 = 101_2 \gg 2 = 1_2 = 1$, giống như $\frac{5}{2^2} = \frac{5}{4} = 1$. Tuy nhiên, đối với máy tính, việc dịch bit nhanh hơn nhiều so với thực hiện phép chia.

- $\ll$: Dịch số sang trái bằng cách thêm các chữ số 0. Tương tự như dịch phải $k$ lần, dịch trái $k$ lần tương đương với phép nhân với $2^k$.
  Ví dụ: $5 \ll 3 = 101_2 \ll 3 = 101000_2 = 40$, giống như $5 \cdot 2^3 = 5 \cdot 8 = 40$.
  Tuy nhiên, lưu ý rằng đối với số nguyên có độ dài cố định, điều này có nghĩa là bỏ các chữ số bên trái nhất, và nếu dịch quá nhiều, bạn sẽ nhận được số 0.

### 5.26.3 Mẹo hữu ích

**Đặt/đảo/xóa một bit**

Sử dụng các phép dịch bit và một số thao tác bitwise cơ bản, ta có thể dễ dàng đặt, đảo hoặc xóa một bit. $1 \ll x$ là một số chỉ có bit thứ $x$ được đặt, trong khi $\sim (1 \ll x)$ là một số có tất cả các bit được đặt trừ bit thứ $x$.

- $n \mid (1 \ll x)$ đặt bit thứ $x$ trong số $n$.

- $n \wedge (1 \ll x)$ đảo bit thứ $x$ trong số $n$.

- $n \,\&\, \sim (1 \ll x)$ xóa bit thứ $x$ trong số $n$.

### Kiểm tra xem một bit có được đặt không

Giá trị của bit thứ $x$ có thể được kiểm tra bằng cách dịch số $x$ vị trí sang phải, sao cho bit thứ $x$ nằm ở vị trí đơn vị, sau đó ta có thể trích xuất nó bằng cách thực hiện phép bitwise AND với 1.

```
bool is_set(unsigned int number, int x) {
    return (number >> x) & 1;
}
```

### Kiểm tra xem một số có chia hết cho lũy thừa của 2 không

Sử dụng phép AND, ta có thể kiểm tra xem một số $n$ có phải là số chẵn không vì $n \,\&\, 1 = 0$ nếu $n$ chẵn, và $n \,\&\, 1 = 1$ nếu $n$ lẻ. Tổng quát hơn, $n$ chia hết cho $2^k$ chính xác khi $n \,\&\, (2^k - 1) = 0$.

```
bool isDivisibleByPowerOf2(int n, int k) {
    int powerOf2 = 1 << k;
    return (n & (powerOf2 - 1)) == 0;
}
```

Ta có thể tính $2^k$ bằng cách dịch trái 1 đi $k$ vị trí. Mẹo này hoạt động vì $2^k - 1$ là một số gồm chính xác $k$ chữ số 1. Và một số chia hết cho $2^k$ phải có các chữ số 0 ở những vị trí đó.

### Kiểm tra xem một số có phải là lũy thừa của 2 không

Một lũy thừa của hai là một số chỉ có một bit duy nhất (ví dụ: $32 = 0010\ 0000_2$), trong khi số trước đó có bit đó không được đặt và tất cả các bit sau nó được đặt (ví dụ: $31 = 0001\ 1111_2$). Vì vậy, phép AND bitwise của một số với số trước nó sẽ luôn là 0, vì chúng không có bit nào chung được đặt. Bạn có thể dễ dàng kiểm tra rằng điều này chỉ xảy ra với các lũy thừa của hai và số 0 vốn đã không có bit nào được đặt.

```
bool isPowerOfTwo(unsigned int n) {
    return n && !(n & (n - 1));
}
```

### Xóa bit được đặt bên phải nhất

Biểu thức $n \,\&\, (n-1)$ có thể được dùng để tắt bit được đặt bên phải nhất của một số $n$. Điều này hoạt động vì biểu thức $n-1$ đảo ngược tất cả các bit sau bit được đặt bên phải nhất của $n$, bao gồm cả bit được đặt bên phải nhất. Vì vậy, tất cả các chữ số đó khác với

số ban đầu, và bằng cách thực hiện phép AND bitwise, chúng đều được đặt thành 0, cho bạn số ban đầu $n$ với bit được đặt bên phải nhất bị tắt.

Ví dụ, xét số $52 = 0011\ 0100_2$:

```
n         = 00110100
n-1       = 00110011
--------------------
n & (n-1) = 00110000
```

### 5.26.4 Thuật toán của Brian Kernighan

Ta có thể đếm số bit được đặt bằng biểu thức trên.

Ý tưởng là chỉ xem xét các bit được đặt của một số nguyên bằng cách tắt bit được đặt bên phải nhất của nó (sau khi đếm nó), để vòng lặp tiếp theo xem xét bit được đặt bên phải tiếp theo.

```
int countSetBits(int n)
{
    int count = 0;
    while (n)
    {
        n = n & (n - 1);
        count++;
    }
    return count;
}
```

### 5.26.5 Đếm số bit được đặt đến $n$

Để đếm số bit được đặt của tất cả các số đến số $n$ (bao gồm cả $n$), ta có thể chạy thuật toán của Brian Kernighan trên tất cả các số đến $n$. Nhưng điều này sẽ dẫn đến "Time Limit Exceeded" trong các bài nộp thi.

Ta có thể sử dụng thực tế rằng đối với các số đến $2^x$ (tức là từ 1 đến $2^x - 1$), có $x \cdot 2^{x-1}$ bit được đặt. Điều này có thể được hình dung như sau:

```
0 ->   0 0 0 0
1 ->   0 0 0 1
2 ->   0 0 1 0
3 ->   0 0 1 1
4 ->   0 1 0 0
5 ->   0 1 0 1
```

```
6 ->   0 1 1 0
7 ->   0 1 1 1
8 ->   1 0 0 0
```

Ta có thể thấy rằng tất cả các cột trừ cột bên trái nhất đều có 4 (tức là $2^2$) bit được đặt mỗi cột, tức là đến số $2^3 - 1$, số bit được đặt là $3 \cdot 2^{3-1}$.

Với kiến thức mới này, ta có thể đưa ra thuật toán sau:

- Tìm lũy thừa cao nhất của 2 nhỏ hơn hoặc bằng số đã cho. Gọi số này là $x$.
- Tính số bit được đặt từ 1 đến $2^x - 1$ bằng công thức $x \cdot 2^{x-1}$.
- Đếm số bit được đặt trong bit quan trọng nhất từ $2^x$ đến $n$ và cộng vào.
- Trừ $2^x$ khỏi $n$ và lặp lại các bước trên với $n$ mới.

```
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        int x = std::bit_width(n) - 1;
        count += x << (x - 1);
        n -= 1 << x;
        count += n + 1;
    }
    return count;
}
```

### 5.26.6 Các mẹo bổ sung

- $n \mathbin{\&} (n+1)$ xóa tất cả các số 1 ở cuối: $0011\ 0111_2 \to 0011\ 0000_2$.
- $n \mid (n+1)$ đặt bit bị xóa cuối cùng: $0011\ 0101_2 \to 0011\ 0111_2$.
- $n \mathbin{\&} -n$ trích xuất bit được đặt cuối cùng: $0011\ 0100_2 \to 0000\ 0100_2$.

Nhiều mẹo khác có thể được tìm thấy trong sách *Hacker's Delight*.

### 5.26.7 Hỗ trợ ngôn ngữ và trình biên dịch

C++ hỗ trợ một số thao tác này từ C++20 thông qua thư viện chuẩn `bit`:

- `has_single_bit`: kiểm tra xem số có phải là lũy thừa của hai không.
- `bit_ceil` / `bit_floor`: làm tròn lên/xuống đến lũy thừa của hai tiếp theo.

- `rotl` / `rotr`: xoay các bit trong số.
- `countl_zero` / `countr_zero` / `countl_one` / `countr_one`: đếm số 0/1 ở đầu/cuối.
- `popcount`: đếm số bit được đặt.

Ngoài ra, còn có các hàm được định nghĩa trước trong một số trình biên dịch giúp làm việc với bit. Ví dụ, GCC định nghĩa một danh sách tại *Built-in Functions Provided by GCC* cũng hoạt động trong các phiên bản cũ của C++:

- `__builtin_popcount(unsigned int)` trả về số bit được đặt (`__builtin_popcount(0b0001'0010 == 4`).
- `__builtin_ffs(int)` tìm chỉ số của bit được đặt đầu tiên (bên phải nhất) (`__builtin_ffs(0b000 == 3`).
- `__builtin_clz(unsigned int)` đếm số 0 ở đầu (`__builtin_clz(0b0001'0010'1100) == 23`).
- `__builtin_ctz(unsigned int)` đếm số 0 ở cuối (`__builtin_ctz(0b0001'0010'1100) == 2`).
- `__builtin_parity(x)` trả về tính chẵn lẻ của số bit 1 trong biểu diễn bit.

Lưu ý rằng một số thao tác (cả các hàm C++20 và các hàm tích hợp của trình biên dịch) có thể khá chậm trong GCC nếu bạn không bật một mục tiêu trình biên dịch cụ thể với `#pragma GCC target("popcnt")`.

## 5.27 Liệt kê tất cả các mặt nạ con

### 5.27.1 Liệt kê tất cả các mặt nạ con của một mặt nạ cho trước

Cho một mặt nạ bit $m$, bạn muốn duyệt qua tất cả các mặt nạ con của nó một cách hiệu quả, tức là các mặt nạ $s$ mà trong đó chỉ các bit được bật trong mặt nạ $m$ được bật.

Hãy xem xét việc triển khai thuật toán này, dựa trên các mẹo với thao tác bit:

```
int s = m;
while (s > 0) {
    ... bạn có thể sử dụng s ...
    s = (s-1) & m;
}
```

Hoặc, sử dụng câu lệnh `for` gọn gàng hơn:

```
for (int s=m; s; s=(s-1)&m)
    ... bạn có thể sử dụng s ...
```

Trong cả hai biến thể của mã, mặt nạ con bằng 0 sẽ không được xử lý. Ta có thể xử lý nó bên ngoài vòng lặp, hoặc sử dụng thiết kế kém thanh lịch hơn, ví dụ:

```
for (int s=m; ; s=(s-1)&m) {
    ... bạn có thể sử dụng s ...
    if (s==0)  break;
}
```

Hãy xem xét tại sao mã trên duyệt qua tất cả các mặt nạ con của $m$, không lặp lại, và theo thứ tự giảm dần.

Giả sử ta có một mặt nạ bit hiện tại $s$, và ta muốn chuyển sang mặt nạ tiếp theo. Bằng cách trừ 1 khỏi mặt nạ $s$, ta sẽ xóa bit được bật bên phải nhất và tất cả các bit bên phải nó sẽ trở thành 1. Sau đó, ta xóa tất cả các bit 1 "thừa" không có trong mặt nạ $m$ và do đó không thể là một phần của mặt nạ con. Ta thực hiện việc xóa này bằng thao tác bitwise $(s-1)\&m$. Kết quả là, ta "cắt" mặt nạ $s-1$ để xác định giá trị cao nhất mà nó có thể nhận, tức là mặt nạ con tiếp theo sau $s$ theo thứ tự giảm dần.

Do đó, thuật toán này tạo ra tất cả các mặt nạ con của mặt nạ này theo thứ tự giảm dần, chỉ thực hiện hai thao tác mỗi lần lặp.

Một trường hợp đặc biệt là khi $s = 0$. Sau khi thực hiện $s-1$, ta nhận được một mặt nạ mà tất cả các bit đều được bật (biểu diễn bit của $-1$), và sau $(s-1)\&m$, $s$ sẽ bằng $m$. Vì vậy, với mặt nạ $s = 0$, hãy cẩn thận — nếu vòng lặp không kết thúc tại 0, thuật toán có thể rơi vào vòng lặp vô hạn.

### 5.27.2 Duyệt qua tất cả các mặt nạ với các mặt nạ con của chúng. Độ phức tạp $O(3^n)$

Trong nhiều bài toán, đặc biệt là những bài sử dụng lập trình động với mặt nạ bit, bạn muốn duyệt qua tất cả các mặt nạ bit và với mỗi mặt nạ, duyệt qua tất cả các mặt nạ con của nó:

```
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        ... s và m ...
```

Hãy chứng minh rằng vòng lặp bên trong sẽ thực hiện tổng cộng $O(3^n)$ lần lặp.

**Chứng minh thứ nhất:** Xét bit thứ $i$. Có chính xác ba lựa chọn cho nó:

- Nó không được bao gồm trong mặt nạ $m$ (và do đó không được bao gồm trong mặt nạ con $s$),

- Nó được bao gồm trong $m$, nhưng không được bao gồm trong $s$,
- Nó được bao gồm trong cả $m$ và $s$.

Vì có tổng cộng $n$ bit, sẽ có $3^n$ tổ hợp khác nhau.

**Chứng minh thứ hai:** Lưu ý rằng nếu mặt nạ $m$ có $k$ bit được bật, thì nó sẽ có $2^k$ mặt nạ con. Vì ta có tổng cộng $\binom{n}{k}$ mặt nạ với $k$ bit được bật (xem hệ số nhị thức), thì tổng số tổ hợp cho tất cả các mặt nạ sẽ là:

$$\sum_{k=0}^{n} \binom{n}{k} \cdot 2^k.$$

Để tính số này, lưu ý rằng tổng trên bằng với khai triển của $(1+2)^n$ theo định lý nhị thức. Do đó, ta có $3^n$ tổ hợp, như ta muốn chứng minh.

## 5.28 Số học chính xác tùy ý

Số học chính xác tùy ý, còn được gọi là "bignum" hoặc đơn giản là "số học dài", là một tập hợp các cấu trúc dữ liệu và thuật toán cho phép xử lý các số lớn hơn nhiều so với những số có thể chứa trong các kiểu dữ liệu chuẩn. Dưới đây là một số loại số học chính xác tùy ý.

### 5.28.1 Số học dài nguyên cổ điển

Ý tưởng chính là số được lưu trữ dưới dạng một mảng các "chữ số" của nó trong một cơ số nào đó. Một số cơ số được sử dụng phổ biến nhất là cơ số thập phân, các lũy thừa của thập phân ($10^4$ hoặc $10^9$) và cơ số nhị phân.

Các phép toán trên các số ở dạng này được thực hiện bằng các thuật toán "học đường" như cộng cột, trừ cột, nhân cột và chia cột. Cũng có thể sử dụng các thuật toán nhân nhanh: biến đổi Fourier nhanh và thuật toán Karatsuba.

Ở đây, chúng ta mô tả số học dài chỉ cho các số nguyên không âm. Để mở rộng các thuật toán xử lý các số nguyên âm, cần phải thêm và duy trì một cờ "số âm" hoặc sử dụng biểu diễn số nguyên bù hai.

#### Cấu trúc dữ liệu

Ta sẽ lưu trữ các số dưới dạng `vector<int>`, trong đó mỗi phần tử là một "chữ số" của số.

```
typedef vector<int> lnum;
```

Để cải thiện hiệu suất, ta sẽ sử dụng $10^9$ làm cơ số, để mỗi "chữ số" của số dài chứa 9 chữ số thập phân cùng một lúc.

```
const int base = 1000*1000*1000;
```

Các chữ số sẽ được lưu trữ theo thứ tự từ ít quan trọng đến quan trọng nhất. Tất cả các phép toán sẽ được triển khai sao cho sau mỗi phép toán, kết quả không có các số 0 đứng đầu, miễn là các toán hạng cũng không có số 0 đứng đầu. Tất cả các phép toán có thể dẫn đến một số có số 0 đứng đầu nên được theo sau bởi mã để loại bỏ chúng. Lưu ý rằng trong biểu diễn này, có hai cách ghi hợp lệ cho số 0: một vector rỗng và một vector chỉ chứa một chữ số 0.

### Xuất dữ liệu

In số nguyên dài là phép toán dễ nhất. Đầu tiên, ta in phần tử cuối cùng của vector (hoặc 0 nếu vector rỗng), tiếp theo là các phần tử còn lại, được đệm thêm các số 0 đứng đầu nếu cần để chúng dài đúng 9 chữ số.

```
printf("%d", a.empty() ? 0 : a.back());
for (int i=(int)a.size()-2; i>=0; --i)
    printf("%09d", a[i]);
```

Lưu ý rằng ta ép kiểu `a.size()` thành kiểu nguyên để tránh tràn dưới số nguyên không dấu nếu vector chứa ít hơn 2 phần tử.

### Nhập dữ liệu

Để đọc một số nguyên dài, đọc biểu diễn của nó vào một chuỗi và sau đó chuyển đổi thành các "chữ số":

```
for (int i=(int)s.length(); i>0; i-=9)
    if (i < 9)
        a.push_back(atoi(s.substr(0, i).c_str()));
    else
        a.push_back(atoi(s.substr(i-9, 9).c_str()));
```

Nếu ta dùng mảng `char` thay vì chuỗi, mã sẽ ngắn hơn:

```
for (int i=(int)strlen(s); i>0; i-=9) {
    s[i] = 0;
    a.push_back(atoi(i>=9 ? s+i-9 : s));
}
```

Nếu đầu vào có thể chứa các số 0 đứng đầu, chúng có thể được loại bỏ như sau:

```
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

### Phép cộng

Tăng số nguyên dài $a$ thêm $b$ và lưu kết quả vào $a$:

```
int carry = 0;
for (size_t i=0; i<max(a.size(),b.size()) || carry; ++i) {
    if (i == a.size())
        a.push_back(0);
    a[i] += carry + (i < b.size() ? b[i] : 0);
    carry = a[i] >= base;
    if (carry)  a[i] -= base;
}
```

### Phép trừ

Giảm số nguyên dài $a$ đi $b$ ($a \geq b$) và lưu kết quả vào $a$:

```
int carry = 0;
for (size_t i=0; i<b.size() || carry; ++i) {
    a[i] -= carry + (i < b.size() ? b[i] : 0);
    carry = a[i] < 0;
    if (carry)  a[i] += base;
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Lưu ý rằng sau khi thực hiện phép trừ, ta loại bỏ các số 0 đứng đầu để giữ đúng tiền đề rằng các số nguyên dài của chúng ta không có số 0 đứng đầu.

### Nhân với số nguyên ngắn

Nhân số nguyên dài $a$ với số nguyên ngắn $b$ ($b < base$) và lưu kết quả vào $a$:

```
int carry = 0;
for (size_t i=0; i<a.size() || carry; ++i) {
    if (i == a.size())
        a.push_back(0);
    long long cur = carry + a[i] * 1ll * b;
    a[i] = int(cur % base);
    carry = int(cur / base);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Tối ưu bổ sung: Nếu thời gian chạy cực kỳ quan trọng, bạn có thể thử thay hai phép chia bằng một phép bằng cách chỉ tìm kết quả nguyên của phép chia (biến `carry`) và sau đó dùng nó để tìm modulo bằng phép nhân. Điều này thường làm mã nhanh hơn, dù không đáng kể.

#### Nhân với số nguyên dài

Nhân các số nguyên dài $a$ và $b$ và lưu kết quả vào $c$:

```
lnum c(a.size()+b.size());
for (size_t i=0; i<a.size(); ++i)
    for (int j=0, carry=0; j<(int)b.size() || carry; ++j) {
        long long cur = c[i+j] + a[i] * 1ll * (j < (int)b.size() ? b[j] : 0) + carry;
        c[i+j] = int(cur % base);
        carry = int(cur / base);
    }
while (c.size() > 1 && c.back() == 0)
    c.pop_back();
```

#### Chia cho số nguyên ngắn

Chia số nguyên dài $a$ cho số nguyên ngắn $b$ ($b < base$), lưu kết quả nguyên vào $a$ và phần dư vào `carry`:

```
int carry = 0;
for (int i=(int)a.size()-1; i>=0; --i) {
    long long cur = a[i] + carry * 1ll * base;
    a[i] = int(cur / b);
    carry = int(cur % b);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

### 5.28.2 Số học dài nguyên theo biểu diễn phân tích thừa số

Ý tưởng là lưu trữ số nguyên dưới dạng phân tích thừa số của nó, tức là các lũy thừa của các số nguyên tố chia hết cho nó.

Phương pháp này rất dễ triển khai và cho phép thực hiện phép nhân và chia dễ dàng (nhanh hơn về mặt tiệm cận so với phương pháp cổ điển), nhưng không hỗ trợ phép cộng hoặc trừ. Nó cũng rất tiết kiệm bộ nhớ so với phương pháp cổ điển.

Phương pháp này thường được sử dụng cho các phép tính modulo một số không nguyên tố $M$; trong trường hợp này, một số được lưu trữ dưới dạng các lũy thừa của các ước của $M$ chia hết cho số đó, cộng với phần dư modulo $M$.

### 5.28.3 Số học dài nguyên trong các modulo nguyên tố (Thuật toán Garner)

Ý tưởng là chọn một tập hợp các số nguyên tố (thường đủ nhỏ để vừa với kiểu dữ liệu nguyên chuẩn) và lưu trữ một số nguyên dưới dạng một vector các phần dư từ phép chia số nguyên đó cho từng số nguyên tố trong tập hợp.

Định lý thặng dư Trung Hoa phát biểu rằng biểu diễn này đủ để khôi phục duy nhất bất kỳ số nào từ 0 đến tích của các số nguyên tố này trừ 1. Thuật toán Garner cho phép khôi phục số từ biểu diễn như vậy về dạng số nguyên thông thường.

Phương pháp này cho phép tiết kiệm bộ nhớ so với phương pháp cổ điển (dù mức tiết kiệm không đáng kể như trong biểu diễn phân tích thừa số). Ngoài ra, nó cho phép thực hiện nhanh các phép cộng, trừ và nhân trong thời gian tỷ lệ với số lượng số nguyên tố được dùng làm modulo (xem bài viết về định lý thặng dư Trung Hoa để biết cách triển khai).

Điểm đánh đổi là việc chuyển đổi số nguyên trở lại dạng thông thường khá phức tạp và đòi hỏi triển khai số học chính xác tùy ý cổ điển với phép nhân. Ngoài ra, phương pháp này không hỗ trợ phép chia.

### 5.28.4 Số học chính xác tùy ý phân số

Phân số xuất hiện trong các cuộc thi lập trình ít thường xuyên hơn số nguyên, và số học dài phức tạp hơn nhiều khi triển khai cho phân số, vì vậy các cuộc thi lập trình chỉ sử dụng một tập hợp con nhỏ của số học dài phân số.

#### Số học trong phân số không rút gọn

Một số được biểu diễn dưới dạng phân số không rút gọn $\frac{a}{b}$, trong đó $a$ và $b$ là các số nguyên. Tất cả các phép toán trên phân số có thể được biểu diễn dưới dạng các phép toán trên tử số và mẫu số nguyên của các phân số này. Thường thì điều này đòi hỏi sử dụng số học chính xác tùy ý cổ điển để lưu trữ tử số và mẫu số, nhưng đôi khi kiểu dữ liệu nguyên 64-bit tích hợp sẵn là đủ.

#### Lưu trữ vị trí dấu phẩy động dưới dạng kiểu riêng

Đôi khi một bài toán yêu cầu xử lý các số rất nhỏ hoặc rất lớn mà không cho phép tràn trên hoặc tràn dưới. Kiểu dữ liệu `double` tích hợp sẵn sử dụng 8-10 byte và cho phép các giá trị của số mũ trong khoảng $[-308; 308]$, đôi khi có thể không đủ.

Phương pháp rất đơn giản: một biến số nguyên riêng biệt được sử dụng để lưu trữ giá trị của số mũ, và sau mỗi phép toán, số dấu phẩy động được chuẩn hóa, tức là đưa về khoảng $[0.1; 1)$ bằng cách điều chỉnh số mũ tương ứng.

Khi hai số như vậy được nhân hoặc chia, số mũ của chúng nên được cộng hoặc trừ, tương ứng. Khi các số được cộng hoặc trừ, chúng phải được đưa về cùng số mũ trước bằng cách nhân một trong số chúng với 10 mũ bằng hiệu giá trị số mũ.

Lưu ý cuối cùng, cơ số của số mũ không nhất thiết phải bằng 10. Dựa trên biểu diễn bên trong của các số dấu phẩy động, việc sử dụng 2 làm cơ số của số mũ là hợp lý nhất.

## 5.29 Biến đổi Fourier Nhanh

Trong bài viết này, chúng ta sẽ thảo luận về một thuật toán cho phép nhân hai đa thức có độ dài $n$ trong thời gian $O(n \log n)$, nhanh hơn so với phép nhân thông thường mất thời gian $O(n^2)$. Rõ ràng, việc nhân hai số dài cũng có thể được quy về nhân đa thức, do đó hai số dài cũng có thể được nhân trong thời gian $O(n \log n)$ (trong đó $n$ là số chữ số của các số).

Việc khám phá ra Biến đổi Fourier Nhanh (FFT) được ghi nhận cho Cooley và Tukey, những người đã công bố một thuật toán vào năm 1965. Tuy nhiên, thực tế FFT đã được phát hiện nhiều lần trước đó, nhưng tầm quan trọng của nó chưa được hiểu rõ cho đến khi máy tính hiện đại ra đời. Một số nhà nghiên cứu cho rằng FFT được Runge và König phát hiện vào năm 1924. Nhưng thực ra, Gauss đã phát triển một phương pháp như vậy từ năm 1805, dù ông không bao giờ công bố nó.

Lưu ý rằng thuật toán FFT được trình bày ở đây chạy trong thời gian $O(n \log n)$, nhưng nó không hoạt động cho việc nhân các đa thức lớn tùy ý với các hệ số lớn tùy ý hoặc nhân các số nguyên lớn tùy ý. Nó có thể dễ dàng xử lý các đa thức có kích thước $10^5$ với hệ số nhỏ, hoặc nhân hai số có kích thước $10^6$, điều này thường đủ để giải các bài toán lập trình thi đấu. Ngoài quy mô nhân các số với $10^6$ bit, phạm vi và độ chính xác của các số thực dấu phẩy động được sử dụng trong quá trình tính toán sẽ không đủ để cho kết quả cuối cùng chính xác, mặc dù có các biến thể phức tạp hơn có thể thực hiện nhân đa thức/số nguyên lớn tùy ý. Ví dụ, vào năm 1971, Schönhage và Strassen đã phát triển một biến thể để nhân các số lớn tùy ý bằng cách áp dụng FFT đệ quy trong cấu trúc vành, chạy trong thời gian $O(n \log n \log \log n)$. Và gần đây (năm 2019), Harvey và van der Hoeven đã công bố một thuật toán chạy trong thời gian thực sự $O(n \log n)$.

### 5.29.1 Biến đổi Fourier Rời rạc

Giả sử có một đa thức bậc $n-1$:

$$A(x) = a_0 x^0 + a_1 x^1 + \cdots + a_{n-1} x^{n-1}$$

Không mất tính tổng quát, ta giả định rằng $n$ - số lượng hệ số - là một lũy thừa của 2. Nếu $n$ không phải là lũy thừa của 2, ta chỉ cần thêm các hạng $a_i x^i$ còn thiếu và đặt các hệ số $a_i$ bằng 0.

Lý thuyết số phức cho ta biết rằng phương trình $x^n = 1$ có $n$ nghiệm phức (được gọi là các nghiệm bậc $n$ của đơn vị), và các nghiệm này có dạng $w_{n,k} = e^{\frac{2k\pi i}{n}}$ với $k = 0, \ldots, n-1$. Ngoài ra, các số phức này có một số tính chất rất thú vị: ví dụ, nghiệm bậc $n$ chính $w_n = w_{n,1} = e^{\frac{2\pi i}{n}}$ có thể được dùng để mô tả tất cả các nghiệm bậc $n$ khác: $w_{n,k} = (w_n)^k$.

Biến đổi Fourier Rời rạc (DFT) của đa thức $A(x)$ (hoặc tương đương với vector hệ số $(a_0, a_1, \ldots, a_{n-1})$) được định nghĩa là các giá trị của đa thức tại các điểm $x = w_{n,k}$, tức là vector:

$$\begin{aligned}\mathrm{DFT}(a_0, a_1, \ldots, a_{n-1}) &= (y_0, y_1, \ldots, y_{n-1}) \\ &= (A(w_{n,0}), A(w_{n,1}), \ldots, A(w_{n,n-1})) \\ &= (A(w_n^0), A(w_n^1), \ldots, A(w_n^{n-1}))\end{aligned}$$

Tương tự, biến đổi Fourier rời rạc ngược được định nghĩa: DFT ngược của các giá trị của đa thức $(y_0, y_1, \ldots, y_{n-1})$ là các hệ số của đa thức $(a_0, a_1, \ldots, a_{n-1})$:

$$\mathrm{InverseDFT}(y_0, y_1, \ldots, y_{n-1}) = (a_0, a_1, \ldots, a_{n-1})$$

Vậy, nếu DFT trực tiếp tính các giá trị của đa thức tại các nghiệm bậc $n$, thì DFT ngược có thể khôi phục các hệ số của đa thức bằng cách sử dụng các giá trị đó.

### 5.29.2 Ứng dụng của DFT: Nhân đa thức nhanh

Giả sử có hai đa thức $A$ và $B$. Ta tính DFT cho từng đa thức: $\mathrm{DFT}(A)$ và $\mathrm{DFT}(B)$.

Điều gì xảy ra nếu ta nhân hai đa thức này? Rõ ràng tại mỗi điểm, các giá trị chỉ đơn giản được nhân với nhau, tức là:

$$(A \cdot B)(x) = A(x) \cdot B(x).$$

Điều này có nghĩa là nếu ta nhân các vector $\mathrm{DFT}(A)$ và $\mathrm{DFT}(B)$ - bằng cách nhân từng phần tử của vector này với phần tử tương ứng của vector kia - thì ta nhận được chính DFT của đa thức $\mathrm{DFT}(A \cdot B)$:

$$\mathrm{DFT}(A \cdot B) = \mathrm{DFT}(A) \cdot \mathrm{DFT}(B)$$

Cuối cùng, áp dụng DFT ngược, ta thu được:

$$A \cdot B = \mathrm{InverseDFT}(\mathrm{DFT}(A) \cdot \mathrm{DFT}(B))$$

Ở vế phải, tích của hai DFT được hiểu là tích từng cặp của các phần tử vector. Điều này có thể được tính trong thời gian $O(n)$. Nếu ta có thể tính DFT và DFT ngược trong

$O(n \log n)$, thì ta có thể tính tích của hai đa thức (và do đó cả hai số dài) với cùng độ phức tạp thời gian.

Cần lưu ý rằng hai đa thức cần có cùng bậc. Nếu không, hai vector kết quả của DFT sẽ có độ dài khác nhau. Ta có thể đạt được điều này bằng cách thêm các hệ số với giá trị 0.

Ngoài ra, vì kết quả của tích hai đa thức là một đa thức bậc $2(n-1)$, ta phải nhân đôi bậc của mỗi đa thức (bằng cách thêm các số 0). Từ một vector với $n$ giá trị, ta không thể tái tạo đa thức mong muốn với $2n-1$ hệ số.

### 5.29.3 Biến đổi Fourier Nhanh

Biến đổi Fourier Nhanh là một phương pháp cho phép tính DFT trong thời gian $O(n \log n)$. Ý tưởng cơ bản của FFT là áp dụng chiến lược chia để trị. Ta chia vector hệ số của đa thức thành hai vector, tính DFT đệ quy cho từng vector, và kết hợp các kết quả để tính DFT của đa thức hoàn chỉnh.

Giả sử có một đa thức $A(x)$ với bậc $n-1$, trong đó $n$ là lũy thừa của 2, và $n > 1$:

$$A(x) = a_0x^0 + a_1x^1 + \cdots + a_{n-1}x^{n-1}$$

Ta chia nó thành hai đa thức nhỏ hơn, một chứa các hệ số ở vị trí chẵn, và một chứa các hệ số ở vị trí lẻ:

$$A_0(x) = a_0x^0 + a_2x^1 + \cdots + a_{n-2}x^{\frac{n}{2}-1}$$
$$A_1(x) = a_1x^0 + a_3x^1 + \cdots + a_{n-1}x^{\frac{n}{2}-1}$$

Dễ thấy rằng:

$$A(x) = A_0(x^2) + xA_1(x^2).$$

Các đa thức $A_0$ và $A_1$ chỉ có số lượng hệ số bằng một nửa so với đa thức $A$. Nếu ta có thể tính $\mathrm{DFT}(A)$ trong thời gian tuyến tính bằng cách sử dụng $\mathrm{DFT}(A_0)$ và $\mathrm{DFT}(A_1)$, thì ta nhận được công thức đệ quy $T_{\mathrm{DFT}}(n) = 2T_{\mathrm{DFT}}\left(\frac{n}{2}\right) + O(n)$ cho độ phức tạp thời gian, dẫn đến $T_{\mathrm{DFT}}(n) = O(n \log n)$ theo định lý chính.

Hãy tìm hiểu cách thực hiện điều này.

Giả sử ta đã tính được các vector $(y_k^0)_{k=0}^{n/2-1} = \mathrm{DFT}(A_0)$ và $(y_k^1)_{k=0}^{n/2-1} = \mathrm{DFT}(A_1)$. Ta cần tìm biểu thức cho $(y_k)_{k=0}^{n-1} = \mathrm{DFT}(A)$.

Với $\frac{n}{2}$ giá trị đầu tiên, ta có thể sử dụng phương trình đã ghi chú trước đó $A(x) = A_0(x^2) + xA_1(x^2)$:

$$y_k = y_k^0 + w_n^k y_k^1, \quad k = 0, \ldots, \frac{n}{2} - 1.$$

Tuy nhiên, với $\frac{n}{2}$ giá trị tiếp theo, ta cần tìm một biểu thức hơi khác:

$$\begin{aligned} y_{k+n/2} &= A\left(w_n^{k+n/2}\right) \\ &= A_0\left(w_n^{2k+n}\right) + w_n^{k+n/2} A_1\left(w_n^{2k+n}\right) \\ &= A_0\left(w_n^{2k} w_n^n\right) + w_n^k w_n^{n/2} A_1\left(w_n^{2k} w_n^n\right) \\ &= A_0\left(w_n^{2k}\right) - w_n^k A_1\left(w_n^{2k}\right) \\ &= y_k^0 - w_n^k y_k^1 \end{aligned}$$

Ở đây, ta đã sử dụng lại $A(x) = A_0(x^2) + xA_1(x^2)$ và hai đẳng thức $w_n^n = 1$ và $w_n^{n/2} = -1$.

Do đó, ta thu được các công thức mong muốn để tính toàn bộ vector $(y_k)$:

$$\begin{aligned} y_k &= y_k^0 + w_n^k y_k^1, \qquad k = 0, \ldots, \frac{n}{2} - 1, \\ y_{k+n/2} &= y_k^0 - w_n^k y_k^1, \qquad k = 0, \ldots, \frac{n}{2} - 1. \end{aligned}$$

(Mẫu này $a + b$ và $a - b$ đôi khi được gọi là "bướm".)

Vậy, ta đã học cách tính DFT trong thời gian $O(n \log n)$.

### 5.29.4 FFT Ngược

Giả sử vector $(y_0, y_1, \ldots, y_{n-1})$ - các giá trị của đa thức $A$ bậc $n-1$ tại các điểm $x = w_n^k$ - được cho trước. Ta muốn khôi phục các hệ số $(a_0, a_1, \ldots, a_{n-1})$ của đa thức. Đây là bài toán nội suy đã biết, và có các thuật toán tổng quát để giải nó. Nhưng trong trường hợp đặc biệt này (vì ta biết các giá trị tại các nghiệm bậc $n$ của đơn vị), ta có thể thu được một thuật toán đơn giản hơn nhiều (gần giống với FFT trực tiếp).

Ta có thể viết DFT theo định nghĩa của nó dưới dạng ma trận:

$$\begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \cdots & w_n^0 \\ w_n^0 & w_n^1 & w_n^2 & w_n^3 & \cdots & w_n^{n-1} \\ w_n^0 & w_n^2 & w_n^4 & w_n^6 & \cdots & w_n^{2(n-1)} \\ w_n^0 & w_n^3 & w_n^6 & w_n^9 & \cdots & w_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{n-1} & w_n^{2(n-1)} & w_n^{3(n-1)} & \cdots & w_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

Ma trận này được gọi là ma trận Vandermonde.

Do đó, ta có thể tính vector $(a_0, a_1, \ldots, a_{n-1})$ bằng cách nhân vector $(y_0, y_1, \ldots, y_{n-1})$

từ bên trái với nghịch đảo của ma trận:

$$\begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \cdots & w_n^0 \\ w_n^0 & w_n^1 & w_n^2 & w_n^3 & \cdots & w_n^{n-1} \\ w_n^0 & w_n^2 & w_n^4 & w_n^6 & \cdots & w_n^{2(n-1)} \\ w_n^0 & w_n^3 & w_n^6 & w_n^9 & \cdots & w_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{n-1} & w_n^{2(n-1)} & w_n^{3(n-1)} & \cdots & w_n^{(n-1)(n-1)} \end{pmatrix}^{-1} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

Kiểm tra nhanh cho thấy nghịch đảo của ma trận có dạng sau:

$$\frac{1}{n} \begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \cdots & w_n^0 \\ w_n^0 & w_n^{-1} & w_n^{-2} & w_n^{-3} & \cdots & w_n^{-(n-1)} \\ w_n^0 & w_n^{-2} & w_n^{-4} & w_n^{-6} & \cdots & w_n^{-2(n-1)} \\ w_n^0 & w_n^{-3} & w_n^{-6} & w_n^{-9} & \cdots & w_n^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{-(n-1)} & w_n^{-2(n-1)} & w_n^{-3(n-1)} & \cdots & w_n^{-(n-1)(n-1)} \end{pmatrix}$$

Do đó, ta thu được công thức:

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j w_n^{-kj}$$

So sánh với công thức cho $y_k$:

$$y_k = \sum_{j=0}^{n-1} a_j w_n^{kj},$$

ta nhận thấy hai bài toán này gần giống nhau, do đó các hệ số $a_k$ có thể được tìm bằng cùng thuật toán chia để trị như FFT trực tiếp, chỉ thay $w_n^k$ bằng $w_n^{-k}$, và cuối cùng ta cần chia các hệ số kết quả cho $n$.

Vậy, việc tính DFT ngược gần giống với tính DFT trực tiếp, và cũng có thể được thực hiện trong thời gian $O(n \log n)$.

### 5.29.5 Triển khai

Dưới đây, ta trình bày một triển khai đệ quy đơn giản của FFT và FFT ngược, cả hai trong một hàm, vì sự khác biệt giữa FFT xuôi và ngược là rất nhỏ. Để lưu trữ số phức, ta sử dụng kiểu `complex` trong STL của C++.

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    if (n == 1)
        return;

    vector<cd> a0(n / 2), a1(n / 2);
    for (int i = 0; 2 * i < n; i++) {
        a0[i] = a[2*i];
        a1[i] = a[2*i+1];
    }
    fft(a0, invert);
    fft(a1, invert);

    double ang = 2 * PI / n * (invert ? -1 : 1);
    cd w(1), wn(cos(ang), sin(ang));
    for (int i = 0; 2 * i < n; i++) {
        a[i] = a0[i] + w * a1[i];
        a[i + n/2] = a0[i] - w * a1[i];
        if (invert) {
            a[i] /= 2;
            a[i + n/2] /= 2;
        }
        w *= wn;
    }
}
```

Hàm này nhận một vector hệ số, và sẽ tính DFT hoặc DFT ngược rồi lưu kết quả lại trong vector này. Tham số invert cho biết nên tính DFT xuôi hay ngược. Trong hàm, ta đầu tiên kiểm tra xem độ dài vector có bằng 1 không, nếu đúng thì không cần làm gì. Nếu không, ta chia vector $a$ thành hai vector $a0$ và $a1$ và tính DFT cho cả hai một cách đệ quy. Sau đó, ta khởi tạo giá trị $wn$ và một biến $w$, chứa lũy thừa hiện tại của $wn$. Tiếp theo, các giá trị của DFT kết quả được tính bằng các công thức trên.

Nếu cờ invert được đặt, ta thay $wn$ bằng $wn^{-1}$, và mỗi giá trị kết quả được chia cho 2 (vì điều này sẽ được thực hiện ở mỗi cấp đệ quy, cuối cùng sẽ chia các giá trị cuối cùng cho $n$).

Sử dụng hàm này, ta có thể tạo một hàm để nhân hai đa thức:

```
vector<int> multiply(vector<int> const& a, vector<int> const& b) {
```

```
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}
```

Hàm này hoạt động với các đa thức có hệ số nguyên, nhưng bạn cũng có thể điều chỉnh để hoạt động với các kiểu khác. Vì có sai số khi làm việc với số phức, ta cần làm tròn các hệ số kết quả ở cuối.

Cuối cùng, hàm để nhân hai số dài gần như không khác gì hàm nhân đa thức. Điều duy nhất ta phải làm sau đó là chuẩn hóa số:

```
    int carry = 0;
    for (int i = 0; i < n; i++) {
        result[i] += carry;
        carry = result[i] / 10;
        result[i] %= 10;
    }
```

Vì độ dài của tích hai số không bao giờ vượt quá tổng độ dài của cả hai số, kích thước vector là đủ để thực hiện tất cả các thao tác nhớ.

### 5.29.6 Triển khai cải tiến: Tính toán tại chỗ

Để tăng hiệu quả, ta sẽ chuyển từ triển khai đệ quy sang lặp. Trong triển khai đệ quy trên, ta đã tách rõ ràng vector $a$ thành hai vector - các phần tử ở vị trí chẵn được gán cho một vector tạm, và các phần tử ở vị trí lẻ cho vector khác. Tuy nhiên, nếu ta sắp xếp lại các phần tử theo một cách nhất định, ta không cần tạo các vector tạm này (tức là tất cả tính toán có thể được thực hiện "tại chỗ", ngay trong vector $A$).

Lưu ý rằng ở cấp đệ quy đầu tiên, các phần tử có bit thấp nhất của vị trí bằng 0 được gán cho vector $a_0$, và những phần tử có bit thấp nhất bằng 1 được gán cho $a_1$. Ở cấp đệ quy thứ hai, điều tương tự xảy ra, nhưng với bit thấp thứ hai, v.v. Do đó, nếu ta đảo ngược các bit của vị trí mỗi hệ số và sắp xếp chúng theo các giá trị đảo ngược này, ta sẽ nhận được thứ tự mong muốn (được gọi là hoán vị đảo bit).

Ví dụ, thứ tự mong muốn cho $n = 8$ có dạng:

$$a = \left\{ \Big[(a_0, a_4), (a_2, a_6)\Big], \Big[(a_1, a_5), (a_3, a_7)\Big] \right\}$$

Thực tế, ở cấp đệ quy đầu tiên (được bao quanh bởi dấu ngoặc nhọn), vector được chia thành hai phần $[a_0, a_2, a_4, a_6]$ và $[a_1, a_3, a_5, a_7]$. Như ta thấy, trong hoán vị đảo bit, điều này tương ứng với việc chia vector thành hai nửa: $\frac{n}{2}$ phần tử đầu tiên và $\frac{n}{2}$ phần tử cuối cùng. Sau đó, có một lệnh gọi đệ quy cho mỗi nửa. Giả sử DFT kết quả cho mỗi nửa được trả về tại vị trí của các phần tử đó (tức là nửa đầu và nửa sau của vector $a$):

$$a = \left\{ \Big[y_0^0, y_1^0, y_2^0, y_3^0\Big], \Big[y_0^1, y_1^1, y_2^1, y_3^1\Big] \right\}$$

Bây giờ ta muốn kết hợp hai DFT thành một cho toàn bộ vector. Thứ tự các phần tử là lý tưởng, và ta cũng có thể thực hiện phép hợp trực tiếp trong vector này. Ta có thể lấy các phần tử $y_0^0$ và $y_0^1$ và thực hiện phép biến đổi bướm. Vị trí của hai giá trị kết quả trùng với vị trí của hai giá trị ban đầu, nên ta được:

$$a = \left\{ \Big[y_0^0 + w_n^0 y_0^1, y_1^0, y_2^0, y_3^0\Big], \Big[y_0^0 - w_n^0 y_0^1, y_1^1, y_2^1, y_3^1\Big] \right\}$$

Tương tự, ta có thể tính phép biến đổi bướm của $y_1^0$ và $y_1^1$ và đặt kết quả vào vị trí của chúng, v.v. Kết quả là ta được:

$$a = \left\{ \Big[y_0^0 + w_n^0 y_0^1, y_1^0 + w_n^1 y_1^1, y_2^0 + w_n^2 y_2^1, y_3^0 + w_n^3 y_3^1\Big], \Big[y_0^0 - w_n^0 y_0^1, y_1^0 - w_n^1 y_1^1, y_2^0 - w_n^2 y_2^1, y_3^0 - w_n^3 y_3^1\Big] \right\}$$

Vậy, ta đã tính được DFT cần thiết từ vector $a$.

Ở đây, ta đã mô tả quá trình tính DFT chỉ ở cấp đệ quy đầu tiên, nhưng rõ ràng điều này cũng áp dụng cho tất cả các cấp khác. Do đó, sau khi áp dụng hoán vị đảo bit, ta có thể tính DFT tại chỗ, không cần bộ nhớ bổ sung.

Điều này còn cho phép ta loại bỏ đệ quy. Ta chỉ cần bắt đầu từ cấp thấp nhất, tức là chia vector thành các cặp và áp dụng phép biến đổi bướm cho chúng. Điều này dẫn đến vector $a$ với công việc của cấp cuối cùng được áp dụng. Ở bước tiếp theo, ta chia vector thành các vector có kích thước 4, và lại áp dụng phép biến đổi bướm, cho ta DFT cho mỗi khối kích thước 4. Và cứ tiếp tục như vậy. Cuối cùng, ở bước cuối, ta thu được kết quả của DFT của cả hai nửa của $a$, và bằng cách áp dụng phép biến đổi bướm, ta thu được DFT cho toàn bộ vector $a$.

```
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; i++) {
        if (num & (1 << i))
            res |= 1 << (lg_n - 1 - i);
    }
    return res;
}

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        lg_n++;

    for (int i = 0; i < n; i++) {
        if (i < reverse(i, lg_n))
            swap(a[i], a[reverse(i, lg_n)]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
```

```
    }
}
```

Đầu tiên, ta áp dụng hoán vị đảo bit bằng cách hoán đổi mỗi phần tử với phần tử ở vị trí đảo ngược. Sau đó, trong $\log n - 1$ trạng thái của thuật toán, ta tính DFT cho mỗi khối có kích thước tương ứng len. Với tất cả các khối này, ta có cùng nghiệm bậc đơn vị wlen. Ta lặp qua tất cả các khối và thực hiện phép biến đổi bướm trên mỗi khối.

Ta có thể tối ưu thêm việc đảo ngược các bit. Trong triển khai trước, ta đã lặp qua tất cả các bit của chỉ số và tạo chỉ số đảo ngược bitwise. Tuy nhiên, ta có thể đảo ngược các bit theo cách khác.

Giả sử $j$ đã chứa đảo ngược của $i$. Khi chuyển sang $i+1$, ta phải tăng $i$, và cũng phải tăng $j$, nhưng trong hệ số "đảo ngược". Cộng một trong hệ nhị phân thông thường tương đương với việc lật tất cả các số 1 ở đuôi thành 0 và lật số 0 ngay trước chúng thành 1. Tương tự, trong hệ số "đảo ngược", ta lật tất cả các số 1 ở đầu, và cả số 0 tiếp theo.

Do đó, ta có triển khai sau:

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
```

```
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}
```

Ngoài ra, ta có thể tính trước hoán vị đảo bit. Điều này đặc biệt hữu ích khi kích thước $n$ là như nhau cho tất cả các lần gọi. Nhưng ngay cả khi ta chỉ có ba lần gọi (cần thiết để nhân hai đa thức), hiệu quả vẫn đáng chú ý. Ta cũng có thể tính trước tất cả các nghiệm bậc đơn vị và lũy thừa của chúng.

### 5.29.7 Biến đổi Số học

Bây giờ ta chuyển hướng mục tiêu một chút. Ta vẫn muốn nhân hai đa thức trong thời gian $O(n \log n)$, nhưng lần này ta muốn tính các hệ số theo modulo một số nguyên tố $p$. Tất nhiên, cho bài toán này, ta có thể dùng DFT thông thường và áp dụng toán tử modulo cho kết quả. Tuy nhiên, làm như vậy có thể dẫn đến sai số làm tròn, đặc biệt khi xử lý các số lớn. Biến đổi Số học (NTT) có ưu điểm là chỉ làm việc với số nguyên, do đó kết quả được đảm bảo chính xác.

Biến đổi Fourier rời rạc dựa trên số phức và các nghiệm bậc $n$ của đơn vị. Để tính nó hiệu quả, ta sử dụng rộng rãi các tính chất của các nghiệm (ví dụ, có một nghiệm sinh ra tất cả các nghiệm khác bằng lũy thừa).

Nhưng các tính chất tương tự cũng đúng với các nghiệm bậc $n$ của đơn vị trong số học modulo. Một nghiệm bậc $n$ của đơn vị trong trường nguyên tố là một số $w_n$ thỏa mãn:

$$\begin{aligned} (w_n)^n &= 1 \pmod p, \\ (w_n)^k &\neq 1 \pmod p, \quad 1 \le k < n. \end{aligned}$$

Các $n-1$ nghiệm khác có thể được thu bằng lũy thừa của nghiệm $w_n$.

Để áp dụng nó trong thuật toán FFT, ta cần một nghiệm tồn tại cho một số $n$ là lũy thừa của 2, và cũng cho tất cả các lũy thừa nhỏ hơn. Ta có thể nhận thấy tính chất thú vị sau:

$$\begin{aligned} (w_n^2)^m &= w_n^n = 1 \pmod p, \quad \text{với } m = \frac{n}{2} \\ (w_n^2)^k &= w_n^{2k} \neq 1 \pmod p, \quad 1 \le k < m. \end{aligned}$$

Vậy, nếu $w_n$ là nghiệm bậc $n$ của đơn vị, thì $w_n^2$ là nghiệm bậc $\frac{n}{2}$ của đơn vị. Và do đó, với tất cả các lũy thừa nhỏ hơn của 2, tồn tại các nghiệm bậc cần thiết, và chúng có thể được tính bằng $w_n$.

Để tính DFT ngược, ta cần nghịch đảo $w_n^{-1}$ của $w_n$. Nhưng với modulo nguyên tố, nghịch đảo luôn tồn tại.

Vậy, tất cả các tính chất ta cần từ các nghiệm phức cũng có trong số học modulo, miễn là ta có một modulo $p$ đủ lớn mà tại đó tồn tại nghiệm bậc $n$ của đơn vị.

Ví dụ, ta có thể lấy các giá trị sau: modulo $p = 7340033$, $w_{2^{20}} = 5$. Nếu modulo này không đủ, ta cần tìm một cặp khác. Ta có thể sử dụng sự thật rằng với các modulo dạng $p = c2^k + 1$ (và $p$ là số nguyên tố), luôn tồn tại nghiệm bậc $2^k$ của đơn vị. Có thể chứng minh rằng $g^c$ là nghiệm bậc $2^k$ của đơn vị, trong đó $g$ là một nghiệm nguyên tố của $p$.

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;

void fft(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
```

```
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }

    if (invert) {
        int n_1 = inverse(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

Ở đây, hàm `inverse` tính nghịch đảo modulo (xem Nghịch đảo Nhân Modulo). Các hằng số `mod`, `root`, `root_pw` xác định modulo và nghiệm, và `root_1` là nghịch đảo của `root` theo modulo `mod`.

Trong thực tế, triển khai này chậm hơn triển khai sử dụng số phức (do số lượng lớn các phép toán modulo), nhưng nó có một số ưu điểm như sử dụng ít bộ nhớ hơn và không có sai số làm tròn.

### 5.29.8 Nhân với modulo tùy ý

Ở đây, ta muốn đạt được mục tiêu tương tự như phần trước: nhân hai đa thức $A(x)$ và $B(x)$, và tính các hệ số theo modulo một số $M$. Biến đổi số học chỉ hoạt động với một số nguyên tố nhất định. Vậy trường hợp modulo không có dạng mong muốn thì sao?

Một lựa chọn là thực hiện nhiều biến đổi số học với các số nguyên tố khác nhau dạng $c2^k + 1$, sau đó áp dụng Định lý Dư Trung Hoa để tính các hệ số cuối cùng.

Một lựa chọn khác là phân phối các đa thức $A(x)$ và $B(x)$ thành hai đa thức nhỏ hơn mỗi đa thức:

$$A(x) = A_1(x) + A_2(x) \cdot C$$
$$B(x) = B_1(x) + B_2(x) \cdot C$$

với $C \approx \sqrt{M}$.

Khi đó, tích của $A(x)$ và $B(x)$ có thể được biểu diễn như sau:

$$A(x) \cdot B(x) = A_1(x) \cdot B_1(x) + (A_1(x) \cdot B_2(x) + A_2(x) \cdot B_1(x)) \cdot C + (A_2(x) \cdot B_2(x)) \cdot C^2$$

Các đa thức $A_1(x)$, $A_2(x)$, $B_1(x)$ và $B_2(x)$ chỉ chứa các hệ số nhỏ hơn $\sqrt{M}$, do đó các hệ số của tất cả các tích xuất hiện nhỏ hơn $M \cdot n$, thường đủ nhỏ để xử lý với các kiểu dấu phẩy động thông thường.

Cách tiếp cận này do đó yêu cầu tính các tích của đa thức với hệ số nhỏ hơn (bằng cách sử dụng FFT và FFT ngược thông thường), sau đó tích ban đầu có thể được khôi phục bằng cách cộng và nhân modulo trong thời gian $O(n)$.

### 5.29.9 Ứng dụng

DFT có thể được sử dụng trong rất nhiều bài toán khác, thoạt nhìn không liên quan đến việc nhân đa thức.

#### Tất cả các tổng có thể

Ta được cho hai mảng $a[]$ và $b[]$. Ta phải tìm tất cả các tổng có thể $a[i] + b[j]$, và với mỗi tổng, đếm xem nó xuất hiện bao nhiêu lần.

Ví dụ, với $a = [1, 2, 3]$ và $b = [2, 4]$, ta được: tổng 3 có thể đạt được theo 1 cách, tổng 4 cũng theo 1 cách, 5 theo 2 cách, 6 theo 1 cách, 7 theo 1 cách.

Ta xây dựng hai đa thức $A$ và $B$ cho các mảng $a$ và $b$. Các số trong mảng sẽ đóng vai trò là số mũ trong đa thức ($a[i] \Rightarrow x^{a[i]}$); và hệ số của hạng này sẽ là tần suất xuất hiện của số đó trong mảng.

Sau đó, bằng cách nhân hai đa thức này trong thời gian $O(n \log n)$, ta nhận được một đa thức $C$, trong đó các số mũ cho ta biết tổng nào có thể đạt được, và các hệ số cho ta biết tần suất. Để minh họa trên ví dụ:

$$(1x^1 + 1x^2 + 1x^3)(1x^2 + 1x^4) = 1x^3 + 1x^4 + 2x^5 + 1x^6 + 1x^7$$

#### Tất cả các tích vô hướng có thể

Ta được cho hai mảng $a[]$ và $b[]$ có độ dài $n$. Ta phải tính các tích của $a$ với mọi dịch chuyển vòng của $b$.

Ta tạo hai mảng mới có kích thước $2n$: Ta đảo ngược $a$ và thêm $n$ số 0 vào sau nó. Và ta chỉ cần nối $b$ với chính nó. Khi ta nhân hai mảng này như đa thức, và xem các hệ số $c[n-1], c[n], \ldots, c[2n-2]$ của tích $c$, ta được:

$$c[k] = \sum_{i+j=k} a[i]b[j]$$

Và vì tất cả các phần tử $a[i] = 0$ với $i \geq n$:

$$c[k] = \sum_{i=0}^{n-1} a[i]b[k-i]$$

Dễ thấy rằng tổng này chính là tích vô hướng của vector $a$ với dịch chuyển vòng trái thứ $(k - (n-1))$ của $b$. Do đó, các hệ số này là câu trả lời cho bài toán, và ta vẫn có thể đạt được nó trong thời gian $O(n \log n)$. Lưu ý rằng $c[2n-1]$ cũng cho ta dịch chuyển vòng thứ $n$, nhưng nó giống với dịch chuyển vòng thứ 0, nên ta không cần xét riêng vào câu trả lời.

### Hai dải

Ta được cho hai dải Boolean (mảng vòng của các giá trị 0 và 1) $a$ và $b$. Ta muốn tìm tất cả các cách gắn dải đầu tiên vào dải thứ hai, sao cho không có vị trí nào mà 1 của dải đầu tiên nằm cạnh 1 của dải thứ hai.

Bài toán này thực ra không khác nhiều so với bài trước. Gắn hai dải chỉ có nghĩa là ta thực hiện dịch chuyển vòng trên mảng thứ hai, và ta có thể gắn hai dải nếu tích vô hướng của hai mảng bằng 0.

### Khớp chuỗi

Ta được cho hai chuỗi, một văn bản $T$ và một mẫu $P$, gồm các chữ cái thường. Ta phải tính tất cả các lần xuất hiện của mẫu trong văn bản.

Ta tạo một đa thức cho mỗi chuỗi ($T[i]$ và $P[i]$ là các số từ 0 đến 25 tương ứng với 26 chữ cái của bảng chữ cái):

$$A(x) = a_0x^0 + a_1x^1 + \cdots + a_{n-1}x^{n-1}, \quad n = |T|$$

với

$$a_i = \cos(\alpha_i) + i\sin(\alpha_i), \quad \alpha_i = \frac{2\pi T[i]}{26}.$$

Và

$$B(x) = b_0x^0 + b_1x^1 + \cdots + b_{m-1}x^{m-1}, \quad m = |P|$$

với

$$b_i = \cos(\beta_i) - i\sin(\beta_i), \quad \beta_i = \frac{2\pi P[m-i-1]}{26}.$$

Lưu ý rằng với biểu thức $P[m-i-1]$, ta đảo ngược mẫu một cách rõ ràng.

Hệ số thứ $(m-1+i)$ của tích hai đa thức $C(x) = A(x) \cdot B(x)$ sẽ cho ta biết liệu mẫu có xuất hiện trong văn bản tại vị trí $i$ hay không:

$$c_{m-1+i} = \sum_{j=0}^{m-1} a_{i+j} \cdot b_{m-1-j} = \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i\sin(\alpha_{i+j})) \cdot (\cos(\beta_j) - i\sin(\beta_j))$$

với $\alpha_{i+j} = \frac{2\pi T[i+j]}{26}$ và $\beta_j = \frac{2\pi P[j]}{26}$.

Nếu có sự khớp, thì $T[i+j] = P[j]$, và do đó $\alpha_{i+j} = \beta_j$. Điều này cho (sử dụng định lý lượng giác Pythagore):

$$\begin{aligned} c_{m-1+i} &= \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i\sin(\alpha_{i+j})) \cdot (\cos(\alpha_{i+j}) - i\sin(\alpha_{i+j})) \\ &= \sum_{j=0}^{m-1} \cos(\alpha_{i+j})^2 + \sin(\alpha_{i+j})^2 = \sum_{j=0}^{m-1} 1 = m \end{aligned}$$

Nếu không có sự khớp, thì ít nhất một ký tự khác nhau, dẫn đến một trong các tích $a_{i+1} \cdot b_{m-1-j}$ không bằng 1, khiến hệ số $c_{m-1+i} \neq m$.

**Khớp chuỗi với ký tự đại diện**

Đây là mở rộng của bài toán trước. Lần này, ta cho phép mẫu chứa ký tự đại diện , có thể khớp với mọi chữ cái có thể. Ví dụ, mẫu $a * c$ xuất hiện trong văn bản *abccaacc* tại đúng ba vị trí, tại chỉ số 0, chỉ số 4 và chỉ số 5.

Ta tạo các đa thức giống hệt như trên, ngoại trừ việc ta đặt $b_i = 0$ nếu $P[m-i-1] = *$. Nếu $x$ là số ký tự đại diện trong $P$, thì ta sẽ có sự khớp của $P$ trong $T$ tại chỉ số $i$ nếu $c_{m-1+i} = m - x$.

## 5.30 Biến đổi Số học

Bây giờ ta chuyển hướng mục tiêu một chút. Ta vẫn muốn nhân hai đa thức trong thời gian $O(n \log n)$, nhưng lần này ta muốn tính các hệ số theo modulo một số nguyên tố $p$. Tất nhiên, cho bài toán này, ta có thể dùng DFT thông thường và áp dụng toán tử modulo cho kết quả. Tuy nhiên, làm như vậy có thể dẫn đến sai số làm tròn, đặc biệt khi xử lý các số lớn. Biến đổi Số học (NTT) có ưu điểm là chỉ làm việc với số nguyên, do đó kết quả được đảm bảo chính xác.

Biến đổi Fourier rời rạc dựa trên số phức và các nghiệm bậc $n$ của đơn vị. Để tính nó hiệu quả, ta sử dụng rộng rãi các tính chất của các nghiệm (ví dụ, có một nghiệm sinh ra tất cả các nghiệm khác bằng lũy thừa).

Nhưng các tính chất tương tự cũng đúng với các nghiệm bậc $n$ của đơn vị trong số học modulo. Một nghiệm bậc $n$ của đơn vị trong trường nguyên tố là một số $w_n$ thỏa mãn:

$$\begin{aligned} (w_n)^n &= 1 \pmod p, \\ (w_n)^k &\neq 1 \pmod p, \quad 1 \leq k < n. \end{aligned}$$

Các $n-1$ nghiệm khác có thể được thu bằng lũy thừa của nghiệm $w_n$.

Để áp dụng nó trong thuật toán Biến đổi Fourier Nhanh, ta cần một nghiệm tồn tại cho một số $n$ là lũy thừa của 2, và cũng cho tất cả các lũy thừa nhỏ hơn. Ta có thể nhận thấy tính chất thú vị sau:

$$\begin{aligned} (w_n^2)^m &= w_n^n = 1 \pmod p, \quad \text{với } m = \frac{n}{2} \\ (w_n^2)^k &= w_n^{2k} \neq 1 \pmod p, \quad 1 \leq k < m. \end{aligned}$$

Vậy, nếu $w_n$ là nghiệm bậc $n$ của đơn vị, thì $w_n^2$ là nghiệm bậc $\frac{n}{2}$ của đơn vị. Và do đó, với tất cả các lũy thừa nhỏ hơn của 2, tồn tại các nghiệm bậc cần thiết, và chúng có thể được tính bằng $w_n$.

Để tính DFT ngược, ta cần nghịch đảo $w_n^{-1}$ của $w_n$. Nhưng với modulo nguyên tố, nghịch đảo luôn tồn tại.

Vậy, tất cả các tính chất ta cần từ các nghiệm phức cũng có trong số học modulo, miễn là ta có một modulo $p$ đủ lớn mà tại đó tồn tại nghiệm bậc $n$ của đơn vị.

Ví dụ, ta có thể lấy các giá trị sau: modulo $p = 7340033$, $w_{2^{20}} = 5$. Nếu modulo này không đủ, ta cần tìm một cặp khác. Ta có thể sử dụng sự thật rằng với các modulo dạng $p = c2^k + 1$ (và $p$ là số nguyên tố), luôn tồn tại nghiệm bậc $2^k$ của đơn vị. Có thể chứng minh rằng $g^c$ là nghiệm bậc $2^k$ của đơn vị, trong đó $g$ là một nghiệm nguyên tố của $p$.

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;

void fft(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
```

```
    }

    if (invert) {
        int n_1 = inverse(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

Ở đây, hàm `inverse` tính nghịch đảo modulo (xem Nghịch đảo Nhân Modulo). Các hằng số `mod`, `root`, `root_pw` xác định modulo và nghiệm, và `root_1` là nghịch đảo của `root` theo modulo `mod`.

Trong thực tế, triển khai này chậm hơn triển khai sử dụng số phức (do số lượng lớn các phép toán modulo), nhưng nó có một số ưu điểm như sử dụng ít bộ nhớ hơn và không có sai số làm tròn.

### 5.30.1 Nhân với modulo tùy ý

Ở đây, ta muốn đạt được mục tiêu tương tự như phần trước: nhân hai đa thức $A(x)$ và $B(x)$, và tính các hệ số theo modulo một số $M$. Biến đổi số học chỉ hoạt động với một số nguyên tố nhất định. Vậy trường hợp modulo không có dạng mong muốn thì sao?

Một lựa chọn là thực hiện nhiều biến đổi số học với các số nguyên tố khác nhau dạng $c2^k + 1$, sau đó áp dụng Định lý Dư Trung Hoa để tính các hệ số cuối cùng.

Một lựa chọn khác là phân phối các đa thức $A(x)$ và $B(x)$ thành hai đa thức nhỏ hơn mỗi đa thức:

$$A(x) = A_1(x) + A_2(x) \cdot C$$
$$B(x) = B_1(x) + B_2(x) \cdot C$$

với $C \approx \sqrt{M}$.

Khi đó, tích của $A(x)$ và $B(x)$ có thể được biểu diễn như sau:

$$A(x) \cdot B(x) = A_1(x) \cdot B_1(x) + (A_1(x) \cdot B_2(x) + A_2(x) \cdot B_1(x)) \cdot C + (A_2(x) \cdot B_2(x)) \cdot C^2$$

Các đa thức $A_1(x)$, $A_2(x)$, $B_1(x)$ và $B_2(x)$ chỉ chứa các hệ số nhỏ hơn $\sqrt{M}$, do đó các hệ số của tất cả các tích xuất hiện nhỏ hơn $M \cdot n$, thường đủ nhỏ để xử lý với các kiểu dấu phẩy động thông thường.

Cách tiếp cận này do đó yêu cầu tính các tích của đa thức với hệ số nhỏ hơn (bằng cách sử dụng FFT và FFT ngược thông thường), sau đó tích ban đầu có thể được khôi phục bằng cách cộng và nhân modulo trong thời gian $O(n)$.

### 5.30.2 Ứng dụng

DFT có thể được sử dụng trong rất nhiều bài toán khác, thoạt nhìn không liên quan đến việc nhân đa thức.

#### Tất cả các tổng có thể

Ta được cho hai mảng $a[]$ và $b[]$. Ta phải tìm tất cả các tổng có thể $a[i] + b[j]$, và với mỗi tổng, đếm xem nó xuất hiện bao nhiêu lần.

Ví dụ, với $a = [1, 2, 3]$ và $b = [2, 4]$, ta được: tổng 3 có thể đạt được theo 1 cách, tổng 4 cũng theo 1 cách, 5 theo 2 cách, 6 theo 1 cách, 7 theo 1 cách.

Ta xây dựng hai đa thức $A$ và $B$ cho các mảng $a$ và $b$. Các số trong mảng sẽ đóng vai trò là số mũ trong đa thức ($a[i] \Rightarrow x^{a[i]}$); và hệ số của hạng này sẽ là tần suất xuất hiện của số đó trong mảng.

Sau đó, bằng cách nhân hai đa thức này trong thời gian $O(n \log n)$, ta nhận được một đa thức $C$, trong đó các số mũ cho ta biết tổng nào có thể đạt được, và các hệ số cho ta biết tần suất. Để minh họa trên ví dụ:

$$(1x^1 + 1x^2 + 1x^3)(1x^2 + 1x^4) = 1x^3 + 1x^4 + 2x^5 + 1x^6 + 1x^7$$

#### Tất cả các tích vô hướng có thể

Ta được cho hai mảng $a[]$ và $b[]$ có độ dài $n$. Ta phải tính các tích của $a$ với mọi dịch chuyển vòng của $b$.

Ta tạo hai mảng mới có kích thước $2n$: Ta đảo ngược $a$ và thêm $n$ số 0 vào sau nó. Và ta chỉ cần nối $b$ với chính nó. Khi ta nhân hai mảng này như đa thức, và xem các hệ số $c[n-1], c[n], \ldots, c[2n-2]$ của tích $c$, ta được:

$$c[k] = \sum_{i+j=k} a[i]b[j]$$

Và vì tất cả các phần tử $a[i] = 0$ với $i \geq n$:

$$c[k] = \sum_{i=0}^{n-1} a[i]b[k-i]$$

Dễ thấy rằng tổng này chính là tích vô hướng của vector $a$ với dịch chuyển vòng trái thứ $(k - (n-1))$ của $b$. Do đó, các hệ số này là câu trả lời cho bài toán, và ta vẫn có thể đạt được nó trong thời gian $O(n \log n)$. Lưu ý rằng $c[2n-1]$ cũng cho ta dịch chuyển vòng thứ $n$, nhưng nó giống với dịch chuyển vòng thứ 0, nên ta không cần xét riêng vào câu trả lời.

### Hai dải

Ta được cho hai dải Boolean (mảng vòng của các giá trị 0 và 1) $a$ và $b$. Ta muốn tìm tất cả các cách gắn dải đầu tiên vào dải thứ hai, sao cho không có vị trí nào mà 1 của dải đầu tiên nằm cạnh 1 của dải thứ hai.

Bài toán này thực ra không khác nhiều so với bài trước. Gắn hai dải chỉ có nghĩa là ta thực hiện dịch chuyển vòng trên mảng thứ hai, và ta có thể gắn hai dải nếu tích vô hướng của hai mảng bằng 0.

### Khớp chuỗi

Ta được cho hai chuỗi, một văn bản $T$ và một mẫu $P$, gồm các chữ cái thường. Ta phải tính tất cả các lần xuất hiện của mẫu trong văn bản.

Ta tạo một đa thức cho mỗi chuỗi ($T[i]$ và $P[i]$ là các số từ 0 đến 25 tương ứng với 26 chữ cái của bảng chữ cái):

$$A(x) = a_0x^0 + a_1x^1 + \cdots + a_{n-1}x^{n-1}, \quad n = |T|$$

với

$$a_i = \cos(\alpha_i) + i\sin(\alpha_i), \quad \alpha_i = \frac{2\pi T[i]}{26}.$$

Và

$$B(x) = b_0x^0 + b_1x^1 + \cdots + b_{m-1}x^{m-1}, \quad m = |P|$$

với

$$b_i = \cos(\beta_i) - i\sin(\beta_i), \quad \beta_i = \frac{2\pi P[m-i-1]}{26}.$$

Lưu ý rằng với biểu thức $P[m-i-1]$, ta đảo ngược mẫu một cách rõ ràng.

Hệ số thứ $(m-1+i)$ của tích hai đa thức $C(x) = A(x) \cdot B(x)$ sẽ cho ta biết liệu mẫu có xuất hiện trong văn bản tại vị trí $i$ hay không:

$$c_{m-1+i} = \sum_{j=0}^{m-1} a_{i+j} \cdot b_{m-1-j} = \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i\sin(\alpha_{i+j})) \cdot (\cos(\beta_j) - i\sin(\beta_j))$$

với $\alpha_{i+j} = \frac{2\pi T[i+j]}{26}$ và $\beta_j = \frac{2\pi P[j]}{26}$.

Nếu có sự khớp, thì $T[i+j] = P[j]$, và do đó $\alpha_{i+j} = \beta_j$. Điều này cho (sử dụng định lý lượng giác Pythagore):

$$\begin{aligned} c_{m-1+i} &= \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i\sin(\alpha_{i+j})) \cdot (\cos(\alpha_{i+j}) - i\sin(\alpha_{i+j})) \\ &= \sum_{j=0}^{m-1} \cos(\alpha_{i+j})^2 + \sin(\alpha_{i+j})^2 = \sum_{j=0}^{m-1} 1 = m \end{aligned}$$

Nếu không có sự khớp, thì ít nhất một ký tự khác nhau, dẫn đến một trong các tích $a_{i+1} \cdot b_{m-1-j}$ không bằng 1, khiến hệ số $c_{m-1+i} \neq m$.

**Khớp chuỗi với ký tự đại diện**

Đây là mở rộng của bài toán trước. Lần này, ta cho phép mẫu chứa ký tự đại diện , có thể khớp với mọi chữ cái có thể. Ví dụ, mẫu $a * c$ xuất hiện trong văn bản *abccaacc* tại đúng ba vị trí, tại chỉ số 0, chỉ số 4 và chỉ số 5.

Ta tạo các đa thức giống hệt như trên, ngoại trừ việc ta đặt $b_i = 0$ nếu $P[m-i-1] = *$. Nếu $x$ là số ký tự đại diện trong $P$, thì ta sẽ có sự khớp của $P$ trong $T$ tại chỉ số $i$ nếu $c_{m-1+i} = m - x$.

# 5.31 Phép toán trên đa thức và chuỗi

Các bài toán trong lập trình thi đấu, đặc biệt là những bài liên quan đến việc liệt kê một số loại nào đó, thường được giải bằng cách quy về việc tính toán một thứ gì đó trên đa thức và chuỗi lũy thừa chính quy.

Điều này bao gồm các khái niệm như nhân đa thức, nội suy, và các khái niệm phức tạp hơn như lôgarit đa thức và lũy thừa đa thức. Trong bài viết này, một cái nhìn tổng quan ngắn gọn về các phép toán như vậy và các cách tiếp cận phổ biến sẽ được trình bày.

## 5.31.1 Khái niệm cơ bản và các dữ kiện

Trong phần này, chúng ta tập trung nhiều hơn vào định nghĩa và các tính chất "trực giác" của các phép toán đa thức khác nhau. Chi tiết kỹ thuật về việc triển khai và độ phức tạp của chúng sẽ được đề cập ở các phần sau.

**Nhân đa thức**

**Định nghĩa**

Đa thức một biến là một biểu thức có dạng:

$$A(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

Các giá trị $a_0, \ldots, a_n$ là các hệ số của đa thức, thường được lấy từ một tập hợp các số hoặc cấu trúc giống số. Trong bài viết này, chúng ta giả định rằng các hệ số được lấy từ một trường, nghĩa là các phép cộng, trừ, nhân và chia được định nghĩa rõ ràng cho chúng (ngoại trừ chia cho 0) và chúng thường hoạt động tương tự như các số thực.

Ví dụ điển hình của trường như vậy là trường các dư modulo số nguyên tố $p$.

Để đơn giản, chúng ta sẽ bỏ qua thuật ngữ "một biến", vì đây là loại đa thức duy nhất được xem xét trong bài viết này. Chúng ta cũng sẽ viết $A$ thay vì $A(x)$ bất cứ khi nào có thể, điều này sẽ được hiểu từ ngữ cảnh. Giả định rằng hoặc $a_n \neq 0$ hoặc $A(x) = 0$.

**Định nghĩa**

Tích của hai đa thức được định nghĩa bằng cách mở rộng nó như một biểu thức số học:

$$A(x)B(x) = \left(\sum_{i=0}^{n} a_i x^i\right)\left(\sum_{j=0}^{m} b_j x^j\right) = \sum_{i,j} a_i b_j x^{i+j} = \sum_{k=0}^{n+m} c_k x^k = C(x).$$

Dãy $c_0, c_1, \ldots, c_{n+m}$ của các hệ số của $C(x)$ được gọi là tích chập của $a_0, \ldots, a_n$ và $b_0, \ldots, b_m$.

**Định nghĩa**

Bậc của một đa thức $A$ với $a_n \neq 0$ được định nghĩa là:

$$\deg A = n.$$

Để nhất quán, bậc của $A(x) = 0$ được định nghĩa là:

$$\deg A = -\infty.$$

Trong cách ký hiệu này, $\deg AB = \deg A + \deg B$ cho bất kỳ đa thức $A$ và $B$ nào.

Tích chập là cơ sở để giải nhiều bài toán liệt kê.

**Ví dụ**

Bạn có $n$ đối tượng loại thứ nhất và $m$ đối tượng loại thứ hai.

Các đối tượng loại thứ nhất có giá trị $a_1, \ldots, a_n$, và các đối tượng loại thứ hai có giá trị $b_1, \ldots, b_m$.

Bạn chọn một đối tượng loại thứ nhất và một đối tượng loại thứ hai. Có bao nhiêu cách để đạt được tổng giá trị $k$?

**Giải pháp**

**Ví dụ**

Bạn tung một con xúc xắc 6 mặt $n$ lần và cộng tổng các kết quả từ tất cả các lần tung. Xác suất để đạt được tổng $k$ là bao nhiêu?

**Giải pháp**

Hệ số trước $x^k$ trong đa thức $A(x)$ được ký hiệu ngắn gọn là $[x^k]A$.

### Chuỗi lũy thừa chính quy

**Định nghĩa**

Chuỗi lũy thừa chính quy là một tổng vô hạn:

$$A(x) = a_0 + a_1 x + a_2 x^2 + \ldots,$$

được xem xét bất kể tính chất hội tụ của nó.

Nói cách khác, khi chúng ta xem xét ví dụ tổng $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots = 2$, chúng ta ngụ ý rằng nó hội tụ về 2 khi số lượng các số hạng tiến tới vô cực. Tuy nhiên, chuỗi chính quy chỉ được xem xét dưới dạng các dãy tạo nên chúng.

**Định nghĩa**

Tích của chuỗi lũy thừa chính quy $A(x)$ và $B(x)$ cũng được định nghĩa bằng cách mở rộng nó như một biểu thức số học:

$$A(x)B(x) = \left(\sum_{i=0}^{\infty} a_i x^i\right)\left(\sum_{j=0}^{\infty} b_j x^j\right) = \sum_{i,j} a_i b_j x^{i+j} = \sum_{k=0}^{\infty} c_k x^k = C(x),$$

nơi các hệ số $c_0, c_1, \ldots$ được định nghĩa là các tổng hữu hạn:

$$c_k = \sum_{i=0}^{k} a_i b_{k-i}.$$

Dãy $c_0, c_1, \ldots$ cũng được gọi là tích chập của $a_0, a_1, \ldots$ và $b_0, b_1, \ldots$, mở rộng khái niệm cho các dãy vô hạn.

Do đó, đa thức có thể được coi là chuỗi lũy thừa chính quy, nhưng với số lượng hệ số hữu hạn.

Chuỗi lũy thừa chính quy đóng vai trò quan trọng trong tổ hợp liệt kê, nơi chúng được nghiên cứu như các hàm sinh cho các dãy khác nhau. Giải thích chi tiết về hàm sinh và trực giác đằng sau chúng, thật không may, nằm ngoài phạm vi của bài viết này, do đó người đọc tò mò được tham khảo thêm ở đây để biết chi tiết về ý nghĩa tổ hợp của chúng.

Tuy nhiên, chúng ta sẽ đề cập ngắn gọn rằng nếu $A(x)$ và $B(x)$ là các hàm sinh cho các dãy liệt kê một số đối tượng theo số lượng "nguyên tử" trong chúng (ví dụ: cây theo số đỉnh), thì tích $A(x)B(x)$ liệt kê các đối tượng có thể được mô tả như các cặp đối tượng thuộc loại $A$ và $B$, liệt kê theo tổng số "nguyên tử" trong cặp.

**Ví dụ**

Giả sử $A(x) = \sum_{i=0}^{\infty} 2^i x^i$ liệt kê các gói đá, mỗi viên đá được tô màu bởi một trong 2 màu (vậy, có $2^i$ gói như vậy với kích thước $i$) và $B(x) = \sum_{j=0}^{\infty} 3^j x^j$ liệt kê các gói đá, mỗi viên đá được tô màu bởi một trong 3 màu. Thì $C(x) = A(x)B(x) = \sum_{k=0}^{\infty} c_k x^k$ sẽ liệt kê các đối tượng có thể được mô tả như "hai gói đá, gói đầu tiên chỉ gồm đá loại $A$, gói thứ hai chỉ gồm đá loại $B$, với tổng số đá là $k$" cho $c_k$.

Tương tự, có một ý nghĩa trực giác cho một số hàm khác trên chuỗi lũy thừa chính quy.

### Chia đa thức dài

Tương tự như số nguyên, có thể định nghĩa phép chia dài trên đa thức.

**Định nghĩa**

Với bất kỳ đa thức $A$ và $B \neq 0$ nào, ta có thể biểu diễn $A$ như sau:

$$A = D \cdot B + R, \quad \deg R < \deg B,$$

nơi $R$ được gọi là dư của $A$ modulo $B$ và $D$ được gọi là thương.

Ký hiệu $\deg A = n$ và $\deg B = m$, cách đơn giản để thực hiện là sử dụng phép chia dài, trong đó bạn nhân $B$ với đơn thức $\frac{a_n}{b_m}x^{n-m}$ và trừ nó khỏi $A$, cho đến khi bậc của $A$ nhỏ hơn bậc của $B$. Phần còn lại của $A$ cuối cùng sẽ là dư (do đó có tên gọi này), và các đa thức mà bạn đã nhân với $B$ trong quá trình, cộng lại với nhau, tạo thành thương.

**Định nghĩa**

Nếu $A$ và $B$ có cùng dư modulo $C$, chúng được gọi là tương đương modulo $C$, được ký hiệu là:

$$A \equiv B \pmod{C}.$$

Phép chia dài đa thức hữu ích vì nó có nhiều tính chất quan trọng:

- $A$ là bội số của $B$ nếu và chỉ nếu $A \equiv 0 \pmod{B}$. - Điều này ngụ ý rằng $A \equiv B \pmod{C}$ nếu và chỉ nếu $A - B$ là bội số của $C$. - Đặc biệt, $A \equiv B \pmod{C \cdot D}$ ngụ ý $A \equiv B \pmod{C}$. - Với bất kỳ đa thức tuyến tính $x - r$, ta có $A(x) \equiv A(r) \pmod{x - r}$. - Điều này ngụ ý rằng $A$ là bội số của $x - r$ nếu và chỉ nếu $A(r) = 0$. - Với modulo là $x^k$, ta có:

$$A \equiv a_0 + a_1 x + \cdots + a_{k-1}x^{k-1} \pmod{x^k}.$$

Lưu ý rằng phép chia dài không thể được định nghĩa đúng cách cho chuỗi lũy thừa chính quy. Thay vào đó, với bất kỳ $A(x)$ nào sao cho $a_0 \neq 0$, có thể định nghĩa một chuỗi lũy thừa chính quy nghịch đảo $A^{-1}(x)$, sao cho $A(x)A^{-1}(x) = 1$. Sự thật này, đến lượt nó, có thể được sử dụng để tính kết quả của phép chia dài cho đa thức.

### 5.31.2 Triển khai cơ bản

Ở đây bạn có thể tìm thấy triển khai cơ bản của đại số đa thức.

Nó hỗ trợ tất cả các phép toán thông thường và một số phương thức hữu ích khác. Lớp chính là `poly<T>` cho các đa thức với hệ số thuộc kiểu `T`.

Tất cả các phép toán số học $+$, $-$, $*$, % và / đều được hỗ trợ, trong đó % và / đại diện cho dư và thương trong phép chia Euclid.

Ngoài ra còn có lớp `modular<m>` để thực hiện các phép toán số học trên các dư modulo một số nguyên tố $m$.

Các hàm hữu ích khác:

- `deriv()`: tính đạo hàm $P'(x)$ của $P(x)$. - `integr()`: tính tích phân không xác định $Q(x) = \int P(x)$ của $P(x)$ sao cho $Q(0) = 0$. - `inv(size_t n)`: tính $n$ hệ số đầu tiên của $P^{-1}(x)$ trong $O(n \log n)$. - `log(size_t n)`: tính $n$ hệ số đầu tiên của $\ln P(x)$ trong $O(n \log n)$. - `exp(size_t n)`: tính $n$ hệ số đầu tiên của $\exp P(x)$ trong $O(n \log n)$. - `pow(size_t k, size_t n)`: tính $n$ hệ số đầu tiên cho $P^k(x)$ trong $O(n \log nk)$. - `deg()`: trả về bậc của $P(x)$. - `lead()`: trả về hệ số của $x^{\deg P(x)}$. - `resultant(poly<T> a, poly<T> b)`: tính kết quả của $a$ và $b$ trong $O(|a| \cdot |b|)$. - `bpow(T x, size_t n)`: tính $x^n$. - `bpow(T x, size_t n, T m)`: tính $x^n \pmod{m}$. - `chirpz(T z, size_t n)`: tính $P(1), P(z), P(z^2), \ldots, P(z^{n-1})$ trong $O(n \log n)$. - `vector<T> eval(vector<T> x)`: đánh giá $P(x_1), \ldots, P(x_n)$ trong $O(n \log^2 n)$. - `poly<T> inter(vector<T> x, vector<T> y)`:

nội suy một đa thức bởi tập hợp các cặp $P(x_i) = y_i$ trong $O(n \log^2 n)$. - Và một số hàm khác, hãy tự do khám phá mã nguồn!

### 5.31.3 Số học

#### Nhân

Phép toán cốt lõi là nhân hai đa thức. Đó là, cho hai đa thức $A$ và $B$:

$$A = a_0 + a_1 x + \cdots + a_n x^n$$

$$B = b_0 + b_1 x + \cdots + b_m x^m$$

Bạn phải tính đa thức $C = A \cdot B$, được định nghĩa là:

$$\boxed{C = \sum_{i=0}^{n} \sum_{j=0}^{m} a_i b_j x^{i+j}} = c_0 + c_1 x + \cdots + c_{n+m} x^{n+m}.$$

Nó có thể được tính trong $O(n \log n)$ thông qua Biến đổi Fourier Nhanh và hầu hết các phương pháp ở đây sẽ sử dụng nó như một chương trình con.

#### Chuỗi nghịch đảo

Nếu $A(0) \neq 0$, luôn tồn tại một chuỗi lũy thừa chính quy vô hạn $A^{-1}(x) = q_0 + q_1 x + q_2 x^2 + \ldots$ sao cho $A^{-1}A = 1$. Việc tính $k$ hệ số đầu tiên của $A^{-1}$ (tức là tính nó modulo $x^k$) thường rất hữu ích. Có hai cách chính để tính nó.

**Chia để trị**

Thuật toán này được đề cập trong bài báo của Schönhage và được lấy cảm hứng từ phương pháp Graeffe. Người ta biết rằng với $B(x) = A(x)A(-x)$, ta có $B(x) = B(-x)$, nghĩa là $B(x)$ là một đa thức chẵn. Điều này có nghĩa là nó chỉ có các hệ số khác không với các số chẵn và có thể được biểu diễn là $B(x) = T(x^2)$. Do đó, ta có thể thực hiện chuyển đổi sau:

$$A^{-1}(x) \equiv \frac{1}{A(x)} \equiv \frac{A(-x)}{A(x)A(-x)} \equiv \frac{A(-x)}{T(x^2)} \pmod{x^k}$$

Lưu ý rằng $T(x)$ có thể được tính với một phép nhân duy nhất, sau đó ta chỉ quan tâm đến nửa đầu các hệ số của chuỗi nghịch đảo của nó. Điều này giảm hiệu quả vấn đề ban đầu của việc tính $A^{-1} \pmod{x^k}$ xuống việc tính $T^{-1} \pmod{x^{\lfloor k/2 \rfloor}}$.

Độ phức tạp của phương pháp này có thể được ước lượng là:

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

**Thuật toán Sieveking–Kung**

Quy trình chung được mô tả ở đây được gọi là nâng Hensel, vì nó xuất phát từ bổ đề Hensel. Chúng ta sẽ đề cập chi tiết hơn ở phần dưới, nhưng hiện tại hãy tập trung vào giải pháp đặc biệt. Phần "nâng" ở đây nghĩa là ta bắt đầu với xấp xỉ $B_0 = q_0 = a_0^{-1}$, là $A^{-1} \pmod{x}$ và sau đó nâng dần từ $\bmod x^a$ lên $\bmod x^{2a}$.

Giả sử $B_k \equiv A^{-1} \pmod{x^a}$. Xấp xỉ tiếp theo cần thỏa mãn phương trình $AB_{k+1} \equiv 1 \pmod{x^{2a}}$ và có thể được biểu diễn là $B_{k+1} = B_k + x^a C$. Từ đó ta có phương trình:

$$A(B_k + x^a C) \equiv 1 \pmod{x^{2a}}.$$

Giả sử $AB_k \equiv 1 + x^a D \pmod{x^{2a}}$, thì phương trình trên ngụ ý:

$$x^a(D + AC) \equiv 0 \pmod{x^{2a}} \implies D \equiv -AC \pmod{x^a} \implies C \equiv -B_k D \pmod{x^a}.$$

Từ đây, ta có thể thu được công thức cuối cùng, đó là:

$$x^a C \equiv -B_k x^a D \equiv B_k(1 - AB_k) \pmod{x^{2a}} \implies \boxed{B_{k+1} \equiv B_k(2 - AB_k) \pmod{x^{2a}}}$$

Do đó, bắt đầu với $B_0 \equiv a_0^{-1} \pmod{x}$, ta sẽ tính dãy $B_k$ sao cho $AB_k \equiv 1 \pmod{x^{2^k}}$ với độ phức tạp:

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

Thuật toán này có thể trông phức tạp hơn một chút so với cách đầu tiên, nhưng nó có lý do rất vững chắc và thực tế đằng sau, cũng như tiềm năng tổng quát hóa lớn nếu nhìn từ góc độ khác, điều này sẽ được giải thích thêm ở phần dưới.

**Chia Euclid**

Xem xét hai đa thức $A(x)$ và $B(x)$ có bậc $n$ và $m$. Như đã nói trước đó, bạn có thể viết lại $A(x)$ như sau:

$$A(x) = B(x)D(x) + R(x), \quad \deg R < \deg B.$$

Giả sử $n \geq m$, điều này ngụ ý rằng $\deg D = n - m$ và $n - m + 1$ hệ số cao nhất của $A$ không ảnh hưởng đến $R$. Điều này có nghĩa là bạn có thể khôi phục $D(x)$ từ $n - m + 1$ hệ số lớn nhất của $A(x)$ và $B(x)$ nếu xem nó như một hệ phương trình.

Hệ phương trình tuyến tính mà chúng ta đang nói đến có thể được viết dưới dạng sau:

$$\begin{bmatrix} a_n \\ \vdots \\ a_{m+1} \\ a_m \end{bmatrix} = \begin{bmatrix} b_m & \dots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ \dots & \dots & b_m & 0 \\ \dots & \dots & b_{m-1} & b_m \end{bmatrix} \begin{bmatrix} d_{n-m} \\ \vdots \\ d_1 \\ d_0 \end{bmatrix}$$

Từ hình dạng của nó, ta có thể kết luận rằng với việc giới thiệu các đa thức đảo ngược:

$$A^R(x) = x^n A(x^{-1}) = a_n + a_{n-1}x + \dots + a_0 x^n$$

$$B^R(x) = x^m B(x^{-1}) = b_m + b_{m-1}x + \cdots + b_0 x^m$$
$$D^R(x) = x^{n-m} D(x^{-1}) = d_{n-m} + d_{n-m-1}x + \cdots + d_0 x^{n-m}$$

hệ có thể được viết lại như sau:

$$A^R(x) \equiv B^R(x) D^R(x) \pmod{x^{n-m+1}}.$$

Từ đây bạn có thể khôi phục tất cả các hệ số của $D(x)$ một cách rõ ràng:

$$\boxed{D^R(x) \equiv A^R(x)(B^R(x))^{-1} \pmod{x^{n-m+1}}}$$

Và từ đây, đến lượt nó, bạn có thể khôi phục $R(x)$ như $R(x) = A(x) - B(x)D(x)$.

Lưu ý rằng ma trận trên là một ma trận Toeplitz tam giác và, như ta thấy ở đây, giải hệ phương trình tuyến tính với ma trận Toeplitz bất kỳ thực tế tương đương với nghịch đảo đa thức. Hơn nữa, ma trận nghịch đảo của nó cũng sẽ là ma trận Toeplitz tam giác và các phần tử của nó, theo thuật ngữ đã sử dụng ở trên, là các hệ số của $(B^R(x))^{-1} \pmod{x^{n-m+1}}$.

## 5.31.4 Tính các hàm của đa thức

### Phương pháp Newton

Hãy tổng quát hóa thuật toán Sieveking–Kung. Xem xét phương trình $F(P) = 0$ nơi $P(x)$ là một đa thức và $F(x)$ là một hàm có giá trị đa thức được định nghĩa như:

$$F(x) = \sum_{i=0}^{\infty} \alpha_i (x - \beta)^i,$$

nơi $\beta$ là một hằng số. Có thể chứng minh rằng nếu ta giới thiệu một biến chính quy mới $y$, ta có thể biểu diễn $F(x)$ như:

$$F(x) = F(y) + (x - y)F'(y) + (x - y)^2 G(x, y),$$

nơi $F'(x)$ là chuỗi lũy thừa chính quy đạo hàm được định nghĩa như:

$$F'(x) = \sum_{i=0}^{\infty} (i + 1)\alpha_{i+1}(x - \beta)^i,$$

và $G(x, y)$ là một chuỗi lũy thừa chính quy của $x$ và $y$. Với kết quả này, ta có thể tìm nghiệm một cách lặp.

Giả sử $F(Q_k) \equiv 0 \pmod{x^a}$. Ta cần tìm $Q_{k+1} \equiv Q_k + x^a C \pmod{x^{2a}}$ sao cho $F(Q_{k+1}) \equiv 0 \pmod{x^{2a}}$.

Thay $x = Q_{k+1}$ và $y = Q_k$ vào công thức trên, ta được:

$$F(Q_{k+1}) \equiv F(Q_k) + (Q_{k+1} - Q_k)F'(Q_k) + (Q_{k+1} - Q_k)^2 G(x, y) \pmod{x}^{2a}.$$

Vì $Q_{k+1} - Q_k \equiv 0 \pmod{x^a}$, nên cũng có $(Q_{k+1} - Q_k)^2 \equiv 0 \pmod{x^{2a}}$, do đó:

$$0 \equiv F(Q_{k+1}) \equiv F(Q_k) + (Q_{k+1} - Q_k)F'(Q_k) \pmod{x^{2a}}.$$

Công thức cuối cùng cho ta giá trị của $Q_{k+1}$:

$$\boxed{Q_{k+1} = Q_k - \frac{F(Q_k)}{F'(Q_k)} \pmod{x^{2a}}}$$

Do đó, biết cách nghịch đảo đa thức và cách tính $F(Q_k)$, ta có thể tìm $n$ hệ số của $P$ với độ phức tạp:

$$T(n) = T(n/2) + f(n),$$

nơi $f(n)$ là thời gian cần để tính $F(Q_k)$ và $F'(Q_k)^{-1}$, thường là $O(n \log n)$.

Quy tắc lặp trên được biết trong phân tích số học là phương pháp Newton.

### Bổ đề Hensel

Như đã đề cập trước đó, chính thức và tổng quát, kết quả này được biết đến như bổ đề Hensel và nó thực sự có thể được sử dụng trong một ý nghĩa rộng hơn khi ta làm việc với một chuỗi các vành lồng nhau. Trong trường hợp đặc biệt này, ta đã làm việc với một chuỗi các dư đa thức modulo $x$, $x^2$, $x^3$ và tiếp tục như vậy.

Một ví dụ khác nơi nâng Hensel có thể hữu ích là các số p-adic, nơi ta thực sự làm việc với chuỗi các dư số nguyên modulo $p$, $p^2$, $p^3$ và tiếp tục như vậy. Ví dụ, phương pháp Newton có thể được sử dụng để tìm tất cả các số tự đẳng hình (số mà khi bình phương thì kết thúc bằng chính nó) với một cơ số cho trước. Bài toán này được để lại như một bài tập cho người đọc. Bạn có thể xem xét bài toán này để kiểm tra xem giải pháp của bạn có hoạt động cho các số cơ số 10 hay không.

### Lôgarit

Đối với hàm $\ln P(x)$, ta biết rằng:

$$\boxed{(\ln P(x))' = \frac{P'(x)}{P(x)}}$$

Do đó, ta có thể tính $n$ hệ số của $\ln P(x)$ trong $O(n \log n)$.

### Chuỗi nghịch đảo

Hóa ra, ta có thể lấy công thức cho $A^{-1}$ bằng phương pháp Newton. Để làm điều này, ta lấy phương trình $A = Q^{-1}$, do đó:

$$F(Q) = Q^{-1} - A$$

$$F'(Q) = -Q^{-2}$$

$$\boxed{Q_{k+1} \equiv Q_k(2 - AQ_k) \pmod{x^{2^{k+1}}}}$$

### Lũy thừa

Hãy học cách tính $e^{P(x)} = Q(x)$. Nó cần thỏa mãn $\ln Q = P$, do đó:

$$F(Q) = \ln Q - P$$

$$F'(Q) = Q^{-1}$$

$$\boxed{Q_{k+1} \equiv Q_k(1 + P - \ln Q_k) \pmod{x^{2^{k+1}}}}$$

### Lũy thừa thứ $k$

Bây giờ ta cần tính $P^k(x) = Q$. Điều này có thể được thực hiện qua công thức sau:

$$Q = \exp\left[k \ln P(x)\right]$$

Tuy nhiên, lưu ý rằng bạn chỉ có thể tính lôgarit và lũy thừa chính xác nếu bạn có thể tìm một $Q_0$ ban đầu.

Để tìm nó, bạn nên tính lôgarit hoặc lũy thừa của hệ số hằng của đa thức.

Nhưng cách hợp lý duy nhất để làm điều đó là nếu $P(0) = 1$ cho $Q = \ln P$ để $Q(0) = 0$ và nếu $P(0) = 0$ cho $Q = e^P$ để $Q(0) = 1$.

Do đó, bạn chỉ có thể sử dụng công thức trên nếu $P(0) = 1$. Nếu không, nếu $P(x) = \alpha x^t T(x)$ với $T(0) = 1$, bạn có thể viết rằng:

$$\boxed{P^k(x) = \alpha^k x^{kt} \exp[k \ln T(x)]}$$

Lưu ý rằng bạn cũng có thể tính một số căn bậc $k$ của đa thức nếu bạn có thể tính $\sqrt[k]{\alpha}$, ví dụ với $\alpha = 1$.

### 5.31.5 Đánh giá và Nội suy

#### Biến đổi Chirp-z

Đối với trường hợp đặc biệt khi bạn cần đánh giá một đa thức tại các điểm $x_r = z^{2r}$, bạn có thể làm như sau:

$$A(z^{2r}) = \sum_{k=0}^{n} a_k z^{2kr}$$

Hãy thay $2kr = r^2 + k^2 - (r-k)^2$. Khi đó tổng này được viết lại như:

$$\boxed{A(z^{2r}) = z^{r^2} \sum_{k=0}^{n} (a_k z^{k^2}) z^{-(r-k)^2}}$$

Điều này, ngoại trừ yếu tố $z^{r^2}$, bằng tích chập của các dãy $u_k = a_k z^{k^2}$ và $v_k = z^{-k^2}$.

Lưu ý rằng $u_k$ có chỉ số từ 0 đến $n$ ở đây và $v_k$ có chỉ số từ $-n$ đến $m$ nơi $m$ là lũy thừa tối đa của $z$ mà bạn cần.

Bây giờ nếu bạn cần đánh giá một đa thức tại các điểm $x_r = z^{2r+1}$, bạn có thể quy nó về bài toán trước bằng phép biến đổi $a_k \to a_k z^k$.

Điều này cho ta một thuật toán $O(n \log n)$ khi bạn cần tính các giá trị ở các lũy thừa của $z$, do đó bạn có thể tính DFT cho các số không phải lũy thừa của hai.

Một quan sát khác là $kr = \binom{k+r}{2} - \binom{k}{2} - \binom{r}{2}$. Khi đó ta có:

$$\boxed{A(z^r) = z^{-\binom{r}{2}} \sum_{k=0}^{n} \left(a_k z^{-\binom{k}{2}}\right) z^{\binom{k+r}{2}}}$$

Hệ số của $x^{n+r}$ của tích các đa thức $A_0(x) = \sum_{k=0}^{n} a_{n-k} z^{-\binom{n-k}{2}} x^k$ và $A_1(x) = \sum_{k \geq 0} z^{\binom{k}{2}} x^k$ bằng $z^{\binom{r}{2}} A(z^r)$. Bạn có thể sử dụng công thức $z^{\binom{k+1}{2}} = z^{\binom{k}{2}+k}$ để tính các hệ số của $A_0(x)$ và $A_1(x)$.

#### Đánh giá đa điểm

Giả sử bạn cần tính $A(x_1), \ldots, A(x_n)$. Như đã đề cập trước đó, $A(x) \equiv A(x_i) \pmod{x - x_i}$. Do đó, bạn có thể làm như sau:

- Tính một cây đoạn sao cho trong đoạn $[l, r)$ là tích $P_{l,r}(x) = (x - x_l)(x - x_{l+1}) \ldots (x - x_{r-1})$. - Bắt đầu với $l = 1$ và $r = n + 1$ tại nút gốc. Gọi $m = \lfloor (l + r)/2 \rfloor$. Di chuyển xuống $[l, m)$ với đa thức $A(x) \pmod{P_{l,m}(x)}$. - Điều này sẽ tính đệ quy $A(x_l), \ldots, A(x_{m-1})$, giờ làm tương tự cho $[m, r)$ với $A(x) \pmod{P_{m,r}(x)}$. - Ghép kết quả từ hai lần gọi đệ quy đầu tiên và thứ hai rồi trả về chúng.

Toàn bộ quy trình sẽ chạy trong $O(n \log^2 n)$.

### Nội suy

Có một công thức trực tiếp của Lagrange để nội suy một đa thức, cho tập hợp các cặp $(x_i, y_i)$:

$$\boxed{A(x) = \sum_{i=1}^{n} y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}}$$

Tính trực tiếp nó là một việc khó khăn nhưng hóa ra, ta có thể tính nó trong $O(n \log^2 n)$ với cách tiếp cận chia để trị:

Xem xét $P(x) = (x - x_1) \dots (x - x_n)$. Để biết các hệ số của mẫu số trong $A(x)$, ta nên tính các tích như:

$$P_i = \prod_{j \neq i} (x_i - x_j)$$

Nhưng nếu bạn xem xét đạo hàm $P'(x)$, bạn sẽ thấy rằng $P'(x_i) = P_i$. Do đó, bạn có thể tính các $P_i$ qua đánh giá trong $O(n \log^2 n)$.

Bây giờ xem xét thuật toán đệ quy được thực hiện trên cùng cây đoạn như trong đánh giá đa điểm. Nó bắt đầu ở các lá với giá trị $\dfrac{y_i}{P_i}$ ở mỗi lá.

Khi quay lại từ đệ quy, ta nên gộp kết quả từ các đỉnh trái và phải như $A_{l,r} = A_{l,m} P_{m,r} + P_{l,m} A_{m,r}$.

Theo cách này, khi bạn quay lại gốc, bạn sẽ có chính xác $A(x)$ ở đó. Toàn bộ quy trình cũng hoạt động trong $O(n \log^2 n)$.

## 5.31.6 UCLN và Kết quả

Giả sử bạn được cho các đa thức $A(x) = a_0 + a_1 x + \dots + a_n x^n$ và $B(x) = b_0 + b_1 x + \dots + b_m x^m$.

Gọi $\lambda_0, \dots, \lambda_n$ là các nghiệm của $A(x)$ và $\mu_0, \dots, \mu_m$ là các nghiệm của $B(x)$, tính cả bội của chúng.

Bạn muốn biết liệu $A(x)$ và $B(x)$ có nghiệm chung nào không. Có hai cách liên kết để làm điều đó.

### Thuật toán Euclid

Chúng ta đã có một bài viết về nó. Với một miền bất kỳ, bạn có thể viết thuật toán Euclid dễ dàng như sau:

```
template<typename T>
T gcd(const T &a, const T &b) {
    return b == T(0) ? a : gcd(b, a % b);
}
```

Có thể chứng minh rằng với các đa thức $A(x)$ và $B(x)$, nó sẽ hoạt động trong $O(nm)$.

### Kết quả

Hãy tính tích $A(\mu_0)\cdots A(\mu_m)$. Nó sẽ bằng không nếu và chỉ nếu một $\mu_i$ nào đó là nghiệm của $A(x)$.

Để đối xứng, ta cũng có thể nhân nó với $b_m^n$ và viết lại toàn bộ tích dưới dạng sau:

$$\boxed{\mathcal{R}(A,B) = b_m^n \prod_{j=0}^{m} A(\mu_j) = b_m^n a_m^n \prod_{i=0}^{n}\prod_{j=0}^{m}(\mu_j - \lambda_i) = (-1)^{mn} a_n^m \prod_{i=0}^{n} B(\lambda_i)}$$

Giá trị được định nghĩa ở trên được gọi là kết quả của các đa thức $A(x)$ và $B(x)$. Từ định nghĩa, bạn có thể tìm thấy các tính chất sau:

- $\mathcal{R}(A,B) = (-1)^{nm}\mathcal{R}(B,A)$. - $\mathcal{R}(A,B) = a_n^m b_m^n$ khi $n=0$ hoặc $m=0$. - Nếu $b_m = 1$ thì $\mathcal{R}(A - CB, B) = \mathcal{R}(A,B)$ với đa thức $C(x)$ bất kỳ và $n, m \geq 1$. - Từ đây suy ra $\mathcal{R}(A,B) = b_m^{\deg(A)-\deg(A-CB)}\mathcal{R}(A-CB,B)$ với $A(x)$, $B(x)$, $C(x)$ bất kỳ.

Kỳ diệu thay, điều này có nghĩa là kết quả của hai đa thức thực sự luôn thuộc cùng vành với các hệ số của chúng!

Các tính chất này cũng cho phép ta tính kết quả cùng với thuật toán Euclid, hoạt động trong $O(nm)$.

```
template<typename T>
T resultant(poly<T> a, poly<T> b) {
    if(b.is_zero()) {
        return 0;
    } else if(b.deg() == 0) {
        return bpow(b.lead(), a.deg());
    } else {
        int pw = a.deg();
        a %= b;
        pw -= a.deg();
        base mul = bpow(b.lead(), pw) * base((b.deg() & a.deg() & 1) ? -1 : 1);
        base ans = resultant(b, a);
        return ans * mul;
    }
}
```

### Thuật toán Half-GCD

Có một cách để tính UCLN và kết quả trong $O(n\log^2 n)$.

Quy trình để làm điều đó triển khai một phép biến đổi tuyến tính $2\times 2$ ánh xạ một cặp đa thức $a(x)$, $b(x)$ thành một cặp khác $c(x), d(x)$ sao cho $\deg d(x) \leq \frac{\deg a(x)}{2}$. Nếu bạn

cẩn thận, bạn có thể tính Half-GCD của bất kỳ cặp đa thức nào với tối đa 2 lần gọi đệ quy đến các đa thức nhỏ hơn ít nhất 2 lần.

Chi tiết cụ thể của thuật toán khá phức tạp để giải thích, tuy nhiên bạn có thể tìm thấy triển khai của nó trong thư viện, dưới dạng hàm `half_gcd`.

Sau khi Half-GCD được triển khai, bạn có thể áp dụng nó lặp đi lặp lại cho các đa thức cho đến khi bạn giảm xuống cặp $\gcd(a, b)$ và 0.

## 5.32 Tính Lũy Thừa Nhị Phân Bằng Phân Tích Thừa Số

Xét bài toán tính $ax^y \pmod{2^d}$, với các số nguyên $a$, $x$, $y$ và $d \geq 3$, trong đó $x$ là số lẻ.

Thuật toán dưới đây cho phép giải bài toán này với $O(d)$ phép cộng và các phép toán nhị phân, cùng một phép nhân duy nhất với $y$.

Do cấu trúc của nhóm nhân modulo $2^d$, bất kỳ số nào $x$ thỏa mãn $x \equiv 1 \pmod 4$ đều có thể được biểu diễn dưới dạng

$$x \equiv b^{L(x)} \pmod{2^d},$$

trong đó $b \equiv 5 \pmod 8$. Không mất tính tổng quát, ta giả sử rằng $x \equiv 1 \pmod 4$, vì ta có thể rút gọn trường hợp $x \equiv 3 \pmod 4$ về $x \equiv 1 \pmod 4$ bằng cách thay thế $x \mapsto -x$ và $a \mapsto (-1)^y a$. Trong cách biểu diễn này, $ax^y$ được biểu diễn là

$$ax^y \equiv ab^{yL(x)} \pmod{2^d}.$$

Ý tưởng cốt lõi của thuật toán là đơn giản hóa việc tính toán $L(x)$ và $b^{yL(x)}$ bằng cách sử dụng thực tế rằng ta đang làm việc modulo $2^d$. Vì những lý do sẽ rõ ràng sau này, ta sẽ làm việc với $4L(x)$ thay vì $L(x)$, nhưng được lấy modulo $2^d$ thay vì $2^{d-2}$.

Trong bài viết này, chúng ta sẽ đề cập đến việc triển khai cho các số nguyên 32 bit. Gọi

- `mbin_log_32(r, x)` là hàm tính $r + 4L(x) \pmod{2^d}$; - `mbin_exp_32(r, x)` là hàm tính $rb^{\frac{x}{4}} \pmod{2^d}$; - `mbin_power_odd_32(a, x, y)` là hàm tính $ax^y \pmod{2^d}$.

Khi đó, `mbin_power_odd_32` được triển khai như sau:

```
uint32_t mbin_power_odd_32(uint32_t rem, uint32_t base, uint32_t exp) {
    if (base & 2) {
        /* số chia được coi là âm */
        base = -base;
        /* kiểm tra xem kết quả có nên là âm không */
        if (exp & 1) {
            rem = -rem;
        }
```

```
    }
    return (mbin_exp_32(rem, mbin_log_32(0, base) * exp));
}
```

### 5.32.1 Tính $4L(x)$ từ $x$

Cho $x$ là một số lẻ sao cho $x \equiv 1 \pmod 4$. Nó có thể được biểu diễn như sau

$$x \equiv (2^{a_1} + 1) \dots (2^{a_k} + 1) \pmod{2^d},$$

trong đó $1 < a_1 < \dots < a_k < d$. Ở đây, $L(\cdot)$ được xác định rõ ràng cho mỗi thừa số, vì chúng bằng 1 modulo 4. Do đó,

$$4L(x) \equiv 4L(2^{a_1} + 1) + \dots + 4L(2^{a_k} + 1) \pmod{2^d}.$$

Vậy, nếu ta tính trước $t_k = 4L(2^n + 1)$ cho tất cả $1 < k < d$, ta sẽ có thể tính $4L(x)$ cho bất kỳ số nào $x$.

Đối với số nguyên 32-bit, ta có thể sử dụng bảng sau:

```
const uint32_t mbin_log_32_table[32] = {
    0x00000000, 0x00000000, 0xd3cfd984, 0x9ee62e18,
    0xe83d9070, 0xb59e81e0, 0xa17407c0, 0xce601f80,
    0xf4807f00, 0xe701fe00, 0xbe07fc00, 0xfc1ff800,
    0xf87ff000, 0xf1ffe000, 0xe7ffc000, 0xdfff8000,
    0xffff0000, 0xfffe0000, 0xfffc0000, 0xfff80000,
    0xfff00000, 0xffe00000, 0xffc00000, 0xff800000,
    0xff000000, 0xfe000000, 0xfc000000, 0xf8000000,
    0xf0000000, 0xe0000000, 0xc0000000, 0x80000000,
};
```

Trong thực tế, ta sử dụng một cách tiếp cận hơi khác so với mô tả ở trên. Thay vì tìm phân tích thừa số cho $x$, ta sẽ liên tục nhân $x$ với $2^n + 1$ cho đến khi biến nó thành 1 modulo $2^d$. Bằng cách này, ta sẽ tìm được biểu diễn của $x^{-1}$, tức là

$$x(2^{a_1} + 1) \dots (2^{a_k} + 1) \equiv 1 \pmod{2^d}.$$

Để làm điều này, ta lặp qua $n$ sao cho $1 < n < d$. Nếu giá trị hiện tại của $x$ có bit thứ $n$ được bật, ta nhân $x$ với $2^n + 1$, điều này được thực hiện dễ dàng trong C++ bằng `x = x + (x « n)`. Điều này sẽ không thay đổi các bit thấp hơn $n$, nhưng sẽ chuyển bit thứ $n$ thành số không, vì $x$ là số lẻ.

Với tất cả những điều này trong tâm trí, hàm `mbin_log_32(r, x)` được triển khai như sau:

```
uint32_t mbin_log_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n < 32; n++) {
        if (x & (1 << n)) {
            x = x + (x << n);
            r -= mbin_log_32_table[n];
        }
    }

    return r;
}
```

Lưu ý rằng $4L(x) = -4L(x^{-1})$, nên thay vì cộng $4L(2^n + 1)$, ta trừ nó khỏi $r$, giá trị ban đầu bằng 0.

### 5.32.2 Tính $x$ từ $4L(x)$

Lưu ý rằng với $k \geq 1$, ta có

$$(a2^k + 1)^2 = a^2 2^{2k} + a2^{k+1} + 1 = b2^{k+1} + 1,$$

từ đó (bằng cách bình phương lặp đi lặp lại) ta có thể suy ra rằng

$$(2^a + 1)^{2^b} \equiv 1 \pmod{2^{a+b}}.$$

Áp dụng kết quả này cho $a = 2^n + 1$ và $b = d - k$, ta suy ra rằng bậc nhân của $2^n + 1$ là một ước của $2^{d-n}$.

Điều này, đến lượt nó, có nghĩa là $L(2^n + 1)$ phải chia hết cho $2^n$, vì bậc của $b$ là $2^{d-2}$ và bậc của $b^y$ là $2^{d-2-v}$, trong đó $2^v$ là lũy thừa cao nhất của 2 chia hết cho $y$, nên ta cần

$$2^{d-k} \equiv 0 \pmod{2^{d-2-v}},$$

do đó $v$ phải lớn hơn hoặc bằng $k - 2$. Điều này hơi phức tạp, và để giảm bớt vấn đề này, ta đã nói từ đầu rằng ta nhân $L(x)$ với 4. Bây giờ nếu ta biết $4L(x)$, ta có thể phân tích nó một cách duy nhất thành tổng của $4L(2^n + 1)$ bằng cách kiểm tra tuần tự các bit trong $4L(x)$. Nếu bit thứ $n$ được đặt thành 1, ta sẽ nhân kết quả với $2^n + 1$ và giảm giá trị hiện tại của $4L(x)$ đi $4L(2^n + 1)$.

Do đó, `mbin_exp_32` được triển khai như sau:

```
uint32_t mbin_exp_32(uint32_t r, uint32_t x) {
    uint8_t n;
```

```
    for (n = 2; n < 32; n++) {
        if (x & (1 << n)) {
            r = r + (r << n);
            x -= mbin_log_32_table[n];
        }
    }

    return r;
}
```

### 5.32.3 Tối ưu hóa thêm

Có thể giảm một nửa số lần lặp nếu bạn lưu ý rằng $4L(2^{d-1}+1) = 2^{d-1}$ và rằng với $2k \geq d$, ta có

$$(2^n+1)^2 \equiv 2^{2n} + 2^{n+1} + 1 \equiv 2^{n+1} + 1 \pmod{2^d},$$

điều này cho phép suy ra rằng $4L(2^n+1) = 2^n$ với $2n \geq d$. Vậy, bạn có thể đơn giản hóa thuật toán bằng cách chỉ lặp đến $\frac{d}{2}$ và sau đó sử dụng thực tế trên để tính phần còn lại bằng các phép toán bit:

```
uint32_t mbin_log_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n != 16; n++) {
        if (x & (1 << n)) {
            x = x + (x << n);
            r -= mbin_log_32_table[n];
        }
    }

    r -= (x & 0xFFFF0000);

    return r;
}

uint32_t mbin_exp_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n != 16; n++) {
        if (x & (1 << n)) {
```

```
            r = r + (r << n);
            x -= mbin_log_32_table[n];
        }
    }

    r *= 1 - (x & 0xFFFF0000);

    return r;
}
```

### 5.32.4 Tính bảng logarit

Để tính bảng logarit, ta có thể sửa đổi thuật toán Pohlig–Hellman cho trường hợp modulo là lũy thừa của 2.

Nhiệm vụ chính của chúng ta ở đây là tính $x$ sao cho $g^x \equiv y \pmod{2^d}$, trong đó $g = 5$ và $y$ là một số dạng $2^n + 1$.

Bình phương cả hai vế $k$ lần, ta được

$$g^{2^k x} \equiv y^{2^k} \pmod{2^d}.$$

Lưu ý rằng bậc của $g$ không lớn hơn $2^d$ (thực tế là không lớn hơn $2^{d-2}$, nhưng ta sẽ giữ ở mức $2^d$ cho tiện), do đó sử dụng $k = d - 1$, ta sẽ có hoặc $g^1$ hoặc $g^0$ ở vế trái, điều này cho phép xác định bit nhỏ nhất của $x$ bằng cách so sánh $y^{2^k}$ với $g$. Bây giờ giả sử rằng $x = x_0 + 2^k x_1$, trong đó $x_0$ là phần đã biết và $x_1$ chưa biết. Khi đó

$$g^{x_0 + 2^k x_1} \equiv y \pmod{2^d}.$$

Nhân cả hai vế với $g^{-x_0}$, ta được

$$g^{2^k x_1} \equiv (g^{-x_0} y) \pmod{2^d}.$$

Bây giờ, bình phương cả hai vế $d - k - 1$ lần, ta có thể lấy được bit tiếp theo của $x$, cuối cùng khôi phục lại tất cả các bit của nó.

# Chương 6

# Quy hoạch động

## 6.1 Giới thiệu về Quy hoạch Động

Bản chất của quy hoạch động là tránh việc tính toán lặp lại. Thường thì các bài toán quy hoạch động có thể được giải một cách tự nhiên bằng đệ quy. Trong những trường hợp như vậy, cách dễ nhất là viết giải pháp đệ quy, sau đó lưu các trạng thái lặp lại vào một bảng tra cứu. Quá trình này được gọi là quy hoạch động từ trên xuống với ghi nhớ (memoization). Từ này được đọc là "memoization" (như việc ghi vào sổ tay) chứ không phải "memorization".

Một trong những ví dụ cơ bản và kinh điển nhất của quá trình này là dãy Fibonacci. Công thức đệ quy của nó là $f(n) = f(n-1) + f(n-2)$ với $n \geq 2$ và $f(0) = 0$ và $f(1) = 1$. Trong C++, điều này sẽ được biểu diễn như sau:

```
int f(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return f(n - 1) + f(n - 2);
}
```

Thời gian chạy của hàm đệ quy này là cấp số mũ - khoảng $O(2^n)$, vì một lời gọi hàm ( $f(n)$) dẫn đến 2 lời gọi hàm có kích thước tương tự ( $f(n-1)$ và $f(n-2)$).

### 6.1.1 Tăng tốc Fibonacci bằng Quy hoạch Động (Ghi nhớ)

Hàm đệ quy hiện tại của chúng ta giải quyết dãy Fibonacci trong thời gian cấp số mũ. Điều này có nghĩa là chúng ta chỉ có thể xử lý các giá trị đầu vào nhỏ trước khi bài toán trở nên quá khó. Chẳng hạn, $f(29)$ dẫn đến hơn 1 triệu lời gọi hàm!

Để tăng tốc độ, ta nhận ra rằng số lượng bài toán con chỉ là $O(n)$. Nghĩa là, để tính $f(n)$, ta chỉ cần biết $f(n-1), f(n-2), \ldots, f(0)$. Do đó, thay vì tính toán lại các bài toán

con này, ta giải chúng một lần rồi lưu kết quả vào bảng tra cứu. Các lời gọi sau sẽ sử dụng bảng tra cứu này và trả về kết quả ngay lập tức, qua đó loại bỏ công việc cấp số mũ!

Mỗi lời gọi đệ quy sẽ kiểm tra bảng tra cứu để xem giá trị đã được tính chưa. Việc này được thực hiện trong thời gian $O(1)$. Nếu đã tính trước đó, trả về kết quả; nếu không, ta tính toán hàm như bình thường. Tổng thời gian chạy là $O(n)$. Đây là một cải tiến vượt bậc so với thuật toán thời gian cấp số mũ trước đó!

```
const int MAXN = 100;
bool found[MAXN];
int memo[MAXN];

int f(int n) {
    if (found[n]) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    found[n] = true;
    return memo[n] = f(n - 1) + f(n - 2);
}
```

Với hàm đệ quy ghi nhớ mới của chúng ta, $f(29)$, vốn trước đây dẫn đến hơn 1 triệu lời gọi, giờ chỉ còn 57 lời gọi, ít hơn gần 20.000 lần! Thật trớ trêu, giờ đây ta bị giới hạn bởi kiểu dữ liệu. $f(46)$ là số Fibonacci cuối cùng có thể chứa vừa trong một số nguyên 32-bit có dấu.

Thông thường, ta cố gắng lưu trạng thái trong các mảng, nếu có thể, vì thời gian tra cứu là $O(1)$ với chi phí tối thiểu. Tuy nhiên, một cách tổng quát hơn, ta có thể lưu trạng thái theo bất kỳ cách nào ta muốn. Các ví dụ khác bao gồm cây tìm kiếm nhị phân (map trong C++) hoặc bảng băm (unordered_map trong C++).

Một ví dụ có thể là:

```
unordered_map<int, int> memo;
int f(int n) {
    if (memo.count(n)) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    return memo[n] = f(n - 1) + f(n - 2);
}
```

Hoặc tương tự:

```
map<int, int> memo;
int f(int n) {
    if (memo.count(n)) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    return memo[n] = f(n - 1) + f(n - 2);
}
```

Cả hai cách này hầu như luôn chậm hơn phiên bản dựa trên mảng cho một hàm đệ quy ghi nhớ thông thường. Những cách lưu trạng thái thay thế này chủ yếu hữu ích khi lưu các vector hoặc chuỗi như một phần của không gian trạng thái.

Cách phân tích thời gian chạy của một hàm đệ quy ghi nhớ theo cách đơn giản là:

$$\text{công việc cho mỗi bài toán con} \times \text{số lượng bài toán con}$$

Sử dụng cây tìm kiếm nhị phân (map trong C++) để lưu trạng thái sẽ dẫn đến thời gian $O(n \log n)$, vì mỗi lần tra cứu và chèn sẽ mất $O(\log n)$ công việc và với $O(n)$ bài toán con duy nhất, ta có tổng thời gian $O(n \log n)$.

Cách tiếp cận này được gọi là từ trên xuống, vì ta có thể gọi hàm với một giá trị truy vấn và việc tính toán bắt đầu từ trên (giá trị được truy vấn) xuống dưới (các trường hợp cơ sở của đệ quy), và tạo các lối tắt qua ghi nhớ trên đường đi.

### 6.1.2 Quy hoạch Động Từ Dưới Lên

Cho đến nay, bạn chỉ thấy quy hoạch động từ trên xuống với ghi nhớ. Tuy nhiên, ta cũng có thể giải các bài toán bằng quy hoạch động từ dưới lên. Từ dưới lên là hoàn toàn ngược lại với từ trên xuống, bạn bắt đầu từ dưới (các trường hợp cơ sở của đệ quy) và mở rộng lên các giá trị lớn hơn.

Để tạo cách tiếp cận từ dưới lên cho số Fibonacci, ta khởi tạo các trường hợp cơ sở trong một mảng. Sau đó, ta chỉ cần sử dụng định nghĩa đệ quy trên mảng:

```
const int MAXN = 100;
int fib[MAXN];

int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++) fib[i] = fib[i - 1] + fib[i - 2];

    return fib[n];
}
```

Tất nhiên, như đã viết, điều này hơi ngớ ngẩn vì hai lý do: Thứ nhất, ta làm công việc lặp lại nếu gọi hàm nhiều lần. Thứ hai, ta chỉ cần sử dụng hai giá trị trước đó để tính phần tử hiện tại. Do đó, ta có thể giảm bộ nhớ từ $O(n)$ xuống $O(1)$.

Một ví dụ về giải pháp quy hoạch động từ dưới lên cho Fibonacci sử dụng bộ nhớ $O(1)$ có thể là:

```
const int MAX_SAVE = 3;
int fib[MAX_SAVE];

int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++)
        fib[i % MAX_SAVE] = fib[(i - 1) % MAX_SAVE] + fib[(i - 2) % MAX_SAVE];

    return fib[n % MAX_SAVE];
}
```

Lưu ý rằng ta đã thay đổi hằng số từ `MAXN` thành `MAX_SAVE`. Điều này là vì tổng số phần tử ta cần truy cập chỉ là 3. Nó không còn tỷ lệ với kích thước đầu vào và, theo định nghĩa, là bộ nhớ $O(1)$. Ngoài ra, ta sử dụng một mẹo phổ biến (dùng toán tử modulo) để chỉ duy trì các giá trị cần thiết.

Đó là tất cả. Đó là những điều cơ bản của quy hoạch động: Đừng lặp lại công việc bạn đã làm trước đó.

Một trong những mẹo để giỏi hơn trong quy hoạch động là nghiên cứu một số ví dụ kinh điển.

### 6.1.3 Các Bài Toán Quy hoạch Động Kinh Điển

| **Tên** | **Mô tả/Ví dụ** |
|---|---|
| Ba lô 0-1 | Cho $W$, $N$, và $N$ vật phẩm với trọng lượng $w_i$ và giá trị $v_i$, giá trị tối đa $\sum_{i=1}^{k} v_i$ cho mỗi tập con các vật phẩm có kích thước $k$ ( $1 \leq k \leq N$) trong khi đảm bảo $\sum_{i=1}^{k} w_i \leq W$ là bao nhiêu? |
| Tổng tập con | Cho $N$ số nguyên và $T$, xác định xem có tồn tại một tập con của tập đã cho mà tổng các phần tử bằng $T$ không. |
| Dãy con tăng dài nhất (LIS) | Bạn được cho một mảng chứa $N$ số nguyên. Nhiệm vụ của bạn là xác định dãy con tăng dài nhất trong mảng, tức là một dãy con mà mỗi phần tử lớn hơn phần tử trước đó. |
| Đếm đường đi trong mảng 2D | Cho $N$ và $M$, đếm tất cả các đường đi khác nhau từ $(1, 1)$ đến $(N, M)$, trong đó mỗi bước là từ $(i, j)$ đến $(i + 1, j)$ hoặc $(i, j + 1)$. |
| Dãy con chung dài nhất | Bạn được cho hai chuỗi $s$ và $t$. Tìm độ dài của chuỗi dài nhất là dãy con của cả $s$ và $t$. |
| Dường đi dài nhất trong đồ thị có hướng không vòng (DAG) | Tìm đường đi dài nhất trong đồ thị có hướng không vòng (DAG). |
| Dãy con đối xứng dài nhất | Tìm dãy con đối xứng dài nhất (LPS) của một chuỗi đã cho. |
| Cắt thanh | Cho một thanh có độ dài $n$ đơn vị, cho một mảng số nguyên `cuts` trong đó `cuts[i]` biểu thị vị trí bạn nên thực hiện một lần cắt. Chi phí của một lần cắt là độ dài của thanh cần cắt. Tổng chi phí tối thiểu của các lần cắt là bao nhiêu? |
| Khoảng cách chỉnh sửa | Khoảng cách chỉnh sửa giữa hai chuỗi là số lượng thao tác tối thiểu cần thiết để biến chuỗi này thành chuỗi kia. Các thao tác bao gồm ["Thêm", "Xóa", "Thay thế"]. |

### 6.1.4 Các Chủ Đề Liên Quan

- Quy hoạch động với mặt nạ bit
- Quy hoạch động trên chữ số

- Quy hoạch động trên cây

Tất nhiên, mẹo quan trọng nhất là thực hành.

## 6.2 Bài toán Cái Túi (Knapsack Problem)

### 6.2.1 Kiến thức cần thiết: Nhập môn Quy hoạch động

### 6.2.2 Giới thiệu

Xét ví dụ sau: [USACO07 Dec] Charm Bracelet

Có $n$ vật phẩm khác nhau và một cái túi có sức chứa $W$. Mỗi vật phẩm có 2 thuộc tính, trọng lượng ($w_i$) và giá trị ($v_i$). Bạn phải chọn một tập hợp con các vật phẩm để cho vào cái túi sao cho tổng trọng lượng không vượt quá sức chứa $W$ và tổng giá trị là lớn nhất.

Trong ví dụ trên, mỗi đối tượng chỉ có hai trạng thái có thể (lấy hoặc không lấy), tương ứng với các giá trị nhị phân 0 và 1. Vì vậy, loại bài toán này được gọi là "bài toán cái túi 0-1".

## 6.3 Bài toán Cái Túi 0-1

### 6.3.1 Giải thích

Trong ví dụ trên, đầu vào của bài toán là: trọng lượng của vật phẩm thứ $i$ ($w_i$), giá trị của vật phẩm thứ $i$ ($v_i$) và tổng sức chứa của cái túi ($W$).

Gọi $f_{i,j}$ là trạng thái quy hoạch động lưu trữ tổng giá trị lớn nhất mà cái túi có thể chứa với sức chứa $j$, khi chỉ xét đến $i$ vật phẩm đầu tiên.

Giả sử rằng tất cả các trạng thái của $i-1$ vật phẩm đầu tiên đã được xử lý, những lựa chọn nào cho vật phẩm thứ $i$?

- Khi nó không được cho vào cái túi, sức chứa còn lại không thay đổi và tổng giá trị không thay đổi. Do đó, giá trị lớn nhất trong trường hợp này là $f_{i-1,j}$.
- Khi nó được cho vào cái túi, sức chứa còn lại giảm đi $w_i$ và tổng giá trị tăng lên $v_i$, vì vậy giá trị lớn nhất trong trường hợp này là $f_{i-1,j-w_i} + v_i$.

Từ đó, ta có thể suy ra phương trình chuyển trạng thái quy hoạch động:

$$f_{i,j} = \max(f_{i-1,j}, f_{i-1,j-w_i} + v_i)$$

Hơn nữa, vì $f_i$ chỉ phụ thuộc vào $f_{i-1}$, ta có thể loại bỏ chiều thứ nhất. Ta thu được quy tắc chuyển trạng thái:

$$f_j \leftarrow \max(f_j, f_{j-w_i} + v_i)$$

cần được thực hiện theo thứ tự giảm dần của $j$ (để $f_{j-w_i}$ ngầm định tương ứng với $f_{i-1,j-w_i}$ chứ không phải $f_{i,j-w_i}$).

Điều quan trọng là phải hiểu quy tắc chuyển trạng thái này, vì hầu hết các chuyển trạng thái cho các bài toán cái túi đều được suy ra theo cách tương tự.

### 6.3.2 Triển khai

Thuật toán được mô tả có thể được triển khai trong $O(nW)$ như sau:

```
for (int i = 1; i <= n; i++)
  for (int j = W; j >= w[i]; j--)
    f[j] = max(f[j], f[j - w[i]] + v[i]);
```

Lưu ý lại thứ tự thực hiện. Cần tuân thủ nghiêm ngặt để đảm bảo bất biến sau: Ngay trước khi cặp $(i, j)$ được xử lý, $f_k$ tương ứng với $f_{i,k}$ với $k > j$, nhưng với $f_{i-1,k}$ với $k < j$. Điều này đảm bảo rằng $f_{j-w_i}$ được lấy từ bước $(i-1)$, chứ không phải từ bước $i$.

## 6.4 Bài toán Cái Túi Hoàn Chỉnh (Complete Knapsack)

Mô hình cái túi hoàn chỉnh tương tự như cái túi 0-1, sự khác biệt duy nhất là một vật phẩm có thể được chọn một số lượng lần không giới hạn thay vì chỉ một lần.

Chúng ta có thể tham khảo ý tưởng của cái túi 0-1 để định nghĩa trạng thái: $f_{i,j}$, giá trị lớn nhất mà cái túi có thể đạt được khi sử dụng $i$ vật phẩm đầu tiên với sức chứa tối đa $j$.

Cần lưu ý rằng mặc dù định nghĩa trạng thái tương tự như của cái túi 0-1, nhưng quy tắc chuyển trạng thái của nó khác với cái túi 0-1.

### 6.4.1 Giải thích

Cách tiếp cận đơn giản là, đối với $i$ vật phẩm đầu tiên, liệt kê số lần mỗi vật phẩm được lấy. Độ phức tạp thời gian của việc này là $O(n^2W)$.

Điều này dẫn đến phương trình chuyển trạng thái sau:

$$f_{i,j} = \max_{k=0}^{\infty}(f_{i-1,j-k\cdot w_i} + k \cdot v_i)$$

Đồng thời, nó đơn giản hóa thành một phương trình "phẳng":

$$f_{i,j} = \max(f_{i-1,j}, f_{i,j-w_i} + v_i)$$

Lý do điều này hoạt động là vì $f_{i,j-w_i}$ đã được cập nhật bởi $f_{i,j-2\cdot w_i}$, v.v.

Tương tự như cái túi 0-1, chúng ta có thể loại bỏ chiều thứ nhất để tối ưu hóa độ phức tạp không gian. Điều này cho chúng ta quy tắc chuyển trạng thái giống như cái túi 0-1.

$$f_j \leftarrow \max(f_j, f_{j-w_i} + v_i)$$

### 6.4.2 Triển khai

Thuật toán được mô tả có thể được triển khai trong $O(nW)$ như sau:

```
for (int i = 1; i <= n; i++)
  for (int j = w[i]; j <= W; j++)
    f[j] = max(f[j], f[j - w[i]] + v[i]);
```

Mặc dù có cùng quy tắc chuyển trạng thái, nhưng đoạn mã trên là không chính xác cho cái túi 0-1.

Quan sát đoạn mã cẩn thận, chúng ta thấy rằng đối với vật phẩm $i$ hiện đang được xử lý và trạng thái hiện tại $f_{i,j}$, khi $j \geqslant w_i$, $f_{i,j}$ sẽ bị ảnh hưởng bởi $f_{i,j-w_i}$. Điều này tương đương với việc có thể cho vật phẩm $i$ vào ba lô nhiều lần, điều này phù hợp với bài toán cái túi hoàn chỉnh chứ không phải bài toán cái túi 0-1.

## 6.5 Bài toán Cái Túi Bội Số (Multiple Knapsack)

Bài toán cái túi bội số cũng là một biến thể của bài toán cái túi 0-1. Sự khác biệt chính là có $k_i$ vật phẩm của mỗi loại thay vì chỉ 1.

### 6.5.1 Giải thích

Một ý tưởng rất đơn giản là: "chọn mỗi vật phẩm $k_i$ lần" tương đương với "chọn $k_i$ vật phẩm giống nhau lần lượt". Do đó, chuyển đổi nó thành một mô hình cái túi 0-1, có thể được mô tả bằng hàm chuyển trạng thái:

$$f_{i,j} = \max_{k=0}^{k_i}(f_{i-1,j-k\cdot w_i} + k \cdot v_i)$$

Độ phức tạp thời gian của quá trình này là $O(W \sum_{i=1}^{n} k_i)$

### 6.5.2 Tối ưu hóa Nhóm Nhị Phân (Binary Grouping Optimization)

Chúng ta vẫn xem xét việc chuyển đổi mô hình cái túi bội số thành mô hình cái túi 0-1 để tối ưu hóa. Độ phức tạp thời gian $O(Wn)$ không thể được tối ưu hóa thêm với phương pháp trên, vì vậy chúng ta tập trung vào thành phần $O(\sum k_i)$.

Gọi $A_{i,j}$ biểu thị vật phẩm thứ $j$ được tách ra từ vật phẩm thứ $i$. Trong phương pháp đơn giản được thảo luận ở trên, $A_{i,j}$ đại diện cho cùng một vật phẩm cho tất cả $j \leq k_i$. Lý do chính cho hiệu quả thấp của chúng ta là chúng ta đang thực hiện rất nhiều công việc lặp đi lặp lại. Ví dụ: xét việc chọn $\{A_{i,1}, A_{i,2}\}$, và chọn $\{A_{i,2}, A_{i,3}\}$. Hai tình huống này hoàn toàn tương đương. Do đó, việc tối ưu hóa phương pháp tách sẽ giảm đáng kể độ phức tạp thời gian.

Việc nhóm được thực hiện hiệu quả hơn bằng cách sử dụng nhóm nhị phân.

Cụ thể, $A_{i,j}$ giữ $2^j$ vật phẩm riêng lẻ ($j \in [0, \lfloor \log_2(k_i + 1) \rfloor - 1]$). Nếu $k_i + 1$ không phải là một lũy thừa nguyên của 2, một bó khác có kích thước $k_i - (2^{\lfloor \log_2(k_i+1) \rfloor} - 1)$ được sử dụng để bù đắp cho nó.

Thông qua phương pháp tách trên, có thể thu được bất kỳ tổng nào $\leq k_i$ vật phẩm bằng cách chọn một vài $A_{i,j}$'s. Sau khi tách từng vật phẩm theo cách được mô tả, chỉ cần sử dụng phương pháp cái túi 0-1 để giải quyết công thức mới của bài toán.

Tối ưu hóa này cho chúng ta độ phức tạp thời gian là $O(W \sum_{i=1}^{n} \log k_i)$.

### 6.5.3 Triển khai

```
index = 0;
for (int i = 1; i <= n; i++) {
  int c = 1, p, h, k;
  cin >> p >> h >> k;
  while (k > c) {
    k -= c;
    list[++index].w = c * p;
    list[index].v = c * h;
    c *= 2;
  }
  list[++index].w = p * k;
  list[index].v = h * k;
}
```

## 6.6 Tối ưu hóa Hàng đợi Đơn điệu (Monotone Queue Optimization)

Trong tối ưu hóa này, chúng ta hướng tới việc chuyển đổi bài toán cái túi thành bài toán hàng đợi tối đa.

Để thuận tiện cho việc mô tả, đặt $g_{x,y} = f_{i,x \cdot w_i + y}, g'_{x,y} = f_{i-1,x \cdot w_i + y}$. Khi đó, quy tắc

chuyển trạng thái có thể được viết là:

$$g_{x,y} = \max_{k=0}^{k_i}(g'_{x-k,y} + v_i \cdot k)$$

Hơn nữa, đặt $G_{x,y} = g'_{x,y} - v_i \cdot x$. Khi đó, quy tắc chuyển trạng thái có thể được biểu diễn là:

$$g_{x,y} \leftarrow \max_{k=0}^{k_i}(G_{x-k,y}) + v_i \cdot x$$

Điều này chuyển đổi thành một dạng tối ưu hóa hàng đợi đơn điệu cổ điển. $G_{x,y}$ có thể được tính toán trong $O(1)$, vì vậy đối với một $y$ cố định, chúng ta có thể tính toán $g_{x,y}$ trong $O(\lfloor \frac{W}{w_i} \rfloor)$ thời gian. Do đó, độ phức tạp của việc tìm tất cả $g_{x,y}$ là $O(\lfloor \frac{W}{w_i} \rfloor) \times O(w_i) = O(W)$. Bằng cách này, tổng độ phức tạp của thuật toán được giảm xuống còn $O(nW)$.

## 6.7 Bài toán Cái Túi Hỗn Hợp (Mixed Knapsack)

Bài toán cái túi hỗn hợp bao gồm sự kết hợp của ba bài toán được mô tả ở trên. Đó là, một số vật phẩm chỉ có thể được lấy một lần, một số có thể được lấy vô hạn lần và một số có thể được lấy tối đa $k$ lần.

Bài toán có vẻ khó khăn, nhưng chỉ cần bạn hiểu các ý tưởng cốt lõi của các bài toán cái túi trước đó và kết hợp chúng lại với nhau, bạn có thể làm được. Mã giả cho giải pháp như sau:

```
for (each item) {
  if (0-1 knapsack)
    Apply 0-1 knapsack code;
  else if (complete knapsack)
    Apply complete knapsack code;
  else if (multiple knapsack)
    Apply multiple knapsack code;
}
```

## 6.8 Quy hoạch động Chia để trị (Divide and Conquer DP)

Quy hoạch động Chia để trị là một kỹ thuật tối ưu hóa quy hoạch động.

### 6.8.1 Điều kiện tiên quyết

Một số bài toán quy hoạch động có công thức truy hồi dạng sau:

$$dp(i,j) = \min_{0 \le k \le j} \{dp(i-1,k-1) + C(k,j)\}$$

trong đó $C(k,j)$ là hàm chi phí và $dp(i,j) = 0$ khi $j < 0$.

Giả sử $0 \le i < m$ và $0 \le j < n$, và việc tính toán $C$ mất thời gian $O(1)$. Khi đó, việc tính toán trực tiếp công thức truy hồi trên là $O(mn^2)$. Có $m \times n$ trạng thái, và $n$ chuyển đổi cho mỗi trạng thái.

Gọi $opt(i,j)$ là giá trị của $k$ mà tối thiểu hóa biểu thức trên. Giả sử rằng hàm chi phí thỏa mãn bất đẳng thức tứ giác, chúng ta có thể chứng minh rằng $opt(i,j) \le opt(i,j+1)$ cho tất cả $i,j$. Điều này được gọi là điều kiện đơn điệu. Khi đó, chúng ta có thể áp dụng quy hoạch động chia để trị. "Điểm chia" tối ưu cho một $i$ cố định tăng khi $j$ tăng.

Điều này cho phép chúng ta giải quyết tất cả các trạng thái hiệu quả hơn. Giả sử chúng ta tính toán $opt(i,n/2)$ cho một $i$ cố định. Khi đó, đối với bất kỳ $j' < j$ nào, chúng ta biết rằng $opt(i,j') \le opt(i,j)$. Điều này có nghĩa là khi tính toán $opt(i,j')$, chúng ta không phải xem xét nhiều điểm chia như vậy!

Để giảm thiểu thời gian chạy, chúng ta áp dụng ý tưởng đằng sau chia để trị. Đầu tiên, tính toán $opt(i,n/2)$. Sau đó, tính toán $opt(i,n/4)$, biết rằng nó nhỏ hơn hoặc bằng $opt(i,n/2)$ và $opt(i,3n/4)$ biết rằng nó lớn hơn hoặc bằng $opt(i,n/2)$. Bằng cách theo dõi đệ quy các cận dưới và cận trên trên $opt$, chúng ta đạt được thời gian chạy $O(mn \log n)$. Mỗi giá trị có thể của $opt(i,j)$ chỉ xuất hiện trong $\log n$ nút khác nhau.

Lưu ý rằng không quan trọng $opt(i,j)$ "cân bằng" như thế nào. Trên một mức cố định, mỗi giá trị của $k$ được sử dụng nhiều nhất hai lần và có nhiều nhất $\log n$ mức.

### 6.8.2 Triển khai tổng quát

Mặc dù việc triển khai khác nhau tùy thuộc vào bài toán, nhưng đây là một mẫu khá tổng quát. Hàm `compute` tính toán một hàng $i$ các trạng thái `dp_cur`, cho hàng trước đó $i-1$ các trạng thái `dp_before`. Nó phải được gọi với `compute(0, n-1, 0, n-1)`. Hàm `solve` tính toán m hàng và trả về kết quả.

```
int m, n;
vector<long long> dp_before, dp_cur;

long long C(int i, int j);

// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr) {
    if (l > r)
```

```
        return;

    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};

    for (int k = optl; k <= min(mid, optr); k++) {
        best = min(best, {(k ? dp_before[k - 1] : 0) + C(k, mid), k});
    }

    dp_cur[mid] = best.first;
    int opt = best.second;

    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}

long long solve() {
    dp_before.assign(n,0);
    dp_cur.assign(n,0);

    for (int i = 0; i < n; i++)
        dp_before[i] = C(0, i);

    for (int i = 1; i < m; i++) {
        compute(0, n - 1, 0, n - 1);
        dp_before = dp_cur;
    }

    return dp_before[n - 1];
}
```

### 6.8.3 Những điều cần lưu ý

Khó khăn lớn nhất với các bài toán Quy hoạch động Chia để trị là chứng minh tính đơn điệu của *opt*. Một trường hợp đặc biệt khi điều này đúng là khi hàm chi phí thỏa mãn bất đẳng thức tứ giác, tức là, $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ cho tất cả $a \leq b \leq c \leq d$. Nhiều bài toán Quy hoạch động Chia để trị cũng có thể được giải bằng mẹo Bao lồi (Convex Hull trick) hoặc ngược lại. Thật hữu ích khi biết và hiểu cả hai!

## 6.9 Tối ưu hóa Knuth (Knuth's Optimization)

Tối ưu hóa Knuth, còn được gọi là Tăng tốc Knuth-Yao, là một trường hợp đặc biệt của quy hoạch động trên các đoạn, có thể tối ưu hóa độ phức tạp thời gian của các giải pháp theo hệ số tuyến tính, từ $O(n^3)$ cho quy hoạch động trên đoạn tiêu chuẩn thành $O(n^2)$.

### 6.9.1 Điều kiện

Tăng tốc được áp dụng cho các chuyển đổi có dạng

$$dp(i,j) = \min_{i \le k < j} [dp(i,k) + dp(k+1,j) + C(i,j)].$$

Tương tự như quy hoạch động chia để trị, gọi $opt(i,j)$ là giá trị của $k$ mà tối thiểu hóa biểu thức trong chuyển đổi ($opt$ được gọi là "điểm chia tối ưu" trong bài viết này). Tối ưu hóa yêu cầu điều sau phải đúng:

$$opt(i,j-1) \le opt(i,j) \le opt(i+1,j).$$

Chúng ta có thể chứng minh rằng điều này đúng khi hàm chi phí $C$ thỏa mãn các điều kiện sau cho $a \le b \le c \le d$:

1. $C(b,c) \le C(a,d)$;
2. $C(a,c) + C(b,d) \le C(a,d) + C(b,c)$ (bất đẳng thức tứ giác [QI]).

Kết quả này được chứng minh thêm bên dưới.

### 6.9.2 Thuật toán

Hãy xử lý các trạng thái quy hoạch động theo cách mà chúng ta tính toán $dp(i,j-1)$ và $dp(i+1,j)$ trước $dp(i,j)$, và khi làm như vậy, chúng ta cũng tính toán $opt(i,j-1)$ và $opt(i+1,j)$. Sau đó, để tính toán $opt(i,j)$, thay vì kiểm tra các giá trị của $k$ từ $i$ đến $j-1$, chúng ta chỉ cần kiểm tra từ $opt(i,j-1)$ đến $opt(i+1,j)$. Để xử lý các cặp $(i,j)$ theo thứ tự này, chỉ cần sử dụng các vòng lặp for lồng nhau trong đó $i$ đi từ giá trị lớn nhất đến giá trị nhỏ nhất và $j$ đi từ $i+1$ đến giá trị lớn nhất.

### 6.9.3 Triển khai tổng quát

Mặc dù việc triển khai khác nhau, nhưng đây là một ví dụ khá tổng quát. Cấu trúc của mã gần như giống hệt với cấu trúc của Quy hoạch động trên đoạn.

```
int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];

    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };

    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }

    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] + cost;
                }
            }
            dp[i][j] = mn;
        }
    }

    return dp[0][N-1];
}
```

### 6.9.4 Độ phức tạp

Độ phức tạp của thuật toán có thể được ước tính bằng tổng sau:

$$\sum_{i=1}^{N}\sum_{j=i+1}^{N}[opt(i+1,j)-opt(i,j-1)] = \sum_{i=1}^{N}\sum_{j=i}^{N-1}[opt(i+1,j+1)-opt(i,j)].$$

Như bạn thấy, hầu hết các số hạng trong biểu thức này triệt tiêu lẫn nhau, ngoại trừ các số hạng dương với $j = N$ và các số hạng âm với $i = 1$. Do đó, toàn bộ tổng có thể

được ước tính là

$$\sum_{k=1}^{N}[opt(k,N) - opt(1,k)] = O(n^2),$$

chứ không phải $O(n^3)$ như khi chúng ta sử dụng quy hoạch động trên đoạn thông thường.

### 6.9.5 Trong thực tế

Ứng dụng phổ biến nhất của tối ưu hóa Knuth là trong Quy hoạch động trên đoạn, với chuyển đổi đã cho. Khó khăn duy nhất là chứng minh rằng hàm chi phí thỏa mãn các điều kiện đã cho. Trường hợp đơn giản nhất là khi hàm chi phí $C(i,j)$ chỉ đơn giản là tổng của các phần tử của mảng con $S[i, i+1, ..., j]$ cho một số mảng (tùy thuộc vào câu hỏi). Tuy nhiên, đôi khi chúng có thể phức tạp hơn.

Lưu ý rằng hơn các điều kiện về chuyển đổi quy hoạch động và hàm chi phí, chìa khóa để tối ưu hóa này là bất đẳng thức trên điểm chia tối ưu. Trong một số bài toán, chẳng hạn như bài toán cây tìm kiếm nhị phân tối ưu (vốn là bài toán ban đầu mà tối ưu hóa này được phát triển), các chuyển đổi và hàm chi phí sẽ ít rõ ràng hơn, tuy nhiên, người ta vẫn có thể chứng minh rằng $opt(i, j-1) \leq opt(i,j) \leq opt(i+1, j)$, và do đó, sử dụng tối ưu hóa này.

### 6.9.6 Chứng minh tính đúng đắn

Để chứng minh tính đúng đắn của thuật toán này về các điều kiện $C(i,j)$, chỉ cần chứng minh rằng

$$opt(i, j-1) \leq opt(i,j) \leq opt(i+1, j)$$

giả sử các điều kiện đã cho được thỏa mãn.

**Bổ đề**

$dp(i,j)$ cũng thỏa mãn bất đẳng thức tứ giác, với các điều kiện của bài toán được thỏa mãn.

**Chứng minh**

Chứng minh cho bổ đề này sử dụng quy nạp mạnh. Nó được lấy từ bài báo Efficient Dynamic Programming Using Quadrangle Inequalities, của F. Frances Yao, người đã giới thiệu Tăng tốc Knuth-Yao (tuyên bố cụ thể này là Bổ đề 2.1 trong bài báo). Ý tưởng là quy nạp trên độ dài $l = d - a$. Trường hợp $l = 1$ là tầm thường. Với $l > 1$ xét 2 trường hợp:

1. $b = c$

   Bất đẳng thức giảm thành $dp(a,b) + dp(b,d) \leq dp(a,d)$ (Điều này giả định rằng $dp(i,i) = 0$ cho tất cả $i$, đó là trường hợp cho tất cả các bài toán sử dụng tối ưu hóa này). Đặt $opt(a,d) = z$.

(a) Nếu $z < j$,
Lưu ý rằng

$$dp(a,b) \le dp_z(a,b) = dp(a,z) + dp(z+1,b) + C(a,b).$$

Do đó,

$$dp(a,b) + dp(b,d) \le dp(a,z) + dp(z+1,b) + dp(b,d) + C(a,b)$$

Từ giả thuyết quy nạp, $dp(z+1,b) + dp(b,d) \le dp(z+1,d)$. Ngoài ra, nó được cho rằng $C(a,b) \le C(a,d)$. Kết hợp 2 sự kiện này với bất đẳng thức trên sẽ cho kết quả mong muốn.

(b) Nếu $z \ge j$, chứng minh của trường hợp này đối xứng với trường hợp trước.

2. $b < c$

Đặt $opt(b,c) = z$ và $opt(a,d) = y$.

(a) Nếu $z \le y$,

$$dp(a,c) + dp(b,d) \le dp_z(a,c) + dp_y(b,d)$$

trong đó

$$dp_z(a,c)+dp_y(b,d) = C(a,c)+C(b,d)+dp(a,z)+dp(z+1,c)+dp(b,y)+dp(y+1,d).$$

Sử dụng QI trên $C$ và trên trạng thái quy hoạch động cho các chỉ số $z+1 \le y+1 \le c \le d$ (từ giả thuyết quy nạp) sẽ cho kết quả mong muốn.

(b) Nếu $z > y$, chứng minh của trường hợp này đối xứng với trường hợp trước.

Điều này hoàn thành chứng minh của bổ đề.

Bây giờ, hãy xem xét thiết lập sau. Chúng ta có 2 chỉ số $i \le p \le q < j$. Đặt $dp_k = C(i,j) + dp(i,k) + dp(k+1,j)$.

Giả sử chúng ta chứng minh rằng

$$dp_p(i,j-1) \ge dp_q(i,j-1) \implies dp_p(i,j) \ge dp_q(i,j).$$

Đặt $q = opt(i,j-1)$, theo định nghĩa, $dp_p(i,j-1) \ge dp_q(i,j-1)$. Do đó, áp dụng bất đẳng thức cho tất cả $i \le p \le q$, chúng ta có thể suy ra rằng $opt(i,j)$ ít nhất là nhiều như $opt(i,j-1)$, chứng minh nửa đầu của bất đẳng thức.

Bây giờ, sử dụng QI trên một số chỉ số $p+1 \le q+1 \le j-1 \le j$, chúng ta nhận được

$$\begin{aligned}
&dp(p+1,j-1) + dp(q+1,j) \le dp(q+1,j-1) + dp(p+1,j)\\
\implies &(dp(i,p) + dp(p+1,j-1) + C(i,j-1)) + (dp(i,q) + dp(q+1,j) + C(i,j))\\
\le &(dp(i,q) + dp(q+1,j-1) + C(i,j-1)) + (dp(i,p) + dp(p+1,j) + C(i,j))\\
\implies &dp_p(i,j-1) + dp_q(i,j) \le dp_p(i,j) + dp_q(i,j-1)\\
\implies &dp_p(i,j-1) - dp_q(i,j-1) \le dp_p(i,j) - dp_q(i,j)
\end{aligned}$$

Cuối cùng,

$$\begin{aligned}
&dp_p(i, j-1) \geq dp_q(i, j-1) \\
&\implies 0 \leq dp_p(i, j-1) - dp_q(i, j-1) \leq dp_p(i, j) - dp_q(i, j) \\
&\implies dp_p(i, j) \geq dp_q(i, j)
\end{aligned}$$

Điều này chứng minh phần đầu tiên của bất đẳng thức, tức là, $opt(i, j-1) \leq opt(i, j)$. Phần thứ hai $opt(i, j) \leq opt(i+1, j)$ có thể được hiển thị với cùng một ý tưởng, bắt đầu với bất đẳng thức $dp(i, p) + dp(i+1, q) \leq dp(i+1, p) + dp(i, q)$.

Điều này hoàn thành chứng minh.

## 6.10 Quy hoạch động trên Profile bị hỏng. Bài toán "Parquet"

Các bài toán phổ biến được giải bằng quy hoạch động trên profile bị hỏng bao gồm:

- Tìm số cách để lấp đầy hoàn toàn một khu vực (ví dụ: bàn cờ/lưới) bằng một số hình (ví dụ: quân domino)
- Tìm một cách để lấp đầy một khu vực với số lượng hình tối thiểu
- Tìm một cách lấp đầy một phần với số lượng không gian (hoặc ô, trong trường hợp lưới) không được lấp đầy tối thiểu
- Tìm một cách lấp đầy một phần với số lượng hình tối thiểu, sao cho không thể thêm hình nào nữa

## 6.11 Bài toán "Parquet"

### 6.11.1 Mô tả bài toán

Cho một lưới có kích thước $N \times M$. Tìm số cách để lấp đầy lưới bằng các hình có kích thước $2 \times 1$ (không ô nào bị bỏ trống và các hình không được chồng lên nhau).

### 6.11.2 Định nghĩa trạng thái quy hoạch động

Gọi trạng thái quy hoạch động là: $dp[i, mask]$, trong đó $i = 1, \ldots N$ và $mask = 0, \ldots 2^M - 1$.

$i$ đại diện cho số hàng trong lưới hiện tại và $mask$ là trạng thái của hàng cuối cùng của lưới hiện tại. Nếu bit thứ $j$ của $mask$ là 0 thì ô tương ứng được lấp đầy, ngược lại thì không được lấp đầy.

Rõ ràng, câu trả lời cho bài toán sẽ là $dp[N, 0]$.

Chúng ta sẽ xây dựng trạng thái quy hoạch động bằng cách lặp qua từng $i = 1, \cdots N$ và từng $mask = 0, \ldots 2^M - 1$, và đối với mỗi $mask$ chúng ta sẽ chỉ chuyển tiếp, tức là, chúng ta sẽ thêm các hình vào lưới hiện tại.

### 6.11.3 Triển khai

```
#include <iostream>
#include <vector>

using namespace std;

int n, m;
vector < vector<long long> > dp;


void calc (int x = 0, int y = 0, int mask = 0, int next_mask = 0)
{
    if (x == n)
        return;
    if (y >= m)
        dp[x+1][next_mask] += dp[x][mask];
    else
    {
        int my_mask = 1 << y;
        if (mask & my_mask)
            calc (x, y+1, mask, next_mask);
        else
        {
            calc (x, y+1, mask, next_mask | my_mask);
            if (y+1 < m && ! (mask & my_mask) && ! (mask & (my_mask << 1)))
                calc (x, y+2, mask, next_mask);
        }
    }
}


int main()
{
    cin >> n >> m;
```

```
    dp.resize (n+1, vector<long long> (1<<m));
    dp[0][0] = 1;
    for (int x=0; x<n; ++x)
        for (int mask=0; mask<(1<<m); ++mask)
            calc (x, 0, mask, 0);

    cout << dp[n][0] << endl;

    return 0;
}
```

## 6.12 Tìm ma trận con toàn số 0 lớn nhất

Cho một ma trận có n hàng và m cột. Tìm ma trận con lớn nhất chỉ bao gồm các số không (ma trận con là một vùng hình chữ nhật của ma trận).

## 6.13 Thuật toán

Các phần tử của ma trận sẽ là a[i][j], trong đó i = 0...n - 1, j = 0... m - 1. Để đơn giản, chúng ta sẽ coi tất cả các phần tử khác không bằng 1.

### 6.13.1 Bước 1: Quy hoạch động phụ trợ

Đầu tiên, chúng ta tính toán ma trận phụ trợ sau: d[i][j], hàng gần nhất có số 1 phía trên a[i][j]. Nói một cách chính thức, d[i][j] là số hàng lớn nhất (từ 0 đến i - 1), trong đó có một phần tử bằng 1 trong cột thứ j. Trong khi lặp từ trên xuống dưới, từ trái sang phải, khi chúng ta đứng ở hàng i, chúng ta biết các giá trị từ hàng trước đó, vì vậy, chỉ cần cập nhật các phần tử có giá trị 1. Chúng ta có thể lưu các giá trị trong một mảng đơn giản d[i], i = 1...m - 1, vì trong thuật toán tiếp theo, chúng ta sẽ xử lý ma trận từng hàng một và chỉ cần các giá trị của hàng hiện tại.

```
vector<int> d(m, -1);
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        if (a[i][j] == 1) {
            d[j] = i;
        }
    }
}
```

### 6.13.2 Bước 2: Giải quyết bài toán

Chúng ta có thể giải quyết bài toán trong $O(nm^2)$ bằng cách lặp qua các hàng, xem xét mọi cột trái và phải có thể cho một ma trận con. Đáy của hình chữ nhật sẽ là hàng hiện tại và sử dụng d[i][j], chúng ta có thể tìm thấy hàng trên cùng. Tuy nhiên, có thể tiến xa hơn và cải thiện đáng kể độ phức tạp của giải pháp.

Rõ ràng là ma trận con toàn số 0 mong muốn được giới hạn ở cả bốn phía bởi một số số một, ngăn nó tăng kích thước và cải thiện câu trả lời. Do đó, chúng ta sẽ không bỏ lỡ câu trả lời nếu chúng ta hành động như sau: đối với mọi ô j trong hàng i (hàng dưới cùng của một ma trận con toàn số 0 tiềm năng), chúng ta sẽ có d[i][j] là hàng trên cùng của ma trận con toàn số 0 hiện tại. Bây giờ, vẫn cần xác định các ranh giới trái và phải tối ưu của ma trận con toàn số 0, tức là đẩy ma trận con này tối đa sang trái và phải của cột thứ j.

Đẩy tối đa sang trái có nghĩa là gì? Có nghĩa là tìm một chỉ số k1 mà d[i][k1] > d[i][j], và đồng thời k1 - chỉ số gần nhất bên trái chỉ số j. Rõ ràng là sau đó k1 + 1 cho số của cột bên trái của ma trận con toàn số 0 cần thiết. Nếu hoàn toàn không có chỉ số nào như vậy, thì đặt k1 = -1 (điều này có nghĩa là chúng ta có thể mở rộng ma trận con toàn số 0 hiện tại sang trái cho đến tận biên của ma trận a).

Một cách đối xứng, bạn có thể xác định một chỉ số k2 cho biên phải: đây là chỉ số gần nhất bên phải j sao cho d[i][k2] > d[i][j] (hoặc m, nếu không có chỉ số nào như vậy).

Vì vậy, các chỉ số k1 và k2, nếu chúng ta học cách tìm kiếm chúng một cách hiệu quả, sẽ cung cấp cho chúng ta tất cả các thông tin cần thiết về ma trận con toàn số 0 hiện tại. Đặc biệt, diện tích của nó sẽ bằng (i - d[i][j]) * (k2 - k1 - 1).

Làm thế nào để tìm kiếm các chỉ số k1 và k2 này một cách hiệu quả với i và j cố định? Chúng ta có thể làm điều đó trong $O(1)$ trung bình.

Để đạt được độ phức tạp như vậy, bạn có thể sử dụng ngăn xếp như sau. Hãy học trước cách tìm kiếm một chỉ số k1 và lưu giá trị của nó cho mỗi chỉ số j trong hàng i hiện tại trong ma trận d1[i][j]. Để làm điều này, chúng ta sẽ xem xét tất cả các cột j từ trái sang phải và chúng ta sẽ chỉ lưu trữ trong ngăn xếp những cột có d[][] lớn hơn d[i][j]. Rõ ràng là khi di chuyển từ một cột j sang cột tiếp theo, cần phải cập nhật nội dung của ngăn xếp. Khi có một phần tử không phù hợp ở đầu ngăn xếp (tức là d[][] <= d[i][j]) hãy bật nó. Thật dễ hiểu rằng chỉ cần loại bỏ khỏi ngăn xếp chỉ từ trên cùng của nó và không từ bất kỳ vị trí nào khác của nó (vì ngăn xếp sẽ chứa một chuỗi d tăng dần của các cột).

Giá trị d1[i][j] cho mỗi j sẽ bằng giá trị nằm ở thời điểm đó trên đỉnh của ngăn xếp.

Động lực học d2[i][j] để tìm các chỉ số k2 được xem xét tương tự, chỉ cần xem các cột từ phải sang trái.

Rõ ràng là vì có chính xác m phần được thêm vào ngăn xếp trên mỗi dòng, nên cũng không thể có nhiều lần xóa hơn, tổng độ phức tạp sẽ là tuyến tính, vì vậy độ phức tạp cuối cùng của thuật toán là $O(nm)$.

Cũng cần lưu ý rằng thuật toán này tiêu thụ bộ nhớ $O(m)$ (không tính dữ liệu đầu vào - ma trận a[][]).

## 6.14 Triển khai

```
#include <iostream>
#include <vector>
#include <stack>

using namespace std;

int zero_matrix(vector<vector<int>> a) {
    int n = a.size();
    int m = a[0].size();

    int ans = 0;
    vector<int> d(m, -1), d1(m), d2(m);
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (a[i][j] == 1)
                d[j] = i;
        }

        for (int j = 0; j < m; ++j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d1[j] = st.empty() ? -1 : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = m - 1; j >= 0; --j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d2[j] = st.empty() ? m : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = 0; j < m; ++j)
```

```
            ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
    }
    return ans;
}

int main() {
  vector<vector<int>> matrix = {
      {0, 0, 1, 0},
      {0, 0, 0, 0},
      {0, 1, 0, 0},
      {0, 0, 0, 0}
  };

  cout << "Largest zero submatrix area: " << zero_matrix(matrix) << endl;
  return 0;
}
```

# Chương 7

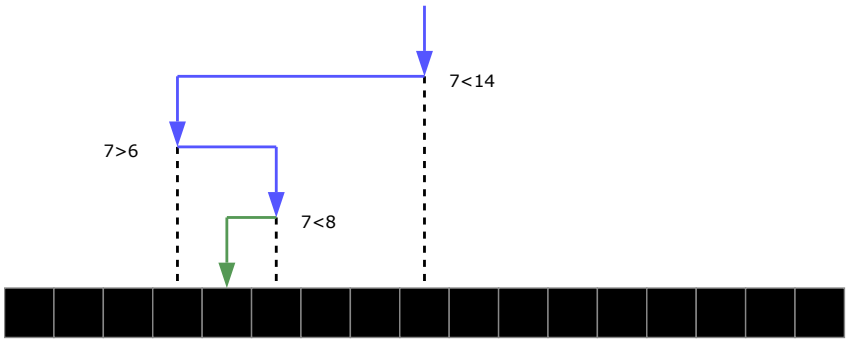
# Phương pháp số

## 7.1 Tìm kiếm nhị phân (Binary Search)

Tìm kiếm nhị phân là một phương pháp cho phép tìm kiếm nhanh hơn bằng cách chia khoảng tìm kiếm thành hai. Ứng dụng phổ biến nhất của nó là tìm kiếm các giá trị trong mảng đã sắp xếp, tuy nhiên ý tưởng chia nhỏ là rất quan trọng trong nhiều nhiệm vụ điển hình khác.

## 7.2 Tìm kiếm trong mảng đã sắp xếp

Bài toán điển hình nhất dẫn đến tìm kiếm nhị phân như sau. Cho một mảng đã sắp xếp $A_0 \leq A_1 \leq \cdots \leq A_{n-1}$, kiểm tra xem $k$ có xuất hiện trong dãy hay không. Giải pháp đơn giản nhất là kiểm tra từng phần tử một và so sánh nó với $k$ (cái gọi là tìm kiếm tuyến tính). Cách tiếp cận này hoạt động trong $O(n)$, nhưng không tận dụng được việc mảng đã được sắp xếp.




Hình 7.1: Tìm kiếm nhị phân

Bây giờ giả sử rằng chúng ta biết hai chỉ số $L < R$ sao cho $A_L \leq k \leq A_R$. Vì mảng đã được sắp xếp, chúng ta có thể suy ra rằng $k$ hoặc xuất hiện trong số $A_L, A_{L+1}, \ldots, A_R$ hoặc không xuất hiện trong mảng. Nếu chúng ta chọn một chỉ số tùy ý $M$ sao cho $L < M < R$ và kiểm tra xem $k$ nhỏ hơn hay lớn hơn $A_M$. Chúng ta có hai trường hợp có thể xảy ra:

1. $A_L \leq k \leq A_M$. Trong trường hợp này, chúng ta giảm bài toán từ $[L, R]$ xuống $[L, M]$;

2. $A_M \leq k \leq A_R$. Trong trường hợp này, chúng ta giảm bài toán từ $[L, R]$ xuống $[M, R]$.

Khi không thể chọn $M$, tức là khi $R = L + 1$, chúng ta so sánh trực tiếp $k$ với $A_L$ và $A_R$. Nếu không, chúng ta muốn chọn $M$ theo cách giảm đoạn hoạt động xuống một phần tử duy nhất nhanh nhất có thể trong trường hợp xấu nhất.

Vì trong trường hợp xấu nhất, chúng ta sẽ luôn giảm xuống đoạn lớn hơn của $[L, M]$ và $[M, R]$. Do đó, trong kịch bản trường hợp xấu nhất, việc giảm sẽ là từ $R - L$ xuống $\max(M - L, R - M)$. Để giảm thiểu giá trị này, chúng ta nên chọn $M \approx \frac{L+R}{2}$, khi đó

$$M - L \approx \frac{R - L}{2} \approx R - M.$$

Nói cách khác, từ góc độ trường hợp xấu nhất, việc luôn chọn $M$ ở giữa $[L, R]$ và chia nó làm đôi là tối ưu. Do đó, đoạn hoạt động giảm một nửa trên mỗi bước cho đến khi nó có kích thước 1. Vì vậy, nếu quá trình cần $h$ bước, cuối cùng nó sẽ giảm sự khác biệt giữa $R$ và $L$ từ $R - L$ xuống $\frac{R-L}{2^h} \approx 1$, cho chúng ta phương trình $2^h \approx R - L$.

Lấy $\log_2$ trên cả hai vế, ta được $h \approx \log_2(R - L) \in O(\log n)$.

Số lượng bước logarit tốt hơn đáng kể so với tìm kiếm tuyến tính. Ví dụ: với $n \approx 2^{20} \approx 10^6$, bạn sẽ cần thực hiện khoảng một triệu thao tác cho tìm kiếm tuyến tính, nhưng chỉ khoảng 20 thao tác với tìm kiếm nhị phân.

## 7.3 Cận dưới và cận trên

Thông thường, việc tìm vị trí của phần tử đầu tiên lớn hơn hoặc bằng $k$ (gọi là cận dưới của $k$ trong mảng) hoặc vị trí của phần tử đầu tiên lớn hơn $k$ (gọi là cận trên của $k$) sẽ thuận tiện hơn là vị trí chính xác của phần tử.

Cùng với nhau, cận dưới và cận trên tạo ra một nửa khoảng có thể trống của các phần tử mảng bằng $k$. Để kiểm tra xem $k$ có xuất hiện trong mảng hay không, chỉ cần tìm cận dưới của nó và kiểm tra xem phần tử tương ứng có bằng $k$ hay không.

## 7.4 Triển khai

Giải thích trên cung cấp một mô tả sơ bộ về thuật toán. Để biết chi tiết triển khai, chúng ta cần chính xác hơn.

Chúng ta sẽ duy trì một cặp $L < R$ sao cho $A_L \leq k < A_R$. Có nghĩa là khoảng tìm kiếm hoạt động là $[L, R)$. Chúng ta sử dụng nửa khoảng ở đây thay vì một đoạn $[L, R]$ vì nó hóa ra đòi hỏi ít công việc trường hợp góc hơn.

Khi $R = L + 1$, chúng ta có thể suy ra từ các định nghĩa trên rằng $R$ là cận trên của $k$. Sẽ thuận tiện hơn nếu khởi tạo $R$ với chỉ số vượt quá cuối, tức là $R = n$ và $L$ với chỉ số trước khi bắt đầu, tức là $L = -1$. Điều đó vẫn ổn miễn là chúng ta không bao giờ đánh giá $A_L$ và $A_R$ trực tiếp trong thuật toán của mình, chính thức coi nó là $A_L = -\infty$ và $A_R = +\infty$.

Cuối cùng, để cụ thể về giá trị của $M$ mà chúng ta chọn, chúng ta sẽ tuân thủ $M = \lfloor \frac{L+R}{2} \rfloor$.

Sau đó, việc triển khai có thể trông như thế này:

```
... // a sorted array is stored as a[0], a[1], ..., a[n-1]
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (k < a[m]) {
        r = m; // a[l] <= k < a[m] <= a[r]
    } else {
        l = m; // a[l] <= a[m] <= k < a[r]
    }
}
```

Trong quá trình thực thi thuật toán, chúng ta không bao giờ đánh giá $A_L$ cũng như $A_R$, vì $L < M < R$. Cuối cùng, $L$ sẽ là chỉ số của phần tử cuối cùng không lớn hơn $k$ (hoặc $-1$ nếu không có phần tử nào như vậy) và $R$ sẽ là chỉ số của phần tử đầu tiên lớn hơn $k$ (hoặc $n$ nếu không có phần tử nào như vậy).

**Lưu ý**. Tính toán m là m = (r + l) / 2 có thể dẫn đến tràn nếu l và r là hai số nguyên dương và lỗi này đã tồn tại khoảng 9 năm trong JDK như được mô tả trong blogpost. Một số cách tiếp cận thay thế bao gồm ví dụ: viết m = l + (r - l) / 2, luôn hoạt động cho số nguyên dương l và r, nhưng vẫn có thể tràn nếu l là một số âm. Nếu bạn sử dụng C++20, nó cung cấp một giải pháp thay thế ở dạng m = std::midpoint(l, r) luôn hoạt động chính xác.

## 7.5 Tìm kiếm trên vị từ tùy ý

Cho $f : \{0, 1, \ldots, n-1\} \to \{0, 1\}$ là một hàm boolean được xác định trên $0, 1, \ldots, n-1$ sao cho nó tăng đơn điệu, tức là

$$f(0) \leq f(1) \leq \cdots \leq f(n-1).$$

Tìm kiếm nhị phân, theo cách được mô tả ở trên, tìm thấy phân vùng của mảng theo vị từ $f(M)$, giữ giá trị boolean của biểu thức $k < A_M$. Có thể sử dụng vị từ đơn điệu tùy ý thay vì $k < A_M$. Điều này đặc biệt hữu ích khi việc tính toán $f(k)$ đòi hỏi quá nhiều thời gian để thực sự tính toán nó cho mọi giá trị có thể. Nói cách khác, tìm kiếm nhị phân tìm thấy chỉ số duy nhất $L$ sao cho $f(L) = 0$ và $f(R) = f(L+1) = 1$ nếu tồn tại một điểm chuyển đổi như vậy, hoặc cho chúng ta $L = n-1$ nếu $f(0) = \cdots = f(n-1) = 0$ hoặc $L = -1$ nếu $f(0) = \cdots = f(n-1) = 1$.

Chứng minh tính đúng đắn giả sử tồn tại một điểm chuyển đổi, tức là $f(0) = 0$ và $f(n-1) = 1$: Việc triển khai duy trì bất biến vòng lặp $f(l) = 0, f(r) = 1$. Khi $r - l > 1$, việc chọn $m$ có nghĩa là $r - l$ sẽ luôn giảm. Vòng lặp kết thúc khi $r - l = 1$, cho chúng ta điểm chuyển đổi mong muốn.

```
... // f(i) is a boolean function such that f(0) <= ... <= f(n-1)
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (f(m)) {
        r = m; // 0 = f(l) < f(m) = 1
    } else {
        l = m; // 0 = f(m) < f(r) = 1
    }
}
```

## 7.6 Tìm kiếm nhị phân trên đáp án

Tình huống như vậy thường xảy ra khi chúng ta được yêu cầu tính toán một số giá trị, nhưng chúng ta chỉ có khả năng kiểm tra xem giá trị này có ít nhất là $i$ hay không. Ví dụ: bạn được cho một mảng $a_1, \ldots, a_n$ và bạn được yêu cầu tìm tổng trung bình sàn tối đa

$$\left\lfloor \frac{a_l + a_{l+1} + \cdots + a_r}{r - l + 1} \right\rfloor$$

trong số tất cả các cặp $l, r$ có thể sao cho $r - l \geq x$. Một trong những cách đơn giản để giải quyết vấn đề này là kiểm tra xem câu trả lời có ít nhất là $\lambda$ hay không, tức là nếu

có một cặp $l, r$ sao cho điều sau là đúng:

$$\frac{a_l + a_{l+1} + \cdots + a_r}{r - l + 1} \geq \lambda.$$

Tương đương, nó viết lại là

$$(a_l - \lambda) + (a_{l+1} - \lambda) + \cdots + (a_r - \lambda) \geq 0,$$

vì vậy bây giờ chúng ta cần kiểm tra xem có một mảng con của một mảng mới $a_i - \lambda$ có độ dài ít nhất là $x + 1$ với tổng không âm hay không, điều này có thể thực hiện được với một số tổng tiền tố.

## 7.7 Tìm kiếm liên tục

Cho $f : \mathbb{R} \to \mathbb{R}$ là một hàm giá trị thực liên tục trên một đoạn $[L, R]$.

Không mất tính tổng quát, giả sử rằng $f(L) \leq f(R)$. Từ định lý giá trị trung gian, suy ra rằng đối với bất kỳ $y \in [f(L), f(R)]$ nào, có $x \in [L, R]$ sao cho $f(x) = y$. Lưu ý rằng, không giống như các đoạn trước, hàm không bắt buộc phải đơn điệu.

Giá trị $x$ có thể được xấp xỉ đến $\pm\delta$ trong $O\left(\log \frac{R-L}{\delta}\right)$ thời gian cho bất kỳ giá trị cụ thể nào của $\delta$. Ý tưởng về cơ bản là giống nhau, nếu chúng ta lấy $M \in (L, R)$ thì chúng ta có thể giảm khoảng tìm kiếm xuống $[L, M]$ hoặc $[M, R]$ tùy thuộc vào việc $f(M)$ lớn hơn $y$ hay không. Một ví dụ phổ biến ở đây sẽ là tìm nghiệm của các đa thức bậc lẻ.

Ví dụ: cho $f(x) = x^3 + ax^2 + bx + c$. Khi đó $f(L) \to -\infty$ và $f(R) \to +\infty$ với $L \to -\infty$ và $R \to +\infty$. Điều đó có nghĩa là luôn có thể tìm thấy $L$ đủ nhỏ và $R$ đủ lớn sao cho $f(L) < 0$ và $f(R) > 0$. Sau đó, có thể tìm thấy với tìm kiếm nhị phân khoảng nhỏ tùy ý chứa $x$ sao cho $f(x) = 0$.

## 7.8 Tìm kiếm với lũy thừa của 2

Một cách đáng chú ý khác để thực hiện tìm kiếm nhị phân là, thay vì duy trì một đoạn hoạt động, hãy duy trì con trỏ hiện tại $i$ và lũy thừa hiện tại $k$. Con trỏ bắt đầu tại $i = L$ và sau đó trên mỗi lần lặp, người ta kiểm tra vị từ tại điểm $i + 2^k$. Nếu vị từ vẫn là 0, con trỏ được chuyển từ $i$ sang $i + 2^k$, nếu không thì nó vẫn giữ nguyên, sau đó lũy thừa $k$ giảm đi 1.

Mô hình này được sử dụng rộng rãi trong các tác vụ xung quanh cây, chẳng hạn như tìm tổ tiên chung thấp nhất của hai đỉnh hoặc tìm tổ tiên của một đỉnh cụ thể có một chiều cao nhất định. Nó cũng có thể được điều chỉnh để ví dụ: tìm phần tử khác không thứ $k$ trong cây Fenwick.

## 7.9 Tìm kiếm Tam phân (Ternary Search)

Cho một hàm $f(x)$ là đơn mode trên một khoảng $[l, r]$. Hàm đơn mode có một trong hai đặc điểm sau:

1. Hàm tăng nghiêm ngặt trước, đạt giá trị lớn nhất (tại một điểm duy nhất hoặc trên một khoảng), và sau đó giảm nghiêm ngặt.

2. Hàm giảm nghiêm ngặt trước, đạt giá trị nhỏ nhất, và sau đó tăng nghiêm ngặt.

Trong bài viết này, chúng ta sẽ giả định kịch bản đầu tiên. Kịch bản thứ hai hoàn toàn đối xứng với kịch bản đầu tiên.

Nhiệm vụ là tìm giá trị lớn nhất của hàm $f(x)$ trên khoảng $[l, r]$.

## 7.10 Thuật toán

Xét hai điểm bất kỳ $m_1$ và $m_2$ trong khoảng này: $l < m_1 < m_2 < r$. Chúng ta đánh giá hàm tại $m_1$ và $m_2$, tức là tìm các giá trị của $f(m_1)$ và $f(m_2)$. Bây giờ, chúng ta nhận được một trong ba tùy chọn:

1. $f(m_1) < f(m_2)$

   Giá trị lớn nhất mong muốn không thể nằm ở phía bên trái của $m_1$, tức là trên khoảng $[l, m_1]$, vì cả hai điểm $m_1$ và $m_2$ hoặc chỉ $m_1$ thuộc về vùng mà hàm tăng. Trong cả hai trường hợp, điều này có nghĩa là chúng ta phải tìm kiếm giá trị lớn nhất trong đoạn $[m_1, r]$.

2. $f(m_1) > f(m_2)$

   Tình huống này đối xứng với tình huống trước: giá trị lớn nhất không thể nằm ở phía bên phải của $m_2$, tức là trên khoảng $[m_2, r]$, và không gian tìm kiếm được giảm xuống đoạn $[l, m_2]$.

3. $f(m_1) = f(m_2)$

   Chúng ta có thể thấy rằng một trong hai điểm này thuộc về vùng mà giá trị của hàm được tối đa hóa, hoặc $m_1$ nằm trong vùng giá trị tăng và $m_2$ nằm trong vùng giá trị giảm (ở đây chúng ta đã sử dụng tính nghiêm ngặt của hàm tăng/giảm). Do đó, không gian tìm kiếm được giảm xuống $[m_1, m_2]$. Để đơn giản hóa mã, trường hợp này có thể được kết hợp với bất kỳ trường hợp nào trước đó.

Do đó, dựa trên so sánh các giá trị ở hai điểm bên trong, chúng ta có thể thay thế khoảng hiện tại $[l, r]$ bằng một khoảng mới, ngắn hơn $[l', r']$. Lặp đi lặp lại quy trình được mô tả cho khoảng, chúng ta có thể nhận được một khoảng ngắn tùy ý. Cuối cùng, độ dài

của nó sẽ nhỏ hơn một hằng số được xác định trước (độ chính xác) và quá trình có thể dừng lại. Đây là một phương pháp số, vì vậy chúng ta có thể giả định rằng sau đó hàm đạt giá trị lớn nhất tại tất cả các điểm của khoảng cuối cùng $[l, r]$. Không mất tính tổng quát, chúng ta có thể lấy $f(l)$ làm giá trị trả về.

Chúng ta không áp đặt bất kỳ hạn chế nào đối với việc chọn các điểm $m_1$ và $m_2$. Lựa chọn này sẽ xác định tốc độ hội tụ và độ chính xác của việc triển khai. Cách phổ biến nhất là chọn các điểm sao cho chúng chia khoảng $[l, r]$ thành ba phần bằng nhau. Như vậy, chúng ta có

$$m_1 = l + \frac{(r-l)}{3}$$

$$m_2 = r - \frac{(r-l)}{3}$$

Nếu $m_1$ và $m_2$ được chọn gần nhau hơn, tốc độ hội tụ sẽ tăng nhẹ.

## 7.11 Phân tích thời gian chạy

$$T(n) = T(2n/3) + O(1) = \Theta(\log n)$$

Nó có thể được hình dung như sau: mỗi lần sau khi đánh giá hàm tại các điểm $m_1$ và $m_2$, chúng ta về cơ bản bỏ qua khoảng một phần ba của khoảng, một trong hai bên trái hoặc bên phải. Do đó, kích thước của không gian tìm kiếm là $2n/3$ của bản gốc.

Áp dụng Định lý Master, chúng ta nhận được ước tính độ phức tạp mong muốn.

## 7.12 Trường hợp đối số số nguyên

Nếu $f(x)$ lấy tham số số nguyên, khoảng $[l, r]$ trở thành rời rạc. Vì chúng ta không áp đặt bất kỳ hạn chế nào đối với việc chọn các điểm $m_1$ và $m_2$, nên tính đúng đắn của thuật toán không bị ảnh hưởng. $m_1$ và $m_2$ vẫn có thể được chọn để chia $[l, r]$ thành 3 phần gần bằng nhau.

Sự khác biệt xảy ra trong tiêu chí dừng của thuật toán. Tìm kiếm tam phân sẽ phải dừng lại khi $(r - l) < 3$, vì trong trường hợp đó, chúng ta không còn có thể chọn $m_1$ và $m_2$ khác nhau cũng như từ $l$ và $r$, và điều này có thể gây ra một vòng lặp vô hạn. Sau khi $(r - l) < 3$, nhóm điểm ứng cử viên còn lại $(l, l + 1, \ldots, r)$ cần được kiểm tra để tìm điểm tạo ra giá trị lớn nhất $f(x)$.

## 7.13 Triển khai

```
double ternary_search(double l, double r) {
    double eps = 1e-9;              //set the error limit here
```

```
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);      //evaluates the function at m1
        double f2 = f(m2);      //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                    //return the maximum of f(x) in [l, r]
}
```

Ở đây eps thực tế là lỗi tuyệt đối (không tính đến các lỗi do tính toán không chính xác của hàm).

Thay vì tiêu chí r - l > eps, chúng ta có thể chọn một số lượng vòng lặp không đổi làm tiêu chí dừng. Số lượng vòng lặp nên được chọn để đảm bảo độ chính xác cần thiết. Thông thường, trong hầu hết các thử thách lập trình, giới hạn lỗi là $10^{-6}$ và do đó 200 - 300 lần lặp là đủ. Ngoài ra, số lượng vòng lặp không phụ thuộc vào các giá trị của $l$ và $r$, vì vậy số lượng vòng lặp tương ứng với lỗi tương đối cần thiết.

## 7.14 Phương pháp Newton để tìm nghiệm

Đây là một phương pháp lặp được phát minh bởi Isaac Newton vào khoảng năm 1664. Tuy nhiên, phương pháp này đôi khi còn được gọi là phương pháp Raphson, vì Raphson đã phát minh ra cùng một thuật toán vài năm sau Newton, nhưng bài báo của ông được xuất bản sớm hơn nhiều.

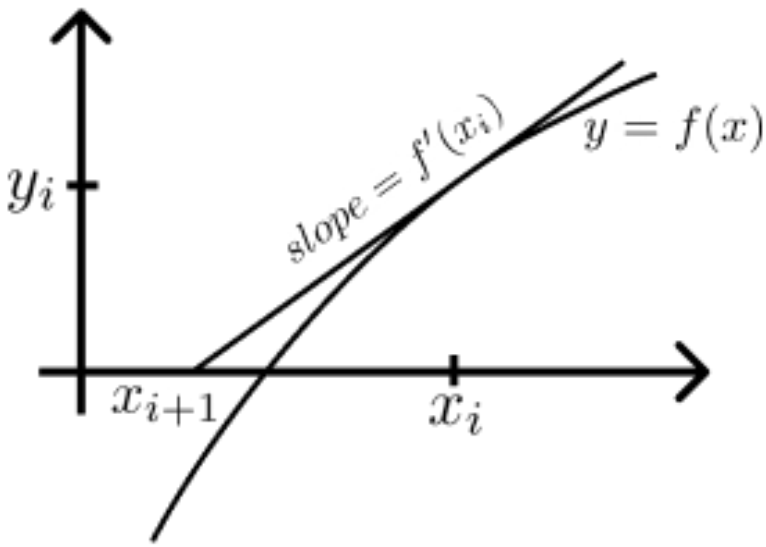
Nhiệm vụ như sau. Cho phương trình sau:

$$f(x) = 0$$

Chúng ta muốn giải phương trình. Chính xác hơn, chúng ta muốn tìm một trong các nghiệm của nó (giả sử rằng nghiệm tồn tại). Giả sử rằng $f(x)$ là liên tục và khả vi trên một khoảng $[a, b]$.

## 7.15 Thuật toán

Các tham số đầu vào của thuật toán không chỉ bao gồm hàm $f(x)$ mà còn cả phép xấp xỉ ban đầu - một số $x_0$, mà thuật toán bắt đầu.




Hình 7.2: Minh họa phương pháp Newton

Giả sử chúng ta đã tính toán $x_i$, tính $x_{i+1}$ như sau. Vẽ tiếp tuyến với đồ thị của hàm $f(x)$ tại điểm $x = x_i$ và tìm điểm giao nhau của tiếp tuyến này với trục $x$. $x_{i+1}$ được đặt bằng tọa độ $x$ của điểm tìm thấy và chúng ta lặp lại toàn bộ quá trình từ đầu.

Không khó để có được công thức sau,

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Đầu tiên, chúng ta tính độ dốc $f'(x)$, đạo hàm của $f(x)$, và sau đó xác định phương trình của tiếp tuyến là,

$$y - f(x_i) = f'(x_i)(x - x_i)$$

Tiếp tuyến giao với trục x tại tọa độ, $y = 0$ và $x = x_{i+1}$,

$$-f(x_i) = f'(x_i)(x_{i+1} - x_i)$$

Bây giờ, giải phương trình, chúng ta nhận được giá trị của $x_{i+1}$.

Rõ ràng một cách trực quan rằng nếu hàm $f(x)$ là "tốt" (mượt) và $x_i$ đủ gần nghiệm, thì $x_{i+1}$ sẽ còn gần hơn với nghiệm mong muốn.

Tốc độ hội tụ là bậc hai, điều kiện nói rằng số lượng chữ số chính xác trong giá trị gần đúng $x_i$ tăng gấp đôi sau mỗi lần lặp.

## 7.16 Ứng dụng để tính căn bậc hai

Hãy sử dụng tính toán căn bậc hai làm ví dụ về phương pháp Newton.

Nếu chúng ta thay thế $f(x) = x^2 - n$, thì sau khi đơn giản hóa biểu thức, chúng ta nhận được:

$$x_{i+1} = \frac{x_i + \frac{n}{x_i}}{2}$$

Biến thể điển hình đầu tiên của bài toán là khi một số hữu tỉ $n$ được cho và căn của nó phải được tính với một độ chính xác eps nào đó:

```
double sqrt_newton(double n) {
    const double eps = 1E-15;
    double x = 1;
    for (;;) {
        double nx = (x + n / x) / 2;
        if (abs(x - nx) < eps)
            break;
        x = nx;
    }
    return x;
}
```

Một biến thể phổ biến khác của bài toán là khi chúng ta cần tính căn nguyên (cho $n$ đã cho, tìm $x$ lớn nhất sao cho $x^2 \leq n$). Ở đây, cần phải thay đổi một chút điều kiện kết thúc của thuật toán, vì có thể xảy ra trường hợp $x$ sẽ bắt đầu "nhảy" gần câu trả lời. Do đó, chúng ta thêm một điều kiện rằng nếu giá trị $x$ đã giảm ở bước trước và nó cố gắng tăng ở bước hiện tại, thì thuật toán phải dừng lại.

```
int isqrt_newton(int n) {
    int x = 1;
    bool decreased = false;
    for (;;) {
        int nx = (x + n / x) >> 1;
        if (x == nx || nx > x && decreased)
            break;
        decreased = nx < x;
        x = nx;
    }
    return x;
}
```

Cuối cùng, chúng ta được đưa ra biến thể thứ ba - cho trường hợp số học bignum. Vì số $n$ có thể đủ lớn, nên việc chú ý đến phép xấp xỉ ban đầu là hợp lý. Rõ ràng, nó càng gần nghiệm, kết quả sẽ đạt được càng nhanh. Đơn giản và hiệu quả để lấy phép xấp xỉ ban đầu là số $2^{\text{bits}/2}$, trong đó bits là số bit trong số $n$. Đây là mã Java minh họa biến thể này:

```
public static BigInteger isqrtNewton(BigInteger n) {
    BigInteger a = BigInteger.ONE.shiftLeft(n.bitLength() / 2);
    boolean p_dec = false;
    for (;;) {
        BigInteger b = n.divide(a).add(a).shiftRight(1);
        if (a.compareTo(b) == 0 || a.compareTo(b) < 0 && p_dec)
            break;
        p_dec = a.compareTo(b) > 0;
        a = b;
    }
    return a;
}
```

Ví dụ: mã này được thực thi trong 60 mili giây cho $n = 10^{1000}$ và nếu chúng ta loại bỏ việc chọn cải tiến phép xấp xỉ ban đầu (chỉ bắt đầu với 1), thì nó sẽ được thực thi trong khoảng 120 mili giây.

## 7.17 Tích phân bằng công thức Simpson

Chúng ta sẽ tính giá trị của một tích phân xác định

$$\int_a^b f(x)dx$$

Giải pháp được mô tả ở đây đã được công bố trong một trong những luận án của Thomas Simpson vào năm 1743.

## 7.18 Công thức Simpson

Cho $n$ là một số tự nhiên nào đó. Chúng ta chia đoạn tích phân $[a, b]$ thành $2n$ phần bằng nhau:

$$x_i = a + ih, \quad i = 0 \ldots 2n,$$

$$h = \frac{b-a}{2n}.$$

Bây giờ chúng ta tính tích phân riêng trên mỗi đoạn $[x_{2i-2}, x_{2i}]$, $i = 1 \ldots n$, và sau đó cộng tất cả các giá trị.

Vì vậy, giả sử chúng ta xem xét đoạn tiếp theo $[x_{2i-2}, x_{2i}], i = 1 \ldots n$. Thay thế hàm $f(x)$ trên đó bằng một parabol $P(x)$ đi qua 3 điểm $(x_{2i-2}, x_{2i-1}, x_{2i})$. Một parabol như vậy luôn tồn tại và là duy nhất; nó có thể được tìm thấy một cách phân tích. Ví dụ: chúng ta có thể xây dựng nó bằng cách sử dụng nội suy đa thức Lagrange. Điều duy nhất còn lại cần làm là tích hợp đa thức này. Nếu bạn làm điều này cho một hàm tổng quát $f$, bạn sẽ nhận được một biểu thức đơn giản đáng kể:

$$\int_{x_{2i-2}}^{x_{2i}} f(x)\ dx \approx \int_{x_{2i-2}}^{x_{2i}} P(x)\ dx = (f(x_{2i-2}) + 4f(x_{2i-1}) + (f(x_{2i}))\,\frac{h}{3}$$

Cộng các giá trị này trên tất cả các đoạn, chúng ta thu được công thức Simpson cuối cùng:

$$\int_a^b f(x)dx \approx (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \ldots + 4f(x_{2N-1}) + f(x_{2N}))\,\frac{h}{3}$$

## 7.19 Sai số

Sai số trong việc xấp xỉ một tích phân bằng công thức Simpson là

$$-\tfrac{1}{90}\left(\tfrac{b-a}{2}\right)^5 f^{(4)}(\xi)$$

trong đó $\xi$ là một số nào đó giữa $a$ và $b$.

Sai số tiệm cận tỷ lệ với $(b-a)^5$. Tuy nhiên, các dẫn xuất trên cho thấy một sai số tỷ lệ với $(b-a)^4$. Quy tắc Simpson đạt được một bậc bổ sung vì các điểm mà tích phân được đánh giá được phân phối đối xứng trong khoảng $[a, b]$.

## 7.20 Triển khai

Ở đây, $f(x)$ là một hàm do người dùng xác định.

```
const int N = 1000 * 1000; // number of steps (already multiplied by 2)

double simpson_integration(double a, double b){
    double h = (b - a) / N;
    double s = f(a) + f(b); // a = x_0 and b = x_2n
    for (int i = 1; i <= N - 1; ++i) { // Refer to final Simpson's formula
        double x = a + h * i;
        s += f(x) * ((i & 1) ? 4 : 2);
```

```
    }
    s *= h / 3;
    return s;
}
```

# Chương 8

# Vài thứ linh tinh

## 8.1 Truy vấn Tìm kiếm Giá trị Nhỏ nhất trong Khoảng (Range Minimum Query)

Cho một mảng $A[1..N]$. Bạn phải trả lời các truy vấn đến có dạng $(L, R)$, yêu cầu tìm phần tử nhỏ nhất trong mảng $A$ giữa các vị trí $L$ và $R$ bao gồm cả hai.

RMQ có thể xuất hiện trực tiếp trong các bài toán hoặc có thể được áp dụng trong một số nhiệm vụ khác, ví dụ: bài toán Tổ tiên Chung Gần nhất (Lowest Common Ancestor).

## 8.2 Giải pháp

Có rất nhiều cách tiếp cận và cấu trúc dữ liệu có thể được sử dụng để giải quyết nhiệm vụ RMQ.

Các cách được giải thích trên trang web này được liệt kê dưới đây.

Đầu tiên là các cách tiếp cận cho phép sửa đổi mảng giữa các lần trả lời truy vấn.

- Phân tích căn bậc hai (Sqrt-decomposition) - trả lời mỗi truy vấn trong $O(\sqrt{N})$, tiền xử lý được thực hiện trong $O(N)$. Ưu điểm: một cấu trúc dữ liệu rất đơn giản. Nhược điểm: độ phức tạp kém hơn.

- Cây phân đoạn (Segment tree) - trả lời mỗi truy vấn trong $O(\log N)$, tiền xử lý được thực hiện trong $O(N)$. Ưu điểm: độ phức tạp thời gian tốt. Nhược điểm: lượng mã lớn hơn so với các cấu trúc dữ liệu khác.

- Cây Fenwick - trả lời mỗi truy vấn trong $O(\log N)$, tiền xử lý được thực hiện trong $O(N \log N)$. Ưu điểm: mã ngắn nhất, độ phức tạp thời gian tốt. Nhược điểm: Cây Fenwick chỉ có thể được sử dụng cho các truy vấn với $L = 1$, vì vậy nó không áp dụng được cho nhiều bài toán.

Và đây là các cách tiếp cận chỉ hoạt động trên các mảng tĩnh, tức là không thể thay đổi giá trị trong mảng mà không cần tính toán lại cấu trúc dữ liệu hoàn chỉnh.

- Bảng thưa (Sparse Table) - trả lời mỗi truy vấn trong $O(1)$, tiền xử lý được thực hiện trong $O(N \log N)$. Ưu điểm: cấu trúc dữ liệu đơn giản, độ phức tạp thời gian tuyệt vời.
- Cây căn bậc hai (Sqrt Tree) - trả lời các truy vấn trong $O(1)$, tiền xử lý được thực hiện trong $O(N \log \log N)$. Ưu điểm: nhanh. Nhược điểm: Khó triển khai.
- Disjoint Set Union / Thủ thuật của Arpa - trả lời các truy vấn trong $O(1)$, tiền xử lý trong $O(n)$. Ưu điểm: ngắn gọn, nhanh chóng. Nhược điểm: chỉ hoạt động nếu tất cả các truy vấn được biết trước, tức là chỉ hỗ trợ xử lý ngoại tuyến các truy vấn.
- Cây Descartes và thuật toán Farach-Colton và Bender - trả lời các truy vấn trong $O(1)$, tiền xử lý trong $O(n)$. Ưu điểm: độ phức tạp tối ưu. Nhược điểm: lượng mã lớn.

**Lưu ý**: Tiền xử lý là quá trình xử lý sơ bộ của mảng đã cho bằng cách xây dựng cấu trúc dữ liệu tương ứng cho nó.

## 8.3 Dãy con tăng dài nhất (Longest Increasing Subsequence)

Cho một mảng gồm $n$ số: $a[0 \dots n-1]$. Nhiệm vụ là tìm dãy con tăng nghiêm ngặt dài nhất trong $a$.

Chính thức, chúng ta tìm kiếm dãy chỉ số dài nhất $i_1, \dots i_k$ sao cho

$$i_1 < i_2 < \dots < i_k, \quad a[i_1] < a[i_2] < \dots < a[i_k]$$

Trong bài viết này, chúng ta thảo luận về nhiều thuật toán để giải quyết nhiệm vụ này. Ngoài ra, chúng ta sẽ thảo luận về một số bài toán khác, có thể được quy về bài toán này.

## 8.4 Giải pháp trong $O(n^2)$ với quy hoạch động

Quy hoạch động là một kỹ thuật rất tổng quát cho phép giải quyết một lớp lớn các bài toán. Ở đây chúng ta áp dụng kỹ thuật này cho nhiệm vụ cụ thể của chúng ta.

Đầu tiên, chúng ta sẽ chỉ tìm kiếm độ dài của dãy con tăng dài nhất và chỉ sau đó mới học cách khôi phục chính dãy con đó.

### 8.4.1 Tìm độ dài

Để hoàn thành nhiệm vụ này, chúng ta định nghĩa một mảng $d[0 \dots n-1]$, trong đó $d[i]$ là độ dài của dãy con tăng dài nhất kết thúc ở phần tử tại chỉ số $i$.

**Ví dụ**

$$\begin{aligned} a &= \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\} \\ d &= \{1, 1, 2, 3, 3, 1, 1, 4, 5, 2\} \end{aligned}$$

Dãy con tăng dài nhất kết thúc tại chỉ số 4 là $\{3, 4, 5\}$ với độ dài là 3, dãy dài nhất kết thúc tại chỉ số 8 là $\{3, 4, 5, 7, 9\}$ hoặc $\{3, 4, 6, 7, 9\}$, cả hai đều có độ dài là 5 và dãy dài nhất kết thúc tại chỉ số 9 là $\{0, 1\}$ có độ dài là 2.

Chúng ta sẽ tính toán mảng này dần dần: đầu tiên $d[0]$, sau đó $d[1]$, v.v. Sau khi mảng này được tính toán, câu trả lời cho bài toán sẽ là giá trị lớn nhất trong mảng $d[]$.

Vậy, cho chỉ số hiện tại là $i$. Tức là chúng ta muốn tính giá trị $d[i]$ và tất cả các giá trị trước đó $d[0], \dots, d[i-1]$ đã được biết. Khi đó có hai lựa chọn:

1. $d[i] = 1$: dãy con yêu cầu chỉ bao gồm phần tử $a[i]$.
2. $d[i] > 1$: Dãy con sẽ kết thúc tại $a[i]$ và ngay trước đó sẽ là một số $a[j]$ với $j < i$ và $a[j] < a[i]$.

Thật dễ dàng để thấy rằng dãy con kết thúc ở $a[j]$ sẽ tự nó là một trong những dãy con tăng dài nhất kết thúc ở $a[j]$. Số $a[i]$ chỉ mở rộng dãy con tăng dài nhất đó thêm một số.

Do đó, chúng ta có thể chỉ cần lặp lại trên tất cả $j < i$ với $a[j] < a[i]$ và lấy dãy dài nhất mà chúng ta nhận được bằng cách nối $a[i]$ vào dãy con tăng dài nhất kết thúc ở $a[j]$. Dãy con tăng dài nhất kết thúc ở $a[j]$ có độ dài $d[j]$, mở rộng nó thêm một cho độ dài $d[j] + 1$.

$$d[i] = \max_{\substack{j<i \\ a[j]<a[i]}} (d[j] + 1)$$

Nếu chúng ta kết hợp hai trường hợp này, chúng ta sẽ nhận được câu trả lời cuối cùng cho $d[i]$:

$$d[i] = \max \left( 1, \max_{\substack{j<i \\ a[j]<a[i]}} (d[j] + 1) \right)$$

### 8.4.2 Triển khai

Đây là một triển khai của thuật toán được mô tả ở trên, tính toán độ dài của dãy con tăng dài nhất.

```
int lis(vector<int> const& a) {
    int n = a.size();
    vector<int> d(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i])
                d[i] = max(d[i], d[j] + 1);
        }
    }

    int ans = d[0];
    for (int i = 1; i < n; i++) {
        ans = max(ans, d[i]);
    }
    return ans;
}
```

### 8.4.3 Khôi phục dãy con

Cho đến nay, chúng ta chỉ học cách tìm độ dài của dãy con, nhưng không phải cách tìm chính dãy con đó.

Để có thể khôi phục dãy con, chúng ta tạo một mảng phụ trợ bổ sung $p[0 \ldots n-1]$ mà chúng ta sẽ tính toán cùng với mảng $d[]$. $p[i]$ sẽ là chỉ số $j$ của phần tử gần cuối trong dãy con tăng dài nhất kết thúc ở $i$. Nói cách khác, chỉ số $p[i]$ là cùng chỉ số $j$ mà giá trị cao nhất $d[i]$ đã đạt được. Mảng phụ trợ $p[]$ này chỉ đến tổ tiên theo một nghĩa nào đó.

Sau đó, để suy ra dãy con, chúng ta chỉ cần bắt đầu tại chỉ số $i$ với $d[i]$ lớn nhất và theo dõi tổ tiên cho đến khi chúng ta suy ra toàn bộ dãy con, tức là cho đến khi chúng ta đạt đến phần tử với $d[i] = 1$.

### 8.4.4 Triển khai khôi phục

Chúng ta sẽ thay đổi mã từ các phần trước một chút. Chúng ta sẽ tính toán mảng $p[]$ cùng với $d[]$ và sau đó tính toán dãy con.

Để thuận tiện, ban đầu chúng ta gán tổ tiên với $p[i] = -1$. Đối với các phần tử có $d[i] = 1$, giá trị tổ tiên sẽ vẫn là $-1$, điều này sẽ thuận tiện hơn một chút cho việc khôi phục dãy con.

```
vector<int> lis(vector<int> const& a) {
    int n = a.size();
    vector<int> d(n, 1), p(n, -1);
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i] && d[i] < d[j] + 1) {
                d[i] = d[j] + 1;
                p[i] = j;
            }
        }
    }

    int ans = d[0], pos = 0;
    for (int i = 1; i < n; i++) {
        if (d[i] > ans) {
            ans = d[i];
            pos = i;
        }
    }

    vector<int> subseq;
    while (pos != -1) {
        subseq.push_back(a[pos]);
        pos = p[pos];
    }
    reverse(subseq.begin(), subseq.end());
    return subseq;
}
```

### 8.4.5 Cách thay thế để khôi phục dãy con

Cũng có thể khôi phục dãy con mà không cần mảng phụ trợ $p[]$. Chúng ta có thể chỉ cần tính toán lại giá trị hiện tại của $d[i]$ và cũng xem cách đạt được giá trị lớn nhất.

Phương pháp này dẫn đến một mã dài hơn một chút, nhưng đổi lại chúng ta tiết kiệm được một số bộ nhớ.

## 8.5 Giải pháp trong $O(n \log n)$ với quy hoạch động và tìm kiếm nhị phân

Để có được một giải pháp nhanh hơn cho bài toán, chúng ta xây dựng một giải pháp quy hoạch động khác chạy trong $O(n^2)$ và sau đó cải thiện nó thành $O(n \log n)$.

Chúng ta sẽ sử dụng mảng quy hoạch động $d[0 \dots n]$. Lần này $d[l]$ không tương ứng với phần tử $a[i]$ hoặc với một tiền tố của mảng. $d[l]$ sẽ là phần tử nhỏ nhất mà một dãy con tăng có độ dài $l$ kết thúc.

Ban đầu, chúng ta giả sử $d[0] = -\infty$ và cho tất cả các độ dài khác $d[l] = \infty$.

Chúng ta sẽ lại xử lý dần dần các số, đầu tiên $a[0]$, sau đó $a[1]$, v.v. và trong mỗi bước duy trì mảng $d[]$ để nó được cập nhật.

**Ví dụ**

Cho mảng $a = \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\}$, đây là tất cả các tiền tố của chúng và mảng quy hoạch động của chúng. Lưu ý rằng các giá trị của mảng không phải lúc nào cũng thay đổi ở cuối.

$$
\begin{aligned}
\text{prefix} &= \{\} & d &= \{-\infty, \infty, \dots\} \\
\text{prefix} &= \{8\} & d &= \{-\infty, 8, \infty, \dots\} \\
\text{prefix} &= \{8, 3\} & d &= \{-\infty, 3, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4\} & d &= \{-\infty, 3, 4, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6\} & d &= \{-\infty, 3, 4, 6, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5\} & d &= \{-\infty, 3, 4, 5, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5, 2\} & d &= \{-\infty, 2, 4, 5, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5, 2, 0\} & d &= \{-\infty, 0, 4, 5, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5, 2, 0, 7\} & d &= \{-\infty, 0, 4, 5, 7, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5, 2, 0, 7, 9\} & d &= \{-\infty, 0, 4, 5, 7, 9, \infty, \dots\} \\
\text{prefix} &= \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\} & d &= \{-\infty, 0, 1, 5, 7, 9, \infty, \dots\}
\end{aligned}
$$

Khi chúng ta xử lý $a[i]$, chúng ta có thể tự hỏi. Các điều kiện để chúng ta viết số hiện tại $a[i]$ vào mảng $d[0 \dots n]$ là gì?

Chúng ta đặt $d[l] = a[i]$, nếu có một dãy con tăng dài nhất có độ dài $l$ kết thúc ở $a[i]$ và không có dãy con tăng dài nhất có độ dài $l$ kết thúc bằng một số nhỏ hơn. Tương tự như cách tiếp cận trước, nếu chúng ta loại bỏ số $a[i]$ khỏi dãy con tăng dài nhất có độ dài $l$, chúng ta sẽ nhận được một dãy con tăng dài nhất khác có độ dài $l - 1$. Vì vậy, chúng ta muốn mở rộng một dãy con tăng dài nhất có độ dài $l - 1$ bằng số $a[i]$ và rõ ràng dãy con tăng dài nhất có độ dài $l - 1$ kết thúc bằng phần tử nhỏ nhất sẽ hoạt động tốt nhất, nói cách khác, dãy có độ dài $l - 1$ kết thúc ở phần tử $d[l - 1]$.

Có một dãy con tăng dài nhất có độ dài $l - 1$ mà chúng ta có thể mở rộng bằng số $a[i]$, chính xác nếu $d[l - 1] < a[i]$. Vì vậy, chúng ta có thể chỉ cần lặp lại trên mỗi độ dài $l$ và kiểm tra xem chúng ta có thể mở rộng một dãy con tăng dài nhất có độ dài $l - 1$ bằng cách kiểm tra các tiêu chí hay không.

Ngoài ra, chúng ta cũng cần kiểm tra xem chúng ta có thể đã tìm thấy một dãy con tăng dài nhất có độ dài $l$ với một số nhỏ hơn ở cuối hay không. Vì vậy, chúng ta chỉ cập nhật nếu $a[i] < d[l]$.

Sau khi xử lý tất cả các phần tử của $a[]$, độ dài của dãy con mong muốn là $l$ lớn nhất với $d[l] < \infty$.

```
int lis(vector<int> const& a) {
    int n = a.size();
```

```
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        for (int l = 1; l <= n; l++) {
            if (d[l-1] < a[i] && a[i] < d[l])
                d[l] = a[i];
        }
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

Bây giờ chúng ta đưa ra hai quan sát quan trọng.

1. Mảng $d$ sẽ luôn được sắp xếp: $d[l-1] < d[l]$ cho tất cả $i = 1 \dots n$.

Điều này là tầm thường, vì bạn có thể chỉ cần loại bỏ phần tử cuối cùng khỏi dãy con tăng có độ dài $l$ và bạn sẽ nhận được một dãy con tăng có độ dài $l-1$ với một số kết thúc nhỏ hơn.

1. Phần tử $a[i]$ sẽ chỉ cập nhật nhiều nhất một giá trị $d[l]$.

Điều này tuân theo ngay lập tức từ việc triển khai trên. Chỉ có thể có một vị trí trong mảng với $d[l-1] < a[i] < d[l]$.

Do đó, chúng ta có thể tìm thấy phần tử này trong mảng $d[]$ bằng cách sử dụng tìm kiếm nhị phân trong $O(\log n)$. Trên thực tế, chúng ta có thể chỉ cần tìm trong mảng $d[]$ số đầu tiên lớn hơn $a[i]$ và chúng ta cố gắng cập nhật phần tử này theo cùng một cách như cách triển khai trên.

### 8.5.1 Triển khai

Điều này cho chúng ta triển khai $O(n \log n)$ được cải thiện:

```
int lis(vector<int> const& a) {
    int n = a.size();
```

```
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l-1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

### 8.5.2 Khôi phục dãy con

Cũng có thể khôi phục dãy con bằng cách sử dụng cách tiếp cận này. Lần này, chúng ta phải duy trì hai mảng phụ trợ. Một mảng cho chúng ta biết chỉ số của các phần tử trong $d[]$. Và một lần nữa, chúng ta phải tạo một mảng "tổ tiên" $p[i]$. $p[i]$ sẽ là chỉ số của phần tử trước đó cho dãy con tối ưu kết thúc ở phần tử $i$.

Thật dễ dàng để duy trì hai mảng này trong quá trình lặp lại trên mảng $a[]$ cùng với các tính toán của $d[]$. Và cuối cùng, không khó để khôi phục dãy con mong muốn bằng cách sử dụng các mảng này.

## 8.6 Giải pháp trong $O(n \log n)$ với cấu trúc dữ liệu

Thay vì phương pháp trên để tính toán dãy con tăng dài nhất trong $O(n \log n)$, chúng ta cũng có thể giải quyết bài toán theo một cách khác: sử dụng một số cấu trúc dữ liệu đơn giản.

Hãy quay lại phương pháp đầu tiên. Hãy nhớ rằng $d[i]$ là giá trị $d[j] + 1$ với $j < i$ và $a[j] < a[i]$.

Do đó, nếu chúng ta xác định một mảng bổ sung $t[]$ sao cho

$$t[a[i]] = d[i],$$

thì bài toán tính toán giá trị $d[i]$ tương đương với việc tìm giá trị lớn nhất trong một

tiền tố của mảng $t[]$:

$$d[i] = \max\left(t[0 \ldots a[i] - 1] + 1\right)$$

Bài toán tìm giá trị lớn nhất của một tiền tố của một mảng (thay đổi) là một bài toán tiêu chuẩn có thể được giải quyết bằng nhiều cấu trúc dữ liệu khác nhau. Ví dụ: chúng ta có thể sử dụng Cây phân đoạn hoặc Cây Fenwick.

Phương pháp này rõ ràng có một số thiếu sót: về độ dài và độ phức tạp của việc triển khai, cách tiếp cận này sẽ tệ hơn phương pháp sử dụng tìm kiếm nhị phân. Ngoài ra, nếu các số đầu vào $a[i]$ đặc biệt lớn, thì chúng ta sẽ phải sử dụng một số thủ thuật, như nén các số (tức là đánh số lại chúng từ 0 đến $n-1$) hoặc sử dụng cây phân đoạn động (chỉ tạo các nhánh của cây quan trọng). Nếu không, mức tiêu thụ bộ nhớ sẽ quá cao.

Mặt khác, phương pháp này cũng có một số ưu điểm: với phương pháp này, bạn không phải suy nghĩ về bất kỳ thuộc tính phức tạp nào trong giải pháp quy hoạch động. Và cách tiếp cận này cho phép chúng ta tổng quát hóa bài toán rất dễ dàng (xem bên dưới).

## 8.7 Các nhiệm vụ liên quan

Dưới đây là một số bài toán liên quan chặt chẽ đến bài toán tìm dãy con tăng dài nhất.

### 8.7.1 Dãy con không giảm dài nhất

Dây thực tế là gần như cùng một bài toán. Chỉ bây giờ được phép sử dụng các số giống hệt nhau trong dãy con.

Giải pháp về cơ bản cũng gần như giống nhau. Chúng ta chỉ cần thay đổi các dấu bất đẳng thức và thực hiện một sửa đổi nhỏ đối với tìm kiếm nhị phân.

### 8.7.2 Số lượng dãy con tăng dài nhất

Chúng ta có thể sử dụng phương pháp được thảo luận đầu tiên, phiên bản $O(n^2)$ hoặc phiên bản sử dụng cấu trúc dữ liệu. Chúng ta chỉ cần lưu trữ thêm số cách chúng ta có thể nhận được các dãy con tăng dài nhất kết thúc ở các giá trị $d[i]$.

Số cách để tạo thành một dãy con tăng dài nhất kết thúc ở $a[i]$ là tổng của tất cả các cách cho tất cả các dãy con tăng dài nhất kết thúc ở $j$ trong đó $d[j]$ là tối đa. Có thể có nhiều $j$ như vậy, vì vậy chúng ta cần tổng hợp tất cả chúng.

Sử dụng Cây phân đoạn, cách tiếp cận này cũng có thể được triển khai trong $O(n \log n)$.

Không thể sử dụng cách tiếp cận tìm kiếm nhị phân cho nhiệm vụ này.

### 8.7.3 Số lượng nhỏ nhất của các dãy con không tăng bao phủ một dãy

Đối với một mảng đã cho gồm $n$ số $a[0 \dots n-1]$, chúng ta phải tô màu các số với số lượng màu nhỏ nhất, sao cho mỗi màu tạo thành một dãy con không tăng.

Để giải quyết điều này, chúng ta nhận thấy rằng số lượng màu cần thiết tối thiểu bằng với độ dài của dãy con tăng dài nhất.

**Chứng minh**: Chúng ta cần chứng minh tính đối ngẫu của hai bài toán này.

Chúng ta ký hiệu $x$ là độ dài của dãy con tăng dài nhất và $y$ là số lượng nhỏ nhất của các dãy con không tăng tạo thành một vùng phủ. Chúng ta cần chứng minh rằng $x = y$.

Rõ ràng là $y < x$ là không thể, vì nếu chúng ta có $x$ phần tử tăng nghiêm ngặt, thì không có hai phần tử nào có thể là một phần của cùng một dãy con không tăng. Do đó, chúng ta có $y \geq x$.

Bây giờ chúng ta chỉ ra rằng $y > x$ là không thể bằng cách mâu thuẫn. Giả sử rằng $y > x$. Sau đó, chúng ta xem xét bất kỳ tập hợp tối ưu nào gồm $y$ dãy con không tăng. Chúng ta chuyển đổi tập hợp này theo cách sau: miễn là có hai dãy con như vậy mà dãy đầu tiên bắt đầu trước dãy thứ hai và dãy đầu tiên bắt đầu bằng một số lớn hơn hoặc bằng số thứ hai, thì chúng ta bỏ số bắt đầu này và gắn nó vào đầu của số thứ hai. Sau một số hữu hạn các bước, chúng ta có $y$ dãy con và các số bắt đầu của chúng sẽ tạo thành một dãy con tăng có độ dài $y$. Vì chúng ta giả định rằng $y > x$, chúng ta đã đạt đến một mâu thuẫn.

Do đó, suy ra rằng $y = x$.

**Khôi phục các dãy**: Việc phân vùng dãy mong muốn thành các dãy con có thể được thực hiện một cách tham lam. Tức là đi từ trái sang phải và gán số hiện tại hoặc dãy con đó kết thúc bằng số nhỏ nhất lớn hơn hoặc bằng số hiện tại.

## 8.8 Tìm mảng con có tổng lớn nhất/nhỏ nhất

Ở đây, chúng ta xem xét bài toán tìm một mảng con có tổng lớn nhất, cũng như một số biến thể của nó (bao gồm cả thuật toán để giải bài toán này trực tuyến).

## 8.9 Phát biểu bài toán

Cho một mảng các số $a[1 \dots n]$. Yêu cầu là tìm một mảng con $a[l \dots r]$ có tổng lớn nhất:

$$\max_{1 \leq l \leq r \leq n} \sum_{i=l}^{r} a[i].$$

Ví dụ: nếu tất cả các số nguyên trong mảng $a[]$ đều không âm, thì câu trả lời sẽ là

chính mảng đó. Tuy nhiên, giải pháp là không tầm thường khi mảng có thể chứa cả số dương và số âm.

Rõ ràng là bài toán tìm mảng con nhỏ nhất về cơ bản là giống nhau, bạn chỉ cần thay đổi dấu của tất cả các số.

## 8.10 Thuật toán 1

Ở đây chúng ta xem xét một thuật toán gần như hiển nhiên. (Tiếp theo, chúng ta sẽ xem xét một thuật toán khác, khó nghĩ ra hơn một chút, nhưng việc triển khai của nó thậm chí còn ngắn hơn.)

### 8.10.1 Mô tả thuật toán

Thuật toán rất đơn giản.

Chúng ta giới thiệu để thuận tiện ký hiệu: $s[i] = \sum_{j=1}^{i} a[j]$. Đó là, mảng $s[i]$ là một mảng các tổng riêng của mảng $a[]$. Ngoài ra, đặt $s[0] = 0$.

Bây giờ chúng ta lặp lại trên chỉ số $r = 1 \dots n$ và học cách nhanh chóng tìm $l$ tối ưu cho mỗi giá trị hiện tại $r$, tại đó tổng tối đa đạt được trên mảng con $[l, r]$.

Chính thức, điều này có nghĩa là đối với $r$ hiện tại, chúng ta cần tìm một $l$ (không vượt quá $r$), sao cho giá trị của $s[r] - s[l-1]$ là lớn nhất. Sau một phép biến đổi tầm thường, chúng ta có thể thấy rằng chúng ta cần tìm trong mảng $s[]$ một giá trị nhỏ nhất trên đoạn $[0, r-1]$.

Từ đây, chúng ta ngay lập tức có được một giải pháp: chúng ta chỉ cần lưu trữ nơi giá trị nhỏ nhất hiện tại nằm trong mảng $s[]$. Sử dụng giá trị nhỏ nhất này, chúng ta tìm thấy chỉ số $l$ tối ưu hiện tại trong $O(1)$ và khi di chuyển từ chỉ số $r$ hiện tại sang chỉ số tiếp theo, chúng ta chỉ cần cập nhật giá trị nhỏ nhất này.

Rõ ràng, thuật toán này hoạt động trong $O(n)$ và là tiệm cận tối ưu.

### 8.10.2 Triển khai

Để triển khai nó, chúng ta thậm chí không cần lưu trữ rõ ràng một mảng các tổng riêng $s[]$ — chúng ta sẽ chỉ cần phần tử hiện tại từ nó.

Việc triển khai được đưa ra trong các mảng được lập chỉ mục 0, không phải trong số 1 như được mô tả ở trên.

Đầu tiên, chúng ta đưa ra một giải pháp tìm một câu trả lời số đơn giản mà không tìm thấy các chỉ số của đoạn mong muốn:

```
int ans = a[0], sum = 0, min_sum = 0;

for (int r = 0; r < n; ++r) {
```

```
    sum += a[r];
    ans = max(ans, sum - min_sum);
    min_sum = min(min_sum, sum);
}
```

Bây giờ chúng ta đưa ra một phiên bản đầy đủ của giải pháp, ngoài ra còn tìm thấy các ranh giới của đoạn mong muốn:

```
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, min_sum = 0, min_pos = -1;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    int cur = sum - min_sum;
    if (cur > ans) {
        ans = cur;
        ans_l = min_pos + 1;
        ans_r = r;
    }
    if (sum < min_sum) {
        min_sum = sum;
        min_pos = r;
    }
}
```

## 8.11 Thuật toán 2

Ở đây chúng ta xem xét một thuật toán khác. Nó khó hiểu hơn một chút, nhưng nó thanh lịch hơn thuật toán trên và việc triển khai của nó ngắn hơn một chút. Thuật toán này được Jay Kadane đề xuất vào năm 1984.

### 8.11.1 Mô tả thuật toán

Bản thân thuật toán như sau. Hãy đi qua mảng và tích lũy tổng riêng hiện tại trong một số biến $s$. Nếu tại một số điểm $s$ là âm, chúng ta chỉ cần gán $s = 0$. Người ta cho rằng giá trị lớn nhất tất cả các giá trị mà biến $s$ được gán trong thuật toán sẽ là câu trả lời cho bài toán.

**Chứng minh**:

Xét chỉ số đầu tiên khi tổng của $s$ trở thành âm. Điều này có nghĩa là bắt đầu với một tổng riêng bằng không, cuối cùng chúng ta nhận được một tổng riêng âm — vì vậy

toàn bộ tiền tố của mảng, cũng như bất kỳ hậu tố nào, có tổng âm. Do đó, mảng con này không bao giờ đóng góp vào tổng riêng của bất kỳ mảng con nào mà nó là tiền tố và có thể chỉ cần bị loại bỏ.

Tuy nhiên, điều này là không đủ để chứng minh thuật toán. Trong thuật toán, chúng ta thực sự bị giới hạn trong việc tìm câu trả lời chỉ cho các đoạn bắt đầu ngay sau những nơi mà $s < 0$ đã xảy ra.

Nhưng, trên thực tế, hãy xem xét một đoạn tùy ý $[l, r]$ và $l$ không ở vị trí "quan trọng" như vậy (tức là $l > p+1$, trong đó $p$ là vị trí cuối cùng như vậy, trong đó $s < 0$). Vì vị trí quan trọng cuối cùng là nghiêm ngặt trước $l-1$, nên hóa ra tổng của $a[p+1 \dots l-1]$ là không âm. Điều này có nghĩa là bằng cách di chuyển $l$ đến vị trí $p+1$, chúng ta sẽ tăng câu trả lời hoặc, trong trường hợp cực đoan, chúng ta sẽ không thay đổi nó.

Bằng cách này hay cách khác, hóa ra là khi tìm kiếm câu trả lời, bạn có thể giới hạn bản thân chỉ trong các đoạn bắt đầu ngay sau các vị trí mà $s < 0$ xuất hiện. Điều này chứng minh rằng thuật toán là chính xác.

### 8.11.2 Triển khai

Như trong thuật toán 1, trước tiên chúng ta đưa ra một triển khai đơn giản hóa chỉ tìm kiếm một câu trả lời số mà không tìm thấy các ranh giới của đoạn tương ứng:

```
int ans = a[0], sum = 0;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    ans = max(ans, sum);
    sum = max(sum, 0);
}
```

Một giải pháp hoàn chỉnh, duy trì các chỉ số của các ranh giới của đoạn tương ứng:

```
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, minus_pos = -1;

for (int r = 0; r < n; ++r) {
    sum += a[r];
    if (sum > ans) {
        ans = sum;
        ans_l = minus_pos + 1;
        ans_r = r;
    }
    if (sum < 0) {
```

```
        sum = 0;
        minus_pos = r;
    }
}
```

## 8.12 Các nhiệm vụ liên quan

### 8.12.1 Tìm mảng con lớn nhất/nhỏ nhất với các ràng buộc

Nếu điều kiện bài toán áp đặt các hạn chế bổ sung đối với đoạn yêu cầu $[l, r]$ (ví dụ: độ dài $r - l + 1$ của đoạn phải nằm trong giới hạn đã chỉ định), thì thuật toán được mô tả có khả năng dễ dàng được tổng quát hóa cho các trường hợp này — dù sao, bài toán vẫn sẽ là tìm giá trị nhỏ nhất trong mảng $s[]$ với các hạn chế bổ sung đã chỉ định.

### 8.12.2 Trường hợp hai chiều của bài toán: tìm kiếm ma trận con lớn nhất/nhỏ nhất

Bài toán được mô tả trong bài viết này được tổng quát hóa một cách tự nhiên cho các chiều lớn. Ví dụ: trong trường hợp hai chiều, nó biến thành một tìm kiếm một ma trận con $[l_1 \ldots r_1, l_2 \ldots r_2]$ của một ma trận đã cho, có tổng các số trong đó là lớn nhất.

Sử dụng giải pháp cho trường hợp một chiều, thật dễ dàng để có được một giải pháp trong $O(n^3)$ cho trường hợp hai chiều: chúng ta lặp lại trên tất cả các giá trị có thể có của $l_1$ và $r_1$ và tính tổng từ $l_1$ đến $r_1$ trong mỗi hàng của ma trận. Bây giờ chúng ta có bài toán một chiều là tìm các chỉ số $l_2$ và $r_2$ trong mảng này, bài toán này đã có thể được giải trong thời gian tuyến tính.

Các thuật toán nhanh hơn để giải quyết bài toán này được biết đến, nhưng chúng không nhanh hơn nhiều so với $O(n^3)$ và rất phức tạp (phức tạp đến mức nhiều thuật toán trong số đó kém hơn thuật toán tầm thường cho tất cả các ràng buộc hợp lý theo hằng số ẩn). Hiện tại, thuật toán được biết đến tốt nhất hoạt động trong $O\left(n^3 \frac{\log^3 \log n}{\log^2 n}\right)$ thời gian (T. Chan 2007 "Nhiều thuật toán hơn cho đường đi ngắn nhất giữa tất cả các cặp trong đồ thị có trọng số")

Thuật toán này của Chan, cũng như nhiều kết quả khác trong lĩnh vực này, thực sự mô tả phép nhân ma trận nhanh (trong đó phép nhân ma trận có nghĩa là phép nhân đã sửa đổi: giá trị nhỏ nhất được sử dụng thay vì phép cộng và phép cộng được sử dụng thay vì phép nhân). Bài toán tìm ma trận con có tổng lớn nhất có thể được quy về bài toán tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh và bài toán này, đến lượt nó, có thể được quy về phép nhân ma trận như vậy.

### 8.12.3 Tìm kiếm một mảng con có giá trị trung bình lớn nhất/nhỏ nhất

Bài toán này nằm ở việc tìm một đoạn $a[l, r]$ sao cho giá trị trung bình là lớn nhất:

$$\max_{l \le r} \frac{1}{r - l + 1} \sum_{i=l}^{r} a[i].$$

Tất nhiên, nếu không có điều kiện nào khác được áp đặt đối với đoạn yêu cầu $[l, r]$, thì giải pháp sẽ luôn là một đoạn có độ dài 1 tại phần tử lớn nhất của mảng. Bài toán chỉ có ý nghĩa nếu có các hạn chế bổ sung (ví dụ: độ dài của đoạn mong muốn bị giới hạn bên dưới).

Trong trường hợp này, chúng ta áp dụng kỹ thuật tiêu chuẩn khi làm việc với các bài toán về giá trị trung bình: chúng ta sẽ chọn giá trị trung bình tối đa mong muốn bằng tìm kiếm nhị phân.

Để làm điều này, chúng ta cần học cách giải quyết bài toán con sau: cho số $x$ và chúng ta cần kiểm tra xem có một mảng con của mảng $a[]$ (tất nhiên, thỏa mãn tất cả các ràng buộc bổ sung của bài toán), trong đó giá trị trung bình lớn hơn $x$ hay không.

Để giải quyết bài toán con này, hãy trừ $x$ khỏi mỗi phần tử của mảng $a[]$. Sau đó, bài toán con của chúng ta thực sự biến thành bài toán này: liệu có các mảng con có tổng dương trong mảng này hay không. Và chúng ta đã biết cách giải quyết bài toán này.

Do đó, chúng ta thu được giải pháp cho tiệm cận $O(T(n) \log W)$, trong đó $W$ là độ chính xác cần thiết, $T(n)$ là thời gian giải quyết nhiệm vụ phụ cho một mảng có độ dài $n$ (có thể thay đổi tùy thuộc vào các hạn chế bổ sung cụ thể được áp đặt).

### 8.12.4 Giải quyết bài toán trực tuyến

Điều kiện của bài toán như sau: cho một mảng gồm $n$ số và một số $L$. Có các truy vấn có dạng $(l, r)$ và để đáp lại mỗi truy vấn, yêu cầu là tìm một mảng con của đoạn $[l, r]$ có độ dài không nhỏ hơn $L$ với giá trị trung bình cộng lớn nhất có thể.

Thuật toán để giải quyết bài toán này khá phức tạp. KADR (Yaroslav Tverdokhleb) đã mô tả thuật toán của mình trên diễn đàn Nga.

## 8.13 Thống kê thứ tự thứ $K$ trong $O(N)$

Cho một mảng $A$ có kích thước $N$ và một số $K$. Bài toán là tìm số lớn thứ $K$ trong mảng, tức là thống kê thứ tự thứ $K$.

Ý tưởng cơ bản - sử dụng ý tưởng của thuật toán sắp xếp nhanh. Trên thực tế, thuật toán này rất đơn giản, khó chứng minh hơn là nó chạy trung bình trong $O(N)$, trái ngược với sắp xếp nhanh.

## 8.14 Triển khai (không đệ quy)

```
template <class T>
T order_statistics (std::vector<T> a, unsigned n, unsigned k)
{
    using std::swap;
    for (unsigned l=1, r=n; ; )
    {
        if (r <= l+1)
        {
            // the current part size is either 1 or 2, so it is easy to find the answe
            if (r == l+1 && a[r] < a[l])
                swap (a[l], a[r]);
            return a[k];
        }

        // ordering a[l], a[l+1], a[r]
        unsigned mid = (l + r) >> 1;
        swap (a[mid], a[l+1]);
        if (a[l] > a[r])
            swap (a[l], a[r]);
        if (a[l+1] > a[r])
            swap (a[l+1], a[r]);
        if (a[l] > a[l+1])
            swap (a[l], a[l+1]);

        // performing division
        // barrier is a[l + 1], i.e. median among a[l], a[l + 1], a[r]
        unsigned
            i = l+1,
            j = r;
        const T
            cur = a[l+1];
        for (;;)
        {
            while (a[++i] < cur) ;
            while (a[--j] > cur) ;
            if (i > j)
                break;
            swap (a[i], a[j]);
```

```
        }

        // inserting the barrier
        a[l+1] = a[j];
        a[j] = cur;

        // we continue to work in that part, which must contain the required element
        if (j >= k)
            r = j-1;
        if (j <= k)
            l = i;
    }
}
```

## 8.15 Ghi chú

- Thuật toán ngẫu nhiên hóa ở trên được đặt tên là quickselect. Bạn nên thực hiện xáo trộn ngẫu nhiên trên $A$ trước khi gọi nó hoặc sử dụng một phần tử ngẫu nhiên làm rào cản để nó chạy đúng cách. Ngoài ra còn có các thuật toán xác định giải quyết bài toán đã chỉ định trong thời gian tuyến tính, chẳng hạn như trung vị của các trung vị.
- Một giải pháp tuyến tính xác định được triển khai trong thư viện chuẩn C++ dưới dạng std::nth_element.
- Tìm $K$ phần tử nhỏ nhất có thể được quy về tìm phần tử thứ $K$ với chi phí tuyến tính, vì chúng chính xác là các phần tử nhỏ hơn thứ $K$.

## 8.16 MEX (minimal excluded) của một dãy

Cho một mảng $A$ có kích thước $N$. Bạn phải tìm phần tử không âm nhỏ nhất không có trong mảng. Số đó thường được gọi là MEX (minimal excluded).

$$\text{mex}(\{0,1,2,4,5\}) = 3$$
$$\text{mex}(\{0,1,2,3,4\}) = 5$$
$$\text{mex}(\{1,2,3,4,5\}) = 0$$

Lưu ý rằng, MEX của một mảng có kích thước $N$ không bao giờ lớn hơn chính $N$.

Cách tiếp cận dễ nhất là tạo một tập hợp tất cả các phần tử trong mảng $A$, để chúng ta có thể nhanh chóng kiểm tra xem một số có phải là một phần của mảng hay không.

Sau đó, chúng ta có thể kiểm tra tất cả các số từ 0 đến $N$, nếu số hiện tại không có trong tập hợp, hãy trả về nó.

## 8.17 Triển khai

Thuật toán sau chạy trong $O(N \log N)$ thời gian.

```
#include <iostream>
#include <vector>
#include <set>

using namespace std;

int mex(vector<int> const& A) {
    set<int> b(A.begin(), A.end());

    int result = 0;
    while (b.count(result))
        ++result;
    return result;
}
```

Nếu một thuật toán yêu cầu tính toán MEX trong $O(N)$, thì có thể sử dụng một vectơ boolean thay vì một tập hợp. Lưu ý rằng, mảng cần phải lớn bằng kích thước mảng lớn nhất có thể.

```
#include <iostream>
#include <vector>

using namespace std;

const int MAX_N = 100000; // Define a maximum value for N

int mex(vector<int> const& A) {
    static bool used[MAX_N+1] = { 0 };

    // mark the given numbers
    for (int x : A) {
        if (x <= MAX_N)
            used[x] = true;
    }
```

```
    // find the mex
    int result = 0;
    while (used[result])
        ++result;

    // clear the array again
    for (int x : A) {
        if (x <= MAX_N)
            used[x] = false;
    }

    return result;
}
```

Cách tiếp cận này nhanh chóng, nhưng chỉ hoạt động tốt nếu bạn phải tính toán MEX một lần. Nếu bạn cần tính toán MEX nhiều lần, ví dụ: vì mảng của bạn liên tục thay đổi, thì nó không hiệu quả. Vì điều đó, chúng ta cần một cái gì đó tốt hơn.

## 8.18 MEX với cập nhật mảng

Trong bài toán, bạn cần thay đổi các số riêng lẻ trong mảng và tính toán MEX mới của mảng sau mỗi lần cập nhật như vậy.

Cần có một cấu trúc dữ liệu tốt hơn để xử lý các truy vấn như vậy một cách hiệu quả.

Một cách tiếp cận là lấy tần số của mỗi số từ 0 đến $N$ và xây dựng một cấu trúc dữ liệu giống như cây trên đó. Ví dụ: một cây phân đoạn hoặc một treap. Mỗi nút đại diện cho một phạm vi số và cùng với tổng tần số trong phạm vi, bạn lưu trữ thêm số lượng số riêng biệt trong phạm vi đó. Có thể cập nhật cấu trúc dữ liệu này trong $O(\log N)$ thời gian và cũng có thể tìm MEX trong $O(\log N)$ thời gian, bằng cách thực hiện tìm kiếm nhị phân cho MEX. Nếu nút đại diện cho phạm vi $[0, \lfloor N/2 \rfloor)$ không chứa $\lfloor N/2 \rfloor$ nhiều số riêng biệt, thì một số bị thiếu và MEX nhỏ hơn $\lfloor N/2 \rfloor$ và bạn có thể đệ quy trong nhánh bên trái của cây. Nếu không, nó ít nhất là $\lfloor N/2 \rfloor$ và bạn có thể đệ quy trong nhánh bên phải của cây.

Cũng có thể sử dụng các cấu trúc dữ liệu thư viện chuẩn map và set (dựa trên một cách tiếp cận được giải thích ở đây). Với một map, chúng ta sẽ nhớ tần số của mỗi số và với set, chúng ta biểu diễn các số hiện đang bị thiếu từ mảng. Vì một set được sắp xếp, *set.begin() sẽ là MEX. Tổng cộng, chúng ta cần $O(N \log N)$ tiền xử lý và sau đó MEX có thể được tính toán trong $O(1)$ và một bản cập nhật có thể được thực hiện trong $O(\log N)$.

```
#include <iostream>
```

```
#include <vector>
#include <map>
#include <set>

using namespace std;

class Mex {
private:
    map<int, int> frequency;
    set<int> missing_numbers;
    vector<int> A;

public:
    Mex(vector<int> const& A) : A(A) {
        for (int i = 0; i <= A.size(); i++)
            missing_numbers.insert(i);

        for (int x : A) {
            ++frequency[x];
            missing_numbers.erase(x);
        }
    }

    int mex() {
        return *missing_numbers.begin();
    }

    void update(int idx, int new_value) {
        if (--frequency[A[idx]] == 0)
            missing_numbers.insert(A[idx]);
        A[idx] = new_value;
        ++frequency[new_value];
        missing_numbers.erase(new_value);
    }
};
```

## 8.19 Các trò chơi trên đồ thị tùy ý

Cho một trò chơi được chơi bởi hai người chơi trên một đồ thị tùy ý $G$. Tức là trạng thái hiện tại của trò chơi là một đỉnh nhất định. Những người chơi thực hiện các nước đi theo

lượt và di chuyển từ đỉnh hiện tại đến một đỉnh liền kề bằng cách sử dụng một cạnh kết nối. Tùy thuộc vào trò chơi, người không thể di chuyển sẽ thua hoặc thắng trò chơi.

Chúng ta xem xét trường hợp tổng quát nhất, trường hợp của một đồ thị có hướng tùy ý có chu trình. Nhiệm vụ của chúng ta là xác định, với một trạng thái ban đầu đã cho, ai sẽ thắng trò chơi nếu cả hai người chơi đều chơi với các chiến lược tối ưu hoặc xác định rằng kết quả của trò chơi sẽ là một trận hòa.

Chúng ta sẽ giải quyết bài toán này rất hiệu quả. Chúng ta sẽ tìm giải pháp cho tất cả các đỉnh bắt đầu có thể có của đồ thị trong thời gian tuyến tính so với số lượng cạnh: $O(m)$.

## 8.20 Mô tả thuật toán

Chúng ta sẽ gọi một đỉnh là một đỉnh chiến thắng, nếu người chơi bắt đầu ở trạng thái này sẽ thắng trò chơi, nếu họ chơi tối ưu (bất kể người chơi kia thực hiện những lượt nào). Tương tự, chúng ta sẽ gọi một đỉnh là một đỉnh thua cuộc, nếu người chơi bắt đầu ở đỉnh này sẽ thua trò chơi, nếu đối thủ chơi tối ưu.

Đối với một số đỉnh của đồ thị, chúng ta đã biết trước rằng chúng là các đỉnh chiến thắng hoặc thua cuộc: cụ thể là tất cả các đỉnh không có cạnh đi ra.

Ngoài ra, chúng ta có các quy tắc sau:

1. Nếu một đỉnh có một cạnh đi ra dẫn đến một đỉnh thua cuộc, thì bản thân đỉnh đó là một đỉnh chiến thắng.

2. Nếu tất cả các cạnh đi ra của một đỉnh nhất định dẫn đến các đỉnh chiến thắng, thì bản thân đỉnh đó là một đỉnh thua cuộc.

3. Nếu tại một số thời điểm vẫn còn các đỉnh chưa xác định và không có đỉnh nào phù hợp với quy tắc đầu tiên hoặc quy tắc thứ hai, thì mỗi đỉnh trong số này, khi được sử dụng làm đỉnh bắt đầu, sẽ dẫn đến một trận hòa nếu cả hai người chơi đều chơi tối ưu.

Do đó, chúng ta có thể xác định một thuật toán chạy trong thời gian $O(nm)$ ngay lập tức. Chúng ta đi qua tất cả các đỉnh và cố gắng áp dụng quy tắc đầu tiên hoặc quy tắc thứ hai và lặp lại.

Tuy nhiên, chúng ta có thể tăng tốc quy trình này và giảm độ phức tạp xuống $O(m)$.

Chúng ta sẽ đi qua tất cả các đỉnh mà ban đầu chúng ta biết liệu chúng có phải là trạng thái chiến thắng hay thua cuộc hay không. Đối với mỗi đỉnh trong số đó, chúng ta bắt đầu tìm kiếm theo chiều sâu. DFS này sẽ di chuyển ngược lại trên các cạnh đảo ngược. Trước hết, nó sẽ không đi vào các đỉnh đã được xác định là các đỉnh chiến thắng hoặc thua cuộc. Và hơn nữa, nếu tìm kiếm đi từ một đỉnh thua cuộc đến một đỉnh chưa xác định, thì chúng ta đánh dấu đỉnh này là một đỉnh chiến thắng và tiếp tục DFS bằng

đỉnh mới này. Nếu chúng ta đi từ một đỉnh chiến thắng đến một đỉnh chưa xác định, thì chúng ta phải kiểm tra xem tất cả các cạnh từ đỉnh này có dẫn đến các đỉnh chiến thắng hay không. Chúng ta có thể thực hiện kiểm tra này trong $O(1)$ bằng cách lưu trữ số lượng cạnh dẫn đến một đỉnh chiến thắng cho mỗi đỉnh. Vì vậy, nếu chúng ta đi từ một đỉnh chiến thắng đến một đỉnh chưa xác định, thì chúng ta tăng bộ đếm và kiểm tra xem số này có bằng số lượng cạnh đi ra hay không. Nếu đúng như vậy, chúng ta có thể đánh dấu đỉnh này là một đỉnh thua cuộc và tiếp tục DFS từ đỉnh này. Nếu không, chúng ta chưa biết liệu đỉnh này có phải là một đỉnh chiến thắng hay thua cuộc hay không và do đó, việc tiếp tục DFS bằng nó không có ý nghĩa gì.

Tổng cộng, chúng ta truy cập mọi đỉnh chiến thắng và mọi đỉnh thua cuộc chính xác một lần (các đỉnh chưa xác định không được truy cập) và chúng ta cũng đi qua mỗi cạnh nhiều nhất một lần. Do đó, độ phức tạp là $O(m)$.

## 8.21 Triển khai

Đây là triển khai của một DFS như vậy. Chúng ta giả định rằng biến adj_rev lưu trữ danh sách kề cho đồ thị ở dạng đảo ngược, tức là thay vì lưu trữ cạnh $(i, j)$ của đồ thị, chúng ta lưu trữ $(j, i)$. Ngoài ra, đối với mỗi đỉnh, chúng ta giả định rằng bậc đi ra đã được tính toán.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

struct State {
    int P, T;
    bool Pstep;
};

vector<vector<int>> adj_rev;

vector<bool> winning;
vector<bool> losing;
vector<bool> visited;
vector<int> degree;

void dfs(int v) {
    visited[v] = true;
```

```
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}
```

## 8.22 Ví dụ: "Cảnh sát và tên trộm"

Đây là một ví dụ cụ thể về một trò chơi như vậy.

Có một bảng $m \times n$. Một số ô không thể vào được. Tọa độ ban đầu của cảnh sát và tên trộm được biết. Một trong các ô là lối ra. Nếu cảnh sát và tên trộm ở cùng một ô tại bất kỳ thời điểm nào, cảnh sát sẽ thắng. Nếu tên trộm ở ô lối ra (mà không có cảnh sát ở trên ô đó), thì tên trộm sẽ thắng. Cảnh sát có thể đi bộ theo cả 8 hướng, tên trộm chỉ theo 4 hướng (dọc theo trục tọa độ). Cả cảnh sát và tên trộm sẽ thay phiên nhau di chuyển. Tuy nhiên, họ cũng có thể bỏ qua một lượt nếu muốn. Nước đi đầu tiên được thực hiện bởi cảnh sát.

Bây giờ chúng ta sẽ xây dựng đồ thị. Để làm được điều này, chúng ta phải chính thức hóa các quy tắc của trò chơi. Trạng thái hiện tại của trò chơi được xác định bởi tọa độ của các văn phòng cảnh sát $P$, tọa độ của tên trộm $T$ và cũng bởi lượt của ai, chúng ta hãy gọi biến này là $P_{\text{turn}}$ (đúng khi đến lượt cảnh sát). Do đó, một đỉnh của đồ thị được xác định bởi bộ ba $(P, T, P_{\text{turn}})$. Sau đó, đồ thị có thể dễ dàng được xây dựng, chỉ cần tuân theo các quy tắc của trò chơi.

Tiếp theo, chúng ta cần xác định những đỉnh nào là chiến thắng và những đỉnh nào là thua cuộc ban đầu. Có một điểm tinh tế ở đây. Các đỉnh chiến thắng / thua cuộc phụ thuộc, ngoài tọa độ, còn phụ thuộc vào $P_{\text{turn}}$ - lượt của ai. Nếu đến lượt cảnh sát, thì đỉnh là một đỉnh chiến thắng, nếu tọa độ của cảnh sát và tên trộm trùng nhau và đỉnh là một đỉnh thua cuộc nếu nó không phải là một đỉnh chiến thắng và tên trộm ở đỉnh lối ra. Nếu đến lượt tên trộm, thì một đỉnh là một đỉnh thua cuộc, nếu tọa độ của hai người chơi trùng nhau và nó là một đỉnh chiến thắng nếu nó không phải là một đỉnh thua cuộc và tên trộm ở đỉnh lối ra.

Điểm duy nhất trước khi triển khai không phải là, bạn cần quyết định xem bạn muốn xây dựng đồ thị một cách rõ ràng hay chỉ xây dựng nó một cách nhanh chóng. Một mặt, việc xây dựng đồ thị một cách rõ ràng sẽ dễ dàng hơn rất nhiều và ít có khả năng mắc lỗi

hơn. Mặt khác, nó sẽ làm tăng lượng mã và thời gian chạy sẽ chậm hơn so với việc bạn xây dựng đồ thị một cách nhanh chóng.

Việc triển khai sau đây sẽ xây dựng đồ thị một cách rõ ràng:

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

struct State {
    int P, T;
    bool Pstep;
};

vector<State> adj_rev[100][100][2]; // [P][T][Pstep]
bool winning[100][100][2];
bool losing[100][100][2];
bool visited[100][100][2];
int degree[100][100][2];

void dfs(State v) {
    visited[v.P][v.T][v.Pstep] = true;
    for (State u : adj_rev[v.P][v.T][v.Pstep]) {
        if (!visited[u.P][u.T][u.Pstep]) {
            if (losing[v.P][v.T][v.Pstep])
                winning[u.P][u.T][u.Pstep] = true;
            else if (--degree[u.P][u.T][u.Pstep] == 0)
                losing[u.P][u.T][u.Pstep] = true;
            else
                continue;
            dfs(u);
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (int i = 0; i < n; i++)
```

```
        cin >> a[i];

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep <= 1; Pstep++) {
                int Px = P/m, Py = P%m, Tx = T/m, Ty = T%m;
                if (a[Px][Py]=='*' || a[Tx][Ty]=='*')
                    continue;

                bool& win = winning[P][T][Pstep];
                bool& lose = losing[P][T][Pstep];
                if (Pstep) {
                    win = Px==Tx && Py==Ty;
                    lose = !win && a[Tx][Ty] == 'E';
                } else {
                    lose = Px==Tx && Py==Ty;
                    win = !lose && a[Tx][Ty] == 'E';
                }
                if (win || lose)
                    continue;

                State st = {P,T,!Pstep};
                adj_rev[P][T][Pstep].push_back(st);
                st.Pstep = Pstep;
                degree[P][T][Pstep]++;

                const int dx[] = {-1, 0, 1, 0, -1, -1, 1, 1};
                const int dy[] = {0, 1, 0, -1, -1, 1, -1, 1};
                for (int d = 0; d < (Pstep ? 8 : 4); d++) {
                    int PPx = Px, PPy = Py, TTx = Tx, TTy = Ty;
                    if (Pstep) {
                        PPx += dx[d];
                        PPy += dy[d];
                    } else {
                        TTx += dx[d];
                        TTy += dy[d];
                    }

                    if (PPx >= 0 && PPx < n && PPy >= 0 && PPy < m && a[PPx][PPy] !=
                        TTx >= 0 && TTx < n && TTy >= 0 && TTy < m && a[TTx][TTy] !=
```

```
                    {
                        adj_rev[PPx*m+PPy][TTx*m+TTy][!Pstep].push_back(st);
                        ++degree[P][T][Pstep];
                    }
                }
            }
        }
    }

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep <= 1; Pstep++) {
                if ((winning[P][T][Pstep] || losing[P][T][Pstep]) && !visited[P][T][Ps
                    dfs({P, T, (bool)Pstep});
            }
        }
    }

    int P_st, T_st;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (a[i][j] == 'P')
                P_st = i*m+j;
            else if (a[i][j] == 'T')
                T_st = i*m+j;
        }
    }

    if (winning[P_st][T_st][true]) {
        cout << "Police catches the thief"  << endl;
    } else if (losing[P_st][T_st][true]) {
        cout << "The thief escapes" << endl;
    } else {
        cout << "Draw" << endl;
    }
}
```

## 8.23 Các trò chơi trên đồ thị tùy ý

Cho một trò chơi được chơi bởi hai người chơi trên một đồ thị tùy ý $G$. Tức là trạng thái hiện tại của trò chơi là một đỉnh nhất định. Những người chơi thực hiện các nước đi theo lượt và di chuyển từ đỉnh hiện tại đến một đỉnh liền kề bằng cách sử dụng một cạnh kết nối. Tùy thuộc vào trò chơi, người không thể di chuyển sẽ thua hoặc thắng trò chơi.

Chúng ta xem xét trường hợp tổng quát nhất, trường hợp của một đồ thị có hướng tùy ý có chu trình. Nhiệm vụ của chúng ta là xác định, với một trạng thái ban đầu đã cho, ai sẽ thắng trò chơi nếu cả hai người chơi đều chơi với các chiến lược tối ưu hoặc xác định rằng kết quả của trò chơi sẽ là một trận hòa.

Chúng ta sẽ giải quyết bài toán này rất hiệu quả. Chúng ta sẽ tìm giải pháp cho tất cả các đỉnh bắt đầu có thể có của đồ thị trong thời gian tuyến tính so với số lượng cạnh: $O(m)$.

## 8.24 Mô tả thuật toán

Chúng ta sẽ gọi một đỉnh là một đỉnh chiến thắng, nếu người chơi bắt đầu ở trạng thái này sẽ thắng trò chơi, nếu họ chơi tối ưu (bất kể người chơi kia thực hiện những lượt nào). Tương tự, chúng ta sẽ gọi một đỉnh là một đỉnh thua cuộc, nếu người chơi bắt đầu ở đỉnh này sẽ thua trò chơi, nếu đối thủ chơi tối ưu.

Đối với một số đỉnh của đồ thị, chúng ta đã biết trước rằng chúng là các đỉnh chiến thắng hoặc thua cuộc: cụ thể là tất cả các đỉnh không có cạnh đi ra.

Ngoài ra, chúng ta có các quy tắc sau:

1. Nếu một đỉnh có một cạnh đi ra dẫn đến một đỉnh thua cuộc, thì bản thân đỉnh đó là một đỉnh chiến thắng.

2. Nếu tất cả các cạnh đi ra của một đỉnh nhất định dẫn đến các đỉnh chiến thắng, thì bản thân đỉnh đó là một đỉnh thua cuộc.

3. Nếu tại một số thời điểm vẫn còn các đỉnh chưa xác định và không có đỉnh nào phù hợp với quy tắc đầu tiên hoặc quy tắc thứ hai, thì mỗi đỉnh trong số này, khi được sử dụng làm đỉnh bắt đầu, sẽ dẫn đến một trận hòa nếu cả hai người chơi đều chơi tối ưu.

Do đó, chúng ta có thể xác định một thuật toán chạy trong thời gian $O(nm)$ ngay lập tức. Chúng ta đi qua tất cả các đỉnh và cố gắng áp dụng quy tắc đầu tiên hoặc quy tắc thứ hai và lặp lại.

Tuy nhiên, chúng ta có thể tăng tốc quy trình này và giảm độ phức tạp xuống $O(m)$.

Chúng ta sẽ đi qua tất cả các đỉnh mà ban đầu chúng ta biết liệu chúng có phải là trạng thái chiến thắng hay thua cuộc hay không. Đối với mỗi đỉnh trong số đó, chúng

ta bắt đầu tìm kiếm theo chiều sâu. DFS này sẽ di chuyển ngược lại trên các cạnh đảo ngược. Trước hết, nó sẽ không đi vào các đỉnh đã được xác định là các đỉnh chiến thắng hoặc thua cuộc. Và hơn nữa, nếu tìm kiếm đi từ một đỉnh thua cuộc đến một đỉnh chưa xác định, thì chúng ta đánh dấu đỉnh này là một đỉnh chiến thắng và tiếp tục DFS bằng đỉnh mới này. Nếu chúng ta đi từ một đỉnh chiến thắng đến một đỉnh chưa xác định, thì chúng ta phải kiểm tra xem tất cả các cạnh từ đỉnh này có dẫn đến các đỉnh chiến thắng hay không. Chúng ta có thể thực hiện kiểm tra này trong $O(1)$ bằng cách lưu trữ số lượng cạnh dẫn đến một đỉnh chiến thắng cho mỗi đỉnh. Vì vậy, nếu chúng ta đi từ một đỉnh chiến thắng đến một đỉnh chưa xác định, thì chúng ta tăng bộ đếm và kiểm tra xem số này có bằng số lượng cạnh đi ra hay không. Nếu đúng như vậy, chúng ta có thể đánh dấu đỉnh này là một đỉnh thua cuộc và tiếp tục DFS từ đỉnh này. Nếu không, chúng ta chưa biết liệu đỉnh này có phải là một đỉnh chiến thắng hay thua cuộc hay không và do đó, việc tiếp tục DFS bằng nó không có ý nghĩa gì.

Tổng cộng, chúng ta truy cập mọi đỉnh chiến thắng và mọi đỉnh thua cuộc chính xác một lần (các đỉnh chưa xác định không được truy cập) và chúng ta cũng đi qua mỗi cạnh nhiều nhất một lần. Do đó, độ phức tạp là $O(m)$.

## 8.25 Triển khai

Đây là triển khai của một DFS như vậy. Chúng ta giả định rằng biến adj_rev lưu trữ danh sách kề cho đồ thị ở dạng đảo ngược, tức là thay vì lưu trữ cạnh $(i, j)$ của đồ thị, chúng ta lưu trữ $(j, i)$. Ngoài ra, đối với mỗi đỉnh, chúng ta giả định rằng bậc đi ra đã được tính toán.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

struct State {
    int P, T;
    bool Pstep;
};

vector<vector<int>> adj_rev;

vector<bool> winning;
vector<bool> losing;
vector<bool> visited;
```

```
vector<int> degree;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}
```

## 8.26 Ví dụ: "Cảnh sát và tên trộm"

Đây là một ví dụ cụ thể về một trò chơi như vậy.

Có một bảng $m \times n$. Một số ô không thể vào được. Tọa độ ban đầu của cảnh sát và tên trộm được biết. Một trong các ô là lối ra. Nếu cảnh sát và tên trộm ở cùng một ô tại bất kỳ thời điểm nào, cảnh sát sẽ thắng. Nếu tên trộm ở ô lối ra (mà không có cảnh sát ở trên ô đó), thì tên trộm sẽ thắng. Cảnh sát có thể đi bộ theo cả 8 hướng, tên trộm chỉ theo 4 hướng (dọc theo trục tọa độ). Cả cảnh sát và tên trộm sẽ thay phiên nhau di chuyển. Tuy nhiên, họ cũng có thể bỏ qua một lượt nếu muốn. Nước đi đầu tiên được thực hiện bởi cảnh sát.

Bây giờ chúng ta sẽ xây dựng đồ thị. Để làm được điều này, chúng ta phải chính thức hóa các quy tắc của trò chơi. Trạng thái hiện tại của trò chơi được xác định bởi tọa độ của các văn phòng cảnh sát $P$, tọa độ của tên trộm $T$ và cũng bởi lượt của ai, chúng ta hãy gọi biến này là $P_{\text{turn}}$ (đúng khi đến lượt cảnh sát). Do đó, một đỉnh của đồ thị được xác định bởi bộ ba $(P, T, P_{\text{turn}})$. Sau đó, đồ thị có thể dễ dàng được xây dựng, chỉ cần tuân theo các quy tắc của trò chơi.

Tiếp theo, chúng ta cần xác định những đỉnh nào là chiến thắng và những đỉnh nào là thua cuộc ban đầu. Có một điểm tinh tế ở đây. Các đỉnh chiến thắng / thua cuộc phụ thuộc, ngoài tọa độ, còn phụ thuộc vào $P_{\text{turn}}$ - lượt của ai. Nếu đến lượt cảnh sát, thì đỉnh là một đỉnh chiến thắng, nếu tọa độ của cảnh sát và tên trộm trùng nhau và đỉnh là một đỉnh thua cuộc nếu nó không phải là một đỉnh chiến thắng và tên trộm ở đỉnh lối ra. Nếu đến lượt tên trộm, thì một đỉnh là một đỉnh thua cuộc, nếu tọa độ của hai người chơi trùng nhau và nó là một đỉnh chiến thắng nếu nó không phải là một đỉnh thua cuộc

và tên trộm ở đỉnh lối ra.

Điểm duy nhất trước khi triển khai không phải là, bạn cần quyết định xem bạn muốn xây dựng đồ thị một cách rõ ràng hay chỉ xây dựng nó một cách nhanh chóng. Một mặt, việc xây dựng đồ thị một cách rõ ràng sẽ dễ dàng hơn rất nhiều và ít có khả năng mắc lỗi hơn. Mặt khác, nó sẽ làm tăng lượng mã và thời gian chạy sẽ chậm hơn so với việc bạn xây dựng đồ thị một cách nhanh chóng.

Việc triển khai sau đây sẽ xây dựng đồ thị một cách rõ ràng:

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

struct State {
    int P, T;
    bool Pstep;
};

vector<State> adj_rev[100][100][2]; // [P][T][Pstep]
bool winning[100][100][2];
bool losing[100][100][2];
bool visited[100][100][2];
int degree[100][100][2];

void dfs(State v) {
    visited[v.P][v.T][v.Pstep] = true;
    for (State u : adj_rev[v.P][v.T][v.Pstep]) {
        if (!visited[u.P][u.T][u.Pstep]) {
            if (losing[v.P][v.T][v.Pstep])
                winning[u.P][u.T][u.Pstep] = true;
            else if (--degree[u.P][u.T][u.Pstep] == 0)
                losing[u.P][u.T][u.Pstep] = true;
            else
                continue;
            dfs(u);
        }
    }
}

int main() {
```

```
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep <= 1; Pstep++) {
                int Px = P/m, Py = P%m, Tx = T/m, Ty = T%m;
                if (a[Px][Py]=='*' || a[Tx][Ty]=='*')
                    continue;

                bool& win = winning[P][T][Pstep];
                bool& lose = losing[P][T][Pstep];
                if (Pstep) {
                    win = Px==Tx && Py==Ty;
                    lose = !win && a[Tx][Ty] == 'E';
                } else {
                    lose = Px==Tx && Py==Ty;
                    win = !lose && a[Tx][Ty] == 'E';
                }
                if (win || lose)
                    continue;

                State st = {P,T,!Pstep};
                adj_rev[P][T][Pstep].push_back(st);
                st.Pstep = Pstep;
                degree[P][T][Pstep]++;

                const int dx[] = {-1, 0, 1, 0, -1, -1, 1, 1};
                const int dy[] = {0, 1, 0, -1, -1, 1, -1, 1};
                for (int d = 0; d < (Pstep ? 8 : 4); d++) {
                    int PPx = Px, PPy = Py, TTx = Tx, TTy = Ty;
                    if (Pstep) {
                        PPx += dx[d];
                        PPy += dy[d];
                    } else {
                        TTx += dx[d];
                        TTy += dy[d];
```

```
                    }

                    if (PPx >= 0 && PPx < n && PPy >= 0 && PPy < m && a[PPx][PPy] !=
                        TTx >= 0 && TTx < n && TTy >= 0 && TTy < m && a[TTx][TTy] !=
                    {
                        adj_rev[PPx*m+PPy][TTx*m+TTy][!Pstep].push_back(st);
                        ++degree[P][T][Pstep];
                    }
                }
            }
        }
    }

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep <= 1; Pstep++) {
                if ((winning[P][T][Pstep] || losing[P][T][Pstep]) && !visited[P][T][Ps
                    dfs({P, T, (bool)Pstep});
            }
        }
    }

    int P_st, T_st;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (a[i][j] == 'P')
                P_st = i*m+j;
            else if (a[i][j] == 'T')
                T_st = i*m+j;
        }
    }

    if (winning[P_st][T_st][true]) {
        cout << "Police catches the thief"  << endl;
    } else if (losing[P_st][T_st][true]) {
        cout << "The thief escapes" << endl;
    } else {
        cout << "Draw" << endl;
    }
}
```

## 8.27 Lên lịch công việc trên một máy

Nhiệm vụ này là tìm một lịch trình tối ưu cho $n$ công việc trên một máy duy nhất, nếu công việc $i$ có thể được xử lý trong thời gian $t_i$, nhưng đối với $t$ giây chờ trước khi xử lý công việc, một khoản phạt là $f_i(t)$ phải được trả.

Do đó, nhiệm vụ yêu cầu tìm một hoán vị các công việc, sao cho tổng hình phạt là tối thiểu. Nếu chúng ta ký hiệu bằng $\pi$ hoán vị của các công việc ($\pi_1$ là mục được xử lý đầu tiên, $\pi_2$ là mục thứ hai, v.v.), thì tổng hình phạt bằng:

$$F(\pi) = f_{\pi_1}(0) + f_{\pi_2}(t_{\pi_1}) + f_{\pi_3}(t_{\pi_1} + t_{\pi_2}) + \cdots + f_{\pi_n}\left(\sum_{i=1}^{n-1} t_{\pi_i}\right)$$

## 8.28 Các giải pháp cho các trường hợp đặc biệt

### 8.28.1 Hàm phạt tuyến tính

Đầu tiên, chúng ta sẽ giải quyết bài toán trong trường hợp tất cả các hàm phạt $f_i(t)$ là tuyến tính, tức là chúng có dạng $f_i(t) = c_i \cdot t$, trong đó $c_i$ là một số không âm. Lưu ý rằng các hàm này không có số hạng không đổi. Nếu không, chúng ta có thể cộng tất cả các số hạng không đổi và giải quyết bài toán mà không có chúng.

Chúng ta cố định một số hoán vị $\pi$ và lấy một chỉ số $i = 1 \ldots n-1$. Cho hoán vị $\pi'$ bằng hoán vị $\pi$ với các phần tử $i$ và $i+1$ được chuyển đổi. Hãy xem hình phạt đã thay đổi bao nhiêu.

$$F(\pi') - F(\pi) =$$

Thật dễ dàng để thấy rằng những thay đổi chỉ xảy ra trong các số hạng thứ $i$ và thứ $(i+1)$:

$$\begin{aligned}
&= c_{\pi'_i} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi'_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi_k} \\
&= c_{\pi_{i+1}} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi_i} \cdot \sum_{k=1}^{i} t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi_k} \\
&= c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i}
\end{aligned}$$

Thật dễ dàng để thấy rằng nếu lịch trình $\pi$ là tối ưu, thì bất kỳ thay đổi nào trong đó đều dẫn đến hình phạt tăng lên (hoặc hình phạt giống hệt nhau), do đó, đối với lịch trình tối ưu, chúng ta có thể viết ra điều kiện sau:

$$c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i} \geq 0 \quad \forall i = 1 \ldots n-1$$

Và sau khi sắp xếp lại, chúng ta nhận được:

$$\frac{c_{\pi_i}}{t_{\pi_i}} \geq \frac{c_{\pi_{i+1}}}{t_{\pi_{i+1}}} \quad \forall i = 1 \ldots n-1$$

Do đó, chúng ta có được lịch trình tối ưu bằng cách chỉ cần sắp xếp các công việc theo phân số $\frac{c_i}{t_i}$ theo thứ tự không tăng.

Cần lưu ý rằng chúng ta đã xây dựng thuật toán này bằng cái gọi là phương pháp hoán vị: chúng ta đã cố gắng hoán đổi hai phần tử liền kề, tính toán hình phạt đã thay đổi bao nhiêu và sau đó suy ra thuật toán để tìm phương pháp tối ưu.

### 8.28.2 Hàm phạt mũ

Cho hàm phạt có dạng như sau:

$$f_i(t) = c_i \cdot e^{\alpha \cdot t},$$

trong đó tất cả các số $c_i$ là không âm và hằng số $\alpha$ là dương.

Bằng cách áp dụng phương pháp hoán vị, thật dễ dàng để xác định rằng các công việc phải được sắp xếp theo thứ tự không tăng của giá trị:

$$v_i = \frac{1 - e^{\alpha \cdot t_i}}{c_i}$$

### 8.28.3 Hàm phạt đơn điệu giống hệt nhau

Trong trường hợp này, chúng ta xem xét trường hợp tất cả $f_i(t)$ đều bằng nhau và hàm này tăng đơn điệu.

Rõ ràng là trong trường hợp này, hoán vị tối ưu là sắp xếp các công việc theo thời gian xử lý không giảm $t_i$.

## 8.29 Định lý Livshits-Kladov

Định lý Livshits-Kladov thiết lập rằng phương pháp hoán vị chỉ có thể áp dụng cho ba trường hợp được đề cập ở trên, tức là:

1. Trường hợp tuyến tính: $f_i(t) = c_i(t) + d_i$, trong đó $c_i$ là các hằng số không âm,
2. Trường hợp mũ: $f_i(t) = c_i \cdot e_{\alpha \cdot t} + d_i$, trong đó $c_i$ và $\alpha$ là các hằng số dương,
3. Trường hợp giống hệt nhau: $f_i(t) = \phi(t)$, trong đó $\phi$ là một hàm tăng đơn điệu.

Trong tất cả các trường hợp khác, phương pháp không thể được áp dụng.

Định lý được chứng minh theo giả định rằng các hàm phạt đủ mịn (tồn tại các đạo hàm bậc ba).

Trong cả ba trường hợp, chúng ta áp dụng phương pháp hoán vị, thông qua đó lịch trình tối ưu mong muốn có thể được tìm thấy bằng cách sắp xếp, do đó trong thời gian $O(n \log n)$.

## 8.30 Lên lịch công việc trên hai máy

Nhiệm vụ này là tìm một lịch trình tối ưu cho $n$ công việc trên hai máy. Mỗi mục phải được xử lý trước trên máy đầu tiên và sau đó trên máy thứ hai. Công việc thứ $i$ mất thời gian $a_i$ trên máy đầu tiên và thời gian $b_i$ trên máy thứ hai. Mỗi máy chỉ có thể xử lý một công việc tại một thời điểm.

Chúng ta muốn tìm thứ tự tối ưu của các công việc, sao cho thời gian xử lý cuối cùng là nhỏ nhất có thể.

Giải pháp được thảo luận ở đây được gọi là quy tắc Johnson (được đặt theo tên S. M. Johnson).

Điều đáng chú ý là nhiệm vụ trở thành NP-hoàn chỉnh, nếu chúng ta có nhiều hơn hai máy.

## 8.31 Xây dựng thuật toán

Lưu ý đầu tiên, chúng ta có thể giả định rằng thứ tự của các công việc cho máy đầu tiên và máy thứ hai phải trùng nhau. Trên thực tế, vì các công việc cho máy thứ hai trở nên khả dụng sau khi xử lý chúng tại máy đầu tiên và nếu có một số công việc khả dụng cho máy thứ hai, thì thời gian xử lý sẽ bằng tổng của $b_i$ của chúng, bất kể thứ tự của chúng. Do đó, chỉ có lợi khi gửi các công việc đến máy thứ hai theo cùng thứ tự như chúng ta đã gửi chúng đến máy đầu tiên.

Xem xét thứ tự của các công việc, trùng với thứ tự đầu vào của chúng $1, 2, \ldots, n$.

Chúng ta ký hiệu bằng $x_i$ thời gian nhàn rỗi của máy thứ hai ngay trước khi xử lý $i$. Mục tiêu của chúng ta là giảm thiểu tổng thời gian nhàn rỗi:

$$F(x) = \sum x_i \ \to \min$$

Đối với công việc đầu tiên, chúng ta có $x_1 = a_1$. Đối với công việc thứ hai, vì nó được gửi đến máy tại thời điểm $a_1 + a_2$ và máy thứ hai được giải phóng tại $x_1 + b_1$, chúng ta có $x_2 = \max\left((a_1 + a_2) - (x_1 + b_1), 0\right)$. Nói chung, chúng ta nhận được phương trình:

$$x_k = \max\left(\sum_{i=1}^{k} a_i - \sum_{i=1}^{k-1} b_i - \sum_{i=1}^{k-1} x_i, 0\right)$$

Bây giờ chúng ta có thể tính tổng thời gian nhàn rỗi $F(x)$. Người ta cho rằng nó có dạng

$$F(x) = \max_{k=1\ldots n} K_i,$$

trong đó

$$K_i = \sum_{i=1}^{k} a_i - \sum_{i=1}^{k-1} b_i.$$

Điều này có thể dễ dàng được xác minh bằng quy nạp.

Bây giờ chúng ta sử dụng phương pháp hoán vị: chúng ta sẽ trao đổi hai công việc lân cận $j$ và $j+1$ và xem điều này sẽ thay đổi tổng thời gian nhàn rỗi như thế nào.

Theo dạng của biểu thức của $K_i$, rõ ràng là chỉ $K_j$ và $K_{j+1}$ thay đổi, chúng ta ký hiệu các giá trị mới của chúng bằng $K'_j$ và $K'_{j+1}$.

Nếu sự thay đổi này từ các công việc $j$ và $j+1$ làm tăng tổng thời gian nhàn rỗi, thì phải là trường hợp:

$$\max(K_j, K_{j+1}) \leq \max(K'_j, K'_{j+1})$$

(Việc chuyển đổi hai công việc cũng có thể không có tác động gì cả. Điều kiện trên chỉ là một điều kiện đủ, nhưng không phải là một điều kiện cần thiết.)

Sau khi loại bỏ $\sum_{i=1}^{j+1} a_i - \sum_{i=1}^{j-1} b_i$ khỏi cả hai vế của bất đẳng thức, chúng ta nhận được:

$$\max(-a_{j+1}, -b_j) \leq \max(-b_{j+1}, -a_j)$$

Và sau khi loại bỏ các dấu âm:

$$\min(a_j, b_{j+1}) \leq \min(b_j, a_{j+1})$$

Do đó, chúng ta có được một bộ so sánh: bằng cách sắp xếp các công việc trên đó, chúng ta có được một thứ tự tối ưu của các công việc, trong đó không có hai công việc nào có thể được chuyển đổi với một cải tiến về thời gian cuối cùng.

Tuy nhiên, bạn có thể đơn giản hóa thêm việc sắp xếp, nếu bạn nhìn vào bộ so sánh từ một góc độ khác. Bộ so sánh có thể được hiểu theo cách sau: Nếu chúng ta có bốn lần $(a_j, a_{j+1}, b_j, b_{j+1})$ và giá trị nhỏ nhất trong số chúng là thời gian tương ứng với máy đầu tiên, thì công việc tương ứng phải được thực hiện trước. Nếu thời gian nhỏ nhất là thời gian từ máy thứ hai, thì nó sẽ diễn ra sau. Do đó, chúng ta có thể sắp xếp các công việc theo $\min(a_i, b_i)$ và nếu thời gian xử lý của công việc hiện tại trên máy đầu tiên nhỏ hơn thời gian xử lý trên máy thứ hai, thì công việc này phải được thực hiện trước tất cả các công việc còn lại và nếu không thì sau tất cả các nhiệm vụ còn lại.

Bằng cách này hay cách khác, hóa ra là theo quy tắc Johnson, chúng ta có thể giải quyết bài toán bằng cách sắp xếp các công việc và do đó nhận được độ phức tạp thời gian là $O(n \log n)$.

## 8.32 Triển khai

Ở đây chúng ta triển khai biến thể thứ hai của thuật toán được mô tả.

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Job {
    int a, b, idx;

    bool operator<(Job o) const {
        return min(a, b) < min(o.a, o.b);
    }
};

vector<Job> johnsons_rule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());
    vector<Job> a, b;
    for (Job j : jobs) {
        if (j.a < j.b)
            a.push_back(j);
        else
            b.push_back(j);
    }
    a.insert(a.end(), b.rbegin(), b.rend());
    return a;
}

pair<int, int> finish_times(vector<Job> const& jobs) {
    int t1 = 0, t2 = 0;
    for (Job j : jobs) {
        t1 += j.a;
        t2 = max(t2, t1) + j.b;
    }
    return make_pair(t1, t2);
}
```

Tất cả thông tin về mỗi công việc được lưu trữ trong struct. Hàm đầu tiên sắp xếp

tất cả các công việc và tính toán lịch trình tối ưu. Hàm thứ hai tính toán thời gian kết thúc của cả hai máy cho một lịch trình đã cho.

## 8.33 Lịch trình tối ưu cho các công việc với thời hạn và thời lượng đã cho

Giả sử chúng ta có một tập hợp các công việc, và chúng ta biết thời hạn và thời lượng của mỗi công việc. Việc thực hiện một công việc không thể bị gián đoạn trước khi nó kết thúc. Yêu cầu là tạo ra một lịch trình để hoàn thành số lượng công việc lớn nhất.

## 8.34 Giải quyết

Thuật toán giải quyết là tham lam. Hãy sắp xếp tất cả các công việc theo thời hạn của chúng và xem xét chúng theo thứ tự giảm dần. Ngoài ra, hãy tạo một hàng đợi $q$, trong đó chúng ta sẽ dần dần đưa các công việc vào và trích xuất một công việc có thời gian chạy ít nhất (ví dụ: chúng ta có thể sử dụng set hoặc priority_queue). Ban đầu, $q$ là trống.

Giả sử chúng ta đang xem xét công việc thứ $i$. Trước hết, hãy đưa nó vào $q$. Hãy xem xét khoảng thời gian giữa thời hạn của công việc thứ $i$ và thời hạn của công việc thứ $i-1$. Đó là đoạn có độ dài $T$. Chúng ta sẽ trích xuất các công việc từ $q$ (theo thứ tự tăng dần thời lượng còn lại của chúng) và thực hiện chúng cho đến khi toàn bộ đoạn $T$ được lấp đầy. Quan trọng: nếu tại bất kỳ thời điểm nào công việc được trích xuất chỉ có thể được thực hiện một phần cho đến khi đoạn $T$ được lấp đầy, thì chúng ta thực hiện công việc này một phần chỉ đến mức có thể, tức là trong thời gian $T$, và chúng ta đưa phần còn lại của công việc trở lại $q$.

Khi thuật toán hoàn thành, chúng ta sẽ chọn giải pháp tối ưu (hoặc ít nhất là một trong số các giải pháp). Thời gian chạy của thuật toán là $O(n \log n)$.

## 8.35 Triển khai

Hàm sau đây nhận một vector các công việc (bao gồm thời hạn, thời lượng và chỉ số của công việc) và tính toán một vector chứa tất cả các chỉ số của các công việc được sử dụng trong lịch trình tối ưu. Lưu ý rằng bạn vẫn cần sắp xếp các công việc này theo thời hạn của chúng nếu bạn muốn viết ra kế hoạch một cách rõ ràng.

```
struct Job {
    int deadline, duration, idx;

    bool operator<(Job o) const {
        return deadline < o.deadline;
```

```
    }
};

vector<int> compute_schedule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());

    std::set<std::pair<int,int>> s;
    std::vector<int> schedule;
    for (int i = jobs.size()-1; i >= 0; i--) {
        int t = jobs[i].deadline - (i ? jobs[i-1].deadline : 0);
        s.insert(std::make_pair(jobs[i].duration, jobs[i].idx));
        while (t && !s.empty()) {
            auto it = s.begin();
            if (it->first <= t) {
                t -= it->first;
                schedule.push_back(it->second);
            } else {
                s.insert(std::make_pair(it->first - t, it->second));
                t = 0;
            }
            s.erase(it);
        }
    }
    return schedule;
}
```